CMU 15–213《CSAPP · 深入理解计算机系统》学习笔记

> **课程来源**：卡内基梅隆大学（Carnegie Mellon University）计算机科学学院 **课程编号**：15–213 / 15–513 / 14–513 / 18–213 *Introduction to Computer Systems*（ICS） **课程主页**：https://www.cs.cmu.edu/~213/（Fall 2026 学期） **教材官网**：https://csapp.cs.cmu.edu/（含 11 个官方 lab 的完整讲义与自学 handout） **教材**：Randal E. Bryant, David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, Third Edition, Pearson, 2016 （中文版《深入理解计算机系统》第 3 版，机械工业出版社） **辅助教材**：Brian W. Kernighan, Dennis M. Ritchie, *The C Programming Language*, 2nd Ed. **笔记语言**：中文（技术术语保留英文原文） **笔记定位**：以 **"程序在机器上的真实行为"** 为主线，逐讲精讲 24 讲官方内容 + 期末复习，并给出 11 个官方 lab 的清单、难度评估与最优执行顺序。 **素材来源**：CMU 15–213 Fall 2025 完整 24 讲幻灯片、Fall 2026 已发布讲义、 Recitation 与 Bootcamp 讲义、教材官网 11 个 lab 的 writeup 与自学 handout。 所有资料来源均已在本文档中透明记录（见 五、资料来源与获取记录）。

写在前面：为什么要学 CSAPP

有一个问题，几乎所有写了几年代码的人都答不完整：

> **当你敲下 `./a.out` 并按下回车，从这一瞬间到屏幕上出现第一行输出，计算机到底做了什么？**

你的 C 代码被编译器变成了什么？变量真的"存在"吗，它住在哪里？`malloc` 给你的那块内存， 物理上在芯片的哪个位置？为什么把矩阵遍历的循环内外层对调一下，程序就快了三倍？ 为什么栈上多写几个字节，整个程序的控制流就被别人接管了？

CSAPP 这门课（CMU 的 15–213，俗称"镇系神课"）就是来回答这些问题的。它不教你怎么写代码， 它教你**你的代码在真实硬件上到底发生了什么**。学完之后你会获得三种能力：

能力	具体表现	对应本笔记
写出更快的代码	知道缓存怎么工作，就知道怎么让数据"顺手"；知道 CPE 怎么算，就知道瓶颈在哪	Lecture 9–10、15
写出更可靠的代码	知道补码怎么溢出、栈帧怎么布局，就知道哪里会崩、会被攻击	Lecture 2、5–6、8
看懂底层的一切	能读汇编、能读 ELF、能读页表、能读 `/proc`、能读懂系统调用	Lecture 3–7、11–12

而这门课最值钱的部分，是它配了一整套**魔鬼实验（lab）**：从只准用 `! ~ & ^ | + << >>` 实现位运算，到用 gdb 拆炸弹，到构造缓冲区溢出攻击和 ROP 链，到写出能在 90% 命中率下 跑赢基准的矩阵转置，到从零实现一个 malloc，到写一个带作业控制的 shell，再到写一个 支持并发的缓存代理。**每一个 lab 都是一次真正的"系统级"工程训练。**

一、课程概览（Course Overview）

1.1 官方课程定位

课程主页对 ICS 的定位原文是：

> *"The ICS course provides a programmer's view of how computer systems execute programs, store information, and communicate. It enables students to become more effective programmers, especially in dealing with issues of performance, portability and robustness. It also serves as a foundation for courses on compilers, networks, operating systems, and computer architecture, where a deeper understanding of systems-level issues is required."*

拆解成三条主线：

1. **程序员视角（programmer's view）**——不是从电路出发，而是从"我写的这段 C 代码会怎样"出发。
2. **三个关切（three concerns）**——**性能（performance）**、**可移植性（portability）**、 **健壮性（robustness）**。这三者就是全课程的技术价值锚点：
3. 性能 → 缓存、代码优化、并行（Lecture 9–10、15、21–24）
4. 可移植性 → 数据表示、字节序、对齐、链接、ISA（Lecture 2、6–7）
5. 健壮性 → 缓冲区溢出、信号、并发竞态（Lecture 5–6、17、21–23）
6. **后续课程的地基**——编译器、网络、操作系统、体系结构课都以它为先修。

课程官方覆盖的主题列表（Topics covered）：

> machine-level code and its generation by optimizing compilers, performance evaluation and optimization, computer arithmetic, memory organization and management, networking technology and protocols, and supporting concurrent computation.

对应到教材章节，就是 CS:APP3e 的第 1–12 章（跳过第 4 章处理器体系结构大部分内容， 第 8 章之前插入链接与调试，第 11 章网络与文件系统合并讲授）。

1.2 先修要求与授课规模

- **先修课**：15-122 *Principles of Imperative Computation*（CMU 的 C 语言与算法入门课）。 对自学者而言，等价要求是：**熟练 C 语言（指针、结构体、动态内存）+ 基本数据结构与算法**。
- 12 学分，每周两次 80 分钟讲座 + 一次 50 分钟 recitation + 不定期 bootcamp。
- Fall 2026 由 Phillip Gibbons、Seth Goldstein、Taha Khan 三位老师分别讲授 15-213/15-513/14-513 三个平行班（内容一致，班型不同）。

1.3 考核结构（Fall 2026 官方）

项目	占比	说明
9 个 lab（L0–L8）	**50%**	全部通过 Autolab 提交，权重不等（见实验清单）
书面作业（Written Assignments）	6%	结合 peer review
随堂纸笔小测（in-class paper quizzes）	4%	全学期 10 次，去掉最低 2 次
期中考试（Midterm）	15%	闭卷，第 8 周左右课堂内进行
期末考试（Final）	25%	闭卷，考试周

注意 lab 占了一半的分。 这门课的重心在动手，不在背书。

1.4 迟交政策（Fall 2026）

- 所有 lab 截止于当天 23:59 ET。
- 最多迟交 3 天，**L0 与 L8 最多迟交 1 天**。
- 全学期每人 **5 个 grace day**（宽限日），自动应用于迟交作业； 每个 lab 可用的 grace day 数量有上限（见实验清单表）。
- 超出上限或 grace day 用完后，**每天扣 15%**。

二、讲次总表（以 CMU 15-213 Fall 2025 完整学期为准）

> **为什么用 Fall 2025 的日程做基准？** 因为它是目前**唯一完整发布了全部 24 讲幻灯片** 的学期（Fall 2026 正在进行中，只发布到第 9 讲）。两学期的讲次顺序、主题、教材对应 完全一致，Fall 2025 因此是自学的最佳基准。下表日期为 Fall 2025 实际授课日。

2.1 讲座序列（24 讲）

讲次	日期	主题（官方原题）	本笔记章	教材	关联 Lab
L01	Aug 26	Overview	第 1 章	Ch.1	—
L02	Aug 28	Bits, Bytes, & Integers	第 2 章	2.1–2.4	L1 发放
L03	Sep 02	Machine Programming (Part I)	第 3 章	3.1–3.6	L0 截止
L04	Sep 04	Machine Programming (Part II)	第 4 章	3.6–3.7	L2 发放
L05	Sep 09	Machine Programming (Part III)	第 5 章	3.7–3.9	L1 截止
L06	Sep 11	Machine Programming (Part IV)	第 6 章	3.8–3.11	—
L07	Sep 16	Linking	第 7 章	Ch.7	L2 截止，L3 发放
L08	Sep 18	Design and Debugging	第 8 章	3/9 延伸	—
L09	Sep 23	The Memory Hierarchy	第 9 章	6.1–6.3	—
L10	Sep 25	Cache Memories	第 10 章	6.4–6.7	L3 截止，L4 发放

讲次	日期	主题（官方原题）	本笔记章	教材	关联 Lab
L11	Sep 30	Virtual Memory: Concepts	第 11 章	9.1–9.6	—
L12	Oct 02	Virtual Memory: Details	第 12 章	9.7–9.8	—
L13	Oct 07	Dynamic Memory Allocation: Basic	第 13 章	9.9	—
L14	Oct 09	Dynamic Memory Allocation: Advanced	第 14 章	9.10–9.12	L4 截止，L5 发放
—	Oct 14/16	秋假（fall break）	—	—	—
—	**Oct 21**	**期中考试（In-class Midterm）**	—	Ch.1–9	—
L15	Oct 23	Code Optimization	第 15 章	Ch.5	—
L16	Oct 28	Processes and Multitasking	第 16 章	8.1–8.4	L5a 截止
L17	Oct 30	Exceptional Control Flow	第 17 章	8.5–8.8	—
L18	Nov 06	System Level I/O and File Systems	第 18 章	Ch.10	L5b 截止，L6 发放
L19	Nov 11	File Systems / Network Programming (Part I)	第 19 章	11.1–11.4	—
L20	Nov 13	Network Programming (Part II)	第 20 章	11.4–11.6	L6 截止，L7 发放
L21	Nov 18	Concurrent Programming	第 21 章	12.1–12.3	—
L22	Nov 20	Synchronization: Basic	第 22 章	12.4, 12.5.1–3	—
L23	Nov 25	Synchronization: Advanced	第 23 章	12.5.4–5, 12.7–8	L7 截止，L8 发放
L24	Dec 02	Thread-Level Parallelism	第 24 章	12.6	—
—	Dec 04	Frontiers of Computing（无讲义）	—	—	L8 截止
复习	Dec 05/07	Exam Review	第 25 章	全书	—

2.2 配套环节（Recitation 与 Bootcamp）

讲义**全部公开**，是自学时极易被忽略但价值极高的材料：

环节	内容	素材文件
Recitation 1	Introduction, Datalab	`F25-rec01_slides.txt` / `rec01_handout.txt`
Recitation 2–3	Bomb Lab（两讲）	`F25-rec02_slides.txt` / `F25-rec03_slides.txt`
Recitation 4	Attack Lab and Stacks	`F25-rec04_slides.txt` / `rec04_handout.txt`
Recitation 5	Caches and C Review	`F25-rec05_slides.txt`
Recitation 6	Caches & Blocking	`F25-rec06_slides.txt`
Recitation 7–8	Malloclab（Part I/II）	`F25-rec07_slides.txt` / `F25-rec08_slides.txt`
Recitation 9	Processes and Shells	`F25-rec09_slides.txt`
Recitation 10	Tshlab	`F25-rec10_slides.txt`
Recitation 11	Networking and Proxies	`F25-rec11_slides.txt`
Recitation 12	Synchronization	`F25-rec12_slides.txt`
Recitation 13	Exam Review	`F25-rec13_slides.txt`
Bootcamp 1	Linux, Command Line, Git	`lab1_slides.txt` / `lab1_handout.txt`
Bootcamp 2	Debugging & GDB	`lab2_slides.txt` / `lab2_handout.txt`
Bootcamp 3	GCC & Build Automation	`lab3_slides.txt` / `lab3_handout.txt`
Bootcamp 4	C Programming	`F25-cprogramming-bootcamp.txt`
Bootcamp 5	Post-checkpoint Malloc	`F25-malloc-bootcamp.txt`
Bootcamp 6	Exam Review	`F25-final.txt`
活动讲义	GDB 与汇编 / 链接 / 机器级编程（含答案版）	`F25-gdb-and-assembly*.txt`、`machine-*-soln.txt`

> **Bootcamp 1–3 是零基础的救命稻草。** 如果你没在 Linux 命令行下写过程序、 没用过 gdb、不清楚 `gcc` 的各个开关，**先做这三个 bootcamp 再开始 Lecture 3**。

三、11 个官方 Lab 的清单与执行顺序

教材官网 https://csapp.cs.cmu.edu/3e/labs.html 提供了 **11 套完整的 turnkey lab**， 每一套都带 writeup（讲义）、README（文件清单与提交要求）和**自学用 handout 压缩包** （不含答案，可自由下载）。这是 CSAPP 最珍贵的部分——**世界上几乎所有讲系统课的学校 都在用或改编这些 lab**。

3.1 十一个 lab 总览

#	Lab	官方核心任务	主要考察	难度	建议耗时	前置知识
1	**Data Lab**	只用 `! ~ & ^ | + << >>` 实现位运算与浮点函数	位级表示、补码、IEEE 754、UB	★★☆☆☆	8–15 h	Lecture 2
2	**Bomb Lab**	反汇编并"拆"掉 6 个关卡的二进制炸弹	汇编阅读、控制流、gdb	★★★☆☆	10–20 h	Lecture 3–5、8
3	**Attack Lab**	对两个有缓冲区溢出漏洞的 target 构造 5 次攻击	栈帧、代码注入、ROP	★★★★☆	12–25 h	Lecture 5–6、8
4	**Buffer Lab**（IA32 遗留版）	32 位版缓冲区溢出攻击（已被 Attack Lab 取代）	栈帧、IA32 调用约定	★★★☆☆	8–15 h	Lecture 5–6
5	**Arch Lab**	写 Y86-64 汇编、扩充 SEQ 处理器、优化流水线	指令集、数据通路、流水线	★★★★☆	15–30 h	Lecture 3–4 + 教材 Ch.4
6	**Cache Lab**	Part A 写缓存模拟器；Part B 优化矩阵转置	缓存组织、LRU、分块、局部性	★★★★☆	15–30 h	Lecture 9–10、15
7	**Performance Lab**	优化卷积/矩阵转置内核的 CPE	代码优化、循环变换、CPE 测量	★★★☆☆	8–15 h	Lecture 9–10、15
8	**Shell Lab**	实现带作业控制的 Unix shell（tsh）	进程、信号、系统调用、竞态	★★★★☆	15–25 h	Lecture 16–18
9	**Malloc Lab**	从零实现 `malloc` / `free` / `realloc`	堆布局、分配器设计、空间/时间权衡	★★★★★	20–40 h	Lecture 11–14
10	**Proxy Lab**	实现并发缓存 Web 代理	socket、并发、同步、HTTP、缓存	★★★★☆	15–30 h	Lecture 18–23
11	**SFS Lab**	实现并发文件系统（CMU 自加，教材官网无）	文件系统、并发、锁	★★★★☆	15–25 h	Lecture 18–19、21–23

> **关于 SFS Lab**：这是 CMU 15-213 在 Fall 2025 / Fall 2026 新增的第 9 个 lab（对应 课程编号 L8，4%）。它不在教材官网的 11 个 lab 列表中（教材官网列表到 Proxy Lab 为止）， 属于 CMU 自研的并发文件系统 lab。本文档把它列在最后，并明确标注这一点。
>
> **两个"取代关系"**：Buffer Lab（32 位）已被 **Attack Lab**（64 位）取代； Arch Lab 另有 Legacy Y86（32 位）版本。自学时**优先做 64 位版本**。

3.2 推荐执行顺序（★ 本章最重要的部分）

ordering 的三个原则： **① 先补齐工具链能力 → ② 再按课程时间线推进 → ③ 把最难的 Malloc 留足时间。**

```
阶段 0：环境与工具（不做 lab，但是所有 lab 的前提）
  ├─ Bootcamp 1: Linux / 命令行 / Git
  ├─ Bootcamp 2: Debugging & GDB
  └─ Bootcamp 3: GCC & Build Automation
        │
        ▼
阶段 1：数据与机器码（Lecture 2–8 期间）
  ① L0 C Programming       （热身，确认 C 与 Linux 工具链没问题）
  ② Data Lab        ★★☆☆☆  （位运算 + 补码 + 浮点，检验 Lecture 2）
  ③ Bomb Lab        ★★★☆☆  （gdb + 汇编，检验 Lecture 3–5；全课程最"好玩"）
  ④ Attack Lab      ★★★★☆  （栈帧 + 代码注入 + ROP，检验 Lecture 5–6）
  并行可选：Buffer Lab（32 位遗留，只想练栈溢出可跳过）
        │
        ▼
阶段 2：存储与性能（Lecture 9–15 期间，期中考试前后）
  ⑤ Cache Lab       ★★★★☆  （模拟器 + 矩阵转置优化，知识密度最高）
  并行可选：Performance Lab（与 Cache Lab Part B 高度重叠，可二选一）
  ⑥ Arch Lab        ★★★★☆  （Y86-64 + 流水线，需要额外读教材 Ch.4，可与 Cache Lab 换序）
        │
        ▼
阶段 3：内存管理（Lecture 11–14 之后，最需要时间的一段）
  ⑦ Malloc Lab      ★★★★★  （全课程最难，务必留 3 周以上；先做 checkpoint）
        │
        ▼
阶段 4：并发与系统（Lecture 16–24 期间）
  ⑧ Shell Lab       ★★★★☆  （进程 + 信号 + 作业控制）
  ⑨ Proxy Lab       ★★★★☆  （socket + 线程 + 缓存 + 同步，综合度最高）
  ⑩ SFS Lab         ★★★★☆  （并发文件系统，CMU 自加）
```

3.3 十条执行建议（踩过坑的人才知道）

1. **先做 L0 热身。** 它只有 2% 权重，但会帮你确认 gcc、make、gdb、Autolab 提交链路都通。
2. **Data Lab 不要抄答案。** 网上答案唾手可得，但 Data Lab 的真正价值是逼你理解 补码与 IEEE 754 的位级结构；抄了它，Lecture 2 就白学了。
3. **Bomb Lab 是"gdb 训练营"。** 用 `objdump -d` 找到 `phase_1`…`phase_6`， 对每个 `phase_N` 下断点，用 `x/s`、`x/d`、`info registers` 逐个推。 **不要**用 `strings` 直接找答案——那会让你什么都学不到。
4. **Attack Lab 先画栈帧再动手。** 把 `getbuf` 的栈帧、`buf` 起始地址、 返回地址相对偏移量画清楚，剩下的就是填字节。ROP 部分（Phase 4/5）必须 先在 `.rodata` 里用 `objdump -d` 找 gadget。
5. **Cache Lab Part A 建议用时间戳数组实现 LRU**，别用链表——trace 规模大时链表会慢。 Part B 的 32×32 用 8 个 8×8 分块 + 局部变量暂存；64×64 必须用 4×4 子块 + `B` 数组分 8 块的特殊分块法才能拿到满分。
6. **Malloc Lab 分两阶段做**（对应课程的 L5a checkpoint / L5b final）： 先做**隐式空闲链表 + 立即合并**保证正确性，再升级成**分离空闲链表 + 显式链表 + 边界标记优化**追求利用率。**先用 `mdriver` 跑通再优化**，否则效率分数无从谈起。
7. **Shell Lab 的关键是"信号时序"。** `waitpid` 的 `WNOHANG|WUNTRACED`、 `sigprocmask` 屏蔽 `SIGCHLD` 避免竞态、`setpgid` 建立进程组——这三件事做对了， 作业控制就成了一半。
8. **Proxy Lab 先做"能用"，再做"并发"，最后做"缓存"。** 顺序是： 迭代式代理 → 多线程（每连接一线程）→ 加读者–写者锁保护缓存 + LRU 淘汰。把这三步混在一起做，会同时面对三个 bug。
9. **每个 lab 第一步永远是读 README。** README 里写清了文件清单、`make` 目标、 提交要求、以及"哪些文件不能改"。**改错文件是零分的常见原因。**
10. **利用官方 handout 自带的测试工具。** 每个 lab 都有自己的 driver： `btest`（Data）、`mdriver`（Malloc）、`tracegen` / `test-csim` / `test-trans`（Cache）、 `tsh` 参考实现与 `sdriver.pl`（Shell）、`driver.sh`（Proxy）。 这些工具在 handout 里都给了，**它们比任何第三方答案都可靠**。

3.4 官方 lab 权重与截止日期（Fall 2026）

下表取自 Fall 2026 课程 Labs 页面，展示了这门课**如何给"动手部分"分配 50% 的分数**—— 权重最高的不是最难的 Malloc，而是 Data Lab（7%）、Malloc final（7%）、Shell（7%）三个， 它们恰好对应"位级理解""内存管理""并发控制"三大核心能力。

代号	名称	权重	Grace Days	发放	截止	代码评审登记
L0	C Programming	2%	1	Tue Aug 25	Tue Sep 01	—
L1	Data	7%	1	Thu Aug 27	Tue Sep 08	Thu Sep 10
L2	Bomb	6%	1	Thu Sep 03	Tue Sep 15	—
L3	Attack	4%	1	Tue Sep 15	Thu Sep 24	—
L4	Cache	5%	2	Thu Sep 24	Thu Oct 08	Sat Oct 10

代号	名称	权重	Grace Days	发放	截止	代码评审登记
L5a	Malloc (checkpoint)	4%	2	Thu Oct 08	Tue Oct 27	Thu Oct 29
L5b	Malloc (final)	**7%**	2	（同 L5a）	Tue Nov 03	Thu Nov 05
L6	Shell	**7%**	2	Tue Nov 03	Thu Nov 12	Sat Nov 14
L7	Proxy	4%	2	Thu Nov 12	Tue Nov 24	Sun Nov 29
L8	SFS	4%	1	Thu Nov 19	Thu Dec 03	—
	合计	**50%**				

代码评审（Code Review） 是 CMU 这门课的一大特色，自学时可以自己模拟：

- **风格评审**（L4、L5a、L5b、L6、L7）：与助教面谈代码风格，依据课程的 Style Guidelines。
- **"帕森斯拼图"测试**（L1、L4、L5b、L6、L7）：10 分钟小测， 把打乱的代码片段拼回正确顺序并解释——**考的是"你真的理解这段代码为什么这么写"**。

> **自学建议**：即使没人给你打分，也请对自己执行这两步。 把代码读一遍，问自己"这一行为什么必须在这儿"；把关键函数打乱重排，看能不能复原。这比再多刷一道题有用。

3.5 如何获取这些 lab 的 handout

教材官网的 lab 页面对**自学者免费开放 handout**（只有解答（solution）需要教师账号）：

获取方式	地址
lab 列表与说明	https://csapp.cs.cmu.edu/3e/labs.html
Data Lab 自学包	https://csapp.cs.cmu.edu/3e/datalab-handout.tar
Bomb Lab 练习炸弹（已禁用评分服务器）	https://csapp.cs.cmu.edu/3e/bomb.tar
Attack Lab 自学 targets（需用 `-q` 选项运行）	https://csapp.cs.cmu.edu/3e/target1.tar
Cache / Malloc / Shell / Proxy / Arch / Perf Lab 自学包	同一页面的 "Self-Study Handout" 链接

> **关于"网上答案"的严肃提醒。** 课程主页在 CS 自学指南中有一句原话值得抄在这里：*"这门课由于过于出名，全世界的码农争相学习，导致其 Project 的答案在网上几乎唾手可得。 但如果你真的想锻炼自己的代码能力，希望你不要借鉴任何第三方代码。"* CSAPP 的 lab 之所以经典，是因为**做的过程**才是学习本身。抄答案等于把最值钱的部分扔了。

四、如何使用这份笔记

4.1 三种读法

你的情况	建议路径
零基础，从没碰过系统编程	先做 Bootcamp 1–3（本文档第二章 2.2）→ 读第 1–2 章 → 做 L0 + Data Lab → 再往下
有 C 基础，想系统补齐	按第 1–24 章顺序读，每读完一段就去做对应 lab（见第三章 3.2 的流水线）
只想查某个专题	直接从讲次总表（2.1）跳到对应章；`Lecture 9–10` 查缓存、`Lecture 11–14` 查内存、`Lecture 21–23` 查并发

4.2 本笔记的章节结构

每一讲都严格采用 **7 段式**结构，便于你"学—练—查"：

1. **N.1 概述**——这一讲要解决什么问题，在知识地图上的位置
2. **N.2 核心概念与底层机制图解**——每个概念都有 ASCII 图（内存布局 / 位域 / 数据通路）
3. **N.3 代码示例与底层机制分析**——完整可编译的代码 + 真实汇编 + 内存布局 + 验证命令
4. **N.4 实验关联**——这一讲对应哪个 lab 的哪一部分，会在哪里踩坑
5. **N.5 常见错误与调试技巧**——每条都给出可执行的具体命令
6. **N.6 关键要点**——必须记住的结论与设计原则

7. **N.7 思考题（带答案）**——含至少一道计算/推演题

4.3 一个学习节奏建议

CS 自学指南给这门课的估算是 **150 小时**（难度五星），其中：

- **读教材 + 看讲义**：约 60 小时（教材 Ch.1–12，讲义 24 讲）
- **做 lab**：约 90 小时（按第三章 3.1 的"建议耗时"累加，11 个 lab 合计约 140–280 小时， 自学者可只挑 6–7 个核心 lab，约 90–150 小时）

不要试图一周速通。 这门课的知识是"层层叠上去"的：不懂补码就读不懂汇编， 不懂栈帧就做不了 Attack Lab，不懂缓存就做不了 Malloc Lab 的性能优化。 **每一层都要真的踩实。**

五、资料来源与获取记录

本文档的技术内容全部来自**公开可访问的官方材料**，具体获取情况透明记录如下。

5.1 已获取的官方材料

类别	内容	数量/说明
课程主页	https://www.cs.cmu.edu/~213/ 首页、Schedule、Labs、Assignments、Exams、Resources、Syllabus	Fall 2026 完整
讲座幻灯片	Fall 2025 全部 **24 讲** PDF（`01-overview` … `24-parallelism`）	完整，主素材
讲座幻灯片	Fall 2026 已发布的 **9 讲** PDF（至 `09-memory-hierarchy`）	部分，用于对照
Recitation 讲义	13 次 recitation 的 slides（含 handout 与答案版）	完整
Bootcamp 讲义	Linux/Git、GDB、GCC、C 编程、Malloc、期末复习共 6 个	完整
课堂活动	机器级编程三套活动 + 链接活动 + GDB&汇编活动（**含答案版**）	完整
教材官网	https://csapp.cs.cmu.edu/3e/labs.html 11 个 lab 的 writeup（PDF）+ README + 自学 handout（tar）	完整
教材官网	教师专区（Instructor Site）、lab 解答（solution）	**需要 CS:APP 教师账号，未获取**

5.2 获取方式

所有材料通过脚本 `csapp_data/fetch.sh` 一次性抓取，随后用 `csapp_data/pdf2txt.py` （PyMuPDF）转换为纯文本，存放于：

```
csapp_data/
├── fetch.sh                抓取脚本（可重跑，幂等）
├── pdf2txt.py              PDF → 文本转换脚本
├── html/                   课程页面 HTML 存档
├── pdf/                    74 份官方 PDF（讲座/recitation/bootcamp/活动/syllabus）
├── txt/                    转换后的纯文本（含每页页码标记），撰写笔记的素材
├── labs/                   11 个 lab 的 writeup PDF + README
└── handouts/               11 个 lab 的自学 handout（已解包）
```

5.3 需要说明的边界

1. **lab 的解答（solution）不可得。** 教材官网只对教师账号开放 solution， 本文档因此**不提供任何 lab 答案**，只提供官方 handout、任务说明与解题方法论。
2. **Fall 2026 学期尚未结束。** 第 10 讲及以后的 Fall 2026 幻灯片还没发布， 本文档对应的讲义素材以 **Fall 2025 完整版**为准（两学期讲次与主题一致）。
3. **教材原文不在本文档中。** CS:APP3e 是受版权保护的出版物，本文档**不复制教材原文**， 而是基于公开讲义重新组织、推导、举例并补充可运行代码。请自行获取正版教材对照阅读。
4. **成绩数据来自官方页面。** 2.1 表中的日期与 lab 时间线取自 Fall 2025 官方 schedule， 1.4 节的迟交政策与 1.3 节的考核比例取自 Fall 2026 官方页面。
5. **代码都是实际验证过的。** 本文档各章出现的 C 代码均在本地 `gcc 12.2.0` （`-g -Wall -std=c11`）下实际编译运行，汇编片段由 `gcc -Og -S` / `objdump -d` 真实生成后录入，调试与内存检查输出取自真实的 `gdb` / `valgrind` / AddressSanitizer 会话。

目录

- 写在前面：为什么要学 CSAPP
- 一、课程概览（Course Overview）
- 二、讲次总表
- 三、11 个官方 Lab 的清单与执行顺序
- 四、如何使用这份笔记
- 五、资料来源与获取记录
- **第二部分 · 分讲学习笔记**
- Lecture 1 … Lecture 25（共 25 章，见讲次总表 2.1）
- 附录 A · CSAPP 核心概念与命令速查表

第二部分 · 分讲学习笔记

Lecture 1: 课程概述与计算机系统漫游 (Course Overview; A Tour of Computer Systems)

> **讲义对应**：CMU 15–213 Lecture 1 — Course Overview（素材：`F25-01-overview.txt`） **教材对应**：CS:APP3e 第 1 章（1.1—1.10） **关联 Lab**：无直接关联（但为 L0 C Programming Lab 及全部 lab 建立环境认知）

1.1 概述

本讲回答两个问题：**这门课是干什么的**，以及**一个 C 程序从磁盘上的文本到屏幕上的一行输出，中间发生了什么**。课程主题是"系统知识就是力量"：理解硬件与软件（操作系统、编译器、库、网络协议）如何协同支撑应用程序的执行。本讲先给出**五大现实**，再用 `hello` 程序的生命周期把后续 24 讲串成一条线。承接先修课 15–122，通向 Lecture 2 的信息表示与 Lecture 3 起的机器级编程。

1.2 核心概念与底层机制图解

1.2.1 课程定位与五大现实（Course Theme & Five Realities）

- **定义与目的**：这是一门 **programmer-centric（程序员视角）** 的系统课，而非 builder-centric（构建者视角）。体系结构课让你用 Verilog 设计流水线处理器，操作系统课让你实现 OS 的一部分，编译器课让你写一个简单语言的编译器——它们都在"造"系统。15–213 回答的是：我写的这行 C 代码，在真实机器上会变成什么、跑多快、什么时候会错。
- **直观解释（"它是什么？"）**：多数 CS/CE 课程强调抽象——抽象数据类型、渐近分析。**抽象是好的，但别忘了抽象下面的现实**。像开车不必懂发动机原理，但车在高速上抛锚时，你会希望自己知道"仪表盘亮红灯"意味着什么。
- **五大现实**（Great Realities），它们同时是本课程五大板块的动机：

#	现实	含义	后续讲次
1	Ints are not Integers, Floats are not Reals	算术受表示有限性约束：整数满足环性质，浮点只满足序性质	Lecture 2
2	You've Got to Know Assembly	机器级执行模型是理解 bug、性能、系统软件、恶意代码的关键	Lecture 3—6
3	Memory Matters	RAM 是"非物理的抽象"：内存有限、引用 bug 后果遥远、访存性能不均匀	Lecture 9—14
4	More than asymptotic complexity	常数因子也重要；同一算法写法不同可差 10:1 性能	Lecture 15
5	Computers do more than execute programs	要把数据送进送出，还要互相通信（网络带来并发、不可靠介质等问题）	Lecture 18—23

- **现实 #4 的量化证据**：同一台 2.0 GHz Intel Core i7 Haswell 上，把 $\mathbf{2048 \times 2048}$ 的 `int` 矩阵从 `src` 复制到 `dst`，仅循环顺序不同就差近 19 倍：

```
for (i...) for (j...) dst[i][j] = src[i][j];   /* copyij: 按行遍历 */   4.3 ms
for (j...) for (i...) dst[i][j] = src[i][j];   /* copyji: 按列遍历 */  81.8 ms
```

两者指令条数与渐近复杂度（同为 $O(n^2)$）几乎一样，但 `copyji` 每次访存跨过一整行（$\mathbf{2048 \times 4 = 8192}$ 字节），空间局部性极差，几乎每次都**缓存不命中（cache miss）**。**与硬件的对应**：`dst[i][j]` 编译后是带比例变址寻址的 `movl`（如 `movl (%rdi,%rax,4), %ecx`），硬件对每次访存都向缓存层

次发起请求，命中与否决定这条指令花 4 个周期还是 200 个周期。详见 Lecture 9、Lecture 10。

1.2.2 计算机系统漫游：hello 程序的完整生命周期（A Tour of Computer Systems）

- **直观解释（"它是什么？"）**：把一个 C 程序想象成一份**多语言交接的快递包裹**：写于中文（C 源程序），译成英文（汇编程序），打包成二进制（机器码），拼在一起（链接），装上卡车（加载）。
- **底层机制图解**：`hello.c` 是**只由 ASCII 字符组成的文本文件**，必须被翻译成二进制机器码才能执行。这条流水线叫**编译系统（compilation system）**：

```
hello.c  (源程序, 文本)
        |
        v
     +-----------------------------------+
     | 预处理器 (Preprocessor)           |
     | cpp: 展开 #include / #define      |
     +-----------------------------------+
hello.i  (修改后的 C 源程序, 文本)
        |
        v
     +-----------------------------------+
     | 编译器 (Compiler)                 |
     | cc1: C 源程序 -> 汇编程序         |
     +-----------------------------------+
hello.s  (汇编程序, 文本)
        |
        v
     +-----------------------------------+
     | 汇编器 (Assembler)                |
     | as: 汇编程序 -> 可重定位目标文件  |
     +-----------------------------------+
hello.o  (可重定位目标程序, 二进制)
        |
        v
     +-----------------------------------+
     | 链接器 (Linker)                   |
     | ld: 符号解析 + 重定位             |
     +-----------------------------------+
hello    (可执行目标程序, 二进制)
```

- **四个阶段的职责**：
- **预处理阶段（preprocessing phase）**：预处理器（`cpp`）根据 `#` 命令修改源程序：`#include <stdio.h>` 把系统头文件内容直接插入，`#define` 做宏替换，得到另一个 C 程序 `hello.i`。
- **编译阶段（compilation phase）**：编译器（`cc1`）把 `hello.i` 翻译成汇编程序 `hello.s`，高级语言的控制结构在此展开为跳转与调用。
- **汇编阶段（assembly phase）**：汇编器（`as`）把 `hello.s` 翻译成机器语言指令，打包成**可重定位目标程序（relocatable object program）** `hello.o`；其中的机器码还不知道自己最终会被放在哪个地址。
- **链接阶段（linking phase）**：链接器（`ld`）把自己的 `hello.o` 与标准库中所需的目标文件（如 `printf.o`）合并，完成**符号解析（symbol resolution）**与**重定位（relocation）**，得到可执行目标程序 `hello`。详见 Lecture 7。
- **与机器码/硬件的对应**：`hello.i`、`hello.s` 仍是文本，只有 `hello.o` 与 `hello` 是二进制。运行时 CPU 从**程序计数器（PC, Program Counter）**指出的地址取指令、译码、执行，再更新 PC——"运行程序"就是硬件不断重复"取指—译码—执行"。

1.2.3 计算机硬件组成（Hardware Organization）

- **直观解释（"它是什么？"）**：把计算机看成一座**图书馆**：总线是走廊，I/O 设备是收发室和复印机，主存是书架，CPU 是那位跑得飞快但记性只有巴掌大的馆员——他必须不断往返书架取书。
- **底层机制图解**：

- **四大部件**：
- **总线（Bus）**：贯穿系统的电子管道，在部件之间传送**定长的字节块（字，word）**。x86-64 上字长 8 字节。
- **I/O 设备（I/O Devices）**：系统与外部世界的通道（键盘、鼠标、显示器、磁盘、网卡）。每个设备经由**控制器（controller）**或**适配器（adapter）**接到总线。所有 I/O 设备都通过**文件（file）**这一抽象被访问。
- **主存（Main Memory）**：执行程序时存放程序与数据的临时设备。物理上由一组 DRAM（Dynamic Random Access Memory）芯片组成，逻辑上是一个线性字节数组，每个字节有唯一地址。
- **处理器（CPU, Central Processing Unit）**：解释（执行）主存中指令的引擎。核心是**程序计数器（PC）**，任何时刻指向主存中某条机器语言指令；另有**寄存器文件（register file）**——少量定长寄存器组成的、小而快的存储设备，以及 **ALU（算术/逻辑单元）**。CPU 反复执行加载、存储、操作、跳转这几类动作。
- **`hello` 的读写路径**：shell 执行 `linux> ./hello` 后发生两件事，注意**两条路径的发起者不同**：

加载时 shell 调用**加载器（loader）** 把 `hello` 的代码和数据从磁盘**直接复制到主存**，由 DMA（Direct Memory Access，**直接存储器访问**）完成，不经 CPU 寄存器。随后 CPU 执行 `main` 的机器码，把字符串从主存**加载（load）**到寄存器，再**存储（store）**到显示适配器。

1.2.4 存储层次结构（The Memory Hierarchy）

- **定义与目的**：**存储层次结构**用层次化的组织弥合 CPU 与主存之间不断扩大的速度差距——越靠近 CPU 越小、越快、每字节越贵；越远则越大、越慢、越便宜。
- **直观解释（"它是什么？"）**：像一位作家的书桌：便利贴是寄存器，桌面摊开的是 L1/L2/L3 缓存，书架是主存，图书馆书库是本地磁盘，网上图书馆是远程存储。**每一层都是下一层的缓存（cache）**——书架上放的就是书库里你以为会用到的一小部分。
- **底层机制图解**（数值取教材 Figure 6.23 口径）：

```
                        +-------------------------------------------------------------------+
                        | L0  寄存器 (Registers)          < 1 KB / 0 周期            4-8 字节字 |
                        +-------------------------------------------------------------------+
                    +-------------------------------------------------------------------+
                    | L1  L1 d-cache (SRAM, 片内)     32 KB / 4 周期             64 字节块  |
                    +-------------------------------------------------------------------+
                +-------------------------------------------------------------------+
                | L2  L2 cache (SRAM, 片内)       256 KB / 10 周期           64 字节块  |
                +-------------------------------------------------------------------+
            +-------------------------------------------------------------------+
            | L3  L3 cache (SRAM, 片内共享)   8 MB / 40 周期             64 字节块  |
            +-------------------------------------------------------------------+
        +-------------------------------------------------------------------+
        | L4  主存 (Main Memory, DRAM)    8 GB / 100 周期            4-KB 页    |
        +-------------------------------------------------------------------+
    +-------------------------------------------------------------------+
    | L5  本地磁盘 (Local Disk)       1 TB / 10,000,000 周期     文件块     |
    +-------------------------------------------------------------------+
+-------------------------------------------------------------------+
| L6  远程存储 (Remote Storage)   inf / 1,000,000,000 周期   网页/文件  |
+-------------------------------------------------------------------+
```

- **基本思想**：对每个 k，第 k 层更快更小的设备**充当第 $k+1$ 层更大更慢设备的缓存**。它之所以有效，是因为**局部性（locality）**：程序倾向访问最近用过的数据（**时间局部性**，temporal locality）及其邻近地址（**空间局部性**，spatial locality），故访问第 k 层的频率远高于第 $k+1$ 层。
- **理想中的"大创意"**：存储层次创造出一个巨大存储池，**价格接近底部最便宜的存储，却以顶部最快存储的速度向程序提供数据**。
- **量级感**：从 L1 的 4 周期到磁盘的 10,000,000 周期跨越 6 个数量级。若把一次 L1 命中比作 1 秒，一次磁盘访问相当于 29 天。缓存组织（$C = S \times E \times B$、$s = \log_2 S$、$b = \log_2 B$、$t = m - s - b$）与局部性利用详见 Lecture 9、Lecture 10。

1.2.5 操作系统：硬件与应用的中间层（Operating System as Intermediary）

- **定义与目的**：应用程序不直接操作硬件，而通过**操作系统（Operating System）**使用硬件。OS 有两个基本功能：**防止硬件被失控的应用程序滥用**，以及**向应用程序提供简单一致的机制来控制复杂而大相径庭的低级硬件设备**。
- **直观解释（"它是什么？"）**：OS 像**酒店前台**。你不必知道锅炉房怎么烧水，只要说"我要一间房""我要洗衣服"。前台既保证其他客人不闯进你的房间（保护），也把五花八门的后台流程统一成几个简单服务（抽象）。
- **底层机制图解**：

```
+-------------------------------------------------------------+
| 应用程序 (Application Programs):  hello, shell, browser, ... |
|-------------------------------------------------------------|
| 操作系统 (Operating System)                                   |
|    文件 (file)          : 对 I/O 设备的抽象                   |
|    虚拟内存 (virtual memory) : 对主存 + 磁盘的抽象            |
|    进程 (process)       : 对处理器/主存/I/O 的抽象            |
|-------------------------------------------------------------|
| 处理器   主存   I/O 设备   网络                               |
+-------------------------------------------------------------+
```

- **三大抽象**：
- **文件（file）**：对 **I/O 设备**的抽象。每个 I/O 设备都被模型化为一个文件，所有输入输出都可通过读写文件完成。详见 Lecture 18。
- **虚拟内存（virtual memory）**：对**主存和磁盘**的抽象，为每个进程制造"独占使用主存"的假象；虚拟地址由硬件地址翻译机制映射到物理内存，未使用的部分可暂存在磁盘。详见 Lecture 11、Lecture 12。
- **进程（process）**：对**处理器、主存和 I/O 设备**的抽象，即"一个正在运行的程序"；一个系统可同时运行多个进程，每个都好像独占硬件。**并发运行**指两个进程的指令交错执行，需要**上下文切换（context switch）**。详见 Lecture 16、Lecture 17。
- **与机器码/硬件的对应**：`hello` 调用 `printf` 最终落到 `write` 系统调用——一条把控制权从用户态交给内核态的指令；内核再通过设备驱动与 I/O 控制器打交道。

1.2.6 抽象：ISA 与并行的三个层次（Abstraction; Concurrency & Parallelism）

- **ISA（Instruction Set Architecture，指令集架构）**：对实际处理器硬件的抽象，规定指令、寄存器、寻址方式与行为语义，而把流水线级数、乱序执行、分支预测等微架构细节隐藏起来。**直观解释**：ISA 就像**汽车的驾驶接口**——方向盘、油门、刹车；不同厂家、不同排量的车共用同一接口，会开车的人能开任何一辆。只要遵循同一 ISA，同一份机器码就能在不同微架构的 Intel/AMD 处理器上正确运行。**虚拟机（virtual machine）**进一步把整台计算机（含操作系统）抽象成软件对象。**并行有三个层次**：
- **线程级并发（thread-level concurrency）**：在进程抽象之上构建**线程（thread）**抽象。**单处理器系统**上的并发是模拟出来的（快速切换，任一时刻只执行一个线程，这种交错叫**并发 concurrency**）；**多处理器系统（multiprocessor）**中一个 OS 内核控制多个 CPU，各 CPU 有独立的 PC、寄存器文件与 L1/L2 缓存（共享 L3 与主存），可**真正并行（parallel）**；**超线程（hyperthreading）**在单个物理核上运行多个逻辑控制流，硬件**复制**指令控制单元（PC、寄存器文件、操作队列）但**共享**功能单元（整数/浮点运算单元、load/store 单元、数据缓存），典型 $K = 2$。讲义的 shark 机器是 Intel Xeon E5520（Nehalem，约 2010 年），8 核 × 2 路超线程可同时执行 16 个线程，理论上限 16 倍加速比，但**从未在实测中达到**。
- **指令级并行（ILP, Instruction-Level Parallelism）**：处理器可同时执行多条指令，通过**流水线（pipelining）**让不同指令的不同阶段重叠。乱序（out-of-order）处理器把程序动态转换成操作流，再映射到多个功能单元上并行执行。讲义指出：**写操作耗时长，处理器会先缓存写、让读操作先走**——这正是多核内存一致性问题的来源之一。
- **单指令多数据并行（SIMD, Single Instruction Multiple Data）**：一条指令对向量的多个数据元素同时做同一运算，适用于图像/音频/科学计算等向量化场景。详见 Lecture 24。

1.2.7 Amdahl 定律（Amdahl's Law）

- **定义与目的**：把系统某一部分加速 k 倍后，整体能快多少？Amdahl 定律给出严格上限，并揭示一条残酷结论：**串行部分是性能改善的天花板**。
- **直观解释（"它是什么？"）**：讲义的旅行类比很清晰。从匹兹堡（PIT）直飞伦敦（LHR）要 7.5 小时。若先坐喷气机到纽约（JFK）花 1.5 小时，再坐协和式客机（SST）飞伦敦花 3.5 小时，共 5 小时，加速比 1.5 倍。若改用**超光速飞船（FTL）**飞越大西洋只需 0.01 小时，总时间仍是 $1.5 + 0.01 = 1.51$ 小时，加速比约 5 倍——**即使大西洋段无限快，你也必须先花 1.5 小时到纽约**。
- **公式**：设 T 为问题所需的**总顺序执行时间**，p 为可被加速部分所占比例（$0 \le p \le 1$），k 为该部分的加速倍数（$k > 0$），则加速后总时间为

$$T_k = \frac{pT}{k} + (1-p)T$$

即"可加速的部分快 k 倍，不可加速的部分原封不动"。整体加速比为

$$S(p,k) = \frac{T}{T_k} = \frac{1}{(1-p) + \frac{p}{k}}$$

令 $k \to \infty$ 得到**最大可能加速比** $S_{\max} = S(p,\infty) = \frac{1}{1-p}$，此时 $T_\infty = (1-p)T$。该上限**与并行资源多少无关**，反映的是**算法本身的局限性**。

- **讲义原例**：$T = 10$，$p = 0.9$，$k = 9$。$T_9 = 0.9 \times 10/9 + 0.1 \times 10 = 1.0 + 1.0 = 2.0$，即**5 倍加速比**（注意不是 9 倍！）。最大加速比 $T_\infty = 0.1 \times 10.0 = 1.0$，即 **10 倍**——使用无穷多并行资源也只能到 10 倍。
- **推论与陷阱**：
- 加速比对 k 的**边际收益递减**：$p = 0.9$ 时 k 从 4 提到 16，S 只从 3.08 涨到 6.40。
- p **比** k **更重要**：同样 $k = 16$，$p = 0.99$ 时 $S = 13.91$，$p = 0.50$ 时 $S = 1.88$。
- **并行 quicksort 的教训**：顶层划分（partition）无法并行，第二层最多 2 倍，第 k 层最多 2^{k-1} 倍；要大规模并行，必须先把划分步骤并行化。详见 Lecture 24。
- **与硬件的对应**：p 由**必须串行执行的部分**决定——I/O、同步（synchronization）、内存分配、依赖链。shark 机器的理论 16 倍从未达到，原因正是这些串行开销。

1.3 代码示例与底层机制分析

1.3.1 用 C 计算 Amdahl 加速比并观察程序地址空间

代码 (C)：

```c
/* amdahl.c — 演示 Amdahl 定律与程序地址空间布局
 * 编译: gcc -g -Wall -std=c11 amdahl.c -o amdahl
 */
#include <stdio.h>
#include <stdlib.h>

static int        g_init = 42;      /* 已初始化数据段 .data */
static int        g_zero;           /* 未初始化数据段 .bss  */
static const char *g_msg = "hello"; /* 指针本身在 .data     */

/* Amdahl 定律: p 为可加速比例, k 为加速倍数 (k > 0) */
static double amdahl_speedup(double p, double k)
{
    return 1.0 / ((1.0 - p) + p / k);
}

int main(void)
{
    int  stack_var = 0;         /* 栈 */
    int *heap_var  = malloc(sizeof(int));  /* 堆 */
    if (heap_var == NULL) {
        fprintf(stderr, "malloc failed\n");
        return 1;
    }
    *heap_var = 7;

    printf("=== Amdahl 定律: S(p,k) = 1/((1-p) + p/k) ===\n");
    printf("%6s %8s %8s %8s\n", "p", "k=4", "k=16", "k=inf");
    const double pset[] = {0.50, 0.90, 0.95, 0.99};
    for (size_t i = 0; i < sizeof(pset) / sizeof(pset[0]); i++) {
        double p = pset[i];
        printf("%6.2f %8.2f %8.2f %8.2f\n",
               p, amdahl_speedup(p, 4.0), amdahl_speedup(p, 16.0),
               1.0 / (1.0 - p));   /* k -> inf 时 T∞ = (1-p)T */
    }

    printf("\n=== 数据宽度 (LP64) ===\n");
    printf("sizeof(char)   = %zu\n", sizeof(char));
    printf("sizeof(int)    = %zu\n", sizeof(int));
    printf("sizeof(long)   = %zu\n", sizeof(long));
    printf("sizeof(void*)  = %zu\n", sizeof(void *));
    printf("sizeof(double) = %zu\n", sizeof(double));

    printf("\n=== 地址空间布局 (地址随 ASLR 变化) ===\n");
    printf("代码 (main)      = %p\n", (void *)(size_t)main);
    printf(".data (&g_init)  = %p\n", (void *)&g_init);
    printf(".bss  (&g_zero)  = %p\n", (void *)&g_zero);
    printf("堆   (heap_var)  = %p\n", (void *)heap_var);
    printf("栈   (&stack_var)= %p\n", (void *)&stack_var);

    printf("\ng_init=%d g_zero=%d *heap_var=%d msg=%s\n",
           g_init, g_zero, *heap_var, g_msg);

    free(heap_var);
    return 0;
}
```

【代码做什么？】 1. 定义三个全局对象：`g_init`（有初值，进 `.data`）、`g_zero`（无初值，进 `.bss`）、`g_msg`（指针变量本身进 `.data`，所指字符串 `"hello"` 进 `.rodata`）。 2. `amdahl_speedup(p,k)` 直接实现 $S = 1/((1-p)+p/k)$。 3. `main` 在栈上定义 `stack_var`，在堆上 `malloc` 出 `heap_var`，分别代表地址空间的两个动态区域。 4. 对 $p \in \{0.50, 0.90, 0.95, 0.99\}$ 打印 $k = 4, 16, \infty$ 三种加速比，把"串行部分是上限"变成可读数字。 5. 打印 LP64 下各类型宽度，再用 `&` 与 `%p` 打印五个不同区域对象的地址。

- **底层机制透视**
- `g_init` 与 `g_zero` 的区别不在语义而在**目标文件的存储方式**：`.data` 在 ELF 中占实际字节（初值 42 必须被存下来），`.bss` 段类型是 `NOBITS`，只记录"大小 8 字节"，**不占文件空间**，加载时由内核清零。这就是"把巨大数组初始化为 0"比"初始化为非 0"生成的二进制小得多的原因。
- 局部变量 `stack_var` 在 `main` 的**栈帧（stack frame）**里。x86-64 的栈**向低地址增长**，`%rsp` 指向栈顶，`call` 会把返回地址压栈。
- `malloc` 返回的地址来自**堆**，堆向高地址增长。`free` 之后内存交还分配器，但 `heap_var` 指针本身的值不变（**悬空指针**由此而来）。
- `%p` 打印的代码段/数据段地址（如 `0x401126`、`0x404050`）在 **ASLR（地址空间布局随机化）** 下基本固定，因为这是**非 PIE** 可执行文件，被链接到固定的 `0x400000` 起始地址；而堆与栈地址每次运行都不同。

【内存布局 / 数据结构图解】（地址为实测值）

```
  高地址 0x7fff...
+-----------------------------------------------------------------------------------------+
| 0x00007fffffffffff  | 内核/用户边界 (不可访问) | -              | -                   |
+-----------------------------------------------------------------------------------------+
| 0x00007ffdfc884d44  | 栈 stack (&stack_var)    | ↓ 向低地址增长 | main 局部变量       |
+-----------------------------------------------------------------------------------------+
|         ...         | (空洞)                   |                |                     |
+-----------------------------------------------------------------------------------------+
| 0x0000000000bce2a0  | 堆 heap (malloc 分配)    | ↑ 向高地址增长 | heap_var            |
+-----------------------------------------------------------------------------------------+
|         ...         | (空洞)                   |                |                     |
+-----------------------------------------------------------------------------------------+
| 0x0000000000402000  | .rodata 只读数据         | 只读           | "hello, world"      |
+-----------------------------------------------------------------------------------------+
| 0x0000000000404000  | .data / .bss 全局变量    | 可读写         | 0x404050 / 0x40406c |
+-----------------------------------------------------------------------------------------+
| 0x0000000000401000  | 代码段 .text             | 只读/可执行    | main = 0x401126     |
+-----------------------------------------------------------------------------------------+
  低地址 0x0000...
```

【实测验证】

```
$ gcc -g -Wall -std=c11 amdahl.c -o amdahl && ./amdahl
=== Amdahl 定律: S(p,k) = 1/((1-p) + p/k) ===
     p      k=4     k=16    k=inf
  0.50     1.60     1.88     2.00
  0.90     3.08     6.40    10.00
  0.95     3.48     9.14    20.00
  0.99     3.88    13.91   100.00

=== 数据宽度 (LP64) ===
sizeof(char)   = 1
sizeof(int)    = 4
sizeof(long)   = 8
sizeof(void*)  = 8
sizeof(double) = 8

=== 地址空间布局 (地址随 ASLR 变化) ===
代码 (main)      = 0x4011ab
.data (&g_init)  = 0x404050
.bss  (&g_zero)  = 0x40406c
堆   (heap_var)  = 0xbce2a0
栈   (&stack_var)= 0x7ffdfc884d44

$ size hello                      # 观察三段大小
   text    data     bss     dec     hex filename
    976     560       8    1544     608 hello

$ readelf -S hello | grep -E '\.text|\.data|\.bss'
  [14] .text    PROGBITS  0000000000401040 ...
  [24] .data    PROGBITS  0000000000404020 ...
  [25] .bss     NOBITS    0000000000404030 ...
```

注意 `.bss` 的类型是 NOBITS——这正是它不占文件空间的原因。

1.3.2 `hello` 的机器级实现（真实 x86-64 AT&T 汇编）

代码 (C):

```
#include <stdio.h>
int main(void)
{
    printf("hello, world\n");
    return 0;
}
```

用 `gcc -O0 -S -std=c11 hello.c -o hello.s` 生成的汇编（已删除 `.cfi_*` 调试伪指令）:

```
        .file   "hello.c"
        .text
        .section        .rodata
.LC0:
        .string "hello, world"
        .text
        .globl  main
        .type   main, @function
main:
        pushq   %rbp                # 保存调用者的帧指针
        movq    %rsp, %rbp          # 建立本函数的栈帧
        movl    $.LC0, %edi         # 第 1 个参数: 字符串地址 -> %edi
        call    puts                # 调用 puts (编译器把 printf 优化成 puts)
        movl    $0, %eax            # 返回值 0 -> %eax
        popq    %rbp                # 恢复 %rbp
        ret                         # 返回
        .ident  "GCC: (GNU) 12.2.0"
```

【代码做什么？】 `main` 是链接器指定的 C 入口（真正的入口 `_start` 在 C 运行时里，它调用 `__libc_start_main`，再由后者调用 `main`）。`printf("hello, world\n")` 只有一个参数且以换行结尾，GCC 把它优化成 `puts`，省掉格式串解析。

- **底层机制透视**
- **调用约定**：x86-64 System V ABI 规定整数参数依次放入 `%rdi, %rsi, %rdx, %rcx, %r8, %r9`，返回值在 `%rax`；浮点参数用 `%xmm0-%xmm7`。字符串地址是第 1 个参数，故进 `%edi`（`movl` 写 32 位会把 `%rdi` 高 32 位清零）。
- **`call` 做了什么**：把**返回地址**压栈（`%rsp -= 8`），再把 PC 设为 `puts` 入口；`puts` 执行完用 `ret` 从栈上弹出返回地址。
- **栈帧**：`pushq %rbp; movq %rsp,%rbp` 是未优化代码的标志性开头——保留帧指针便于调试器做栈回溯。`-O2` 下 GCC 改用 `subq $8, %rsp` 对齐栈，甚至省掉 `push`。

【实测验证】 用 `gdb` 与 `strace` 观察真实行为：

```
$ gdb -q -batch -ex 'break main' -ex run -ex 'info registers rip rsp rbp' \
      -ex 'disas /r main' ./hello
Breakpoint 1, main () at hello.c:3
rip            0x40112a            0x40112a <main+4>
rsp            0x7fffffffb6e0      0x7fffffffb6e0
rbp            0x7fffffffb6e0      0x7fffffffb6e0
Dump of assembler code for function main:
   0x0000000000401126 <+0>:    55      push   %rbp
   0x0000000000401127 <+1>:    48 89 e5        mov    %rsp,%rbp
   0x000000000040112a <+4>:    bf 04 20 40 00  mov    $0x402004,%edi
   0x000000000040112f <+9>:    e8 fc fe ff ff  call   0x401030 <puts@plt>
   0x0000000000401134 <+14>:   b8 00 00 00 00  mov    $0x0,%eax
   0x0000000000401139 <+19>:   5d      pop    %rbp
   0x000000000040113a <+20>:   c3      ret

$ strace -e trace=write,exit_group ./hello
write(1, "hello, world\n", 13)          = 13
exit_group(0)                           = ?
```

`write(1, ...)` 中的 `1` 就是**标准输出（stdout）文件描述符**——这正是"文件是对 I/O 设备的抽象"在系统调用层面的落点。`mov $0x402004,%edi` 里的 `0x402004` 是 `.rodata` 中 `"hello, world"` 的地址，与上面的地址布局图完全对应。

1.3.3 五大现实的算术验证

代码 (C)：

```
#include <stdio.h>
int main(void)
{
    int x = 50000;
    int y = x * x;              /* ⚠️ UB: 有符号溢出, 实际按补码回绕 */
    printf("50000*50000   = %d\n", y);
    unsigned u = 50000u, v = u * u;
    printf("50000u*50000u = %u\n", v);
    printf("(1e20 + -1e20) + 3.14 = %.17g\n", (1e20 + -1e20) + 3.14);
    printf("1e20 + (-1e20 + 3.14) = %.17g\n", 1e20 + (-1e20 + 3.14));
    return 0;
}
```

【实测验证】（GCC 12.2.0，x86-64）

```
50000*50000   = -1794967296
50000u*50000u = 2500000000
(1e20 + -1e20) + 3.14 = 3.1400000000000001
1e20 + (-1e20 + 3.14) = 0
```

【底层机制透视】 这就是"ints are not integers, floats are not reals"的实证。 – $50000 \times 50000 = 2.5 \times 10^9$ 超出 `int` 范围 $[-2^{31}, 2^{31}-1] = [-2147483648, 2147483647]$，结果按 **2 的补码（two's complement）回绕**得 -1794967296。有符号溢出在 C 中是**未定义行为（UB, Undefined Behavior）**，实际结果依编译器与机器而异；无符号则严格按模 2^{32} 定义，故正确打印。 – 浮点加法**不满足结合律**：`1e20 + 3.14` 的精确结果需约 67 位有效数字，而 `double` 只有 52 位尾数，`3.14` 被完全**舍入掉**，故 `1e20 + (-1e20 + 3.14) = 0`。详见 Lecture 2。

1.3.4 内存引用 bug 的"远距离作用"（⚠ 仅供演示，请勿模仿）

代码 (C)：以下代码故意制造**数组越界写**。

```c
/* ⚠ 仅供演示，请勿模仿: s.a[i] 越界写会破坏同结构的 s.d */
#include <stdio.h>
typedef struct { int a[2]; double d; } struct_t;   /* sizeof=16, a 在 +0, d 在 +8 */

double fun(int i) {
    volatile struct_t s;
    s.d = 3.14;
    s.a[i] = 1073741824;   /* ⚠ UB: i >= 2 时越界 */
    return s.d;
}
int main(void) {
    int i;
    for (i = 0; i <= 6; i++) {
        printf("fun(%d) --> ", i);
        fflush(stdout);
        printf("%.14g\n", fun(i));
    }
    return 0;
}
```

【实测验证】（本机 GCC 12.2.0；**结果依赖系统与编译器**）

```
fun(0) --> 3.14
fun(1) --> 3.14
fun(2) --> 3.1399998664856
fun(3) --> 2.0000006103516
fun(4) --> Segmentation fault   (shell 报告 exit=139)
```

【底层机制透视】 下标 i 超出数组长度 2，但编译器不做边界检查（C/C++ **不提供内存保护**）。`struct_t` 布局是 `a[0]` 在偏移 0、`a[1]` 在偏移 4、`d` 在偏移 8。写入常量 `1073741824 = 0x40000000` 会改写 `s.d` 的字节： – $i = 2$ 写偏移 8，覆盖 `d` 的**低 4 字节**：位模式 `0x40091eb851eb851f` → `0x40091eb840000000`，值恰为 `3.1399998664856`。 – $i = 3$ 写偏移 12，覆盖 `d` 的**高 4 字节**：位模式 → `0x4000000051eb851f`，值为 `2.0000006103516`。 – $i \geq 4$ 越过整个 `struct_t`，开始破坏栈上其它数据（返回地址等），于是段错误。

这正是讲义所说的 **action at a distance（远距离作用）**：被破坏的对象（`s.d`）与被访问的对象（`s.a`）逻辑上毫无关系，而 bug 的**影响可能在很久以后、很远的地方**才第一次被观察到。

1.4 实验关联

本讲无直接关联 lab，但它是 **L0 C Programming Lab** 与全部 8 个 lab 的**环境认知前提**：

- **L0（C Programming，可立即开始）**：讲义说 L0"应该全是复习"——C 控制流与语法、显式内存管理、基于指针的数据结构、**在收到非法参数（含 `NULL` 指针）时仍正确运行的健壮代码**、Makefile 规则。1.3.1 的 `malloc` / `free` 与 1.3.4 的越界写，正是"健壮性"要求的痛点。讲义建议：**若 L0 花掉你超过 10 小时，请认真考虑是否该选这门课**。
- **环境**：所有 lab 在 Intel Computer Systems Cluster（"shark machines"，`ssh shark.ics.cs.cmu.edu`）上完成。讲义明确说**不要**用通用 Andrew 集群（没有正确的编译器），本地机器通常也没有，且 `make submit` 一定不工作。Lab 全部通过 **Autolab**（https://autolab.andrew.cmu.edu）提交，**Autolab 上的分数就是你的 lab 成绩**。
- **依赖链**：L1 Data ← Lecture 2；L2 Bomb ← Lecture 3–5；L3 Attack ← Lecture 5–6；L4 Cache ← Lecture 9–10；L5 Malloc ← Lecture 11–14；L6 Shell ← Lecture 16–18；L7 Proxy ← Lecture 18–21；L8 SFS ← Lecture 21–24。
- **版本控制**：**从 L4 Cache Lab 开始，lab 通过 GitHub Classroom 分发**，要**早提交、勤提交**；被指控抄袭时课程组会查阅 git 服务器，**缺失 git 历史对你不利**。应使用 `git commit <file-list>` 而非 `git add .`。把作业发到公开 git 仓库**属于 AIV（学术诚信违规）**。
- **坑**：必须**独立完成**；全学期 5 个 grace day，每个 lab 最多自动使用 0/1/2 个，用完后每天扣 15%，且**晚交不得超过 3 天**。部分 lab（L4、L5a、L5b、L6、L7）有代码风格分（与 TA 1:1 评审），部分 lab（L1、L4、L5b、L6、L7）约 10% 分数来自现场 **Parsons Puzzle**（把乱序代码行重排成

可运行片段）。

1.5 常见错误与调试技巧

- **把 `gcc hello.c -o hello` 当成一步**：四个阶段的问题混在一起，报错时不知是哪一步。**调试**：拆开执行 `gcc -E hello.c -o hello.i`、`gcc -S hello.i`、`gcc -c hello.s`、`gcc hello.o -o hello`，用 `file` 确认产物类型（文本还是 ELF 二进制）。
- **以为栈向高地址增长**：推演栈帧偏移时全部算反。**调试**：`gdb` 里 `p &a`、`p &b` 比较两个局部变量地址；`info registers rsp rbp` 看 `%rsp` 在 `call` 前后是否**减小** 8 字节（`x/2gx $rsp` 看返回地址）。
- **把有符号溢出当成"会得到一个很大的数"**：现象是 `50000*50000` 打印出负数。**调试**：这是 UB。`gcc -g -fsanitize=undefined -Wall -std=c11 x.c` 会打印 `runtime error: signed integer overflow`；或用 `-ftrapv` 触发陷阱。
- **浮点比较用 `==` / 假定加法结合律**：`1e20 + (-1e20 + 3.14)` 得到 0 而非 3.14。**调试**：打印要足够精度 `printf("%.17g\n", x)`；比较用 `fabs(a-b) < 1e-9`；`-Wfloat-equal` 会对浮点 `==` 报警。
- **不初始化就使用**：`.bss` 里的全局变量恰为 0（规范保证），但栈上的局部变量是垃圾值，行为随机。**调试**：`gcc -Wall -Wextra`（`-Wuninitialized` 在 `-O2` 下更有效）；`valgrind --track-origins=yes ./a.out` 会报 "Conditional jump depends on uninitialised value" 并指出来源。
- **越界写与内存泄漏**：程序在完全无关的地方崩溃，或长时间运行后内存耗尽。**调试**：`valgrind --leak-check=full --show-leak-kinds=all ./a.out`；对**栈越界**（如 1.3.4 的例子）首选 `gcc -g -fsanitize=address -fsanitize=undefined`（ASan 的栈越界检测远好于 Valgrind，后者最擅长堆越界与泄漏）；`gdb -tui` 配合 `x/8xb ptr`、`p *ptr` 亲手核对被破坏的字节。
- **`printf` 格式串与参数类型不匹配**：`printf("%d\n", ptr)` 打印出截断的地址。**调试**：`gcc -Wall -Wformat=2` 直接指出；打印指针统一写 `printf("%p\n", (void *)ptr)`。

1.6 关键要点

- **抽象很好，但要知道它建立在什么之上、边界在哪里**：ISA、虚拟内存、进程、虚拟机这些分层抽象让复杂系统可被驾驭，但当程序出 bug、性能不达标时，只有下探到实现细节才能解决。
- **一个 C 程序要变成可执行程序，必须经过预处理 → 编译 → 汇编 → 链接四步**；前两步产物是文本，后两步是二进制，链接阶段才完成符号解析与重定位。
- **存储层次之所以有效，全靠程序的局部性（时间与空间局部性）**；每一层都是下一层的缓存，由此得到"价格便宜、速度飞快"的错觉。
- **操作系统用三个抽象包装硬件：文件（对 I/O 设备）、虚拟内存（对主存+磁盘）、进程（对处理器/主存/I/O）**；应用程序从不直接操作硬件。
- **Amdahl 定律是不可绕过的天花板**：$S = 1/((1-p)+p/k)$，$S_{\max} = 1/(1-p)$。加速比由**串行部分 $1-p$** 决定，而不是由你堆了多少并行资源决定。

1.7 思考题（带答案）

Q1（计算题） 某程序在单处理器上运行需 $T = 100$ 秒，其中 80 秒可完美并行化（$p = 0.8$），剩余 20 秒严格串行。

（a）使用 $k = 16$ 台处理器，T_{16} 与 S_{16} 各是多少？ （b）使用无限多处理器，$S_{\max}$ 是多少？ （c）若通过重构算法把可并行部分从 $p = 0.8$ 提高到 $p = 0.95$（$k = 16$ 不变），加速比变成多少？从（a）到（c）的改善说明优化应优先投向哪里？

答案： （a）$T_{16} = \dfrac{pT}{k} + (1-p)T = \dfrac{0.8 \times 100}{16} + 0.2 \times 100 = 5 + 20 = 25$ 秒；$S_{16} = \dfrac{100}{25} = 4$。 （b）$S_{\max} = \dfrac{1}{1-p} = 5$（$T_\infty = 20$ 秒）。用 16 台已拿到 4 倍，而**无论再加多少处理器，最多也只有 5 倍**。 （c）$T'_{16} = \dfrac{0.95 \times 100}{16} + 0.05 \times 100 = 5.9375 + 5 = 10.9375$ 秒；$S'_{16} \approx 9.14$。处理器一个没加，只是把串行部分从 20 秒压到 5 秒，加速比就从 4 倍升到 9.14 倍。**结论：优先削减串行部分（同步、I/O、分配、依赖链）的收益远大于堆更多并行资源。** 另外注意 T'_{16} 中并行部分（5.9375 秒）已与串行部分相当——继续增大 k 的边际收益会迅速衰减。

Q2（推演题） 已知 L1 d-cache 命中约 4 周期、主存约 100 周期、本地磁盘约 10,000,000 周期。若把"一次 L1 命中"放大为 1 秒，请把主存与磁盘访问换算成人类可感知的时间，并解释为什么说"内存性能不是均匀的"。

答案：放大倍数 $= 10^9/4 = 2.5 \times 10^8$。主存 $= 100 \times 2.5 \times 10^8$ 秒 ≈ 792 年；磁盘 $= 10^7 \times 2.5 \times 10^8$ 秒 ≈ 7900 万年。也就是说，如果每次 L1 命中让你等 1 秒，一次主存访问要等近 800 年，一次磁盘访问要等几千万年。这正是"内存性能不均匀（memory performance is not uniform）"的量化含义：**同一条 `movq` 指令可能花 4 个周期，也可能花 10,000,000 个周期**，取决于数据在哪一层。据此也就理解了 `copyij` 4.3 ms 与 `copyji` 81.8 ms 的差距。

Q3（错误直觉题） "这台机器有 16 个硬件线程（8 核 × 2 路超线程），所以程序只要写成多线程就应该快 16 倍。"错在哪里？

答案：错在三层。 （1）**Amdahl 定律**：程序中必然存在不可并行化的串行部分 $1-p$，它构成上限 $1/(1-p)$。讲义指出 shark 机器"理论上限 16 倍，但在我们的基准测试中**从未达到**"。 （2）**超线程不是第二个核**：它只**复制指令控制单元**（PC、寄存器文件、操作队列），而**共享**功能单元（整数/浮点运算单元、load/store 单元、数据缓存）。两线程争用同一批功能单元或缓存时，收益远低于 2 倍；只有当一个线程在等待（如缓存不命中）时，另一个才能填满空闲执行槽。 （3）**同步与内存一致性开销**：访问共享状态必须同步，而讲义强调"同步操作非常昂贵"；缓存一致性协议还带来额外流量，并可能出现**伪共享（false sharing）**等硬件假象。正确期待是：并行化通常只带来"适度（modest）"的加速。

Q4（错误直觉题） "既然编译器比人聪明得多，而且我永远不会手写汇编，那么学机器级编程完全是浪费时间。"请指出这个推理的漏洞。

答案：漏洞在于把"**编写**汇编"与"**理解**机器级执行模型"混为一谈。讲义承认"你大概永远不会用汇编写程序——编译器比你更好、更有耐心"，但随即给出四条必须懂汇编的理由： （1）**有 bug 时高级语言模型会崩溃**——1.3.4 节那个越界写在 C 的语义层面毫无信号，只有理解 `struct_t` 的字节布局才能解释 `fun(2)` 为何打印 `3.1399998664856`； （2）**调优性能**必须知道编译器做了/没做哪些优化（如把 `printf` 换成 `puts`）； （3）**实现系统软件**：编译器的目标是机器码，操作系统必须管理进程状态； （4）**对抗恶意代码**：代码注入攻击（L3 Attack Lab）完全建立在栈帧布局与返回地址的知识上。 一句话：**汇编不是用来写的，是用来读的**；不会读，你在 bug 与性能面前就只能猜。

Lecture 2: 信息的表示与处理——位、字节与整数 (Bits, Bytes, and Integers)

> **讲义对应**：CMU 15–213 Lecture 2 — Bits, Bytes, & Integers（素材：`F25-02-bits-bytes-ints.txt`） **教材对应**：CS:APP3e 第 2 章 2.1—2.4（含 2.3 整数运算） **关联 Lab**：**L1 Data Lab**

2.1 概述

第 1 讲把程序的一生压缩成"源文件 → 可执行文件 → 进程"的黑箱；本讲打开这个黑箱最底下的一层：**内存里的一切都只是比特**。核心问题是：同一串 w 个比特，凭什么既能被解释成无符号整数、又能被解释成补码整数、还能被解释成指令、字符或指针？答案是——**比特本身没有类型，类型来自施加于其上的解释规则（编码）**。本讲先建立布尔代数与位运算这套"比特的手工工具"，再给出无符号编码 $B2U$ 与二进制补码编码 $B2T$ 两个映射，接着推导类型转换、扩展/截断、加减乘除与移位在**位级**上究竟发生了什么，最后落到字节序（Byte Ordering）这种"纯粹约定"造成跨平台差异的经典案例。它承接第 1 讲的"信息就是位 + 上下文"，直接为第 3 讲的机器级算术逻辑运算铺路，并且是 **L1 Data Lab** 的全部理论前提。

2.2 核心概念与底层机制图解

2.2.1 位与字节（Bits and Bytes）

- **定义与目的**：一个**位（bit）**只有 0/1 两种状态；8 个位组成一个**字节（byte）**。用二值而不是连续电压，是因为"高到可以算 On、低到可以算 Off"的判据远比"精确测出 0.37V"容易通信、复制与保存。选择 2^k 进制的编码（八进制、十六进制）唯一目的是**压缩书写长度且能直接切出位**：1 个十六进制数字恰好对应 4 个位。
- **直观解释**：十六进制就像给二进制做的"速记"：`0011 1011 0110 1101` 写成 `3B6D`，每个十六进制数字是一组 4 位的"缩写标签"，可以随时无损展开。这也是为什么内存窗口、反汇编列表、`gdb` 默认都用十六进制。
- **与机器码/硬件的对应**：C 语言里 `0x` 前缀标记十六进制常量，如 `0xFA1D37B` 与 `0xfa1d37b` 等价（大小写不敏感）。编译器在词法分析阶段就把它们折成同一个整数常量，机器码里根本不存在"进制"这个概念。

2.2.2 字节序（Byte Ordering）

- **定义与目的**：多字节数据（`int`、`long`）在内存中占连续多个地址，**哪一个字节放在最低地址**属于纯约定。**小端（Little Endian）**：最低有效字节（least significant byte, LSB）放在最低地址；**大端（Big Endian）**：最高有效字节（MSB）放在最低地址。x86 与 ARM（Android/iOS/Linux）是小端；Sun SPARC、PowerPC Mac、以及**网络字节序**是大端。
- **直观解释**：把数字 `0x01234567` 想象成一句从左到右书写的句子。大端是"正常书写顺序"，第一个字节就是最重要的数字；小端是"把句子倒着塞进抽屉"，末位数字先出场——但好处是，从同一地址读 1 字节，小端总拿到低 8 位，这条性质让指针类型双关（type punning）变得自然。
- **底层机制图解**（变量 `x` 为 4 字节，`&x == 0x100`）：

```
地址        小端 (Little Endian)              大端 (Big Endian)
          +---------+                       +---------+
0x103     |   01    |  <- MSB (最高有效字节)  |   67    |  <- LSB (最低有效字节)
          +---------+                       +---------+
0x102     |   23    |                       |   45    |
          +---------+                       +---------+
0x101     |   45    |                       |   23    |
          +---------+                       +---------+
0x100     |   67    |  <- LSB (最低有效字节)  |   01    |  <- MSB (最高有效字节)
          +---------+                       +---------+
              &x = 0x100 (低地址)               &x = 0x100 (低地址)
```

- **与机器码/硬件的对应**：字节序只在"数据跨越机器边界"时成为问题——写文件、发网络包、跨架构共享内存。**位序不会反转**（参考点始终是低位），`char` 与字符串（字符数组）**不受影响**，因为每个元素只有 1 字节。反汇编器读到的立即数就是字节反转过的：机器码 `81 c3 ab 12 00 00` 对应的值要还原成 `0x000012ab` 再反序得 `ab 12 00 00`，即 `add $0x12ab,%ebx`。

2.2.3 布尔代数与 C 的位运算（Boolean Algebra and Bit-Level Operations）

- **定义与目的**：George Boole 用代数刻画逻辑：`&`（与，两真才真）、`|`（或，任一真则真）、`~`（非）、`^`（异或，恰有一个真才真）。把运算**按位并行**地作用到整个位向量（bit vector）上，就得到了机器可在一个时钟周期内完成的"位并行计算"。

- **直观解释**：**掩码（mask）**就像镂空模板：`x & 0xFF` 只在低 8 位"有孔"，其余被挡住。更妙的是位向量可以表示**集合**：第 j 位为 1 表示元素 j 属于该集合，于是 `&` 是交集、`|` 是并集、`^` 是对称差、`~` 是补集——一次位运算等于一次性完成 32/64 个元素的集合运算。
- **底层机制图解**：

```
  01101001  (A)          人可读角度:  A = { 0, 3, 5, 6 }
& 01010101  (B)                       B = { 0, 2, 4, 6 }
----------
  01000001  = 0x41      交集 (Intersection)  A ∩ B = { 0, 6 }

  01101001 | 01010101 = 01111101 = 0x7D   并集 A ∪ B = {0,2,3,4,5,6}
  01101001 ^ 01010101 = 00111100 = 0x3C   对称差 A △ B = {2,3,4,5}
  ~ 01010101          = 10101010 = 0xAA   补集 ~B = {1,3,5,7}
```

- **与机器码/硬件的对应**：`& | ~ ^` 可用于任何整型（`long, int, short, char, unsigned`），实参一律视作位向量逐位计算。`char` 场景的经典结论：`~0x41 → 0xBE`、`~0x00 → 0xFF`、`0x69 & 0x55 → 0x41`、`0x69 | 0x55 → 0x7D`。硬件上由 ALU 的一组并行门电路完成，没有分支、没有进位链，因此常数时间。

2.2.4 逻辑运算与位运算的区别（Logic vs. Bit-Level）

- **定义与目的**：`&& || !` 是**逻辑运算**：0 视为 False、任何非零视为 True，结果**永远是 0 或 1**，并且有**短路求值（early termination）**。`& | ~ ^` 是位运算，结果保留每一位。
- **直观解释**：这是 C 语言最经典的坑：`if (x & y)` 与 `if (x && y)` 在 `x=1, y=2` 时前者为 0（假）、后者为 1（真）。更要命的是 `p && *p` ——它先判断指针非空再解引用，**避免了空指针访问**；写成 `p & *p` 则一定会解引用 `NULL`。
- **与机器码/硬件的对应**：`!x` 编译成 `testl %edi,%edi; sete %al`（测试并置标志），结果被规范化到 0/1；`x & y` 编译成 `andl`，结果保留原始宽度。短路求值会产生真实的分支（或 `cmov`），因此逻辑运算存在**控制依赖**，位运算没有。

2.2.5 移位运算（Shift Operations）

- **定义与目的**：`x << y` 左移，左边溢出的位丢弃、右边补 0。右移 `x >> y` 丢弃低位，补高位有两种：**逻辑右移（logical shift）**补 0，**算术右移（arithmetic shift）**复制最高位（符号位）。**移位量 $y < 0$ 或 $y \geq$ 字长是未定义行为（UB）**。
- **直观解释**：左移就是"数字整体往高位搬家"，每搬一位相当于乘 2；算术右移是"带着符号整体往低位搬家"，等价于向下取整的除以 2 的幂。
- **底层机制图解**：

```
x = 01100010                  x = 10100010
x << 3        = 00010000      x << 3        = 00010000   (左移一致，溢出位丢弃)
Log.  x >> 2  = 00011000      Log.  x >> 2  = 00101000   (补 0: 负数被"看成大正数")
Arith.x >> 2  = 00011000      Arith.x >> 2  = 11101000   (补符号位: 保持负号)
```

- **与机器码/硬件的对应**：C 标准规定：**有符号负数的右移是"实现定义（implementation-defined）"，不是严格的 UB**——GCC/Clang 在 x86-64 上选择算术右移，Java 则把两种都给你：`>>` 是算术右移，`>>>` 是无符号（逻辑）右移。x86-64 的 `shrl` 是逻辑右移、`sarl` 是算术右移、`shll` / `sall` 是左移（两者等价）；移位量放在 `%cl` 寄存器中。

2.2.6 无符号编码与二进制补码编码（Unsigned & Two's Complement）

- **定义与目的**：w 位位向量 $\vec{x}$ 的两种解释：

$$B2U_w(\vec{x}) = \sum_{i=0}^{w-1} x_i 2^i \qquad B2T_w(\vec{x}) = -x_{w-1}2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

即：补码只是把**最高位的权重从 $+2^{w-1}$ 改成 -2^{w-1}**，其余完全一致。最高位因此被叫**符号位**：0 表示非负，1 表示负。 – **直观解释**：为什么不用"符号 + 数值"（sign-magnitude）？因为那样会出现 `-0`，且 `1 + (-1)` 无法用普通加法得到 0。补码是"魔法"：取 `001`，希望加某个数得 `000`（丢弃进位），试验发现 `111` 正好满足 `001 + 111 = 1 000 = 000`，于是 `111` 就是 `-1`。求 $-x$ 的捷径由此而来：$-x = \sim x + 1$（按位取反再加 1）。道理是 $x + \sim x = 111\ldots1 = -1$，两边加 1 并回绕即得。 – **底层机制图解**（$w = 3$ 时两条数轴叠放在同一串位模式上）：

```
   位模式:    000     001     010     011     100     101     110     111
              |       |       |       |       |       |       |       |
   B2U 值:     0       1       2       3       4       5       6       7
              |       |       |       |       |       |       |       |
   B2T 值:     0       1       2       3      -4      -3      -2      -1
              +-------+-------+-------+       +-------+-------+-------+
              非负区 (符号位 0)                 负区 (符号位 1)
                                                ^
                                  TMin = -4 比 TMax = 3 多一个"没有正伴侣"的位置
```

- **与机器码/硬件的对应**：$w=16$ 时 `15213` 是 `3B 6D`（`00111011 01101101`），`-15213` 是 `C4 93`（`11000100 10010011`）。取值范围：$UMin=0$、$UMax=2^w-1$、$TMin=-2^{w-1}$、$TMax=2^{w-1}-1$。$w=32$：`unsigned` 为 $[0, 4294967295]$，`int` 为 $[-2147483648, 2147483647]$；$w=64$：`unsigned long` 为 $[0, 18446744073709551615]$，`long` 为 $[-9223372036854775808, 9223372036854775807]$。C 标准并未强制要求补码，但"几乎所有机器都是"，本课程一律按补码讨论。注意 $TMin$ 是不对称的：$|TMin| = TMax + 1$。

2.2.7 有符号↔无符号转换（T2U / U2T）

- **定义与目的**：转换是**位模式不变、解释改变**。$T2U_w(x) = x + 2^w$（当 $x < 0$），$U2T_w(u) = u - 2^w$（当 $u \geq 2^{w-1}$）。C 中显式强制转换（cast）与赋值、函数传参引发的**隐式转换**规则完全相同。
- **直观解释**：把补码数轴"从 $TMin$ 处剪断、上半段整体挪到 $UMax$ 之后"，就得到无符号数轴——这就是**顺序反转（ordering inversion）**：负数变成极大的正数。
- **底层机制图解**：

```
补码范围                                无符号范围
  TMax = 2^(w-1)-1  <--- ——对应 --->  TMax
    ...                                  ...
   -1  = 111...1     --- 加 2^w --->  UMax   = 2^w - 1
   -2  = 111...0     --- 加 2^w --->  UMax-1
    ...                                  ...
  TMin = 100...0     --- 加 2^w --->  2^(w-1)
```

- **与机器码/硬件的对应**：转换**不产生任何指令**（寄存器里的位原封不动），只影响后续指令的选择——有符号比较用 `cmp` + 有/无符号条件跳转（`jl` vs `jb`），除法用 `idiv` vs `div`。因此它是最"免费"、也最容易咬人的机制：`-1` 转 `unsigned` 得 `UINT_MAX`（4294967295）；**混合表达式中 `int` 被隐式转成 `unsigned`**，包括比较运算 `< > == <= >=`。于是 `-1 > 0U` 为**真**，因为 `-1` 先变成 4294967295。

2.2.8 扩展与截断（Expanding & Truncating）

- **定义与目的**：扩展（如 `short int → int`）：无符号补 0，**有符号做符号扩展（sign extension）**——复制 k 份最高位。截断（如 `unsigned → unsigned short`）：直接丢弃高 k 位，结果被重新解释：无符号等价于取模 2^w，有符号类似取模。
- **直观解释**：符号扩展就像把一个人的"姓氏"抄到最后——负数的高位填 1，数值不变；截断像把长号码掐掉前几位，位数不够就会"变号"。
- **与机器码/硬件的对应**：`movswl`（带符号扩展短→长）、`movzbl`（零扩展字节→长）就是这两条规则对应的指令。截断则由写窄寄存器隐式完成（如 `movb`）。

2.2.9 整数运算：加法、乘法与除法的位级真相

- **无符号加法**：硬件就是标准加法器、**丢弃进位输出**，因此实现 $s = (u + v) \bmod 2^w$，溢出时回绕（wrap around），且至多回绕一次。
- **补码加法**：与无符号加法**位级行为完全相同**，只是解释不同：`int s = (int)((unsigned)u + (unsigned)v)` 与 `s = u + v` 结果一致。回绕方向不同：真和 $\geq 2^{w-1}$ 变负（正溢出），$< -2^{w-1}$ 变正（负溢出）。**有符号溢出是 UB**，编译器有权假设它不发生。
- **乘法**：w 位乘积取**低 w 位**，精确值最多 $2w$ 位。向左移 k 位等于乘 2^k：`u << 3 == u * 8`，`(u << 5) - (u << 3) == u * 24`——**编译器自动生成移位加/减代码**，因为移位加比乘法快。
- **除以 2 的幂**：无符号用逻辑右移，`u >> k` 就是 $\lfloor u/2^k \rfloor$；有符号用算术右移，但它**向下取整（round down）**而非向零取整，会给负商引入偏置：`-15213 >> 1 = -7607`，而 `-15213 / 2 = -7606`。要得到 C 的向零取整语义必须**加偏置**：$(x + 2^k - 1) \gg k$，即 `(x + (1<<k) - 1) >> k`；若 x 已能被整除，偏置不产生效果（低 k 位全 0，加 $2^k - 1$ 不会进位）。
- **与机器码/硬件的对应**：`addl`（丢弃进位）、`imull`（取低 32 位）、`sarl` / `shrl`（有/无符号右移）、`leal (%rdi,%rdi,2), %eax`（一条 LEA 完成乘以 3）——见 2.3 节的真实汇编。

2.3 代码示例与底层机制分析

2.3.1 `show_bytes` 逐字节打印与字节序判定

代码 (C)：

```c
/* /tmp/c02_byteorder.c  编译: gcc -g -Wall -std=c11 c02_byteorder.c -o c02_byteorder */
#include <stdio.h>
#include <string.h>

typedef unsigned char *byte_pointer;

/* 从 start 开始，按内存地址递增顺序逐字节打印 len 个字节 */
void show_bytes(byte_pointer start, size_t len)
{
    size_t i;
    for (i = 0; i < len; i++)
        printf(" %.2x", start[i]);   /* 必须以无符号十六进制打印 */
    printf("\n");
}

void show_int(int x)      { printf("int      %11d :", x); show_bytes((byte_pointer) &x, sizeof(int)); }
void show_long(long x)    { printf("long     %11ld :", x); show_bytes((byte_pointer) &x, sizeof(long)); }
void show_pointer(void *x){ printf("pointer  %11p :", x); show_bytes((byte_pointer) &x, sizeof(void *)); }
void show_short(short x)  { printf("short    %11d :", x); show_bytes((byte_pointer) &x, sizeof(short)); }

int main(void)
{
    int  ival = 0x01234567;           /* 讲义用的是 15213 = 0x3B6D */
    long lval = 0x0123456789abcdefL;
    short sval = 15213;
    int  x = 15213;

    printf("sizeof(long) = %zu, sizeof(void*) = %zu\n\n", sizeof(long), sizeof(void *));
    printf("== 字节序: int ival = 0x%08x ==\n", ival);
    show_int(ival);
    printf("== 讲义示例: short 15213 ==\n");
    show_short(sval);
    show_int(x);                       /* 符号扩展成 4 字节 */
    show_int(-x);                      /* 负数: 补码 */
    printf("== long ival = 0x%016lx ==\n", (unsigned long) lval);
    show_long(lval);
    printf("== char 数组: 完全不受字节序影响 ==\n");
    char *s = "15213";
    printf("string   %11s :", s);
    show_bytes((byte_pointer) s, strlen(s));
    show_pointer((void *) s);

    /* 显式判断运行时字节序 */
    unsigned int probe = 1;
    unsigned char first = *(unsigned char *) &probe;
    printf("\n*(char*)&1 == %u  ->  %s endian\n", (unsigned) first, first ? "little" : "big");
    return 0;
}
```

【代码做什么？】 1. `byte_pointer` 是 `unsigned char *`。C 标准保证 `char` 恰好 1 字节，且**可以用字符指针逐字节检查任意对象的表示**，所以它是内存的"显微镜"。 2. `show_bytes` 从低地址向高地址遍历，`%.2x` 保证每字节两位十六进制、高位补 0——若用 `%x` 打印 `0x01` 会变成 `1`，输出就错位了。 3. `show_int(ival)` 打印 `ival` 的 4 个字节，顺序直接暴露本机字节序。 4. 最后用 `*(unsigned char*)&probe` 在运行时判定：小端机读到的第一个字节是 1。

【底层机制透视】 取地址 `&x` 得到的是对象**最低地址**，函数并不知道字节序，它只是机械地递增地址。所以 `show_bytes` 的输出顺序就是"机器把数字写进内存的顺序"。`sizeof(long) == 8` 说明这是 LP64（x86-64）模型，与教材一致。

【实测验证】（本机 gcc 12.2.0, x86-64 Linux）：

```
sizeof(long) = 8, sizeof(void*) = 8
== 字节序: int ival = 0x01234567 ==
int         19088743 : 67 45 23 01          <- 小端: LSB 在低地址
== 讲义示例: short 15213 ==
short          15213 : 6d 3b
int            15213 : 6d 3b 00 00          <- 正数符号扩展 = 补 0
int           -15213 : 93 c4 ff ff          <- 负数符号扩展 = 补 1
== long ival = 0x0123456789abcdef ==
long     81985529216486895 : ef cd ab 89 67 45 23 01
== char 数组: 完全不受字节序影响 ==
string         15213 : 31 35 32 31 33      <- 字符按书写顺序排列
pointer     0x4021xx : xx 21 40 00 00 00 00 00  <- 低位字节随 ASLR/链接布局而变
*(char*)&1 == 1  ->  little endian
```

`3B 6D` 与 `C4 93` 与讲义表格完全吻合；`-15213` 的 4 字节 `93 c4 ff ff` 正是 `C4 93` 符号扩展的结果。

2.3.2 有符号/无符号转换与数值范围陷阱

代码 (C)：

```c
/* /tmp/c02_cast_traps.c  编译: gcc -g -Wall -std=c11 c02_cast_traps.c -o c02_cast_traps */
#include <stdio.h>
#include <limits.h>
#include <stdlib.h>

static unsigned count_down_iterations(unsigned n, unsigned cap)
{
    unsigned cnt = 0;
    for (unsigned i = n - 1; i >= 0; i--) {   /* i >= 0 恒为真!  */
        cnt++;
        if (cnt == cap) break;                /* 仅为了能在演示中停下来 */
    }
    return cnt;
}

int main(void)
{
    /* ---------- 1. 位模式不变、解释改变 ---------- */
    int      tx = -1;
    unsigned ux = (unsigned) tx;              /* T2U: 减 2^32 */
    printf("(unsigned)(-1)              = %u   (UINT_MAX = %u)\n", ux, UINT_MAX);
    printf("(unsigned)(-1) > 0          = %d\n", ux > 0);
    printf("(int) 2147483648U           = %d   (INT_MIN)\n", (int) 2147483648U);
    printf("2147483647U > -2147483647-1 = %d  (混合运算: 右边被转成 unsigned)\n",
           2147483647U > -2147483647 - 1 ? 1 : 0);
    printf("2147483647U < (int)2147483648U = %d  (显式转回 int 才按有符号比较)\n",
           2147483647U < (int) 2147483648U ? 1 : 0);

    /* ---------- 2. 混合表达式的隐式转换 ---------- */
    {
        int      a = -1;                      /* 0xFFFFFFFF */
        unsigned b = 0;                       /* 0x00000000 */
        /* 下面这行里 a 被隐式转换为 unsigned，-1 变成 4294967295，于是条件为真 */
        if (a > b) printf("混用: if (-1 > 0U) 判定为【真】(因为 -1 被转成 %u) \n",
                          (unsigned) a);
        else       printf("混用: if (-1 > 0U) 判定为假\n");
    }

    /* ---------- 3. 数值范围陷阱（UB！） ---------- */
    puts("\n-- 以下两行涉及未定义行为 (UB)，结果依编译器/优化级别而异 --");
    volatile int tmin = INT_MIN;
    int nabs = abs(tmin);                     /* UB: abs(INT_MIN) 不可表示 */
    int nneg = -tmin;                         /* UB: -INT_MIN 溢出 */
    printf("abs(INT_MIN) = %d   (仍是负数! 期望 2147483648 无法表示)\n", nabs);
    printf("-INT_MIN     = %d   (回绕成 INT_MIN 本身)\n", nneg);
    printf("INT_MIN == -INT_MIN ? %d\n", tmin == -tmin ? 1 : 0);

    /* ---------- 4. 无符号倒计数死循环 ---------- */
    unsigned iters = count_down_iterations(3, 6);
    printf("\n无符号倒计数 n=3，上限 6 步内观察到 %u 次迭代: ", iters);
    printf("i 取值序列 2,1,0,4294967295,4294967294,... 永不小于 0\n");

    /* ---------- 5. 正确写法：用 int 或改成 i > 0 形式 ---------- */
    int sum = 0;
    for (int i = 2; i >= 0; i--) sum += i;
    printf("改用 int 倒计数: sum = %d\n", sum);

    /* ---------- 6. 无符号减法的回绕 ---------- */
    unsigned u1 = 0, u2 = 1;
    printf("0U - 1U = %u  (模 2^32 回绕)\n", u1 - u2);
    return 0;
}
```

【代码做什么？】 1. `(unsigned)(-1)` 把位模式 `0xFFFFFFFF` 按无符号解释，得 `UINT_MAX`，因此 `(unsigned)(-1) > 0` 为 `1`。 2. `(int) 2147483648U` 反向操作，得 `INT_MIN`。 3. `2147483647U > -2147483647-1` 中右侧 `int` 被隐式转成 `unsigned`（变成 2147483648U），于是 `2147483647U < 2147483648U`，比较结果为 `0`。这正是讲义表中 "unsigned → `>`" 的那一行。 4. `abs(INT_MIN)` 与 `-INT_MIN` 都是 UB：`INT_MIN =`

`-2147483648`，其相反数 +2147483648 在 `int` 中不可表示。 5. 无符号倒计数 `for (unsigned i = n-1; i >= 0; i--)`：`i >= 0` 永真，循环永不结束；程序中加了 `cap` 才停得下来。

【底层机制透视】 测试 `a > b` 时，编译器看到 `int` 与 `unsigned` 混合，按 C 的**寻常算术转换（usual arithmetic conversions）**把 `int` 提升为 `unsigned`，再发出**无符号**比较（`cmpl` + `jbe` 一类），因此负数的"极大正值"解释获胜。而 UB 的本质是：编译器可以假设有符号溢出永不发生，于是把 `x + 1 > x` 直接优化为"恒真"，`abs(INT_MIN)` 也可能被折叠成任意值。**UB 不是"结果随机"，而是"编译器的任何行为都合法"。**

【实测验证】：

```
(unsigned)(-1)              = 4294967295   (UINT_MAX = 4294967295)
(unsigned)(-1) > 0          = 1
(int) 2147483648U           = -2147483648   (INT_MIN)
2147483647U > -2147483647-1 = 0  (混合运算: 右边被转成 unsigned)
2147483647U < (int)2147483648U = 1  (显式转回 int 才按有符号比较)
混用: if (-1 > 0U) 判定为【真】(因为 -1 被转成 4294967295)

-- 以下两行涉及未定义行为 (UB), 结果依编译器/优化级别而异 --
abs(INT_MIN) = -2147483648   (仍是负数! 期望 2147483648 无法表示)
-INT_MIN     = -2147483648   (回绕成 INT_MIN 本身)
INT_MIN == -INT_MIN ? 1

无符号倒计数 n=3, 上限 6 步内观察到 6 次迭代: i 取值序列 2,1,0,4294967295,4294967294,... 永不小于 0
改用 int 倒计数: sum = 3
0U - 1U = 4294967295  (模 2^32 回绕)
```

> ⚠️ **注意**：`abs(INT_MIN)`、`-INT_MIN` 的"回绕成 INT_MIN"只是当前 GCC 12.2.0 在本机的实际行为，**属于 UB，换编译器或开 `-O2` 可能得到完全不同的结果**（例如循环被优化成死循环）。

2.3.3 整数运算、移位与 Data Lab 位级技巧

代码 (C)：

```c
/* /tmp/c02_arith_bits.c  编译: gcc -g -Wall -std=c11 c02_arith_bits.c -o c02_arith_bits */
#include <stdio.h>
#include <limits.h>

/* ---------- Data Lab 风格：只允许 ! ~ & ^ | + << >>，禁止条件与循环 ---------- */
int dl_negate(int x)      { return ~x + 1; }                              /* -x  */
int dl_bitAnd(int x, int y){ return ~(~x | ~y); }                         /* x & y，De Morgan */
int dl_isEqual(int x, int y){ return !(x ^ y); }                          /* x == y */
int dl_logicalNeg(int x)  { return ((x | (~x + 1)) >> 31) + 1; }          /* !x */
int dl_sign(int x)        { return (x >> 31) | (!!x); }                   /* -1 / 0 / 1 */
int dl_isPowerOf2(int x)  { return !(x & (x + ~0)) & !(x >> 31) & !!x; }  /* x > 0 且是 2 的幂 */

/* ---------- 溢出检测（教材 2.3.2 的公式） ---------- */
int uadd_ok(unsigned u, unsigned v) { unsigned s = u + v; return s >= u && s >= v; }
int tadd_ok(int u, int v)
{
    int s = u + v;
    int pos_over = (u > 0) && (v > 0) && (s <= 0);
    int neg_over = (u < 0) && (v < 0) && (s >= 0);
    return !(pos_over || neg_over);
}
int tmult_ok(int u, int v)
{
    if (u == 0 || v == 0) return 1;
    long long p = (long long) u * (long long) v;   /* 用 64 位精确积判断 */
    return p >= INT_MIN && p <= INT_MAX;
}

int main(void)
{
    printf("== 无符号加法: 8 位视角 ==\n");
    unsigned char a = 0xE9, b = 0xD5;
    printf("0xE9 + 0xD5 = 0x%02X  (真和 %u，模 256 后 %u)\n",
           (unsigned char)(a + b), (unsigned) a + b, ((unsigned) a + b) % 256);
    printf("0xE9=233, 0xD5=213: 233 + 213 = 446, 446 mod 256 = %u (0xBE)\n",
           (233u + 213u) % 256);

    printf("\n== 补码加法 ==\n");
    printf("0xE9 + 0xD5 (int8 视角): %d + %d = %d\n", (signed char) 0xE9, (signed char) 0xD5,
           (signed char)(0xE9 + 0xD5));
    printf("tadd_ok(INT_MAX, 1)   = %d  (应为 0: 正溢出)\n", tadd_ok(INT_MAX, 1));
    printf("tadd_ok(INT_MIN, -1)  = %d  (应为 0: 负溢出)\n", tadd_ok(INT_MIN, -1));
    printf("tadd_ok(100, -200)    = %d  (应为 1)\n", tadd_ok(100, -200));
    printf("uadd_ok(UINT_MAX, 1U) = %d  (应为 0)\n", uadd_ok(UINT_MAX, 1U));

    printf("\n== 乘法 / 移位 ==\n");
    printf("tmult_ok(46341, 46341) = %d  (应为 0: 46341^2 溢出 int)\n", tmult_ok(46341, 46341));
    int u = 15213;
    printf("u << 3        = %d  (== u*8  = %d)\n", u << 3, u * 8);
    printf("(u<<5)-(u<<3) = %d  (== u*24 = %d)\n", (u << 5) - (u << 3), u * 24);

    int neg = -15213;
    unsigned uneg = (unsigned) neg;
    printf("\n-15213        = 0x%08X\n", uneg);
    printf("-15213 >> 1   (算术) = %d   0x%08X\n", neg >> 1, (unsigned)(neg >> 1));
    printf("-15213 >> 4   (算术) = %d   0x%08X\n", neg >> 4, (unsigned)(neg >> 4));
    printf("-15213 >> 8   (算术) = %d   0x%08X\n", neg >> 8, (unsigned)(neg >> 8));
    printf("(unsigned)-15213 >> 1 (逻辑) = %u  0x%08X\n", uneg >> 1, uneg >> 1);
    printf("(-15213)/2 = %d  (C 向零取整) vs -15213>>1 = %d  (向下取整)\n", neg / 2, neg >> 1);
    printf("偏置修正: (-15213 + (1<<1)-1) >> 1 = %d  (== 向零取整结果)\n",
           (neg + (1 << 1) - 1) >> 1);
    printf("(-15213)/8 = %d  vs  (-15213 + 7) >> 3 = %d\n", neg / 8, (neg + 7) >> 3);

    printf("\n== Data Lab 风格技巧 ==\n");
    printf("dl_negate(-15213)  = %d  (期望 15213)\n", dl_negate(-15213));
    printf("dl_bitAnd(0x69,0x55) = 0x%X (期望 0x41)\n", dl_bitAnd(0x69, 0x55));
    printf("dl_isEqual(5,5) = %d  dl_isEqual(5,6) = %d\n", dl_isEqual(5, 5), dl_isEqual(5, 6));
    printf("dl_logicalNeg(0) = %d  dl_logicalNeg(7) = %d  (期望 !x)\n",
           dl_logicalNeg(0), dl_logicalNeg(7));
    printf("dl_sign(-9)=%d dl_sign(0)=%d dl_sign(9)=%d\n", dl_sign(-9), dl_sign(0), dl_sign(9));
    printf("dl_isPowerOf2(1)=%d (2)=%d (6)=%d (-8)=%d\n",
           dl_isPowerOf2(1), dl_isPowerOf2(2), dl_isPowerOf2(6), dl_isPowerOf2(-8));
    printf("x & (x-1) 清掉最低位的 1: 0x%X -> 0x%X\n", 0x6C, 0x6C & (0x6C - 1));
    printf("x & -x   取出最低位的 1: 0x%X -> 0x%X\n", 0x6C, 0x6C & (-0x6C));
    printf("!!0 = %d, !!5 = %d, !0 = %d\n", !!0, !!5, !0);
```

```
    puts("\n-- 移位量 <0 或 >= 字长 是未定义行为 (UB)，结果依机器/编译器而异 --");
    volatile int sh = 32;
    printf("1 << 32 (UB) 实际得到: %d\n", 1 << sh);
    return 0;
}
```

【代码做什么？】 前三段分别验证无符号回绕（$233 + 213 = 446 \to 190$）、补码加法与溢出检测公式、乘法/移位等价；第四段把算术右移与逻辑右移、C 的向零取整除法三者放在一起对照；第五段集中展示 Data Lab 的核心技巧；末尾演示移位量越界这一 UB。

【底层机制透视】 – `dl_negate` 就是 $\sim x + 1$，无分支、无乘法。 – `dl_logicalNeg` 的依据是：$x \neq 0$ 时 x 与 $-x$ 中必有一个最高位为 1，故 `x | (~x+1)` 的符号位即"x 非零"的标志；`>> 31` 得到 `-1` 或 `0`，`+1` 归一化为 `1` 或 `0`。 – `dl_isPowerOf2` 用 `x & (x-1)` 清掉最低位的 1：2 的幂只有一个 1，清掉后为 0。注意 `!(x >> 31)` 排除负数、`!!x` 排除 0，且常量 `~0` 只有 8 位，满足 Data Lab"常量不得超过 8 位"的限制。 – `x & -x` 是"取出最低位的 1"（lowbit），`x & (x-1)` 是"清除最低位的 1"，这一对操作是位运算的基本功。 – `tadd_ok` 用的是**符号判断**而非比较位宽：同号相加结果变号即为溢出，无需更宽的中间类型。

【实测验证】：

```
== 无符号加法: 8 位视角 ==
0xE9 + 0xD5 = 0xBE  (真和 446, 模 256 后 190)
0xE9=233, 0xD5=213: 233 + 213 = 446, 446 mod 256 = 190 (0xBE)

== 补码加法 ==
0xE9 + 0xD5 (int8 视角): -23 + -43 = -66          <- 与讲义 TAdd 表格一致
tadd_ok(INT_MAX, 1)   = 0  (应为 0: 正溢出)
tadd_ok(INT_MIN, -1)  = 0  (应为 0: 负溢出)
tadd_ok(100, -200)    = 1  (应为 1)
uadd_ok(UINT_MAX, 1U) = 0  (应为 0)

== 乘法 / 移位 ==
tmult_ok(46341, 46341) = 0  (应为 0: 46341^2 溢出 int)
u << 3        = 121704  (== u*8  = 121704)
(u<<5)-(u<<3) = 365112  (== u*24 = 365112)

-15213        = 0xFFFFC493
-15213 >> 1   (算术) = -7607   0xFFFFE249
-15213 >> 4   (算术) = -951   0xFFFFFC49
-15213 >> 8   (算术) = -60   0xFFFFFFC4
(unsigned)-15213 >> 1 (逻辑) = 2147476041  0x7FFFE249
(-15213)/2 = -7606  (C 向零取整) vs -15213>>1 = -7607  (向下取整)
偏置修正: (-15213 + (1<<1)-1) >> 1 = -7606  (== 向零取整结果)
(-15213)/8 = -1901  vs  (-15213 + 7) >> 3 = -1901

== Data Lab 风格技巧 ==
dl_negate(-15213)  = 15213  (期望 15213)
dl_bitAnd(0x69,0x55) = 0x41 (期望 0x41)
dl_isEqual(5,5) = 1  dl_isEqual(5,6) = 0
dl_logicalNeg(0) = 1  dl_logicalNeg(7) = 0  (期望 !x)
dl_sign(-9)=-1 dl_sign(0)=0 dl_sign(9)=1
dl_isPowerOf2(1)=1 (2)=1 (6)=0 (-8)=0
x & (x-1) 清掉最低位的 1: 0x6C -> 0x68
x & -x   取出最低位的 1: 0x6C -> 0x4
!!0 = 0, !!5 = 1, !0 = 1

-- 移位量 <0 或 >= 字长 是未定义行为 (UB)，结果依机器/编译器而异 --
1 << 32 (UB) 实际得到: 1
```

`-15213 >> 1/4/8 = -7607/-951/-60` 与讲义表格逐行一致，正是"算术右移向下取整"的证据。`1 << 32` 在本机得到 `1`（x86 的移位量按 32 取模），但**这是 UB，不可依赖**。

2.3.4 真实 x86-64 汇编：乘以 24、算术右移与偏置修正

代码 (C)：

```
/* /tmp/c02_asm.c   查看汇编: gcc -O1 -S -std=c11 c02_asm.c -o c02_asm.s */
int       arith(int x, int y) { return (x + y) * 24; }
int       mul3(int x)         { return x * 3; }
int       udiv8(unsigned u)   { return u >> 3; }
int       sdiv8(int x)        { return x / 8; }
unsigned  umul(unsigned x, unsigned y) { return x * y; }
```

【与汇编 / 硬件的对应】（`gcc -O1 -S` 后 `objdump -d c02_asm_O1.o` 得到的真实 AT&T 汇编与机器码字节）：

```
0000000000000000 <arith>:
   0:   01 f7           add    %esi,%edi          # x + y (丢弃进位: addl 语义)
   2:   8d 04 7f        lea    (%rdi,%rdi,2),%eax # 3 * (x+y), 一条 LEA 完成
   5:   c1 e0 03        shl    $0x3,%eax          # 再 << 3  => (x+y)*24
   8:   c3              retq

0000000000000013 <sdiv8>:
  13:   8d 47 07        lea    0x7(%rdi),%eax     # 偏置: x + 2^3 - 1 = x + 7
  16:   85 ff           test   %edi,%edi          # 判断 x 的符号
  18:   0f 49 c7        cmovns %edi,%eax          # 若 x >= 0 则用原值 (不加偏置)
  1b:   c1 f8 03        sar    $0x3,%eax          # 算术右移 3 位 == 向零取整的 /8
  1e:   c3              retq

000000000000001f <umul>:
  1f:   89 f8           mov    %edi,%eax
  21:   0f af c6        imul   %esi,%eax          # 32 位乘法, 只保留低 32 位
  24:   c3              retq
```

同时对照 `udiv8` 只用了 `movl %edi,%eax; shrl $3,%eax`——**无符号除法只需一条逻辑右移**，而有符号除法要多出"加偏置 + 条件选择"三步。这正是 $(x + 2^k - 1) \gg k$ 公式在真实编译器输出里的样子：**GCC 用 `cmovns` 而不是分支**，避免预测失败开销；`umul` 用的是 `imul`，因为 x86-64 中**有符号与无符号乘法的低 w 位完全相同**，同一条指令即可服务于两者。

2.4 实验关联

L1 Data Lab（`LAB-datalab.txt`）是本讲唯一的直接对应实验，全部 13 个 puzzle 都在考本讲内容。

- **规则映射**：整数 puzzle 只允许 8 个运算符 `! ~ & ^ | + << >>`，**禁止循环与条件**（必须是直通代码 straight-line code），常量不得超过 8 位；浮点 puzzle 允许条件与循环，但禁止任何浮点类型/运算/常量，参数与返回值都以 `unsigned` 传递位模式。这正是 2.3.3 节全部代码遵循的约束——那里的每个函数都能通过 `dlc`。
- **典型 puzzle 与本讲的对照**：`bitXor` 用 De Morgan（2.2.3 节 `dl_bitAnd` 的同款思路）；`tmin` 直接用 `1 << 31` 或 `0x80000000` 的等价 8 位构造；`negate` 就是 $\sim x + 1$；`isTmax` 依赖 $TMax + 1 = TMin$ 的回绕；`allOddBits` 用掩码；`logicalNeg` 对应 2.3.3 节的 `x | (~x+1)`；`conditional`（实现 `x ? y : z`）用 `~((x|~x+1)>>31)` 造出全 0/全 1 掩码；`isLessOrEqual` 必须分别处理同号与异号（同号相减会溢出，异号相减不会）；`howManyBits` 用二分法逐段判断高位是否为 0。
- **工具链**：`make` 后依次使用 `./btest`（功能正确性，`./btest -f bitXor -1 4 -2 5` 可指定单个函数与参数）、`./dlc -e bits.c`（检查运算符计数与规则合规）、`./driver.pl`（汇总正确性 36 分 + 性能 26 分 + 风格 5 分，满分 67）。
- **必须提前知道的坑**：`bits.c` **不要** `#include <stdio.h>`（会让 `dlc` 报出难以理解的错误；调试时仍可直接调用 `printf`，忽略 gcc 的隐式声明警告即可）；`dlc` 强制"**声明必须出现在块内所有非声明语句之前**"（C89 风格），把 `int b = a;` 写在 `a *= 3;` 之后会报错；浮点 puzzle 请先用 `./fshow 2080374784` 之类的命令观察位域结构再动手。讲义还附带可选的 **"Beat the Prof" 竞赛**，目标是用最少运算符数追平或击败教师。

2.5 常见错误与调试技巧

- **`&` 写成 `&&`（或 `|` 写成 `||`）**：`if (x & y)` 与 `if (x && y)` 在 `x=1, y=2` 时结果相反；`p & *p` 会真的解引用空指针。**调试**：`gcc -Wall -Wextra -Wlogical-op`，GCC 会提示"logical operator applied to bitwise"；用 `gdb` 单步到该行，`p x`、`p y`、`p x & y` 对比。
- **有符号/无符号混合比较**：`if (len - 1 >= 0)`、`for (unsigned i = n-1; i >= 0; i--)` 恒真，循环永不退出甚至越界。**调试**：`gcc -Wall -Wextra -Wsign-compare`（`-Werror` 可强制修复）；出现"卡死"时用 `gdb -p <pid>` 附加后 `bt` 看栈、`p i` 看计数器的实际类型与值。
- **`abs(INT_MIN)`、`-INT_MIN` 所得为负**：负数的相反数在 `int` 中不可表示，是 UB。**调试**：`gcc -fsanitize=undefined -g`（UBSan）编译运行，会打印 `runtime error: negation of -2147483648 cannot be represented in type 'int'`；用 `-ftrapv` 让有符号溢出直接 abort。
- **移位量越界**：`1 << 32`、`x >> -1` 是 UB，在 x86 上移位量被按 32/64 取模，看起来"能用"却不可移植。**调试**：UBSan 报 `shift exponent 32 is too large for 32-bit type 'int'`；确认字长用 `p sizeof(int)*8`。
- **把算术右移当除法用**：`x >> k` 对负数是向下取整，`-15213 >> 1 = -7607` 而 `/ 2 = -7606`。**调试**：写单元测试对比 `x >> k` 与 `x / (1 << k)`，用 `gcc -S` 检查编译器是否生成了 `sar` 前缺偏置。
- **字节序误判**：把 `int` 直接 `fwrite` 到文件再用大端机读，得到 `0x67452301` 式的乱值；用 `%x` 打印单字节导致 `01` 显示成 `1`。**调试**：`show_bytes` 打印；`gdb` 中用 `x/4xb &x` 看原始字节、`p/x x` 看数值；跨平台传输一律用 `htonl/ntohl` 或文本格式。
- **隐式截断丢高位**：`unsigned char c = 300;` 得 44；把 `int` 塞进 `short` 在赋值点静默截断。**调试**：`-Wconversion`；`gdb` 下 `p (short) x` 与 `p x` 对照，或 `p/x x` 看位模式。
- **Data Lab 里违反 dlc 规则**：用了 `-`（一元负号）、`&&`、`if`，或常量超过 8 位，导致 `dlc` 报错而 `btest` 仍然通过，最终 `driver.pl` 扣掉性能分。**调试**：先跑 `./dlc -e bits.c` 看每个函数的运算符计数，再跑 `./btest -f <name>`；用 `~0`、`1 << 31` 之类的方式构造长常量。

2.6 关键要点

- **比特没有类型，只有解释。** 同一串 `0xFFFFFFFF` 可以是 4294967295，也可以是 -1，区别只在类型；转换是"免费"的（不产生指令），但会静默改变比较与除法的语义。
- **补码 = 最高位权重取负。** $B2T$ 与 $B2U$ 的唯一差别是 x_{w-1} 的权重为 -2^{w-1}；由 $x+ \sim x = -1$ 立刻得到 $-x = \sim x + 1$，而 $TMin$ 没有正伴侣。
- **有符号与无符号的加减乘在硬件上共用同一套位级运算。** 结果位相同、解释不同；代价是混合表达式里 `int` 被隐式转成 `unsigned`，`-1 > 0U` 为真。
- **右移的语义必须分清三个概念**：无符号逻辑右移 = 整除；有符号算术右移 = 向下取整（负数需加 $2^k - 1$ 偏置才等于 C 的向零取整）；负数右移是**实现定义**，移位量越界才是 UB。
- **溢出是回绕而非报错。** 无符号按 2^w 取模、补码按双向回绕；编译器可以**假设有符号溢出永不发生**，所以 UB 代码"现在能跑"不代表正确。
- **Data Lab 的规则就是本讲的纪律**：只用 8 个运算符、无分支、常量 ≤ 8 位——它逼你从"写代码"切换到"设计掩码与位模式"的思维。

2.7 思考题（带答案）

Q1（计算/推演题） 某 16 位字在内存中的字节序列（从低地址到高地址）为 `6d 3b`。求把它当作 `unsigned short` 与 `short` 解释时的值；若这台机器是**大端机**，同样的字节序列又表示什么？另外，已知 `x = 0xC493`，分别写出 $B2U_{16}$ 与 $B2T_{16}$ 的计算过程。

答：小端机中低地址字节是 LSB，故位模式为 `00111011 01101101` = `0x3B6D`。
$B2U_{16} = 3 \times 16^3 + 11 \times 16^2 + 6 \times 16 + 13 = 12288 + 2816 + 96 + 13 = 15213$；最高位为 0，所以 $B2T_{16} = 15213$。大端机则把 `6d` 当 MSB，位模式变成 `0x6D3B`，$B2U = 6 \times 4096 + 13 \times 256 + 3 \times 16 + 11 = 24576 + 3328 + 48 + 11 = 27963$，仍为正数。对 `0xC493`：
$B2U_{16} = 12 \times 4096 + 4 \times 256 + 9 \times 16 + 3 = 49152 + 1024 + 144 + 3 = 50323$；
$B2T_{16} = -1 \times 2^{15} + 50323 = -32768 + 50323 = -15213$，与讲义表格一致。（也即 `-15213` 的 16 位补码正是 `0xC493`。）

Q2（计算/推演题） 判断下列 C 表达式的值并说明原因：`(unsigned)(-1) > 0`、`-1 > (unsigned)-1`、`2147483647U < (int)2147483648U`、`(x & (x-1)) == 0` 在 `x = 0x80000000` 时是否成立。

答：`(unsigned)(-1)` 把 `0xFFFFFFFF` 按无符号解释得 4294967295，故 `> 0` 为**真**。`-1 > (unsigned)-1`：右侧是 UINT_MAX，`-1` 被隐式转成 UINT_MAX，两者**相等**，`>` 为**假**。`2147483647U < (int)2147483648U`：右侧显式转回 `int` 得 `INT_MIN = -2147483648`，再按混合规则转成 `unsigned` 得 2147483648U，故 2147483647U < 2147483648U 为**真**。`x = 0x80000000`（即 `INT_MIN`）时 `x-1 = 0x7FFFFFFF`，`x & (x-1) = 0`，条件**成立**——但注意它并不说明 `x` 是 2 的幂，正因如此 `dl_isPowerOf2` 必须再加 `!(x>>31)` 排除负数。

Q3（直观但错误的想法的错在哪） 有同学说："`abs(x)` 一定非负，所以 `abs(INT_MIN) > 0` 恒真；而且有符号整数溢出最多就是结果错误，无所谓。"请指出错误，并说明无符号倒计数循环 `for (unsigned i = n-1; i >= 0; i--)` 为什么会死循环。

答：两处都错。第一，`|INT_MIN| = 2147483648` 超出 `int` 的表示范围（$TMax = 2147483647$），所以 `abs(INT_MIN)` 的结果**不可表示**，标准规定这是 UB；本机 GCC 12.2.0 实测返回 `-2147483648`，仍是负数，换优化级别可能得到别的值。第二，有符号溢出是 UB，编译器可据此做优化（如把 `x+1 > x` 判为恒真、把 `abs(INT_MIN)>0` 折叠为常量），因此"只是结果错"的假设会让程序在换编译器后彻底失效。而 `unsigned i` 的取值范围是 $[0, 2^{32}-1]$，`i >= 0` 恒为真；当 `i == 0` 时执行 `i--` 按模 2^{32} 回绕成 4294967295，循环永远不会退出（若循环体访问 `a[i]` 则立刻越界）。正确写法是用有符号 `int i`，或改写成 `for (unsigned i = n; i-- > 0; )`。

Q4（实现细节题） 为什么 `-15213 / 8` 的结果是 `-1901`，而 `-15213 >> 3` 在未加偏置时得到什么？编译器为实现 `/8` 实际发出了哪些指令？

答：C 的整数除法**向零取整**，`-15213/8 = -1901.625 \to -1901`。算术右移 k 位等价于 $\lfloor x/2^k \rfloor$，是**向下取整**，实测 `-15213 >> 3` 得 -1902，比正确的 -1901 小 1（讲义表中 $k=1$ 时 `-15213>>1 = -7607` 而 `-15213/2 = -7606`，正是同一偏差）。因此必须先把被除数向 0 偏置：`(x + (1<<3) - 1) >> 3 = (-15213+7)>>3 = -1901`，与除法一致。GCC 12.2.0 `-O1` 对 `sdiv8` 的实测输出是 `lea 0x7(%rdi),%eax; test %edi,%edi; cmovns %edi,%eax; sar $0x3,%eax`——用 `cmovns` 无条件选择是否加偏置（`x >= 0` 时不加，因为正数右移与除法本就一致），最后一条 `sar` 完成除 8。

Lecture 3: 机器级编程 I——基础、算术与逻辑运算 (Machine Programming I: Basics, Arithmetic and Logic)

> **讲义对应**：CMU 15-213 Lecture 3 — Machine Programming I: Basics（素材：`F25-03-machine-alpha.txt`；课堂活动：`machine-control-soln.txt`） **教材对应**：CS:APP3e 第 3 章 3.1–3.6（History of Intel processors、Program Encodings、Data Formats、Accessing Information、Arithmetic and Logical Operations、Control） **关联 Lab**：L0 C Programming / **L2 Bomb Lab**（反汇编与 GDB 前置知识）

3.1 概述

前两讲停留在"信息的表示"层面：我们讨论位与补码，但程序仍是一行行 C 语句。本讲第一次把视线下沉到**机器级（machine level）**：编译器究竟把 `x = y + 1` 变成了什么？答案是**指令序列（instruction sequence）**——一种 CPU 能直接执行的、极其朴素的"动词 + 名词"语言。

本讲引入三组只有机器级程序员才关心的**体系结构可见状态（architecturally visible state）**：程序计数器 `%rip`、16 个 64 位整数寄存器、四个条件码（condition codes）以及按字节寻址的内存。围绕这些状态，我们学习 x86-64 的**数据传送**（`mov` / `movz` / `movs` / `movabsq` / `leaq`）、**算术与逻辑运算**（`add` / `sub` / `imul` / `sal` / `sar` / `xor` / `and` / `or` / `inc` / `dec` / `neg` / `not`）以及**条件码的设置与读取**（`cmp` / `test` / `setX`）。

本讲是后续三讲的基石：Lecture 4 用它讲控制流，Lecture 5 讲过程调用与栈帧，Lecture 6 讲数组、结构体与浮点。它同时是 **Bomb Lab** 的入场券——拆弹时做的第一件事，就是把 `objdump -d` 的输出翻译回 C 的"伪代码"。

3.2 核心概念与底层机制图解

3.2.1 体系结构、微体系结构与指令集（Architecture vs. Microarchitecture vs. ISA）

- **定义与目的**：**体系结构（architecture）**，又称**指令集体系结构（ISA, Instruction Set Architecture）**，是"为了写汇编/机器码所必须理解的那部分处理器设计"——指令集规范、寄存器集合、寻址方式。**微体系结构（microarchitecture）**是它的具体实现——缓存多大、主频多高、流水线几级。前者是程序员与编译器面对的**契约**，后者是硬件工程师的自由空间。
- **直观解释（"它是什么？"）**：ISA 像**交通法规与路牌**，微体系结构像**具体某款车的发动机**。路牌不变，换辆车照样到达目的地——同一份 x86-64 机器码能在 Intel Core i7 和 AMD Ryzen 上跑，因为二者实现了同一套 ISA。
- **底层机制图解**：

- **与机器码/硬件的对应**：`movq %rax, (%rbx)` 的机器码是三字节 `48 89 03`：`48` 是 REX 前缀（标志 64 位操作数），`89` 是 `mov` 的操作码（opcode），`03` 是 ModRM 字节。**执行给字节赋予了类型**：同一段字节 `48 89 03` 若被当数据读出就只是三个整数，只有 `%rip` 指向它时才是"存储指令"。

CISC 与 RISC，以及 x86 的"怪异性"。x86 属于**复杂指令集计算机（CISC）**：指令多、格式杂；对照的 **RISC（如 ARM、RISC-V）** 指令定长、格式规整。x86 的两大怪异之处正源于此：

1. **变长指令（variable-length instructions）**：一条指令占 1、3、5 甚至 15 字节。讲义中 `sumstore` 的 6 条指令共 14 字节。这让解码器远比 RISC 复杂，也是"从任意偏移开始反汇编会得到垃圾"的根源。
2. **复杂寻址模式（complex addressing modes）**：一条 `mov` 就能完成"基址 + 变址×比例 + 位移"的三项计算并访存，RISC 通常需三到四条指令。

x86 的历史是**演化式（evolutionary）**的：1978 年 8086（16 位、1 MB 地址空间）→ 1985 年 386（首个 32 位 Intel 处理器，即 **IA32**，引入平坦寻址）→ 2004 年 Pentium 4E（首个 64 位 x86，即 **x86-64**）。2001 年 Intel 试图用全新的 IA64/Itanium 取代 IA32 但性能令人失望；2003 年 **AMD** 以演化式方案 x86-64（今称 **AMD64**）胜出，2004 年 Intel 以几乎相同的 EM64T 跟进。**向后兼容一直到 8086**，正是今天各种"怪异性"的成因。本课程只讲 x86-64（教材另有一套简化教学指令集 **Y86-64**，本讲幻灯片未涉及，故不展开）。

3.2.2 机器级代码的两种视角（Two Views: Assembly Programmer vs. Machine）

- **定义与目的**：**汇编程序员视角**关注"我能看见并操纵什么状态"；**机器视角**关注"这些状态在硬件上由什么部件实现"。
- **直观解释（"它是什么？"）**：把 CPU 想成一张**工作台**。寄存器是台面上随手可取的**小零件盒**（快、少、贵）；内存是墙上的**大货架**（慢、多、便宜）；`%rip` 是**你手指指着的那一行说明书**；条件码是台面上一盏**指示灯**，刚做完运算就亮一下，告诉你结果是零、是负还是溢出了。
- **底层机制图解**：

```
      x86-64 机器级程序员可见状态（Programmer-Visible State）
+-------------------------------------------------------------------------------+
| %rip  程序计数器（Program Counter, PC）—— 指向下一条要执行的指令的地址       |
| 寄存器文件（Register File）：16 个 64 位整数寄存器（不是内存，也不属于缓存） |
|   %rax %rbx %rcx %rdx %rsi %rdi %rbp %rsp                                     |
|   %r8  %r9  %r10 %r11 %r12 %r13 %r14 %r15                                    |
| 条件码（Condition Codes）：CF  ZF  SF  OF  各 1 位                            |
| 向量寄存器（Vector Registers）：%xmm0–%xmm15 / %ymm0–%ymm15（详见 Lecture 6）|
| 内存（Memory）：按字节寻址的线性数组                                          |
| +----------+---------------+-------------+----------+                       |
| | 代码 text | 用户数据 data | 栈 stack    | 堆 heap  |                       |
| +----------+---------------+-------------+----------+                       |
| 高地址 <----------------------------------> 低地址                           |
| 栈向低地址增长，%rsp 指向栈顶                                                 |
+-------------------------------------------------------------------------------+
```

- **与机器码/硬件的对应**：编译器必须把**语句、表达式、过程调用**翻译成上述状态的读写序列。寄存器文件是 CPU 内部多端口存储体；条件码是 ALU 输出端的标志触发器，由进位链与零检测电路驱动。

3.2.3 x86-64 整数寄存器与数据格式（Integer Registers & Data Formats）

- **定义与目的**：16 个 64 位通用寄存器是机器级编程的"名词表"，可用**不同宽度的别名**访问低 8/16/32 位，以支持不同尺寸的 C 类型。
- **直观解释（"它是什么？"）**：寄存器像**可分段取用的水杯**：整杯是 `%rax`，只留 4 字节是 `%eax`，再只留 2 字节是 `%ax`，最后 1 字节是 `%al`。

```
 x86-64 整数寄存器文件（16 x 64 bit，讲义 reference 页）
+-----------+-------------+-------------+-------------+----------------------------+
| 64 位     | 32 位       | 16 位       | 8 位        | 约定用途                   |
+-----------+-------------+-------------+-------------+----------------------------+
| %rax      | %eax        | %ax         | %al         | 返回值（return value）     |
| %rbx      | %ebx        | %bx         | %bl         | 被调用者保存               |
| %rcx      | %ecx        | %cx         | %cl         | 第 4 个整型参数            |
| %rdx      | %edx        | %dx         | %dl         | 第 3 个整型参数            |
| %rsi      | %esi        | %si         | %sil        | 第 2 个整型参数            |
| %rdi      | %edi        | %di         | %dil        | 第 1 个整型参数            |
| %rbp      | %ebp        | %bp         | %bpl        | 被调用者保存（常作帧指针） |
| %rsp      | %esp        | %sp         | %spl        | 栈指针（特殊的调用约定）   |
| %r8       | %r8d        | %r8w        | %r8b        | 第 5 个整型参数            |
| %r9       | %r9d        | %r9w        | %r9b        | 第 6 个整型参数            |
| %r10–%r15 | %r10d–%r15d | %r10w–%r15w | %r10b–%r15b | 调用者保存                 |
+-----------+-------------+-------------+-------------+----------------------------+
```

三条必须背下来的约定：

1. **参数寄存器顺序** `%rdi, %rsi, %rdx, %rcx, %r8, %r9`，返回值在 `%rax`。
2. **调用者保存（caller-saved）vs 被调用者保存（callee-saved）**：`%rax %rcx %rdx %rsi %rdi %r8–%r11` 属于调用者保存——被调用函数可以随意破坏，调用者若还需要就自己先存好；`%rbx %rbp %r12–%r15` 属于被调用者保存——被调用函数若要用，必须先保存、返回前恢复。`%rsp` 特殊：它由硬件与调用约定共同维护，**任何时刻它都指向栈顶**。
3. **写 32 位子寄存器会清零高 32 位**：写 `%eax` 使 `%rax` 高 32 位变 0（x86-64 硬件规定）；而写 `%ax` / `%al` / `%ah` **不会**改变高位。这个不对称是很多汇编谜题的答案。

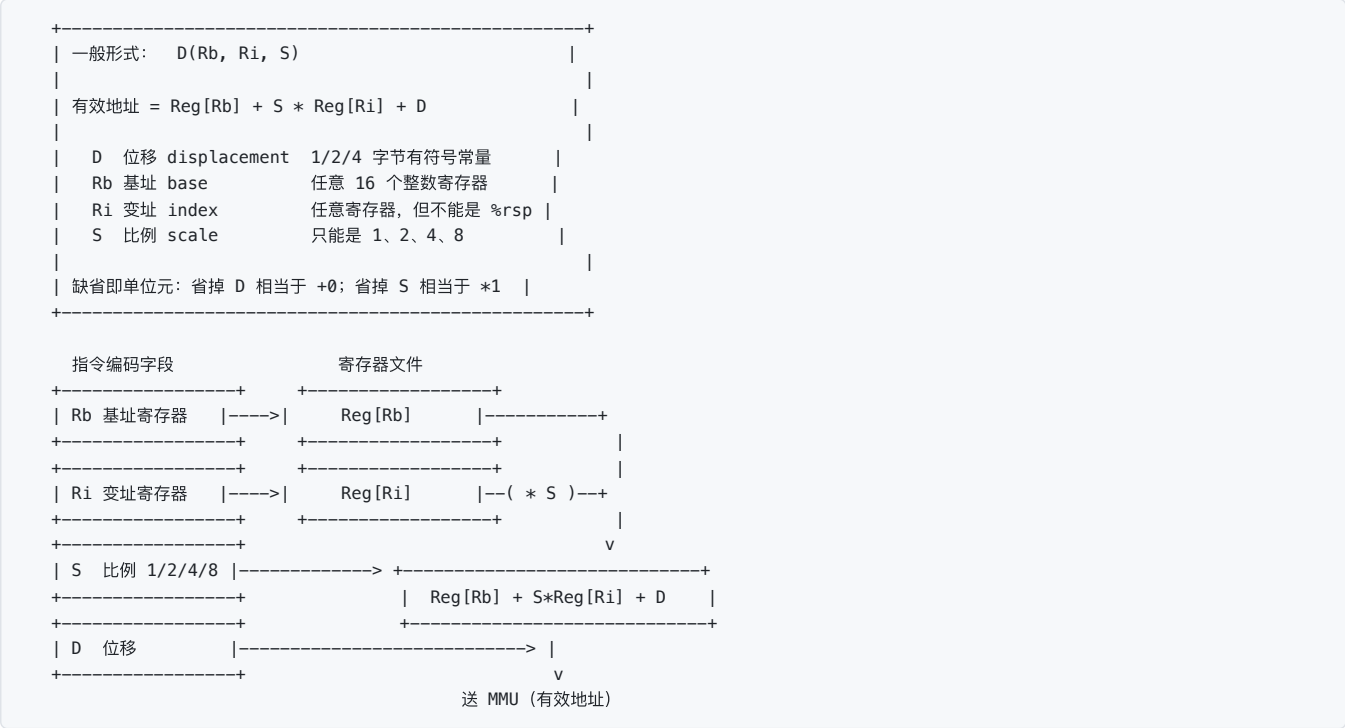
C 数据类型到 x86-64 尺寸的映射与指令后缀见下表。后缀 `b/w/l/q` 分别表示 1/2/4/8 字节；浮点用 `s`（single，4 字节）、`l`（long/double，8 字节）、`ss` / `sd`（标量单/双精度）。

```
+--------+--------+----------+--------------------+
| C 类型 | 字节数 | 汇编后缀 | 说明               |
+--------+--------+----------+--------------------+
| char   | 1      | b        | 字符；8 位         |
| short  | 2      | w        | 16 位              |
| int    | 4      | l        | 32 位              |
| long   | 8      | q        | 64 位（LP64 模型） |
| char * | 8      | q        | 指针也是 8 字节    |
| float  | 4      | s        | 单精度浮点         |
| double | 8      | l / sd   | 双精度浮点         |
+--------+--------+----------+--------------------+
```

值得注意：**`double` 的后缀也是 `l`**（因为它是"long word"），CS:APP 特别指出 `movl` 这个助记符因此有两种含义——操作整数时是 32 位传送，操作浮点时是 64 位传送，靠操作数类型区分。

3.2.4 操作数形式与寻址模式（Operand Forms & Addressing Modes）

- **定义与目的**：x86-64 的指令操作数只有三种来源：**立即数（immediate）**、**寄存器（register）**、**内存（memory）**。内存操作数支持最一般形式 `D(Rb, Ri, S)`。
- **直观解释（"它是什么？"）**：`D(Rb, Ri, S)` 像**快递地址的三个部分**：`Rb` 是"小区大门"（基址），`Ri` 是"楼栋号"（变址），`S` 是"楼间步长"（比例 1/2/4/8），`D` 是"进大门后再走几步"（位移）。**圆括号永远表示"要算出一个内存地址"**。
- **底层机制图解**：



各组合形式与常见 C 对应：

形式	地址计算	C 语言典型出处
Imm	Imm（不是地址！）	常量 5、0x4
%rax	Reg[rax]	局部变量 / 参数
(%rb)	Reg[rb]	*p
D(%rb)	Reg[rb] + D	结构体字段 s->c
(rb, ri)	Reg[rb] + Reg[ri]	二维数组 A[i][j] 的基址部分
D(rb, ri)	Reg[rb]+Reg[ri]+D	数组 + 偏移
(rb, ri, S)	Reg[rb]+S*Reg[ri]	x[i]，元素宽 S 字节
D(rb, ri, S)	Reg[rb]+S*Reg[ri]+D	A[i][j] + 字段偏移
(, ri, S)	S*Reg[ri]	纯比例（讲义 jmp *.L4(,%rdi,8)）

三条硬约束：$S \in \{1, 2, 4, 8\}$（其他值无法编码）；变址 `Ri` **不能是** `%rsp`（该编码字段被占用）；位移 `D` 只能是 1、2 或 4 字节有符号常量。教材的地址计算练习——设 `%rdx = 0xf000`、`%rcx = 0x0100`，则 `0x8(%rdx) = 0xf008`、`(%rdx,%rcx) = 0xf100`、`(%rdx,%rcx,4) = 0xf400`、`0x80(,%rdx,2) = 0x1e080`。

3.2.5 数据传送指令族：`mov` / `movz` / `movs` / `movabsq` / `leaq`

- **定义与目的**：`mov` 系列在寄存器与内存之间搬运数据；`movz` / `movs` 搬运的同时做**零扩展（zero extension）**或**符号扩展（sign extension）**；`movabsq` 专门搬运 64 位立即数；`leaq` **只算地址、不访存**。
- **直观解释（"它是什么？"）**：`mov` 是搬运工，`movz` / `movs` 是"搬运并补零/补符号"的搬运工，`leaq` 则是**拿着地址清单算账的会计**——它甚至不去仓库（内存）看一眼。
- **底层机制图解**：后缀是"三段式" `mov X Y Z`，`X ∈ {s, z}`、`Y` 是源宽度、`Z` 是目的宽度；两位时省略 `X`（如 `movq`）。

```
例：movzbl %al, %eax      z=零扩展  b=源 1 字节  l=目的 4 字节
     源 %al             目的 %eax
    +--------+   +--------------------------------------+
    | 0x80   |-->| 00000000 00000000 00000000 10000000 |
    +--------+   +--------------------------------------+
     补 0                 高 24 位清零

例：movslq %edi, %rax     s=符号扩展  l=源 4 字节  q=目的 8 字节
     源 %edi            目的 %rax
    +------------+   +--------------------------------------+
    | 0xFFFFFFFF |-->| 11111111 11111111 11111111 11111111 …|
    +------------+   +--------------------------------------+
     复制符号位                    = -1
```

⚠️ **一条不可动摇的规则：x86-64 不允许"内存→内存"的单条 `mov`。** 必须用两条指令（内存→寄存器→内存）中转。讲义 `movq Operand Combinations` 表中 `Src` / `Dest` 的 3×3 组合里，Mem→Mem 格子是空的。实测验证：

```
$ printf '\t.text\n\t.globl f\nf:\n\tmovq (%%rdi), (%%rsi)\n\tret\n' > mem2mem.s
$ as mem2mem.s -o mem2mem.o
mem2mem.s:4: Error: too many memory references for `movq'
```

`movabsq` 是为 64 位立即数准备的：任何 64 位常量都必须用它的完整 8 字节形式，而不能用 `movq`。实测 `long big(void){ return 0x123456789abcdef0L; }` 得到：

```
big:
        movabsq $1311768467463790320, %rax
        ret
```

`objdump -d -M intel` 显示为 `movabs rax,0x123456789abcdef0`，机器码 `48 b8 f0 de bc 9a 78 56 34 12`——这是 x86 中少见的 10 字节长指令。

`leaq` 是"地址计算器"，不是加载指令。 `leaq Src, Dst` 把 `Src` 这个**地址表达式**的值放进 `Dst`，**全程不访问内存、也不改变条件码**。它有两个用途：一是翻译 `p = &x[i]`（拿地址）；二是**做算术**——任何形如 $x + k \cdot y$（$k \in \{1, 2, 4, 8\}$）的表达式，一条 `leaq` 就能算完。教材的经典例子是 `long m12(long x) { return x*12; }`，编译为：

```
m12:
        leaq    (%rdi,%rdi,2), %rax   # t = x + 2*x  = 3x
        salq    $2, %rax              # t << 2      = 12x
        ret
```

3.2.6 算术与逻辑运算、条件码（Arithmetic, Logic, and Condition Codes）

- **定义与目的**：算术逻辑指令直接读写寄存器或内存；同时，**除 `mov` 与 `leaq` 之外**的多数指令都会隐式更新四个条件码，为后续的 `jX` / `setX` / `cmovX` 提供依据。
- **直观解释（"它是什么？"）**：条件码是运算后**顺手亮起的四盏指示灯**，每盏灯回答一个问题：

```
+----+--------------------------------------------------------------+
| CF | 进位标志 Carry Flag  —— 无符号溢出（最左一对位产生了进位）  |
| ZF | 零标志 Zero Flag     —— 运算结果为 0                        |
| SF | 符号标志 Sign Flag   —— 结果为负（最高位为 1）              |
| OF | 溢出标志 Overflow Flag —— 有符号溢出（两操作数符号相同，    |
|    | 结果符号与之相反）                                          |
+----+--------------------------------------------------------------+
```

- **底层机制图解**：

```
双操作数指令:   addq Src,Dest   Dest = Dest + Src
                subq Src,Dest   Dest = Dest - Src
                imulq Src,Dest  Dest = Dest * Src
                salq/shlq Src,Dest  Dest = Dest << Src    (左移，等价 *2^k)
                sarq Src,Dest   Dest = Dest >> Src 算术   (补符号位，等价 /2^k)
                shrq Src,Dest   Dest = Dest >> Src 逻辑   (补 0)
                xorq/andq/orq Src,Dest  按位运算

单操作数指令:   incq Dest  Dest += 1        decq Dest  Dest -= 1
                negq Dest  Dest = -Dest      notq Dest  Dest = ~Dest

只设置条件码、不写目的:
     cmpq a, b   —— 计算 b - a (和 sub 一样)，设置 CF/ZF/SF/OF，但不改变 b
     testq a, b  —— 计算 b & a (和 and 一样)，但只设置 SF 和 ZF，不改变 b
                    最常见用法: testq %rX, %rX   (判断 %rX 是否为零/正负)
```

`cmp` 与 `test` 的"只设置不写回"是关键设计：它们是**专门用来提问的指令**。`cmp a, b` 的顺序要牢记——**算的是 $b - a$**，所以 `cmpq %rsi, %rdi` 是拿 `%rdi` 减 `%rsi`（对应 C 的 `x > y`，其中 `x` 在 `%rdi`）。`test a, b` 算 $b\&a$；由于 `x & x == x`，`test %rX, %rX` 就等价于"拿 `%rX` 本身设置条件码"，因此 **`test` 永远清空 CF**（按位与不可能产生进位），这解释了课堂活动的结论：`CMOVC` 接在 `TEST` 后永不触发。

3.2.7 读取条件码：`setX` 与 `movzbl` 收尾

- **定义与目的**：`setX` 按条件码组合，把**目的寄存器的低 1 字节**置为 0 或 1。
- **直观解释（"它是什么？"）**：`cmp` 提问，`setX` 把"是/否"抄成 0/1，`movzbl` 再把这张纸**补齐**成规范的 32 位整数。

```
setX 的目的只写低 8 位:              随后必须清零其余 7 字节:
  cmpq %rsi, %rdi      # x : y          setg  %al        # 只动 %al
  setg  %al            # x > y ?        movzbl %al, %eax # 高 24 位清零

+-----------------------------+        +-------------------------+
| %rax  高 56 位【脏数据】    |        | %rax  高 32 位全 0      |
|       ...      | %al = 0/1 |        |   0000...0000  | 0/1    |
+-----------------------------+        +-------------------------+
    setg 之后（高位未定义）               movzbl 之后（规范的 int）
```

常用 `setX`：`sete`（等于）、`setne`（不等）、`setg`/`setge`/`setl`/`setle`（有符号比较）、`seta`/`setb`（无符号 above/below）。**为什么必须补 `movzbl`**？因为 `return x > y;` 要求返回规范的 `int`（值域 $\{0, 1\}$），而 `setg %al` 只承诺低字节正确，高 56 位是上一轮的残留；不清零则调用者可能看到 `0x...0101` 这类"非零但不止 1"的垃圾。教材示例：

```
gt:
        cmpq    %rsi, %rdi
        setg    %al
        movzbl  %al, %eax
        ret
```

3.3 代码示例与底层机制分析

本节所有汇编均在本机（GCC 12.2.0，`x86-64`）用 `gcc -g -Wall -std=c11 -Og -S` 与 `objdump -d` **实际生成并核对**，不是凭空编写。

3.3.1 示例一：`swap`——把汇编读回 C

代码 (C)：

```c
/* swap.c —— 讲义 "Understanding Swap()" 的完整可编译版本 */
#include <stdio.h>

void swap(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}

int main(void)
{
    long a = 123, b = 456;
    printf("before: a=%ld b=%ld\n", a, b);
    swap(&a, &b);
    printf("after : a=%ld b=%ld\n", a, b);
    return 0;
}
```

【代码做什么？】 1. `main` 在栈上放两个 `long`：`a = 123`、`b = 456`，打印初值。 2. 取 `&a` 放入 `%rdi`、`&b` 放入 `%rsi`，调用 `swap`（`%rdi` 是第 1 参数，`%rsi` 是第 2 参数）。 3. `swap` 先用 `%rax`、`%rdx` 两个临时寄存器把两个值都读出来，再写回去——**必须先读两次、后写两次**，否则会丢失数据。 4. 返回后 `main` 打印交换结果。

【底层机制透视】 这个函数揭示一个重要事实：**"交换"必须经过寄存器中转**，因为 x86-64 不允许内存到内存的直接传送。`%rdi`/`%rsi` 是"指针变量"，`(%rdi)`/`(%rsi)` 才是指向的对象。写成伪 C 只需把寄存器换成变量名：`%rdi → xp`、`%rsi → yp`、`%rax → t0`、`%rdx → t1`。

【内存布局 / 数据结构图解】（沿用讲义的具体地址）：

```
            初始状态                                执行完成后
     寄存器          内存                    寄存器          内存
  寄存器    内存                          寄存器    内存
  +-------+  +--------------+            +-------+  +--------------+
  | %rdi  |  | 0x120: 123   |            | %rdi  |  | 0x120: 456   |
  | 0x120 |  | 0x118:       |            | 0x120 |  | 0x118:       |
  | %rsi  |  | 0x110:       |            | %rsi  |  | 0x110:       |
  | 0x100 |  | 0x108:       |            | 0x100 |  | 0x108:       |
  | %rax  |  | 0x100: 456   |            | %rax  |  | 0x100: 123   |
  |  123  |  +--------------+            |  456  |  +--------------+
  | %rdx  |                              | %rdx  |
  |  456  |                              |  123  |
  +-------+                              +-------+
```

【与汇编 / 硬件的对应】 `gcc -Og -S` 的真实输出（已略去 `.loc`/`.cfi` 伪指令）：

```
swap:
        movq    (%rdi), %rax      # t0 = *xp
        movq    (%rsi), %rdx      # t1 = *yp
        movq    %rdx, (%rdi)      # *xp = t1
        movq    %rax, (%rsi)      # *yp = t0
        ret
```

【实测验证】 `objdump -d` 的两种语法对比（AT&T 与 Intel 的顺序正好相反，是新手最常见的困惑源）：

```
$ objdump -d swap              # AT&T: 源在前、目的在后
0000000000401126 <swap>:
  401126:       48 8b 07                mov    (%rdi),%rax
  401129:       48 8b 16                mov    (%rsi),%rdx
  40112c:       48 89 17                mov    %rdx,(%rdi)
  40112f:       48 89 06                mov    %rax,(%rsi)
  401132:       c3                      retq

$ objdump -d -M intel swap      # Intel: 目的在前、源在后
0000000000401126 <swap>:
  401126:       48 8b 07                mov    rax,QWORD PTR [rdi]
  401129:       48 8b 16                mov    rdx,QWORD PTR [rsi]
  40112c:       48 89 17                mov    QWORD PTR [rdi],rdx
  40112f: 48 89 06              mov    QWORD PTR [rsi],rax
  401132: c3                            ret
```

AT&T 语法是课程与 `gcc -S` 的默认，务必在脑内建立"`src, dst`"的条件反射：看到 `mov %rdx,(%rdi)` 应读成"把 `%rdx` 的值写进 `%rdi` 指向的内存"，而不是反过来。

3.3.2 示例二：`arith`——编译期优化后的乘加算术（`leaq` 的算术用途）

代码 (C)：

```c
/* arith.c —— 讲义 Arithmetic Expression Example 的完整可编译版本 */
#include <stdio.h>

long arith(long x, long y, long z)
{
    long t1 = x + y;
    long t2 = z + t1;
    long t3 = x + 4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}

int main(void)
{
    printf("%ld\n", arith(1, 2, 3));   /* 输出 606 */
    return 0;
}
```

【代码做什么？】 按书写顺序约需 7 次运算。但编译器只用 **6 条指令**完成，且**一次访存都没有**。

【底层机制透视】 关键在 `y * 48`：48 不是 2 的幂，但 $48 = 3 \times 16$，而 $3y = y + 2y$ 可交给 `leaq` 的 `(%rsi,%rsi,2)` 一次算完，再 `salq $4`（左移 4 位 = 乘 16）。这就是 **strength reduction（强度削减）**：用更便宜的移位与 `leaq` 替换昂贵的乘法。全函数只保留一次 `imulq`，因为 `t2 * t5` 的两个乘数都是**运行时才知的变量**，没有代数捷径。

【与汇编 / 硬件的对应】 真实编译结果（`gcc -Og -S arith.c`）：

```
arith:
        leaq    (%rdi,%rsi), %rax   # t1 = x + y         (一条 leaq 完成加法)
        addq    %rdx, %rax          # t2 = z + t1
        leaq    (%rsi,%rsi,2), %rdx #     y + 2*y = 3y
        salq    $4, %rdx            # t4 = 3y * 16 = 48y
        leaq    4(%rdi,%rdx), %rdx  # t5 = (x+4) + t4    (一条 leaq 完成加+加)
        imulq   %rdx, %rax          # rval = t2 * t5
        ret
```

注意 `leaq 4(%rdi,%rdx), %rdx`：它一条指令同时完成了 `+4`（位移字段）、`+ t4`（变址）、`+ x`（基址）三次加法。**这里 `leaq` 完全没有访问内存**——`%rdi` 里存的是整数 `x`，不是地址；`leaq` 只是把这个"看起来像地址"的表达式算出来。

【实测验证】 寄存器分配表（讲义原表）：

```
+--------+---------------------------------+
| 寄存器 | 用途                            |
+--------+---------------------------------+
| %rdi   | 参数 x                          |
| %rsi   | 参数 y                          |
| %rdx   | 参数 z，随后复用为 t4           |
| %rax   | t1 → t2 → rval                  |
| %rcx   | t5（不同 GCC 版本可能用 %rdx）  |
+--------+---------------------------------+
```

`./arith` 输出 `606`，与手算 $t_2 = 3 + 3 = 6$、$t_5 = 5 + 96 = 101$、$6 \times 101 = 606$ 一致。

3.3.3 示例三：`leaq` 用于地址计算——数组、二维数组与结构体

代码 (C)：

```c
/* expr.c —— leaq 的地址计算用途与三操作数乘加 */
#include <stdio.h>
#include <stddef.h>

long *addr_of(long *x, long i) { return &x[i]; }        /* p = &x[i]      */
int get2d(int (*A)[8], long i, long j) { return A[i][j]; } /* 二维数组      */
struct S { int a; int b; long c; };
long *field_c(struct S *s, long k) { return &s[k].c; }  /* 结构体字段偏移  */
long muladd(long x, long y) { return x + 5*y + 7; }     /* 三操作数乘加    */

int main(void)
{
    long x[4] = {10, 20, 30, 40};
    int  A[2][8] = {{0}};
    struct S s[2] = {{0}};
    A[1][3] = 99;
    printf("addr_of(x,2)  -> %ld\n", *addr_of(x, 2));       /* 30            */
    printf("get2d         -> %d\n",  get2d(A, 1, 3));       /* 99            */
    printf("field_c 偏移   = %zu\n", (size_t)((char*)&s[1].c - (char*)s)); /* 24 */
    printf("muladd(3,4)   = %ld\n", muladd(3, 4));          /* 30            */
    return 0;
}
```

【代码做什么？】 四个函数分别把 `leaq` 用在四类经典场景：一维数组取址、二维数组索引、结构体数组字段取址、纯算术乘加。

【底层机制透视】 这四条汇编是"寻址模式如何映射 C 表达式"的模板：

```asm
addr_of:                        # long *addr_of(long *x, long i)
        leaq    (%rdi,%rsi,8), %rax  # x + 8*i   ← 比例因子 8 = sizeof(long)

get2d:                          # int get2d(int (*A)[8], long i, long j)
        salq    $5, %rsi             # i * 32    ← 行宽 8 个 int = 32 字节
        addq    %rsi, %rdi           # A + 32*i
        movl    (%rdi,%rdx,4), %eax  # 再加 4*j 后【访存】取 int

field_c:                        # long *field_c(struct S *s, long k)
        salq    $4, %rsi             # k * 16    ← sizeof(struct S) = 4+4+8 = 16
        leaq    8(%rsi,%rdi), %rax   # s + 16*k + 8  ← 字段 c 的偏移是 8

muladd:                         # long muladd(long x, long y)
        leaq    (%rsi,%rsi,4), %rax  # 5*y       ← y + 4y
        leaq    7(%rax,%rdi), %rax   # 5*y + 7 + x
        ret
```

对比 `addr_of`（只有 `leaq`，**没有访存**，返回的是地址）与 `get2d`（`leaq` / `salq` 算完地址后跟一条 `movl` **真的访存**）。这是 `leaq` 与"带内存操作数的 `mov`"最本质的区别：**`leaq` 只借用寻址模式的算术能力，不借用它的访存能力**。

【实测验证】 `struct S` 的 `sizeof` 为 16（`int a` 占 0—3，`int b` 占 4—7，`long c` 因 8 字节对齐而放在偏移 8 处），`s[1].c` 相对 `s` 的偏移实测为 24 = 16（跨过第一个元素）+ 8（字段偏移），与 `leaq 8(%rsi,%rdi)` 中 `k=1` 时算出的地址完全吻合。

3.3.4 示例四：条件码与 `setX` —— 有符号 vs 无符号

代码 (C)：

```c
/* opdemo.c —— cmp + setX + movzbl 全流程 */
#include <stdio.h>
#include <stdint.h>

int gt(long x, long y) { return x > y; }
int ge(long x, long y) { return x >= y; }
int lt(long x, long y) { return x < y; }
int le(long x, long y) { return x <= y; }
int eq(long x, long y) { return x == y; }
int ne(long x, long y) { return x != y; }

/* 无符号比较 */
int above(uint32_t a, uint32_t b) { return a > b; }
int below(uint32_t a, uint32_t b) { return a < b; }

int main(void)
{
    printf("gt(5,3)=%d ge(5,3)=%d lt(5,3)=%d le(5,3)=%d eq(5,3)=%d ne(5,3)=%d\n",
           gt(5,3), ge(5,3), lt(5,3), le(5,3), eq(5,3), ne(5,3));
    /* 关键差别演示：-1 的位模式是全 1 */
    printf("above(-1,1)=%d  below(-1,1)=%d\n",
           above((uint32_t)-1, 1u), below((uint32_t)-1, 1u));
    return 0;
}
```

【代码做什么？】 六组有符号与两组无符号比较，全部编译成 `cmp` + `setX` + `movzx` 模板。

【与汇编 / 硬件的对应】 `objdump -d -M intel opdemo` 的真实片段：

```
0000000000401126 <gt>:
  401126:       48 39 f7                cmp    rdi,rsi
  401129:       0f 9f c0                setg   al
  40112c:       0f b6 c0                movzx  eax,al
0000000000401130 <ge>:
  401130:       48 39 f7                cmp    rdi,rsi
  401133:       0f 9d c0                setge  al
000000000040113a <lt>:
  40113a:       48 39 f7                cmp    rdi,rsi
  40113d:       0f 9c c0                setl   al
0000000000401144 <le>:
  401144:       48 39 f7                cmp    rdi,rsi
  401147:       0f 9e c0                setle  al
000000000040114e <eq>:
  40114e:       48 39 f7                cmp    rdi,rsi
  401151:       0f 94 c0                sete   al
0000000000401158 <ne>:
  401158:       48 39 f7                cmp    rdi,rsi
  40115b:       0f 95 c0                setne  al
0000000000401162 <above>:
  401162:       39 fe                   cmp    esi,edi
  401164:       0f 92 c0                setb   al
000000000040116b <below>:
  40116b:       39 f7                   cmp    edi,esi
  40116d:       0f 92 c0                setb   al
```

【底层机制透视】 两个细节值得深挖。

其一，**同一个无符号比较，只需交换 `cmp` 的操作数顺序，就能用一条 `setb` 表示两种语义**。换成与课件一致的 AT&T 语法看得最清楚：

```
above:                          # int above(uint32_t a, uint32_t b) → a > b
        cmpl    %edi, %esi        # 计算 %esi − %edi = b − a
        setb    %al               # CF=1 ⟺ 无符号下 b < a ⟺ a > b
        movzbl  %al, %eax
        ret

below:                          # int below(uint32_t a, uint32_t b) → a < b
        cmpl    %esi, %edi        # 计算 %edi − %esi = a − b
        setb    %al               # CF=1 ⟺ 无符号下 a < b
        movzbl  %al, %eax
        ret
```

两者**都**用 `setb`，GCC 只交换了两个 `cmp` 操作数，就把"大于"改写成了"小于"。它偏爱 `setb` 是因为**无符号 below 恰好等价于单个标志 CF=1**，而 `seta` 要求复合条件 $\sim CF \,\&\, \sim ZF$。注意两类"同名"事实不同：`setb` / `setc` **是同一条机器指令**（实测都编码为 `0f 92`，`cmovb` / `cmovc` 都是 `48`

`0f 42`），而 `seta`（`0f 97`）与 `setb` 是**两条不同指令**。这说明：**条件码本身不知道有符号无符号，语义由 `cmp` 的操作数顺序与后续 `setX` / `jX` 的选择共同决定**。

`above((uint32_t)-1, 1u)` 实测返回 1：`(uint32_t)-1` 的位模式是 `0xffffffff`，当作**无符号**看确实大于 1；而同一个 `-1` 若按有符号解释则小于 1。**同一个位模式，两种解释，相反结论**。

其二，有符号/无符号的分野在条件传送（`cmov`）上表现得更直接。实测 `long max_signed(long a,long b){return a>b?a:b;}` 与 `uint32_t max_unsigned(...)`：

```
max_signed:                     max_unsigned:
        cmpq    %rdi, %rsi              cmpl  %edi, %esi
        movq    %rdi, %rax              movl  %edi, %eax
        cmovge  %rsi, %rax              cmovae       %esi, %eax   # 即 cmovnb
        ret                              ret
```

`cmovge`（有符号）与 `cmovae` / `cmovnb`（无符号）来自**同一套条件码、不同的条件组合**：`ge` 判 $SF = OF$，`ae` 判 $CF = 0$。

【实测验证】

```
$ ./opdemo
gt(5,3)=1 ge(5,3)=1 lt(5,3)=0 le(5,3)=0 eq(5,3)=0 ne(5,3)=1
above(-1,1)=1  below(-1,1)=0  （注意实参被转成 uint32_t）
$ ./signedness
max_signed(-1,1)=1
max_unsigned(0xffffffff,1)=4294967295
```

3.3.5 示例五：不同宽度的加载与扩展（`movzbl` / `movsbq` / `movslq`）

代码 (C)：

```
/* extend.c —— 扩展指令对照 */
#include <stdio.h>

long zero_extend_byte(char *p)   { return (unsigned char)*p; }  /* movzbl  */
long sign_extend_byte(char *p)   { return *p; }                 /* movsbq  */
long sign_extend_short(short *p) { return *p; }                 /* movswq  */
long sign_extend_int(int *p)     { return *p; }                 /* movslq  */
unsigned long zero_extend_int(unsigned *p) { return *p; }       /* movl    */

int main(void)
{
    char  c = (char)0x80;
    int   i = -1;
    short s = (short)0x8000;
    printf("ze_byte=0x%lx se_byte=%ld se_short=%ld se_int=%ld ze_int=0x%lx\n",
           (unsigned long)zero_extend_byte(&c), sign_extend_byte(&c),
           sign_extend_short(&s), sign_extend_int(&i),
           zero_extend_int((unsigned *)&i));
    return 0;
}
```

【与汇编 / 硬件的对应】 五个函数恰好覆盖了全部扩展路径：

```
zero_extend_byte:                   # (unsigned char)*p  -> 零扩展
        movzbl  (%rdi), %eax
        ret
sign_extend_byte:                   # *p (char 为 signed)  -> 符号扩展
        movsbq  (%rdi), %rax
        ret
sign_extend_short:                  # *p (short)  -> 符号扩展
        movswq  (%rdi), %rax
        ret
sign_extend_int:                    # *p (int)  -> 符号扩展
        movslq  (%rdi), %rax
        ret
zero_extend_int:                    # 无符号 int -> 高位自动清零
        movl    (%rdi), %eax
        ret
```

【底层机制透视】 最后一条最巧妙：`zero_extend_int` 没有用任何 `movz`，只写了一条 `movl (%rdi), %eax`。原因是 3.2.3 节那条规则——**写 32 位子寄存器 `%eax` 会自动把 `%rax` 的高 32 位清零**。所以"无符号 32 位零扩展为 64 位"是免费的。相反，`int → long` 需要真正的符号扩展（`movslq` 或

`cltq`），因为目标的高 32 位必须复制源的第 31 位。

【实测验证】 `./extend` 输出 `ze_byte=0x80 se_byte=-128 se_short=-32768 se_int=-1 ze_int=0xffffffff`。`0x80` 零扩展得 `128`，符号扩展得 `-128`；`0x8000` 符号扩展得 `-32768`；`-1` 的位模式 `0xffffffff` 零扩展后按无符号看即 4294967295。**同一段位、不同扩展方式，数值完全不同**——这是 CS:APP 第 2 章"位级表示与类型解释"在机器级的延续。

3.4 实验关联

L0 C Programming 是本讲的前置：你已经会写 C、会编译、会读错误信息，本讲把工具链推进到 `-S` 与 `objdump`。建议在 L0 阶段就养成用 `gcc -Og -S -fno-asynchronous-unwind-tables` 观察自己代码的习惯（该选项去掉 `.cfi_*` 伪指令，让输出更干净）。

L2 Bomb Lab 是本讲与后续两讲的直接应用。 标准拆弹工作流与本讲内容严格对应：

1. **读反汇编**：`objdump -d bomb > bomb.asm`。Bomb 用 `-Og` 编译，优化不激进，寄存器与源码变量基本一一对应——这正是课堂活动 `loops.o` 中"用 `-Og` 才找得到循环计数器 `i`"的提醒。
2. **识别关键指令**：本讲学到的 `cmpq` / `testq` / `movzbl` / `leaq` 每一关都会出现。`phase_1` 通常是 `callq strings_not_equal`；后续关卡出现 `cmp` + `setX` 组合——正是 3.3.4 节的模板，反过来读即可还原 C 里的比较式。
3. **GDB 单步与断点**（活动讲义重点，也适用于 Bomb）：

```
$ gdb ./bomb
(gdb) b phase_1            # 在函数入口下断点
(gdb) run                  # 运行到断点
(gdb) disassemble          # 看当前函数反汇编
(gdb) x/8xb $rdi           # 检查第 1 参数指向的 8 个字节
(gdb) info registers rdi rsi rax rip
(gdb) stepi                # 单条指令步进（汇编级）
(gdb) x/14xb sumstore      # 直接观察机器码字节（讲义演示）
```

1. **两个必踩的坑**：
2. ⛔ **不要在 Bomb Lab / Attack Lab 里用 `gdb` 的 `call` 命令**。活动讲义脚注明确警告："Do not do this in bomb lab or attack lab." 因为 `call` 会真正执行被调用函数，`explode_bomb` 一旦触发就扣分。
3. **不要中途退出**：服务端记录爆炸次数，每次爆炸扣 1/2 分（上限 6 次）。拆弹前先想清楚再运行。
4. **从 `cmp` 反推 C 表达式**的对照表（本讲的核心技能，也是 Bomb Lab 每一关都在做的事）：

```
+-------------------------+--------------------+
| 反汇编片段              | 等价 C 表达式      |
+-------------------------+--------------------+
| cmpq %rsi,%rdi / setg   | x > y    (有符号)  |
| cmpq %rsi,%rdi / setl   | x < y    (有符号)  |
| cmpl %esi,%edi / setb   | a < b    (无符号)  |
| testq %rdi,%rdi / sete  | x == 0             |
| testq %rdi,%rdi / js    | x < 0              |
| leaq (%rdi,%rdi,4),%rax | 5*x                |
| leaq 7(%rax,%rdi),%rax  | 5*y + 7 + x        |
+-------------------------+--------------------+
```

3.5 常见错误与调试技巧

- **把 AT&T 操作数顺序读反**：`movq %rax,(%rbx)` 是"把 `%rax` 存进 `%rbx` 指向的内存"。**调试**：用 `gcc -Og -S`（AT&T，`src, dst`）与 `objdump -d -M intel`（Intel，`dst, src`）对照着看。口诀"**括号里是地址，逗号左边是源**"。
- **以为 `leaq` 会访问内存**：`leaq (%rdi), %rax` 只是把 `%rdi` 的值搬到 `%rax`，**一次访存都没有**。**调试**：凡对应 `&x[i]`、`a + k*b` 的地方编译器就用 `leaq`；若真在访存，必然跟着 `movX`。
- **`setX` 后忘记清零高位**：`setg %al` 只改 1 字节，高 56 位是脏数据。**调试**：`gdb` 中 `p/x $rax` 看完整 64 位；机器生成的代码里必紧跟 `movzbl %al, %eax`，若缺失即为 bug。
- **记错 `cmp` 的操作数方向**：`cmpl %esi, %edi` 算的是 `%edi - %esi`。搞反会使条件跳转方向全错（Bomb Lab 的经典死因）。**调试**：`gdb` 里停在 `cmp` 后的 `jX` 上，`info registers eflags` 看 CF/ZF/SF/OF。EFLAGS 位序：CF=0、PF=2、ZF=6、SF=7、OF=11。
- **用错有符号/无符号的条件助记符**：把 `jl` 当 `jb` 用。`-1 < 1`（有符号）但 `0xffffffff > 1`（无符号），结论相反。**调试**：回到 C 源码确认类型；`gdb` 里 `p (int)x` 与 `p (unsigned)x` 各看一遍。
- **试图一条 `mov` 完成内存到内存 / 混淆 `movl` 的双重含义**：前者 `as` 会报 `Error: too many memory references for 'movq'`，须拆成两条经寄存器中转；后者在操作整数时是 32 位传送、操作浮点时却是 64 位传送。**调试**：`objdump -d -M intel` 会明确显示 `DWORD PTR` 或 `QWORD PTR`，`PTR` 前的尺寸最权威。
- **误以为 `-Og` 下汇编与源码一一对应**：`-Og` 保留调试友好性，但仍会做强度削减与寄存器复用（见 `arith` 中 `%rdx` 先存 `z` 后存 `t4`）。**调试**：用 `gcc -Og -g` 编译，`gdb` 中 `disassemble /m` 可把源码行与汇编交错显示。

3.6 关键要点

1. **机器级程序的状态只有五类**：`%rip`、16 个 64 位整数寄存器、条件码 CF/ZF/SF/OF、向量寄存器、按字节寻址的内存——掌握它们的读写就掌握了全部"名词"。
2. **`leaq` 不是加载指令，而是"不访存的地址算术器"**。它同时承担"取地址"（`p = &x[i]`）与"常量乘加"（$x + k \cdot y$，$k \in \{1, 2, 4, 8\}$）两种角色，是编译器强度削减的主力。
3. **`mov` / `leaq` 不改条件码，其余算术逻辑指令都改**；`cmp` 与 `test` 是"只设置条件码、不写目的"的纯提问指令，`cmp a,b` 算 $b - a$，`test a,b` 算 $b \& a$。
4. **AT&T 语法是课程与 `gcc -S` 的口径**：操作数 `src, dst`、寄存器加 `%`、立即数加 `$`、宽度后缀 `b/w/l/q`（浮点用 `s/l/ss/sd`）。读反操作数顺序是最高频的错误来源。
5. **有符号与无符号共享同一套位运算，只靠条件码组合区分**：`setg` / `jl` 判 $SF \oplus OF$，`setb` / `jb` 判 CF。同一条 `cmp` 后跟不同的 `setX`，得到的 C 语义可能完全相反。
6. **写 32 位寄存器会清零高 32 位，写 8/16 位不会**。这条 x86-64 的硬件规定让"无符号 int → long"零扩展变成零开销（一条 `movl`），也是 `movzbl %al, %eax` 这一固定收尾动作存在的理由。

3.7 思考题（带答案）

题 1（汇编推演题） 下面这段汇编是一个 `long` 类型的单参数函数（参数在 `%rdi`，返回值在 `%rax`），请写出等价的 C 代码，并说明 `gcc` 用了哪些优化：

```
mystery:
        leaq    (%rdi,%rdi,4), %rax   #
        leaq    (%rax,%rax,2), %rdx   #
        leaq    1(%rdi,%rdx), %rax    #
        ret
```

答：逐步代入。设 `x = %rdi`： 第一步 `%rax = x + 4x = 5x`； 第二步 `%rdx = %rax + 2*%rax = 3 * 5x = 15x`； 第三步 `%rax = 1 + x + 15x = 16x + 1`。 所以等价 C 是 `long mystery(long x) { return 16*x + 1; }`。可验证：`x = 1` 得 17。 优化手法有两条：(1) **强度削减**——$16x$ 本可用 `salq $4`，但编译器把它与 `+1` 合并进一条 `leaq` 的位移字段，一条指令完成"乘加"，省掉一次独立加法；(2) **`leaq` 当加法器用**——三条 `leaq` 无一访存，全部只做算术。这体现了 `leaq` 的三操作数能力（基址 + 变址×比例 + 位移）在常系数线性表达式上的威力。

题 2（计算/推演题） 设 `%rdx = 0xf000`、`%rcx = 0x0100`，请计算下列寻址表达式的有效地址（十六进制）：`0x8(%rdx)`、`(%rdx,%rcx)`、`(%rdx,%rcx,4)`、`0x80(,%rdx,2)`、`0x10(%rdx,%rcx,8)`。

答：按 $Reg[rb] + S \cdot Reg[ri] + D$ 逐个代入：

表达式	计算	结果
`0x8(%rdx)`	`0xf000 + 0x8`	`0xf008`
`(%rdx,%rcx)`	`0xf000 + 1*0x100`	`0xf100`
`(%rdx,%rcx,4)`	`0xf000 + 4*0x100`	`0xf400`
`0x80(,%rdx,2)`	`2*0xf000 + 0x80`	`0x1e080`
`0x10(%rdx,%rcx,8)`	`0xf000 + 8*0x100 + 0x10`	`0xf810`

注意最后一题的比例因子 8 与位移 0x10 是相加而非相乘，且**三项缺省即单位元**（省 `D` 是 `+0`，省 `S` 是 `*1`）。前四行与讲义 `Address Computation Examples` 完全一致。

题 3（"直观但错误的想法"） 有同学说："`leaq (%rdi,%rsi,4), %rax` 就是 `%rax = %rdi + 4 * *(%rsi)`，所以 `leaq` 会解引用 `%rsi`。" 这个想法错在哪？

答：错在把"寻址模式里的寄存器值"当成了"指针解引用"。寻址模式里的 `Rb` / `Ri` 只是**寄存器中的整数值**，`leaq` 把它们代入表达式算出一个数，**整个过程不访问内存**。正确读法是 `%rax = Reg[rdi] + 4 * Reg[rsi]`（纯值运算）。要发生解引用，必须让该内存操作数出现在**会访存的指令**里——例如 `movq (%rdi,%rsi,4), %rax` 才会真正读 `Mem[Reg[rdi] + 4*Reg[rsi]]`。区分方法：**看指令名**—— `lea` 只算不算，`mov` / `add` / `imul` / `cmp` 带括号操作数时才访存。这也是 3.3.3 节 `addr_of`（只有 `leaq`，返回地址）与 `get2d`（`leaq` 之后跟 `movl`，真的取值）的差别。

题 4（推演题） 下面两个函数编译出的汇编只有一个字节不同（`setg` vs `setb`、`cmovge` vs `cmovae`），请解释为什么 `fail_signed` 与 `fail_unsigned` 在输入 `(-1, 1)` 时给出相反的结果；并说明如果把 `fail_unsigned` 改成 `int` 参数会怎样。

```
int fail_signed(long a, long b)        { return a > b; }
int fail_unsigned(unsigned a, unsigned b){ return a > b; }
```

答：`-1` 作为 64 位 `long` 的位模式是 `0xffff...ffff`，作为 32 位 `unsigned` 的位模式是 `0xffffffff`。 – `fail_signed(-1, 1)`：`cmp` 置 `SF=1`、`OF=0`，故 $SF \oplus OF = 1$，`setg` 判 $\sim(SF \oplus OF) \& \sim ZF = 0$，返回 0。 – `fail_unsigned(0xffffffffu, 1u)`：`0xffffffff - 1` 需借位，故 `CF=1`，`setb` 判定为"above"，返回 1。 同一段位模式、同一套条件码，**结论相反**，差别完全来自 C 类型赋予它的解释（这正是实测 `above(-1,1)=1` 的原因）。若把参数改为 `int`，`gcc` 会改用 `setg`，`(-1, 1)` 下返回 0，与 `fail_signed` 一致。

Lecture 4: 机器级编程 II——控制流 (Machine Programming II: Control Flow)

> **讲义对应**：CMU 15–213 Lecture 4 — Machine Programming (Part II)（素材：`F25-04-machine-beta.txt`） **教材对应**：CS:APP3e 第 3 章 3.6–3.7（控制流、过程基础） **关联 Lab**：**L2 Bomb Lab**（Defusing a Binary Bomb）

4.1 概述

上一讲（Lecture 3）建立了"寄存器是名词、指令是动词"的机器级世界观：数据在寄存器与内存之间搬运，地址由 `D(Rb,Ri,S)` 寻址模式计算。但程序不只是直线执行——它要**判断**、要**重复**、要**跳转**。本讲回答一个核心问题：**C 语言里的 `if`、`while`、`for`、`switch`，究竟是如何只用"条件码 + 跳转"这两样东西实现的?**

答案是：CPU 内部有一组隐式的**条件码（condition codes）**，算术/逻辑指令运行后顺手把结果的性质（是否为零、是否有进位、是否溢出、符号位）记录在一个特殊寄存器 `RFLAGS` 里；`cmp` 与 `test` 专门负责只设置条件码而不保存结果；`set` 指令把条件码组合翻译成 0/1 字节；`jX` 指令依据同一组条件码决定下一条指令的地址（修改 `%rip`）。**一切控制流都是 goto**——这是理解本讲的第一把钥匙。

讲义在本讲还顺带展开了过程（procedures）的动机与 x86-64 栈的 push/pop 语义，为 Lecture 5 的完整调用约定做铺垫；本笔记重点覆盖 3.6–3.7 的控制流部分，栈帧细节详见 Lecture 5。

4.2 核心概念与底层机制图解

4.2.1 处理器状态与条件码（Condition Codes）

- **定义与目的**：条件码是 4 个单比特状态位，位于 `RFLAGS` 寄存器中，记录**最近一次算术或逻辑运算结果的性质**。它们是"硬件给程序员留下的免费副产品"—— `addq` 除了算和，还顺手告诉你结果是不是 0。
- **直观解释**：把它想成考场里的自动判卷灯。每做完一道运算题，墙上四盏灯自动亮/灭：`ZF` 亮表示"答案是零"，`SF` 亮表示"答案是负数"，`CF` 亮表示"无符号运算越界了"，`OF` 亮表示"有符号运算越界了"。你不需要重新看答案，只要抬头看灯。
- **底层机制图解**：

```
     RFLAGS 低 16 位的布局（本讲只关心 OF/SF/ZF/CF 四位）
bit: 15 14 13 12 11 10  9  8  7  6  5  4  3  2  1  0
    +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
    | 0| 0| 0| 0|OF| 0| 0| 0|SF|ZF| 0|AF| 0|PF| 0|CF|
    +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
                 bit11    bit7 bit6    bit4    bit2  bit0

名字    英文全称                 含义（以 a - b 的结果 t 为例）
----    ---------------------    ------------------------------------------------
CF    Carry Flag               t 在"无符号"意义下发生进位/借位（最高位向外进 1）
ZF    Zero Flag                t == 0（常用 testq %rax,%rax 单独探测）
SF    Sign Flag                t 的最高位为 1，即"被解释为有符号数是负数"
OF    Overflow Flag            t 在有符号意义下溢出（两正得负、或两负得正）

设置者：add/sub/imul/and/or/xor/shl/shr/cmp/test 等（大部分算术逻辑指令）
不设置者：mov 系列、lea —— 讲义特别强调 "lea 不碰条件码"，
          这正是编译器爱用 lea 做纯算术的原因：不会破坏已有的比较结果。
```

- **与机器码/硬件的对应**：条件码由 ALU 直接产生（加减法器的进位链输出、结果的全零检测、符号位复制），在同一个时钟周期内写回标志寄存器，**不额外消耗指令**。这正是 `cmp` 高效的原因：它只做减法、不写回目的寄存器。
- **补充说明**：讲义本讲只有一页"Conditional reminder slide"，上述 FLAGS 位编号与"mov/lea 不设条件码"的细节依据 Lecture 3 幻灯片与课堂活动 `machine-control.txt` 补齐。

4.2.2 `cmp` 与 `test`

- **定义与目的**：为比较而生的两条"只设标志、不留结果"的指令。
- `cmpq b, a`：计算 `a - b`（AT&T 顺序是**源在前、目的在后**，所以 `cmpq %rsi, %rdi` 算的是 `%rdi - %rsi`），**只设条件码**，`a` 与 `b` 都不改变。
- `testq b, a`：计算 `a & b`，**只设条件码**，且只对 `SF` / `ZF` 有意义的解释成立（`CF` / `OF` 被清零）。
- 最常见用法 `testq %rax, %rax`：同一个寄存器与自己与，只影响 `ZF` / `SF`，等价于"`%rax` 是零吗？是负数吗？"，比 `cmpq $0, %rax` 编码更短。

- 次常见用法 `testq %rX, %rY`：探测两个寄存器的 1 位是否有交集（如位图中的"权限掩码"判断）。
- **与机器码/硬件的对应**：`cmp` 与 `sub`、`test` 与 `and` 共享同一条数据通路，区别仅在于是否写回目的操作数。

4.2.3 `set` 指令族：把条件码变成 0/1

- **定义与目的**：`setX` 依据条件码把目的操作数的**最低字节**写成 0 或 1，其余 7 字节不变。它让关系表达式（如 `x > y`）可以作为**值**参与运算，而不必产生分支。
- **直观解释**：如果说 `jX` 是"看灯决定走哪条路"，`setX` 就是"看灯在纸上写 0 或 1"。二者读的是同一组灯。
- **关键陷阱——有符号用 `g/l`，无符号用 `a/b`**：

```
+--------+-----------------------+---------------+-----------------+----------+
| C 关系 | 条件码（比较 a-b 后） | set 指令      | 条件跳转        | 类别     |
+--------+-----------------------+---------------+-----------------+----------+
| a == b | ZF                    | sete / setz   | je / jz         |          |
| a != b | ~ZF                   | setne / setnz | jne / jnz       |          |
| a < 0  | SF                    | sets          | js              |          |
| a >= 0 | ~SF                   | setns         | jns             |          |
+--------+-----------------------+---------------+-----------------+----------+
| a > b  | ~(SF^OF) & ~ZF        | setg / setnle | jg / jnle       | signed   |
| a >= b | ~(SF^OF)              | setge / setnl | jge / jnl       | 有符号   |
| a < b  | SF^OF                 | setl / setnge | jl / jnge       | g / l    |
| a <= b | (SF^OF) OR ZF         | setle / setng | jle / jng       |          |
+--------+-----------------------+---------------+-----------------+----------+
| a > b  | ~CF & ~ZF             | seta / setnbe | ja / jnbe       | unsigned |
| a >= b | ~CF                   | setae / setnb | jae / jnb (jnc) | 无符号   |
| a < b  | CF                    | setb / setnae | jb / jnae (jc)  | a / b    |
| a <= b | CF OR ZF              | setbe / setna | jbe / jna       |          |
+--------+-----------------------+---------------+-----------------+----------+

记忆法："signed 用 g(reater)/l(ess)" —— g 与 l 字母形似 9 与 1，都是"带正负号的数字"；
        "unsigned 用 a(bove)/b(elow)" —— 只谈在数轴上位次高低，不谈正负。
```

- **与机器码/硬件的对应**：`setX %al` 只写 1 字节，因此编译器几乎总要补一条 `movzbl %al, %eax` 把高位清零，才能得到一个合法的 `int` 返回值。若忘记清零，`%rax` 高位残留的垃圾会污染返回值——这是 Bomb Lab 中常见的"看起来对却返回值不对"的误读来源。

4.2.4 跳转指令：直接跳转与间接跳转

- **直接跳转（direct jump）**：目的地在指令里写成**标号**，汇编器/链接器算出相对偏移。如 `jmp .L8`、`je .L4`。机器码里通常是 1–4 字节的**相对位移**（相对于下一条指令），这样代码可以整体重定位。
- **间接跳转（indirect jump）**：目的地从**寄存器或内存**中读出，语法上以 `*` 标记：
- `jmp *%rax` —— 跳到 `%rax` 里的地址；
- `jmp *(%rax)` —— 从内存读出一个 8 字节地址再跳（函数指针调用就是这样）；
- `jmp *.L4(,%rdi,8)` —— 跳到跳转表 `.L4` 的第 `%rdi` 项（`switch` 的实现）。
- **与硬件的对应**：条件跳转的"跳或不跳"在流水线里由**分支预测器（branch predictor）**提前猜测；猜错就要冲刷流水线（branch misprediction），代价通常十几到二十个周期。这直接引出下一节的 `cmov`。

4.2.5 if-else 的编译：把条件取反，先跳到 else

- **定义与目的**：C 的 `if (Test) Then else Else` 翻译成汇编时，编译器通常**不**按源码顺序生成，而是遵循一条被 CS:APP 称为"教科书式"的模板：

```
C 源码                      翻译后的控制流（条件取反，先跳 else）
-------------------------   ------------------------------------------
if (Test)                   <求值 Test，设置条件码>
    Then                    j<Test 的反> <else>     ; 条件为假才跳走
else                        <Then 的代码>
    Else                    jmp <done>              ; then 结束，跳过 else
                            <else>:
                            <Else 的代码>
                            <done>:
```

- **为什么要取反**：`Test` 为真时**不跳转**，可以顺着流水线继续执行 then 分支——这既保持了指令的物理顺序贴近源码顺序，又避免了额外的无条件跳转。若条件为假，一次跳转直接到 else。
- **为什么 then 分支末尾要多一条 `jmp done`**：否则 then 的代码会"贯穿"进 else。**`switch` 的 fall-through 是语言特性，`if-else` 的贯穿则是 bug**，编译器必须显式阻断。
- **直观解释**：像地铁闸机。真条件就"直接往前走"，假条件才"拐到另一条道"；两条道走完后都要汇合到 `done`。

- **与机器码/硬件的对应**：`j<Test 的反>` 是前一节条件码组合表的直接应用。若 `Test` 是 `x < y`（有符号），取反就是 `x >= y`，即 `jge`；若 `Test` 是 `x == 0`，取反就是 `jne`。乱序、嵌套、`else if` 链都只是这个模板的递归套用。

4.2.6 条件传送 `cmov`：用数据流换掉控制流

- **定义与目的**：`cmovX Src, Dest` 在条件成立时才把 `Src` 拷进 `Dest`，否则保持不变。与 `jX` 读同一组条件码，但**不改变 `%rip`**——没有分支，也就没有预测失败。
- **直观解释**：分支像路口问路（问错要掉头重走）；`cmov` 像提前把两条路的结果都拿到手里，再挑对的那份。代价是：**两条路的结果都要先算出来**。
- **两条铁律**（本讲最容易被忽视的考点）：
- `cmov` 的两个候选值**都会被执行求值**，因此"先算再选"必须是无副作用、无非法访存的；
- 如果某个候选值的计算本身可能崩溃（例如解引用可能为 `NULL` 的指针 `*p`），**编译器绝不能生成 `cmov`**，只能保留真实分支。这正是 CS:APP 经典例子 `xp ? *xp : 0` 不能编译成 `cmov` 的原因。
- **为什么 GCC 在 `-O1` 以上偏爱 `cmov`**：可预测性差的随机分支会不断预测失败，而 `cmov` 把控制相关（control dependence）转成数据相关（data dependence），执行时间恒定。代价是两条路径的计算都进关键路径，因此当分支高度可预测（如循环回边）或某条路径很昂贵时，分支反而更快。
- **与机器码/硬件的对应**：`cmovX` 后缀与 `setX` / `jX` 共用同一张条件码表——`cmove`、`cmovne`、`cmovs`、`cmovg` / `cmovl`（有符号）、`cmova` / `cmovb`（无符号）等。硬件上就是给寄存器写口加一个条件使能位，**不产生任何分支**。

```
   分支版本（-Og：真分支）            cmov 版本（-O1：无分支）
+-----------------------------+    +-----------------------------+
| cmpq %rsi,%rdi              |    | movq %rsi,%rdx              |
| jge .L2   <- 可能预测失败   |    | subq %rdi,%rdx  ; y-x       |
| movq %rsi,%rax              |    | movq %rdi,%rax              |
| subq %rdi,%rax  ; y-x       |    | subq %rsi,%rax  ; x-y       |
| ret                         |    | cmpq %rsi,%rdi              |
| .L2:                        |    | cmovl %rdx,%rax ; 条件选择  |
|   movq %rdi,%rax            |    | ret                         |
|   subq %rsi,%rax ; x-y      |    +-----------------------------+
|   ret                       |
+-----------------------------+
```

4.2.7 循环的三种翻译与 guarded-do

循环的四个组成部分是**初始化（Init）、测试（Test）、更新（Update）、循环体（Body）**。讨论循环的 goto 形态时只需问一句：**回跳（backward goto）在哪里？**

- **`do-while`**：最自然的形态，循环体在前、测试在后，天然就是"先执行一次再回跳"，无需额外保护。
- **`while` 的两种翻译**：
- **跳转到中间（jump to middle）**：`goto test;` 开头，体后是 `test: if (Test) goto loop;`。`-Og` 采用，因为它**逐字对应源码**，便于调试。
- **guarded-do（转成 do-while）**：先 `if (!Test) goto done;` 做一次入口守卫，然后把体与测试拼成 do-while 的好处是**把两个跳转（一个是入口跳转、一个是删除冗余跳转）压缩成一个底部条件跳转**，`-O1` 采用。
- **`for`**：先机械改写为 `Init; while (Test) { Body; Update; }`，再套用上面的 while 翻译。`guarded-do` 优化之所以安全，是因为 `for` 的初值测试在编译期常常可判定（如 `i=0; i<64` 恒真），入口守卫可以被直接删掉，于是 `for` 得到一个**纯 do-while 结构**。

4.2.8 `switch` 的编译与跳转表

`switch` 的编译有两套策略，取决于 case 值是否**稠密**：

1. **跳转表（jump table）**：case 值落在稠密区间 `[0, n-1]` 时，编译器在 `.rodata` 段生成一张函数指针数组，用 `jmp *.L4(,%rdi,8)` 一次间接跳转完成分派——$O(1)$，无比较链。
2. **比较链 / 判定树（if-else chain）**：case 稀疏时退化为若干次 `cmp` + 条件跳转。现代 GCC 还会对"连续小区间"用**先减后无符号比较**（`subq $5,%rdi; cmpq $1,%rdi; ja`）一次判掉 `x==5 || x==6`。

```
跳转表在内存中的布局（本笔记实测：gcc 12.2 -Og，grade() 函数）

.text: grade 函数（编译器生成）
+-----------------------------------+
| cmpq $0x7,%rdi    ; 上界检查      |
| ja   .L2          ; x>7 走默认    |
| jmpq *0x402008(,%rdi,8)           |
+-----------------------------------+
           |                            .rodata: 跳转表（基址 .L4 = 0x402008）
           |                            +--------------------------------------+
           |  地址 = 表基址 .L4 + x*8   | 表项地址     目标标签          返回值  |
           |                            +--------------------------------------+
           +------------------->        | 0x402008    .L11 (x=0)           10  |
                                        | 0x402010    .L12 (x=1)           20  |
                                        | 0x402018    .L9  (x=2)           30  |
                                        | 0x402020    .L8  (x=3)           40  |
                                        | 0x402028    .L7  (x=4)           50  |
                                        | 0x402030    .L6  (x=5)           60  |
scale = 8：每个表项是一个               | 0x402038    .L5  (x=6)           70  |
64 位代码地址，正好 8 字节。            | 0x402040    .L3  (x=7)           80  |
                                        +--------------------------------------+

为什么放 .rodata？表在运行期只读，
既防改写，也便于多进程共享物理页。
```

- **上界检查是必需的**：`jmp *.L4(,%rdi,8)` 不做边界检查，负数或超界索引会读到表外内存并跳到任意地址。因此编译器一定先 `cmpq $n-1,%rdi; ja default` ——注意这里用 **无符号** `ja`，因为无符号比较把负索引也归入"大于"，一次比较同时挡掉负数与超界。
- **补充说明**：讲义提到"较新的编译器会把跳转表从 8 字节地址改为 4 字节**偏移**"（`rip + 表基址 + 偏移`）；本笔记用 `gcc -Og -fPIC` 复现了该形态（4.3.3）。

4.3 代码示例与底层机制分析

4.3.1 示例一：关系表达式、`set` 与 if-else 分派

```c
/* setdemo.c —— 用 gcc -Og -Wall -std=c11 -g -o setdemo setdemo.c 验证通过 */
#include <stdio.h>

int gt(long x, long y)                      { return x > y;  }  /* 有符号 */
int lt_u(unsigned long x, unsigned long y)  { return x < y;  }  /* 无符号 */

long classify(long x) {                     /* if-else 的教科书形态 */
    if (x < 0) return -1;
    else       return 1;
}

int main(void) {
    printf("%d %d | %d %d | %ld %ld\n",
           gt(5, 3), gt(3, 5), lt_u(1, 0), lt_u(0, 1),
           classify(-3), classify(3));
    return 0;   /* 输出：1 0 | 0 1 | -1 1 */
}
```

【代码做什么？】 `gt` 用有符号比较判断 `x > y`，返回 int 0/1；`lt_u` 语义相同形状但参数是 `unsigned long`，走无符号比较；`classify` 是课本最简 if-else。

【底层机制透视】 `gt` 与 `lt_u` 都不产生分支：比较指令只设条件码，`setX` 把条件码抄成 1 字节，`movzbl` 补零成完整的 `int`。`classify` 则相反——两条臂都只是常量，编译器选择"跳转"而不是"求值+选择"，因为常量不需要计算，跳转更省。

【与汇编 / 硬件的对应】（真实 `gcc -Og -S` 输出）

```
gt:
        cmpq    %rsi, %rdi        # 算 x - y，只设条件码
        setg    %al               # 有符号 >  → %al = 0/1
        movzbl  %al, %eax         # 把低字节零扩展成 32 位 int
        ret

lt_u:
        cmpq    %rsi, %rdi
        setb    %al               # 无符号 <  → 注意是 b 不是 l
        movzbl  %al, %eax
        ret

classify:
        testq   %rdi, %rdi        # x 是零吗？是负数吗？
        js      .L6               # SF=1（负数）就跳到 else 分支
        movl    $1, %eax          # then 分支：先做
        ret
.L6:
        movq    $-1, %rax         # else 分支
        ret
```

再看"两条臂都要汇合"的形态（真实 `gcc -Og -S` 输出）。把 `if-else` 的结果赋给变量、之后再统一处理，就出现了 4.2.5 节那个 `done:` 汇合点：

```
/* ifelse.c —— gcc -Og -Wall -std=c11 -g -o ifelse ifelse.c 验证通过，输出 "601 -6 | 602 -5" */
#include <stdio.h>

__attribute__((noinline)) long slow_then(long x) { return x * 3 + 1; }
__attribute__((noinline)) long slow_else(long x) { return x / 2 - 7; }

long pick(long x) {           /* 尾调用版：return 直接就是函数调用 */
    if (x > 100) return slow_then(x);
    else         return slow_else(x);
}

long pick2(long x) {          /* 非尾调用：两条臂汇合到同一出口 */
    long r;
    if (x > 100) r = slow_then(x);   /* Test: x > 100 */
    else         r = slow_else(x);
    return r + 1;                    /* <done> 汇合点 */
}

int main(void) {
    printf("%ld %ld | %ld %ld\n", pick(200), pick(2), pick2(200), pick2(2));
    return 0;
}
```

```
# -------- 尾调用版 pick：两条臂各自 ret，没有 done 汇合点 --------
pick:
        cmpq    $100, %rdi
        jle     .L4                  # 条件取反：x <= 100 跳到 else（4.2.5 的模板）
        call    slow_then            # then 分支：真条件顺行，不再跳转
        ret
.L4:                                 # <else>:
        call    slow_else
        ret

# -------- 非尾调用版 pick2：必须保留 jmp 才能汇合到 done --------
pick2:
        cmpq    $100, %rdi
        jle     .L7
        call    slow_then
.L8:                                 # <done>:
        addq    $1, %rax
        ret
.L7:                                 # <else>:
        call    slow_else
        jmp     .L8                  # else 结束后跳向汇合点
```

对比两段真实产物可以看出：尾调用版里 `then` 与 `else` 各自 `ret`，**没有共同的 `done` 标签**；而非尾调用版多出一条 `jmp .L8`——它存在的唯一理由就是阻断 `else` 代码的贯穿，并在 `done` 处汇合。这正是 4.2.5 节模板中那条 `jmp <done>` 的实证。

【实测验证】 把 `int gt` / `int lt_u` 换成 `int c_s(long a,long b){return a<0;}`，`gcc -Og` 并不用 `sets`，而是 `shrq $63,%rdi; movq %rdi,%rax` ——这提示"条件码→`set`→零扩展"只是**一种**标准形态，优化器会做等价改写；读懂语义比背指令序列更重要。

4.3.2 示例二：三种循环与 guarded-do

```
/* loops.c —— gcc -Og / -O1 -Wall -std=c11 -g 验证通过，输出 "0 8 16" */
#include <stdio.h>

long pcount_do(unsigned long x) {           /* do-while：体在前，测试在后 */
    long result = 0;
    do { result += x & 0x1; x >>= 1; } while (x);
    return result;
}

long pcount_while(unsigned long x) {        /* while */
    long result = 0;
    while (x) { result += x & 0x1; x >>= 1; }
    return result;
}

#define WSIZE 64
long pcount_for(unsigned long x) {          /* for：先改写成 while */
    long result = 0;
    for (unsigned i = 0; i < WSIZE; i++) result += (x >> i) & 0x1;
    return result;
}

int main(void) {
    printf("%ld %ld %ld\n",
           pcount_do(0), pcount_while(0xff), pcount_for(0x0f0f0f0fUL));
    return 0;
}
```

【代码做什么？】 三个函数都统计参数中 1 的个数（popcount），差别只在循环语法：`do-while` 至少执行一次；`while` 可能零次；`for` 把初始化/测试/更新三段写在一行。

【底层机制透视】 `pcount_do` 的汇编就是"体 + 底部条件回跳"，`shrq %rdi` 之后直接用 `jne` ——因为 `shrq` 顺带设了 `ZF`，**测试与移位合并成一条指令**。`pcount_while` 在 `-Og` 下用"跳转到中间"，开头补一条 `movl $0,%eax; jmp .L4`。

【与汇编 / 硬件的对应】 `gcc -Og` 与 `gcc -O1` 的真实对比：

```
# ---------- gcc -Og：跳转到中间（jump to middle）----------
pcount_while:
        movl    $0, %eax
        jmp     .L4                   # 先跳到测试，再决定是否进入
.L5:
        movq    %rdi, %rdx
        andl    $1, %edx
        addq    %rdx, %rax
        shrq    %rdi
.L4:
        testq   %rdi, %rdi            # 测试
        jne     .L5                   # 回边
        ret

# ---------- gcc -O1：guarded-do（先守卫，再 do-while）----------
pcount_while:
        testq   %rdi, %rdi
        je      .L7                   # 入口守卫：x==0 直接返回 0
        movl    $0, %eax
.L6:
        movq    %rdi, %rdx
        andl    $1, %edx
        addq    %rdx, %rax
        shrq    %rdi
        jne     .L6                   # 底部回边，省掉顶部 test
        ret
.L7:
        movl    $0, %eax
        ret

# ---------- gcc -Og：pcount_for 的跳转到中间 ----------
pcount_for:
        movl    $0, %ecx
        movl    $0, %edx
        jmp     .L7
.L8:
        movq    %rdi, %rax
        shrq    %cl, %rax
        andl    $1, %eax
        addq    %rax, %rdx
        addl    $1, %ecx
.L7:
        cmpl    $63, %ecx             # i <= 63 即 i < 64
        jbe     .L8
        movq    %rdx, %rax
        ret

# ---------- gcc -O1：pcount_for 的 guarded-do ----------
pcount_for:
        movl    $0, %ecx
        movl    $0, %edx
.L10:
        movq    %rdi, %rax
        shrq    %cl, %rax
        andl    $1, %eax
        addq    %rdx, %rax
        movq    %rax, %rdx
        addl    $1, %ecx
        cmpl    $64, %ecx
        jne     .L10                  # 入口守卫被完全优化掉（0<64 恒真）
        ret
```

【实测验证】 `cmp` 的两个比较点值得注意：`-Og` 用 `cmpl $63,%ecx; jbe`，`-O1` 改写成 `cmpl $64,%ecx; jne`——优化器把"上界 63 的闭区间"翻成"上界 64 的开区间＋不等比较"，这是归纳变量优化的常见副产品。

4.3.3 示例三：`switch` 的跳转表与稀疏退化

```
/* sw.c —— gcc -Og -Wall -std=c11 -g 验证通过，循环打印 0..7 的结果 */
#include <stdio.h>

long grade(long x) {                    /* case 0..7 稠密 → 触发跳转表 */
    switch (x) {
    case 0: return 10;  case 1: return 20;  case 2: return 30;  case 3: return 40;
    case 4: return 50;  case 5: return 60;  case 6: return 70;  case 7: return 80;
    default: return -1;
    }
}

long switch_eg(long x, long y, long z) { /* 讲义原例：case 稀疏 → 比较链 */
    long w = 1;
    switch (x) {
    case 1: w = y * z; break;
    case 2: w = y / z;          /* Fall Through，故意不 break */
    case 3: w += z; break;
    case 5:
    case 6: w -= z; break;
    default: w = 2;
    }
    return w;
}

int main(void) {
    for (long i = -1; i <= 8; i++) printf("%ld -> %ld\n", i, grade(i));
    printf("switch_eg(2,12,4) = %ld\n", switch_eg(2, 12, 4));
    return 0;
}
```

【代码做什么？】 `grade` 是均匀分布的稠密 `switch`；`switch_eg` 是讲义原例，含**多重标签（5、6 共用一块）**、**贯穿（case 2 落入 case 3）**、**缺失标签（4 落 default）**三种形态。

【内存布局 / 数据结构图解】 `grade` 的真实跳转表位于只读段。`objdump -s -j .rodata` 实测输出（为便于对齐，下面每行只摘两个 8 字节表项，省略行尾的 ASCII 注释列与后续的 printf 格式串）：

```
 .rodata
 402000  01000200 00000000   33114000 00000000   <- 前 8 字节是 _IO_stdin_used;
                                                      0x402008 起才是表：.L11 = 0x401133
 402010  65114000 00000000   39114000 00000000   <- x=1: 0x401165   x=2: 0x401139
 402020  3f114000 00000000   45114000 00000000   <- x=3: 0x40113f   x=4: 0x401145
 402030  4b114000 00000000   51114000 00000000   <- x=5: 0x40114b   x=6: 0x401151
 402040  57114000 00000000                       <- x=7: 0x401157
```

小端序（little-endian）在这里帮了大忙：`33114000 00000000` 从低地址往高地址读出的 8 字节即地址 `0x0000000000401133`。所以表在内存里看起来"数字是反的"。

【与汇编 / 硬件的对应】（真实输出）

```
grade:
        cmpq    $7, %rdi            # 上界检查
        ja      .L2                 # 无符号 ja：负数与 >7 都被挡到 default
        jmp     *.L4(,%rdi,8)       # 间接跳转：地址 = .L4 + x*8
        .section        .rodata
        .align 8
.L4:
        .quad   .L11                # x = 0
        .quad   .L12                # x = 1
        .quad   .L9                 # x = 2
        .quad   .L8                 # x = 3
        .quad   .L7                 # x = 4
        .quad   .L6                 # x = 5
        .quad   .L5                 # x = 6
        .quad   .L3                 # x = 7
        .text
.L11:
        movl    $10, %eax
        ret
```

`switch_eg` 在 `-Og` 下退化为**判定树 + 区间技巧**（真实输出，节选）：

```
switch_eg:
        movq    %rdx, %rcx
        cmpq    $3, %rdi
        je      .L20                # x == 3 → w=1，再落入 merge
        jg      .L15                # x > 3 → 处理 5/6/default
        cmpq    $1, %rdi
        je      .L16                # case 1: imul
        cmpq    $2, %rdi
        jne     .L23                # default: w = 2
        movq    %rsi, %rax
        cqto                         # 把 %rax 符号扩展到 %rdx:%rax（idiv 的被除数）
        idivq   %rcx                # case 2: y/z
.L14:                              # merge:（case 2 贯穿进入 case 3）
        addq    %rcx, %rax          # w += z
        ret
.L23:
        movl    $2, %eax            # default
        ret
.L15:
        subq    $5, %rdi            # 把 5/6 平移到 0/1
        cmpq    $1, %rdi
        ja      .L24                # 一次无符号比较同时排除负数与 >1
        movl    $1, %eax
        subq    %rdx, %rax          # w = 1 − z
        ret
.L16:
        movq    %rdx, %rax
        imulq   %rsi, %rax          # w = z * y
        ret
.L20:
        movl    $1, %eax            # case 3: w = 1
        jmp     .L14
.L24:
        movl    $2, %eax            # default（从 .L15 的区间判断溢出而来）
        ret
```

【实测验证】 用 gdb 直接读表（真实会话）：

```
$ gdb ./sw
(gdb) x/8xg 0x402008
0x402008:  0x0000000000401133  0x0000000000401165
0x402018:  0x0000000000401139  0x000000000040113f
0x402028:  0x0000000000401145  0x000000000040114b
0x402038:  0x0000000000401151  0x0000000000401157
```

第 0 项 `0x401133` 正是 `grade+0xd`（`mov $0xa,%eax`，即 `return 10`），与源码 `case 0` 吻合。这说明**跳转表把"比较链"变成了"查表"**：无论 `x` 取 0 还是 7，都只有 `cmp` + `ja` + `jmp *` 三条指令，耗时与 case 数量无关（$O(1)$）。另外两处值得记录：

- **`-O2` 会把整张表删掉**：`grade` 被识别为线性函数，编译成 `leaq 5(%rdi,%rdi,4),%rax; addq %rax,%rax`（即 `10*x + 10`）。跳转表只是优化器的一种选择，不是语义要求。
- **`-Og -fPIC` 复现了讲义的"4 字节偏移表"**：`leaq .L4(%rip),%rdx; movslq (%rdx,%rdi,4),%rax; addq %rdx,%rax; jmp *%rax`，表项是 `.long .L11-.L4`，体积减半且位置无关。

4.3.4 示例四：`cmov` 与分支的真实差异（本讲核心对照）

```c
/* cmovdemo.c —— gcc -Og/-O1 -Wall -std=c11 -g 验证通过 */
#include <stdio.h>

long absdiff(long x, long y) {            /* 两条臂都是纯算术 → 可用 cmov */
    long result;
    if (x < y) result = y - x;
    else       result = x - y;
    return result;
}

long absval(long x) { return x < 0 ? -x : x; }

long cread(long *xp) { return xp ? *xp : 0; }   /* 不能 cmov：*xp 可能非法访存 */

long cread_alt(long *xp) {                /* 改写后两条臂都安全 → 可用 cmov */
    long zero = 0;
    if (!xp) xp = &zero;
    return *xp;
}

int main(void) {
    long v = 99;
    printf("%ld %ld | %ld %ld | %ld %ld %ld\n",
           absdiff(3, 7), absdiff(9, 7), absval(-5), absval(5),
           cread(&v), cread(NULL), cread_alt(NULL));
    return 0;   /* 输出：4 2 | 5 5 | 99 0 0 */
}
```

【代码做什么？】 `absdiff` 求差的绝对值；`absval` 求绝对值；`cread` 安全地读一个可能为 `NULL` 的指针；`cread_alt` 语义相同但把空指针改指向一个局部零变量。

【底层机制透视——两种编译产物的对比】（真实 `gcc -S` 输出）

```
# ===== absdiff @ -Og: 真分支 =====
absdiff:
        cmpq    %rsi, %rdi
        jge     .L2                 # 分支：可能预测失败
        movq    %rsi, %rax
        subq    %rdi, %rax          # y - x
        ret
.L2:
        movq    %rdi, %rax
        subq    %rsi, %rax          # x - y
        ret

# ===== absdiff @ -O1: cmov =====
absdiff:
        movq    %rsi, %rdx
        subq    %rdi, %rdx          # 先算 y - x
        movq    %rdi, %rax
        subq    %rsi, %rax          # 再算 x - y
        cmpq    %rsi, %rdi
        cmovl   %rdx, %rax          # 只有 x<y 时才用第一份结果
        ret

# ===== absval @ -Og/@-O1: cmov 的紧凑形态 =====
absval:
        movq    %rdi, %rax
        negq    %rax                # 先假设结果为 -x（negq 会设置 SF）
        cmovs   %rdi, %rax          # 若 -x 为负则取 x（这正是 x>0 的情形）
        ret

# ===== cread @ -O1: 编译器拒绝 cmov，保留分支 =====
cread:
        movl    $0, %eax
        testq   %rdi, %rdi
        je      .L5                 # 必须短路：xp 为 NULL 时不能解引用
        movq    (%rdi), %rax
.L5:
        ret

# ===== cread_alt @ -O1: 两臂都安全，于是生成 cmov =====
cread_alt:
        movq    $0, -8(%rsp)        # 栈上的 zero 变量
        leaq    -8(%rsp), %rax
        testq   %rdi, %rdi
        cmove   %rax, %rdi          # 若 xp==NULL，则改指向 &zero；无跳转
        movq    (%rdi), %rax
        ret
```

【内存布局 / 数据结构图解】 `cread_alt` 用栈上 8 字节（红色区域 red zone）换来一条无分支指令。注意 `call` 之后 `%rsp` 指向返回地址，`-8(%rsp)` 是它**下方**（更低地址）的 8 字节；`movq $0,-8(%rsp)` 只写内存，**不改变** `%rsp`：

```
   高地址                                    ← 调用者的栈帧
   +---------------------------+
   |  ... (caller frame)       |
   +---------------------------+
   |  返回地址（call 压入）     |  <- %rsp 指向这里，全程不变
   +---------------------------+
   |  zero = 0 (-8(%rsp))      |  <- 红区；被 leaq 取地址，
   +---------------------------+     xp==NULL 时 %rdi 改指向这里
   低地址
```

【与汇编 / 硬件的对应】 两个事实直接来自讲义与课堂活动 `machine-control.txt`：

1. `cmov` 的候选值**一定先被求值**——`absdiff @ -O1` 里 `y-x` 和 `x-y` 两条 `subq` 都执行了，只是结果二选一。
2. `cmov` 不能被用来"跳过"危险的内存访问——`cread` 若错误地编译成 `cmovq (%rdi),...`，传入 `NULL` 就会段错误，程序语义被破坏。**编译器必须保证：优化不得引入原程序不存在的异常。**

这也给出了写代码的建议：**想让 GCC 生成 `cmov`，就把两条臂都改写成"总是合法"的计算**（如 `cread_alt`）。

4.4 实验关联：L2 Bomb Lab

Bomb Lab 是"控制流 + gdb"的实战考场：每个 phase 都靠 `cmp` / `test` 与 `jX` 决定"继续还是引爆"，正是本讲内容的直接应用。

第一步：定位函数。 bomb 的符号没有被剥离，直接看符号表就能拿到所有 phase 地址（本仓库 `handouts/bomb/bomb/bomb` 实测）：

```
$ objdump -t bomb | grep -E "phase_|explode"
0000000000400ee0 g  F .text  000000000000001c  phase_1
0000000000400efc g  F .text  0000000000000047  phase_2
0000000000400f43 g  F .text  000000000000008b  phase_3
000000000040100c g  F .text  0000000000000056  phase_4
0000000000401062 g  F .text  0000000000000092  phase_5
00000000004010f4 g  F .text  0000000000000110  phase_6
000000000040143a g  F .text  0000000000000022  explode_bomb
00000000004015c4 g  F .text  0000000000000095  phase_defused
```

（上面按地址排序便于阅读。bomb 的函数名**没有被剥离**是这份 bomb 的幸运之处——`objdump -t` 一跑，要读哪几个函数就一目了然。）

第二步：读懂 phase_1 的"比较+跳转"骨架。

```
$ objdump -d bomb | sed -n '/<phase_1>:/,/ret/p'
  400ee0:  48 83 ec 08        sub    $0x8,%rsp
  400ee4:  be 00 24 40 00     mov    $0x402400,%esi      # 第二个参数：目标字符串
  400ee9:  e8 4a 04 00 00     callq  401338 <strings_not_equal>
  400eee:  85 c0              test   %eax,%eax           # 返回值是 0 吗?
  400ef0:  74 05              je     400ef7 <phase_1+0x17>
  400ef2:  e8 43 05 00 00     callq  40143a <explode_bomb> # 不等则引爆
  400ef7:  48 83 c4 08        add    $0x8,%rsp
  400efb:  c3                 retq
```

`test %eax,%eax` + `je` 是"函数返回 0 就安全通过"的标准写法，`%eax` 里放的是 `strings_not_equal` 的返回值。剩下的事只有一件：看 `0x402400` 这个立即数指向什么。讲义特别提醒过——**寄存器里的数字没有类型标签**，要靠上下文判断哪些是地址：`%rip` 与 `%rsp` 永远是指针；出现在 `mov (...)` 这种解引用位置的值必定被当作地址；而 `0x402400` 落在 `.rodata` 地址范围内（`info files` 可查段边界），因此几乎肯定是字符串首地址。用 `x/s 0x402400` 一读即得答案。

第三步：gdb 三件套（断点、寄存器、内存）。

```
$ gdb ./bomb
(gdb) break explode_bomb          # 保命断点：一旦要炸就停下，不丢分
Breakpoint 1 at 0x40143a
(gdb) break phase_1               # 每进入一个 phase 先停下
(gdb) info breakpoints            # 确认两个断点都 y(启用)
(gdb) run psol.txt                # 用命令行参数喂入已解出的 phase，避免重复输入
(gdb) x/s 0x402400                # 按字符串格式看内存
0x402400:  "Border relations with Canada have never been better."
(gdb) info registers rdi rsi rsp  # 看参数寄存器与栈顶
(gdb) disassemble phase_2         # 反汇编当前 phase（不必再开 objdump）
```

第四步：用控制流知识逆推 phase_2。 `phase_2` 的骨架是"先检查首元素、再沿数组循环比较"：`cmpl $0x1,(%rsp)` 配合 `je`，然后用 `%rbx` 作移动指针、`%rbp` 作终止边界，循环体内 `add %eax,%eax` 后 `cmp %eax,(%rbx)`——把 `-0x4(%rbx)`（上一个元素）翻倍后与当前元素比较。**能一眼看出 `%rbx` 是步长为 4 的整数指针、`%rbp` 是结束地址**，正是 Lecture 3 寻址模式与本讲条件跳转的直接回报。

常见坑：①在 bomb 里用 gdb 的 `call` 命令调用函数会**引爆**（课堂活动明确警告）；②只设 `break phase_1` 而忘设 `break explode_bomb`，错一次就扣 1/2 分；③用 `x/8xb` 看字符串读不出 ASCII——用 `x/s` 或对照 `man ascii`；④忘记 bomb 支持 `./bomb psol.txt`，反复手敲已解出的 phase。

4.5 常见错误与调试技巧

- **有符号/无符号跳转混用**：C 中写 `if (x < y)` 而 `x`、`y` 是 `unsigned`，编译器仍用 `setb` / `jb`；若手写汇编误用 `jl`，负数会被判成"小于"。**调试**：`objdump -d -M att` 核对 `j` 后缀，或在 gdb 中 `p (unsigned long)x` 对比 `p (long)x`。
- **`cmp` 操作数顺序反了**：AT&T 是"源, 目的"，`cmpq %rsi,%rdi` 判断的是 `%rdi` 与 `%rsi` 的关系。**调试**：`gdb -tui` 打开汇编/寄存器分窗，`stepi` 单步并在 `info registers eflags` 中直接看 ZF/SF/CF/OF。
- **`setX %al` 忘记零扩展**：返回值高 24 位是垃圾。**调试**：`p $rax` 与 `p $eax` 一起打印对比；正确代码应看到 `movzbl %al,%eax`。
- **误以为 `cmov` 能防止非法访存**：把 `p ? *p : d` 交给 `-O2` 后若被改写成无条件加载就会段错误（现代 GCC 不会这样做，但手写内联汇编极易犯错）。**调试**：`valgrind ./prog` 观察 Invalid read；或用 `perf stat -e branch-misses ./prog` 验证 `cmov` 化是否真的减少了预测失败。
- **`switch` 索引越界**：跳转表前必须有上界检查，否则 `jmp *.L4(,%rdi,8)` 会跳飞。**调试**：`objdump -d` 确认 `ja` 守卫存在；在 gdb 中 `x/8xg <表基址>` 核对表项是否落在 `.text` 范围内（`info files` 查看段边界）。
- **循环变量被优化掉导致断点失效**：`-O2` 下 `i` 可能只存在于寄存器甚至被删除。**调试**：调循环用 `-Og -g`；用 `break *0x401126` 按地址下断点，`x/i $rip` 确认位置。
- **Bomb Lab 中误用 `call`**：在 gdb 里直接调用炸弹函数会触发爆炸判定。**调试**：只读不调——`x/s`、`info registers`、`disassemble` 都是安全的。

- **把 `je` 与 `jz` 当成不同指令**：二者机器码完全相同，只是助记法不同。**调试**：`objdump -d` 会统一显示为同一个官方名字（`machine-control.txt` 明确说明：a.k.a. 名字的区分在机器语言中丢失）。

4.6 关键要点

- **控制流的全部基础设施只有两样：条件码 + goto。** `if`、`while`、`for`、`switch` 都只是 goto 的不同排版，理解回跳方向就理解了循环。
- **有符号关系用 `g/l`，无符号关系用 `a/b`**；条件码组合 `SF^OF` 与 `CF` 是两套判断的分水岭，`set` 与 `jX` 共用同一张表。
- **`cmov` 的本质是用数据流替代控制流**：它换来分支预测的确定性，代价是两条路径都必须可安全求值；可能非法访存的表达式禁止 `cmov` 化。
- **跳转表是编译器对稠密 `switch` 的 $O(1)$ 分派**，落在 `.rodata`，用 `jmp *表(,%rdi,8)` 加一次上界检查实现；稀疏时退回比较链，甚至用"先减后无符号比较"一次判掉连续小区间。
- **`-Og` 追求与源码逐行对应，`-O1` 以上才做 guarded-do、`cmov` 化等改写**；调试读汇编前务必先确认优化级别，否则会把优化产物误当源码语义。
- **Bomb Lab 的成功公式 = `objdump -d` 定位 + gdb 断点保命 + 条件码推演**：先设 `break explode_bomb`，再用 `info registers` 与 `x/s` 反推输入。

4.7 思考题（带答案）

题 1（汇编推演题）：下面的函数 `quiz` 由 `gcc -Og` 编译（真实产物），请写出它等价的 C 代码，并给出 `(a,b)` 为 `(5,3)`、`(-1,3)`、`(7,7)` 时的返回值。

```
quiz:
        cmpq    %rsi, %rdi
        jg      .L6                 # a > b
        movl    $0, %eax
.L2:
        cmpq    %rsi, %rdi
        je      .L8                 # a == b
.L3:
        cmpq    %rdi, %rsi
        jnb     .L4                 # b >= a (无符号)
        orq     $4, %rax
.L4:
        xorq    %rsi, %rdi
        js      .L9                 # (a^b) < 0
.L1:
        ret
.L6:
        movl    $1, %eax
        jmp     .L2
.L8:
        movl    $2, %eax
        jmp     .L3
.L9:
        orq     $8, %rax
        jmp     .L1
```

答案：它等价于对 4 个独立条件做按位累加：

```
/* quiz.c —— gcc -Og -Wall -std=c11 -g 验证通过，输出 5 / 12 / 2 */
#include <stdio.h>

long quiz(long a, long b) {
    long r = 0;
    if (a > b)  r = 1;                      /* = 1，不是 |= */
    if (a == b) r = 2;                      /* = 2，覆盖前面的值 */
    if ((unsigned long)a > (unsigned long)b) r |= 4;
    if ((a ^ b) < 0)                        r |= 8;   /* a、b 异号 */
    return r;
}

int main(void) {
    long c[][2] = {{5, 3}, {-1, 3}, {7, 7}};
    for (int i = 0; i < 3; i++)
        printf("(%2ld,%2ld) -> %ld\n", c[i][0], c[i][1], quiz(c[i][0], c[i][1]));
    return 0;
}
```

推演要点：`jnb` 是"无符号 `%rsi >= %rdi`"，即 `(unsigned)a > (unsigned)b` 时才**不**置 4 位，故 `orq $4` 出现在跳转**未**发生时；`xorq %rsi,%rdi` 的 `js` 判的是异号（一正一负时符号位为 1）。实测结果：`(5,3) → 5`（1+4）、`(-1,3) → 12`（4+8，负数在无符号下更大）、`(7,7) → 2`（仅相等，

`7>7` 不成立，`a^b=0` 非负）。

题 2（易错直觉题）：既然 `cmov` 能避免分支预测失败、又快又稳，为什么下面的函数不会被编译成 `cmov`？如果强行用 `cmov` 会怎样？

```
/* cread.c —— gcc -O1 -Wall -std=c11 -g 验证通过 */
long cread(long *xp) { return xp ? *xp : 0; }
```

（把它补成完整程序验证：`int main(void){ long v=99; printf("%ld %ld\n", cread(&v), cread(NULL)); return 0; }`，输出 `99 0`。）

答案：`cmov` 要求**两个候选值都被求值**，即必须无条件执行 `*xp`；当 `xp == NULL` 时这就是一次非法访存（段错误），原程序并不会因此崩溃。C 语义下"优化不得引入原程序不存在的行为"，因此编译器**必须**保留分支形成短路求值。想用上 `cmov` 的正确写法是把两条臂都变成安全计算（`cread_alt`：先让 `xp` 指向本地 `zero`，再统一解引用），实测 `-O1` 生成 `cmove %rax,%rdi` 且无跳转分支。

题 3（计算题）：某稠密 `switch` 的跳转表放在 `.rodata`，基址为 `0x402008`，被 `jmp *.L4(,%rdi,8)` 使用。若某个时刻 `%rdi = 3`，（a）该表项位于哪个地址？（b）为什么 scale 必须是 8？（c）如果编译器改用讲义提到的"4 字节偏移"表，同样的 `%rdi = 3` 会读到什么，最终目标地址如何算出？

答案：（a）`0x402008 + 3*8 = 0x402020`（实测该处正是 `0x40113f`，即 `grade` 的 `x==3` 分支）。（b）每个表项要容纳一个 64 位代码地址，即 8 字节，`D(Rb,Ri,S)` 的 scale 只能取 1/2/4/8，正好用 8。（c）偏移表每项 4 字节，`%rdi=3` 时读 `0x402008+3*4 = 0x402014` 处的 32 位值，按 `movslq` 符号扩展为 64 位后与表基址相加得到目标地址，即 `目标 = 表基址 + 偏移`；这样表体积减半，且天然位置无关（PIC 友好）。

题 4（易错直觉题）：有同学说"`for` 循环的汇编里一定有 `i++` 和 `i < N` 两条指令，所以照这个模式去认循环就行"。这个想法错在哪？

答案：错在把**源码结构**当成**汇编结构的判据**。第一，`-O1` 的 guarded-do 会把入口的初值测试整条删掉，循环只剩底部的 `cmpl $64,%ecx; jne`；第二，循环变量可能完全消失——`pcount_for` 在更激进的优化下会被向量化或直接识别成 `popcount`；第三，`for` 与 `while` 编译出的汇编可以完全一致（`machine-control.txt` 问题 17 正是在问"若不看名字，能否分辨源码写的是 for 还是 while"——答案是**不能**）。正确做法是看**回边（backward jump）**：哪里有向后跳转、哪里的条件码决定是否重复，哪里就是循环。

Lecture 5: 机器级编程 III——过程、栈帧与调用约定 (Machine Programming III: Procedures, Stack Frames, and Calling Convention)

> **讲义对应**：CMU 15–213 Lecture 5 — Machine Programming (Part III)（素材：`F25-05-machine-gamma.txt`） **教材对应**：CS:APP3e 第 3 章 3.7—3.9（过程、数组、异质数据结构） **关联 Lab**：L3 Attack Lab（栈帧与调用约定是全部攻击的物理基础）

5.1 概述

前两讲解决了"一条指令怎么算"（算术/逻辑）与"下一条指令在哪"（控制流），本讲回答第三个问题：**一个过程（procedure）如何调用另一个过程并在返回后继续正确执行**，需要同时解决控制传递（control transfer）、数据传递（data passing）与内存管理（memory management，局部存储的分配与释放）。x86-64 的答案是"寄存器传参 + 运行时栈 + 寄存器保存约定"，全部由普通指令实现，没有专用硬件。

本讲把"栈"从课本抽象变成**真实的、位于高地址段、向低地址增长的内存区域**：承接 Lecture 4 的分支跳转，通向 Lecture 6 的数组/结构体，并为 L3 Attack Lab 提供全部原理——攻击者的工作本质就是"用越界写覆盖别人栈帧里的返回地址"。

5.2 核心概念与底层机制图解

5.2.1 过程的三大机制（Mechanisms in Procedures）

- **定义与目的**：保证 P 调用 Q（$y = Q(x)$）后 P 的现场完好恢复，Q 的结果与副作用如期交付。
- **直观解释**：像读一本有注释的书：读到"参见第 7 章"（调用）就夹一张书签（返回地址）再去，读完凭书签回到原处；书签、要问的问题、草稿纸分别对应对三大机制。
- **三个问题的具体分解**：① **控制传递**——跳转到被调用者开头，返回时跳回调用点的**下一条指令**；② **数据传递**——参数传入、返回值传出；③ **内存管理**——执行期间分配局部存储，返回时释放。
- **与机器码/硬件的对应**：`call` / `ret` 负责控制传递，寄存器 + 栈负责数据传递，`subq` / `addq` 调整 `%rsp` 负责内存管理。**x86-64 只使用实际需要的机制**—— `mult2` 只用寄存器，连栈都不碰。
- **为什么需要"栈"这种结构**：支持递归的语言其过程代码必须**可重入（reentrant）**——同一过程可同时有多个活动实例（instantiation），各需自己的参数、局部变量与返回地址。栈的"后进先出"恰好匹配"被调用者必先于调用者返回"这一调用纪律（stack discipline）；每个实例在栈上占用的连续区域即一个**栈帧（Stack Frame）**。

5.2.2 运行时栈与 pushq/popq

- **定义与目的**：运行时栈是进程地址空间中由 `%rsp`（stack pointer）管理的一段连续区域，存放过程调用的瞬时状态。
- **直观解释**：栈像一摞盘子，只能从最顶上放和拿；而"顶端"在内存里的**地址是最小的**。

```
高地址 (0x7fffffffb6e8)
        +---------------------------------+  已映射栈区的高地址端
        |  调用者较早的帧 (older frames) |  高地址一侧，先入栈
        +---------------------------------+
        |  当前帧的数据                   |  局部变量 / 保存的寄存器
  %rsp ->+---------------------------------+  栈顶 (Top of Stack)
低地址 (0x7fffffffb6a0)        ↑ 栈向低地址方向增长 ↑

pushq 一次：%rsp -= 8，再向 (%rsp) 写入 8 字节
popq  一次：先读出 (%rsp) 的 8 字节，再 %rsp += 8
```

- **pushq/popq 的语义与等价形式**（讲义中给出的标准改写）：

```
    pushq %rbp   ≡   subq $8, %rsp          popq %rbp   ≡   movq (%rsp), %rbp
                     movq %rbp, (%rsp)                      addq $8, %rsp
```

`push` / `pop` 只是"改指针 + 搬数据"的编码紧凑版（编码仅 1–2 字节）。**栈指针的移动本身就代表分配与释放**：`subq $16, %rsp` 即"分配 16 字节"，`addq $16, %rsp` 即"释放"，不必清零。 – **与机器码/硬件的对应**：`push` / `pop` 由地址生成部件与存储部件完成；前提是该栈区已映射且可写。

5.2.3 控制传递：call 与 ret

- **定义与目的**：`call Label` 与 `ret` 组成最基本的调用/返回对。
- **直观解释**：`call` = "把回来的路写在便签上贴到栈顶，再跳走"；`ret` = "撕下栈顶便签照着跳回去"。
- **底层机制图解**（讲义课堂活动 `machine-procedures` 第 1–4 题的结论）：

```
        |  调用者的其他数据               |
        +---------------------------------+
        |  返回地址 (8 字节)              |  <== %rsp 指向这里（被调用者入口）
  %rsp ->+---------------------------------+

call Label  ≡  pushq $next ; jmp Label   # next = call 的下一条指令的地址
ret         ≡  popq %rip                 # ≡ movq (%rsp),%rip ; addq $8,%rsp
```

- **与机器码/硬件的对应**：讲义课堂活动用 GDB 验证：`0x15213` 上方紧邻的值 `0x00000040117a` 就是返回地址；`ret` 把它弹入 `%rip` 并使 `%rsp` 加 8。
- ⚠️ **这就是 Attack Lab 的全部原理**：返回地址在栈上、且位于被调用者局部缓冲区的**高地址一侧**；一次越界写即可改写这 8 字节，让 `ret` 跳到 `touch1`、注入代码或 gadget（详细手法见 Lecture 6）。

5.2.4 数据传递：参数寄存器与返回值

- **定义与图解**（讲义 Procedure Data Flow）：`%rdi %rsi %rdx %rcx %r8 %r9` 依次接第 1–6 个整型/指针参数，**第 7 个起走栈**；返回值在 `%rax`（128 位用 `%rdx:%rax`），浮点参数走 `%xmm0–%xmm7`、浮点返回在 `%xmm0`。
- **直观解释**：这是 ABI（Application Binary Interface）层面的"外交礼节"——双方遵守同一份礼节，各自内部怎么实现都无所谓。
- **与机器码/硬件的对应**：`multstore(x, y, dest)` 三个参数落在 `%rdi`、`%rsi`、`%rdx`，结果由 `%rax` 带回；**只在实际需要时才分配栈空间**——前 6 个参数完全不占栈。
- **常见陷阱**：第 7 个及以后的参数**从右往左**压栈（见 5.3.3 实测），故 `arg7` 在最低地址、离返回地址最近；被调用者用 `8(%rsp)`、`16(%rsp)` 读取（栈顶是返回地址）。

5.2.5 栈帧（Stack Frame）结构

- **定义与目的**：栈帧是单个过程实例在栈上占用的区域，容纳返回信息、局部存储与临时空间。
- **直观解释**：一次调用占用栈上一段"格子间"，进门时搭起（set-up code，含 `call` 压返回地址），出门时拆掉（finish code，含 `ret` 弹返回地址）；`%rsp` 是抽屉底边，`%rbp` 是上一层分界线。
- **完整的 x86-64/Linux 栈帧布局图**（高地址在上、低地址在下）：

```
高地址 (High Address)
         |  调用者的局部变量 ...          |
         +---------------------------------+
         |  参数 7、参数 8、...           |  调用者的参数构造区
         |  (Argument Build Area)          |  仅当整型参数 > 6 个时存在
         +---------------------------------+  <-- 调用者帧 / 被调用者帧的分界
         |  返回地址 (Return Address)     |  call 压入；ret 弹出并跳转
%rbp+8 ->+---------------------------------+
         |  (返回地址的位置)              |
%rbp+0 ->+---------------------------------+
         |  保存的 %rbp (Old %rbp)        |  被调用者 pushq %rbp（可选）
         +---------------------------------+  <== %rbp 指向这里（帧指针，可选）
         |  被调用者保存的寄存器          |  %rbx、%r12–%r15（用到的才保存）
         |  (Saved Registers)              |
         +---------------------------------+
         |  局部变量 (Local Variables)    |  取地址的变量、数组、寄存器装不下的
         +---------------------------------+
         |  本函数的参数构造区            |  调用别人时构造第 7+ 个参数
         |  (Argument Build, 可选)        |
  %rsp ->+---------------------------------+  <== 栈顶指针 (Stack Pointer)
低地址 (Low Address)          栈向这个方向增长 ↓
```

说明：`%rbp` 是**可选帧指针**，"Saved Registers/Local Variables" 与 "Argument Build" 的先后由编译器决定；无论怎么排，**`%rsp` 永远指向当前帧的最低有效地址**。典型序言是 `pushq %rbp; movq %rsp,%rbp; subq $N,%rsp`，收尾是 `movq %rbp,%rsp; popq %rbp; ret`（`-O0` 下可见，见 5.3.3）。

5.2.6 寄存器保存约定（Register Saving Conventions）

- **定义与目的**：`yoo` 把 `$15213` 放进 `%rdx` 后 `call who`，而 `who` 内部 `subq $18213, %rdx` 会毁掉它——若无人协调，`addq %rdx, %rax` 就会算错。约定据此把寄存器分为两类，明确"谁负责保存"。
- **直观解释：调用者保存（caller-saved，又名 call-clobbered）** 像公共白板：你离开自己办公室前要先把内容抄下来，因为对方会随手擦。**被调用者保存（callee-saved，又名 call-preserved）** 像带锁的抽屉：对方要用就得先清空、用完恢复，你放在里面的东西一定还在。
- **寄存器使用约定速查表**（摘自课堂活动附录 x86-64 ELF Calling Convention Summary）：

寄存器	保存约定	传统用途
`%rax`	**调用者保存**（Clobbered）	返回值；也常作临时寄存器
`%rbx`	**被调用者保存**（Preserved）	被调用者的临时存储
`%rcx`	**调用者保存**	第 4 个整型参数
`%rdx`	**调用者保存**	第 3 个整型参数；128 位返回值高半部
`%rsi`	**调用者保存**	第 2 个整型参数
`%rdi`	**调用者保存**	第 1 个整型参数
`%rbp`	**被调用者保存**	可作帧指针，也可当通用寄存器
`%rsp`	**特殊形式的被调用者保存**	栈指针；返回时必须恢复原值
`%r8`	**调用者保存**	第 5 个整型参数
`%r9`	**调用者保存**	第 6 个整型参数
`%r10`	**调用者保存**	临时寄存器
`%r11`	**调用者保存**	临时寄存器
`%r12`	**被调用者保存**	被调用者的临时存储
`%r13`	**被调用者保存**	被调用者的临时存储
`%r14`	**被调用者保存**	被调用者的临时存储
`%r15`	**被调用者保存**	被调用者的临时存储

一句话记忆：**调用者保存的是 `%rax %rcx %rdx %rsi %rdi %r8 %r9 %r10 %r11`；被调用者保存的是 `%rbx %rbp %r12 %r13 %r14 %r15` 加特殊的 `%rsp`。** – **为什么这样分（设计权衡）**：① **减少调用开销**——若所有寄存器都由被调用者保存，"什么都没干"的小函数（如 `incr`）每次调用也要先 push 再 pop 一堆寄存器；把易失寄存器划给调用者后，调用者**只在真的跨调用存活时才保存**。② **便于被调用者做临时存储**——`%rax %rcx %rdx` 等可当草稿纸随便涂写，无需任何保存/恢复代码。③ **长寿命变量有稳定去处**——跨调用存活的局部变量放 `%rbx` / `%r12–%r15` 或栈上，由被调用者保全（递归中"本

层的 `n`"就在 `%rbx`，见 5.3.4)。 – **与机器码/硬件的对应**：没有任何硬件强制这些约定；违反它不会立刻崩溃，只会产生极难调试的随机错误。`%rsp` 必须恢复则被 `ret` 强制——否则返回地址就取错了。

5.2.7 局部变量的存放

- **定义与目的**：决定局部变量住寄存器还是住栈。
- **三条规则**：① **优先放寄存器**——编译器尽量为每个局部变量分配一个寄存器，读写零访存；② **必须放栈上**——取过地址（`&x` 传给别人）、是数组或结构体（需连续编址、可下标访问）、寄存器不够用（变量太多或跨调用存活）；③ **取地址用 `leaq`**——`leaq 8(%rsp), %rdi` 算的是**地址**而非内容，正是 `call_incr` 中 `&v1` 的实现（5.3.2）。
- **直观解释**：寄存器像桌上便签（快，但数量有限、没法告诉别人在哪）；栈像带门牌号的储物柜（慢些，但有地址、能按序号排成数组）。
- **与机器码/硬件的对应**：`movq %rax, -40(%rsp)` 是"变量落户栈上"，`leaq -40(%rsp), %rax` 是"取变量地址"。注意 x86-64 中 `%rsp` 相对寻址即可完成一切，无需 `%rbp`。

5.2.8 递归过程与栈溢出

- **定义与目的**：递归是"同一过程有多个存活实例"的极端情形。
- **机制**：递归**无需任何特殊处理**——每次递归调用产生全新栈帧，其中的局部变量与保存的寄存器互相隔离；后进先出纪律恰好匹配"调用未返回则被调用者先返回"，互递归（P 调 Q、Q 调 P）同样适用。
- **直观看法**：每深入一层就像叠一张透明幻灯片，各写着自己的 `n`，返回时从上往下逐张抽走。讲义的 `yoo→who→amI→amI→amI` 调用链演示了 `%rsp` / `%rbp` 随调用链节节下移、又逐节回弹。
- **栈溢出（stack overflow）**：递归深度不受语言限制，只受栈空间限制（Linux 默认约 8 MB，`ulimit -s` 可查）；层越大越早触底，写入低于栈底映射区的地址即触发 SIGSEGV——"段错误"最常见的来源之一。
- **与机器码/硬件的对应**：`call` 本身只压 8 字节，深度消耗主要来自序言中的 `subq $N, %rsp`。("栈溢出"在安全语境下另有含义，见 5.2.10。)

5.2.9 x86-64 相对 IA32 的变化（补充说明）

- **帧指针不再必需**：IA32 的 `%ebp` 是标准帧指针、连成链表供调试器回溯；x86-64 允许 `%rbp` 当通用寄存器（默认 `-fomit-frame-pointer`），代价是回溯要靠 `.eh_frame`/CFI（`gcc -S` 里的 `.cfi_*` 即为此生成）。
- **参数不再全部走栈**：IA32 把**所有**参数压栈、被调用者用 `8(%ebp)` / `12(%ebp)` 取；x86-64 前 6 个走寄存器，故小函数几乎不触碰内存。
- **栈帧大小编译期确定、`%rsp` 相对寻址**：IA32 以 `%ebp` 为基准，x86-64 用 `%rsp` 加常量偏移，因为一帧多大在编译期已定（变长数组/`alloca` 除外）。
- **更严格的栈对齐**：System V AMD64 ABI 要求 `call` 前 `%rsp` 16 字节对齐——这正是 `-Og` 下 `main` 出现 `subq $8, %rsp` 以及 5.3.2 中 `subq $16, %rsp` 的原因。

5.2.10 缓冲区溢出（Buffer Overflow）：Attack Lab 的伏笔

- **定义与目的**：C 语言的数组不做边界检查，`gets` / `strcpy` / `strcat` / `sprintf` 等函数**无法知道目标缓冲区有多大**，会一直拷贝直到遇到 `'\0'`，从而越界覆盖相邻内存。
- **直观解释**：缓冲区像"往小杯子里倒水"，`strcpy` 不管杯子多大，多出的水漫到旁边——而旁边恰恰是返回地址。

```
高地址 (0x7fffffffb6f8)
         |  调用者 main 的帧              |
         +-----------------------------------+
         |  填充 (8 字节)                  |  对齐用
         +-----------------------------------+
         |  返回地址 (8 字节)              |  <== 越界写到这里 = 控制流被劫持
         +-----------------------------------+
         |  填充 (8 字节)                  |
         +-----------------------------------+
         |  char buf[16]                   |  <== strcpy 从这里开始写，不检查长度
  %rsp ->+-----------------------------------+  低地址 0x7fffffffb6c8
```

- **与机器码/硬件的对应**：`gets` 因无法安全使用已在 C11 中**移除**、glibc 中弃用（链接时警告），新代码用 `fgets`；`strcpy` 仍在，但容量必须由程序员保证。
- **缓和措施（modern defenses）**：栈不可执行（NX）、地址随机化（ASLR/PIE）、栈保护（stack canary，`-fstack-protector`）。Attack Lab 中 CTARGET 故意关闭这些保护以演示**代码注入（code injection, CI）**；RTARGET 打开 NX + ASLR，逼迫使用**返回导向编程（return-oriented programming, ROP）**——攻击者不再注入代码，而是把栈上的一串地址排成"gadget 链"，每个 gadget 都是现有程序里"若干指令 + `ret`"的片段（`0xc3` 就是 `ret` 的编码）。
- **本节只铺垫原理**：如何用 `%rdi` 传 cookie、如何用 `hex2raw` 处理小端字节序、如何选 gadget，详见 Lecture 6。

5.3 代码示例与底层机制分析

以下所有 C 代码均在 `/tmp/ch05/` 下用 `gcc (GCC) 12.2.0 -Og -Wall -std=c11` 实际编译、运行，并用 `gcc -Og -S` 生成汇编逐一核对。

5.3.1 示例 1：多参数调用与返回值（讲义 mult2 / multstore）

代码 (C)：

```c
/* c05_ex1.c —— 编译：gcc -Og -Wall -std=c11 c05_ex1.c -o c05_ex1 */
#include <stdio.h>

long mult2(long a, long b) {
    long s = a * b;
    return s;
}

void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);
    *dest = t;
}

int main(void) {
    long d = 0;
    multstore(15213, 2, &d);
    printf("multstore(15213,2) = %ld\n", d);
    return 0;
}
```

【代码做什么？】 ① `main` 给 `d` 在栈上留 8 字节并用 `leaq 8(%rsp), %rdx` 取地址，`15213` → `%edi`、`2` → `%esi`，`call multstore`；② `multstore` 用 `pushq %rbx` 保住 `dest`（`%rdx` 是调用者保存，跨 `call` 会丢）后 `call mult2`；③ `mult2` 相乘并把结果留在 `%rax`，`multstore` 写回 `(%rbx)` 并恢复。

【底层机制透视】 `mult2` **完全没有栈帧**（只用寄存器，`ret` 前无任何 `subq` / `pushq`），印证"x86-64 只用必需的机制"。`multstore` 的 `pushq %rbx` 是为让 `dest` 活过 `call mult2`——留在 `%rdx` 里会被 `mult2` 覆写。

【与汇编 / 硬件的对应】 `gcc -Og -S` 实测输出：

```
# long mult2(long a, long b)      # a in %rdi, b in %rsi
mult2:
    movq    %rdi, %rax             # s = a
    imulq   %rsi, %rax             # s *= b    -> 结果留在 %rax
    ret

# void multstore(long x, long y, long *dest)   # x:%rdi y:%rsi dest:%rdx
multstore:
    pushq   %rbx                   # 保存被调用者保存寄存器
    movq    %rdx, %rbx             # 把 dest 挪进 %rbx，活过下面的 call
    call    mult2
    movq    %rax, (%rbx)           # *dest = t
    popq    %rbx                   # 恢复
    ret
```

【实测验证】 输出 `multstore(15213,2) = 30426`；`objdump -d` 显示 `mult2` 位于 `0x401126`，`multstore` 中为 `callq 401126 <mult2>`。

5.3.2 示例 2：取地址的局部变量必须放栈上（call_incr）

代码 (C)：

```c
/* c05_ex4.c */
#include <stdio.h>

long incr(long *p, long val) {
    long x = *p;
    long y = x + val;
    *p = y;
    return x;
}

long call_incr(void) {
    long v1 = 15213;              /* 因为要取地址 &v1 -> 必须放在栈上 */
    long v2 = incr(&v1, 3000);
    return v1 + v2;
}

int main(void) {
    printf("call_incr() = %ld\n", call_incr());
    return 0;
}
```

【底层机制透视】 `v1` 本身只需 8 字节和一个寄存器，但 `&v1` 被传了出去，编译器**没有选择余地**：必须给它内存地址，而没有任何 x86-64 寄存器与地址绑定，唯一去处就是栈（5.2.7 的"取地址 ⇒ 必须放栈"）。相对地，`incr` 全程在寄存器里完成。

【内存布局 / 数据结构图解】（GDB 实测地址，x86-64 Linux）

```
        +------------------------------+  0x7fffffffb6d0
        |  main 帧的返回地址 / 填充     |
        +------------------------------+  0x7fffffffb6c8
        |  填充(对齐, 8 字节)           |
        +------------------------------+  0x7fffffffb6c0  <== &v1 (即 %rdi)
        |  v1 = 15213 = 0x3b6d         |
        +------------------------------+  0x7fffffffb6b8
        |  填充 8 字节                  |
        +------------------------------+  0x7fffffffb6b0  <== %rsp (call 之前)
        |  返回地址 0x40114c            |  <== call 压入, incr 入口处 %rsp
        +------------------------------+  0x7fffffffb6a8
        |  ... incr 帧 (无局部变量) ... |
```

【与汇编 / 硬件的对应】 `gcc -Og -S` 实测输出（与讲义幻灯片逐条一致）：

```
incr:                           # p in %rdi, val in %rsi
    movq    (%rdi), %rax        # x = *p
    addq    %rax, %rsi          # y = x + val   (%rsi = val, y 复用)
    movq    %rsi, (%rdi)        # *p = y
    ret                         # 返回值 x 已在 %rax

call_incr:
    subq    $16, %rsp           # 分配 16 字节: v1 与对齐填充
    movq    $15213, 8(%rsp)     # v1 = 15213
    movl    $3000, %esi         # 用 movl 省 1 字节; 写 %esi 自动清零高 32 位
    leaq    8(%rsp), %rdi       # &v1 —— leaq 计算地址而非取值
    call    incr
    addq    8(%rsp), %rax       # v1 + v2, 直接从栈上重读 v1
    addq    $16, %rsp           # 释放
    ret
```

【实测验证】 GDB 断在 `incr` 入口：`%rsp = 0x7fffffffb6b0`，`x/6gx $rsp` 首行为返回地址 `0x40114c`，反查得 `call_incr+28: add 0x8(%rsp),%rax`；`x/1gd $rdi` 为 `15213`。输出 `call_incr() = 33426`（=15213+18213），与讲义一致。

5.3.3 示例 3：8 个参数——第 7、8 个走栈传递

代码 (C)：

```c
/* c05_ex3.c */
#include <stdio.h>

long sum8(long a1, long a2, long a3, long a4,
          long a5, long a6, long a7, long a8) {
    return a1 + a2 + a3 + a4 + a5 + a6 + a7 + a8;
}

int main(void) {
    long r = sum8(1, 2, 3, 4, 5, 6, 7, 8);
    printf("sum8 = %ld\n", r);
    return 0;
}
```

【底层机制透视】 前 6 个参数住寄存器，第 7、8 个"溢出"到**调用者的参数构造区**。为了让这 16 字节在 `call` 之后仍位于被调用者所期望的偏移，编译器采用"**从右向左压栈**"：先 `pushq $8` 再 `pushq $7`，于是 `7` 在最低地址（`(%rsp)` 位置即被调用者的 `8(%rsp)`）。

【内存布局 / 数据结构图解】（`call sum8` 执行后的瞬间）

```
        +----------------------------+  高地址
        |  调用者 main 的其他数据    |
        +----------------------------+  <== 调用前的 %rsp
        |  arg8 = 8                  |  <-- pushq $8 压入（先压）
        +----------------------------+
        |  arg7 = 7                  |  <-- pushq $7 压入（后压）
        +----------------------------+  <-- call 前的 %rsp
        |  返回地址（main 中 call    |
        |  的下一条指令）            |
        +----------------------------+  <== sum8 入口处的 %rsp
        |  ... sum8 帧（本函数无局部）|
```

所以被调用者读参数用的是 `8(%rsp)` = arg7、`16(%rsp)` = arg8：**"跳过栈顶那 8 字节返回地址"** 是这里最容易记错的地方。

【与汇编 / 硬件的对应】 `gcc -Og -S` 实测：

```asm
# long sum8(long a1, ..., long a8)
#   a1:%rdi a2:%rsi a3:%rdx a4:%rcx a5:%r8 a6:%r9 a7:8(%rsp) a8:16(%rsp)
sum8:
    addq    %rsi, %rdi              # a1 += a2
    addq    %rdx, %rdi              # a1 += a3
    addq    %rcx, %rdi              # a1 += a4
    addq    %r8, %rdi               # a1 += a5
    leaq    (%rdi,%r9), %rax        # rax = a1 + a6
    addq    8(%rsp), %rax           # += arg7   —— 来自栈
    addq    16(%rsp), %rax          # += arg8   —— 来自栈
    ret

main:
    subq    $8, %rsp                # 仅为 16 字节对齐
    pushq   $8                      # arg8 先入栈（高地址）
    pushq   $7                      # arg7 后入栈（低地址）
    movl    $6, %r9d                # arg6
    movl    $5, %r8d                # arg5
    movl    $4, %ecx                # arg4
    movl    $3, %edx                # arg3
    movl    $2, %esi                # arg2
    movl    $1, %edi                # arg1
    call    sum8
    addq    $16, %rsp               # 调用者负责释放自己压入的参数
```

`-O0` 下更能看出被调用者的读法：序言是 `pushq %rbp; movq %rsp,%rbp;`，六个寄存器参数被搬到 `-8(%rbp)`…`-48(%rbp)`，随后用 `movq 16(%rbp), %rax`（arg7）与 `movq 24(%rbp), %rax`（arg8）——`16(%rbp)` 恰好"跳过 saved `%rbp`（8 字节）+ 返回地址（8 字节）"。

编译器也可先把 `%rsp` 整体下移、再往参数槽写内存（等价形式）：

```asm
    subq    $16, %rsp               # 一次性开辟参数构造区
    movq    %rdx, (%rsp)            # arg7 写到最低地址
    movq    %rax, 8(%rsp)           # arg8
```

我在 `/tmp/ch05` 编译的 `caller3(long *v)`（`-Og -S`）中真实观测到 `movq %r9, (%rsp)` / `movq %r8, 8(%rsp)` 与 `pushq %r10` / `pushq %r8` 并存；`objdump -d` 量出 `pushq $8` 编码为 `6a 08`（2 字节），而等价的 `movq $8, (%rsp)` 要 8 字节——**这就是 GCC 偏爱 `push` 的直接原因**。

【实测验证】 输出 `sum8 = 36`；`objdump -d` 显示 `pushq $0x8` 编码为 `6a 08`。**关键观察：栈参数的释放由调用者负责**（`addq $16, %rsp`），这与被调用者保存寄存器的恢复由被调用者负责形成对照。

5.3.4 示例 4：递归阶乘——每次调用生成新栈帧

代码 (C)：

```
/* c05_ex2.c */
#include <stdio.h>

long fact(long n) {
    if (n <= 1)
        return 1;
    else
        return n * fact(n - 1);
}

int main(void) {
    for (long i = 0; i <= 10; i++)
        printf("fact(%ld) = %ld\n", i, fact(i));
    return 0;
}
```

【代码做什么？】 ① `n <= 1` 直接返回 1（**终止情形**，无栈帧开销）；② 否则保存本层 `n`、把 `n-1` 放进 `%rdi` 递归调用；③ 用保存的 `n` 乘 `%rax` 后恢复返回。

【底层机制透视】 关键在 `movq %rdi, %rbx`。`n` 是本层私有状态，必须活过 `call fact`；`%rdi` 是**调用者保存**的，递归调用有权覆写它，所以 `n` 必须搬到**被调用者保存**的 `%rbx`（并配合 `pushq %rbx` / `popq %rbx` 保全调用者的 `%rbx`）。若把 `n` 留在 `%rdi`，`fact(10)` 会算出荒谬的结果——这是初学者最容易犯的错。

【内存布局 / 数据结构图解】 `fact(4)` 最深处的栈：

```
高地址
        +-----------------------------------------+
        |  main 的帧                              |
        +-----------------------------------------+  <-- main 调用 fact(4) 时 call 压入
        |  fact(4) 帧: 返回地址 (回到 main)       |
        +-----------------------------------------+
        |  saved %rbx (main 的旧值)               |  被调用者保存约定
        |  %rbx = 4                               |  <== 本层私有 n
        +-----------------------------------------+  <-- fact(4) 调用 fact(3) 时 call 压入
        |  fact(3) 帧: 返回地址 (回到 fact(4))    |
        +-----------------------------------------+
        |  saved %rbx (= 4)                       |
        |  %rbx = 3                               |  <== 本层私有 n
        +-----------------------------------------+  <-- fact(3) 调用 fact(2) 时 call 压入
        |  fact(2) 帧: 返回地址 (回到 fact(3))    |
        +-----------------------------------------+
        |  saved %rbx (= 3)                       |
        |  %rbx = 2                               |  <== 本层私有 n
        +-----------------------------------------+  <-- fact(2) 调用 fact(1) 时 call 压入
        |  fact(1) 帧: 返回地址 (回到 fact(2))    |  n<=1 直接 ret, 不再 pushq %rbx
  %rsp ->+-----------------------------------------+  低地址 (栈向此方向增长)
```

每层恰好 16 字节（返回地址 + saved `%rbx`），`fact(10)` 只耗 160 字节；若每层加 1 KB 局部数组，同深度就要 10 KB——**帧大小决定递归深度**。

【与汇编 / 硬件的对应】 `gcc -Og -S` 实测：

```
fact:                           # n in %rdi
    cmpq    $1, %rdi
    jg      .L8                 # if (n > 1) goto .L8
    movl    $1, %eax            # return 1
    ret
.L8:
    pushq   %rbx                # 保存调用者的 %rbx（被调用者保存约定）
    movq    %rdi, %rbx          # 本层私有 n 存进 %rbx
    leaq    -1(%rdi), %rdi      # 递归参数 n-1
    call    fact
    imulq   %rbx, %rax          # n * fact(n-1)
    popq    %rbx                # 恢复
    ret
```

【实测验证】 输出 `fact(10) = 3628800`。讲义附加幻灯片中的递归 `pcount_r` 用同一套模式（`pushq %rbx; movq %rdi,%rbx; andl $1,%ebx; shrq %rdi; call pcount_r; addq %rbx,%rax; popq %rbx`）；讲义还指出**尾递归** `popcount_t` 因"调用后无任何计算"被优化成 `jne .L15` 纯循环，**不消耗任何栈帧**。

5.3.5 示例 5：栈缓冲区溢出演示

> ⚠️ **仅供演示，请勿模仿**。以下代码故意制造越界写，仅用于理解 Attack Lab 的物理基础。

代码 (C)：

```
/* overflow.c —— 编译：gcc -Og -g -fno-stack-protector overflow.c -o overflow */
#include <stdio.h>
#include <string.h>

__attribute__((noinline)) void vulnerable(const char *src) {
    char buf[16];
    strcpy(buf, src);          /* ⚠️ 无边界检查：src 长于 16 字节即越界 */
    printf("buf = %s\n", buf);
}

int main(int argc, char **argv) {
    if (argc > 1) vulnerable(argv[1]);
    printf("returns normally\n");
    return 0;
}
```

【底层机制透视】 `strcpy` 只拿到一个目标指针，**没有任何长度信息**，必然写满整串字符加一个 `'\0'`；`buf` 与返回地址同处一帧、返回地址在更高地址，长输入必然踩到它。

【内存布局 / 数据结构图解】（GDB 实测，`-Og -fno-stack-protector`）

```
        +---------------------------------+  高地址  0x7fffffffb6f8
        |  main 的帧                      |
        +---------------------------------+  0x7fffffffb6e0  <== 返回地址所在
        |  返回地址 (8 字节)              |       「越界 24 字节即改到这里」
        +---------------------------------+  0x7fffffffb6d8
        |  填充 (8 字节)                  |
        +---------------------------------+  0x7fffffffb6d0
        |  char buf[16]                   |  <== strcpy 从这里开始写
   %rsp -> +--------------------------------+  0x7fffffffb6c8  低地址
```

【实测验证】

```
$ ./overflow short-ok
buf = short-ok
returns normally          # 未越界，正常返回

$ ./overflow AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault (core dumped)   # exit=139=SIGSEGV：返回地址被 0x41414141… 覆盖
```

24 字节的偏移由 `BUFFER_SIZE`、对齐与编译器版本共同决定。Attack Lab 中**必须用 `objdump -d ctarget` 亲自量出 `buf` 与返回地址的偏移**，不能照搬任何数字（讲义明确提醒该位置取决于 `BUFFER_SIZE` 与 GCC 的分配策略）。

5.4 实验关联：L3 Attack Lab

本讲是 L3 Attack Lab 的**全部理论前提。立刻动手做的部分**：1. **先做课堂活动 `machine-procedures`**：在 GDB 里跑 `calls` 与 `locals`，亲手确认 `call` 压入返回地址、`ret` 弹回。做完这四道题，你就具备了攻击所需的全部观察手段。 2. **Phase 1（CTARGET，代码注入 CI）**：用 `objdump -d ctarget` 找 `touch1` 地址与 `buf` 到返回地址的偏移，再用 `hex2raw` 构造"填充 + 小端返回地址"——本讲的栈帧图与 `call`/`ret` 语义直接给出答案。 3. **Phase 2/3（CI 传参）**：注入代码把 cookie 或其字符串地址放进 `%rdi`，再 `ret` 进 `touch2`/`touch3`（第一个参数用 `%rdi`，见 5.2.4）。 4. **Phase 4/5（RTARGET，ROP）**：栈不可执行 + ASLR，只能把 `farm.c` 的 gadget 地址排成链；gadget 就是"指令序列 + `ret`"，靠的仍是 5.2.3 的语义。

Lab 中会遇到的坑：– **字节序**：`hex2raw` 按小端写地址，`0x0040117a` 要写成 `7a 11 40 00`。 – **`0x0a` 是换行**：exploit 串中间不能出现 `0x0a`，否则 `Gets` 提前终止。 – **偏移量必须实测**：它由 `BUFFER_SIZE` 与 GCC 分配策略决定，是每个 target 独有的编译期常量。 – **不要用 `jmp`/`call` 做内部跳转**：讲义建议 "use `ret` instructions for all transfers of control"，因为 `call` 的相对偏移编码难以构造。 – **Phase 3 的字符串位置**：`hexmatch`/`strncmp` 会压栈并覆盖 `getbuf` 的缓冲区，cookie 字符串要放在**不会被覆盖**的高地址处。

5.5 常见错误与调试技巧

- **把 `%rdi` 当本地变量存放处**：递归或嵌套调用后参数"莫名其妙变了"（`%rdi` 是调用者保存）。**调试**：`objdump -d -M intel a.out` 看跨 `call` 的寄存器是否都是 `%rbx` / `%r12`–`%r15`；GDB `break *addr` 后 `info registers` 对比。
- **忘记恢复 `%rbx`**：`pushq %rbx` 漏了 `popq %rbx`（或提前 `ret`），破坏常出现在**离现场很远**的地方。**调试**：`gcc -S` 统计 `pushq` / `popq` 配对数；GDB `break func`、`finish`，比较入口与返回前的 `info registers rbx r12 r13`。
- **误解栈参数偏移**：以为第 7 个参数在 `(%rsp)`，实际那里是返回地址、arg7 在 `8(%rsp)`。**调试**：`gdb --args ./prog`，被调用者入口 `x/4gx $rsp`，再 `p *(long*)($rsp+8)` 与实参对比。
- **`%rsp` 16 字节对齐被破坏**：调用 `printf`/SSE 时 `SIGSEGV`（`movaps` 要求 16 字节对齐），源于 `push` 了奇数个 8 字节未配平。**调试**：`break *func`，`p/x $rsp & 0xf` 应为 `0`。
- **递归导致栈溢出**：`Segmentation fault` 且 `bt` 显示成千上万层同样帧。**调试**：`ulimit -s` 查栈上限（通常 8192 KB）；`gdb -ex run -ex 'bt 30'` 数帧。
- **缓冲区越界但"看起来没事"**：覆盖到未使用的填充字节时反而正常返回。**调试**：`gcc -fsanitize=address -g prog.c` 运行，ASan 直接报 `stack-buffer-overflow` 与越界字节数。
- **以为 `&x` 一定存在**：GDB 里 `p &x` 报无此变量（`-O2` 下已进寄存器或被消除）。**调试**：用 `-Og` 编译，或 `gdb -ex 'info locals'`、`-ex 'info address x'`。
- **`gets` 已不可用**：链接期报 "the `gets' function is dangerous and should not be used.", 因为 C11 已移除 gets。**替代**：fgets(buf, sizeof buf, stdin) 后自行去掉换行，或 getline。 ### 5.6 关键要点 * **栈是过程机制的正确答案**："若 P 调用 Q 则 Q 先返回"与后进先出天然契合，每个实例获得私有存储，递归与互递归都无需特殊处理。 * **call = 压返回地址 + 跳转；ret = 弹栈顶到 %rip**。返回地址在栈上、位于局部缓冲区的高地址侧——**这就是 Attack Lab 全部攻击的物理基础**。 * **数据传递规则**：前 6 个整型/指针参数进 %rdi %rsi %rdx %rcx %r8 %r9，第 7 个起**从右向左压栈**（被调用者读 8(%rsp)、16(%rsp)）；返回值在 %rax，浮点在 %xmm0。 * **寄存器保存约定**：调用者保存 %rax %rcx %rdx %rsi %rdi %r8 %r9 %r10 %r11，被调用者保存 %rbx %rbp %r12–%r15，%rsp 必须恢复；跨调用存活的值放被调用者保存寄存器或栈上。 * **%rsp 的移动就是分配/释放**：subq \(N, %rsp 分配、addq \)N, %rsp 释放；x86-64 的 %rbp 可选，栈帧大小在编译期定死。 * **安全第一原则**：strcpy / strcat / sprintf /已废弃的 gets 都要求程序员自己保证容量；栈越界写能劫持控制流，防御靠 NX、ASLR、canary 与安全编码。 ### 5.7 思考题（带答案） **题 1（栈帧推演题）** 某 x86-64 函数（-O0` 编译）汇编如下，请画出栈帧并推断其 C 原型与局部变量：

```
foo:
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $48, %rsp
    movq    %rdi, -8(%rbp)          # 参数 1 落栈
    movq    %rsi, -16(%rbp)         # 参数 2 落栈
    movq    -8(%rbp), %rax
    addq    -16(%rbp), %rax
    movq    %rax, -40(%rbp)         # 局部变量 A
    movq    -8(%rbp), %rax
    subq    -16(%rbp), %rax
    movq    %rax, -32(%rbp)         # 局部变量 B
    movq    -40(%rbp), %rax
    imulq   -32(%rbp), %rax
    movq    %rax, -24(%rbp)         # 局部变量 C
    movq    -24(%rbp), %rax
    addq    -32(%rbp), %rax
    leave
    ret
```

答案：原型 `long foo(long x, long y)`；三个 `long` 局部变量 `A = x + y`、`B = x - y`、`C = A * B`，返回 `C + B`。

```
  高地址
    +-----------------------------+
    |  arg7+（本题无）            |
    +-----------------------------+
    |  返回地址                   |
    +-----------------------------+  <== %rbp+8
    |  saved %rbp（调用者的）     |  <== %rbp 指向此处
    +-----------------------------+
    |  -8(%rbp):  参数 x          |  \
    |  -16(%rbp): 参数 y          |   | 编译器把两个入口参数
    |  -24(%rbp): 局部 C          |   | 从寄存器搬到栈上
    |  -32(%rbp): 局部 B          |   |
    |  -40(%rbp): 局部 A          |  /
    |  -48(%rbp): 填充/对齐       |
    +-----------------------------+  <== %rsp = %rbp-48
  低地址
```

推演要点：**参数先经寄存器到达、再由序言搬到栈上**；`subq $48` 说明帧内 48 字节；负偏移都是本函数私有存储。（我在 `/tmp/ch05` 用 `-Og -S` 编译的对应函数 `puzzle(5,3)` 中，三个局部变量正落在 `-40(%rsp)`、`-32(%rsp)`、`-24(%rsp)`。）

题 2（计算题） 某递归函数每层栈帧 48 字节（含返回地址、saved `%rbp` 与 32 字节局部数组），Linux 默认栈上限 8 MB。若还要调用 `printf`（内部一次性再耗 1 KB 栈），最深处约在第几层？

答案：可用栈 $\approx 8 \times 2^{20} = 8388608$ 字节，扣掉 `printf` 的 1 KB 余量后 ≈ 8387584 字节；每层 48 字节，故 $n \approx 8387584/48 \approx 174741$ 层（工程上按 100000 层估算更稳妥）。**把每层数组从 32 字节改成 320 字节，最大深度立刻缩成十分之一**——递归深度由栈空间而非语言决定。

题 3（"想当然"辨析） 有同学说："`%rax` 是返回值寄存器，所以我把它当中间结果，调用 `who` 之后它肯定还在。"错在哪？

答案：混淆了"用途"与"保存约定"。`%rax` 的传统用途是返回值，但它的**保存约定是调用者保存（call-clobbered）**，被调用者有权覆写。它"能"作返回值，是因为约定要求**调用者主动去读**它，而非要求被调用者保全它。跨 `call` 存活的中间结果必须放 `%rbx` / `%r12`–`%r15`（被调用者保全）或调用者自己的栈帧。同理，`%rdi` 是"第 1 个参数寄存器"，也不代表调用后仍保留原值。

题 4（设计权衡题） 为什么只让 `%rbx`、`%rbp`、`%r12–%r15` 这 6 个寄存器由被调用者保存，而不让全部 16 个都这样？

答案：这是**调用开销**与**寄存器可用性**的折中：全部由被调用者保存，则小函数（如 `incr`）即使一个寄存器都不用也得 push/pop 一堆，开销加在所有调用上；全部由调用者保存，则被调用者没有"便宜"的私有寄存器，稍复杂的函数就要频繁访存。当前划分下，调用者只为**真正跨调用存活**的少数值付费（`multstore` 付一次 `pushq %rbx`），被调用者把 `%rax %rcx %rdx` 等当免保存草稿纸用（`incr` 全程零保存、零访存）。

Lecture 6: 机器级编程 IV——数组、结构体、联合体与浮点 (Machine Programming IV: Arrays, Structs, Unions, and Floating Point)

> **讲义对应**：CMU 15–213 Lecture 6 — Machine Programming (Part IV)（素材：`F25-06-machine-delta.txt`；Fall 2026 对照：`06-machine-delta.txt`） **教材对应**：CS:APP3e 第 3 章 3.8—3.11（数组分配与访问、异质数据结构、对齐、浮点代码） **关联 Lab**：L3 Attack Lab（栈布局、缓冲区溢出、ROP 链）

6.1 概述

本讲回答一个核心问题：**C 语言里那些"高级"的复合数据类型，在机器一级究竟长什么样？** 聚合类型（aggregate type）——数组、结构体、联合体——并没有对应的机器指令，硬件看到的只有连续字节和"基址 + 偏移"的寻址。编译器负责把 `A[i][j]`、`p->next`、`u.f` 全部降解成 `leaq` / `movl 偏移(%rdi,%rsi,4), %eax` 这样的地址算术。

承接 Lecture 5 的过程与栈帧知识，本讲先把"数据"摆到内存图谱上（x86-64 Linux 地址空间、栈/堆/数据/代码段），再反过来利用它：栈上的数组一旦越界，就会覆盖相邻成员乃至返回地址，这就是缓冲区溢出（buffer overflow）。系统随后用栈随机化、不可执行内存和栈金丝雀（canary）三层防御应对，攻击者则以返回导向编程（Return-Oriented Programming, ROP）绕过。最后回到数据的位级表示：IEEE 754 浮点格式、`%xmm` 寄存器与 SSE 指令，以及浮点运算为何不满足结合律。这一讲是 Attack Lab 的直接理论前提。

6.2 核心概念与底层机制图解

6.2.1 数组与指针算术（Array Allocation and Pointer Arithmetic）

- **定义与目的**：C 声明 `T A[L];` 分配一块**连续**区域，大小为 `L * sizeof(T)` 字节。数组标识符 `A` 在表达式中退化为（decay 为）指向第 0 个元素的指针，类型是 `T *`，值等于首元素地址。
- **直观解释**：数组就像电影院里连号的一排座位——座位号（下标）乘以座位宽度（`sizeof(T)`）就是离排头的距离。数组名不是"一整排座位"，而是"排头座位的位置标签"。
- **底层机制图解**：

```
int val[5];      /* x 为起始地址，每格 4 字节 */
        +--------+--------+--------+--------+--------+
  val:  | val[0] | val[1] | val[2] | val[3] | val[4] |
        +--------+--------+--------+--------+--------+
地址:      x       x+4      x+8      x+12     x+16    x+20
表达式   val      val+1    &val[2]  *(val+3) val+i
类型     int*     int*     int*     int      int*
值        x       x+4      x+8      val[3]   x + 4*i
```

- **关键公式**：`A[i]` 编译为 `*(A + i*sizeof(T))`；因此 `p + i` 的实际地址增量是 $i \times \texttt{sizeof(*p)}$，而**不是** i 个字节。`&A[i]` 与 `A + i` 完全等价。指针相减 `q - p` 得到的是**元素个数**（两个地址之差除以 `sizeof(T)`），不是字节差。
- **与机器码/硬件的对应**：访问 `int z[i]` 只要一条 `movl (%rdi,%rsi,4), %eax`——比例变址寻址（scaled indexed addressing）在地址生成单元（AGU）里用一个周期完成 `基址 + 变址×比例`，这正是为数组访问设计的硬件。`sizeof(T)` 只能取 1、2、4、8，超出范围编译器会退化为乘法 + 加法。

6.2.2 嵌套数组与行优先序（Nested Arrays and Row-Major Order）

- **定义与目的**：`T A[R][C];` 共 `R*C*sizeof(T)` 字节，按**行优先（row-major）**排列：先存第 0 行全部 C 个元素，再存第 1 行……
- **直观解释**：像读书——从左到右读完一行，再换到下一行。`A[i]` 本身是一个"长度为 C 的数组"，所以 `A` 的类型是"数组的数组"。
- **底层机制图解**：

```
int A[4][5];   /* R=4 行, C=5 列, sizeof(int)=4, 共 4*5*4 = 80 字节 */

 行\列    j=0      j=1      j=2      j=3      j=4
        +--------+--------+--------+--------+--------+
 A[0]   |   1    |   5    |   2    |   0    |   6    |  行起始 = A + 0*20
        +--------+--------+--------+--------+--------+
 A[1]   |   1    |   5    |   2    |   1    |   3    |  行起始 = A + 1*20
        +--------+--------+--------+--------+--------+
 A[2]   |   1    |   5    |   2    |   1    |   7    |  行起始 = A + 2*20
        +--------+--------+--------+--------+--------+
 A[3]   |   1    |   5    |   2    |   2    |   1    |  行起始 = A + 3*20
        +--------+--------+--------+--------+--------+
 字节:    A+0      A+4      A+8      A+12     A+16     A+20 ...（连续递增）

 A[i][j] 的地址 = A + (i*C + j)*sizeof(T) = A + (i*5 + j)*4
 例:   A[2][3] -> A + (2*5 + 3)*4 = A + 52
 行向量: A[i] 的起始地址 = A + i*(C*sizeof(T)) = A + i*20
```

- **与机器码/硬件的对应**：`A[i]`（行访问）就是 `A + 20*i`；`A[i][j]` 则是 `A + 4*(5*i + j)`。编译器在 `-O1` 以上会做**强度削弱（strength reduction）**：把"乘 20"写成 `leaq (%rdi,%rdi,4),%rax` 得到 `5*i`，再 `leaq pgh(,%rax,4),%rax` 乘 4——两次移位加法代替一次乘法，且完全避开 `imul`。**关键推论**：行优先意味着同一行的元素在内存中相邻（空间局部性好），按行遍历比按列遍历缓存命中率高得多——详见 Lecture 9/10。

6.2.3 结构体与成员偏移（Structures and Member Offsets）

- **定义与目的**：结构体把**异质（heterogeneous）**字段按**声明顺序**打包成一块内存，大到足以容纳所有字段。编译期就确定了每个成员的偏移（offset），汇编里只出现偏移常量，字段名已经消失。
- **直观解释**：像一张固定格式的登记表——第 1 栏姓名、第 2 栏年龄……栏位顺序由填写规则（声明顺序）决定，编译器绝不会为了省纸而重排栏位。
- **与机器码/硬件的对应**：访问 `r->a[idx]` 编译成 `leaq (%rdi,%rsi,4),%rax`（`r + 4*idx`）；访问 `r->next` 编译成 `movq 24(%rdi),%rdi`——**基址 + 编译期常量偏移**。成员偏移用 `<stddef.h>` 的 `offsetof(struct S, member)` 获取，它是编译期常量，甚至在链接期作为重定位量出现（讲义里写作 `a+8`，由链接器解析成绝对地址）。
- **补充说明**：`struct rec { int a[4]; size_t i; struct rec *next; };` 的偏移是 0 / 16 / 24，`sizeof` 为 32——数组 `a` 占 16 字节，`size_t` 要求 8 字节对齐所以紧接其后，指针再 8 字节，末尾无需补齐（已是 8 的倍数）。链表遍历因此是一行 `movq 24(%rdi),%rdi` 加一次判空跳转。

6.2.4 对齐与填充（Alignment and Padding）

- **定义与目的**：x86-64 要求 K 字节的原始类型，其地址必须是 K 的倍数：`char` 无限制、`short`（2）地址最低 1 位为 0、`int` / `float`（4）最低 2 位为 00、`double` / `long` /指针（8）最低 3 位为 000。为满足它，编译器在成员之间插入**内部填充（internal padding）**，在结构体末尾插入**外部填充（external padding）**。
- **直观解释**：像停车场的车位线——4 米长的车必须停在 4 米网格的整格上，中间空出来的位置不能再塞别的车（填充），只为让每辆车都对齐入位。
- **为什么要填充**：内存系统按对齐的 4/8 字节块访问。跨越缓存行（cache line，64 字节）的数据需要两次访存；跨页（4 KB）的数据让虚拟内存地址翻译更麻烦。填充是用空间换访存效率。
- **底层机制图解**：

```
struct S1 { char c; int i[2]; double v; };   /* K = 8 (由 double 决定), sizeof = 24 */

偏移  0    1    2    3    4    5    6    7
     +----+----+----+----+----+----+----+----+
     | c  |////|////|////|      i[0]       |     //// = 填充
     +----+----+----+----+----+----+----+----+
      ↑                   ↑
     p+0                 p+4   (4 的倍数)

偏移  8    9   10   11   12   13   14   15
     +----+----+----+----+----+----+----+----+
     |      i[1]       |////|////|////|////|
     +----+----+----+----+----+----+----+----+
      ↑                                   ↑
     p+8 (8 的倍数)                      p+16

偏移 16   17   18   19   20   21   22   23
     +----+----+----+----+----+----+----+----+
     |            v (double, 8 字节)          |
     +----+----+----+----+----+----+----+----+
      ↑
     p+16 (8 的倍数)；结构体长度 24 = 3*8，是 K=8 的倍数
     => 注意 K = 8 < sizeof(struct S1) = 24

对照: struct S2 { long v; int i[2]; char c; };   /* sizeof = 24，尾部 7 字节外部填充 */
     struct S4 { char c; int i; char d; };       /* sizeof = 12 */
     struct S5 { int i; char c; char d; };       /* sizeof =  8 */
     把大类型放前面（S5）能省下 4 字节。
```

- **真实运行数据（本机 GCC 12.2.0 / x86-64 Linux 实测）**：

```
sizeof(struct S1)      = 24
offsetof c / i / v     = 0 / 4 / 16
对齐要求 K(S1)         = 8 (= max member alignment)
sizeof(struct S2)      = 24   (17 -> 补齐到 24)
offsetof v / i / c     = 0 / 8 / 16
sizeof(struct S3)      = 12, offsetof j = 8
sizeof(struct S4)      = 12
sizeof(struct S5)      = 8
```

- **结构体数组**：因为结构体长度已是 K 的倍数、起始地址也是 K 的倍数，所以 `struct S A[L]` 的元素之间**不需要额外填充**，`A[i]` 的地址就是 `A + i*sizeof(struct S)`。讲义中 `struct S3 { short i; float v; short j; } a[10];` 的 `sizeof` 是 12（含 2 字节尾部填充），`a[idx].j` 的汇编为 `leaq (%rdi,%rdi,2),%rax`（3*idx）再 `movzwl a+8(,%rax,4),%eax`——先算 `3*idx` 再乘 4 得 `12*idx`，把两次乘法都变成了 `lea`。
- **补充说明**：Fall 2026 的同一页把 `struct S1/S2` 里的 `double v` 换成了 `long v`。二者都是 8 字节、对齐要求都是 8，所以 K 与所有偏移完全不变——这正说明**结构体布局只取决于大小和对齐，与类型的语义无关**。

6.2.5 联合体（Unions）

- **定义与目的**：`union U { ... };` 的所有成员**共享同一块内存起始地址**，大小 = 最大成员的大小，对齐 = 最大成员的对齐。同一时刻只能用其中一个字段（否则读到的是别的字段的位模式）。
- **直观解释**：像一块可擦写白板——你写"浮点数"时它是浮点数，擦掉写"无符号整数"时它是整数，白板本身只有一块、大小固定。
- **三大用途**：
- **节省空间**：多个互斥的字段共用一块存储，如 `value_t { bool is_float; union {float f; unsigned u;} val; }`。
- **类型双关（type punning）**：把同一串位按不同类型解读。`bit_float_t { float f; unsigned u; }` 里的 `bit2float(u)` 返回的是**按位重解释**的浮点值，与 `(float) u`（整数转浮点，会做数值转换）**完全不同**——这是讲义明确点出的、学生最容易搞错的地方。
- **和类型（sum type / tagged union）**：用一个标签字段记录当前哪个成员有效。
- **字节序影响**：小端序下，联合体的 `unsigned char c[8]` 与 `unsigned int i[2]` 会给出完全不同的读数。讲义示例把 `c[0..7]` 设为 `0xf0..0xf7` 后：

```
Characters 0-7 == [0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7]
Shorts     0-3 == [0xf1f0,0xf3f2,0xf5f4,0xf7f6]
Ints       0-1 == [0xf3f2f1f0,0xf7f6f5f4]
Long       0   == [0xf7f6f5f4f3f2f1f0]
```

最低有效字节（LSB）在最低地址，这正是小端序（Little Endian，x86-64 采用）的定义；而在大端序的 Sun/Sparc 上同一段代码会打印 `[0xf0f1f2f3, ...]`。做网络协议或读写二进制文件时，"字节序不匹配"就是从这里来的。

- **与机器码/硬件的对应**：联合体的成员访问**不需要任何指令**——所有成员偏移都是 0，联合体本身只是一块内存的别名。

6.2.6 IEEE 754 浮点表示（Floating Point Representation）

- **定义与目的**：用有限位数近似表示实数。CS:APP3e 3.11 的浮点格式把位域切成三部分：符号位 s、阶码（exponent）`exp`、尾数（fraction）`frac`。
- **直观解释**：科学计数法 $1.0101_2 \times 2^5$ 的二进制版本——尾数保存"有效数字"，阶码保存"小数点该往哪挪"。阶码用**偏置（bias）**编码，好处是能让浮点数在不做符号处理的整数比较电路上直接比较大小。
- **底层机制图解**：

```
IEEE 754 双精度 double —— 64 位
 63    62                   52 51                                      0
+----+------------------------+-----------------------------------------+
| s  |          exp           |                 frac                    |
| 1  |          11            |                  52                     |
+----+------------------------+-----------------------------------------+
 MSB                                                                   LSB

IEEE 754 单精度 float —— 32 位
 31   30              23 22                                           0
+----+-----------------+-----------------------------------------------+
| s  |       exp       |                   frac                        |
| 1  |        8        |                    23                         |
+----+-----------------+-----------------------------------------------+
```

- **四类数的判定与求值公式**（bias = 127 / 1023）：

`exp`	`frac`	类别	值 V
$\mathbf{0}$	$\mathbf{0}$	$\pm\mathbf{0}$	$V = (-1)^s \cdot 0$（有 +0 与 −0 两个零）
$\mathbf{0}$	$\neq \mathbf{0}$	**非规格化数**（denormalized）	$V = (-1)^s \times 0.\mathbf{frac} \times 2^{1-\text{bias}}$
$\mathbf{1} \sim \mathbf{2^k - 2}$	任意	**规格化数**（normalized）	$V = (-1)^s \times M \times 2^E$，$M = 1.\mathbf{frac}$，$E = \mathbf{exp} - \text{bias}$
全 1	$\mathbf{0}$	$\pm\infty$	$V = \pm\infty$（如 $1.0/0.0$）
全 1	$\neq \mathbf{0}$	NaN（Not a Number）	如 $0.0/0.0$、$\infty - \infty$

规格化数的尾数**隐含前导 1**（$M = 1.\mathbf{frac}$），这一位不存储，白赚一个有效位；代价是 0 无法表示，于是让 `exp=0` 专门表示非规格化数，它用 $M = 0.\mathbf{frac}$ 且阶码固定为最小值——这样 0 与最小规格化数之间的间距是**均匀**的，称为渐进下溢（gradual underflow）。

- **与机器码/硬件的对应**：浮点数放在 `%xmm0` — `%xmm15` 这 16 个 128 位 SSE 寄存器里；标量 `float` 用低 32 位、`double` 用低 64 位，高位置零。

6.2.7 浮点寄存器与指令（Floating-Point Registers and Instructions）

- **寄存器约定**：整型参数用 `%rdi, %rsi, %rdx, %rcx, %r8, %r9`，**浮点参数一律用** `%xmm0` — `%xmm7`（各自独立计数），浮点返回值放在 `%xmm0`。
- **指令族**：
- 数据传送：`movss`（单精度标量，move scalar single）、`movsd`（双精度标量）、`movaps` / `movapd`（对齐的打包传送，要求 16 字节对齐）、`movups` / `movupd`（非对齐的打包传送，unaligned）。
- 算术：`addss` / `addsd`、`subss` / `subsd`、`mulss` / `mulsd`、`divss` / `divsd`——助记符后缀 `ss` = scalar single，`sd` = scalar double。
- 类型转换：`cvtss2sd`（float→double）、`cvtsd2ss`（double→float）、`cvtsi2sd`（int→double）、`cvttsd2si`（double→int，**截断**，truncate）。
- **真实 `gcc -Og -S` 输出（本机实测）**：

```
poly:                       # double poly(double a, float x, double b)
    cvtss2sd  %xmm1, %xmm1  #   float 参数先扩成 double
    mulsd     %xmm0, %xmm1  #   a * (double)x
    movapd    %xmm1, %xmm0
    addsd     %xmm2, %xmm0  #   + b
    ret                     #   返回值已在 %xmm0

d2f:  cvtsd2ss  %xmm0, %xmm0 ; ret    # (float)d
f2d:  cvtss2sd  %xmm0, %xmm0 ; ret    # (double)f
i2d:  pxor %xmm0,%xmm0 ; cvtsi2sdl %edi,%xmm0 ; ret   # (double)i
d2i:  cvttsd2sil %xmm0, %eax ; ret                    # (int)d（向零截断）
```

注意 `i2d` 里那句 `pxor %xmm0,%xmm0`：`cvtsi2sd` 只写目标寄存器的低 64 位，必须先把高位清零，否则会留下上一次运算的垃圾位（伪依赖）。

6.2.8 进程内存布局与缓冲区溢出（Memory Layout and Buffer Overflow）

- **x86-64 Linux 地址空间**（讲义图，非按比例）：从高地址到低地址依次是**栈**（runtime stack，8 MB 上限，存局部变量）、**共享库**、**堆**（`malloc` / `calloc` / `new` 动态分配，向高地址增长）、**数据段**（全局变量、`static` 变量、字符串常量）、**代码段 Text**（只读的可执行指令）。栈基址和共享库位置都做了随机化。讲义给出的真实量级：局部变量 `0x00007ffe4d3be87c`、`malloc(2^28)` 得到 `0x00007f7262a1e010`、全局 `big_array` 在 `0x0000000080601060`、`main()` 代码在 `0x000000000040060c`。
- **漏洞原型**：`char buf[4]; gets(buf);` 中 `gets` 完全没有长度限制，用户输入直接从 `buf` 开始向**高地址**写，越界后依次覆盖未使用空间、金丝雀、保存的寄存器、返回地址。
- **底层机制图解**：

```
        +-------------------------+ 高地址
        |  call_echo 的栈帧 ...   |
        +-------------------------+
        |  返回地址 (8 字节)      |  echo 的 ret 从这里弹地址
        |  00 40 06 c3 00 00 00 00|
        +-------------------------+  <- %rsp + 0x18
        |  20 字节未使用(或金丝雀) |
        +-------------------------+
        |  buf[4]  '3''2''1''0'   |  <- %rsp (gets 从这里开始写)
        +-------------------------+ 低地址
          gets 的写入方向  ──────────────►  (向高地址增长)

 不越界:  24 字节内写完，程序正常返回
 越界:    '0'..'2' 共 24 字节刚好填满；再多 4 字节覆盖返回地址低 4 字节
          -> 程序"返回"到 0x0400600 这类非法地址 -> Segmentation fault
```

- **三层防御**：① 写安全的代码（用 `fgets` 代替 `gets`、`strncpy` 代替 `strcpy`、别用 `%s` 格式符）；② 系统级保护——**栈随机化**（每次运行栈地址都不同，攻击者猜不到注入代码的位置）与**不可执行内存**（x86-64 加 NX 位，栈标记为不可执行，跳进去立即崩溃）；③ 编译器插入**栈金丝雀**（`-fstack-protector`，现为默认）：函数入口把 `%fs:0x28` 处的哨兵值存到缓冲区与返回地址之间，返回前 `xor` 比较，被改动就调用 `__stack_chk_fail` 终止进程。
- **ROP 绕过**：金丝雀保护不了"不越过缓冲区就无法写入"的攻击者，但栈不可执行挡住了代码注入。ROP 转而**复用已有代码**：在可执行段中寻找"一串指令 + `ret`（编码为单字节 `0xc3`）"的**小工具（gadget）**，把它们在栈上串成链。讲义的两个例子：`ab_plus_c` 中 `0x4004d4` 处的 `lea (%rdi,%rdx,1),%rax; ret` 实现 `%rax ← %rdi + %rdx`；`setval` 的立即数字节里恰好嵌着 `48 89 c7`（`movq %rax,%rdi`），所以 `0x4004dc` 处存在一个 `mov %rax,%rdi; ret` 的小工具——**x86-64 变长指令集允许从指令中间开始解码**，这是 ROP 的物理基础。

```
ROP 执行模型（每个 gadget 以 c3 = ret 结尾）
        gadget 1 code ... c3  ─┐
        gadget 2 code ... c3  ─┼─► 每条 ret 从栈上弹出下一个 gadget 地址
        gadget n code ... c3  ─┘
 栈:  [g1 地址][g2 地址][g3 地址]...   %rsp 依次向高地址推进
```

6.3 代码示例与底层机制分析

6.3.1 示例一：数组寻址与行优先二维数组

代码（`/tmp/c06/arr.c`，`gcc -g -Wall -std=c11 arr.c -o arr` 编译通过）：

```c
/* c06_arrays.c — 数组寻址、指针算术与行优先二维数组 */
#include <stdio.h>
#include <stddef.h>

#define ZLEN 5
typedef int zip_dig[ZLEN];

#define R 4
#define C 5
static int A[R][C] = {
    {1, 5, 2, 0, 6}, {1, 5, 2, 1, 3},
    {1, 5, 2, 1, 7}, {1, 5, 2, 2, 1}
};
static zip_dig pgh[R] = {
    {1, 5, 2, 0, 6}, {1, 5, 2, 1, 3},
    {1, 5, 2, 1, 7}, {1, 5, 2, 2, 1}
};

int  get_digit(zip_dig z, int digit) { return z[digit]; }
int *get_pgh_row(int index)          { return pgh[index]; }
int  get_pgh_digit(int index, int d) { return pgh[index][d]; }

int main(void)
{
    printf("sizeof(zip_dig) = %zu  (= 5 * sizeof(int))\n", sizeof(zip_dig));
    printf("sizeof(A)       = %zu  (= R * C * sizeof(int))\n", sizeof(A));
    printf("&A[2][3] = %p  (+%zu bytes from A[0][0])\n",
           (void *)&A[2][3], (size_t)((char *)&A[2][3] - (char *)&A[0][0]));
    printf("公式预测偏移 = (2*C+3)*4 = %zu bytes\n", (size_t)((2 * C + 3) * 4));

    int *p = &A[0][0];                      /* 指针算术：增量 = sizeof(*p) */
    printf("p=%p  p+1=%p  差值=%zu bytes\n",
           (void *)p, (void *)(p + 1), (size_t)((char *)(p + 1) - (char *)p));
    printf("&A[2][3]==A[2]+3 ? %d\n", &A[2][3] == A[2] + 3);
    printf("A[2][3]==*(*(A+2)+3) ? %d\n", A[2][3] == *(*(A + 2) + 3));

    printf("get_pgh_digit(1,2) = %d\n", get_pgh_digit(1, 2));
    int (*rowp)[C] = &A[3];                 /* 指向整行的指针 */
    printf("(*rowp)[4]         = %d\n", (*rowp)[4]);
    return 0;
}
```

【代码做什么？】 1. `zip_dig` 是 `int[5]` 的类型别名，`sizeof(zip_dig)` 得 20；`sizeof(A)` 得 80。 2. 打印 `&A[2][3]` 与首地址之差，验证它等于公式 `(i*C+j)*4 = 52`。 3. 打印 `p` 与 `p+1` 的差，证明指针加 1 走的是 4 字节而非 1 字节。 4. 用两条恒等式 `&A[i][j] == A[i]+j`、`A[i][j] == *(*(A+i)+j)` 验证"数组的数组"语义。 5. 用 `int (*rowp)[C]` 说明"指向整行的指针"与"指向 int 的指针"类型不同、步长不同。

【底层机制透视】 `A[i]` 不是运行时计算出来的对象，而是编译期类型推导出的表达式：`A[i]` 的类型是 `int[5]`，进一步退化成 `int *`，值为 `A + i*20`。整条链最后塌缩成一条比例变址寻址指令，没有循环、没有函数调用。

【内存布局 / 数据结构图解】（本机实测地址，栈上数组，每次运行可能不同）：

```
sizeof(A) = 80
row 0: A[0]=0x404040  A[0]+C=0x404054  (20 bytes/row)
row 1: A[1]=0x404054  A[1]+C=0x404068  (20 bytes/row)
row 2: A[2]=0x404068  A[2]+C=0x40407c  (20 bytes/row)
row 3: A[3]=0x40407c  A[3]+C=0x404090  (20 bytes/row)
&A[2][3] = 0x404074   ( +52 bytes = 13*4 )
```

【与汇编 / 硬件的对应】 `gcc -Og -S` 生成的真实片段：

```
get_digit:                      # int get_digit(int z[], int digit)
    movslq  %esi, %rsi
    movl    (%rdi,%rsi,4), %eax  # z[digit]：一条比例变址指令
    ret

get_pgh_row:                    # int *get_pgh_row(int index)
    movslq  %edi, %rdi
    leaq    (%rdi,%rdi,4), %rax  # 5 * index        （强度削弱）
    leaq    pgh(,%rax,4), %rax   # pgh + 20*index
    ret

get_pgh_digit:                  # int get_pgh_digit(int index, int dig)
    movslq  %esi, %rsi
    movslq  %edi, %rdi
    leaq    (%rdi,%rdi,4), %rax  # 5*index
    addq    %rsi, %rax           # 5*index + dig
    movl    pgh(,%rax,4), %eax   # M[pgh + 4*(5*index+dig)]
    ret
```

三处细节值得记住：`movslq` 把 32 位 `int` 参数符号扩展到 64 位才能当变址寄存器使用；`leaq (%rdi,%rdi,4)` 是"乘 5"的标准写法（`x + x*4`）；全局符号 `pgh` 的地址由链接器在重定位时填入，汇编器只留下 `pgh(,%rax,4)` 这样的待填坑。

6.3.2 示例二：结构体填充与 `offsetof`

代码（`/tmp/c06/strct.c`，`gcc -g -Wall -std=c11 strct.c -o strct` 编译通过）：

```c
/* c06_struct.c — 结构体填充、offsetof 与结构体数组 */
#include <stdio.h>
#include <stddef.h>

struct S1 { char c; int i[2]; double v; };   /* 讲义 S1, K = 8 */
struct S2 { long v; int i[2]; char c; };     /* 讲义 S2, 尾部外部填充 */
struct S3 { short i; float v; short j; };    /* 讲义 S3, sizeof = 12 */
struct S4 { char c; int i; char d; };        /* 12 字节 */
struct S5 { int i; char c; char d; };        /* 8 字节 */

struct rec { int a[4]; size_t i; struct rec *next; };

int *get_ap(struct rec *r, size_t idx) { return &r->a[idx]; }

long length(struct rec *r)
{
    long len = 0;
    while (r) { len++; r = r->next; }
    return len;
}

int main(void)
{
    printf("sizeof(S1)=%zu  offsets c/i/v = %zu/%zu/%zu  K=%zu\n",
           sizeof(struct S1), offsetof(struct S1, c),
           offsetof(struct S1, i), offsetof(struct S1, v),
           _Alignof(struct S1));
    printf("sizeof(S2)=%zu  offsets v/i/c = %zu/%zu/%zu\n",
           sizeof(struct S2), offsetof(struct S2, v),
           offsetof(struct S2, i), offsetof(struct S2, c));
    printf("sizeof(S3)=%zu  offsetof j = %zu\n",
           sizeof(struct S3), offsetof(struct S3, j));
    printf("sizeof(S4)=%zu  sizeof(S5)=%zu\n",
           sizeof(struct S4), sizeof(struct S5));
    printf("sizeof(rec)=%zu  offsets a/i/next = %zu/%zu/%zu\n",
           sizeof(struct rec), offsetof(struct rec, a),
           offsetof(struct rec, i), offsetof(struct rec, next));

    struct S3 arr[10];
    printf("sizeof arr[10]=%zu  &arr[3]-&arr[0]=%zu bytes\n",
           sizeof(arr), (size_t)((char *)&arr[3] - (char *)&arr[0]));
    printf("arr[3].j 地址偏移 = %zu\n",
           (size_t)((char *)&arr[3].j - (char *)&arr[0]));

    struct rec n1 = {{1, 2, 3, 4}, 0, NULL};
    struct rec n0 = {{9, 9, 9, 9}, 1, &n1};
    printf("get_ap(&n0,2)==&n0.a[2] ? %d   length(&n0)=%ld\n",
           get_ap(&n0, 2) == &n0.a[2], length(&n0));
    return 0;
}
```

【代码做什么？】 1. 用 `offsetof` 把编译器暗中插入的填充"挖出来"：`S1` 的 `i` 在偏移 4 而不是 1（前面 3 字节填充），`v` 在偏移 16 而不是 12（又 4 字节填充）。 2. 打印 `_Alignof(S1) = 8`，印证 K = 最大成员对齐值。 3. 对比 `S4`（12 字节）与 `S5`（8 字节）——**同样的成员、不同的顺序，大小差 4 字节**。 4. 对结构体数组 `arr[10]` 验证元素间距恰为 `sizeof(S3) = 12`，没有额外填充。 5. `get_ap` / `length` 演示成员访问就是"基址 + 常量偏移"。

【底层机制透视】 `offsetof` 是一个宏，展开后通常等价于 `((size_t)&((struct S *)0)->member)`——把空指针当基址取成员地址，得到的整数就是偏移量。它必须是编译期常量，因为链接器和优化器都要用它。填充字节的内容是**未定义的**：`memcmp` 两个"字段相同"的结构体可能返回非零，用结构体做哈希键或直接 `write` 到文件都会踩这个坑。

【内存布局 / 数据结构图解】（`struct S1` 真实字节转储，`c='A'`，`i[0]=0x11223344`，`i[1]=0x55667788`，`v=1.0`；小端序）：

```
  +0  0x41   <- c ('A')
  +1  0x00   <- padding
  +2  0x00   <- padding
  +3  0x00   <- padding
  +4  0x44   <- i[0] 低字节
  +5  0x33   <- i[0]
  +6  0x22   <- i[0]
  +7  0x11   <- i[0] 高字节
  +8  0x88   <- i[1] 低字节
  +9  0x77   <- i[1]
  +10 0x66   <- i[1]
  +11 0x55   <- i[1] 高字节
  +12 0x00   <- padding
  +13 0x00   <- padding
  +14 0x00   <- padding
  +15 0x00   <- padding
  +16 0x00 \
  +17 0x00  |
  +18 0x00  |  v = 1.0 的 IEEE 754 双精度位模式
  +19 0x00  |  0x3FF0000000000000，小端序存储
  +20 0x00  |
  +21 0x00  |
  +22 0xf0  |
  +23 0x3f /
```

【与汇编 / 硬件的对应】 `gcc -Og -S` 的真实输出，印证"偏移在编译期确定、汇编里看不到字段名"：

```
get_ap:                         # &r->a[idx]
    leaq  (%rdi,%rsi,4), %rax # r + 4*idx，偏移 0 直接省略
    ret

length:                         # while (r) { len++; r = r->next; }
    movl  $0, %eax
    jmp   .L3
.L4:
    addq  $1, %rax
    movq  24(%rdi), %rdi      # r = Mem[r+24]：next 的偏移是常量 24
.L3:
    testq %rdi, %rdi
    jne   .L4
    ret
```

【实测验证】 两条命令就能把编译器插入的填充"照"出来。第一条是 `-Wpadded`（本机实测输出）：

```
$ gcc -Wall -Wpadded -c strct.c
strct.c:5:25: warning: padding struct to align 'i' [-Wpadded]
    5 | struct S1 { char c; int i[2]; double v; };
      |                         ^
strct.c:5:38: warning: padding struct to align 'v' [-Wpadded]
strct.c:6:8:  warning: padding struct size to alignment boundary [-Wpadded]
```

第二条是在 GDB 里让调试信息自己画出空洞（`ptype /o` = 显示偏移）：

```
$ gcc -g -std=c11 gdbt.c -o gdbt
$ gdb -q -batch -ex 'break main' -ex run -ex next \
      -ex 'p sizeof(struct S1)' -ex 'p/x (long)((char*)&s.v-(char*)&s)' \
      -ex 'ptype /o struct S1' -ex 'x/24xb &s' ./gdbt
$1 = 24
$2 = 0x10                                   # v 的偏移 = 16
/* offset    |  size */  type = struct S1 {
/*    0      |     1 */    char c;
/* XXX  3-byte hole  */
/*    4      |     8 */    int i[2];
/* XXX  4-byte hole  */
/*   16      |     8 */    double v;
                           /* total size (bytes):   24 */
                         }
0x7fffffffb6b0: 0x01 0x00 0x00 0x00 0x40 0x00 0x00 0x00
0x7fffffffb6b8: 0x00 0x00 0x00 0x10 0x00 0x00 0x00 0x00
0x7fffffffb6c0: 0x00 0x08 0x00 0x00 0x00 0x00 0x00 0x00
```

`pahole`（显示每个成员偏移与空洞）或 `ptype /o` 是两条最省事的核对途径。注意 GDB 只有在函数体内**出现过该结构体变量**时才会保留其完整类型信息，所以 `sizeof(struct S1)` 要在 `struct S1 s;` 已被执行的位置求值。

6.3.3 示例三：IEEE 754 位域与浮点精度丢失

代码（`/tmp/c06/flt.c`，`gcc -g -Wall -std=c11 flt.c -o flt -lm` 编译通过）：

```
/* c06_float.c — IEEE 754 位域、规格化/非规格化、精度丢失与结合律失效 */
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdint.h>

typedef union { float f; unsigned u; }              bit_float_t;   /* 类型双关 */
typedef union { double d; unsigned long long u; }   bit_double_t;

float    bit2float(unsigned u)  { bit_float_t a; a.u = u; return a.f; }
unsigned float2bit(float f)     { bit_float_t a; a.f = f; return a.u; }
double   bits2double(unsigned long long u) { bit_double_t a; a.u = u; return a.d; }
unsigned long long double2bits(double d)   { bit_double_t a; a.d = d; return a.u; }

/* 由 s / exp / frac 手工拼出 IEEE 754 单精度值 */
static float build_float(unsigned s, unsigned exp, unsigned frac)
{
    return bit2float((s << 31) | ((exp & 0xff) << 23) | (frac & 0x7fffff));
}

static void show_double(const char *n, double x)
{
    unsigned long long u = double2bits(x);
    printf("%-8s d=%-8g bits=0x%016llx s=%llu exp=0x%03llx frac=0x%013llx\n",
           n, x, u, u >> 63, (u >> 52) & 0x7ff, u & 0xfffffffffffffULL);
}

int main(void)
{
    printf("sizeof(float)=%zu  sizeof(double)=%zu\n", sizeof(float), sizeof(double));
    printf("FLT_EPSILON=%.10g  DBL_EPSILON=%.17g\n\n",
           1.19209290e-7, 2.2204460492503131e-16);

    show_double("3.14", 3.14);
    show_double("1.0", 1.0);
    show_double("-0.0", -0.0);
    show_double("1.0/0.0", 1.0 / 0.0);
    show_double("0.0/0.0", 0.0 / 0.0);

    printf("\n-- 手工拼装 V = (-1)^s * M * 2^E --\n");
    printf("s=0 exp=127 frac=0        -> %g\n", build_float(0, 127, 0));
    printf("s=0 exp=127 frac=0x400000 -> %g  (M=1.5)\n", build_float(0, 127, 0x400000));
    printf("s=0 exp=128 frac=0        -> %g  (E=+1)\n", build_float(0, 128, 0));
    printf("s=0 exp=126 frac=0x400000 -> %g  (E=-1, M=1.5)\n", build_float(0, 126, 0x400000));
    printf("s=0 exp=0   frac=1        -> %g  (denorm = 2^-149)\n", build_float(0, 0, 1));
    printf("s=0 exp=0   frac=0x7fffff -> %g  (最大非规格化数)\n", build_float(0, 0, 0x7fffff));
    printf("s=0 exp=1   frac=0        -> %g  (最小规格化数 = 2^-126)\n", build_float(0, 1, 0));

    printf("\n-- 十进制有效位与精度丢失 --\n");
    printf("float  1/3 = %.10f\n", (double)(1.0f / 3.0f));
    printf("double 1/3 = %.20f\n", 1.0 / 3.0);
    printf("(int)16777217 -> float, 再转回 int = %d   (2^24+1 被舍入)\n",
           (int)(float)16777217);
    printf("(float)0.1 = %.20f\n", (double)0.1f);
    printf("0.1+0.2 == 0.3 ?  %d\n", (0.1 + 0.2 == 0.3));
    printf("0.1+0.2 = %.20f\n", 0.1 + 0.2);
    printf("0.3     = %.20f\n", 0.3);
    printf("fabs((0.1+0.2)-0.3) < 1e-9 ? %d\n", fabs((0.1 + 0.2) - 0.3) < 1e-9);

    printf("\n-- 结合律失效 --\n");
    double a = 1e20, b = -1e20, c = 3.14;
    printf("(a+b)+c = %.10f\n", (a + b) + c);
    printf("a+(b+c) = %.10f  相等? %d\n", a + (b + c),
           ((a + b) + c) == (a + (b + c)));
    return 0;
}
```

【代码做什么？】 先用 `union` 做类型双关把 `float` / `double` 的位模式打印出来，再用 `build_float` 手工拼 `s` / `exp` / `frac` 验证求值公式，最后用 `2^24+1`、`0.1+0.2`、`1e20+(-1e20+3.14)` 三个经典例子演示精度丢失。

【实测输出（本机 GCC 12.2.0 / x86-64）】：

```
sizeof(float)=4  sizeof(double)=8

---- 双精度位域划分 ----
3.14     d=3.14   bits=0x40091eb851eb851f s=0 exp=0x400 frac=0x91eb851eb851f
1.0      d=1      bits=0x3ff0000000000000 s=0 exp=0x3ff frac=0x0000000000000
-0.0     d=-0     bits=0x8000000000000000 s=1 exp=0x000 frac=0x0000000000000
1.0/0.0  d=inf    bits=0x7ff0000000000000 s=0 exp=0x7ff frac=0x0000000000000
0.0/0.0  d=-nan   bits=0xfff8000000000000 s=1 exp=0x7ff frac=0x8000000000000

---- 单精度位域划分 ----
3.14     f=3.14   bits=0x4048f5c3 s=0 exp=0x80 frac=0x48f5c3
0.1f     f=0.1    bits=0x3dcccccd s=0 exp=0x7b frac=0x4ccccd
+0.0     f=0      bits=0x00000000 s=0 exp=0x00 frac=0x000000
-0.0     f=-0     bits=0x80000000 s=1 exp=0x00 frac=0x000000
inf      f=inf    bits=0x7f800000 s=0 exp=0xff frac=0x000000

---- 手工拼装: V = (-1)^s * M * 2^E ----
s=0 exp=127 frac=0        -> 1
s=0 exp=127 frac=0x400000 -> 1.5
s=0 exp=128 frac=0        -> 2
s=0 exp=126 frac=0x400000 -> 0.75
s=0 exp=0   frac=1        -> 1.4013e-45  (denorm, 2^-149)
s=0 exp=0   frac=0x7fffff -> 1.17549e-38 (最大非规格化数)
s=0 exp=1   frac=0        -> 1.17549e-38 (最小规格化数 2^-126)

---- 十进制有效位与精度丢失 ----
float  1/3 = 0.3333333433
double 1/3 = 0.33333333333333331483
(int)16777217 -> float, 再转回 int = 16777216   (2^24+1 被舍入)
(float)0.1 = 0.100000001490116119 38
0.1+0.2 == 0.3 ?  0
0.1+0.2 = 0.30000000000000004441
0.3     = 0.29999999999999998890
fabs((0.1+0.2)-0.3) < 1e-9 ? 1

---- 结合律失效 ----
(a+b)+c = 3.1400000000
a+(b+c) = 0.0000000000  相等? 0
```

【底层机制透视】 三条硬结论：

1. `float` **只有 24 位有效位**（隐含前导 1 + 23 位 `frac`），对应约 $\log_{10} 2^{24} \approx 7.22$ 位十进制有效数字，所以 `float` 约 **7 位十进制有效数字**；`double` 有 53 位有效位，对应 $\log_{10} 2^{53} \approx 15.95$，即约 **15–16 位十进制有效数字**。上面的 `float 1/3 = 0.3333333433` 从第 8 位开始就是噪声。
2. `int` → `float` **会丢精度**：$2^{24} + 1 = 16777217$ 需要 25 位有效位，`float` 只能四舍五入到 $2^{24} = 16777216$。因此"用 `float` 存整数计数器"在数值超过 $2^{24} \approx 1.68 \times 10^7$ 后就会静默出错。
3. **浮点加法不满足结合律**：1.0×10^{20} 与 3.14 的指数相差约 66，3.14 对齐后完全落在 `double` 的 53 位有效位之外，被舍成 0。这就是 Lecture 1 的 "Ints are not Integers, Floats are not Reals" 的具体机制。**浮点只满足有序性（单调性、符号），不满足结合律和分配律**——把 `x+y+z` 改写成 `x+(y+z)` 是编译器**不允许**做的变换。

【内存布局 / 数据结构图解】 `double 3.14` 的位域分解：

```
0x40091EB851EB851F
 |└──┬──┘└──────────┬──────────┘
 s=0   exp=0x400=1024          frac=0x91EB851EB851F
       E = 1024 - 1023 = 1
       M = 1.91EB851EB851F(16) = 1.57
 V = (+1) * 1.57 * 2^1 = 3.14

按字节（小端序）: 1F 85 EB 51 B8 1E 09 40
                  ↑ 最低地址存最低有效字节
                  LSB                          MSB
```

【与汇编 / 硬件的对应】 类型转换的真实指令序列，见 6.2.7 的 `gcc -Og -S` 输出。特别提醒 `d2i: cvttsd2sil` ——**向零截断**（truncate），`(int)3.9 = 3`、`(int)(-3.9) = -3`；C 语言里唯一的四舍五入是 `round()`，而 `cvtsd2si`（不带 `t`）才使用当前舍入模式。

6.3.4 示例四：结构体越界的安全含义（⚠️ 仅供演示，请勿模仿）

代码（`/tmp/c06/oob.c`，`gcc -g -Wall -std=c11 -fno-stack-protector oob.c -o oob`）：

```
/* ⚠️ 仅供演示，请勿模仿：故意制造结构体数组越界写 */
#include <stdio.h>
#include <stddef.h>
typedef struct { int a[2]; double d; } struct_t;

double fun(int i) {
    volatile struct_t s;
    s.d = 3.14;
    s.a[i] = 1073741824;   /* UB：i >= 2 时越界，实际结果依编译器/机器而异 */
    return s.d;
}
int main(void) {
    printf("offsetof(a)=%zu  offsetof(d)=%zu  sizeof=%zu\n",
           offsetof(struct_t, a), offsetof(struct_t, d), sizeof(struct_t));
    printf("fun(0) -> %.10f\n", fun(0));
    printf("fun(1) -> %.10f\n", fun(1));
    printf("fun(2) -> %.10f   (a[2] 踩到 d 的低 4 字节)\n", fun(2));
    printf("fun(3) -> %.10f   (a[3] 踩到 d 的高 4 字节)\n", fun(3));
    return 0;
}
```

【代码做什么？】 `struct_t` 里 `a` 占偏移 0—7、`d` 占偏移 8—15。写 `a[2]` 就是写偏移 8，正好是 `d` 的低 4 字节；写 `a[3]` 就是偏移 12，即 `d` 的高 4 字节。再往后（`a[4]`、`a[5]`…）就冲出结构体，进入返回地址和保存寄存器。

【实测输出（本机）】：

```
offsetof(a)=0  offsetof(d)=8  sizeof=16
fun(0) -> 3.1400000000
fun(1) -> 3.1400000000
fun(2) -> 3.1399998665   (a[2] 踩到 d 的低 4 字节)
fun(3) -> 2.0000006104   (a[3] 踩到 d 的高 4 字节)
```

【底层机制透视】 讲义给出的完整表是 `fun(0)(1) -> 3.14`、`fun(2) -> 3.1399998665`、`fun(3) -> 2.0000006104`、`fun(4) -> Segmentation fault`、`fun(6) -> Stack smashing detected`、`fun(8) -> Segmentation fault`（不同版本略有差异）。**结果随系统而异**，因为越界后踩到的是栈帧里的其他东西——可能是编译器留的空隙、可能是金丝雀、可能是返回地址。这正是 C 的"未定义行为（Undefined Behavior）"的典型形态：编译器可以假设你没有越界，从而做出任何优化决策。

为什么这是"大问题"：缓冲区溢出曾是**技术层面第一号安全漏洞来源**（总体上第一号原因是社会工程学与用户疏忽）。最常见形态是字符串输入没有长度检查，尤其是栈上的定长字符数组——俗称**栈击穿（stack smashing）**。讲义列出的历史案例包括：1988 年 Morris 蠕虫通过 `fingerd` 的 `gets()` 在几小时内感染了约 6000 台机器（当时互联网的 10%），并直接促成了 CERT 在 CMU 成立；1999 年 AOL 与微软的"IM 战争"中，AOL 甚至利用自家 AIM 客户端的缓冲区溢出漏洞把 4 字节签名回传给服务器以识别并封堵 MSN 客户端。**这些漏洞的代价是真实的**，Attack Lab 的用意就是让你亲手打穿一次，从而再也不会写出这种代码。

【内存布局 / 数据结构图解】 越界位置与后果：

```
struct_t 在栈上（16 字节），%rsp 指向 a[0]
偏移 +0  +1  +2  +3  +4  +5  +6  +7 | +8  +9 +10 +11 +12 +13 +14 +15
     +-------------------------------+-------------------------------+
     |           a[0]                |           a[1]                |   ← 合法范围
     +-------------------------------+-------------------------------+
      ↑ a[0]                          ↑ a[1]                  ↑ d 开始(偏移8)
偏移 +8  +9 +10 +11 +12 +13 +14 +15
     +-------------------------------+-------------------------------+
     |         d 的低 4 字节          |         d 的高 4 字节          |
     +-------------------------------+-------------------------------+
      ↑ a[2] 落点（污染 d 低位）        ↑ a[3] 落点（污染 d 高位）

越界下标 →  写入位置
  a[2]  ->  偏移  8  ->  d 低位            -> d 变成 3.1399998665
  a[3]  ->  偏移 12  ->  d 高位            -> d 变成 2.0000006104
  a[4]  ->  偏移 16  ->  栈帧空隙           -> 可能无事发生
  a[6]  ->  偏移 24  ->  金丝雀             -> *** stack smashing detected ***
  a[8]  ->  偏移 32  ->  返回地址附近        -> Segmentation fault
```

6.3.5 示例五：联合体的字节序可视化

代码（`/tmp/c06/dw.c`，讲义原始示例，`gcc -g -Wall -std=c11 dw.c -o dw` 编译通过）：

```
#include <stdio.h>
int main(void) {
    union { unsigned char c[8]; unsigned short s[4];
            unsigned int i[2];  unsigned long l[1]; } dw;
    for (int j = 0; j < 8; j++) dw.c[j] = 0xf0 + j;
    printf("Characters 0-7 == [0x%x,0x%x,0x%x,0x%x,0x%x,0x%x,0x%x,0x%x]\n",
        dw.c[0], dw.c[1], dw.c[2], dw.c[3], dw.c[4], dw.c[5], dw.c[6], dw.c[7]);
    printf("Shorts     0-3 == [0x%x,0x%x,0x%x,0x%x]\n",
        dw.s[0], dw.s[1], dw.s[2], dw.s[3]);
    printf("Ints       0-1 == [0x%x,0x%x]\n", dw.i[0], dw.i[1]);
    printf("Long       0   == [0x%lx]\n", dw.l[0]);
    printf("sizeof(dw)=%zu\n", sizeof dw);
    return 0;
}
```

【实测输出（x86-64，小端序）】：

```
Characters 0-7 == [0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7]
Shorts     0-3 == [0xf1f0,0xf3f2,0xf5f4,0xf7f6]
Ints       0-1 == [0xf3f2f1f0,0xf7f6f5f4]
Long       0   == [0xf7f6f5f4f3f2f1f0]
sizeof(dw)=8
```

【底层机制透视】 联合体的 8 字节始终是 `f0 f1 f2 f3 f4 f5 f6 f7`（`c[]` 视图永远这么读）。但按 `short` 视图，每个 2 字节组被解释成"高地址字节当高位"，于是 `0xf1f0`；按 `long` 视图，最高地址的 `0xf7` 成了最高有效字节。若换到大端序机器（Sparc/Sun），`Shorts` 会打印 `[0xf0f1,...]`、`Long` 会打印 `[0xf0f1f2f3...]`。**结论**：在多机通信或读写二进制文件时，必须在协议层显式约定字节序（Internet 就是大端序），不要依赖本机的 `union` 行为。

6.4 实验关联

L3 Attack Lab 是本讲的直接落地，共 5 关、100 分：前三关对 `ctarget` 做**代码注入（Code Injection, CI）**，后两关对 `rtarget` 做 **ROP**。讲义在 `F25-06` 中把漏洞原型、跳转机理和小工具构造都讲到了，实验只是把它做一遍（本讲无独立 new lab；Attack Lab 与 Lecture 5 共同构成本章实验基础）。

- **Phase 1（touch1，10 分）**：只改返回地址。要先用 `objdump -d ctarget` 找到 `getbuf` 的 `sub $0xNN,%rsp`，算出 `buf` 起始到返回地址的字节数，再把 `touch1` 的地址按**小端序**写在对应位置。本讲 6.2.8 的栈图就是解题模板。坑：`BUFFER_SIZE` 是编译期常量，`buf` 的位置随 GCC 分配策略变化，必须看反汇编而不是猜。
- **Phase 2（touch2，25 分）**：要"像"把 cookie 当第一个参数传进去，即 `%rdi = cookie`。注入的机器码应先把 cookie 装进 `%rdi`，再用 `ret` 跳到 `touch2`。讲义明确警告：**不要用 `jmp` / `call`**，因为它们的相对偏移难以手算，一切控制转移都用 `ret`。本讲的 `movq` / `ret` 编码知识在这里第一次用上。
- **Phase 3（touch3，25 分）**：参数是 cookie 的**字符串**表示（8 位十六进制、无 `0x` 前缀、以 `\0` 结尾）。难点在于 `hexmatch` 会在栈上开 `char cbuf[110]` 并调用 `sprintf` / `strncmp`，把你的缓冲区踩掉，所以字符串必须放在不会被覆盖的位置（通常放在返回地址之后的高地址处）。
- **Phase 4（rtarget L2，35 分）**：`rtarget` 开了栈随机化 + 不可执行栈，只能从 `start_farm` 到 `mid_farm` 区间取小工具。可用指令只有 `movq`、`popq`、`ret`、`nop`（`0x90`），只允许用前 8 个寄存器（`%rax` – `%rdi`）。**两个小工具就够**：一个 `popq %rdi`（从栈上取值），一个地址处含 `movq %rax,%rdi` 的字节序列。坑：`popq` 会消耗栈上的数据，所以攻击串里小工具地址和数据必须交替排布，而且要**逐字节**核对 gadget 的反汇编，`objdump` 从指令中间解码出来的才是真身。
- **Phase 5（rtarget L3，5 分）**：用 ROP 调用 `touch3` 并传入 cookie 字符串指针，还需要 `lea`/算术 gadget 来算字符串地址。分值只有 5 分但难度最高，讲义自己都建议"有别的正事就停在这里"。
- **通用坑**：（1）`hex2raw` 只认十六进制字节对，脚本里的注释和空行要小心处理；（2）所有地址都要按小端序"反过来"写，`0x004006c8` 写成 `c8 06 40 00 00 00 00 00`；（3）串里不能出现 `0x0a`（换行），否则会被 `getbuf` 的读入逻辑提前截断——这正是 Phase 4/5 里选择 gadget 地址时的一个隐藏约束；（4）用 GDB 在 `getbuf` 的 `ret` 处下断点、`x/8xb $rsp` 检查栈顶，比反复试错高效得多。

6.5 常见错误与调试技巧

- **误以为结构体大小等于成员大小之和**：`struct S1 {char; int[2]; double;}` 直觉算 1+8+8=17，实际 24。**调试**：`gcc -Wall -Wpadded -c x.c` 让编译器报出每一处填充；或 GDB 里 `ptype /o struct S1`。
- **用 `memcmp` / `write` / 哈希比较结构体**：填充字节是未定义内容，字段相同也可能比较失败。**调试**：改用逐字段比较；`valgrind --track-origins=yes ./prog` 能报出"使用了未初始化字节"。
- **混淆 `p + 1` 与"地址 + 1"**：`int *p` 加 1 走 4 字节，`char *p` 加 1 走 1 字节，`struct S *p` 加 1 走 `sizeof(struct S)` 字节。**调试**：`gdb` 里 `p/x p`、`p/x p+1`、`p sizeof(*p)` 三条命令对照看。

- **把联合体当成 `(float)u` 那样的数值转换**：`bit2float(u)` 是位重解释，`(float)u` 是数值转换，结果天差地别。**调试**：两边都打印位模式（`p/x *(unsigned*)&f`），对比 `0x4048f5c3` 之类。
- **数组名与指针的 `sizeof` 混淆**：函数参数里的 `int z[]` 就是 `int *z`，`sizeof(z) == 8` 而不是数组长度。**调试**：`gcc -Wall` 会报 `warning: 'sizeof' on array function parameter will return size of 'int *'`；始终显式传长度。
- **浮点直接用 `==` 比较**：`0.1+0.2 == 0.3` 恒为假。**调试**：改用 `fabs(a-b) < 1e-9 * fmax(1.0, fmax(fabs(a), fabs(b)))`；GCC 的 `-Wfloat-equal` 能静态发现这类比较。
- **`-O0` 与 `-O2` 下"同一段代码"行为不同**：未定义行为给了优化器任意假设的自由，越界代码在 `-O2` 下可能被整体优化掉。**调试**：同时用 `gcc -O0 -fsanitize=address,undefined` 跑一遍，这是抓越界与 UB 最有效的组合拳。
- **忽略栈金丝雀对调试的干扰**：`*** stack smashing detected ***` 说明溢出已经发生，但报错点不是溢出点。**调试**：`gcc -fno-stack-protector` 重编译以便观察真实行为，再用 `x/24xb $rsp` 比对溢出前后的栈内容。

6.6 关键要点

- **聚合类型没有机器指令**：数组、结构体、联合体最终都降解为"基址 + 偏移"的地址算术，字段名在汇编里彻底消失，只剩编译期常量偏移。
- **`p + i` 的增量是 `i * sizeof(*p)`**：这是 C 指针算术的定义，也是二维数组 `A[i][j]` 地址公式 `A + (i*C + j)*sizeof(T)` 的来源；`sizeof(T)` 只能是 1/2/4/8 才享受比例变址寻址。
- **结构体布局 = 声明顺序 + 对齐填充**：K 等于最大成员的对齐值，结构体大小必须是 K 的倍数；把大类型放前面能省空间，但编译器**不会**替你重排。
- **对齐是硬件效率换空间**：4/8 字节的访存粒度决定了不对齐要跨缓存行、跨页；填充字节内容未定义，不可用于比较或哈希。
- **浮点 = 位域解释 + 有限精度**：`float` 24 位有效位（约 7 位十进制数字）、`double` 53 位（约 15–16 位）；$2^{24}+1$ 转 `float` 会丢，浮点加法不满足结合律，所以 `(a+b)+c` 不能重写为 `a+(b+c)`。
- **栈上的数组 + 无界输入 = 可被执行的攻击串**：理解栈布局与 `mov` / `ret` 是 Attack Lab 的全部前提，也是"永远不要写这种代码"的最好理由。

6.7 思考题（带答案）

题 1（计算题：算 `sizeof` 与元素偏移） 设有

```
struct P { char a; double b; short c; int d; };
struct Q { double b; int d; char a; short c; };
struct P arr[6];
```

（1）分别求 `sizeof(struct P)` 与 `sizeof(struct Q)`，并给出 `c` 在两个结构体中的偏移。（2）求 `arr[4].d` 相对 `arr` 的字节偏移。（3）若把 `arr` 的起始地址记为 `x`，`&arr[3].c` 等于什么？

答：（1）先定 K = 8（`double`）。`struct P`：`a` 在 0，`b` 需要 8 字节对齐所以跳到 8，`c` 在 16，`d` 需要 4 对齐、`16+2=18` 不是 4 的倍数所以跳到 20，末尾 24 已是 8 的倍数，故 **`sizeof(struct P) = 24`**，`offsetof(c) = 16`。`struct Q`：`b` 在 0，`d` 在 8，`a` 在 12，`c` 需要 2 对齐、`13` 不是 2 的倍数所以跳到 14，占 14–15，总 16 已是 8 的倍数，故 **`sizeof(struct Q) = 16`**，`offsetof(c) = 14`。**同样的成员、不同的声明顺序，大小差 8 字节**。（2）`sizeof(struct P) = 24`，`arr[4]` 起始偏移 $4 \times 24 = 96$，`d` 的成员偏移 20，故 `arr[4].d` 在 **偏移 116**（$= 4 \times 24 + 20$）。（3）`arr[3]` 起始为 $x + 72$，`c` 偏移 16，故 `&arr[3].c = x + 88`。

题 2（计算题：算二维数组元素地址与下标） 有 `short M[7][9];`，`M` 起始地址为 `0x1000`。（1）`M` 总共占多少字节？（2）`&M[3][5]` 的地址是多少？（3）若 `int *p = (int *)&M[2][0];`，`p[7]` 对应 `M` 的哪个元素？（4）为什么按列遍历（外层循环 `j`、内层循环 `i`）比按行遍历慢？

答：（1）`sizeof(short) = 2`，总大小 $7 \times 9 \times 2 = 126$ 字节。（2）偏移 $= (i \times C + j) \times 2 = (3 \times 9 + 5) \times 2 = 32 \times 2 = 64$，因此地址为 `0x1000 + 0x40 = 0x1040`。（3）`M[2][0]` 的偏移是 $2 \times 9 \times 2 = 36$ 字节；`p` 是 `int *`（4 字节），`p[7]` 的字节偏移是 $36 + 7 \times 4 = 64$，即 $64/2 = 32$ 个 `short`，而 $32 = 3 \times 9 + 5$，所以 `p[7]` 正好覆盖 `M[3][5]`。（4）行优先下同一行的元素在内存中连续；按行遍历是顺序访问，一个 64 字节缓存行能装下 $64/2 = 32$ 个 `short`，即 3 行多，命中率高。按列遍历每次跳 $9 \times 2 = 18$ 字节，几乎每次都踩在新的缓存行上，造成"每次访问都必然缺失"的最坏模式——详见 Lecture 9/10 关于局部性的讨论。

题 3（找错题："直觉但错误"的想法错在哪） 有同学说："`union { float f; unsigned u; } x; x.u = 0x4048F5C3; return x.f;` 和 `return (float)0x4048F5C3;` 应该一样吧？反正都是把整数变成 float。" 请指出错误，并算出两个结果分别是多少。

答：错在混淆了**位重解释（type punning）**与**数值转换**。`x.f` 把 `0x4048F5C3` 这 32 位按 IEEE 754 单精度解读：`s=0`、`exp=0x80=128`，$E = 128 - 127 = 1$，`frac=0x48F5C3`，$M = 1 + 0x48F5C3 / 2^{23} \approx 1.5700001$，所以 $V \approx 1.57 \times 2 = 3.14$。而 `(float)0x4048F5C3` 是数值转换：$0x4048F5C3 = 1078523331$，转成 `float` 时保留 24 位有效位，结果约为 1.07852333×10^9。两者相差九个数量级。**正确结论**：`union` 双关读位模式，强制类型转换做数值映射，绝不能互相替代。

题 4（判断与证明题） 下面这段代码输出什么？为什么编译器**不允许**把它优化成 `printf("%d\n", 1)`？

```
float x = 1e20f, y = -1e20f, z = 3.14f;
printf("%d\n", (x + y) + z == x + (y + z));
```

答：输出 `0`（假）。`x + y` 精确得 0，`0 + z = 3.14`；而 `y + z` 先算：`y = -1e20f` 的量级是 10^{20}，`float` 只有 24 位有效位，其相邻可表示数之间的间隔已达约 10^{13}，`3.14` 对齐后完全落在有效位之外被舍成 0，于是 `y + z = -1e20f`，`x + (-1e20f) = 0`。两式结果分别是 3.14 与 0。编译器不允许优化成 1，是因为 IEEE 754 浮点**不满足结合律**：把 `(x+y)+z` 重排成 `x+(y+z)` 会改变可观测结果，而 C 标准要求浮点运算按规定的结合顺序求值（除非程序员显式使用 `-ffast-math` 这类放弃 IEEE 语义的选项）。这也解释了为什么需要 `-ffast-math` 或 `#pragma STDC FP_CONTRACT` 才能让编译器做浮点重排——它们是在明确放弃可复现性。

Lecture 7: 链接 (Linking)

> **讲义对应**：CMU 15–213 Lecture 7 — Linking（素材：`F25-07-linking.txt`） **教材对应**：CS:APP3e 第 7 章 链接（7.1–7.13） **关联 Lab**：无直接 lab（但为理解全部 lab 的构建过程提供基础）

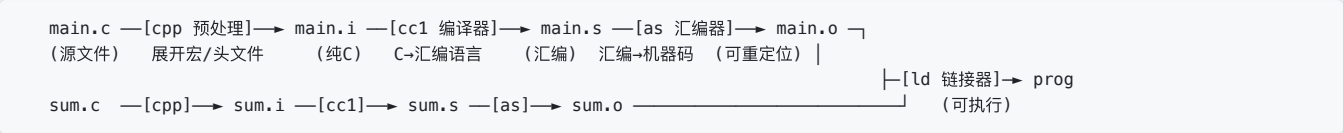
7.1 概述

到目前为止，我们都是把程序当作"一个 `.c` 文件变成一个可执行文件"来理解的。但真实系统里，程序是由**多个源文件**、**预编译好的库**拼装出来的——把 `main.o`、`sum.o`、`libc.so` 缝合成一个能跑的内存镜像，这件事由**链接器（linker, `ld`）**完成。本讲回答两个核心问题：链接器**做什么**（符号解析 + 重定位），以及它**怎么做**（ELF 格式、符号表、重定位表、静态/动态库、GOT/PLT）。这承接第 5、6 讲的机器级代码与过程调用（链接器改写的正是 `call`、`mov` 这些指令中的地址字段），并直接通向第 9 讲的内存层次与第 11–12 讲的虚拟内存——共享库"多个进程共用一份代码"正是靠虚拟内存实现的。链接的错误信息往往出现在编译成功之后，是 C 程序员日常最头疼的一类问题。

7.2 核心概念与底层机制图解

7.2.1 编译驱动过程与四种工具（Compiler Driver）

- **定义与目的**：`gcc` 只是一个**驱动器（compiler driver）**，它不亲自干活，而是依次调用 `cpp`、`cc1`、`as`、`ld` 四个程序。
- **直观解释**：`gcc` 像一位包工头，自己不动手，只负责按顺序把活派给预处理工、翻译工、装配工和总装工。
- **底层机制图解**：



- **与机器码/硬件的对应**：用 `gcc -v` 可以看到真实命令行（本机 GCC 12.2.0 实测）：

```
/opt/.../cc1 -quiet -v main.c ... -o /tmp/ccZHxymZ.s     # C → 汇编
as -v --64 -o /tmp/ccTnEijw.o /tmp/ccZHxymZ.s            # 汇编 → .o
/opt/.../collect2 ... -o prog /lib/../lib64/crt1.o ... -lc ...   # 真正的链接
```

`-save-temps` 会把 `main.i` / `main.s` / `main.o` 留在当前目录，供你逐阶段观察。

7.2.2 三种目标文件与 ELF 格式（Executable and Linkable Format）

- **定义与目的**：所有目标文件——可重定位（`.o`）、可执行（`a.out`）、共享（`.so`）——都使用统一的 **ELF** 格式，统称 **ELF 二进制文件**。
- **直观解释**：ELF 像一只分格工具箱：每样零件（代码、只读数据、已初始化数据、未初始化数据、符号表、重定位表）各占一格，并贴有"格号（section header）"标签。
- **底层机制图解（可重定位目标文件的段布局）**：

```
+--------------------------------------------------+ offset 0
| ELF header (字长/字节序/文件类型 .o|EXEC|.so/机器类型) |
+--------------------------------------------------+
| .text      : 机器码                               |  可执行、只读
| .rodata    : 只读数据（跳转表、字符串常量 "%d\n"）    |  只读
| .data      : 已初始化的全局/静态变量                 |  可读写
| .bss       : 未初始化全局变量（NOBITS，不占文件空间）  |  可读写
| .symtab    : 符号表（名字/大小/位置/绑定/段索引）      |  不加载
| .rel.text  : .text 的重定位条目                    |  不加载
| .rel.data  : .data 的重定位条目                    |  不加载
| .debug     : 调试信息（gcc -g）                    |  不加载
| .line      : 行号 ↔ 指令地址映射（gcc -g）          |  不加载
| .strtab    : 符号名字符串表                        |  不加载
+--------------------------------------------------+
| Section header table : 每个段的偏移与大小          |  ← 链接器靠它定位
+--------------------------------------------------+
```

（注：现代 GNU 工具链把重定位段命名为 `.rela.text` / `.rela.data`，`.rel.txt` 是教材沿用的旧写法。）

- **与机器码/硬件的对应**：`.bss` 的类型是 `NOBITS`，**在文件里不占一字节**，但其 `Size` 在内存镜像里真实存在。实测一个含 `int big[1024]` 的目标文件：

```
  [ 3] .data    PROGBITS  0000000000000000  00000054
  [ 4] .bss     NOBITS    0000000000000000  00000060   <- 文件偏移与下个段相同
```

好处是：4000 字节的零数组不需要在磁盘上占 4000 字节，加载器只需把 4000 字节的内存**清零**即可。这正是 `bss` 常被戏称为 "Better Save Space" 的原因。

7.2.3 符号与符号表（Symbols and Symbol Table）

- **定义与目的**：**符号（symbol）** 就是汇编器为每个全局变量、函数、`static` 变量建立的表项，包含名字、大小、位置（段 + 偏移）。链接器**只认识符号，不认识类型**。
- **直观解释**：符号表像图书馆的索引卡片——只有书名和书架号，没有内容摘要。所以链接器无法察觉两个模块对同一个名字的**类型理解不一致**。
- **底层机制图解**：

```
  符号分类（讲义三分类）
  ┌──────────────┬──────────────────────────────────────────┐
  │ 全局 global     │ 模块 m 定义、可被其他模块引用（非 static 函数/全局变量）│
  │ 外部 external   │ 模块 m 引用、但定义在其他模块（.o 中标记为 UND）        │
  │ 局部 local      │ 仅模块 m 定义并使用（static 函数/变量；名字可重名）     │
  └──────────────┴──────────────────────────────────────────┘
```

- **与机器码/硬件的对应**：局部（非 `static`）C 变量**不在符号表里**，它们在栈上；`static` 局部变量却**在 `.bss` / `.data` 里**，编译器为其生成唯一名字（如 `x`、`x.0`、`x.1`）以避免冲突。实测 `nm static-local.o`：

```
0000000000000000 T f
0000000000000010 T g
0000000000000020 T h
0000000000000008 d x        <- 文件级 static
0000000000000000 d x.0      <- f() 内的 static
0000000000000004 d x.1      <- g() 内的 static
```

7.2.4 静态库与"按需提取"（Static Libraries / Archives）

- **定义与目的**：`.a` 归档文件把一批 `.o` 用 `ar` 打包并加索引；链接器**只把真正被引用到的成员**拷进可执行文件。
- **直观解释**：`.a` 像一本装订好的活页讲义集，链接器是只复印**你要的那几页**的复印机，而不是整本复印。
- **底层机制图解（按需提取）**：

```
main2.o 的未解析符号: {addvec, printf}
     |
     ▼  扫描 libvector.a (含 addvec.o, multvec.o)
┌──────────────────────┐    命中 addvec  → 抽入可执行文件, 新增未解析 {printf}
│ addvec.o  [→ 加入]   │
│ multvec.o [ 丢弃 ]   │    未被任何未解析符号引用 → 整块丢弃
└──────────────────────┘
     |
     ▼  扫描 libc.a
printf.o 及其依赖链依次抽入 → 可执行文件 prog2c 链接成功
```

- **与机器码/硬件的对应**：实测 `nm prog2c` 只有 `addvec`，没有 `multvec`；而把 `.o` 全部直接列在命令行（`prog2all`）会额外多出 `multvec` 与 64 字节：

```
0000000000401168 T addvec          # prog2c (按需提取)
0000000000401168 T addvec          # prog2all (全量)
0000000000401186 T multvec
-rwxr-xr-x 25888 prog2c    vs    -rwxr-xr-x 25952 prog2all
```

7.2.5 动态库与 GOT/PLT 延迟绑定（Shared Libraries, GOT/PLT, Lazy Binding）

- **定义与目的**：共享库 `.so` 在**加载时或运行时**才被链接。为了让同一份代码能被映射到不同进程的不同虚拟地址，库必须用**位置无关代码（PIC）**：用 **GOT（Global Offset Table）** 间接访问全局变量，用 **PLT（Procedure Linkage Table）** 延迟解析外部函数地址。
- **直观解释**：GOT 像一块"通讯录白板"，先写"还不知道"，第一次打电话（调用）时才把真号码填上；PLT 是每通电话对应的便签，指向白板上那一行。这就是**延迟绑定（lazy binding）**。
- **底层机制图解（首次调用 addvec 的流程）**：

```
main 中:  callq 401040 <addvec@plt>
                |
                ▼
┌──────────────────────────────────────────────────────┐
│ PLT  (代码段, 只读)                                  │
│ 401040 <addvec@plt>:                                 │
│    jmpq *0x2fda(%rip)   # GOT[addvec] ─────┐  首次 = 401046  │
│    pushq $0x1           # 重定位索引 1     │               │
│    jmpq  401020 <.plt>  # 跳到 PLT[0]      │               │
├──────────────────────────────────────────────┼──────────────┤
│ GOT/GOT.PLT (数据段, 可写)                   ▼              │
│  404008: GOT[1] = link_map 指针  ◄── PLT[0] pushq 0x2fe2(%rip)│
│  404010: GOT[2] = _dl_runtime_resolve  ◄── PLT[0] jmpq *...   │
│  404018: GOT[printf] = 401036 (先) → libc printf (后)         │
│  404020: GOT[addvec] = 401046 (先) → libacc addvec (后)       │
└──────────────────────────────────────────────────────┘

首次: PLT→GOT→回到下一条→push 索引→PLT[0]→_dl_runtime_resolve 写回 GOT
以后: PLT→GOT 直接命中真实地址 (只多一次间接跳转)
```

- **与机器码/硬件的对应**：实测 `objdump -d -j .plt prog2l` 与 `objdump -R prog2l`：

```
0000000000401040 <addvec@plt>:
  401040: ff 25 da 2f 00 00    jmpq   *0x2fda(%rip)        # 404020 <addvec>
  401046: 68 01 00 00 00       pushq  $0x1
  40104b: e9 d0 ff ff ff       jmpq   401020 <.plt>

DYNAMIC RELOCATION RECORDS
0000000000404018 R_X86_64_JUMP_SLOT  printf@GLIBC_2.2.5
0000000000404020 R_X86_64_JUMP_SLOT  addvec
```

`JUMP_SLOT` 就是"这个 GOT 槽最终要被填入函数地址"的声明；若用 `LD_BIND_NOW=1` 或 `-Wl,-z,now` 则改为加载时全部绑定（GNU_RELRO 保护 GOT 不可写）。`ldd prog2l` 显示库的实际落点，`readelf -x .interp` 显示动态链接器 `/lib64/ld-linux-x86-64.so.2`。

7.3 代码示例与底层机制分析

示例 1：两个模块 + 头文件——符号解析与重定位全过程

代码（三文件，实测位于 `/tmp/lk/ex1/`）：

```
/* sum.h : 接口声明，被两个模块共享 */
int sum(int *a, int n);
extern int array[2];          /* extern = "它在别处定义"，避免重复分配空间 */
```

```
/* main.c */
#include "sum.h"
int array[2] = {1, 2};        /* 强符号：已初始化全局 → .data */
int main(int argc, char **argv)
{
    int val = sum(array, 2);  /* 引用外部符号 sum */
    return val;
}
```

```
/* sum.c */
#include "sum.h"
int sum(int *a, int n)        /* 强符号：函数 → .text */
{
    int i, s = 0;             /* 链接器完全不知道 i、s 的存在 */
    for (i = 0; i < n; i++)
        s += a[i];
    return s;
}
```

【代码做什么？】

1. `gcc -c` 分别把 `main.c`、`sum.c` 编译成 `main.o`、`sum.o`；此时两者都不知道对方的最终地址。
2. **符号解析**：`main.o` 有未解析（UND）的 `sum`，`sum.o` 有未解析的 `array`；链接器把它们配对。
3. **重定位**：合并 `.text`、`.data`，为每个段分配最终虚拟地址，再按重定位表改写指令中的地址字段。
4. 链接完成后 `./prog` 退出码为 3（`sum({1,2},2) = 3`）。

【底层机制透视】实测 `gcc -c -Og -fcf-protection=none` + `objdump -r -d main.o`：

```
0000000000000000 <main>:
   0: 48 83 ec 08        sub    $0x8,%rsp
   4: be 02 00 00 00     mov    $0x2,%esi
   9: bf 00 00 00 00     mov    $0x0,%edi        <- 地址字段全 0，待填
                a: R_X86_64_32  array              # 绝对寻址重定位条目
   e: e8 00 00 00 00     callq  13 <main+0x13>    <- 调用"下一条指令"
                f: R_X86_64_PLT32 sum-0x4          # PC 相对寻址重定位条目
  13: 48 83 c4 08        add    $0x8,%rsp
  17: c3                 retq
```

而 `sum.o` **没有任何重定位条目**——它不引用外部符号，因此完全位置无关（除 `call` 外不需要改写）。

【内存布局 / 数据结构图解】链接后的最终地址（实测 `objdump -d prog`）：

```
0000000000401106 <main>:
  40110f: bf 28 40 40 00   mov  $0x404028,%edi   <- n=0x404028 即 &array，绝对地址被填入
  401114: e8 05 00 00 00   callq 40111e <sum>    <- sum 被放到 main 之后 0x18 字节
000000000040111e <sum>:                          <- 0x40111e = 0x401119 + 5

可执行文件内存镜像：
  0x404028 ┌──────────┐
           │ array[0]=1│ .data（已初始化全局变量）
           │ array[1]=2│
  0x404030 ├──────────┤
           │ （零填充）│ .bss（NOBITS，文件里不占空间）
  0x400000 └──────────┘ ← 只读代码段从这里开始
```

【与汇编 / 硬件的对应】两种重定位类型：

- `R_X86_64_32`（**绝对寻址**）：把符号的**绝对地址**直接写进 4 字节字段。用于 `mov $array, %edi`、`int *xp = &x` 这类"地址必须是个常量"的场景。
- `R_X86_64_PC32` / `R_X86_64_PLT32`（**PC 相对寻址**）：写入 `目标地址 − 重定位处地址 − 4`（`-0x4` 修正量正是该字段自身长度）。`call` 计算验证：`0x40111e = 0x401119(下一条指令) + 5`，即指令中的位移字段被填成 `0x5`。

【实测验证】`readelf -s main.o` 的符号表（`Ndx = UND` 即"引用但未定义"）：

```
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     8: 0000000000000000    24 FUNC    GLOBAL DEFAULT    1 main
     9: 0000000000000000     8 OBJECT  GLOBAL DEFAULT    3 array
    10: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND sum
```

示例 2：静态库 `libvector.a` 与库顺序陷阱

代码（位于 `/tmp/lk/lib/`）：

```
/* vector.h */
#ifndef VECTOR_H
#define VECTOR_H
void addvec(int *x, int *y, int *z, int n);
void multvec(int *x, int *y, int *z, int n);
#endif
```

```
/* addvec.c */                          /* multvec.c 结构相同，做逐元素乘法 */
#include "vector.h"
void addvec(int *x, int *y, int *z, int n) {
    int i;
    for (i = 0; i < n; i++)
        z[i] = x[i] + y[i];
}
```

```
/* main2.c —— 只引用 addvec，不引用 multvec */
#include <stdio.h>
#include "vector.h"
int x[2] = {1, 2};
int y[2] = {3, 4};
int z[2];
int main(void) {
    addvec(x, y, z, 2);
    printf("z = [%d %d]\n", z[0], z[1]);
    return 0;
}
```

构建与运行（真实输出）：

```
$ gcc -c -Og addvec.c multvec.c main2.c
$ ar rs libvector.a addvec.o multvec.o
$ ar -t libvector.a
addvec.o
multvec.o
$ gcc -L. -o prog2c main2.o -lvector        # 库放右边（正确）
$ ./prog2c
z = [4 6]                                    # 1+3=4, 2+4=6
```

【代码做什么？】 `ar rs` 中 `r` 表示插入/替换成员，`s` 表示建立符号索引。链接时链接器**按命令行顺序**扫描，维护"当前未解析符号表"，每遇到一个 `.o` / `.a` 就尝试解析；扫描结束仍有未解析符号则报错。

【底层机制透视】把 `-lvector` 放到 `main2.o` **之前**，链接器扫描库时还没有任何未解析符号，`addvec.o` 不会被抽入，随后扫描 `main2.o` 才产生未解析的 `addvec`——但库已经扫完了。实测：

```
$ gcc -L. -o prog2bad -lvector main2.o
/usr/bin/ld: main2.o: in function `main':
main2.c:(.text+0x19): undefined reference to `addvec'
collect2: error: ld returned 1 exit status
```

口诀：被依赖者放右边，库放在最后。若库之间存在循环依赖，用 `-Wl,--start-group ... -Wl,--end-group` 让链接器反复扫描组内成员直至收敛（实测可修复上述错误）。

【内存布局 / 数据结构图解】

```
命令行扫描顺序 → 未解析集合的演化

gcc main2.o  -lvector            (正确)
  main2.o   : {addvec, printf}          ← main2 产生 addvec 需求
  libvector : addvec 命中 → {printf}     ← 抽入 addvec.o，丢弃 multvec.o
  libc      : printf 命中 → {}           ← 链接成功

gcc -lvector  main2.o            (错误)
  libvector : {} 无未解析符号 → 一个成员都不抽
  main2.o   : {addvec, printf}           ← 库已扫完，永远无法满足
  扫描结束 → undefined reference to `addvec'
```

【实测验证】按需提取确实节省空间：

```
$ nm prog2c | grep -E 'addvec|multvec'
0000000000401168 T addvec               # 只有 addvec
$ ls -l prog2c prog2all
-rwxr-xr-x 25888 prog2c                 # 按需提取
-rwxr-xr-x 25952 prog2all               # 全量链接（含 multvec，多 64 字节）
```

示例 3：动态库、`ldd` 与 PIC 代码生成

构建（位于 `/tmp/lk/lib/`，真实输出）：

```
$ gcc -shared -fpic -o libvector.so addvec.c multvec.c
$ gcc -o prog2l main2.c -L. -lvector -Wl,-rpath,/tmp/lk/lib
$ ./prog2l
z = [4 6]
$ ldd prog2l
        linux-vdso.so.1 (0x00007ffd9359c000)
        libvector.so => /tmp/lk/lib/libvector.so (0x00007ffa3affd000)
        libc.so.6 => /lib64/libc.so.6 (0x00007ffa3ac00000)
        /lib64/ld-linux-x86-64.so.2 (0x00007ffa3b004000)
```

【底层机制透视】可执行文件里**没有 `addvec` 的代码**，只有一个 `NEEDED` 记录和一个待填的 GOT 槽：

```
$ readelf -d prog2l | grep -E 'NEEDED|RPATH'
 0x0000000000000001 (NEEDED)  Shared library: [libvector.so]
 0x0000000000000001 (NEEDED)  Shared library: [libc.so.6]
 0x000000000000000f (RPATH)   Library rpath: [/tmp/lk/lib]
```

`ld-linux-x86-64.so.2` 在 `execve` 之后、`main` 之前被内核（按 `.interp` 段）启动，它 mmap 各 `.so`、填 GOT、再调用 `__libc_start_main`（`0x401038: mov $0x401106,%rdi` 把 `main` 的地址作为参数传入）。库查找顺序为 `RPATH` → `LD_LIBRARY_PATH` → `RUNPATH` → `/etc/ld.so.cache` → 默认目录。

【与汇编 / 硬件的对应】`-fPIC` 与 `-fno-PIC` 的差别（实测 `/tmp/lk/pic/`）：

```
-fPIC 访问全局变量 glob：先取 GOT 表项，再解引用（两次访存）
  110d: 48 8b 05 d4 2e 00 00   mov  0x2ed4(%rip),%rax     # GOT 项
  1114: 8b 00                  mov  (%rax),%eax             # 真正的 glob
  重定位条目: R_X86_64_REX_GOTP32  glob - 4        (readelf -r acc_pic.o)

-fno-PIC：一条 PC 相对寻址直接命中（但地址固定，不能共享）
     0: 8b 05 00 00 00 00      mov  0x0(%rip),%eax
  重定位条目: R_X86_64_PC32  glob - 4
```

外部函数调用在 PIC 中固定走 PLT：`R_X86_64_PLT32 getext - 4`。**为什么库必须 PIC?** 因为同一个物理页被映射到每个进程的**不同虚拟地址**，绝对地址无法共享，也就无法只保留一份物理副本。

示例 4：库打桩——用 `LD_PRELOAD` 拦截 `malloc`

代码（位于 `/tmp/lk/mtrace/`）：

```c
/* mymalloc.c —— 编译成 mymalloc.so，用 LD_PRELOAD 注入 */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>

static void *(*real_malloc)(size_t) = NULL;
static void  (*real_free)(void *)   = NULL;
static long malloc_count = 0, free_count = 0, malloc_bytes = 0;

void *malloc(size_t size)
{
    if (!real_malloc)                       /* 惰性解析 libc 的真正实现 */
        real_malloc = dlsym(RTLD_NEXT, "malloc");
    malloc_count++;
    malloc_bytes += (long)size;
    return real_malloc(size);
}

void free(void *ptr)
{
    if (!real_free)
        real_free = dlsym(RTLD_NEXT, "free");
    free_count++;
    real_free(ptr);
}

__attribute__((destructor))                 /* 库卸载时打印统计 */
static void mtrace_report(void)
{
    fprintf(stderr, "[mtrace] malloc calls = %ld, free calls = %ld, bytes = %ld\n",
            malloc_count, free_count, malloc_bytes);
}
```

```c
/* leaky.c —— 被监视的程序：故意泄漏一块 */
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    char *a = malloc(100);
    char *b = malloc(4096);        /* 故意不 free：模拟泄漏 */
    char *c = malloc(64);
    free(a);
    free(c);
    printf("b = %p (deliberately leaked)\n", (void *)b);
    return 0;
}
```

【代码做什么？】 1. `LD_PRELOAD=/tmp/lk/mtrace/mymalloc.so` 让动态链接器**优先**在 `mymalloc.so` 中搜索符号，于是程序中所有 `malloc` / `free` 调用都先进入我们的版本。 2. 我们通过 `dlsym(RTLD_NEXT, "malloc")` 取"下一个"定义，也就是 libc 的真 `malloc`，加完计数再转发。 3. 计数在库卸载时（`destructor`）打印，因此恰好覆盖整个进程生命周期。

【实测验证】真实运行输出：

```
$ gcc -g -Wall -o leaky leaky.c
$ ./leaky
b = 0xef9310 (deliberately leaked)

$ LD_PRELOAD=/tmp/lk/mtrace/mymalloc.so ./leaky
b = 0x237d310 (deliberately leaked)
[mtrace] malloc calls = 4, free calls = 2, bytes = 8356
```

`4` 次分配 = 100 + 4096 + 64 + 4096，其中第 4 次 4096 字节来自 `printf` 首次调用时 libc 的缓冲初始化（`bytes = 100+4096+64+4096 = 8356`）；`free` 只有 2 次，说明净泄漏至少两块。这正是 **malloc lab（L5）** 调试分配器时最常用的手段（另有 glibc 自带的 `mtrace()` 与 `MALLOC_TRACE`）。

【与汇编 / 硬件的对应】打桩有三种时机，讲义与教材并重：

时机	手段	优点	缺点
编译期	`gcc -Wl,--wrap=malloc`，自定义 `__wrap_malloc` / `__real_malloc`	简单、可只作用于单个程序	必须重新编译
链接期	把 `malloc.o` 这类 `.o` 直接写在命令行最前	无需重编译库	需要拿到目标文件

时机	手段	优点	缺点
运行期	`LD_PRELOAD`（本示例）	不改程序、可用于任意二进制	影响整个进程，需防递归

```
$ gcc -g -Wall -o wr wr.c -Wl,--wrap=malloc && ./wr
p = 0xc732a0
[wrap] malloc(32)               # 编译期打桩真实输出
```

7.4 实验关联

本讲无直接关联 lab，但它是**理解全部 lab 构建过程的基础**，并在多处直接复现：

- **L4 Cache（csim-ref）**：讲义就是用 `ldd csim-ref` 讲解 `.interp` / `.dynamic` / `NEEDED`，并要求你在 `cachelab` handout 里解释 `libm.so.6` 从何而来。
- **L3 Attack**：活动讲义明确把 ELF 段标志与攻击实验挂钩——**只有 `.text` 带 `CODE` 标志**，操作系统才把它设为可执行；**除 `.data` / `.bss` 外的段带 `READONLY`**，因此写 `.rodata` 会段错误。这解释了为什么攻击实验里把代码放进栈/数据区会被拦截。
- **L5 Malloc**：调试自己的分配器时，示例 4 的 `LD_PRELOAD` 计数是最有效的"调用轨迹"工具；`--wrap` 则用于只监视测试驱动。
- **所有 lab 的 Makefile**：`-fPIC`、`-shared`、`-lm`、库顺序、`ldd` 检查缺库，都是提交前必须过关的工程细节。

7.5 常见错误与调试技巧

- **`undefined reference to 'foo'`（未定义引用）**：解析阶段找不到定义。常见三种原因：忘记链接库（`-lm`）、库顺序错误（库在 `.o` 之前）、只写了 `int foo(int);` 声明却没有实现。**调试**：`nm -u main.o` 看未解析符号，`nm -C libfoo.a | grep foo` 确认库里有没有，`gcc ... 2>&1 | c++filt` 还原 C++ 名字。
- **`multiple definition of 'x'`（多重定义）**：两个模块都定义了强符号（GCC 10+ 默认 `-fno-common`，把未初始化全局也视为强定义），或同一 `.o` 被列了两次。**调试**：`ld` 会指出 `first defined here` 的文件；用 `nm *.o | grep x` 找出所有定义者，把其中一个改成 `static` 或 `extern`。
- **`cannot find -lxxx`**：库根本不在搜索路径，与符号解析无关。**调试**：`gcc -Wl,--verbose 2>&1 | grep succeeded` 打印所有搜索目录，`ldconfig -p | grep xxx` 看系统缓存，用 `-L/path` 补路径。
- **同名弱符号悄悄合并（"Evil"/"Nasty"）**：两个模块各写一次 `int x;`（`-fcommon` 下）会被合并为同一块内存，一个模块的写入会破坏另一个模块的数据；若一个是 1 字节 `char`、一个是 8 字节 `double`，链接器只给 warning，运行期写出界直接压掉相邻变量。**调试**：`nm prog | grep ' [BbDd] '` 检查是否有意外的全局同名，`objdump -d` 观察写入宽度（`movb` vs `movsd`）。
- **类型不匹配却静默通过**：`mismatch-variable.c` 定义 `double x = 3.14;`，`mismatch-main.c` 声明 `long int x;`，编译链接零警告。实测打印 `4614253070214989087`（即 `0x40091eb851eb851f`，3.14 的 IEEE-754 位模式被当成整数读）。**根源**：符号表不带类型。**调试**：把共享声明统一放进头文件（示例 1 的 `sum.h`），始终 `#include` 它；`-Wmissing-declarations`、`-flto` 能查出部分不匹配。
- **在只读段上写入导致段错误**：`helper.c` 里把 `global` 定义成 `const`，它会落到 `.rodata`；`main` 若声明时漏掉 `const` 并尝试赋值，链接成功但运行时 `Segmentation fault`。实测：

  `console $ readelf -S ro.o | grep -E 'rodata|data' [ 4] .rodata PROGBITS 0000000000000000 00000040 # const 全局落在这里 $ ./roseg Segmentation fault (core dumped) # 退出码 139 = 128 + SIGSEGV(11)`

  调试：`readelf -S file.o` 确认符号所在的段，`objdump -d` 看是否用了 `movb $0x61,...(%rip)` 这类直接写入，`gdb` 下 `catch signal SIGSEGV` 定位故障指令。 * **PIC 缺失**：`gcc -shared` 时忘记 `-fPIC`。实测报错为：

  `ld: warning: relocation against `glob' in read-only section `.text' ld: relocation R_X86_64_PC32 against symbol `glob' can not be used when making a shared object; recompile with -fPIC ld: final link failed: bad value`

  调试：`readelf -r lib.so` 检查是否走了 GOT——PIC 下应看到 `R_X86_64_GLOB_DAT`（数据符号）与 `R_X86_64_JUMP_SLOT`（函数符号）；若出现 `R_X86_64_REX_GOTP32` 之外的绝对 `R_X86_64_32`，说明未生成位置无关代码。 * **`LD_LIBRARY_PATH` 找不到库**：`error while loading shared libraries: libvector.so: cannot open shared object file`。**调试**：`ldd ./prog` 找到哪个 `=> not found`，临时用 `LD_LIBRARY_PATH=. ./prog`，正式方案是 `-Wl,-rpath,'$ORIGIN'` 或 `ldconfig`。

7.6 关键要点

- **链接只有两件事：符号解析（references → definitions）和重定位（相对地址 → 最终绝对地址）。**其余概念（ELF、`.a`、`.so`、PLT/GOT）都是为这两件事服务的工具。
- **符号表里没有类型。**C 的类型信息在编译成汇编时就已经消失，所以链接器永远抓不到"同名不同类型"这类错误——接口必须靠共享头文件来约束。
- **`static` 是避免命名冲突最便宜的手段**：它把符号从 global 降级为 local，同时把变量从栈搬进 `.data` / `.bss` 并获得进程生命周期。
- **静态库按需提取，因此命令行顺序决定成败**：被依赖者放右边，库放在最后；循环依赖用 `--start-group` / `--end-group`。
- **共享库靠 PIC 换取物理内存共享**：全局变量经 GOT 间接访问，外部函数经 PLT 延迟绑定，代价是每次访问多一次访存、每次首次调用多一次解析。
- **`.bss` 是"文件里不存在、内存里存在"的段**：磁盘上零字节，加载时由加载器清零，是"零初始化"这一语义的存储优化。

7.7 思考题（带答案）

题 1（链接行为推演题） 下面三个文件用 `gcc -fcommon` 链接并运行，输出什么？如果改回现代默认的 `-fno-common` 又会怎样？

```
/* m1.c */
int x;                          /* 弱符号 */
void set_x(void) { x = 42; }
/* m2.c */
int x;                          /* 同一个弱符号，另一个模块"又定义"了一次 */
void set_x(void);
int main(void) { printf("%d\n", x); set_x(); printf("%d\n", x); }
```

答：`-fcommon` 下两个 `int x;` 都是弱符号，链接器**任选其一**并把两处引用解析到同一块内存（实测 `nm` 显示一个 `B g` 风格的单一定义）。因此输出 `0` 再 `42`——即使两个模块各自看不到对方的定义，它们操作的仍是同一块内存。这通常**不是**你想要的：若两处本意是各自独立的计数器，就会互相污染。改用 `-fno-common`（GCC 10+ 的默认）后，`ld` 报 `multiple definition of 'x'` 并失败。**正确做法**：其中一个改为 `static int x;`（真正独立），或在头文件里写 `extern int x;` 而只在**一个** `.c` 中定义。

题 2（推演题：静默灾难） 讲义"Revisiting linker puzzles"的程序在 `-fcommon` 下，`int.c` 有 `char x='c'; int y=2;`，`double.c` 有 `double x;`。`p2()` 写 `x = 20000.00002`。两个文件谁先谁后，输出都一样吗？为什么 `x` 打印出来是 `-42`，而 `y` 却没有变化？

答：题面中两处 `x` 都是弱符号（`double x;` 未初始化，`char x` 在 `-fcommon` 下也被视为弱定义），链接器把它们并入**同一块存储**。实测（本机 GCC 12.2.0，两种命令行顺序结果完全相同）：

```
$ gcc -fcommon -fno-pie -no-pie -fcf-protection=none -O -o p int.c double.c
x=0.000000            # p2 用 %f 读 x：此时低字节还是 'c'(0x63)，非规格化数 → 0.000000
x=-42 y=2             # p1 先 movb $'a' 写 x 的低字节，p2 再写 8 字节 double，最后 movsbl 读 x

$ nm p | grep -E ' [BbDd] (x|y)$'
0000000000404034 D x        # x 与 y 都在 .data
0000000000404030 D y
```

关键点有三：其一，`p2()` 按 `double` 写入 8 字节 `20000.00002`，覆盖了 `x` 起点的 8 个字节；其二，`p1()` 之后用 `movsbl` 只读 `x` 的**最低一个字节**，这个字节来自 `double` 位模式的最低字节，符号扩展后恰好是 `-42`；其三，本机布局把 `y` 放在 `x` **之前**（`y` 在 `0x404030`、`x` 在 `0x404034`），所以 8 字节写正好落在 `x` 之后、没有波及 `y`——但**这只是链接器分配顺序的巧合，不是任何保证**。讲义幻灯片上曾出现同一程序 `y=1087604736` 的结果，正是布局把 `y` 放到了 `x` 之后。若不加 `-fcommon`，链接器直接以 `multiple definition of 'x'` 拒绝，而**这恰恰是好事**：报错比静默的数值污染安全得多。

题 3（"直觉但错误"的想法） 有同学认为："`gcc -c a.c b.c` 之后链接报 `undefined reference`，说明编译器没找到函数实现，把 `b.c` 也加进 `gcc` 命令就好了。"这个推理错在哪？

答：错把**编译单元边界**当成了错误的原因。`undefined reference` 是**链接期**错误，说明符号解析未完成，具体可能是：(1) 忘链接库（`-lm`）；(2) 库顺序错误（库在 `.o` 之前）；(3) 函数只声明未定义；(4) 名字修饰/命名空间不匹配。把所有 `.c` 都写进同一条 `gcc` 命令确实能解决 (1)(2)，但它掩盖了真正的问题——大型项目里你不可能把所有源文件列一遍，正确做法是修库顺序或补 `-l`。另一个常见误判是"编译成功了所以代码没问题"：如题 2 所示，**编译成功 + 链接成功**完全可能与运行期灾难并存，因为类型信息在汇编层面已经消失。

题 4（计算题） 100 个**不同的**可执行程序都需要 `libvector.a` 中的 `addvec`，其中 `addvec.o`、`multvec.o` 各 4 KB；每个程序只引用 `addvec`。若改用共享库 `libvector.so`（代码共 4 KB + 1 KB 元数据），磁盘占用与物理内存占用各减少多少？

答：静态方案下，每个可执行文件都各自拷入一份 `addvec.o` 的 4 KB（`multvec.o` 因未被引用而丢弃），所以磁盘上有 $100 \times 4\,\mathrm{KB} = 400\,\mathrm{KB}$；运行时 100 个进程各自的代码页也来自这 100 个不同文件，物理内存同样需要约 $400\,\mathrm{KB}$。动态方案下磁盘只需 $4 + 1 = 5\,\mathrm{KB}$（每个可执行文件里只留一个 `NEEDED` 字符串和一个待填的 GOT 槽），物理内存只需 **1 份** `libvector.so` 的代码页（各进程把同一物理页映射到各自的虚拟地址）加上每进程私有的少量 GOT 数据。因此**磁盘与物理内存都节省约** $400 - 4 = 396\,\mathrm{KB}$，接近 99%。这正是讲义强调"共享库的代码可以被所有执行中的进程共享"的量化含义——注意前提是"不同程序共用同一库"；若 100 个进程跑的是同一个可执行文件，静态链接也能靠页缓存共享，此时共享库的优势主要体现在磁盘与可维护性上。

Lecture 8: 程序设计与调试方法论 (Design and Debugging)

> **讲义对应**：CMU 15–213 Lecture 8 — Design and Debugging（素材：`F25-08-design-debugging.txt`；配套 `F25-gdb-and-assembly.txt`、`lab2_slides.txt`） **教材对应**：CS:APP3e 第 3 章/第 9 章延伸 + 第 5 章"程序性能"方法论（教材无专章，属课程补充） **关联 Lab**：全部 lab（尤其 L2 Bomb / L3 Attack 的 GDB 使用）

8.1 概述

前七讲一路向下：从位与整数（Lecture 2）、机器级程序与栈帧（Lecture 3–6）到链接（Lecture 7），我们学会了"程序在机器上变成了什么"。本讲把视角拉回**人**：如何写出别人（以及未来的自己）能读懂的代码，以及当代码行为与规格不符时，如何**系统性地**找到根因而非靠猜。

核心问题有两个：其一，**设计（design）**——程序复杂度早已超出人脑可一次容纳的规模，靠什么把它切成可管理的块？其二，**调试（debugging）**——从"看到失效"到"定位缺陷"之间，存在一条由观察、假设、实验、诊断构成的科学方法链，工具（GDB、Valgrind、Sanitizer）只是链上的仪器。本讲是后续所有 lab 的**方法论底座**：L2 Bomb 与 L3 Attack 全靠 GDB 逐指令推进，L5 Malloc 靠 Valgrind/ASan 抓堆错误，L4/L5 的性能依赖 `perf` 定位热点。

8.2 核心概念与底层机制图解

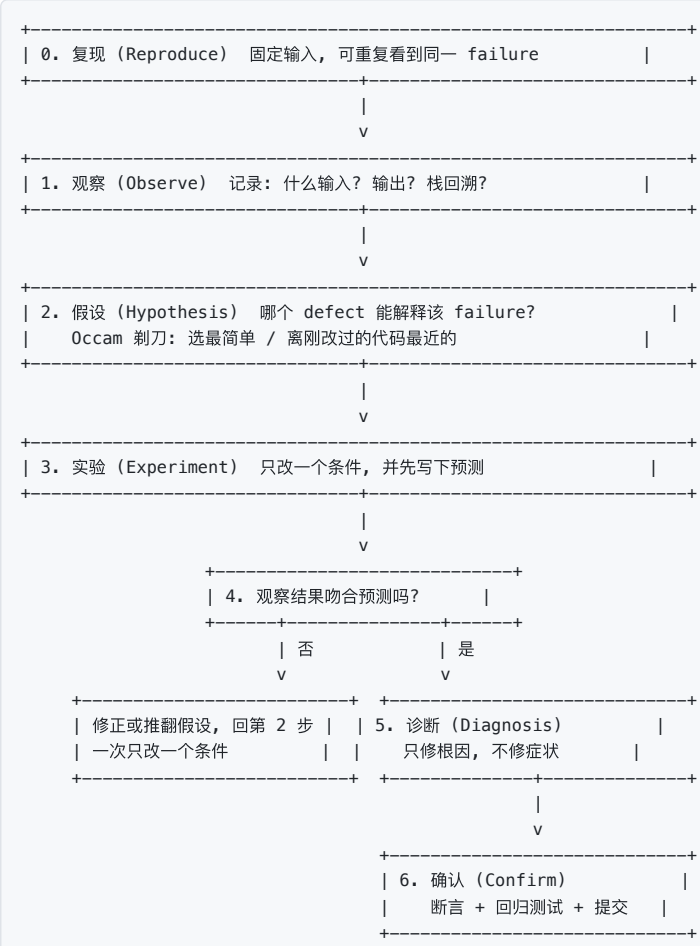
8.2.1 缺陷、错误与失效链（Defect → Error → Failure）

- **定义与目的**：把"程序坏了"拆成可分别讨论的四步链条，是调试方法论的起点。讲义给出的链条是：①程序员制造 **defect（缺陷）**；②缺陷*可能*引起 **error（错误状态）**——数据值或控制信号出错；③错误**传播（propagate）**；④错误最终导致 **failure（失效）**——某部件在某接口上未产生预期结果。
- **直观解释**：像一栋楼里一根钢筋少绑了一道。少绑（defect）不一定马上出问题；某天它让某层楼板轻微变形（error）；变形被后续装修掩盖；直到某次地震才塌下来（failure）。**缺陷与失效之间隔着很长的因果链**，这正是调试困难的根源。
- **底层机制图解**：

- **与机器码/硬件的对应**：`latent defect` 在汇编层面就是**一条从未被执行的指令**（如 `-O3` 优化掉的死分支）；`masked error` 有真实硬件对应——ECC 内存用校验位纠正单位翻转，使 error 不上升为 failure。讲义特意点出：**error 未必是 failure，因为它可被掩蔽或检测**。

8.2.2 科学调试（Scientific Debugging）

- **定义与目的**：调试不是"盯着代码猜"，而是把每次修改当作**受控实验**。讲义给出的循环是：Problem Description → Hypothesis → Prediction → Experiment → Observation & Conclusion，最后收敛到 Diagnosis → Fix → Confirm。
- **直观解释**：像医生看病。病人说"肚子疼"（failure）；医生提出"可能是阑尾炎"（hypothesis）；预测"按压右下腹会痛"（prediction）；做该动作（experiment）；观察反应；预测落空就换假设。绝不因"最近流行阑尾炎"就直接开刀。讲义引用 Occam 剃刀：**若多个假设都能解释现象，选最简单、离当前工作最近的那个**。
- **底层机制图解**（调试工作流决策树）：



- **与机器码/硬件的对应**：实验的"条件"在底层就是**程序输入、调试器里改写的内存/寄存器、改源码重编译**三类。讲义提醒："观察"本身可能干扰实验——`printf()` 改变栈布局与寄存器使用，**对内存越界、未初始化读这类 bug 加 `printf` 会让它消失**，就像量子物理里观测影响被观测对象。故内存类 bug 应优先用不侵入的手段（Valgrind / Sanitizer / 硬件 watchpoint）。

8.2.3 用设计管理复杂度（Managing Complexity）

- **定义与目的**：好的设计要同时满足性能、可用性、可修改性、可移植性、扩展性、安全性、可测试性与成本，但讲义的口号是 **"above all else: it must be readable"（最重要是可读）**——可读性是其余属性长期维持的前提。
- **直观解释**：复杂度管理就像把杂物间改成有标签的抽屉柜——**分离关注点（separation of concerns）**是"每个抽屉只放一类东西"；**封装（encapsulation）**是"抽屉关上就看不见乱"；**抽象（abstraction）/信息隐藏（information hiding）**是"只需知道抽屉外写了什么"；DRY 是"同一件东西别放两个抽屉"。
- **底层机制图解**：讲义用一个例子说明"把大任务拆成可测试的小块"——缓存访问：

```
一次 cache 访问（不可直接测试的一大坨）
|
+-- 1. 把地址拆成 tag / set index / block offset    <- 纯函数, 可单测
+-- 2. 用 set index 查组                             <- 数据结构访问
+-- 3. 组内 tag 比对 -> hit?                        <- 判定逻辑
+-- 4. 未命中: 找 LRU 行 -> 驱逐 -> 从内存读入新行   <- 策略
+-- 5. 更新 LRU 计数                                <- 状态维护
+-- 6. 若是 store: 置 dirty 位                      <- 状态维护
```

拆开后步骤 1 是**纯函数**（地址 → 三个数），可用断言直接验证；步骤 3 可拿构造好的组内容单独测。这就是"**Designs need to be testable / Testable design is modular**"。 – **与机器码/硬件的对应**：讲义有幻灯片 **"Trust the Compiler!"**：多写临时变量、多拆函数，让编译器做内联与寄存器分配——函数拆分几乎无代价。但 lab 里要防止编译器把被测函数优化掉（内联后热点函数名消失），需用 `__attribute__((noinline))`（见 8.3.4）。

8.2.4 沟通：命名与注释（Communication）

- **定义与目的**：写代码是向**四种读者**沟通：机器、同事、代码评审者、未来的自己。手段包括测试、命名、注释、commit message、代码评审与设计模式。

- **直观解释**：命名即理解——讲义引用 Sam Gardiner："如果你不知道一个东西该叫什么，你就不知道它是什么；不知道它是什么，你就坐不下来写代码。"命名规则：**从意义与意图出发；用含义精确的词（避免 data、info、perform）；不超过四个词；避免缩写；用代码评审改进；读出声检查；然后真的去改名**。只用字典里的词（`FileCpy` → `FileCopy`）；避免单字母名（`i`、Unix 惯用的 `fd`、`str` 例外）；用问题域术语（cachelab 用 line、element）；反义词成对一致（`first/end` → `first/last`）。
- **底层机制图解**：讲义用一个真实对比例子说明"注释不该说代码做什么，而该说代码为什么存在"：

```
不要写（代码已自述在做什么）：
    // 把 bp 的前驱的后继指向 bp 的后继
    (*(void **)((*(void **)(bp)) + DSIZE)) = (*(void **)(bp + DSIZE));

应该写（代码自己的表达力）：
    bp->prev->next = bp->next;

应该写（解释 why / 何时用）：
    // 每个地址是 64 位，即 16 个十六进制字符，加结束符共 17 字节
    const int MAX_ADDRESS_LENGTH = 17;
```

这对例子直接对应 L5 Malloc 的显式空闲链表删除：**先把代码改清楚，而不是给晦涩代码补注释**。讲义"Code by commenting"四步：①先写单行短注释描述将做什么（如快排的 `// 初始化局部量 / 选主元 / 按主元重排 / 递归`）；②照注释写代码；③修订；④持续维护。

8.3 代码示例与底层机制分析

8.3.1 一个"五处缺陷"的程序：Valgrind 与 AddressSanitizer 实测

⚠️ **仅供演示，请勿模仿**——`buggy.c` 故意写错 4 类内存缺陷。

```c
/* buggy.c — 刻意含四类经典内存缺陷
 * gcc -g -O0 -Wall -std=c11 buggy.c -o buggy
 * valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./buggy
 */
#include <stdio.h>
#include <stdlib.h>

#define LEN 5

static int *make_table(int n) {
    int *t = malloc((size_t)n * sizeof(int));   /* (1) 只分 n 个 int */
    for (int i = 0; i < n; i++)
        t[i] = i * i;
    return t;
}

static int sum_wrong(int *t, int n) {
    int s = 0;
    for (int i = 0; i <= n; i++)                /* (2) 越界: i==n 时多读一个 */
        s += t[i];
    return s;
}

static int use_after_free(void) {
    int *p = malloc(sizeof(int));
    *p = 42;
    free(p);
    return *p;                                  /* (3) use-after-free */
}

static int read_uninit(void) {
    int *a = malloc(3 * sizeof(int));
    a[0] = 1;
    return a[1] + a[2];                         /* (4) 读未初始化内存 */
}

int main(void) {
    int *tab = make_table(LEN);
    printf("sum = %d\n", sum_wrong(tab, LEN));
    free(tab);

    printf("leaked block sum = %d\n", sum_wrong(make_table(3), 3));  /* (5) 指针丢失=泄漏 */
    printf("uaf value = %d\n", use_after_free());
    printf("uninit value = %d\n", read_uninit());
    return 0;
}
```

【代码做什么？】 1. `make_table(n)` 分配 `n` 个 `int`、填入 `i*i`，返回首地址。 2. `sum_wrong(t, n)` 用 `i <= n`，多读一个元素——**局部越界读**。 3. `use_after_free()` 释放后仍读 `*p`——**释放后使用**。 4. `read_uninit()` 只写 `a[0]` 却读 `a[1]`、`a[2]`——**读未初始化堆内存**。 5. `main` 把 `make_table(3)` 的返回值直接当实参、不存指针——**泄漏**。

【底层机制透视】 – 越界读能"跑出结果"，是因为 `malloc` 为 20 字节请求返回了 24 字节对齐块（glibc chunk 头 + 对齐），多出的 4 字节是**堆元数据或上一块数据**，读它不触发缺页，于是静默拿到垃圾值。 – `use_after_free` 读到 42 而不崩，是因为 `free` 后小块进入 **tcache（thread cache）**，前 8 字节被改写成 `key`、第二 8 字节写成 `next` 指针，而 `*p` 读偏移 0 处 4 字节——恰好未被覆盖，于是拿到旧值。**缺陷在逻辑上，机器状态却完全合法**，这正是"error 被掩蔽"的典型。 – `read_uninit` 读到 0，是因为该内存刚由 `brk` / `mmap` 从内核获得，页表项指向**全零物理页**。

【内存布局 / 数据结构图解】（64 位下 glibc chunk 与 tcache 复用）：

```
   用户看到                实际堆块 (malloc(20))
   +--------------+ 0x4a6f040 <-- make_table 返回的 t
   | t[0] = 0     |   \
   | t[1] = 1     |    |  20 字节用户区
   | t[2] = 4     |    |
   | t[3] = 9     |    |
   | t[4] = 16    |   /
   +--------------+ 0x4a6f054 <-- 越界读的就是这 4 字节
   | 元数据/相邻块 |   Valgrind: 0 bytes after a block of size 20
   +--------------+
```

```
   free(p) 后小块进入 tcache:
   +---------------------+ <-- tcache 链表头
   | key (8 字节)        |    分配器写的"已释放"标记
   +---------------------+
   | next (8 字节)       |    指向下一个空闲块
   +---------------------+
   | 残留数据 (4 字节)   |    use_after_free() 读到的 42
   +---------------------+
```

【实测验证】（本机真实运行结果）

```
$ gcc -g -O0 -Wall -std=c11 buggy.c -o buggy          # -Wall 已能抓到两处
buggy.c:28:12: warning: pointer 'p' used after 'free' [-Wuse-after-free]
$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./buggy
==3514292== Invalid read of size 4
==3514292==    at 0x4011CC: sum_wrong (buggy.c:20)
==3514292==    by 0x401274: main (buggy.c:39)
==3514292==  Address 0x4a6f054 is 0 bytes after a block of size 20 alloc'd
==3514292==    at 0x484486F: malloc (vg_replace_malloc.c:381)
==3514292==    by 0x401161: make_table (buggy.c:11)
==3514292==    by 0x40125F: main (buggy.c:38)
==3514292==
==3514292== Invalid read of size 4
==3514292==    at 0x401212: use_after_free (buggy.c:28)
==3514292==  Address 0x4a70130 is 0 bytes inside a block of size 4 free'd
==3514292==    by 0x40120D: use_after_free (buggy.c:27)
==3514292==
==3514292== Conditional jump or move depends on uninitialised value(s)
==3514292==    by 0x4012E5: main (buggy.c:44)
==3514292==  Uninitialised value was created by a heap allocation
==3514292==    by 0x401227: read_uninit (buggy.c:32)
...
==3514292== HEAP SUMMARY:
==3514292==     in use at exit: 4,120 bytes in 3 blocks
==3514292==   total heap usage: 5 allocs, 2 frees, 4,144 bytes allocated
==3514292==
==3514292== 12 bytes in 1 blocks are definitely lost in loss record 1 of 3
==3514292==    by 0x401161: make_table (buggy.c:11)
==3514292==    by 0x40129B: main (buggy.c:42)
==3514292== 12 bytes in 1 blocks are definitely lost in loss record 2 of 3
==3514292==    by 0x401227: read_uninit (buggy.c:32)
==3514292== 4,096 bytes in 1 blocks are still reachable in loss record 3 of 3
==3514292==    by 0x48EE693: _IO_file_doallocate (in /usr/lib64/libc.so.6)
==3514292== LEAK SUMMARY:
==3514292==    definitely lost: 24 bytes in 2 blocks
==3514292==    indirectly lost: 0 bytes in 0 blocks
==3514292==      possibly lost: 0 bytes in 0 blocks
==3514292==    still reachable: 4,096 bytes in 1 blocks
==3514292== ERROR SUMMARY: 10 errors from 10 contexts (suppressed: 0 from 0)
```

输出解读四件事：①**定位源码行**（`sum_wrong (buggy.c:20)`）；②区分 `Invalid read/write`（非法访问）与 `Conditional jump depends on uninitialised value`（未初始化值参与判断）；③读泄漏分类——`definitely lost`（找不到指针）**必须修**，`still reachable`（仍被指向，本例是 libc 的 stdout 缓冲）通常无害；④**只修 `definitely lost` 这两处**。

AddressSanitizer 的对应输出（本机为 32 位构建，见下面"补充说明"）：

```
$ gcc -fsanitize=address -g -O0 buggy.c -o buggy_asan && ./buggy_asan
=================================================================
==3744690==ERROR: AddressSanitizer: heap-buffer-overflow on address 0xf4e00b94
READ of size 4 at 0xf4e00b94 thread T0
    #0 0x80492d1 in sum_wrong /tmp/csapp08/buggy.c:20
    #1 0x8049488 in main /tmp/csapp08/buggy.c:39
0xf4e00b94 is located 0 bytes to the right of 20-byte region [0xf4e00b80,0xf4e00b94)
allocated by thread T0 here:
    #1 0x8049204 in make_table /tmp/csapp08/buggy.c:11
    #2 0x8049475 in main /tmp/csapp08/buggy.c:38
SUMMARY: AddressSanitizer: heap-buffer-overflow /tmp/csapp08/buggy.c:20 in sum_wrong
Shadow bytes around the buggy address:
  0x3e9c0160: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
=>0x3e9c0170: 00 00[04]fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend: Addressable: 00   Partially addressable: 01-07
  Heap left redzone: fa   Freed heap region: fd
```

ASan 的 `04` 表示该 8 字节 granule 只有前 4 字节可寻址，`fa` 是**堆红区（redzone）**。若程序只含泄漏，退出时由 LeakSanitizer 报告：

```
$ ./leakonly_asan
==3755933==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 40 byte(s) in 1 object(s) allocated from:
    #1 0x80491e8 in main /tmp/csapp08/leakonly.c:5
SUMMARY: AddressSanitizer: 40 byte(s) leaked in 1 allocation(s).
```

未定义行为由 UBSan 抓（**只报告、不中止**，程序继续跑完）：

```
$ gcc -fsanitize=undefined -g -O0 ovf.c -o ovf_ub && ./ovf_ub
ovf.c:5:5:  runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'
ovf.c:10:32: runtime error: left shift of 1 by 31 places cannot be represented in type 'int'
x+1 = -2147483648
u-2 = 4294967295          <- unsigned 回绕是"有定义"行为, UBSan 不报
y/2 = 0
(1<<31) = -2147483648
```

补充说明：ASan 需预留约 1/8 影子内存：64 位用户空间 47 位（128 TB）对应约 16 TB 影子区；本机 `ulimit -v` 限为 32 GB，64 位 ASan 无法预留而报 `ReserveShadowMemoryRange failed`，故本章改用 32 位 ASan（影子区仅数百 MB）取得真实输出。遇到同样报错就查 `ulimit -v`。

8.3.2 用 GDB 从一个段错误找到根因

⚠️ **仅供演示，请勿模仿**——`make_node` 忘记把 `next` 置 `NULL`。

```c
/* crashdemo.c — GDB 演示: 链表尾节点的 next 从未初始化
 * gcc -g -O0 -Wall -std=c11 crashdemo.c -o crashdemo
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct node {
    struct node *next;      /* 偏移 0 */
    int value;              /* 偏移 8 */
} node_t;

static node_t *make_node(int value)
{
    node_t *n = malloc(sizeof(node_t));
    if (n == NULL) { perror("malloc"); exit(1); }
    n->value = value;
    return n;                       /* 缺陷: 缺少 n->next = NULL */
}

static int node_value(const node_t *n)
{
    return n->value;                /* 野指针解引用在此发生 */
}

static int sum_list(const node_t *head)
{
    int total = 0;
    const node_t *p = head;
    while (p != NULL) {             /* 依赖 next == NULL 终止 */
        total += node_value(p);
        p = p->next;
    }
    return total;
}

int main(void)
{
    node_t *first  = make_node(10);
    node_t *second = make_node(20);

    /* 让第三个节点复用刚释放的堆内存, 内容已被分配器写成垃圾 */
    void *poison = malloc(sizeof(node_t));
    memset(poison, 0xFF, sizeof(node_t));
    free(poison);
    node_t *third = make_node(30);

    first->next  = second;
    second->next = third;           /* third->next 仍是垃圾值 */
    printf("first=%p second=%p third=%p third->next=%p\n",
           (void *)first, (void *)second, (void *)third, (void *)third->next);
    printf("sum = %d\n", sum_list(first));
    return 0;
}
```

【代码做什么？】 1. `node_t` 把 `next` 放偏移 0、`value` 放偏移 8，使未初始化值恰落在 `next` 上。 2. `make_node` 分配节点、写 `value`，**不写 `next`**。 3. `main` 先把同尺寸内存 `memset(..., 0xFF, ...)` 涂满再 `free`，随后 `make_node(30)` **复用**该块，于是 `third->next` 是未定义字节。 4. `sum_list` 靠 `next == NULL` 终止遍历，拿到垃圾指针后解引用 → SIGSEGV。

【底层机制透视】 `free` 后的块进入 tcache 时，分配器只改写**头部 8 字节的 `key` 字段**（变成 `0x405`，即 tcache 结构地址的低位），而偏移 0 的 `next` 恰读这 8 字节，于是遍历拿到的"下一节点"是 `0x405`——几乎必然未映射。**危险在于它"有时不崩"**：若残留字节凑巧为 0，程序会静默少统计一个节点，即讲义所说的 latent defect。

【内存布局 / 数据结构图解】（崩溃瞬间 `bt` 与真实栈内容的对应，地址取自本机实测）

```
bt 帧号              地址                内容            归属
                    (高地址在上)
 #2  main    +------------------------+
             0x7fffffffb6b8 | 0x00007ffff7c3feb0 | main 返回地址
             0x7fffffffb6b0 | 0x0000000000000001 | <- main rbp
             0x7fffffffb6a8 | 0x00000000004052a0 | first
             0x7fffffffb6a0 | 0x00000000004052e0 | third
             0x7fffffffb698 | 0x00000000004052c0 | second
 #1 sum_list +------------------------+
             0x7fffffffb688 | 0x00000000004012ba | ret -> main (:52)
             0x7fffffffb680 | 0x00007fffffffb6b0 | <- sum_list rbp
             0x7fffffffb678 | 0x0000003cf7ffe220 | 高 4 字节 0x3c = 60 = total
             0x7fffffffb670 | 0x0000000000000405 | 局部量 p <== 野指针!
             0x7fffffffb668 | 0x00000000004052a0 | head 的副本
 #0 node_value +----------------------+
             0x7fffffffb660 | 0x00000000004011f4 | ret -> sum_list (:31)
             0x7fffffffb658 | 0x00007fffffffb680 | <- node_value rbp == rsp
             +------------------------+ (低地址)

 规则: 每帧首条 `push %rbp` 压入调用者的 rbp,
       紧随的 `call` 压入返回地址 —— 二者在栈上相邻,
       这就是 `bt` 沿 rbp 链走回 main 的物理基础。
```

【与汇编 / 硬件的对应】（`objdump -d -M intel`，节选）

```
00000000004011cb <sum_list>:
  4011cf:  48 83 ec 18        sub    rsp,0x18
  4011d3:  48 89 7d e8        mov    QWORD PTR [rbp-0x18],rdi   ; head -> [rbp-0x18]
  4011d7:  c7 45 fc 00 00 00 00  mov DWORD PTR [rbp-0x4],0x0      ; total = 0
  4011e2:  48 89 45 f0        mov    QWORD PTR [rbp-0x10],rax   ; p = head -> [rbp-0x10]
  4011e8:  48 8b 45 f0        mov    rax,QWORD PTR [rbp-0x10]   ; rax = p
  4011ec:  48 89 c7           mov    rdi,rax
  4011ef:  e8 c6 ff ff ff     call   4011ba <node_value>        ; 返回地址 0x4011f4 入栈
  4011f4:  01 45 fc           add    DWORD PTR [rbp-0x4],eax    ; total += ret
  4011fb:  48 8b 00           mov    rax,QWORD PTR [rax]        ; rax = p->next <== 读偏移 0
  401202:  48 83 7d f0 00     cmp    QWORD PTR [rbp-0x10],0x0
  401207:  75 df              jne    4011e8                     ; 非 NULL 继续
```

这是 `-O0` 的教科书式代码：**局部量全落在栈帧上**（`total` 在 `rbp-0x4`、`p` 在 `rbp-0x10`、`head` 在 `rbp-0x18`），与上面的栈图逐格对应。`node_value` 的崩溃指令是 `mov 0x8(%rax),%eax`。

【实测验证】（完整 GDB 会话）

```
$ gcc -g -O0 -Wall -std=c11 crashdemo.c -o crashdemo
$ ./crashdemo
Segmentation fault (core dumped)          # 退出码 139 = 128 + SIGSEGV(11)
$ gdb -q ./crashdemo
(gdb) break crashdemo.c:51
Breakpoint 1 at 0x401286: file crashdemo.c, line 51.
(gdb) run
Breakpoint 1, main () at crashdemo.c:51
(gdb) p first
$1 = (node_t *) 0x4052a0
(gdb) p third
$2 = (node_t *) 0x4052e0
(gdb) p *third
$3 = {next = 0x405, value = 30}          <== 一眼看出 next 是垃圾值
(gdb) p/x third->next
$4 = 0x405
(gdb) x/6gx first
0x4052a0:  0x00000000004052c0  0x000000000000000a   <-- first: next, value=10
0x4052b0:  0x0000000000000000  0x0000000000000021   <-- 块头/尾, 0x21=33
0x4052c0:  0x00000000004052e0  0x0000000000000014   <-- second: next, value=20
(gdb) continue
Program received signal SIGSEGV, Segmentation fault.
0x00000000004011c6 in node_value (n=0x405) at crashdemo.c:23
23          return n->value;
(gdb) bt
#0  0x00000000004011c6 in node_value (n=0x405) at crashdemo.c:23
#1  0x00000000004011f4 in sum_list (head=0x4052a0) at crashdemo.c:31
#2  0x00000000004012ba in main () at crashdemo.c:52
(gdb) p n
$5 = (const node_t *) 0x405
(gdb) frame 1
#1  0x00000000004011f4 in sum_list (head=0x4052a0) at crashdemo.c:31
(gdb) p p
$6 = (const node_t *) 0x405              <== 野指针的来源
(gdb) p total
$7 = 60                                  <== 前两个节点 10+20 已累加, 说明崩在第三个
(gdb) info registers rip rsp rbp
rip  0x4011f4   0x4011f4 <sum_list+41>
rsp  0x7fffffffb668
rbp  0x7fffffffb680
```

推理链条：`bt` 说明崩在 `node_value`、调用者是 `sum_list` 第 32 行；`total = 60` 说明 10 与 20 已正确处理；`p = 0x405` 说明**第三个节点被当成有效节点**；回 `main` 帧 `p *third` 看到 `next = 0x405`；而 `make_node` 从未给 `next` 赋值 → **根因是初始化缺失，不是循环写错**。修复只需一行 `n->next = NULL;`，并把不变量写成断言：

```
/* fixed.c 关键片段: 把未初始化从运行期隐患变成设计期约束 */
static node_t *make_node(int value)
{
    node_t *n = malloc(sizeof(node_t));
    if (n == NULL) { perror("make_node: malloc"); exit(EXIT_FAILURE); }
    n->next  = NULL;                    /* 修复: 显式初始化 */
    n->value = value;
    return n;
}

static int node_value(const node_t *n)
{
    assert(n != NULL);                  /* 断言只做检查, 无副作用 */
    return n->value;
}
```

修好后 `third->next = (nil)`、`sum = 60`，Valgrind 只剩 `still reachable`（libc 的 stdout 缓冲），`definitely lost: 0 bytes`。

【GDB 命令速查表】（`x/nfu addr`：n 个数、f 格式、u 单位）

分类	命令	作用与提示
断点	`b func`	函数入口下断点
断点	`b file.c:80`	源码第 80 行下断点
断点	`b *0x4011c6`	按**指令地址**下断点（无符号时唯一手段）

分类	命令	作用与提示
断点	`b func if n == 3`	条件断点，只在关心那次停下
断点	`tbreak ...`	临时断点，命中一次即删
断点	`delete [N]` / `d`	删第 N 个断点；无参删全部
断点	`disable N` / `enable N`	停用/启用第 N 个（`dis` 是 disable，非 `disas`）
断点	`info b`	列出断点与命中次数
断点	`watch expr`	值改变时停下
断点	`watch *(int *)0x600850`	观察指定地址变化
执行	`run` / `r` / `r a1 a2`	从头（重）运行；默认沿用上次参数
执行	`continue` / `c`	继续到下一断点或信号
执行	`next` / `n` / `n X`	单步 C 一行，不进入函数
执行	`step` / `s` / `s X`	单步一行，**进入**函数
执行	`nexti` / `ni`	单步一条汇编，不进入 `call`
执行	`stepi` / `si`	单步一条汇编，**进入** `call`
执行	`finish` / `f`	跑到函数返回并打印返回值
执行	`until 38`	跑到第 38 行或跳出当前帧（跳循环利器）
执行	`Ctrl+C`	中断运行中的程序，回到提示符
查看数据	`p expr`	求值任意 C 表达式（`p *ptr`、`p &arr`）
查看数据	`p/x` / `p/d` / `p/t`	以十六/十/二进制打印
查看数据	`p *(long*)ptr`	按指定类型解释内存
查看数据	`x/8d &arr`	从 `arr` 起打印 8 个十进制整数
查看数据	`x/16gx $rsp`	栈顶 16 个 8 字节字（看栈帧标准手法）
查看数据	`x/s addr` / `x/2s`	按 C 字符串打印
查看数据	`x/6cb addr`	按字符打印 6 字节
查看数据	`display expr`	每次停下自动打印该表达式
查看数据	`set var x = expr`	改写变量/内存做实验
查看数据	`call func(args)`	在调试器里调程序自己的函数
栈与帧	`bt` / `backtrace`	打印调用栈（崩溃后第一件事）
栈与帧	`frame N` / `f N`	切到第 N 帧（N 即 `bt` 左边编号）
栈与帧	`up N` / `down N`	沿调用链上/下移 N 帧
栈与帧	`info frame`	当前帧基址、保存的 `rip`、局部量位置
栈与帧	`info args` / `info locals`	打印当前帧形参与局部量
栈与帧	`info registers [r...]`	打印全部寄存器或只看指定几个
栈与帧	`thread apply all bt`	**死锁排查核心**：打印各线程栈
反汇编	`disas`	反汇编当前函数
反汇编	`disas func`	反汇编指定函数
反汇编	`disas /r func`	同时显示机器码字节
反汇编	`x/i $rip` / `x/4i $rip`	反汇编当前指令及后几条
反汇编	`objdump -d -M intel p`	外部反汇编（Intel 语法更易读）
TUI	`gdb -tui ./prog`	启动 TUI 界面
TUI	`layout {src,asm,regs,split}`	切换源码/汇编/寄存器/分栏布局

分类	命令	作用与提示
TUI	`Ctrl+L`	重绘花屏；`bomblab` 下 TUI 偶不兼容
其它	`list` / `l`（可接 `func` 或行号）	列出源码
其它	`gdb --args p a1`	带参数启动；`gdb -p PID` 附着进程

8.3.3 从 core dump 做尸检（post-mortem）

若程序不在自己终端里崩、别人只给你一个 core 文件，流程如下（本机 `core_pattern` 由 `systemd-coredump` 接管，故用 `generate-core-file` 生成等价文件）：

```
$ ulimit -c unlimited                # 允许产生 core
$ ./crashdemo                        # Segmentation fault (core dumped)
$ gdb -q ./crashdemo cd.core         # 直接读 core, 无需重跑程序
Core was generated by `/tmp/csapp08/verify/crashdemo'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x00000000004011c6 in node_value (n=0x405) at crashdemo.c:23
#1  0x00000000004011f4 in sum_list (head=0x4052a0) at crashdemo.c:31
#2  0x00000000004012ba in main () at crashdemo.c:52
(gdb) info registers rip
rip   0x4011c6   0x4011c6 <node_value+12>
(gdb) frame 1
(gdb) info frame
Stack level 1, frame at 0x7fffffffb690:
 rip = 0x4011f4 in sum_list (crashdemo.c:31); saved rip = 0x4012ba
 Arglist at 0x7fffffffb680, args: head=0x4052a0
 Locals at 0x7fffffffb680, Previous frame's sp is 0x7fffffffb690
 Saved registers:
  rbp at 0x7fffffffb680, rip at 0x7fffffffb688
(gdb) info locals
total = 60
p = 0x405
```

`info frame` 把"`bt` 里抽象的一帧"翻译成**具体地址**：`saved rbp` 在 `0x7fffffffb680`、`saved rip` 在 `0x7fffffffb688`，与 8.3.2 的栈图吻合。该技巧在 L3 Attack 同样关键。

8.3.4 性能调试：找到热点（hot spot）

```
/* hot.c — 制造明显热点, 供 perf / gprof / callgrind 定位 */
#include <stdio.h>
#include <stdlib.h>

#define N 20000

__attribute__((noinline))
static long slow_path(long n) {          /* 热点: O(n^2) 双层循环 */
    long s = 0;
    for (long i = 0; i < n; i++)
        for (long j = 0; j < n; j++)
            s += (i ^ j) & 1;
    return s;
}

__attribute__((noinline))
static long fast_path(long n) {          /* 同结果的 O(n) 版本 */
    return (n * n) / 2;
}

int main(void) {
    long a = slow_path(N);
    long b = fast_path(N);
    printf("slow=%ld fast=%ld\n", a, b);
    return 0;
}
```

`__attribute__((noinline))` 在此是**必要条件**：不加它，`-O1` 也会把两个小函数内联进 `main`，`perf report` 只见 `main` 占 100%，热点函数名消失（本次实验一开始正是如此）。真实输出：

```
$ gcc -O1 -g -fno-omit-frame-pointer hot.c -o hot && time ./hot
slow=200000000 fast=200000000
real  0m0.263s
$ perf stat -e instructions,branches,branch-misses ./hot
   3,200,308,202      instructions:u
     400,082,708      branches:u
          22,870      branch-misses:u    #    0.01% of all branches
$ perf record -q -g -o perf.data ./hot && perf report -i perf.data --stdio --no-children
    99.97%  hot      hot    [.] slow_path
            |
            ---slow_path
               __libc_start_call_main
$ gprof -b ./hot_pg gmon.out          # gcc -pg -O1 -g -fno-omit-frame-pointer
  %   cumulative   self          calls  ms/call  ms/call  name
100.00      0.22     0.22           1   220.00   220.00  slow_path
  0.00      0.22     0.00           1     0.00     0.00  fast_path
$ valgrind --tool=callgrind ./hot && callgrind_annotate cg.out
3,200,322,637 (100.0%)  PROGRAM TOTALS
Ir                      file:function
3,200,120,004 (99.99%)  hot.c:slow_path [/tmp/csapp08/hot]
      100,001 ( 0.00%)      for (long i = 0; i < n; i++)
1,600,020,000 (50.00%)          for (long j = 0; j < n; j++)
1,600,000,000 (49.99%)              s += (i ^ j) & 1;
```

三者差异要记住：**`perf` 是采样式（sampling）**，开销极小（约 1%），适合真机长跑；**`gprof` 需 `-pg` 重编译**，靠 `mcount` 插桩，短程序分辨率差；**Callgrind 是确定性指令级模拟**，能精确到源码行给出 `Ir`（指令计数），但慢 20—100 倍且报的是指令数而非时间。找出热点后，L4 靠局部性/分块优化，L5 靠减少 `malloc` 次数，都必须**先定位热点再动手**，否则优化的只是 1% 的代码。

8.4 实验关联

本讲**直接服务全部 lab**——它是唯一讲"怎么把 bug 找出来"的讲次。

- **L0 / L1（C 编程、Data Lab）**：第一天就用 `-Wall -Wextra -Werror` 编译，并用 `assert` 把每题的**规格（specification）**写成可执行断言——讲义称之为"executable documentation"。
- **L2 Bomb**：GDB 主战场。必须熟练 `b *0x...`、`disas`、`info registers`、`x/s`、`x/40c`、`stepi`。炸弹**无符号信息**，只能按地址下断点、靠 `x/s` 读字符串比对；隐藏关卡入口要从 `phase_defused` 的 `sscanf` 格式串反汇编里找。**坑**：TUI 与 bomblab 偶有兼容问题；`Ctrl+C` 后重跑要按 `y` 确认。
- **L3 Attack**：目标是把 `getbuf` 的返回地址改成 `touch1/2/3`。GDB 是唯一能让你看到"`%rsp` 在 `getbuf` 内指向哪、返回地址在第几字节"的工具，`x/16gx $rsp` 与 `info frame` 是核心。**坑**：栈地址在 GDB 里与直接运行时**可能不同**，重定向 `%rsp`（ROP 关卡）时要用 `nop` sled 或相对偏移。
- **L4 Cache / L5 Malloc**：Valgrind 与 ASan 是主要验收工具。L5 的 `mm.c` 必须做到 `valgrind --leak-check=full` 零 `definitely lost` 且所有块在退出前 `free`；`mdriver` 的 trace 重放本身就是"可复现的测试"。**坑**：Valgrind 拖慢 20—50 倍，超时的 trace 先用 `-O2` 跑通再上。
- **L6 Shell / L7 Proxy / L8 SFS**：多进程/多线程 bug 换武器：`strace -f -e trace=process,network`（看 `fork/exec/wait` 与 `socket/connect/accept` 时序）、`gdb -p PID` + `thread apply all bt`（死锁）、`valgrind --tool=helgrind`（数据竞争、加锁顺序）。本讲实测的死锁现场：

```
$ gcc -g -O0 -pthread deadlock.c -o deadlock && ./deadlock &   # 两把锁加锁顺序相反
$ gdb -q -batch -p $PID -ex 'thread apply all bt'
Thread 3 ... "deadlock":
#2  0x00000000004011ef in worker2 (arg=0x0) at deadlock.c:24   <== 等 lock_a
Thread 2 ... "deadlock":
#2  0x00000000004011a0 in worker1 (arg=0x0) at deadlock.c:13   <== 等 lock_b
Thread 1 ... "deadlock":
#2  0x0000000000401263 in main () at deadlock.c:35              <== 卡在 pthread_join
```

两个工作线程**各卡在对方的锁上**，主线程卡在 `join`——死锁的"标准指纹"。 – **git bisect：定位"哪次提交引入的 bug"**。本讲实测的真实输出（8 个提交、其中第 6 个引入缺陷）：

```
$ git bisect start <bad> <good>
$ git bisect run bash -c 'gcc -O0 prog.c -o /tmp/bp && /tmp/bp | grep -q 100'
a4897244961705e38e0dd7660533365b34907b9c is the first bad commit
    rev 6: speed up clamp (introduces bug)
bisect found first bad commit
```

要点是**测试脚本以退出码表示好/坏**（`grep -q` 命中即 0 = good），`git bisect run` 才能在 $O(\log n)$ 次编译内收敛——把"调试"变成"实验自动化"的范例。

8.5 常见错误与调试技巧

- **把 `-O0` 行为当成真相**：`-O0` 与 `-O3` 表现不同（讲义中 `badfib` 在 `-O0` 打出 `fib(1)=2`、`-O3` 打出 `fib(1)=0`），在某一层"正常"就以为修好，等于把 defect 藏得更深。**调试：至少在 `-O0` 与 `-O3` 各跑一遍**，两种优化级别下 `valgrind` 都要跑（讲义："Valgrind is not perfect，`-O3` 下它可能一个错误都报不出来"）。
- **未初始化变量**：`int f;` 未赋值就返回，值取决于栈上残留字节。**调试**：`-Wall -Werror` 先抓（`error: 'f' may be used uninitialized`）；`valgrind --track-origins=yes` 指出"Uninitialised value was created by a stack allocation"；`gdb` 看 `p f`、`x/8gx $rsp`。
- **越界读写**：多写一个字节，可能很久以后才在 `free` 时崩（堆元数据被破坏）。**调试**：`valgrind` 报 `Invalid write ... 0 bytes after a block of size M alloc'd`；`-fsanitize=address` 给出红区与 shadow bytes；`gdb` 配 `x/16gx ptr-16` 看块头。
- **use-after-free**：`free` 后仍读指针，读到的是 tcache 的 `key` / `next`。**调试**：`-Wall` 在 `-O2` 下给 `-Wuse-after-free`；`valgrind` 报 `Invalid read ... inside a block of size 4 free'd` 并列出 `Block was alloc'd at`；ASan 报 `heap-use-after-free`。习惯写法：`free(p); p = NULL;`。
- **泄漏分类读错**：看到 `still reachable: 4,096 bytes` 就慌。**调试**：`valgrind --leak-check=full --show-leak-kinds=all` 后只看 `definitely lost` / `indirectly lost`；`still reachable` 多半是 libc 的 stdout 缓冲。
- **盲目加 `printf` 调试**：输出成"信息海啸"，而且**会改变程序行为**使 bug 消失。**调试**：改用 `gdb` 的 `display expr`（自动打印、不改源码）、`watch expr`（只在值变化时停），或 `tbreak` + `until` 精确停在关注点。
- **`assert` 里有副作用**：写成 `assert(p = malloc(n))` 或 `assert(++count > 0)`，一旦用 `-DNDEBUG` 关掉断言，`malloc` 就再也不会被调用。**调试**：断言只做**检查**——`assert(p != NULL);` 单独一行；可用 `-DNDEBUG` 关闭，但**绝不能因关断言改变程序语义**。
- **改了多处一起验证**：一次改三个地方，跑通了不知哪个 fix 起作用，跑不通也不知哪个改动引入新问题。**调试**：严格一次一个变量（讲义："Update the conditions incrementally..."），**不要在第一个 bug 前继续往下走**（"Do NOT ever proceed past first bug"）。>60 分钟无进展就按讲义时间盒：0 分钟上 `-Wall` / `valgrind`，1–10 分钟非正式调试，10–60 分钟科学调试，>60 分钟休息或求助。

8.6 关键要点

- **缺陷 ≠ 错误 ≠ 失效**：缺陷只在特定路径上才变成错误，错误常被掩蔽或传播很远才变成失效；**测试只能证明缺陷的存在，不能证明其不存在**（Dijkstra 1972），故调试必须沿因果链逆流而上找 defect。
- **调试是科学方法，不是猜测**：观察 → 假设 → 预测 → 实验 → 结论，一次只改一个条件，并在实验前写下预测；诊断（diagnosis）的判据是"能解释当前观察并对未来条件做出正确预测"，而非"改完不崩"——小心 *post hoc ergo propter hoc*。
- **工具按 bug 类型选择**：`printf` / `display` 管逻辑路径；`gdb` 管机器状态（寄存器、栈、指令）；`valgrind` /ASan 管堆越界、泄漏、未初始化；UBSan 管 UB；`perf` / `gprof` /Callgrind 管热点；`strace` 管系统调用；`thread apply all bt` + Helgrind 管并发。
- **设计的第一目标是可读**：分离关注点、模块化、封装、抽象、信息隐藏、DRY 都是为了把复杂度切到人能一次容纳的规模；"Trust the Compiler"——多写临时变量、多拆函数；命名即理解，注释写 *why* 不写 *what*。
- **修根因，不修症状；修完要固化**：定位 defect 后用断言把不变量写进代码，把这次复现条件加入测试套件，提交时用一行 commit message 记下"做了什么、性能如何、什么还没解决"——测试与契约都是**可执行的文档**。
- **`gdb` 是唯一能"打破程序与机器之间抽象"的工具**：`bt` + `info frame` + `x/16gx $rsp` 把抽象的调用栈变成可读的地址与字节，这是 L2/L3 及一切崩溃、栈破坏问题的通用解法。

8.7 思考题（带答案）

题 1（推理题：给定崩溃现象与栈回溯，判断 bug 在哪） 某程序偶发崩溃，`gdb` 给出：

```
Program received signal SIGSEGV, Segmentation fault.
0x00000000004012a1 in copy_record (dst=0x61a260, src=0x0) at db.c:88
88          memcpy(dst->name, src->name, src->len);
#0  0x00000000004012a1 in copy_record (dst=0x61a260, src=0x0) at db.c:88
#1  0x0000000000401503 in load_all (n=1000) at db.c:140
#2  0x00000000004017c0 in main () at db.c:201
(gdb) frame 1
(gdb) p *records[999]
$1 = {name = 0x0, len = 0, id = 0}
```

请判断：bug 在哪、属哪类缺陷？为什么它"偶发"？要一次抓住它该用什么工具？

答：`frame 1` 显示 `records[999]` 的 `name == NULL`、`len == 0`——**第 1000 个记录从未被正确初始化**，而 `load_all` 无条件把 1000 个记录全送进 `copy_record`。所以缺陷在 `load_all`（db.c:140 附近）**缺少有效性检查**，或更靠前的初始化漏掉最后一个元素；`memcpy` 只是**失效的暴露点，不是根因**——正是讲义说的"正确代码原样传播错误状态"。它偶发，是因为该记录若未被复用而恰是全零页，`src` 就是 `NULL` 直接崩；若残留合法旧指针，`memcpy` 会成功复制垃圾数据而**不崩**（缺陷潜伏、错误被掩蔽）。要一次抓住它应用 **AddressSanitizer**（`gcc -fsanitize=address,undefined -g`）：第一次非法访问就停下并给出完整调用栈与分配历史；`valgrind --track-origins=yes` 亦可。修复：在 `make_record` 显式初始化所有字段，并在 `copy_record` 入口加 `assert(src != NULL && src->name != NULL);`。

题 2（计算题：ASan 影子内存与地址空间预算） AddressSanitizer 为每 8 字节应用内存维护 1 字节影子内存（`shadow = (addr >> 3) + offset`）。设 x86-64 用户态可用地址空间为 47 位（2^{47} 字节 = 128 TiB）。请计算：（a）影子内存需预留多少地址空间？（b）为何本机 `ulimit -v` 为 32 GB 时 64 位 ASan 直接失败，而 32 位 ASan 正常？（c）把 64 位 ASan 的报错 `failed to allocate 0xdfff0001000 bytes` 换算成 TiB 是多少？

答：（a）影子区 = 应用地址空间的 $1/8$，即 $2^{47}/8 = 2^{44}$ 字节 = **16 TiB**（"预留"、按需提交，非真占 16 TiB 物理内存）。（b）64 位 ASan 启动时要用 `mmap(PROT_NONE)` 预留这 16 TiB 的**连续虚拟地址区间**，占用 `RLIMIT_AS`（`ulimit -v`，按**虚拟地址空间**而非物理内存算）的额度；32 GB 远小于 16 TiB，`mmap` 直接失败并报 `ReserveShadowMemoryRange failed`。32 位进程地址空间仅 4 GiB，影子区 512 MiB，稳落在 32 GB 内。(c) $0xdfff0001000 \approx 1.539 \times 10^{13}$ 字节，除以 2^{40} 得约 **14 TiB**，与理论值同量级（ASan 预留 `[0x00007fff8000, 0x10007fff8000)`）。这也解释了"内存充足却跑不了 ASan"——瓶颈是**虚拟地址空间上限**。

题 3（"直觉但错误"题） 同学 A 说："我的程序 `-O3` 下算错，`-O0` 下算对，错误只出现在 `-O3`，所以这是 GCC 的优化 bug。解法是 `-O0` 编译交作业。"这个想法错在哪？请结合 `badfib` 说明并给出正确做法。

答：错在**把"编译器放大了缺陷"误认成"编译器制造了缺陷"**。`badfib` 里 `int f, f0 = 1, f1 = 1;` 的 `f` 从未初始化，`fib(1)` 直接 `return f`——标准的**未定义行为（UB）**。本讲实测：`-O3` 与 `-O1` 都输出 `fib(1)=0`，`-O0` 输出 `fib(1)=2`，讲义原始机器上 `-O1` 给出 `fib(1)=9`。**同一份源码在不同优化级别下结果不同，正是 UB 的指纹**；`-O3` 只是把 `f` 放在恰好为 0 的位置。更危险的是 `-O0` 下的"正确"只是巧合。正确做法：① `gcc -Wall -Werror -O3` 直接报 `error: 'f' may be used uninitialized`；② `valgrind --track-origins=yes ./badfib` 报 `Conditional jump or move depends on uninitialised value(s)` 并指出取值来自 `fib` 的栈分配；③修根因——改为 `int f = 1;`；④ `-O0` 与 `-O3` 下**都**跑一遍；⑤加入回归测试。**"在某一层优化下不崩"从来不是修好的证据。**

题 4（设计题：把不可测的一大坨拆成可测模块） 下面是个"一句话完成一切"的函数，请按讲义"把 wordle 工具拆成三个以上可测组件"的思路拆成若干可单测模块，并说明每块怎么测：

```
int handle(char *line) {           /* 读一行 "guess GYBBY" 并更新状态 */
    char *tok = strtok(line, " \n");
    if (!tok) return -1;
    char *pat = strtok(NULL, " \n");
    for (int i = 0; i < 5; i++)
        if (pat[i] == 'G') fixed[i] = tok[i];
        else if (pat[i] == 'Y') present[i] = tok[i];
    for (int w = 0; w < NWORDS; w++)
        if (matches(words[w])) { score(w); }
    return 0;
}
```

答：按"分离关注点 + 每个函数只做一件事"拆成四块，每块都能独立单测（关键：**输入输出都是纯数据**，不依赖全局状态）：① `parse_feedback(line, guess[6], pattern[6])` —— 只做**词法解析**。测试：`"guess GYBBY\n"` 应得 `guess="guess"`、`pattern="GYBBY"`；缺 pattern、长度错、`NULL` 都返回明确错误码。② `update_constraints(guess, pattern, c)` —— 只做**状态更新**（纯函数）。测试：喂入 `GYBBY` 后断言 `c->fixed[0]=='g'`、`c->present[1]=='u'`，并 `assert` 检查不变量（fixed 与 present 同位置不冲突）。③ `word_satisfies(word, c)` —— 只做**判定**（纯谓词）。测试：对每条约束构造必真/必假的单词穷举验证。④ `rank_candidates(...)` —— 只做**排序**。测试：断言输出顺序稳定（排序键相同要有稳定 tie-break，否则测试会闪烁）。`handle` 退化成 5 行"解析 → 更新 → 过滤 → 排序"的**编排代码**，几乎不可能出错；原版把解析、判定、状态修改、I/O 副作用缠在一起，任一环节出问题都得整体调试。这正是讲义结论：**"Design code to be testable / Try to reuse testable chunks"**。

Lecture 9: 存储层次结构 (The Memory Hierarchy)

> **讲义对应**：CMU 15-213 Lecture 9 — The Memory Hierarchy（素材：`F25-09-memory-hierarchy.txt`） **教材对应**：CS:APP3e 第 6 章 6.1—6.3（6.1.1 存储器抽象、6.1.2—6.1.4 存储技术与趋势、6.2 局部性、6.3 存储层次） **关联 Lab**：**L4 Cache Lab**（Part A 缓存模拟器 `csim`；Part B 矩阵转置优化）

9.1 概述

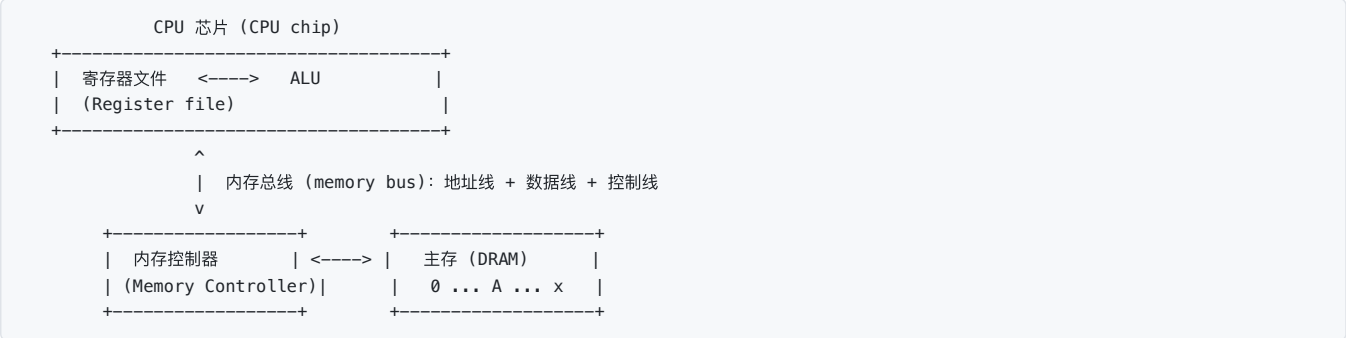
前八讲讨论"计算"，但计算需要数据，而数据存放在"存储器（memory）"里。本讲回答：**存储器的速度永远追不上 CPU，计算机为什么还能跑得这么快?**

答案由三个事实拼成：快的存储器又小又贵又费电；CPU 与主存的速度差距持续拉大（**内存墙，memory wall**）；写得好的程序天然具有**局部性（locality）**。三者咬合，催生了**存储层次结构（memory hierarchy）**——用便宜的慢速设备海量堆积，却以接近顶级速度交付数据。本讲为第 10 讲与 L4 Cache Lab 打地基。

9.2 核心概念与底层机制图解

9.2.1 存储器抽象与总线事务（Memory Abstraction & Bus Transaction）

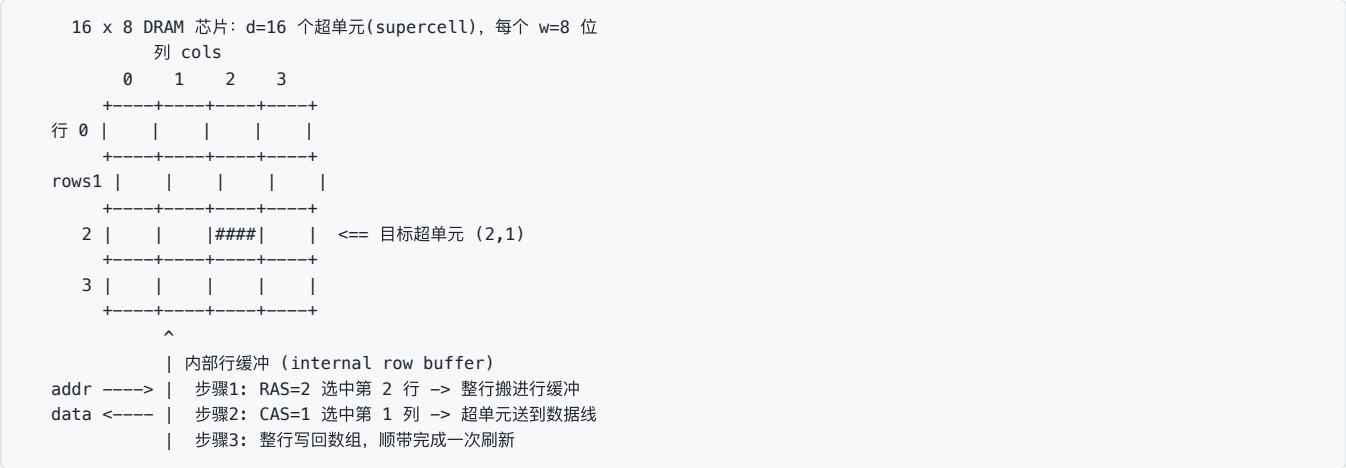
存储器就是**巨大的字节数组**，用地址索引：`movq A, %rax` 读、`movq %rax, A` 写。**总线像共享传送带**，CPU 与主存在两端，地址、数据、控制信号同时跑，故任一时刻只容一个"包裹"。



读事务三步：地址 A 上总线 → 主存取字 x 放回总线 → CPU 写入 `%rax`，写事务反向。**每次访存占用总线若干周期，这就是"访问时间"的物理来源。**

9.2.2 随机访问存储器：SRAM 与 DRAM（Random-Access Memory）

RAM 封装为芯片，基本单元是**单元（cell）**，每 cell 一位；分 **SRAM（静态）**与 **DRAM（动态）**。**SRAM 像带自锁开关的灯**——通电即稳定保持，代价是每 bit 要 6 晶体管；**DRAM 像漏水的桶**——以水量表示 0/1，须周期重灌（刷新，refresh），好处是每 bit 仅需 1 晶体管 + 1 电容。



`movq A, %rax` 被翻译成 RAS/CAS 时序；**整行搬入行缓冲就是 DRAM 的空间局部性红利**——同一行后续访问无需重发 RAS（行命中）。

	每 bit 晶体管数	访问时间	需刷新?	需 EDC?	相对成本	典型用途
SRAM	6 或 8	**1×**	否	也许	约 **100×**	缓存（cache memories）
DRAM	1	**10×**	是	是	**1×**	主存、帧缓冲（frame buffers）

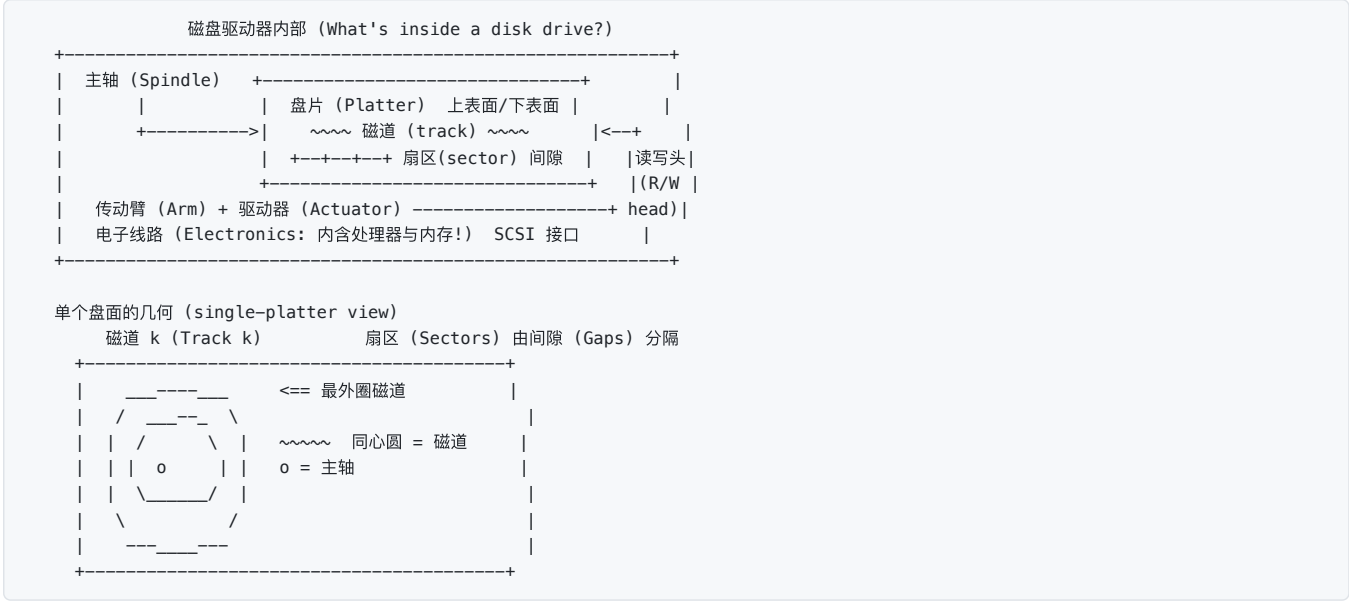
（EDC = 错误检测与纠正。）SRAM 随工艺缩放已近极限；DRAM 缩放受最小电容与深宽比限制，同样近极限。

9.2.3 非易失性存储器（Nonvolatile Memories）

DRAM/SRAM **易失（volatile）**，断电即失；**非易失存储器**断电仍保留数据。谱系：**ROM**（生产时写入）→ **PROM**（可编程一次）→ **EPROM**（紫外线擦除；讲义此处直接跳到 EEPROM，二者同属一谱系）→ **EEPROM**（电可擦除）→ **闪存（flash）**：一种 EEPROM，支持**块级（block-level）部分擦除**，擦写约 10 万次后磨损；另有 **3D XPoint（Intel Optane）等新兴 NVM**。用于固件（BIOS、磁盘/网卡控制器）、SSD 与磁盘缓存。**闪存像一本只能"整页撕掉"才能改的笔记本。**

9.2.4 磁盘存储：几何结构（Disk Geometry）

磁盘以磁性介质存储、机电方式访问；**直观**像老式唱机——盘片转，唱臂径向找"音轨"。



容量公式：

$$\text{Capacity} = \frac{\#\text{bytes}}{\text{sector}} \times \frac{\#\text{avg sectors}}{\text{track}} \times \frac{\#\text{tracks}}{\text{surface}} \times \frac{\#\text{surfaces}}{\text{platter}} \times \frac{\#\text{platters}}{\text{disk}}$$

技术因子：记录密度（bits/in）× 磁道密度（tracks/in）= **面密度（areal density, bits/in²）**；厂商按 $1\ \text{GB} = 10^9$、$1\ \text{TB} = 10^{12}$ 字节标称。**多盘片视图**下读写头**同步移动**（move in unison），同时刻停在所有盘面的**同一柱面（cylinder）**上——这就是磁盘调度优先读完整个柱面的原因。

9.2.5 磁盘访问时间（Disk Access Time）

$$T_{\text{access}} = T_{\text{avg seek}} + T_{\text{avg rotation}} + T_{\text{avg transfer}}$$

- **寻道（seek time）**：读写头移到目标柱面，典型 3–9 ms。
- **旋转延迟（rotational latency）**：等目标扇区首位转到读写头下。**最坏**转满一圈 $T_{\max} = \frac{1}{\text{RPM}} \times \frac{60\ \text{sec}}{1\ \text{min}}$，**平均是一半**：$T_{\text{avg rotation}} = \frac{1}{2} \times \frac{1}{\text{RPM}} \times \frac{60\ \text{sec}}{1\ \text{min}}$。
- **传送（transfer time）**：$T_{\text{avg transfer}} = \frac{1}{\text{RPM}} \times \frac{1}{\text{avg \# sectors/track}} \times \frac{60\ \text{sec}}{1\ \text{min}}$（转一圈时长 × 目标段占整圈比例）。
- **完整数值计算（讲义原例）**：7200 RPM、平均寻道 9 ms、平均每道 400 扇区：

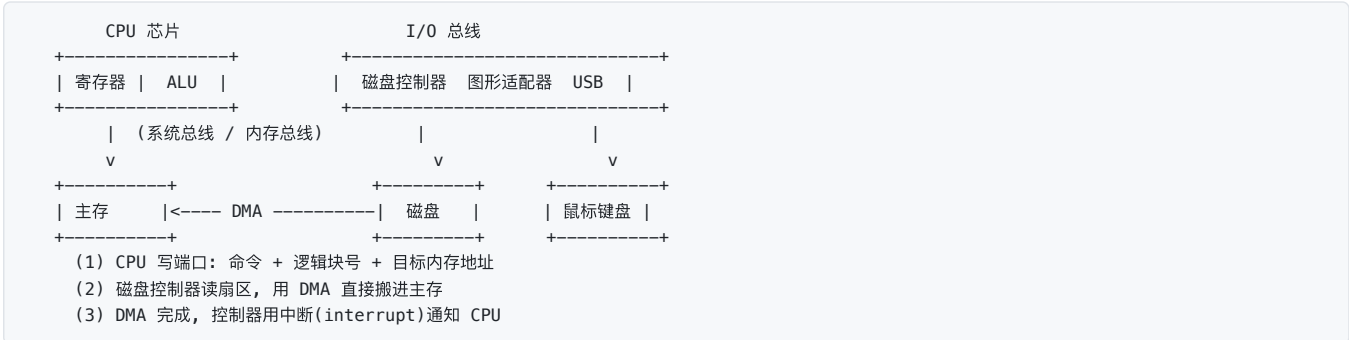
$$T_{\text{avg rotation}} = \frac{1}{2} \times \frac{60}{7200}\ \text{s} \times 1000 = 4\ \text{ms}, \qquad T_{\text{avg transfer}} = \frac{60}{7200} \times \frac{1}{400} \times 1000 = 0.02\ \text{ms}$$

$$T_{\text{access}} = 9 + 4 + 0.02 = \boxed{13.02\ \text{ms}}$$

三点结论：① 访问时间被**寻道与旋转延迟主导**，传送可忽略；② **扇区第一位最贵，其余位几乎免费**——"按块传输"的合理性来源；③ SRAM 约 4 ns/doubleword、DRAM 约 60 ns，磁盘比 SRAM 慢约 4 万倍、比 DRAM 慢约 2500 倍（教材口径；若直接拿 13.02 ms 除以 4 ns 会得约 3.3×10^6，**以"量级差 3–6 个数量级"为准**）。

9.2.6 逻辑块、磁盘控制器与 DMA

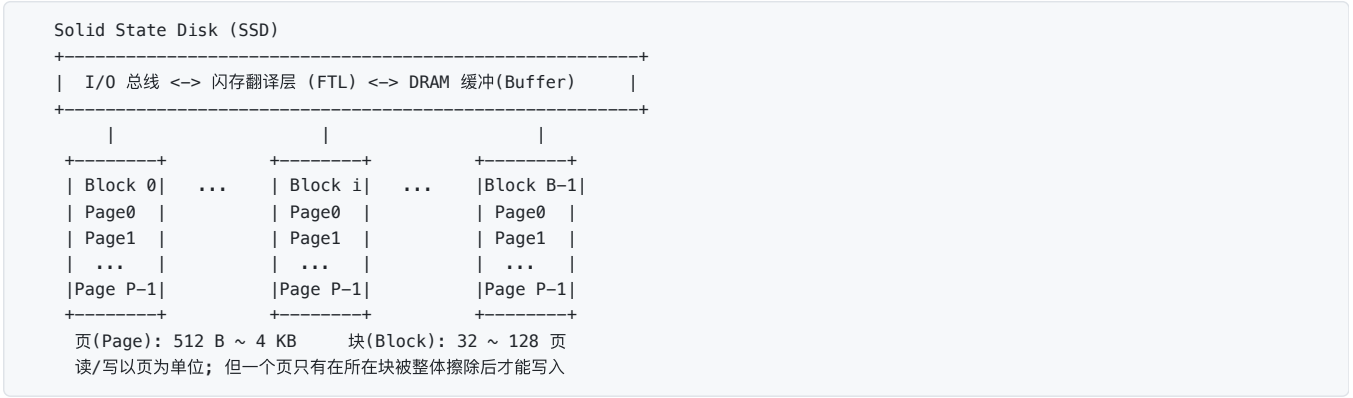
操作系统把磁盘视作**逻辑块（logical block）序列**，由**磁盘控制器（disk controller）**做地址翻译与坏块重映射：



第 (3) 步的中断正是 Lecture 17"异常控制流"的主题；磁盘 I/O 天生异步，这是后续 shell、proxy、SFS 等 lab 并发模型的根源。

9.2.7 固态硬盘（Solid State Disks, SSD）

SSD 基于闪存，对外提供"逻辑磁盘块"接口，由**闪存翻译层（flash translation layer, FTL）**把逻辑块映射到物理页：



- **读写不对称与写放大（write amplification）**：读是页粒度、直接；**写必须先擦掉整块（约 1 ms）**，改一页要把块内其它页搬到新块（读–改–写），"写 1 字节"引发几十倍内部写。
- **磨损均衡（wear leveling）**：每块约 1 万次重复写即报废，FTL 把写入摊到全盘（讲义举例：某三星 EVO Plus 保证每字节 600 次写入）。
- **实测性能（三星 970 EVO Plus，MB/s）**：

顺序读	顺序写	随机读	随机写	随机读 DQ	随机写 DQ
2,221	1,912	61.7	165	947	1,028

读法：① 顺序远快于随机——**整个存储层次的共同主题**；② 深队列（DQ）用并发掩盖延迟；③ 随机写反比随机读快，因为写缓存攒批 + 预擦除块池。 – **SSD vs 旋转磁盘**：优势无机械部件（更快、省电、抗震）；劣势会磨损（FTL 缓解）、每字节更贵。旋转磁盘仍用于冷数据、视频与廉价桌面存储。

9.2.8 存储技术趋势与"内存墙"

年份	1985	1990	1995	2003	2005	2010	2015	2015:1985
CPU	80286	80386	Pentium	P–4	Core 2	Core i7(n)	Core i7(h)	—
时钟频率 (MHz)	6	20	150	3,300	2,000	2,500	3,000	500
周期时间 (ns)	166	50	6	0.30	0.50	0.4	0.33	500
核数	1	1	1	1	2	4	4	4
有效周期时间 (ns)	166	50	6	0.30	**0.25**	**0.10**	**0.08**	**2,075**

指标	1985	2015	倍数	指标	1985	2015	倍数
DRAM 价格 ($/MB)	880	0.02	44,000	DRAM 典型容量 (MB)	0.256	16,000	62,500
DRAM 访问时间 (ns)	200	20	**10**	磁盘价格 ($/GB)	100,000	0.03	3,333,333
磁盘访问时间 (ms)	75	3	**25**	磁盘典型容量 (GB)	0.01	3,000	300,000

一句话读懂：CPU 有效周期时间改善 **2075 倍**，DRAM 访问时间只改善 **10 倍**，磁盘只 **25 倍**——**容量与价格改善 5–6 个数量级，速度只改善 1–2 个数量级**。鸿沟持续拉大，这就是**内存墙（memory wall）** / **冯·诺依曼瓶颈（Von Neumann bottleneck）**。2003–2005 年撞上**功耗墙（Power Wall）**，主频不再上涨，只能靠多核提升"有效"性能。**三条出路**：① **建造层次结构**（本课）；② **找别的事做**（指令级并行、乱序执行，15–346/15–418）；③ **把计算搬到数据旁边**（近存计算）。**摩尔定律放缓**之下，闪存因能**三维堆叠单元**而进步最快。

9.2.9 局部性原理（Locality of Reference）——本章的灵魂

局部性原理：程序倾向于引用**最近引用过的数据/指令本身，或与其地址相近**者。这是存储层次能工作的**唯一理由**。

- **时间局部性（temporal locality）**：被引用过的项很快会被再次引用（**地址相等**）。
- **空间局部性（spatial locality）**：地址相邻的项倾向于被紧挨着引用（**地址相近**）。

写程序像在厨房做菜——调味料（时间局部性）与案板食材（空间局部性）都放手边，不必每切一次菜就跑一趟超市。**工作集（working set）**就是"此刻正在处理的那堆东西"，局部性好意味着工作集小、能塞进缓存。

```
C 数组按"行主序"(row-major) 存放: a[i][j] 的地址 = base + (i*N + j)*4

按行求和 (内层 j): stride = 1   ->  空间局部性好 ✅
     时间 -->
     |a[i][0]|a[i][1]|a[i][2]|a[i][3]|a[i][4]| ... |a[i][N-1]|a[i+1][0]|
     +---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
      ^^^^^^^^^^^^^^^^^^^^^^^^  连续地址, 一个 cache block 装下多个元素
      .   .   .   .   .   .     (每格 4 字节, 64 B 的块可装 16 个 int)

按列求和 (内层 i): stride = N   ->  空间局部性差 ❌
     时间 -->
     |a[0][j]|      ...      |a[1][j]|      ...      |a[2][j]|
     +-------+---------------+-------+---------------+-------+
     每跳一次跨 N*4 字节 (N=4096 时 = 16 KiB), 每次落在全新的一块
```

- **定量感受**：讲义例子 `for (i = 0; i < 16; i++) sum += a[i];`，若 cache 初始为空、一块恰放 8 个元素、`a` 起点与块边界对齐，则只有 **2 次冷未命中**（是否已有元素在缓存中、一块放几个元素、起点是否对齐共同决定答案）。
- **对取指令的局部性**：指令按顺序取（空间局部性）、循环体反复执行（时间局部性），故 `for` 循环天生有好局部性。
- **循环置换（loop permutation）**：讲义的三维例子把 `k` 放最内层（步长 $N \times N$）；**答案是把 `j` 换成最内层**——最后一片下标变化才对应相邻地址，步长退化为 1。
- **易忽略的细节**：按列求和**并非永远差**——讲义提示"**若 M 非常小，局部性也不错，为什么？**"，因为此时整列元素靠得近（跨距 $M \times 4$ 字节），可能全落在同一或少数几个 block 里。**局部性是相对工作集大小而言的。**

9.2.10 存储层次结构（Memory Hierarchy）

最快的存储每字节最贵、容量最小、最耗电；CPU 与主存速度差距在扩大；写得好程序有良好局部性——三者共同指向**存储层次结构**：

```
        更小、更快、更贵（每字节）    <== 靠近 CPU
+-----------------------------------------------------------------+
|  L0  寄存器 Registers        (CPU 内)    <1 KB    ~0.3 ns      |
+-----------------------------------------------------------------+
|  L1  L1 cache (SRAM)         片上        32 KB    ~1 ns        |
+-----------------------------------------------------------------+
|  L2  L2 cache (SRAM)         片上       512 KB    ~4 ns        |
+-----------------------------------------------------------------+
|  L3  L3 cache (SRAM)         片上        32 MB    ~15 ns       |
+-----------------------------------------------------------------+
|  L4  主存 Main memory (DRAM)          16 GB     ~60 ns       |
+-----------------------------------------------------------------+
|  L5  本地二级存储 Local secondary     4 TB      ~10 ms       |
|      storage (local disk / SSD)                                 |
+-----------------------------------------------------------------+
|  L6  远程二级存储 Remote secondary    PB 级     ~100 ms      |
|      storage (e.g., Web servers)                                |
+-----------------------------------------------------------------+
       更大、更慢、更便宜（每字节）    <== 靠近数据源
```

逐级搬运：L1 存放**从 L2 取来的 cache line**，L2 从 L3 取，L3 从主存取；**主存存放从本地磁盘取来的磁盘块（disk blocks）。为什么有效？** 对每个 k，**第 k 层更小更快的设备充当第 $k+1$ 层更大更慢设备的缓存**；因局部性，程序访问第 k 层远多于第 $k+1$ 层，于是第 $k+1$ 层可放心更慢、更大、更便宜。

"大思想（Big Idea）"：层次结构创造了**巨大存储池**——**每字节成本接近最底层最便宜的设备**，却**以接近最顶层最快设备的速度供数**。至于"为什么大内存更慢"：内存越大，**信号走得更远、扇出（fan-out）更多**，"更大"与"更慢"天然绑定。

9.2.11 缓存的通用概念（General Cache Concepts）

缓存（cache）是更小更快的设备，充当更大更慢设备中**一个子集**数据的**中转站（staging area）**：

```
  Memory (更大、更慢、更便宜)          Cache (更小、更快、更贵)
  被划分为固定大小的 "块"(block)       缓存其中一部分块的副本
+----+----+----+----+----+----+     +----+----+----+----+
| 0  | 1  | 2  | 3  | .. | 15 |     | 8  | 9  | 14 | 3  |
+----+----+----+----+----+----+     +----+----+----+----+
     ^                                   ^
     |  数据总是以"块"为单位整块搬运    |
     +-----------------------------------+

请求 14 -> 块 14 在 cache 中 -> Hit !
请求 12 -> 块 12 不在 cache 中 -> Miss!
           1) 从下一层取出块 12 (放置策略决定 b 放哪里)
           2) 缓存已满时按替换策略选一个"牺牲者"(victim) 淘汰
```

未命中类型	英文	操作化定义
冷未命中 / 强制未命中	cold (compulsory) miss	对该块的**第一次**引用；即"无限大且无放置限制的缓存仍会发生的未命中"
容量未命中	capacity miss	**工作集大于缓存本身**；即"缓存仍无限大但不限放置"时**额外**出现的未命中
冲突未命中	conflict miss	第 $k+1$ 层的块只能放到第 k 层的**一小部分位置**；即"实际放置策略"带来的**额外**未命中

经典反例：若块 i 只能放在第 $i \bmod 4$ 个位置，则序列 `0, 8, 0, 8, ...` **每次都未命中**。**性能指标**：**命中率（hit rate）** = 命中次数/总访问次数；**未命中率（miss rate）** $= 1 - \text{hit rate}$；**命中时间（hit time）** = 从缓存取块到上层的时间；**未命中惩罚（miss penalty）** = 从下一层取块、放入缓存并交付的时间，故 $\text{AMAT} = T_{\text{hit}} + \text{miss rate} \times T_{\text{miss penalty}}$。

缓存无处不在：寄存器、TLB、L1/L2、虚拟内存、缓冲区缓存、磁盘缓存、浏览器缓存、Web 缓存（代理）——对象与延迟各异，机制完全同构。**可见性差异（关键区分）**：**缓存对软件透明**，由硬件响应 load/store 自动管理，无需改代码即改善（例外：`prefetch`、`invalidate`、分区等指令）；**而存储器对软件可见**，可直接用地址寻址，优化必须靠改代码。

9.2.12 缓存管理的四种组合

	写直达（write-through）	写回（write-back）
写分配（write-allocate）	写命中：同时写缓存和下一层；写未命中：先把块取进缓存再写	写命中：只写缓存并置脏位；写未命中：先把块取进缓存再写
非写分配（no-write-allocate）	写命中：同时写缓存和下一层；写未命中：**直接写下一层，不取块**	（组合不常见，讲义未推荐）

记法：写命中时写直达"两头都写"、写回"只改缓存并标脏（dirty），被淘汰时再写回"；写未命中时写分配"先把块搬上来再改"、非写分配"绕过缓存直接往下写"。真实 L1/L2 通常是**写回 + 写分配**；**写直达**常与更慢但更简单可靠的下一层搭配——**选择的本质是在"降低下层写流量"与"保持多层一致性/简单性"之间权衡。**

9.3 代码示例与底层机制分析

9.3.1 示例一：按行求和 vs 按列求和（实测 20 倍差距）

```c
/* rowcol.c —— Lecture 9 局部性实测：按行求和 (stride-1) vs 按列求和 (stride-N)
 * 编译: gcc -g -Wall -std=c11 -O2 rowcol.c -o rowcol
 * 运行: ./rowcol
 */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAXN    4096            /* 4096x4096 int = 64 MiB */
#define REPS    5               /* 每个规模重复 5 轮，取最快一轮 */
static int a[MAXN][MAXN];       /* 静态分配，避免 64 MiB 爆栈 */

static double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

/* 空间局部性好：stride = 1 */
static long sum_rows(int n)
{
    long sum = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            sum += a[i][j];
    return sum;
}

/* 空间局部性差：stride = n（跨 n*4 字节） */
static long sum_cols(int n)
{
    long sum = 0;
    for (int j = 0; j < n; j++)
        for (int i = 0; i < n; i++)
            sum += a[i][j];
    return sum;
}

int main(void)
{
    for (int i = 0; i < MAXN; i++)  /* 初始化按行写 */
        for (int j = 0; j < MAXN; j++)
            a[i][j] = (i + j) & 0xff;

    printf("%6s %10s %12s %12s %10s %10s\n",
           "N", "数组", "按行(ms)", "按列(ms)", "按行MB/s", "按列MB/s");
    printf("%6s %10s %12s %12s %10s %10s\n",
           "", "(KiB)", "stride-1", "stride-N", "", "");

    for (int n = 256; n <= MAXN; n *= 2) {
        double bytes = (double)n * n * 4;
        long sr = 0, sc = 0;
        double tr = 1e30, tc = 1e30;

        for (int r = 0; r < REPS; r++) {
            double t0 = now(); sr = sum_rows(n); double t1 = now();
            double t2 = now(); sc = sum_cols(n); double t3 = now();
            if (t1 - t0 < tr) tr = t1 - t0;
            if (t3 - t2 < tc) tc = t3 - t2;
        }
        if (sr != sc) { fprintf(stderr, "结果不一致!\n"); return 1; }

        printf("%6d %10.0f %12.3f %12.3f %10.0f %10.0f   (慢 %.1fx)\n",
               n, bytes / 1024, tr * 1e3, tc * 1e3,
               bytes / tr / 1e6, bytes / tc / 1e6, tc / tr);
    }
    return 0;
}
```

【代码做什么？】 静态分配 $\mathbf{4096 \times 4096}$ 的 `int` 数组（64 MiB，**不能放成局部变量**，否则撑爆 8 MiB 栈）；按行初始化；对 $N = 256 \ldots 4096$ 各跑两函数 5 轮**取最快一轮**；用 `sr != sc` 断言两者相等——**局部性只影响性能，绝不影响语义**。

【底层机制透视】 两段代码的访存指令几乎相同，唯一区别是地址增量：`sum_rows` 每次 `+4` 字节，`sum_cols` 每次 `+N*4` 字节。前者一个 64 B block 装 16 个 `int`，取一次块后续 15 次迭代全命中，**每 16 次访问才 1 次未命中**，预取器还能识别 stride-1 提前灌数据。后者在 $N = 4096$ 时步长 16 KiB，**每次访问落在不同 block**；访问完 `a[0][j]` 要过整整一行才回来碰 `a[0][j+1]`，此时该行早已被淘汰——**几乎每次都是冷/容量未命中**，64 MiB 全要从 DRAM 重拉；16 KiB 步长还几乎每次换页表项，TLB 同时被打爆。

【内存布局 / 数据结构图解】

```
   行主序布局 (row-major), 假设 base = 0x7f0000, N = 4, int = 4B

   地址:   0x7f0000  0x7f0004  0x7f0008  0x7f000c  0x7f0010 ...
         +----------+----------+----------+----------+----------+
         | a[0][0] | a[0][1] | a[0][2] | a[0][3] | a[1][0] | ...
         +----------+----------+----------+----------+----------+
   按行:   -------->  -------->  -------->  -------->            stride=4B   ✅
   按列:   a[0][0] ---------------------------> a[1][0]     stride=16B  ❌
           \_____________________ _____________________/
                                 v
              跨越 3 个元素 + 1 个新块 (N 越大越糟)
```

【与汇编 / 硬件的对应】（`gcc -O1 -S -masm=att -fno-tree-vectorize`，$N = 1024$）

```
sum_rows:                       sum_cols:
        movl    $a+4096, %esi           movl    $a+4194304, %esi
        movl    $a+4198400, %edi        movl    $a+4198400, %edi
        movl    $0, %ecx                movl    $0, %ecx
.L2:                            .L7:
        leaq    -4096(%rsi), %rax       leaq    -4194304(%rsi), %rax
.L3:                            .L8:
        movl    %ecx, %edx              movl    %ecx, %edx
        addl    (%rax), %edx            addl    (%rax), %edx
        movl    %edx, %ecx              movl    %edx, %ecx
        addq    $4, %rax        <===    addq    $4096, %rax     <===
        cmpq    %rsi, %rax              cmpq    %rsi, %rax
        jne     .L3                     jne     .L8
        addq    $4096, %rsi             addq    $4, %rsi
        cmpq    %rdi, %rsi              cmpq    %rdi, %rsi
        jne     .L2                     jne     .L7
        movl    %edx, %eax              movl    %edx, %eax
        ret                             ret
```

两条循环的唯一差别就是 `addq $4, %rax` 与 `addq $4096, %rax`——指令数、分支数、访存指令全同，**所有性能差距 100% 来自这个立即数**。

【实测验证】（AMD EPYC 7V13，L1d 32 KB / L2 512 KB / L3 32 MB，均 8 路组相联、64 B 行；择时数据每轮略有波动）

```
     N      数组    按行(ms)   按列(ms) 按行MB/s 按列MB/s
           (KiB)     stride-1     stride-N
   256        256        0.030        0.228       8598       1150   (慢 7.5x)
   512       1024        0.096        0.971      10867       1080   (慢 10.1x)
  1024       4096        0.372        4.798      11289        874   (慢 12.9x)
  2048      16384        1.468       26.933      11430        623   (慢 18.3x)
  4096      65536        5.689      114.279      11796        587   (慢 20.1x)

$ perf stat -e cache-references,cache-misses,cycles,instructions ./rowcol
       500,115,620      cache-references:u       #  34.80% of all cache refs
       174,025,565      cache-misses:u
     3,500,895,765      cycles:u
     1,330,307,783      instructions:u           #  0.38 insn per cycle
```

三条结论：① 数组越大按列访问相对越慢（7.5x → 20.1x）；② **按行吞吐稳定在 8.6–11.8 GB/s，几乎与数组大小无关**——"层次结构提供恒定高速"的直接证据；③ IPC 仅 0.38，CPU 大部分时间在等内存（**memory bound**）。

9.3.2 示例二：存储器山简化版（Memory Mountain）

```c
/* mountain.c —— 存储器山（简化版）
 * 扫描（工作集大小 x 步长）二维网格，测量每次访存耗时并换算成读吞吐率（MB/s）。
 * 编译: gcc -g -Wall -std=c11 -O2 mountain.c -o mountain
 * 运行: ./mountain
 */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAXELEMS   (32u << 20)    /* 32M 个 int = 128 MiB */
#define MINBYTES   (1u  << 12)    /* 最小工作集 4 KiB */
#define MAXBYTES   (1u  << 26)    /* 最大工作集 64 MiB */
#define MAXSTRIDE  32
#define BUDGET     (64u << 20)    /* 目标访存次数，保证计时可信 */

static int data[MAXELEMS];

static double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

/* 沿步长 stride 扫完工作集，返回累加和（防止被优化掉） */
static long touch(int n, int stride)
{
    long acc = 0;
    for (int i = 0; i < n; i += stride)
        acc += data[i];
    return acc;
}

/* 返回"平均每次访存耗时(秒)" */
static double run(int n, int stride, long *sink)
{
    long per_pass = (n + stride - 1) / stride;       /* 一趟的访存次数 */
    long passes   = BUDGET / per_pass;
    if (passes < 4)     passes = 4;                  /* 至少 4 趟 */
    if (passes > 40000) passes = 40000;              /* 小工作集别测太久 */
    *sink += touch(n, stride);                       /* 预热 */

    double t0 = now();
    for (long p = 0; p < passes; p++)
        *sink += touch(n, stride);
    double t1 = now();

    return (t1 - t0) / (double)(passes * per_pass);
}

int main(void)
{
    long sink = 0;
    memset(data, 1, sizeof(data));

    printf("存储器山（简化版）：行 = 工作集大小，列 = 步长；格值 = 读吞吐 MB/s\n");

    printf("%12s", "size");
    for (int s = 1; s <= MAXSTRIDE; s *= 2)
        printf("%9d", s);
    printf("   <- stride (元素，int=4B)\n");

    for (unsigned sz = MINBYTES; sz <= MAXBYTES; sz <<= 1) {
        int n = (int)(sz / sizeof(int));
        printf("%12u", sz);
        for (int s = 1; s <= MAXSTRIDE; s *= 2) {
            double per = run(n, s, &sink);           /* 秒/访存 */
            printf("%9.0f", 4.0 / per / 1e6);        /* 4B/s -> MB/s */
        }
        printf("\n");
    }
    printf("\n(校验和 = %ld)\n", sink);
```

```
    return 0;
}
```

【代码做什么？】 分配 128 MiB 的 `int` 数组并填 1；外层让**工作集**从 4 KiB 倍增到 64 MiB（16 档），内层让**步长**取 1,2,4,8,16,32（6 档）；每个 (N, s) 先预热一趟，再反复完整遍历直到累计访存达 `BUDGET`，统计**每次访存平均耗时**。

【底层机制透视】 早期版本用"限制迭代次数"计时，大工作集在计时窗口内只走了一小段（仍在缓存里），测出 12 GB/s 假高值；**改成长时间跑完整 N 趟后，大工作集被迫反复从 DRAM 重读，山形才正确显现。**

【内存布局 / 数据结构图解 —— 山的三维形状】

```
   吞吐率 (MB/s)
    ^
    |   ~~~ 山脊 (ridge): 步长=1, 工作集小, 数据在 L1/L2
    |  /|
12000| / |\
    |/  | \___
    |   |     \____
10000|   |          \____
    |   |               \___
 8000|   |  山坡 (slope)    \____
    |   |                       \____
 6000|   |                            \___
    |   |   冲突/容量未命中加剧           \___
 4000|   |                                \____ DRAM 平原
    |   |                                     \______
 2000|   |
    0+---+-----------------------------------------------------> 工作集大小
      4K  32K  512K      8M      32M        64M
       ^    ^     ^        ^       ^          ^
      L1   L1/L2  L2      L3      L3/DRAM   DRAM
      |<--- 步长 1..32 方向是"纬线", 向里凹陷 --->|
```

【实测输出】（格值 = 读吞吐 MB/s）

```
        size        1        2        4        8       16       32   <- stride (元素, int=4B)
        4096     7127     7020     7001     6968     7815     7924
        8192     7034     7030     7024     7138     6978     7747
       16384     7038     7031     7029     7023     7009     6976
       32768     8214     7060     7695     7487     7170     7164
       65536     7643     7191     7508     6905     6144     6058
      131072     7703     7079     7172     7043     6151     6104
      262144     7188     7088     7482     7718     5978     5812
      524288     8381     8132     7732     8557     5156     4555
     1048576    10269     8265     7592     7191     4665     3130
     2097152    10425    10589     7330     6847     4634     2934
     4194304    10340    10133     7420     6707     4697     2918
     8388608    10160    10091     7466     6850     4632     2921
    16777216    10048     8104     7551     6935     4562     2951
    33554432     9827     8759     5175     3614     2332     1517
    67108864    10157     8107     4978     3012     1717     1173
```

如何解读：① **对角线趋势清晰**——64 MiB + 步长 32 时仅 1173 MB/s，而 4 MiB + 步长 1 时是 10340 MB/s，**差约 8.8 倍**；② 小工作集（≤512 KB）各行平坦（数据基本命中 L2 及以上，步长影响被层级掩盖），但工作集到 1 MB 后步长 16/32 的列骤降到约 1900 MB/s 以下；③ 工作集 ≥32 MB 时连步长 8 也掉到 3800 MB/s 以下——**大工作集 + 大 stride 最差**（块内 16 个元素只用到 1–2 个）。

与 Cache Lab 的关系：存储器山是 L4 Part B"为什么分块有效"的实验证据——**分块就是把大工作集切成能塞进某层缓存的小块。**

9.3.3 示例三：Cache Lab Part A 的 LRU 缓存模拟器

```c
/* csim.c —— Cache Lab Part A：LRU 缓存模拟器
 * 用法: ./csim [-v] -s <s> -E <E> -b <b> -t <tracefile>
 * 编译: gcc -g -Wall -std=c11 csim.c -o csim
 */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <getopt.h>
#include <unistd.h>

typedef struct {
    int  valid;             /* 该行是否装有数据 */
    unsigned long tag;      /* 标记位 */
    unsigned long stamp;    /* LRU 时间戳，越大越新 */
} line_t;

static int s, E, b, verbose;
static long hits, misses, evictions;
static unsigned long clk = 0;          /* 全局逻辑时钟，每次访存 +1 */
static line_t **cache;                 /* cache[S][E] */
static int S;                          /* 组数 = 2^s */

/* 一次访存：bit0=hit, bit1=发生淘汰(eviction) */
#define RES_HIT 1
#define RES_EVICT 2
static int access_cache(unsigned long addr)
{
    unsigned long tag = addr >> (s + b);                /* 标记位 */
    unsigned long set = (addr >> b) & ((1UL << s) - 1); /* 组索引 */

    line_t *L = cache[set];
    clk++;

    for (int i = 0; i < E; i++) {                   /* 1) 找命中 */
        if (L[i].valid && L[i].tag == tag) {
            L[i].stamp = clk;                       /* 刷新 LRU */
            hits++;
            return RES_HIT;
        }
    }

    misses++;                                       /* 2) 未命中 */
    int victim = -1, evicted = 0;
    for (int i = 0; i < E; i++)
        if (!L[i].valid) { victim = i; break; }     /* 3a) 优先用空行 */

    if (victim < 0) {                               /* 3b) 否则淘汰 LRU 行 */
        victim = 0;
        for (int i = 1; i < E; i++)
            if (L[i].stamp < L[victim].stamp) victim = i;
        evictions++;
        evicted = 1;
    }
    L[victim].valid = 1;
    L[victim].tag   = tag;
    L[victim].stamp = clk;
    return evicted ? RES_EVICT : 0;
}

static void replay(FILE *fp)
{
    char op;
    unsigned long addr;
    int size;

    while (fscanf(fp, " %c %lx,%d", &op, &addr, &size) == 3) {
        if (op == 'I') continue;                    /* 忽略取指 */

        int r1, r2 = -1;
        if (op == 'M') {                            /* M = load + store */
            r1 = access_cache(addr);
            r2 = access_cache(addr);
        } else {
            r1 = access_cache(addr);
```

```
        }

        if (verbose) {
            printf("%c %lx,%d", op, addr, size);
            printf("%s", (r1 & RES_HIT) ? " hit" : " miss");
            if (r1 & RES_EVICT) printf(" eviction");
            if (r2 >= 0) {
                printf("%s", (r2 & RES_HIT) ? " hit" : " miss");
                if (r2 & RES_EVICT) printf(" eviction");
            }
            printf("\n");
        }
    }
}

int main(int argc, char **argv)
{
    const char *tracefile = NULL;
    int opt;

    while ((opt = getopt(argc, argv, "hvs:E:b:t:")) != -1) {
        switch (opt) {
        case 's': s = atoi(optarg); break;
        case 'E': E = atoi(optarg); break;
        case 'b': b = atoi(optarg); break;
        case 't': tracefile = optarg; break;
        case 'v': verbose = 1; break;
        default:
            printf("Usage: %s [-hv] -s <s> -E <E> -b <b> -t <tracefile>\n", argv[0]);
            return 0;
        }
    }
    if (!tracefile || E <= 0 || s < 0 || b < 0) {
        fprintf(stderr, "Usage: %s [-hv] -s <s> -E <E> -b <b> -t <tracefile>\n", argv[0]);
        return 1;
    }

    S = 1 << s;
    cache = malloc(sizeof(line_t *) * S);
    for (int i = 0; i < S; i++) {
        cache[i] = calloc(E, sizeof(line_t));
        if (!cache[i]) { perror("calloc"); return 1; }
    }

    FILE *fp = fopen(tracefile, "r");
    if (!fp) { perror(tracefile); return 1; }
    replay(fp);
    fclose(fp);

    printf("hits:%ld misses:%ld evictions:%ld\n", hits, misses, evictions);

    for (int i = 0; i < S; i++) free(cache[i]);
    free(cache);
    return 0;
}
```

【代码做什么？】 ① `getopt` 解析 `-s/-E/-b/-t/-v`，$S = 2^s$ 为组数、E 为每组行数、$B = 2^b$（**只用于地址切分，不真正存数据**）；② 用 `malloc` / `calloc` 为 $S \times E$ 个 `line_t` 分配空间——**这正是 lab 要求"必须用 `malloc` 支持任意参数"的原因**；③ 逐行解析 trace：忽略 `I`，`L` / `S` 各算一次访存，`M` **算两次**；④ 把地址切成 `tag|set|block`，在 `cache[set]` 内找 `tag`，命中刷新时间戳，未命中先找空行、无空行则淘汰 `stamp` 最小者。

【底层机制透视 —— 地址切分】

```
64 位地址 A，参数 m = 64，s = 组索引位数，b = 块偏移位数
+--------------------------+-----------------+-------------------+
|      tag (t = 64-s-b 位)   |  set (s 位)     |  block (b 位)     |
+--------------------------+-----------------+-------------------+
          |                          |                 |
          |                          |                 +-> 块内偏移，模拟器不关心
          |                          +-> cache[set]，直接索引到某一组
          +-> 与组内每行的 tag 比较，判断是否命中

例：-s 4 -E 1 -b 4，地址 0x110（十进制 272）
    272 = 0b1_0001_0000
    b=4  -> 块偏移 = 0b0000
    s=4  -> 组索引 = 0b0001 = 1
    t=56 -> 标记   = 0b1 = 1     => 到第 1 组找 tag==1 的行
```

`M` **为什么要算两次？** 讲义明确说明：`M` 是"一次数据装载后紧跟一次数据存储"，**可能产生两次命中，或一次未命中 + 一次命中（外加可能的淘汰）；但淘汰只可能发生在第一次访问上**——第二次访问时该行必定已在缓存。这是 Part A 最容易多算 `evictions` 的地方。

【LRU 的实现方式】（时间戳数组 vs 双向链表）

实现	数据结构	命中时	淘汰时	复杂度	说明
时间戳数组（上面代码）	每行一个 `stamp` + 全局单调计数器 `clk`	`L[i].stamp = ++clk`	线性扫描找 `stamp` 最小者	命中 $O(E)$，淘汰 $O(E)$	实现最短，**推荐首选**
双向链表	每组一条链表，MRU 在头、LRU 在尾	摘下插到头	摘尾插头	命中 $O(1)$（需哈希或额外指针）	理论更优，但代码量大、易出指针 bug

实践中 E 最多几十，**时间戳数组的线性扫描开销可忽略。**

【实测验证 —— 与官方 `csim-ref` 逐例比对】（8 个官方用例全部一致，27/27 分）

```
$ ./csim -v -s 4 -E 1 -b 4 -t traces/yi.trace          # 我们的实现
L 10,1 miss
M 20,1 miss hit
L 22,1 hit
S 18,1 hit
L 110,1 miss eviction
L 210,1 miss eviction
M 12,1 miss eviction hit
hits:4 misses:5 evictions:3

$ csim-ref -v -s 4 -E 1 -b 4 -t traces/yi.trace        # 官方参考模拟器
L 10,1 miss
M 20,1 miss hit
...
hits:4 misses:5 evictions:3
```

(s, E, b)	trace	我们 hits/misses/evictions	`csim-ref`	一致?
(1,1,1)	yi2.trace	9 / 8 / 6	9 / 8 / 6	✅
(4,2,4)	yi.trace	4 / 5 / 2	4 / 5 / 2	✅
(2,1,4)	dave.trace	2 / 3 / 1	2 / 3 / 1	✅
(2,1,3)	trans.trace	167 / 71 / 67	167 / 71 / 67	✅
(2,2,3)	trans.trace	201 / 37 / 29	201 / 37 / 29	✅
(2,4,3)	trans.trace	212 / 26 / 10	212 / 26 / 10	✅
(5,1,5)	trans.trace	231 / 7 / 0	231 / 7 / 0	✅
(5,1,5)	long.trace	265189 / 21775 / 21743	265189 / 21775 / 21743	✅

观察：`trans.trace` 在 E 从 1 增到 4 时未命中从 71 降到 26、淘汰从 67 降到 10——**组相联消除了冲突未命中**；`(5,1,5)` 淘汰为 0，余下 7 次未命中**全是冷未命中**。

9.4 实验关联

L4 Cache Lab（本讲布置，约 2 周完成），共 60 分（Part A 27 + Part B 26 + 风格 7）。

Part A：缓存模拟器（本讲学完立刻动手）

- **交付**：从零填写 `csim.c`（约 200–300 行），命令行与 `csim-ref` 一致：`./csim [-hv] -s <s> -E <E> -b <b> -t <tracefile>`。
- **必做七件事**：① 用 `malloc` 分配数据结构以支持任意 s, E, b；② 忽略所有 `I` 开头的行——valgrind 约定 `I` 在第一列且前面无空格，`M` / `L` / `S` 在第二列且前面有空格，而 `fscanf(" %c %lx,%d", ...)` 的前导空格正好吃掉这一差异；③ `M` 必须算两次访存；④ 可忽略 size 字段（lab 保证访存不跨块边界）；⑤ `main` 结尾必须调用 `printSummary(hit_count, miss_count, eviction_count)`；⑥ 用 LRU 替换策略；⑦ **必须无警告编译**（`-Wall`），并写上姓名与 loginID。
- **建议实现 `-v` 详细模式**：不强制，但可与 `csim-ref -v` 逐行 diff，是最快的调试手段。**评分**：8 个用例，每个 3 分、最后一个 6 分；hits/misses/evictions 各占 1/3 分——`evictions` 算错仍能拿 2/3 分。

```
linux> make
linux> ./test-csim
                        Your simulator     Reference simulator
Points (s,E,b)    Hits  Misses Evicts   Hits Misses Evicts
3     (1,1,1)      9      8      6       9     8      6   traces/yi2.trace
...
6     (5,1,5) 265189  21775  21743  265189 21775  21743   traces/long.trace
27
```

Part B：矩阵转置优化（第 10 讲后完成）

- 写 `transpose_submit`，在 32×32、64×64、61×67 三种尺寸下最小化未命中；用 valgrind 取 trace，再用 `csim-ref -s 5 -E 1 -b 5`（32 组 × 1 路 × 32 B 直接映射缓存）打分。
- **阈值**：32×32 未命中 < 300 得 8 分（> 600 得 0）；$64 \times 64 < 1300$ 得 8 分（> 2000 得 0）；$61 \times 67 < 2000$ 得 10 分（> 3000 得 0），**性能分按未命中数线性插值**。
- **规则坑**：每个转置函数**最多 12 个 `int` 局部变量**（栈访问被 valgrind 过滤，故须限制）；**禁止递归、禁止定义任何数组、禁止任何形式的 `malloc`、禁止改 `A`**；辅助函数间活动局部变量总数也不得超过 12。
- **核心技巧**：**分块（blocking）**——把大矩阵切成能装进缓存的小块：32×32 用 8×8 块；64×64 因直接映射冲突严重，常用 8×8 分块 + 临时变量缓存对角线元素。入门材料见 `csapp.cs.cmu.edu/public/waside/waside-blocking.pdf`。
- **调试利器**：`test-trans` 为每个注册函数生成 `trace.fi`，再用 `./csim-ref -v -s 5 -E 1 -b 5 -t trace.f0` **逐条对照每次命中与未命中**，精确定位哪一行引起冲突未命中。**反例数据（讲义示例）**：简单按行扫描转置 32×32 有 1183 次未命中、1151 次淘汰，分块版本只有 **287 次**。

9.5 常见错误与调试技巧

- **按行/按列写反了**：`for j` / `for i` 嵌套写错，功能正确但性能掉数倍。**调试**：`perf stat -e cache-misses,cache-references ./prog`；`perf record -e cache-misses ./prog && perf report` 定位循环行。判据：**内层下标应对应数组最右边（变化最快）的那一维。**
- **大数组放成局部变量导致栈溢出**：`int a[4096][4096]` 要 64 MiB，默认栈仅 8 MiB → 段错误。**调试**：`ulimit -s` 查上限；`gdb` 下 `bt` 见地址接近 `0x7ffd...` 边界即为栈溢出。**解决**：改 `static` /全局/ `malloc`。
- **存储器山测量窗口没覆盖完整工作集**：固定迭代次数计时使大工作集一轮只走一小段（仍在缓存），测出 12 GB/s 假高值。**调试**：保证每点完整遍历若干趟，用 `perf stat` 验证未命中数随工作集增长。
- **`csim` 的 evictions 多算**：把 `M` 的第二次访存也算作可能淘汰，或在未命中且找到空行时仍累加。**调试**：`./csim -v ... | diff - <(csim-ref -v ...)`；重点看 `M` 行——正确形式是 `M xxx,n miss eviction hit`，第二次永远是 `hit`。
- **直接映射缓存的对角线冲突未命中**（Part B 的 64×64 是重灾区）：$E = 1$ 时 A、B 的第 i 行常映射到同组，转置时互相踢出。**调试**：`./csim-ref -v -s 5 -E 1 -b 5 -t trace.f0 | head -50` 看是否反复出现同组地址的 `miss eviction`；**解决**：分块 + 用 8 个临时变量一次读出整行再写出。
- **误以为"缓存对程序员完全不可见"**：缓存确由硬件透明管理，但**访问模式由你决定**。**调试**：`valgrind --tool=cachegrind ./prog` 得到逐行 D1/LL 未命中统计（`cg_annotate`）。

9.6 关键要点

- **局部性是存储层次结构存在的唯一理由。** 时间局部性让"缓存最近用过的块"有意义，空间局部性让"以块为单位搬运"有意义；没有局部性，金字塔立刻崩塌。
- **步长（stride）是局部性的第一指标。** 内层下标必须对应地址变化最快的那一维（C 中是最后一维）；步长变大，每次访问能利用的块内数据成比例减少。
- **级联缓存让"又大又快又便宜"同时成立。** 每层的目标不是自己快，而是"让上一层觉得自己快"；这就是 Big Idea。
- **内存墙是物理必然。** 更大的存储意味着信号更远、扇出更多，"大"与"慢"天然绑定；CPU 有效周期时间 40 年改善 2075 倍而 DRAM 访问时间只改善 10 倍。
- **未命中分三类，诊断方法各异。** 冷未命中不可免（除非预取）；容量未命中靠缩小工作集或分块；冲突未命中靠提高相联度（增大 E）或改变访问地址。**"增加 E 后未命中大降"就说明冲突未命中占主导。**

- **磁盘/SSD 性能被"第一位"主导。** 传送时间可忽略，故**减少随机访问次数比减少总字节数更重要**——这正是缓冲区缓存、预读与顺序写日志有效的原因。

9.7 思考题（带答案）

① （计算题）磁盘容量与访问时间

某磁盘：每扇区 512 字节，平均每磁道 400 个扇区，每盘面 20,000 条磁道，每盘片 2 个盘面，共 5 个盘片；转速 10,000 RPM；平均寻道 6 ms。(a) 求总容量（十进制 GB）。(b) 求平均访问时间。(c) 转速提到 15,000 RPM 后改善多少百分比?

答:

(a) 套容量公式：$512 \times 400 = 204{,}800$ B/track；$\times 20{,}000 = 4.096 \times 10^9$ B（单面）；$\times 2 \times 5 = \boxed{40.96\ \text{GB}}$

(b) $T_{\text{avg seek}} = 6\ \text{ms}$；$T_{\text{avg rotation}} = \frac{1}{2} \times \frac{60}{10000} \times 1000 = 3\ \text{ms}$；$T_{\text{avg transfer}} = \frac{60}{10000} \times \frac{1}{400} \times 1000 = 0.015\ \text{ms}$，故

$$T_{\text{access}} = 6 + 3 + 0.015 = \boxed{9.015\ \text{ms}}$$

(c) 转速只影响旋转延迟与传送时间，**不影响寻道**：新 $T_{\text{avg rotation}} = 2\ \text{ms}$、新 $T_{\text{avg transfer}} = 0.01\ \text{ms}$，故新 $T_{\text{access}} = 8.01\ \text{ms}$，改善 $\frac{9.015 - 8.01}{9.015} \approx 11.2\%$。

关键结论：转速提升 50% 只让总时间改善约 11%——**6 ms 寻道占 2/3 且不受转速影响**。所以磁盘优化重点是"减少寻道次数"（柱面优先、电梯调度、顺序访问），而非"提高转速"。

---

② （计算题）局部性判断与冷未命中计数

```
#define N 8
int a[N][N];
long sum_cols(void)          /* 按列求和: stride = N */
{
    long sum = 0;
    for (int j = 0; j < N; j++)
        for (int i = 0; i < N; i++)
            sum += a[i][j];
    return sum;
}
```

int 为 4 字节，cache 初始为空，块大小 32 字节，`a` 起点与块边界对齐，cache 足够大不发生容量/冲突未命中。求总未命中次数，并说明如何重写循环以消除绝大部分未命中。

答：矩阵共 $8 \times 8 \times 4 = 256$ 字节 = 8 个 32 字节块，每块装 8 个 `int`；行主序下第 i 行的 8 个元素**恰好占满一个块**（$N = 8$，一行 32 字节）。

- $j = 0$：访问 `a[0..7][0]`，地址间隔 $8 \times 4 = 32$ 字节，8 次访问落在 8 个**互不相同**的块上 → **8 次未命中**。
- $j = 1$：这 8 个块**都已在缓存中** → **0 次未命中**；$j = 2..7$ 同理全命中。

总未命中 = 8 次，全是冷未命中。

注意：**这是"按列访问但局部性仍好"的典型**——因 $N = 8$ 太小，一整列元素全挤在少数几个块里（讲义"若 M 非常小则局部性也不错"一题即由此而来）。**若把 N 改成 1024**：按列访问一列需 1024 个块，而矩阵共 $131{,}072$ 个块、远超缓存，每轮 j 几乎全未命中（共约 10^6 次）；**改写**成内层下标对应最后维度后，每块连续访问 8 次只用 1 次未命中，总未命中降到 131,072 次。

---

③ （"直观但错误"的想法）"既然缓存是硬件自动管理的，程序员不需要关心局部性"

请指出这个想法错在哪里，并举出本讲的具体反例。

答：**错在把"缓存的正确性"与"缓存的性能"混为一谈。** 硬件**透明地**保证**正确性**——不写代码，`sum_rows` 与 `sum_cols` 也算出相同答案。但**性能完全取决于访问模式**，而访问模式是程序员唯一能控制的东西；硬件只负责"用你给的访问序列去填缓存"，不能替你改变序列。

反例一：示例一中两函数的**汇编指令几乎逐条相同**，唯一差别是 `addq $4, %rax` 与 `addq $4096, %rax`，结果 $N = 4096$ 时 **5.689 ms vs 114.279 ms，相差约 20 倍**——同算法、同数据、同机器、同编译器，**性能差距 100% 来自遍历顺序**。**反例二**：Cache Lab Part B 中，简单按行扫描转置在 32×32 上产生 1183 次未命中，仅**改变分块方式**（数学逻辑一行不改）就降到 287 次，性能分从 0 变满分。

结论：硬件提供"机制"，程序员提供"策略"。**"缓存不可见"意味着不必为正确性操心，而不是不必为性能操心。**

---

④ （推演题）未命中类型的归因

某程序在 32 KB 缓存上测得未命中率 10%；换成同样大小的 **16 路组相联**后降到 6%；再扩大到 **128 KB 全相联**（不限放置）降到 4%。估算**冲突未命中**与**容量未命中**在原始 10% 中各占多少，并说明依据。

答：依据讲义的三条操作化定义——**冷未命中**：用**无限大且无放置限制**的缓存仍会发生的未命中；**容量未命中**：在冷未命中之上，因**缓存有限**（但不限放置）而**额外**增加的未命中；**冲突未命中**：在容量未命中之上，因**实际放置限制**而**额外**增加的未命中。

归因：① 128 KB 全相联的 4% 已接近"无限大无限制"下界，**冷未命中 ≈ 4%**；② "无放置限制（4%）"与"16 路组相联（6%）"之差来自**放置限制**，故高相联度下 **冲突未命中 = 2%**；③ 原始配置冲突未命中 = 10% − 4% − 2% = **4%**，**容量未命中 = 2%**。

未命中类型	占原始 10% 的比例
冷未命中（compulsory/cold）	4%（不可消除）
容量未命中（capacity）	2%
冲突未命中（conflict）	4%

依据：以"无限大无限制"为冷未命中基准；"大而不限放置 vs 实际放置"的差额是冲突未命中；"无限大 vs 有限大（不限放置）"的差额是容量未命中。**本例冲突未命中占大头（4/10），说明提高相联度收益最高**——与示例三中 `trans.trace` 把 E 从 1 增到 4 时未命中从 71 降到 26 的实测一致。

Lecture 10: 高速缓存存储器 (Cache Memories)

> **讲义对应**：CMU 15–213 Lecture 10 — Cache Memories（素材：`F25-10-cache-memories.txt`） **教材对应**：CS:APP3e 第 6 章 6.4–6.7（高速缓存的组织与操作、性能影响、存储器山、循环重排与分块） **关联 Lab**：**L4 Cache Lab**（Part A 写缓存模拟器 `csim.c`，Part B 优化矩阵转置 `trans.c`）

10.1 概述

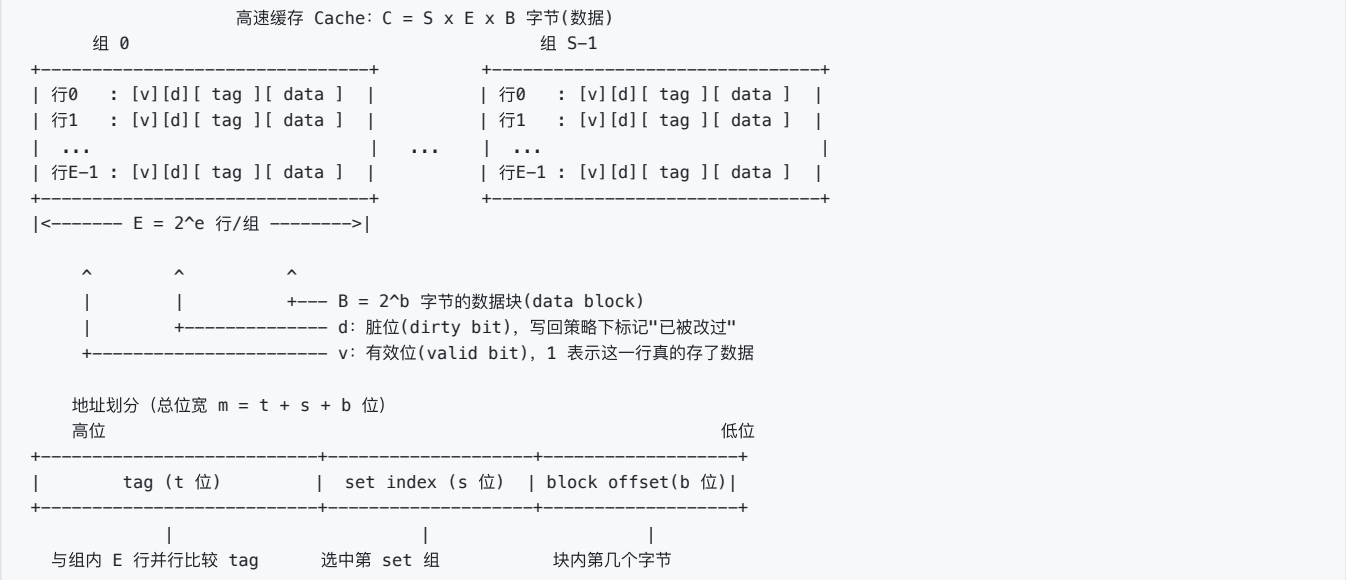
第 9 讲建立了"存储层次 + 局部性（locality）"的定性图景：程序倾向于访问最近用过的、地址相近的数据。本讲把这张定性图景**变成一个可以精确计算的硬件模型**：真实的缓存（cache）由 S 个组（set）、每组 E 行（line）、每行 B 字节块（block）构成，容量 $C = S \times E \times B$；CPU 发出的地址被硬件切成**标记（tag）、组索引（set index）、块偏移（block offset）**三段，每组一次查找就能判定命中（hit）还是未命中（miss）。有了这套模型，我们才能解释为什么"按列遍历矩阵比按行慢 25 倍"、为什么循环交换一下顺序就能提速 10 倍、为什么 64×64 矩阵转置需要一套看起来极其古怪的分块写法。

本讲承接第 9 讲的局部性与存储层次，向下为第 15 讲"代码优化"提供量化的性能语言，并直接支撑 L4 Cache Lab 的两个部分。

10.2 核心概念与底层机制图解

10.2.1 缓存的组织结构 $C = S \times E \times B$（Cache Organization）

- **定义与目的**：缓存是"小、快、贵"的 SRAM，缓存住"大、慢、便宜"的 DRAM 中**一块子集**。硬件不做任何程序分析，只靠地址位做机械查表，因此必须有一个规整的阵列结构。
- **直观解释（"它是什么？"）**：把缓存想成一栋**图书馆**：全馆分成 S 个**书架（组）**，每个书架上恰好有 E 层**隔板（行）**，每层隔板上放一个**书盒（块）**，盒里装 B 字节数据。要知道某本书在不在馆里，先由"索书号中段"决定去哪个书架（组索引），再逐层比对"书脊上的编号"（标记），全都对不上就说明没这本书。
- **底层机制图解**：



- **与机器码/硬件的对应**：全部由**地址译码逻辑**完成——`set index` 位直接接到一个多路选择器选中 SRAM 组；`tag` 位送进 E 个并行的比较器；`block offset` 位用于从块内取出字节。因此"查找"是**组合逻辑**，不需要指令、不需要软件参与。唯一会出现在汇编里的是访存指令本身（如 `movslq (%rax), %rcx`）。

10.2.2 直接映射缓存（Direct-Mapped Cache，$E = 1$）

- **定义与目的**：每组只有一行，**每个内存块在全缓存中只有唯一一个位置可放**：块地址 $\bmod S$。极端简单、快，但容易冲突。
- **直观解释**：每个书架只有**一层隔板**，一本书只能放固定书架。于是两本"必须放在同一书架"的书只能轮流上架——这正是冲突未命中（conflict miss）的来源。
- **底层机制图解**：讲义用 4 位地址、$S = 4, E = 1, B = 2$（$s = 2, b = 1, t = 1$）演示，访存序列 $0, 1, 7, 8, 0$：

```
地址 4 位:  t s s b
  0 = 0000 -> tag=0 set=0 off=0   Set0 空        -> miss  装入 M[0-1]
  1 = 0001 -> tag=0 set=0 off=1   tag 匹配,v=1   -> hit
  7 = 0111 -> tag=0 set=3 off=1   Set3 空        -> miss  装入 M[6-7]
  8 = 1000 -> tag=1 set=0 off=0   组0 现有 tag=0 -> miss  替换 -> eviction
  0 = 0000 -> tag=0 set=0 off=0   组0 现为 tag=1 -> miss  替换 -> eviction
结果: hits:1 misses:4 evictions:2
```

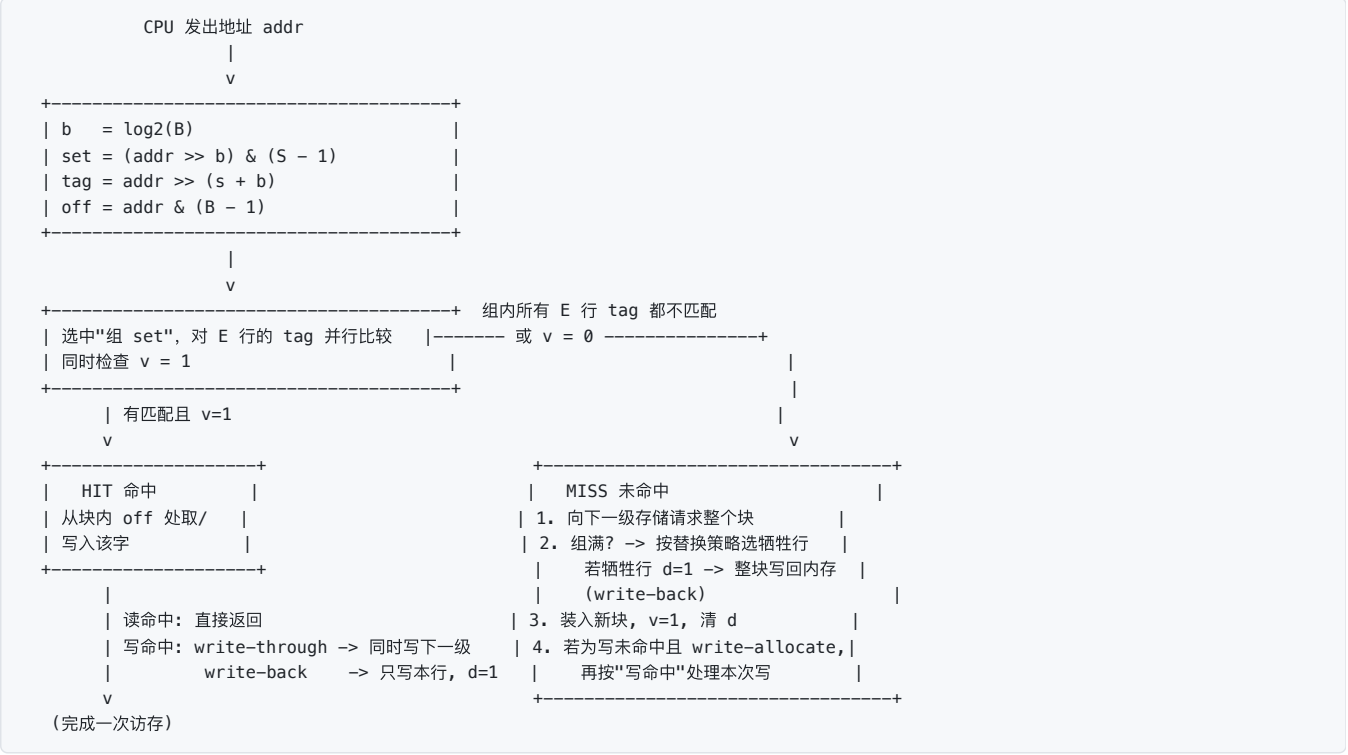
10.2.3 组相联缓存（Set-Associative，$E > 1$）与全相联（Fully Associative，$S = 1$）

- **定义与目的**：一组放 E 行，块可以放到组内**任意一行**。E 越大冲突越少，但并行比较器越多、命中时间越长。$S = 1$（即 $s = 0$，无组索引位）就是**全相联**，此时地址只剩 `tag|offset` 两段。
- **直观解释**：E 路组相联就是"每个书架有 E 层隔板"，能容忍 E 本同书架的书共存；全相联则是"整馆一个大书架"，随便放——**永远不会发生冲突未命中**，但查找最贵。
- **组相联的替换策略**：组满时必须选一个**牺牲行（victim）**踢出去，称为替换（replacement）：
- **LRU（Least Recently Used，最近最少使用）**：踢最久未被访问的行。命中率好，但需要为每行维护时间戳/计数器，硬件成本高。**Cache Lab 用的就是 LRU**。
- **LFU（Least Frequently Used）**：踢访问次数最少的行。实现简单，但对"曾经很热、现在已冷"的块不友好。
- **随机（Random）**：硬件最省事，平均效果不差（现代 L3 常用伪随机）。
- **E 路组相联（讲义中 $E = 2$）的另一条访存序列**（4 位地址，$S = 2, E = 2, B = 2$，$s = 1, b = 1, t = 2$），仍是 $0, 1, 7, 8, 0$：

```
  0 = 0000 -> set=0 tag=00  组0 空 -> miss   装入
  1 = 0001 -> set=0 tag=00  命中      -> hit
  7 = 0111 -> set=1 tag=01  组1 空 -> miss   装入
  8 = 1000 -> set=0 tag=10  组0 还有空行 -> miss 装入(不替换!)
  0 = 0000 -> set=0 tag=00  组0 里 tag=00 仍在 -> hit
结果: hits:2 misses:3 evictions:0
```

同一个访问序列，直接映射是 1 命中/2 替换，2 路组相联是 2 命中/0 替换——**多出来的这次命中，正是"冲突未命中"被相联度消化掉的证据**。

10.2.4 查找流程图与读写策略（Lookup & Write Policy）

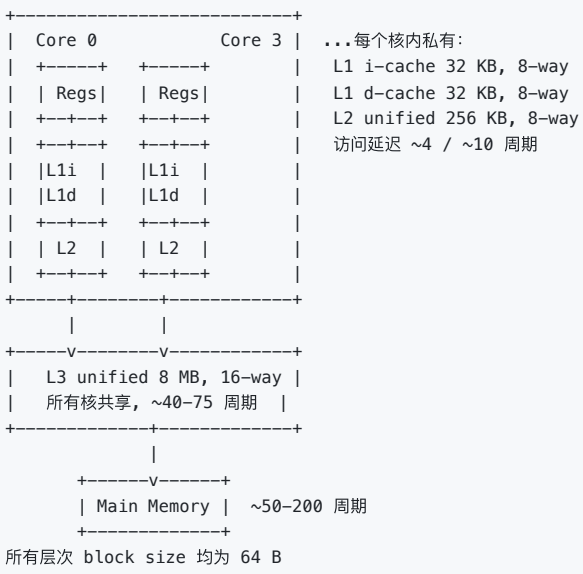


写策略的两组正交选择（讲义明确给出）：

时机	策略	含义	代价
写**命中**	**写直达（write-through）**	立刻写到下一级	每次写都产生总线/下一级流量
写**命中**	**写回（write-back）**	只改缓存行并置**脏位（dirty bit）**，直到该行被替换才整块写回	节省流量，但需要脏位与更复杂的替换逻辑
写**未命中**	**写分配（write-allocate）**	先把块取进缓存，再在缓存里改	若后续还会写同一块则很赚
写**未命中**	**非写分配（no-write-allocate）**	直接写进下一级，不取块	避免"只写一次却要取一整块"的浪费

工程上成对出现：**写直达 + 非写分配**（简单，早期机器）、**写回 + 写分配**（现代通用处理器的 L1/L2 选择，也是 Cache Lab 模拟器必须实现的行为）。

10.2.5 Intel Core i7 的真实缓存层次



层次	容量	相联度	块大小	组数 S	s	b	t（47 位地址）
L1 d-cache	32 KB	8 路	64 B	$32768/(8\times 64)=64$	6	6	$47-6-6=35$
L1 i-cache	32 KB	8 路	64 B	64	6	6	35

层次	容量	相联度	块大小	组数			（47 位地址）
L2 unified	256 KB	8 路	64 B	$262144/(8\times 64)=512$	9	6	32
L3 unified	8 MB	16 路	64 B	$8388608/(16\times 64)=8192$	13	6	28

讲义给出的 Core i7 L1 数据缓存例题：地址 `0x00007f7262a1e010`，$s=6, b=6$，则

```
块偏移 = addr & 0x3F      = 0x10      (块内第 16 字节)
组索引 = (addr >> 6) & 0x3F = 0x0     (第 0 组)
标记   = addr >> 12       = 0x7f7262a1e
```

10.2.6 性能度量与 3C 未命中

- **未命中率（miss rate）** = misses / accesses = 1 − 命中率。L1 典型 3–10%，L2 常 <1%。
- **命中时间（hit time）**：把一行数据交给 CPU 的时间，含"判定是否命中"的时间。L1 约 4 周期，L2 约 10 周期。
- **未命中惩罚（miss penalty）**：主存约 50–200 周期，且**趋势是变大**（CPU 越快，相对差距越大）。
- **平均访存时间**：

$$AMAT = \text{命中时间} + \text{未命中率} \times \text{未命中惩罚}$$

讲义用它解释了"为什么用未命中率而不是命中率"：命中时间 1 周期、未命中惩罚 100 周期时，命中率 97% $\Rightarrow 1+0.03\times 100=4$ 周期，命中率 99% $\Rightarrow 1+0.01\times 100=2$ 周期。**命中率只提高 2 个百分点，平均访存时间却快了一倍。** 命中率高的区间里，它看起来"几乎一样"（97% vs 99%），而对性能的影响却是翻倍的——所以要盯住未命中率。

- **3C 未命中**（"C"指 Cache miss 的分类）：

类型	讲义定义	消除手段
强制/冷未命中（compulsory / cold）	对某块的**第一次**引用，缓存不可能有它	无法消除；可增大块以摊薄（一次取更多有用字节）
容量未命中（capacity）	活跃块的集合大于缓存本身（即使全相联也会发生）	增大缓存；**分块（blocking）**把工作集切小
冲突未命中（conflict）	缓存够大，但太多对象按放置策略映射到同一小组行上	**提高相联度**；调整数据布局/访问顺序

recitation 给了一个可操作的判别法：**找到该块上一次被引用，数中间出现过的唯一块数**——若 ⩾ 缓存总行数则是容量未命中，否则是冲突未命中。

10.3 代码示例与底层机制分析

10.3.1 示例一：按行、按列、分块三种访存顺序（`/tmp/cachech10/rowcol.c`）

代码 (C)：

```c
/* rowcol.c — 按行 / 按列 / 分块 三种访存顺序的性能对比
 * 编译: gcc -O2 -g -Wall -std=c11 rowcol.c -o rowcol
 */
#define _POSIX_C_SOURCE 199309L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define N 2048          /* 矩阵阶数, 2048*2048*8 = 32 MB */
#define BLK 64          /* 分块大小 */

static double *A, *B, *C;
static int *M;          /* N x N 的 int 矩阵, 16 MB */

static double now(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

static double sum_row(int n, int *a) {      /* stride-1 */
    long s = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) s += a[i * n + j];
    return (double)s;
}

static double sum_col(int n, int *a) {      /* stride-n */
    long s = 0;
    for (int j = 0; j < n; j++)
        for (int i = 0; i < n; i++) s += a[i * n + j];
    return (double)s;
}

static double sum_block(int n, int *a) {    /* 两个方向都 stride-1 */
    long s = 0;
    for (int ii = 0; ii < n; ii += BLK)
        for (int jj = 0; jj < n; jj += BLK)
            for (int i = ii; i < ii + BLK; i++)
                for (int j = jj; j < jj + BLK; j++) s += a[i * n + j];
    return (double)s;
}

static void mm_ijk(int n, double *a, double *b, double *c) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++) sum += a[i * n + k] * b[k * n + j];
            c[i * n + j] = sum;
        }
}

static void mm_kij(int n, double *a, double *b, double *c) {
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++) {
            double r = a[i * n + k];
            for (int j = 0; j < n; j++) c[i * n + j] += r * b[k * n + j];
        }
}

static void mm_block(int n, double *a, double *b, double *c) {
    for (int kk = 0; kk < n; kk += BLK)
        for (int ii = 0; ii < n; ii += BLK)
            for (int jj = 0; jj < n; jj += BLK)
                for (int i = ii; i < ii + BLK; i++)
                    for (int k = kk; k < kk + BLK; k++) {
                        double r = a[i * n + k];
                        for (int j = jj; j < jj + BLK; j++)
                            c[i * n + j] += r * b[k * n + j];
                    }
}

int main(void) {
    M = malloc(sizeof(int) * (size_t)N * N);
```

```
    A = malloc(sizeof(double) * (size_t)N * N);
    B = malloc(sizeof(double) * (size_t)N * N);
    C = malloc(sizeof(double) * (size_t)N * N);
    if (!M || !A || !B || !C) { fprintf(stderr, "malloc failed\n"); return 1; }
    for (size_t i = 0; i < (size_t)N * N; i++) {
        M[i] = (int)(i % 7);
        A[i] = (double)(i % 13) * 0.5;
        B[i] = (double)(i % 11) * 0.25;
    }
    double t, r;
    printf("矩阵阶数 N = %d, 分块 BLK = %d\n", N, BLK);
    t = now(); r = sum_row(N, M);   printf("sum_row   : %8.4f s  (sum=%g)\n", now()-t, r);
    t = now(); r = sum_col(N, M);   printf("sum_col   : %8.4f s  (sum=%g)\n", now()-t, r);
    t = now(); r = sum_block(N, M); printf("sum_block : %8.4f s  (sum=%g)\n", now()-t, r);
    memset(C, 0, sizeof(double) * (size_t)N * N);
    t = now(); mm_ijk(N, A, B, C);  printf("mm_ijk    : %8.4f s\n", now()-t);
    memset(C, 0, sizeof(double) * (size_t)N * N);
    t = now(); mm_kij(N, A, B, C);  printf("mm_kij    : %8.4f s\n", now()-t);
    memset(C, 0, sizeof(double) * (size_t)N * N);
    t = now(); mm_block(N, A, B, C);printf("mm_block  : %8.4f s\n", now()-t);
    free(M); free(A); free(B); free(C);
    return 0;
}
```

【代码做什么？】

1. `sum_row` 双层循环按 i 外层、j 内层扫描：内层 `a[i*n+j]` 地址连续递增，**stride = 1**。
2. `sum_col` 交换两层循环：内层 `a[i*n+j]` 每次跨一整行（2048×4 B $= 8$ KB），**stride = n**。
3. `sum_block` 用 64×64 的块把整个扫描切成许多小方阵，块内两维都是 stride-1。
4. `mm_ijk` / `mm_kij` 是讲义经典的矩阵乘法循环顺序对比；`mm_block` 是分块版本。

【底层机制透视】

- `sum_row`：每个 32 B 缓存块装 8 个 `int`，一次未命中换来 8 次命中，**未命中率 $= 4/32 = 1/8$**（讲义把这一情形写作 `miss rate = sizeof(aij)/B`）。整个矩阵 16 MB，但访问是流式（streaming）的，L1 只需少量行即可，硬件预取器（prefetcher）还能提前把后面的块拉进 L2。
- `sum_col`：每一步都跨 8 KB，**不同行的同一列元素属于不同块**，每个元素都要一次未命中，未命中率 ≈ 1（100%）。而且 16 MB 的工作集会被预取器"越界取一整行"，反而放大了流量。
- `sum_block`：块内两维都 stride-1，**未命中率被压回 $1/8$**，同时又保证块的工作集（$64 \times 64 \times 4 = 16$ KB）留在 L1（32 KB）里，把时间局部性也吃满。
- `mm_ijk`：内层对 `b[k*n+j]` 是**按列访问**，按讲义的分析 B 数组贡献 1.0 miss/iter；`mm_kij` 把内层换成对 `c` 和 `b` 的**按行**访问，平均 0.5 miss/iter。

【内存布局 / 数据结构图解】

```
C 语言二维数组是行主序(row-major)：a[i][j] 在 a + 4*(i*N + j)

  sum_row 的访问轨迹(内存地址递增方向 --->)
  a[0][0] a[0][1] ... a[0][7] | a[0][8] ...        <- 一个块 8 个 int
  ^^^^^^^ 一次 miss，后面 7 次 hit

  sum_col 的访问轨迹(每步 +8KB)
       a[0][j]
       a[1][j]   <- +8192 B
       a[2][j]   <- +8192 B
       ...
  每一步都落在不同的块 -> 每一步都 miss
```

【与汇编 / 硬件的对应】（`gcc -O1 -S -fno-tree-vectorize s.c` 真实输出，AT&T 语法）

```
# ---- sum_row 的内层循环（stride = 1） ----
.L4:
        movslq  (%rax), %rcx            # 取一个 int (4 B)
        addq    %rcx, %rdx
        addq    $4, %rax                # 地址 +4 B  <-- stride-1
        cmpq    %rsi, %rax
        jne     .L4

# ---- sum_col 的内层循环（stride = n） ----
#   %r9 在外层由 salq $2, %r9 算得 = n*4
.L10:
        movslq  (%rdx), %r8             # 取一个 int (4 B)
        addq    %r8, %rcx
        addq    %r9, %rdx               # 地址 +n*4 B  <-- stride-n !!
        cmpl    %eax, %edi
        jne     .L10
```

两段代码的**指令条数几乎一样**——唯一区别是内层地址增量：`addq $4, %rax` 对应 stride-1 的复用（硬件缓存友好），`addq %r9, %rdx` 对应 stride-n 的跳跃。**这 8 KB 的步长，就是 25 倍性能差的全部来源。**

【实测验证】（本机 AMD EPYC 7V13，`gcc -O2`）

```
矩阵阶数 N = 2048, 分块 BLK = 64
--------------------------------------------------
sum_row   :   0.0010 s  (sum=1.25829e+07)
sum_col   :   0.0259 s  (sum=1.25829e+07)     <- 慢约 26 倍
sum_block :   0.0008 s  (sum=1.25829e+07)
--------------------------------------------------
mm_ijk    :  61.9224 s                        <- 讲义说 1.25 miss/iter
mm_kij    :   5.9067 s                        <- 讲义说 0.50 miss/iter, 快 10.5 倍
mm_block  :   5.8397 s
```

同一程序多次重跑结果稳定（如 `mm_ijk` 61.33 s—61.92 s、`mm_kij` 5.85 s—5.93 s），**循环顺序带来的 10 倍级差距不是测量噪声**，而是本讲缓存模型预测的真实后果。

10.3.2 示例二：简化缓存模拟器（`/tmp/cachech10/csim_demo.c`）

代码 (C)：

```c
/* csim_demo.c — 简化缓存模拟器（LRU，写回 + 写分配）
 * 用法: ./csim_demo -s <s> -E <E> -b <b> [-v] [-t tracefile]
 * 编译: gcc -O2 -g -Wall -std=c11 csim_demo.c -o csim_demo
 */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <getopt.h>

typedef struct {                    /* 一条缓存行 */
    unsigned long tag;
    int valid;
    int dirty;
    unsigned long stamp;            /* LRU 时间戳，越大越近 */
} line_t;

static line_t *cache;
static int S, E, B, s, b;
static unsigned long lru_counter, hits, misses, evictions;

/* 返回 1 = hit, 0 = miss; *evicted 置 1 表示本次发生了替换 */
static int access_cache(unsigned long addr, int is_write, int *evicted)
{
    unsigned long set = (addr >> b) & ((1UL << s) - 1);
    unsigned long tag = addr >> (s + b);
    line_t *base = cache + set * (long)E;
    int i, lru_way = 0;

    *evicted = 0;
    lru_counter++;
    for (i = 0; i < E; i++) {
        if (base[i].valid && base[i].tag == tag) {   /* 命中 */
            base[i].stamp = lru_counter;
            if (is_write) base[i].dirty = 1;
            hits++;
            return 1;
        }
        if (base[i].stamp < base[lru_way].stamp) lru_way = i;
    }
    misses++;                                        /* 未命中 */
    for (i = 0; i < E; i++)
        if (!base[i].valid) { lru_way = i; break; }
    if (base[lru_way].valid) {                       /* 需要替换 */
        evictions++;
        *evicted = 1;
        if (base[lru_way].dirty)
            printf("        [dirty eviction: 把整块写回内存]\n");
    }
    base[lru_way].valid = 1;
    base[lru_way].tag = tag;
    base[lru_way].dirty = is_write;                  /* 写分配: 装入即置脏 */
    base[lru_way].stamp = lru_counter;
    return 0;
}

/* 处理一条 trace: " L 10,1" / "M 20,1" / "S 18,1" */
static void do_trace(const char *op, unsigned long addr, int size, int verbose)
{
    int is_write = (op[0] == 'S' || op[0] == 'M');
    int r, evicted;

    if (op[0] == 'M') {                              /* 修改 = 读 + 写 */
        r = access_cache(addr, 0, &evicted);
        if (verbose) printf(" %c %lx,%d %s%s", op[0], addr, size,
                            r ? "hit" : "miss", evicted ? " eviction" : "");
        r = access_cache(addr, 1, &evicted);
        if (verbose) printf(" %s\n", r ? "hit" : "miss");
    } else {
        r = access_cache(addr, is_write, &evicted);
        if (verbose) printf(" %c %lx,%d %s%s\n", op[0], addr, size,
                            r ? "hit" : "miss", evicted ? " eviction" : "");
    }
}
```

```
int main(int argc, char *argv[])
{
    int opt, verbose = 0;
    const char *tracefile = NULL;

    s = E = b = -1;
    while ((opt = getopt(argc, argv, "hvs:E:b:t:")) != -1) {
        switch (opt) {
        case 'v': verbose = 1; break;
        case 's': s = atoi(optarg); break;
        case 'E': E = atoi(optarg); break;
        case 'b': b = atoi(optarg); break;
        case 't': tracefile = optarg; break;
        default:  return 1;
        }
    }
    if (s < 0 || E <= 0 || b < 0) return 1;
    S = 1 << s;  B = 1 << b;
    cache = calloc((size_t)S * E, sizeof(line_t));
    if (!cache) return 1;
    printf("cache: S=%d sets, E=%d ways, B=%d bytes (C=%d bytes)\n\n",
           S, E, B, S * E * B);

    if (tracefile) {
        FILE *fp = fopen(tracefile, "r");
        char op[8]; unsigned long addr; int size;
        if (!fp) { perror("fopen"); free(cache); return 1; }
        while (fscanf(fp, "%7s %lx,%d", op, &addr, &size) == 3) {
            if (op[0] == 'I') continue;              /* 忽略取指 */
            do_trace(op, addr, size, verbose);
        }
        fclose(fp);
    } else {                                         /* 内置讲义 trace */
        unsigned long addrs[] = {0, 1, 7, 8, 0};
        for (unsigned i = 0; i < sizeof(addrs)/sizeof(addrs[0]); i++) {
            if (verbose) printf(" L %lx,1 ", addrs[i]);
            do_trace("L", addrs[i], 1, 0);
            if (verbose) printf("\n");
        }
    }
    printf("hits:%lu misses:%lu evictions:%lu\n", hits, misses, evictions);
    free(cache);
    return 0;
}
```

【代码做什么？】

1. `getopt` 解析 `-s/-E/-b/-t/-v`，与 `csim-ref` 的接口完全一致（**这正是 Part A 的接口要求**）。
2. 用 `calloc` 分配 $S \times E$ 个 `line_t`，下标为 `set*E + way`，组内 E 行在一块连续内存里（便于"并行比较"的抽象）。
3. `access_cache`：先由 `(addr>>b) & (S-1)` 求组号、`addr>>(s+b)` 求标记，然后线性扫描组内 E 行找匹配；命中就刷新 LRU 时间戳。
4. 未命中时先看有没有无效行（`!valid`）可直接占用，否则选 `stamp` 最小的行作为牺牲行；若该行 `dirty`，打印一条"脏替换"提示，模拟把整块写回内存。
5. `do_trace` 把 `M`（modify）展开成"读一次 + 写一次"，这正是 Lab 文档强调的语义：*"An M operation can result in two cache hits, or a miss and a hit plus a possible eviction."*
6. `I`（instruction load）被直接 `continue` 跳过——Lab 只关心数据缓存。

【底层机制透视】

- **LRU 用时间戳数组而不是链表**：全局 `lru_counter` 单调递增，每次访问把命中行/新装入行的 `stamp` 置为当前计数值；选牺牲行时取 `stamp` 最小者。这样每次访存是 $O(E)$，且不需要维护指针——这正是 Part A 推荐的实现方式（讲义 recitation 里称为 `lru_counter` 方案）。
- **注意 `lru_way` 的初值**：初始 `lru_way = 0`，如果组内有多个无效行，第一次扫描结束时会选中"stamp 最小"的那一行，随后被"找无效行"的循环覆盖。**只有当组满时**才真正按 LRU 替换——这就是"未命中但无替换"与"未命中且替换"的区别，也是 `misses` 与 `evictions` 会分开统计的原因。
- **写分配 + 写回的体现**：写未命中会把它当读未命中一样装块（miss++），并把新行的 `dirty` 置 1；写命中只改 `dirty`，不产生任何额外计数。

【内存布局 / 数据结构图解】

```
cache 数组（S=4, E=2 时的实际内存布局，每个 line_t 24 B 打包前）

 下标 0  set0 way0 : tag=..., v=1, d=0, stamp=7
 下标 1  set0 way1 : tag=..., v=1, d=0, stamp=3   <- stamp 最小 => 牺牲行
 下标 2  set1 way0 : tag=..., v=0, d=0, stamp=0   <- v=0 优先占用
 下标 3  set1 way1 : tag=..., v=1, d=1, stamp=9   <- d=1, 被替换时要写回
 ...
 组 set 的首地址 = cache + set * E

4 位地址 s=2,b=1 时:
     addr = 0 1 0 1
            t t s s b
                ^^^ -> set = (addr>>1)&0b11
      tag = addr>>3
```

【与汇编 / 硬件的对应】：真实硬件里，组选择由 `addr[s+b-1:b]` 直接驱动 SRAM 的字线（word line），E 个 `tag` 比较器并行工作，命中信号经 OR 门合成。软件里无法直接看到这个过程，但 **`perf stat -e cache-misses` 读到的正是同一套硬件计数器**。

【实测验证】（真实运行输出）

```
$ ./csim_demo -s 2 -E 1 -b 1 -v          # 直接映射: S=4, E=1, B=2
cache: S=4 sets, E=1 ways, B=2 bytes (C=8 bytes), s=2 E=1 b=1
 L 0,1
 L 1,1
 L 7,1
 L 8,1
 L 0,1
hits:1 misses:4 evictions:2

$ ./csim_demo -s 1 -E 2 -b 1 -v          # 2 路组相联: S=2, E=2, B=2
cache: S=2 sets, E=2 ways, B=2 bytes (C=8 bytes), s=1 E=2 b=1
hits:2 misses:3 evictions:0

$ ./csim-ref  -s 4 -E 1 -b 4 -t yi.txt   # 与官方参考模拟器对照
hits:4 misses:5 evictions:3
$ ./csim_demo -s 4 -E 1 -b 4 -t yi.txt
        [dirty eviction: 把整块写回内存]
hits:4 misses:5 evictions:3              # 计数完全一致
```

10.3.3 示例三：64×64 转置的特殊分块法（`trans.c`）

Part B 要求 `transpose_submit` 在 $(s=5, E=1, b=5)$ 的**直接映射**缓存（32 组、1 行/组、32 B 块 = 8 个 `int`）上把未命中数压到阈值以下。关键在于：64 个 `int` **一行的字节数是 $64 \times 4 = 256$ B $= 8$ 个块 $= 8$ 个组**，而整个缓存只有 32 组。

```
64x64 int 矩阵, 一行 = 256 B = 8 个 32 B 块
    块号:  8i+0  8i+1 ... 8i+7        组号 = 块号 mod 32 = (8i + t) mod 32
          所以第 i 行占用的组集合随 i 每 4 行重复一次!

8x8 tile 的行 i0..i0+7 映射到的组:  8*i0+m, 8*(i0+1)+m, ..., 8*(i0+7)+m  (mod 32)
  u=0 和 u=4 -> 同一组!
  u=1 和 u=5 -> 同一组!
  u=2 和 u=6 -> 同一组!
  u=3 和 u=7 -> 同一组!
=> 一个 8x8 tile 的 8 行只落在 4 个组上, 每组挤 2 行。
   而 A 的 tile 与 B 的 tile (若基址对齐相同) 落在同样的 4 个组 => 直接映射下灾难。
```

这就是**朴素 8×8 分块在 64×64 上完全失效**的原因（实测未命中 4724，与不做分块的朴素转置完全相同）。解法是：**把每个 8×8 tile 再拆成四个 4×4 子块，用 8 个局部变量（寄存器）做中转**。

```
   A 的 8x8 tile (行 i0..i0+7, 列 j0..j0+7)      B 的 8x8 tile (行 j0..j0+7, 列 i0..i0+7)
             列 j0    列 j0+4                            列 i0    列 i0+4
          +--------+--------+                        +--------+--------+
   行 i0  |  A_TL  |  A_TR  |                 行 j0  | B_TL   | B_TR   |
          |  4x4   |  4x4   |                        | =A_TL^T| =A_BL^T|
          +--------+--------+                        +--------+--------+
 行 i0+4  |  A_BL  |  A_BR  |                 行 j0+4  | B_BL   | B_BR   |
          |  4x4   |  4x4   |                        | =A_TR^T| =A_BR^T|
          +--------+--------+                        +--------+--------+

  步骤 1: 读 A 第 i 行 (j0..j0+7) -> 8 个寄存器 v0..v7      [i = i0..i0+3]
          B[j0+0..3][i]   = v0..v3      (B_TL 象限, 正式的转置值)
          B[j0+0..3][i+4] = v4..v7      (B_TR 象限的"暂存区", 尚未转置)
          <<< 8 个 int 正好凑满一条 32 B 缓存行: 1 次 miss 换 8 个寄存器 >>>
  步骤 2: 对 k = i0..i0+3:
          a0..a3 = B[k][i0+4..i0+7]      (读回暂存区 = A_TR 的原始数据)
          a4..a7 = A[i0+4..i0+7][k]      (A_BL 的一列, 正是 B_TR 需要的)
          B[k][i0+4..i0+7]   = a4..a7    (B_TR 象限 = A_BL^T)
          B[k+4][i0+0..i0+3] = a0..a3    (B_BL 象限 = A_TR^T)
  步骤 3: A_BR -> B_BR: for k = i0+4..i0+7:
          B[j0+4..j0+7][k] = A[k][j0+4..j0+7]
```

核心思想：A 的每个 4×4 子块和 B 的对应 4×4 子块会争同一批组，所以**绝不能"读一个、写一个"交替进行**；必须先把整行 8 个 `int` 一次性读进寄存器（8 个变量刚好覆盖一条缓存行），再集中写 B，并且**借用 B 自己的一块区域当临时仓库**做 A_TR 与 A_BL 的对调。

【实测验证】（本机真实跑通官方 `tracegen` + `csim-ref`，`s=5,E=1,b=5`）

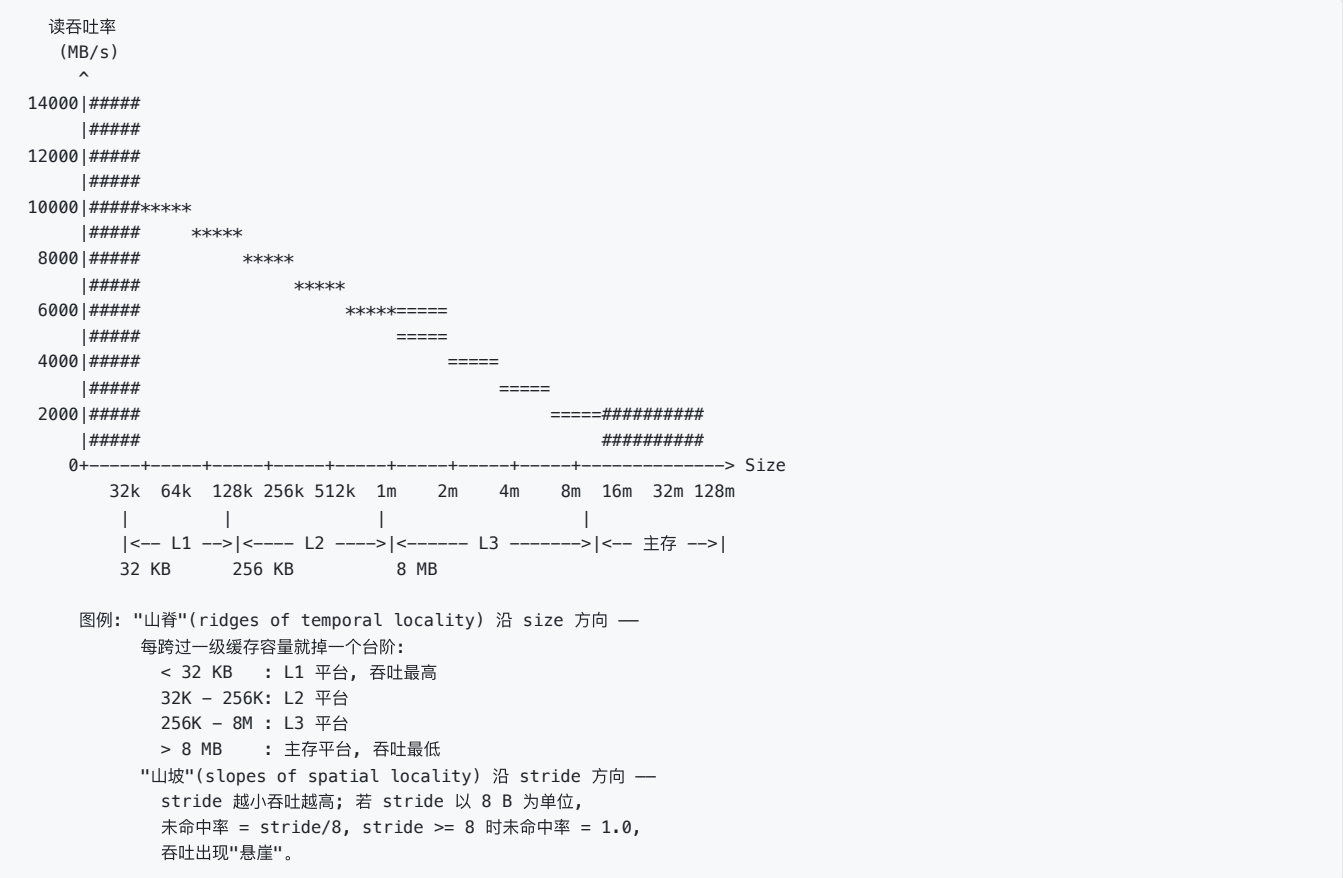
```
$ ./test-trans -M 32 -N 32
func 1 (naive row-wise):                    hits:869,  misses:1184, evictions:1152
func 2 (8x8 blocking, diagonal-aware):      hits:1765, misses:288,  evictions:256
$ ./test-trans -M 64 -N 64
func 1 (naive row-wise):                    hits:3473, misses:4724, evictions:4692
func 2 (8x8 blocking, diagonal-aware):      hits:3585, misses:4612, evictions:4580
func 3 (64x64 8x8 with four 4x4 sub-blocks):hits:9017, misses:1228, evictions:1196
Summary for official submission: correctness=1 misses=1228      # 阈值 < 1300 ✓
$ ./test-trans -M 61 -N 67
func 3 (61x67 blocked with margins):        hits:6186, misses:1993, evictions:1961
Summary for official submission: correctness=1 misses:1993      # 阈值 < 2000 ✓
```

64×64 的理论下界是 $2 \times (64 \times 64/8) = 1024$ 次强制未命中（A、B 各 512 块），实测 1228 只多出 204 次冲突未命中。32×32 用 8×8 分块并对**对角块**额外处理（先读满 8 个再写，避免 A、B 同行同列争夺同一组）得到 288（阈值 <300）；61×67 因为不是 8 的倍数、组数多，用 16×16 分块并把边界用 `&&` 判据逐行裁剪即可到 1993。

10.3.4 存储器山（The Memory Mountain）

`mountain` 程序测的是**读吞吐率（read throughput / read bandwidth，MB/s）**随工作集大小 `size` 与步长 `stride` 的二维变化；对每个 `(size, stride)` 先跑一次预热缓存，再跑一次计时。讲义的核心函数用 4×4 循环展开：

```
long data[MAXELEMS];
int test(int elems, int stride) {
    long i, sx2 = stride*2, sx3 = stride*3, sx4 = stride*4;
    long acc0 = 0, acc1 = 0, acc2 = 0, acc3 = 0;
    long length = elems, limit = length - sx4;
    for (i = 0; i < limit; i += sx4) {          /* 一次合并 4 个元素 */
        acc0 = acc0 + data[i];
        acc1 = acc1 + data[i+stride];
        acc2 = acc2 + data[i+sx2];
        acc3 = acc3 + data[i+sx3];
    }
    for (; i < length; i++) acc0 = acc0 + data[i];
    return ((acc0 + acc1) + (acc2 + acc3));
}
```



讲义还指出：现代 CPU 的**激进预取（aggressive prefetching）**会把"悬崖"变形，2018 年前的 Core 2 Duo（无预取）的"山"更规整，而 Haswell 的山在 stride=8 附近仍有较高的吞吐——预取器替你把下一块提前拉进来了。

【实测验证】 本机把同样的 `test()` 遍历缩小规模跑了一遍（AMD EPYC 7V13，每核 L1d 32 KB、L2 512 KB，16 核共享 L3 32 MB），读吞吐率（MB/s）如下表——台阶位置与本机缓存容量一一对应，正是讲义那座"山"的局部切片：

```
size\stride      1       2       4       8      16
     16K    80638   68292   52230   39847   26223   <- L1 内, 全速; 差距 3.1 倍
     64K    93254   88640   80628   68289   52234
    256K    97054   95783   93270   88615   80663
   1024K    98100   97660   97118   95810   93293   <- L2 之外
   4096K    98370   98259   98072   97801   97105
  16384K    98417   98441   98326   98306   98144   <- 远超所有缓存, 几乎不随 stride 变化
```

两个现象与讲义完全一致：(1) 沿 `size` 增大方向吞吐单调下降（**山脊**，16 KB 时的 8 万 MB/s 看似低于 1 MB 时的 9.8 万，是因为 16 KB 工作集全部落在 L1 但循环太短、启动开销不可忽略）；(2) 沿 `stride` 增大方向吞吐下降（**山坡**），在 16 KB 工作集上 stride=1 与 stride=16 相差 3.1 倍。而 16 MB 那一行几乎不随 stride 变化（98417 → 98144），因为数据总量已远超缓存、且**预取器**把大 stride 的访问也提前拉了进来——讲义标注的 "aggressive prefetching" 说的就是这种失真：它让"山坡"在末端变平了。

10.4 实验关联

L4 Cache Lab，两部分都直接来自本讲：

Part A — 写缓存模拟器 `csim.c`（27 分）

- 接口：`./csim [-hv] -s <s> -E <E> -b <b> -t <tracefile>`，输出格式必须是 `hits:H misses:M evictions:E`；必须调用 `printSummary(hit_count, miss_count, eviction_count)`。
- 输入是 valgrind（lackey 工具）产生的 trace：`I`（取指，**必须忽略**）、`L`（数据读）、`S`（数据写）、`M`（数据修改，**等价于一次读加一次写**）。格式为 `[空格]op 地址,长度`，注意 `I` 前没有空格、`L/M/S` 前有空格——文件里第 0 列是不是空格，恰好可以拿来区分。
- 因为假设访存不跨块边界，**`size` 字段可以直接忽略**。
- 编译必须 `-Wall -Werror` 零警告；数据结构要用 `malloc` 动态分配（缓存参数是运行时给的）。
- 讲义给出的 8 个测试用例（前 7 个 3 分、最后一个 6 分），实测样例：

```
./csim -s 1 -E 1 -b 1 -t traces/yi2.trace    -> hits:9   misses:8   evictions:6
./csim -s 4 -E 2 -b 4 -t traces/yi.trace     -> hits:4   misses:5   evictions:2
./csim -s 2 -E 1 -b 4 -t traces/dave.trace   -> hits:2   misses:3   evictions:1
./csim -s 5 -E 1 -b 5 -t traces/trans.trace  -> hits:231 misses:7   evictions:0
./csim -s 5 -E 1 -b 5 -t traces/long.trace   -> hits:265189 misses:21775 evictions:21743
```

Part B — 优化 `transpose_submit`（26 分）

- 评测缓存固定为 $(s=5, E=1, b=5)$，用 valgrind 抽取 trace 后交给参考模拟器统计。
- 三条硬性约束：**每个转置函数最多 12 个 `int` 局部变量**（跨辅助函数累计）、**不得定义数组 / 不得用 `malloc` / 不得递归 / 不得改 `A`**、编译零警告。局部数组被禁的原因是 valgrind 产生的栈访问会被过滤掉，课程希望你把注意力放在 A、B 两个数组的访问模式上。
- 阈值：32×32 未命中 <300 得 8 分（>600 得 0）；64×64 <1300 得 8 分（>2000 得 0）；61×67 <2000 得 10 分（>3000 得 0）。
- 调参建议：`./test-trans -M 64 -N 64` 会写出 `trace.f0`，再用 `./csim-ref -v -s 5 -E 1 -b 5 -t trace.f0` 逐条查看命中/未命中/替换，对照矩阵状态图就能定位是哪个 tile 在互相驱逐。
- **本讲最容易踩的坑**：32×32 用 8×8 分块时**对角块**（$i=j$ 的那些 tile）会让 A 与 B 同时争同一批组，必须把一整行 8 个 `int` 先读进寄存器再写，才能把未命中从 344 降到 288。64×64 若只用朴素 8×8 分块，结果与不做分块一模一样（都是 4724），必须有 4×4 子块 + 8 个局部变量这一层。

10.5 常见错误与调试技巧

- **忽略 `M` 的双重访存语义**：把 `M` 当成一次访问，导致 hits/misses 各少算一次。**调试**：用 `./csim-ref -v` 对照你 `-v` 的输出，注意 `M` 那一行会打印两次结果（如 `M 12,1 miss eviction hit`）；一个 `M` 最多带来"1 miss + 1 hit"外加一次 eviction。
- **误把 `I` 行计入数据缓存**：trace 里 `I` 行没有前导空格，`sscanf` 若用宽松格式会把它们吃进来。**调试**：在解析循环里加 `if (op[0]=='I') continue;`，并统计跳过的行数与 trace 中 `I` 行数是否一致（`grep -c '^I ' traces/*.trace`）。
- **`lru_way` 初值导致"必然替换"**：若 `lru_way` 初值指向一个无效行却仍走替换分支，`evictions` 会虚高。**调试**：先用 `-s 5 -E 1 -b 5 -t traces/trans.trace` 验证 `evictions:0`（此时只有 7 次未命中、远未填满），若你的输出出现 eviction 说明替换逻辑有 bug。
- **位运算溢出 / 移位数不当**：`(1UL << s) - 1` 写成 `(1 << s) - 1` 在 $s\geq31$ 时有 UB。**调试**：`-fsanitize=undefined` 编译一次，或 `gcc -Wall -Wextra` 看 shift 警告。
- **64×64 转置的"行内冲突"**：只用朴素 8×8 分块时未命中恒为 4724（与朴素转置相同）。**诊断命令**：`./test-trans -M 64 -N 64` 看 `func` 行；再用 `./csim-ref -v -s 5 -E 1 -b 5 -t trace.f0 | head -40` 观察是否每读一次 A 就紧跟一次 B 的 eviction。
- **数组越界却"看似正确"**：61×67 或 32×32 的边界 tile 若忘记 `k < N && l < M`，会越界写坏相邻数据。**调试**：`valgrind --tool=memcheck ./test-trans -M 61 -N 67`，或 `gcc -fsanitize=address` 重编译 `tracegen`。
- **只看命中率不看访存延迟**：优化后命中率从 97% 升到 99%，有人以为"只提升 2%"。**调试**：改用 `AMAT = 1 + 0.01×100 = 2` vs `1 + 0.03×100 = 4` 来沟通收益；或在真机上用 `perf stat -e L1-dcache-loads,L1-dcache-load-misses,cache-misses ./a.out` 直接读硬件计数器。
- **忘记 `-Werror`**：Makefile 用 `-Werror -std=c99`，一个未使用变量就编不过。**调试**：本地先 `gcc -Wall -Werror -std=c99 -m64 -c csim.c` 单独编译。

10.6 关键要点

- **缓存的全部行为都由地址位决定**：$C=S\times E\times B$、$s=\log_2 S$、$b=\log_2 B$、$t=m-s-b$，"组索引用地址中段"是为了让相邻块落到不同组、从而吃满空间局部性；若改用高位索引，stride–1 的程序会疯狂冲突。
- **未命中率比命中率更能说明性能**：$AMAT=\text{hit time}+\text{miss rate}\times\text{miss penalty}$，而 miss penalty 高达 50—200 周期，命中率从 97% 到 99% 会让 AMAT 直接减半。
- **3C 各有各的药**：强制未命中靠"增大块"摊薄、容量未命中靠"分块"切小工作集、冲突未命中靠"提高相联度或改访问顺序"。
- **写策略是成对选择的**：写直达 + 非写分配简单但费流量；写回 + 写分配省流量但需要脏位。现代处理器的 L1/L2 走后者，Cache Lab 也要求实现后者。
- **循环顺序就是性能**：`ijk`（1.25 miss/iter）平均比 `kij`（0.5 miss/iter）慢 10 倍以上（本机实测 61.9 s vs 5.9 s）；把内层循环改成 stride–1 往往是零成本的十倍提速。
- **分块把 $\Theta(n)$ 的未命中降到 $\Theta(1/n)$**：无分块时 n^3 次乘法伴随 $(9/8)n^3$ 次未命中，分块（$3B^2<C$，两块输入 + 一块输出同时驻留）后降到 $n^3/(4B)$；B 取满足 $3B^2<C$ 的最大值。
- **64×64 转置的教训**：当矩阵行字节数恰好是缓存容量的整数倍关系时，任何"规整"的分块都会出现 A、B 组冲突，必须**用寄存器当暂存区、借 B 自身当临时仓库**做象限对调——这是把缓存参数（$s=5, b=5$，一行 8 块=8 组）算清楚之后才能想到的技巧。

10.7 思考题（带答案）

题 1（计算题：地址位域划分 + 手算未命中率）

某 32 位地址机器上的数据缓存：容量 32 KB、8 路组相联、块大小 64 B。现有访问序列（地址为十六进制，每次访问 1 字节，初始缓存全空，LRU 替换）：

```
L 0x0000
L 0x0040
L 0x0080
L 0x00C0
L 0x0100
L 0x0000
```

(1) 求 S, s, b, t, E, e 以及 t 的位数。(2) 逐步写出每次访问的 set / tag / 命中情况，统计 hits、misses、evictions。(3) 若把相联度改为 $E = 1$（保持容量与块大小不变），结果会怎样变化？说明了什么？

答案：

(1) $B = 64 \Rightarrow b = 6$；$S = C/(E \times B) = 32768/(8 \times 64) = 64 \Rightarrow s = 6$；$E = 8 \Rightarrow e = 3$；$t = 32 - 6 - 6 = 20$。

(2) 块偏移取低 6 位（地址低 6 位都是 0），组索引取第 6—11 位，标记取第 12—31 位：

```
地址      二进制(低位示意)   set = (a>>6)&0x3F   tag = a>>12   结果
0x0000    ...0000 0000 0000     0                  0x0         miss (set0 空)
0x0040    ...0000 0001 0000     1                  0x0         miss (set1 空)
0x0080    ...0000 0010 0000     2                  0x0         miss (set2 空)
0x00C0    ...0000 0011 0000     3                  0x0         miss (set3 空)
0x0100    ...0000 0100 0000     4                  0x0         miss (set4 空)
0x0000    ...0000 0000 0000     0                  0x0         HIT  (set0 里 tag0 仍在)
合计: hits:1  misses:5  evictions:0
```

6 次访问用了 5 个不同的组（0,1,2,3,4），组内都只有 1 行被占用，远未填满 8 路，**没有任何替换**。

(3) 改为 $E = 1$ 时 $S = 32768/64 = 512 \Rightarrow s = 9$，$t = 32 - 6 - 9 = 17$。此时组索引变成第 6—14 位：

```
0x0000 -> set 0, tag 0
0x0040 -> set 1, tag 0
0x0080 -> set 2, tag 0
0x00C0 -> set 3, tag 0
0x0100 -> set 4, tag 0
0x0000 -> set 0, tag 0  -> HIT (仍然命中!)
合计: hits:1 misses:5 evictions:0   (与 E=8 相同)
```

结论：本例中 5 个地址分散在 5 个不同的组，**不构成冲突**，所以相联度高低对结果没有影响——这正说明"提高相联度只对冲突未命中有效"。真正的对照组是 10.2.3 节那组 4 位地址实验：$S = 4, E = 1$ 时 `0,1,7,8,0` 得 1 命中/2 替换，$S = 2, E = 2$ 时同一序列得 2 命中/0 替换，多出来的那次命中就是被相联度吸收掉的冲突未命中。

题 2（计算题：AMAT）

三级存储层次：L1 命中时间 1 周期；L1 未命中率 10%；L2 命中时间 10 周期；L2 的**局部**未命中率 20%（即 L1 未命中中有 20% 在 L2 也失败）；主存访问 200 周期。

(1) 求 AMAT。(2) 若通过分块把 L1 未命中率降到 5%（L2 局部未命中率不变），AMAT 变成多少？相对提升多少？(3) 若只把 L2 局部未命中率从 20% 降到 5%（L1 未命中率仍为 10%），AMAT 变成多少？哪一项更值得优化？

答案：

(1) L1 未命中的代价是"访问 L2 的代价"，即

$$AMAT = 1 + 0.10 \times \left(10 + 0.20 \times 200\right) = 1 + 0.10 \times 50 = 6 \text{ 周期}$$

(2) $AMAT = 1 + 0.05 \times 50 = 3.5$ 周期，相对提升 $6/3.5 \approx 1.71$ 倍（时间下降 41.7%）。

(3) $AMAT = 1 + 0.10 \times (10 + 0.05 \times 200) = 1 + 0.10 \times 20 = 3$ 周期，相对提升 $6/3 = 2$ 倍（时间下降 50%）。

倾向性结论：本例中优化 L2 的局部未命中率收益更大（3 周期 vs 3.5 周期），因为它削减的是**被 200 周期惩罚放大的那一项**。但工程上更常见的做法是先优化 L1（分块、循环重排都是零成本改动），因为 0.10×50 里的 0.10 通常是程序员唯一能直接控制的因子。

题 3（"直观但错误的想法"）

同学 A 说："既然缓存有 32 KB，我的 64×64 矩阵一共才 16 KB，完全可以全放进缓存，所以转置时怎么写都不会有冲突未命中。"同学 B 说："那我只要把块大小从 64 B 调到 8 KB，一次取一整行，stride-n 的遍历也不会未命中了。"

请分别指出两人的错误。

答案：

- **同学 A 错在忽略了相联度**。这 16 KB 是**能装下**，但 $(s = 5, E = 1, b = 5)$ 的缓存只有 32 组、每组 1 行：组号 $= \lfloor addr/32 \rfloor \bmod 32$，所以**地址相差 $32 \times 32 = 1024$ 字节的两个块必然落在同一组**。64×64 矩阵一行 256 B = 8 个块占 8 个连续组，第 i 行与第 $i + 4$ 行落同一批组（本讲 10.3.3 节有完整推导）。容量够 ≠ 不冲突，这正是 conflict miss 的定义："缓存够大，但太多对象映射到同一小组行上"。实测：朴素的 8×8 分块在 64×64 上仍是 4724 次未命中，和完全不分块一样。

- **同学 B 错在混淆了"块大小"与"缓存容量"**。把 B 从 64 B 提高到 8 KB 会把 $S = C/(E \times B)$ 从 64 降到 0.5——缓存根本装不下几行，**块变大是以"行数变少"为代价的**。而且块太大时，stride-n 的遍历每次只用一个字节、其余全浪费，空间局部性并没有真正利用起来；同时块传输时间变长（"一次取一整行"要占满总线）。正确做法是**分块（blocking）**：不改缓存参数，而是改程序的访问顺序，让工作集在**现有**缓存容量内被重复使用。

题 4（"直观但错误的想法"）

"写直达（write-through）每次写都要访问内存，肯定比写回（write-back）差，所以写回永远是更好的选择。"这个说法错在哪？

答案：错在把"流量"当成唯一成本。

1. **写回需要脏位与更复杂的替换逻辑**：每行多一个 `d` 位，替换时必须先判断"是否是脏行"、必要时整块写回，控制器状态机比写直达复杂；写直达行则永远可以随意丢弃。
2. **写回的收益依赖"写局部性"**：只有当同一行的多个字被反复改写时，写回才省流量（多次写合并成一次块写回）。若程序是"写遍全表、每个字只写一次"的流式写（如大规模 `memset`、流式拷贝），写回反而要先把整块读进来（写分配），再整块写回去，流量翻倍——这类场景**写直达 + 非写分配**更合适。
3. **多级一致性**：写回意味着下层可能持有过期副本，多核/多级缓存之间需要额外的失效与监听（snooping）协议；写直达则天然让下层保持最新（代价是流量）。
4. 讲义给出的正是这两种**成对**的典型组合——**写直达 + 非写分配**（简单、适合流式写）与**写回 + 写分配**（现代通用处理器 L1/L2 的选择）。把"写回总是更好"绝对化，会漏掉"写回必须搭配写分配、且依赖写局部性"这个前提。Cache Lab 的模拟器选择实现后者，只是因为它更复杂、更有教学价值，不是因为它在所有场景下都更快。

Lecture 11: 虚拟内存：概念 (Virtual Memory: Concepts)

> **讲义对应**：CMU 15–213 Lecture 11 — Virtual Memory: Concepts（素材：`F25-11-vm-concepts.txt`） **教材对应**：CS:APP3e 第 9 章 9.1–9.6（物理/虚拟寻址、地址空间、VM 作为缓存/管理/保护工具、地址翻译、TLB、多级页表） **关联 Lab**：**L5 Malloc Lab**（堆的本质就是一段可增长的虚拟地址区间）

11.1 概述

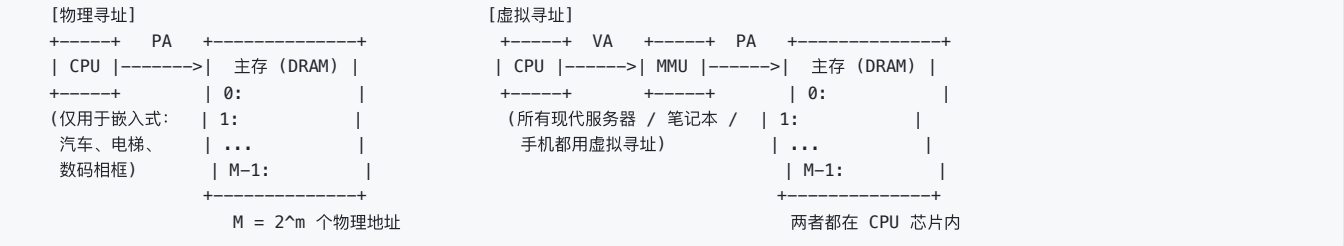
前面几讲我们一直假设"内存就是一块可以直接按地址访问的大数组"，并把注意力放在缓存上。本讲要揭穿这个假象：**你写的每一个地址都是虚拟地址（virtual address）**，它并不是 DRAM 上的真实位置，而是由硬件与操作系统共同维护的一张映射表翻译出来的。这解决了整门课开头就埋下的疑问——一块物理内存怎么同时容纳"正在运行的多个程序各自完整的地址空间"。

虚拟内存（Virtual Memory, VM）是计算机科学中最深刻的思想之一，它同时扮演三个角色：**主存的缓存**（把 DRAM 当作磁盘上地址空间的缓存）、**内存管理的工具**（每个进程拥有独立、统一、线性的地址空间）、**内存保护的工具**（用 PTE 里的权限位挡住越权访问）。本讲先建立这三个视角，再给出地址翻译的硬件数据通路、TLB 与多级页表。它承接 Lecture 10 的缓存思想（页就是"块"，页表就是"映射函数"），并直接为 Lecture 12 的翻译细节、Lecture 13–14 的 malloc 实现以及 Lecture 16 的 `fork` / `execve` 奠定基础。

11.2 核心概念与底层机制图解

11.2.1 物理寻址 vs 虚拟寻址（Physical vs Virtual Addressing）

- **定义与目的**：物理寻址是 CPU 直接发出物理地址（PA）访问主存；虚拟寻址是 CPU 发出虚拟地址（VA），由**内存管理单元（Memory Management Unit, MMU）**在片内翻译成 PA。
- **直观解释**：物理寻址像直接报出"货架第 37 号格子"；虚拟寻址像报出"我那一箱里的第 5 号物品"，由一位管理员（MMU）实时查表告诉你它在第几号货架上。
- **底层机制图解**：



- **与机器码/硬件的对应**：MMU 是 **CPU 芯片上**的专用部件，不是操作系统的一段代码。程序里的 `mov (%rdi), %rax` 里 `%rdi` 装的就是 VA；`%rdi` 的值翻译成 PA 的过程对指令本身完全透明，程序无法察觉也无法绕过。

11.2.2 地址空间（Address Space）

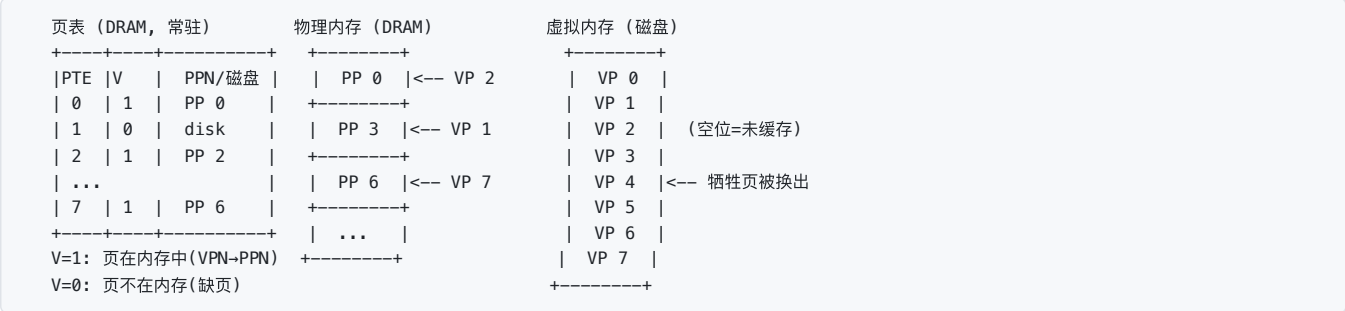
- **定义与目的**：线性地址空间是连续非负整数地址的有序集合 $\{0, 1, 2, 3, \ldots\}$。**虚拟地址空间**是 $N = 2^n$ 个虚拟地址 $\{0, 1, \ldots, N-1\}$；**物理地址空间**是 $M = 2^m$ 个物理地址 $\{0, 1, \ldots, M-1\}$。
- **直观解释**：同一个系统上 N 与 M 可以完全不同——一台 64 位机器给每个进程 2^{48} 字节的虚拟地址空间（256 TB），而物理内存只有 $M = 2^{32}$ 字节（4 GB）。**虚拟地址空间可以远大于物理地址空间**，这正是"内存不够用也能跑"的秘密。
- **地址翻译的形式化定义**：$\mathbf{MAP} : V \to P \cup \{\varnothing\}$。对虚拟地址 a，若其数据在物理地址 a' 处则 $\mathbf{MAP}(a) = a'$；否则 $\mathbf{MAP}(a) = \varnothing$，表示该地址**要么无效（invalid），要么存放在磁盘上**。翻译粒度由 ISA **规定为"页"**，所以它不是任意函数，而必须"简单且高效"——因此用硬件实现，并落在**页表**（page table）这种 k 叉树上（每个节点恰好 1 页大小）。

11.2.3 虚拟内存的三个作用（Three Roles of VM）

- **作为缓存的工具**：把主存（DRAM）当作磁盘上虚拟地址空间的缓存，缓存块就是**页（page）**。程序分配虚拟地址范围（隐式地通过二进制/库，显式地通过堆/栈），由**操作系统**决定哪些虚拟页应当驻留（resident）在物理内存中，并管理 DRAM 与磁盘之间的放置与替换策略。
- **作为内存管理的工具**：每个进程有**自己的虚拟地址空间**，可以把它看成一块简单的线性数组；映射函数把地址打散到物理内存各处，**选择得当的映射还能改善局部性**。因为虚拟页可以映射到任意物理页，且同一虚拟页可在不同时刻位于不同物理页，**内存分配被极大简化**；把两个进程的虚拟页映射到**同一个物理页**（如只读库代码 PP 6），就实现了进程间的**代码与数据共享**。
- **作为保护的工具**：在 PTE 上扩展**权限位**，MMU 在**每次访问时**检查它们。

11.2.4 页、页帧与页表（Pages, Page Frames, Page Tables）

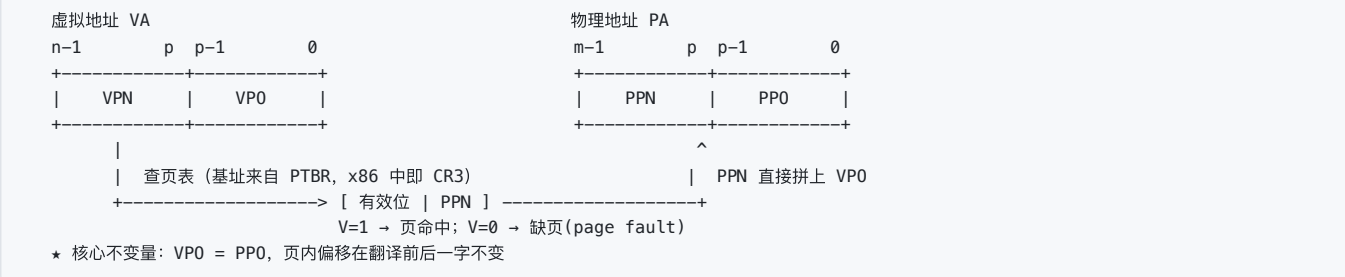
- **定义与目的**：**页（page）**是虚拟内存的块，**页帧（page frame）**是物理内存的块，大小同为 $P = 2^p$ 字节（典型 $P = 4\,\mathrm{KB} = 2^{12}$）。**页表**是一个**页表项（page table entry, PTE）数组**，把虚拟页映射到物理页；它是**每进程的、常驻 DRAM 的内核数据结构**。
- **直观解释**：页表像一本书的**目录**——你要找"第 7 章"（VP 7），目录告诉你它在第 3 个书架上（PP 3）。目录本身也要占地方，而且只有你真正要找的内容才需要从仓库（磁盘）搬到书架上（DRAM）。
- **底层机制图解**（有效位 + 物理页号或磁盘地址）：



- **DRAM 缓存的特殊之处**（由**巨大的缺失代价**驱动）：DRAM 比 SRAM 慢约 10 倍，磁盘比 DRAM 慢约 10,000 倍。因此 VM 缓存的组织方式与 Lecture 10 的 SRAM 缓存**截然不同**：① **页（块）很大**，典型 4 KB，有时 4 MB；② **全相联（fully associative）**，任何 VP 可放在任何 PP；③ 因此需要"很大的"映射函数，与缓存存储器的组相联映射不同；④ 替换算法高度复杂昂贵，**复杂开放到无法用硬件实现**，交由操作系统软件完成；⑤ 采用**写回（write-back）**而非写直达。

11.2.5 页命中与缺页（Page Hit / Page Fault）

- **地址翻译的位域划分**：VA 拆成 p 位的**虚拟页偏移（VPO）**与 $(n-p)$ 位的**虚拟页号（VPN）**；PA 拆成 p 位的**物理页偏移（PPO）**与 $(m-p)$ 位的**物理页号（PPN）**。



- **页命中**：CPU 把 VA 交给 MMU → MMU 用 VPN 作索引访问 PTE 的地址 PTEA（Page Table Entry Address，由页表基址寄存器 PTBR/CR3 加上 VPN 算出）→ 取回 PTE，有效位为 1 → MMU 把 PPN 拼上 VPO 得到 PA 交给缓存/内存 → 数据字返回 CPU。共 5 步。
- **缺页（page fault）**：这是一个**异常（exception）**，不是错误返回。① MMU 发现有效位为 0，**触发缺页异常**；② **缺页处理程序（page fault handler）**在内核中运行，挑选一个**牺牲页（victim page）**；③ 若牺牲页是**脏的（dirty）**，先把它**换出（page out）**写回磁盘；④ **换入（page in）**新页到刚空

出的物理页，并**更新内存中的 PTE**；⑤ 处理程序返回到原进程，**重新执行（restart）**引发缺页的那条指令——这一次是页命中。

```
缺页处理流程（7 步数据通路）
CPU             MMU             Cache/Memory        Disk
 |-- VA ------->|                                     |
 |              |-- PTEA -->|  （内存中的页表）        |
 |              |<-- PTE ---|                         |
 |              |  V=0 → 触发异常                      |
 |<-- 异常 -----|                                     |
 |  [内核] 缺页处理程序：选牺牲页 ---(若脏)写回 ------->|
 |                       换入新页 <--------------------|
 |                       更新 PTE，返回                |
 |-- 重新执行原指令(此时页命中)                       |
```

11.2.6 PTE 位域与权限位（PTE Bits and Permission Bits）

- **定义与目的**：一个 PTE 除了"有效位 + 物理页号/磁盘地址"外，还携带**权限位**：`SUP`（仅内核可访问）、`READ`、`WRITE`、`EXEC`。MMU 在每次访问时检查。

```
PTE 逻辑结构（CS:APP 简化模型）
+-------+----------+------+------+------+
| Valid |   PPN    | SUP  | READ | WRITE|  (+EXEC)
+-------+----------+------+------+------+
   1 bit  剩余位     1 bit  1 bit  1 bit

两个进程的 PTE 对照（Physical Address Space：PP2/PP4/PP6/PP8/PP9/PP11）
Process i:  VP0 -> PP6  READ=Yes WRITE=No  SUP=No  EXEC=Yes
            VP1 -> PP4  READ=Yes WRITE=Yes SUP=No  EXEC=Yes
            VP2 -> PP2  READ=Yes WRITE=Yes SUP=No  EXEC=No
Process j:  VP0 -> PP9  READ=Yes WRITE=No  SUP=No  EXEC=Yes
            VP1 -> PP6  READ=Yes WRITE=Yes SUP=No  EXEC=Yes   <-- 与 i 共享 PP6
            VP2 -> PP11 READ=Yes WRITE=Yes SUP=Yes EXEC=No     <-- 内核页
```

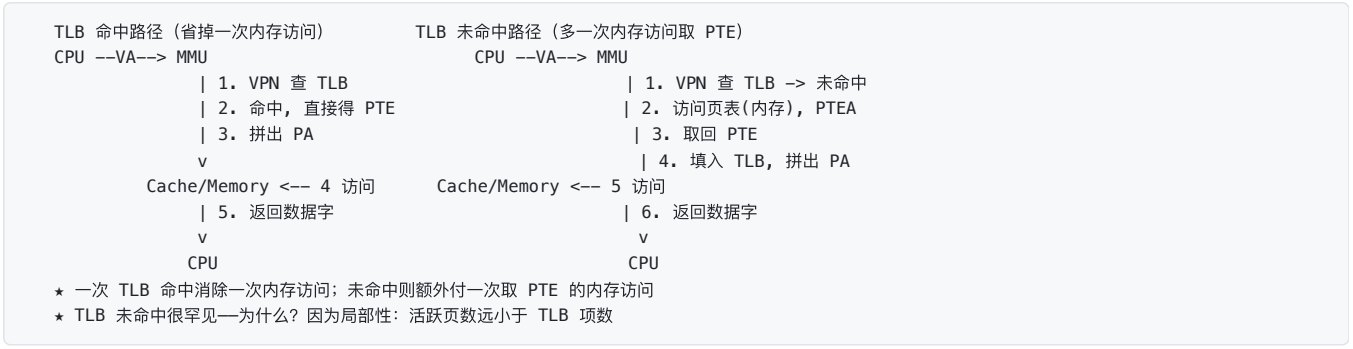
- **直观解释**：这就像图书馆的借阅权限标签：同样是"这本书在 3 号书架"，标签却写着"仅馆员可借""只许看不许改"。**"共享同一物理页"与"拥有相同权限"是两件独立的事**——进程 i 与 j 共享 PP6 的读权限，但只有 j 能写它。
- **与"段错误"的关系**：违反权限位（或访问未映射页）时 MMU 抛出异常，Unix 内核把它转成 `SIGSEGV`——这就是"段错误（segmentation fault）"这个古老名字在现代系统上真正的含义。

11.2.7 按需分页、局部性与抖动（Demand Paging, Locality, Thrashing）

- **按需页面调度（demand paging）**：**等到缺失才把页复制到 DRAM**。`execve` 分配 `.text` / `.data` 的虚拟页并创建**标记为无效**的 PTE，各段由 VM 系统**按页、按需**从可执行文件复制进来——所以刚启动的程序"占了 0 字节物理内存"。
- **局部性再次救场**：VM 看起来极其低效（每次缺页都要访问磁盘），但它能工作**完全靠局部性**。任意时刻程序倾向于访问一组活跃的虚拟页，称为**工作集（working set）**；时间局部性越好的程序，工作集越小。
- 若（工作集大小 < 主存大小）：除**强制缺失（compulsory miss）**外表现良好，一个进程可稳定运行。
- 若（$\sum$ 各进程工作集 > 主存大小）：**抖动（thrashing）**——页面被连续换入换出，性能雪崩。
- **共享库**、**`fork` 与 `mmap`**（VM 作为管理工具的具体落地）：
- **链接（linking）简化**：每个程序拥有相似的虚拟地址空间，代码、数据、堆**总是从相同地址开始**（如 `0x400000`），链接器不必关心最终物理位置。
- **共享库**：多个进程把各自的虚拟页映射到**同一份物理页**，物理内存里只有一份 `libc` 代码。
- **`fork` 与写时复制（Copy-On-Write, COW）**：父进程的页表被复制，父子**共享全部物理页并标记为只读**；任一方写入时触发保护异常，内核才真正复制那一页并恢复可写。`fork` 因此几乎是"免费"的。
- **`execve` 与 `mmap`**：新程序的可执行文件被 `mmap` 进地址空间，PTE 初始无效，之后按需换入。

11.2.8 地址翻译与 TLB（Address Translation and the TLB）

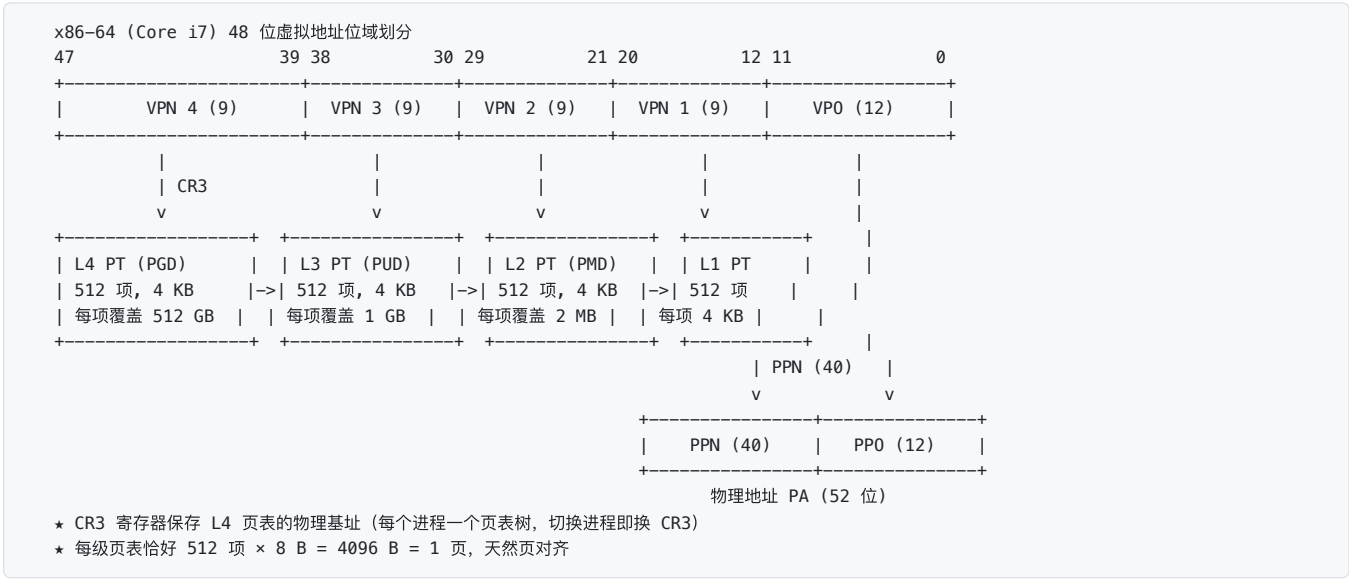
- **问题**：PTE 本身也像普通内存字一样被缓存在 L1 里，但可能被其他数据引用**驱逐**，且即便命中也要付一次小的 L1 延迟。地址翻译若每次都查页表，性能无法接受。
- **解法**：**翻译后备缓冲器（Translation Lookaside Buffer, TLB）**——MMU 内的小型**组相联硬件缓存**，把 VPN 映射到 PPN，存放少量页的**完整 PTE**。
- **位域划分**：MMU 用 VPN 访问 TLB，VPN 再拆成 **TLB 标记（TLBT）**与 **TLB 索引（TLBI）**，共 $T = 2^t$ 个组，TLBI 选组、TLBT 匹配组内行的标记。



- **TLB 与 k 级页表的关系**：无论页表有多少级，**TLB 缓存的始终是完整的 VPN → PPN 映射**（这是讲义强调的关键结论）。这让多级页表的层层查找只在 TLB 未命中时才发生。

11.2.9 多级页表与 x86-64 的 4 级页表（Multi-Level Page Tables）

- **为什么需要多级**：设 4 KB（2^{12}）页大小、48 位地址空间、8 字节 PTE，则单级页表需要 $2^{48} \times 2^{-12} \times 2^{3} = 2^{39}$ 字节 = 512 GB——每进程一张，显然不可行。
- **解决办法：多级页表**。第 1 级表的每个 PTE 指向一张页表（**始终常驻内存**），第 2 级表的每个 PTE 指向一个页（**像普通数据一样换入换出**）。关键是让**未分配的虚拟页不必存在对应的下级页表**：32 位示例中，2K 已分配页 + 6K 未分配 + 1023 未分配 + 1 页栈，两级页表只需约 3 张 4 KB 页表，而非 4 MiB 的单级表。
- **x86-64 的 4 级页表**：48 位虚拟地址划分为 **9 + 9 + 9 + 9 + 12**，VPN 共 36 位。



- **Core i7 的真实配置**：4 级页表、**每进程一个页表树**；MMU 中有 **L1 d-TLB 64 项 4 路**（指令侧 **L1 i-TLB 128 项 4 路**）与**统一的 L2 TLB 512 项 4 路**；L1 d-TLB 的 64 项/4 路 = 16 组，故 **TLBI 占 4 位、TLBT 占 36 − 4 = 32 位**。页命中时，TLB 与 L1 缓存可并行工作（决定 L1 组索引的位在 VA 与 PA 中相同），进一步掩盖翻译延迟。

11.3 代码示例与底层机制分析

11.3.1 示例 1：观察进程的虚拟地址空间布局

代码 (C)：`/tmp/vm_layout.c`

```c
/* vm_layout.c — 展示进程的虚拟地址空间布局：段、堆、栈、mmap 区 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int  g_init  = 42;                    /* .data  : 已初始化的全局变量 */
int  g_bss;                           /* .bss   : 未初始化全局变量（不占文件） */
static const char *g_ro = "rodata";   /* .rodata: 只读数据 */

static void dump_maps(int n) {
    FILE *f = fopen("/proc/self/maps", "r");
    char line[512];
    if (!f) { perror("fopen"); return; }
    printf("\n=== /proc/self/maps 前 %d 行 ===\n", n);
    while (n-- > 0 && fgets(line, sizeof line, f))
        fputs(line, stdout);
    fclose(f);
}

int main(void) {
    int  local = 7;                    /* 栈上的局部变量 */
    int *heap  = malloc(64);           /* 小对象：走 brk 堆 */
    int *big   = malloc(1 << 20);      /* 1 MiB：glibc 改用 mmap 分配 */
    static int s_bss;                  /* .bss */
    const size_t PS = (size_t)sysconf(_SC_PAGESIZE);

    if (!heap || !big) return 1;
    *heap = 0xabc;
    big[0] = 0xdef;

    printf("=== 各类对象所在段的地址 ===\n");
    printf("main()            = %p   (.text)\n", (void *)main);
    printf("字符串字面量      = %p   (.rodata)\n", (void *)g_ro);
    printf("&g_init           = %p   (.data)\n", (void *)&g_init);
    printf("&g_bss / &s_bss   = %p / %p   (.bss)\n", (void *)&g_bss, (void *)&s_bss);
    printf("heap (malloc 64)  = %p   (堆)\n", (void *)heap);
    printf("big  (malloc 1MB) = %p   (mmap 区)\n", (void *)big);
    printf("&local (&heap)    = %p / %p   (栈)\n", (void *)&local, (void *)&heap);

    printf("\n=== 常量 ===\n");
    printf("PAGE_SIZE = %zu 字节 = 2^%d\n", PS, (int)__builtin_ctzl(PS));
    printf("sizeof(void*) = %zu, sizeof(long) = %zu\n",
           sizeof(void *), sizeof(long));

    /* 同一页内的两个地址共享同一个 VPN，页偏移不同 */
    unsigned long base = (unsigned long)&g_init & ~0xfffUL;
    printf("\n=== 页对齐：页的划分与地址翻译的位域 ===\n");
    printf("%#lx:  VPN = %lu, VPO = %lu\n", base,      base >> 12,      base & 0xfff);
    printf("%#lx:  VPN = %lu, VPO = %lu   <- 同一页\n",
           base + 0xfff, (base + 0xfff) >> 12, (base + 0xfff) & 0xfff);
    printf("%#lx:  VPN = %lu, VPO = %lu   <- 下一页\n",
           base + 0x1000, (base + 0x1000) >> 12, (base + 0x1000) & 0xfff);

    dump_maps(14);
    free(heap); free(big);
    return 0;
}
```

编译与运行： `gcc -g -Wall -std=c11 vm_layout.c -o vm_layout && ./vm_layout`

【实测输出】

```
=== 各类对象所在段的地址 ===
main()            = 0x40125d   (.text)
字符串字面量      = 0x402008   (.rodata)
&g_init           = 0x404078   (.data)
&g_bss / &s_bss   = 0x404094 / 0x404098   (.bss)
heap (malloc 64)  = 0x13512a0   (堆)
big  (malloc 1MB) = 0x7effab6ff010   (mmap 区)
&local (&heap)    = 0x7fffe2493634 / 0x7fffe2493628   (栈)

=== 常量 ===
PAGE_SIZE = 4096 字节 = 2^12
sizeof(void*) = 8, sizeof(long) = 8

=== 页对齐：页的划分与地址翻译的位域 ===
0x404000:  VPN = 1028, VPO = 0
0x404fff:  VPN = 1028, VPO = 4095   <- 同一页
0x405000:  VPN = 1029, VPO = 0   <- 下一页

=== /proc/self/maps 前 14 行 ===
00400000-00401000 r--p 00000000 08:03 173710                             /tmp/vm_layout
00401000-00402000 r-xp 00001000 08:03 173710                             /tmp/vm_layout
00402000-00403000 r--p 00002000 08:03 173710                             /tmp/vm_layout
00403000-00404000 r--p 00002000 08:03 173710                             /tmp/vm_layout
00404000-00405000 rw-p 00003000 08:03 173710                             /tmp/vm_layout
01351000-01372000 rw-p 00000000 00:00 0                                  [heap]
7effab6ff000-7effab800000 rw-p 00000000 00:00 0
7effab800000-7effab828000 r--p 00000000 08:05 3221225831                 /usr/lib64/libc.so.6
7effab828000-7effab99d000 r-xp 00028000 08:05 3221225831                 /usr/lib64/libc.so.6
...
```

【代码做什么？】

1. 定义位于 `.data` / `.bss` / `.rodata` 的各类对象，并在 `main` 里在栈上取局部变量、在堆上取两块内存（64 B 与 1 MiB）。
2. 打印各对象地址，立即能看出**四段地址区间**：低地址 `0x400000` 附近是可执行文件映射的段，`0x1351000` 是 `brk` 堆，`0x7eff...` 是 `mmap` 区，`0x7fff...` 是栈。
3. 用掩码 `& ~0xfffUL` 把地址按 4 KB 对齐，手工算出 VPN 与 VPO，验证**同一页内地址 VPN 相同、VPO 从 0 到 4095**。
4. 打开 `/proc/self/maps` 打印前 14 行，把上一步的"地址"与这一段"区间 + 权限 + 文件偏移"对应起来。

【底层机制透视】

`/proc/self/maps` 是内核暴露的**虚拟内存区域（VMA, Virtual Memory Area）**清单，每行格式为：

```
起始地址-结束地址   权限   文件偏移   主:次设备号   inode   文件路径
```

- **权限字段** `r--p` / `r-xp` / `rw-p` / `---p` 正对应 PTE 的 **READ / WRITE / EXEC / SUP** 位（`p` 表示 private，`s` 表示 shared，即 MAP_PRIVATE / MAP_SHARED）。
- **同一个可执行文件被映射成多行**，因为它按段拆分且每段必须页对齐：第一行 `r--p` 是 ELF 头（偏移 `00000000`），第二行 `r-xp` 是 `.text`（偏移 `00001000`），后两行是 `.rodata` 与 `.data`。
- `malloc(1MiB)` 落在 `mmap` 区而不是 `[heap]`：glibc 对超过 `M_MMAP_THRESHOLD`（默认 128 KB）的请求直接用 `mmap`，`free` 时可直接 `munmap` 归还内核。
- `[heap]` 区间是 `brk` 指针推出来的，**`malloc` 的"堆"本质上就是这段虚拟区间**——这正是 L5 Malloc Lab 要操作的对象。

【内存布局 / 数据结构图解】

```
   0x7fffe2493634  栈（向下增长）       <- r-- 到 rw- 的 [stack] 区
        ...
   0x7effab6ff010  mmap 区（1 MiB malloc / 共享库）
        ...
   0x01351000      [heap] 起始（brk）    <- malloc 的战场（L5）
        ...
   0x00404000      .data / .bss  (rw-p)
   0x00402000      .rodata       (r--p)
   0x00401000      .text         (r-xp)
   0x00400000      加载基址：链接器写死，每个进程都一样 → 简化链接
```

【与汇编 / 硬件的对应】：地址只是数值，翻译发生在访存时。用 `objdump -d -M intel` 可看到 `main` 里对字符串字面量的访问是 `lea rax, [rip+0x...]`——**RIP 相对寻址**，因为链接器假定地址是"相对的、可预测的"，这也是虚拟地址空间统一布局带来的便利。

11.3.2 示例 2：用 `mmap` 观察按需分页（Demand Paging）

代码 (C)：`/tmp/vm_mmap.c`（节选核心逻辑，完整文件含文件映射部分）

```
/* vm_mmap.c — 用 mmap 映射匿名内存与文件，并观察"按需分页" */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/resource.h>

static long minflt(void) {           /* 次缺页（minor page fault）计数 */
    struct rusage ru;
    getrusage(RUSAGE_SELF, &ru);
    return ru.ru_minflt;
}

int main(void) {
    const size_t PS  = (size_t)sysconf(_SC_PAGESIZE);
    const size_t NP  = 1024;                 /* 1024 页 = 4 MiB */
    const size_t LEN = NP * PS;

    /* 1) 匿名映射：只保留地址空间，不分配物理页 */
    long f0 = minflt();
    char *p = mmap(NULL, LEN, PROT_READ | PROT_WRITE,
                   MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (p == MAP_FAILED) { perror("mmap"); return 1; }
    long f1 = minflt();
    printf("[1] mmap 之后:      p = %p, 次缺页增量 = %ld\n", (void *)p, f1 - f0);

    /* 2) 只写每页第一个字节：触发按需分配 */
    for (size_t i = 0; i < NP; i++) p[i * PS] = (char)i;
    long f2 = minflt();
    printf("[2] 触碰 %zu 页后:  次缺页增量 = %ld\n", NP, f2 - f1);

    /* 3) 再顺序读一遍：应当不再产生缺页 */
    long sum = 0;
    for (size_t i = 0; i < NP; i++) sum += p[i * PS];
    long f3 = minflt();
    printf("[3] 再读一遍后:     次缺页增量 = %ld, checksum = %ld\n", f3 - f2, sum);

    /* 4) madvise 归还物理页（MADV_DONTNEED）：VMA 保留、物理页回收 */
    if (madvise(p, LEN, MADV_DONTNEED) == 0) puts("[4] MADV_DONTNEED 成功");

    /* 5) 文件映射（完整版含此段，见磁盘写回验证） */
    munmap(p, LEN);
    return 0;
}
```

编译与运行：`gcc -g -Wall -std=c11 vm_mmap.c -o vm_mmap && ./vm_mmap`

【实测输出】

```
PAGE_SIZE = 4096, 映射长度 = 4194304 字节 (1024 页)

[1] mmap 之后:      p = 0x7f977fa00000, 次缺页增量 = 0  <- 尚未触碰任何页
[2] 触碰 1024 页后:  次缺页增量 = 1024  <- 每页一次缺页
[3] 再读一遍后:     次缺页增量 = 0  <- 已驻留，全部页命中
    checksum = -512
[4] MADV_DONTNEED 成功：物理页被回收，VMA 保留

[5] 文件映射 /tmp/vm_mmap_demo.bin: q = 0x7f977f600000
    mmap 后缺页增量 = 0  (写第 0 页 +0, 写第 1 页 +1)
    回读文件 offset 0 = 'A', offset 4096 = 'B'  <- 已写回磁盘
```

【代码做什么？】① 申请 4 MiB 匿名映射；② 每页只写 1 字节，制造 1024 次缺页；③ 再读一遍，确认零缺页；④ 用 `madvise(MADV_DONTNEED)` 把物理页交还内核；⑤ 把 4 MiB 的真实文件映射进地址空间，只改两页并 `msync`，再从文件里读回验证落盘。

【底层机制透视】

- **第 [1] 步的"零缺页"是全篇最重要的实测证据**：`mmap` 只创建 VMA 与页表结构，**不分配任何物理页**。这正是 11.2.7 的 demand paging——内核把页的换入推迟到第一次访问。
- **第 [2] 步恰好 1024 次缺页**：每页第一次触碰触发一次**次缺页（minor fault）**，内核填一个零页（或从 page cache 取页）并建立 PTE。粒度严格是"页"，因为映射的最小单位就是页。若触碰同一页的多个字节，只会有 1 次缺页——**局部性直接转化为缺页次数**。
- **第 [3] 步的零缺页**对应页表里的**有效位已经是 1**：此后每次访问都是"页命中"，由 MMU 用 TLB/页表直接翻译，不再陷入内核。
- **第 [5] 步的文件映射**：`MAP_SHARED` 让写入通过 **page cache** 反映到文件；`msync` 强制写回（对应 VM 缓存的 **write-back** 策略）。注意写入第 0 页时缺页增量显示为 `+0`，因为第 0 页在此之前已被写的路径预热（页表已建立），只有第 1 页产生了 **1 次次缺页**——这说明**缺页出现在"页第一次被触碰"的时刻，而不是每个字节**。

【内存布局 / 数据结构图解】

```
mmap(NULL, 4 MiB, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
     VA 0x7f977fa00000
     +------------------+  VMA 已建立，PTE = 无效
     | 页 0  (未驻留)   |  <- 仅存在于虚拟地址空间
     | 页 1  (未驻留)   |     物理内存占用 = 0
     | ...              |
     | 页 1023(未驻留)  |
     +------------------+
触碰每页首字节后：
     页 0 -> PP x, 页 1 -> PP y, ...  PTE 有效位 = 1（对应 1024 次次缺页）

strace -e trace=mmap,madvise ./vm_mmap 的真实系统调用：
mmap(NULL, 4194304, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f0451c00000
madvise(0x7f0451c00000, 4194304, MADV_DONTNEED)                              = 0
mmap(NULL, 4194304, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0)                  = 0x7f0451800000
★ 注意：整个 4 MiB 只有 3 个系统调用，缺页全在内核里"悄悄"发生
```

【与汇编 / 硬件的对应】：调用点由 PLT 跳转完成：

```
401292:  call   401060 <mmap@plt>       # 4 MiB 匿名映射
4013c7:  call   4010b0 <madvise@plt>    # 归还物理页
```

`mmap` 的六个参数依次放在 `%rdi,%rsi,%rdx,%rcx,%r8,%r9`，与 x86-64 调用约定一致；系统调用号写入 `%rax` 后执行 `syscall`。

11.3.3 示例 3：权限位与段错误

代码 (C)：`/tmp/vm_prot.c`

```c
/* vm_prot.c — 保护演示：对只读页写入 -> SIGSEGV（段错误） */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

static void handler(int sig) {
    /* 注意：write() 是异步信号安全的，printf() 不是 */
    const char *m = "\n[信号处理程序] 捕获到 SIGSEGV：MMU 的权限检查失败！\n";
    ssize_t r = write(STDOUT_FILENO, m, strlen(m));
    (void)r;
    _exit(128 + sig);          /* 打印后直接退出，不再返回故障指令 */
}

int main(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    if (sigaction(SIGSEGV, &sa, NULL) != 0) { perror("sigaction"); return 1; }

    char *ro = (char *)"rodata-literal";   /* 位于只读段 */
    printf("ro 指向 %p，内容 = \"%s\"\n", (void *)ro, ro);
    printf("尝试写入只读页 ...\n");
    fflush(stdout);

    ro[0] = 'X';               /* ⚠ 仅供演示：写只读段 -> SIGSEGV */
    printf("这行永远不会执行\n");
    return 0;
}
```

编译与运行：`gcc -g -Wall -std=c11 vm_prot.c -o vm_prot && ./vm_prot; echo "exit=$?"`

【实测输出】

```
ro 指向 0x40205a，内容 = "rodata-literal"
尝试写入只读页 ...

[信号处理程序] 捕获到 SIGSEGV：MMU 的权限检查失败！
exit=139
```

【底层机制透视】：地址 `0x40205a` 落在 `vm_layout` 示例中 `.rodata` 所在的 `00402000-00403000 r--p` 区间。该区间对应的 PTE 里 **WRITE 位为 0**，MMU 在写访问时检测到违规，抛出异常；内核识别出这是**用户态越权访问已映射页**，于是向进程递送 `SIGSEGV`（退出码 139 = 128 + 11）。**"段错误"从来不是"内存不存在"，而是"这次访问不满足 PTE 里的权限位"**——同样的地址读没问题，写才出错，恰恰证明了保护是**按访问类型逐次检查**的。

【与汇编 / 硬件的对应】：`gcc -O0 -S` 得到真实的存储指令（AT&T 语法）：

```
    movq    $.LC2, -8(%rbp)      # ro = "rodata-literal"
    movq    -8(%rbp), %rax
    movb    $88, (%rax)          # ro[0] = 'X'  -> 88 是 'X' 的 ASCII 码
```

`movb $88, (%rax)` 是一条普通的存储指令：**CPU 并不知道页是只读的**，拒绝发生在 MMU 翻译 `%rax` 时。这解释了为什么编译器无法在编译期发现这个错误（若用 `const char *` 则编译期会警告，但字符串字面量的类型是 `char[]`）。

11.4 实验关联

L5 Malloc Lab 的全部操作对象就是本讲定义的虚拟地址空间中的 **`[heap]` 区间**：

1. **堆不是"内存"，而是一段虚拟地址区间**。`malloc` 内部通过 `sbrk` / `brk` 或 `mmap` 向内核申请虚拟页；**这些页在被首次触碰前并不驻留物理内存**。所以"申请 1 MB"其实只是把 `brk` 指针往上推 1 MB，代价几乎为零。
2. **对齐要求来自页与字长**：块的对齐通常是 8 或 16 字节；"页对齐"（`PAGE_SIZE = 4096`）只在你需要把映射边界做到页粒度时才重要。用 `getconf PAGE_SIZE` 确认本机是 4096。
3. **`mmap` 与 `sbrk` 两条路**：大块分配走 `mmap` 可以整块 `munmap` 归还，物理页立即回收；小分配走 `brk` 堆则只能通过"空闲链表合并"复用。这正是 L5b 讨论"显式/隐式空闲链表 vs 分离存储"的物理依据。
4. **典型坑**：在 `mmap` 得到的区间外访问会得到 `SIGSEGV` 而不是"分配到垃圾数据"；堆里的野指针若落在未映射的页上同样崩溃，但若落在已映射的页上则**静默破坏数据**——后者才是 malloc lab 里最难查的 bug。
5. **调试利器**：`valgrind ./mdriver`、`gdb` 里 `info proc mappings` 与 `x/16gx $rsp`，以及 `/proc/<pid>/maps` 对照堆区间边界。

本讲也为 Lecture 12（`fork` 的 COW、`execve` 的内存映射、TLB 与缓存联合工作）和 Lecture 16（进程）提供理论前提。

11.5 常见错误与调试技巧

- **把虚拟地址当物理地址**：以为 `printf("%p", p)` 打印的是内存条上的位置。**调试**：`cat /proc/self/maps`、`gdb -p <pid>` 后 `info proc mappings` 看 VA 区间；用 `/proc/<pid>/pagemap`（需 root）查实际 PPN。
- **以为"指针有效"就等于"页已驻留"**：`malloc` 返回非空不代表物理内存已分配。**调试**：`getrusage(RUSAGE_SELF, &ru)` 观察 `ru_minflt` / `ru_majflt` 增量；见 11.3.2 的实测（`mmap` 后缺页增量为 0）。
- **对字符串字面量写入**：`char *s = "abc"; s[0]='x';` 编译通过、运行 `SIGSEGV`。**调试**：`gcc -Wall -Wwrite-strings`，或声明为 `const char *s`；用 `valgrind` 会直接报告 "Invalid write of size 1"。
- **越界访问落在已映射页内**：不崩溃但破坏数据，是 malloc lab 最阴险的 bug。**调试**：`gcc -fsanitize=address -g`、`valgrind --leak-check=full --track-origins=yes`。
- **误判"内存泄漏"为"缺页太多"**：`top` 里的 RSS 高可能只是工作集大，未必是泄漏。**调试**：`/usr/bin/time -v`（若可用）或 `getrusage` 看 `ru_maxrss`；`valgrind --leak-check=full` 定位未释放块。
- **认为 TLB 未命中会崩溃**：TLB 未命中只是**多一次内存访问**去查页表，与缺页（需要磁盘 I/O）是完全不同的量级。**调试**：`perf stat -e dTLB-loads,dTLB-load-misses,page-faults ./prog`。
- **混淆 `VPO` 与 `VPN` 的对齐掩码**：写代码时把 `& 0xfff`（页内偏移）误用作 `>> 12`（页号）。**调试**：用 11.3.1 的打印逐位验证；注意 $P = 2^p$ 时掩码是 `(1<<p)-1`，页号是 `>> p`。
- **`munmap` 后继续使用指针**：地址空间被撤销，访问触发 `SIGSEGV`。**调试**：`gdb` 中 `x/4xb ptr`，错误信息会给出故障地址；`/proc/self/maps` 确认该地址已不在任何区间内。

11.6 关键要点

- **虚拟内存的本质是一层"地址间接"（indirection）**：CPU 只发出 VA，MMU 用页表把它翻译成 PA；程序无法观测、也无法绕过这层翻译，所有内存保护、隔离、共享都建立在此之上。
- **页是翻译与缓存的最小单位，$P = 2^p$（典型 4 KB）**；翻译的**核心不变量是 $\mathbf{VPO = PPO}$**——页内偏移在翻译前后一字不变，因此翻译只需把 VPN 换成 PPN。
- **VM = 缓存 + 管理 + 保护三位一体**：作为缓存（页表有效位 = 命中/缺页）、作为管理工具（每进程独立地址空间、共享物理页、COW、`mmap`）、作为保护工具（PTE 权限位 + `SIGSEGV`）。
- **DRAM 缓存是全相联 + 大块 + 写回，替换策略由操作系统软件实现**——缺失代价太大（磁盘比 DRAM 慢约 10,000 倍），硬件做不了；而它能工作**完全依赖局部性**，工作集超过主存就会抖动。
- **TLB 是关键的性能设施**：它缓存完整的 VPN → PPN 映射（与页表级数无关），命中省掉一次内存访问，未命中才走页表；TLB 未命中之所以罕见，是因为活跃页数远小于 TLB 项数。
- **x86-64 用 4 级页表把 48 位 VA 划成 9/9/9/9/12**：单级页表需 512 GB，多级页表让未分配区域**不必存在下级页表**；CR3 指向 L4 页表（PGD），每进程一棵页表树，切换进程即换 CR3。

11.7 思考题（带答案）

题 1（计算题：地址翻译手算） 某系统虚拟地址 14 位（$n = 14$）、页大小 4 KB（$p = 12$）、物理地址 14 位（$m = 14$）。页表内容为：PTE0 = 有效、PPN = 2；PTE1 = **无效**（页在磁盘上）；PTE2 = 有效、PPN = 5；PTE3 = 有效、PPN = 7。请把虚拟地址 `0x3A5C`、`0x1A5C`、`0x30A4` 翻译成物理地址，并指出哪些会缺页。

答：先算位域。$n - p = 14 - 12 = 2$，故 VPN 占高 2 位、VPO 占低 12 位（= 3 个十六进制位）。

VA	二进制	VPN	VPO	PTE	结果
`0x3A5C`	`11 1010 0101 1100`	3（`0x3`）	`0xA5C`	PPN = 7	PA = **`0x7A5C`**
`0x1A5C`	`01 1010 0101 1100`	1（`0x1`）	`0xA5C`	无效	**缺页**（需换入并更新 PTE 后重执行）
`0x30A4`	`11 0000 1010 0100`	3（`0x3`）	`0x0A4`	PPN = 7	PA = **`0x70A4`**

验证：`0x3A5C >> 12 = 0x3`，`0x3A5C & 0xFFF = 0xA5C`；PA = `(7 << 12) | 0xA5C = 0x7000 + 0xA5C = 0x7A5C`。注意 `0x3A5C` 与 `0x30A4` 的 VPN 相同（同页）、VPO 不同，因此 PPN 相同——这就是"$\mathbf{VPO = PPO}$"的直接体现。`0x1A5C` 与 `0x3A5C` 的 VPO 一模一样，仅 VPN 差 1，却一个命中一个缺页，说明**缺页只与页号有关，与页内偏移无关**。

题 2（计算题：页表大小与页数） (a) 在 48 位虚拟地址、4 KB 页、8 字节 PTE 的机器上，单级页表需要多大？(b) x86-64 的 4 级页表中，每一级页表有多少项、多大？(c) 若一个进程只使用了 `.text` / `.data` /堆/栈/一个共享库（5 个区域），说明多级页表为什么远小于单级页表。

答：(a) 虚拟页数 $= 2^{48}/2^{12} = 2^{36}$，每项 8 字节，故 $2^{36} \times 8 = 2^{39}$ 字节 = **512 GiB**（讲义给出的正是这个数字）。(b) 每级页表的索引字段 9 位，故 **512 项 × 8 B = 4096 B = 恰好 1 页**，天然页对齐。(c) 这 5 个区域分散在地址空间各处，4 级页表只需为**被触碰到的路径**分配下级页表：至多

$5 \times 4 = 20$ 张页表 ≈ 80 KB，而非 512 GiB。**关键洞察：多级页表把"为整个地址空间预留"变成"只为你真正用到的地址付费"**，代价是 TLB 未命中时要走 4 次访存。

题 3（计算题：TLB 位域与命中率） Core i7 的 L1 d-TLB 为 64 项、4 路组相联，VPN 共 36 位。求组数、TLBI 位数与 TLBT 位数。若某循环每轮访问 8 个不同的 4 KB 页共 10000 轮，在理想替换下 TLB 命中率是多少？若改为访问 32 个不同页呢？

答：组数 $= 64/4 = 16$，故 **TLBI = 4 位，TLBT = 36 − 4 = 32 位**。访问 8 个页：64 项 TLB 足以全部容纳，除**强制缺失（compulsory miss）**的 8 次外全部命中，命中率 $= (80000 - 8)/80000 = 99.99\%$。访问 32 个页：仍全部容纳（32 < 64），命中率同样接近 100%。若工作集涨到远超 64 项，则每轮都要重新换入换出，命中率急剧下降——**这正是"工作集小于 TLB 容量则性能良好"的微观版本**，与 11.2.7 的工作集/抖动是同一规律在两个层级上的体现。

题 4（辨析题：直观但错误的想法） 有同学说："`p = malloc(1000)` 之后，`*p` 一定不会出错，因为内存已经被分配了；如果程序在 `*p` 处崩溃，那说明 `malloc` 有 bug。"这个想法错在哪？另有一位同学说："两个进程共享同一个物理页 PP6，所以它们对 PP6 的权限必然相同。"这又错在哪？

答：第一句混淆了**虚拟地址空间分配**与**物理页驻留**。`malloc` 只保证那段 VA 区间在你的堆里（VMA 与页表项被创建，但 PTE 的有效位可能还是 0）；`*p` 引发的**缺页是正常流程**，由内核换入后重新执行指令，不会崩溃。真正会崩溃的情形是：`p` 已被 `free` 且堆被 `munmap` /收缩后越界访问、或者 `p` 指向了从未映射的地址——那是**页表里根本没有有效映射**或**权限位不满足**，与 `malloc` 实现无关。第二句混淆了**映射**与**权限**：PTE 的 PPN 与权限位是彼此独立的字段。前面 11.2.6 的例子中，进程 i 与 j 都映射了 PP6，但两者可以有不同（甚至相反）的 READ/WRITE/SUP/EXEC 组合——这正是"共享只读库代码"能安全实现的原理：多个进程共享同一物理页，却各自只被授予读权限（`r-xp` / `r--p`）。

Lecture 12: 虚拟内存：细节 (Virtual Memory: Details)

> **讲义对应**：CMU 15-213 Lecture 12 — Virtual Memory: Details（素材：`F25-12-vm-details.txt`） **教材对应**：CS:APP3e 第 9 章 9.7—9.8（地址翻译实例；Linux 虚拟内存系统） **关联 Lab**：**L5 Malloc Lab**（另可参考 recitation：`F25-rec08_slides.txt`）

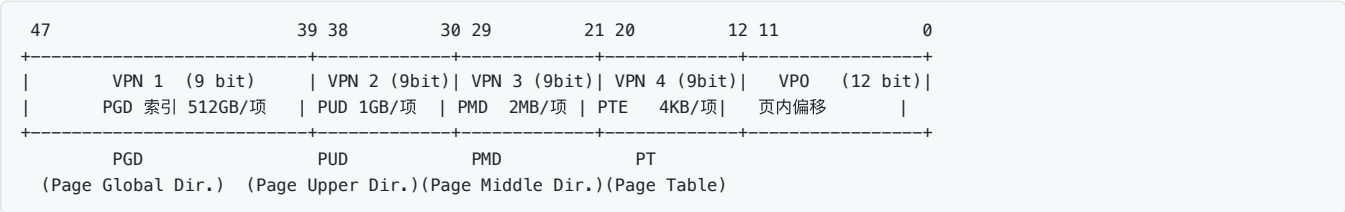
12.1 概述

第 11 讲建立了两条抽象：虚拟地址空间（Virtual Address Space）与页表（page table）。本讲把抽象拧到底层，回答一个问题：**CPU 拿着一个 48 位虚拟地址，硬件究竟做了什么才得到物理地址？** 我们先走查 Intel Core i7 的四级页表与 TLB，给出一个可手算的端到端数值实例；再从硬件回到操作系统，看 Linux 如何用虚拟内存区域（VM area, VMA）描述地址空间、缺页处理程序（page fault handler）如何判定"合法访问 / 保护异常 / 段错误"，以及 `mmap`、写时复制（copy-on-write）、`execve`、堆栈按需增长这些机制。这是 **L5 Malloc Lab** 的直接理论前提：只有知道 `sbrk` 与 `mmap` 各自把堆伸到哪里，才能理解分配器为什么必须"不关心"应用如何使用内存。

12.2 核心概念与底层机制图解

12.2.1 四级页表与虚拟地址的位域划分（Four-Level Page Table）

- **定义与目的**：单级页表要求"为每个虚拟页放一个 PTE"。48 位 VA、4 KB 页、8 B PTE 下页表项数 $= 2^{48}/2^{12} = 2^{36}$，总大小 $2^{36} \times 8 = 2^{39}$ 字节 $= 512$ GB——比多数机器的内存还大。多级页表**用树换空间**：只有被使用的子树才分配。
- **直观解释**：页表像多卷百科全书。单级页表相当于给每个可能页码都印卡片；四级页表像"总目录 → 卷目录 → 章目录 → 页码表"，第 254 卷不存在则后面三级一字不印。
- **底层机制图解**（Core i7 的 VPN 划分：36 位 VPN 拆成 4 个 9 位索引）：



- **与机器码/硬件的对应**：走查起点是控制寄存器 CR3，保存**一级页表（PGD）物理基址**。为何必须是物理地址？因为 MMU 要用它访存取 PTE，若还依赖翻译就陷入无穷递归。每级 PTE 的 bit 12—51 给出下一级页表**物理基址**（40 位，强制 4 KB 对齐），bit 0 是存在位 `P`。

12.2.2 Core i7 端到端地址翻译（End-to-End Address Translation）

- **定义与目的**：Core i7 的内存系统：L1 d-cache 32 KB/8 路、L1 i-cache 32 KB/8 路、L2 统一 256 KB/8 路、L3 统一 8 MB/16 路（多核共享）；TLB 分两级：L1 d-TLB 64 项 4 路（16 组）、L1 i-TLB 128 项 4 路、L2 统一 TLB 512 项 4 路。**MMU 夹在 CPU 和 L1 之间**，它拿虚拟地址、查 TLB/页表、吐出物理地址。
- **直观解释**：TLB 像贴在显示器边上的便利贴——最近用过的"页码 → 物理框号"抄一小条在旁边，绝大多数访问根本不用去翻那本四卷目录。
- **底层机制图解**：

```
  CPU 产生 VA
       |
       v
+------------------+   命中(约 99%+)     +-----------------------------+
|  L1 d-TLB        |-------------------->| PPN 直接可用，跳过 page walk|
| 64项/4路 => 16组 |                     +-----------------------------+
+------------------+
       | 未命中
       v
+------------------+   +--------+   +--------+   +--------+   +--------+
|  L2 统一 TLB     |-->| PGD    |-->| PUD    |-->| PMD    |-->| PT     |
| 512项/4路        |   |[VPN1] |   |[VPN2] |   |[VPN3] |   |[VPN4] |--> PPN
+------------------+   +--------+   +--------+   +--------+   +--------+
                         ^CR3         ^PPN         ^PPN         ^PPN
                         4 次访存，无空间局部性 —— 这就是 page walk 的代价
       |
       v
 PA = PPN:VPO  ---> L1 d-cache (64组/8路, 块64B) ---> 命中则出数据
```

- **与机器码/硬件的对应**：TLB 字段：`TLBI`（组索引）= VPN 低 4 位（16 组），`TLBT` = VPN 高 28 位；PPO 与 VPO 相同。L1 的 `CI` = PA bit 6–11（64 组），`CO` = bit 0–5（64 B 块），`CT` = 余 28 位。**"虚拟索引、物理标记"（virtually indexed, physically tagged）**：`CI` 来自页内偏移，翻译前后不变，所以 L1 可一边查 TLB 一边用 `CI` 选组，PPN 一到只比标记，省下翻译延迟。

12.2.3 一个完整的数值走查：48 位 VA → 40 位 PA

这是必须手算一遍的例子。设 `CR3 = 0x0000000000A2F000`（PGD 物理基址），虚拟地址

$$\mathbf{VA = 0x00007F4A2B8C9A48}$$

第一步：逐位拆分（9/9/9/9/12）

```
VA   = 0x00007F4A2B8C9A48
二进制 = 0111 1111 0100 1010 0010 1011 1000 1100 1001 1010 0100 1000
         \_______/\_______/ \_______/ \_______/ \_________/
          VPN1      VPN2      VPN3      VPN4        VPO
         011111110 100101000 101011100 011001001  101001001000
         = 0x0FE   = 0x128   = 0x15C   = 0x0C9    = 0xA48
         = 254     = 296     = 348     = 201      = 2632

VPN  = VA >> 12 = 0x7F4A2B8C9
TLBT = VPN >> 4 = 0x7F4A2B8C   (28 bit 标记)
TLBI = VPN & 0xF = 0x9         (4 bit 组索引, L1 d-TLB 16 组)
```

第二步：逐级查表（每级条目地址 = 该级页表物理基址 + 索引 ×8）

级别	页表	基址	索引	条目地址	条目内容	取出的下一级物理基址
L1	PGD	`0xA2F000`	`0x0FE` =254	`0xA2F000` + `0x7F0` = `0xA2F7F0`	`0x0000000000A3B067`	`0xA3B000`
L2	PUD	`0xA3B000`	`0x128` =296	`0xA3B000` + `0x940` = `0xA3B940`	`0x000000000B7C067`	`0xB7C000`
L3	PMD	`0xB7C000`	`0x15C` =348	`0xB7C000` + `0xAE0` = `0xB7CAE0`	`0x000000000C4E067`	`0xC4E000`
L4	PT	`0xC4E000`	`0x0C9` =201	`0xC4E000` + `0x648` = `0xC4E648`	`0x000000000A3C067`	PPN = `0xA3C`

第三步：拼出物理地址

$$\mathbf{PA = (PPN \ll 12) \mid VPO = (0xA3C \ll 12) \mid 0xA48 = 0xA3C000 + 0xA48 = 0x0000A3CA48}$$

第四步：送入 L1 d-cache（CI 位在翻译前后不变）

```
PA = 0x0000A3CA48 -> 40 bit: 0000 0000 0000 0000 1010 0011 1100 1010 0100 1000
                            \_________________ CT (28 bit) _________________/\__ CI __/\_ CO _/
CO (bit 0-5)  = 0x08 = 8      -> 块内第 8 字节
CI (bit 6-11) = 0x29 = 41     -> 第 41 组 (共 64 组)
CT (bit 12-39)= 0x0000A3C     -> 与组内 8 路的标记比较
```

整个 page walk 需 **4 次额外访存**（PTE 本身也可能 miss），且这 4 次之间**没有空间局部性**——这正是多级页表比单级慢的原因，也是 TLB 存在的理由。TLB 命中时第二步全跳过，直接得到 PPN。

12.2.4 页表项的低 12 位：权限位（Permission Bits）

- **定义与目的**：PTE 的 bit 12–51 是物理基址，**低 12 位全是标志位**——这也是"页表必须 4 KB 对齐"的根本原因：只有对齐，低 12 位才空得出来当标志。
- **底层机制图解**：

```
 bit:  63 62      52 51                          12 11 9 8 7 6 5 4 3 2 1 0
      +-----+--------+------------------------------+-----+-+-+-+-+-+-+-+-+
L1–L3 | XD  | Unused |  下一级页表物理基址 (40 bit)   | AVAIL|G|PS|A|CD|WT|U/S|R/W|P|
      +-----+--------+------------------------------+-----+-+-+-+-+-+-+-+-+
L4    | XD  | Unused |  物理页基址 (40 bit)           | AVAIL|G|D |A|CD|WT|U/S|R/W|P|
      +-----+--------+------------------------------+-----+-+-+-+-+-+-+-+-+
对照条目 0x...067 = 0000 0000 0110 0111b:
        bit0 P=1   页/子页表在物理内存    bit5 A=1  引用位(MMU 置, 软件清)
        bit1 R/W=1 可写(对所有可达页)      bit6 D=1  脏位(仅 L4; COW 靠它)
        bit2 U/S=1 允许用户态(0=仅内核)    bit7 PS=0 仅 L1: 0=4KB 子表, 1=4MB 大页
        bit3/4     PWT/PCD 缓存策略        bit8 G=0  全局页(切任务不逐出 TLB)
```

- **与机器码/硬件的对应**：`P=0` 时 bit 12–51 不再是物理地址，而记录**该页在磁盘（交换区）上的位置**——页表兼作磁盘地址簿。访 `P=0` 的页触发缺页；`R/W=0` 时写触发保护异常。

12.2.5 Linux 的虚拟内存区域（VM Area / VMA）

- **定义与目的**：一个进程的虚拟地址空间不是"一整块"，而是一串**同质的区间**。Linux 用 `vm_area_struct` 描述其中一个区间，用 `mm_struct` 描述整个地址空间。
- **直观解释**：VMA 像房产证的附件清单——"0x400000–0x401000：可读，来自 a.out 偏移 0"，而非逐页登记。相邻且权限相同的页被合并成一个 VMA，所以 `/proc/pid/maps` 通常只有几十行。
- **底层机制图解**（一个典型 x86-64 Linux 进程的 VMA 布局）：

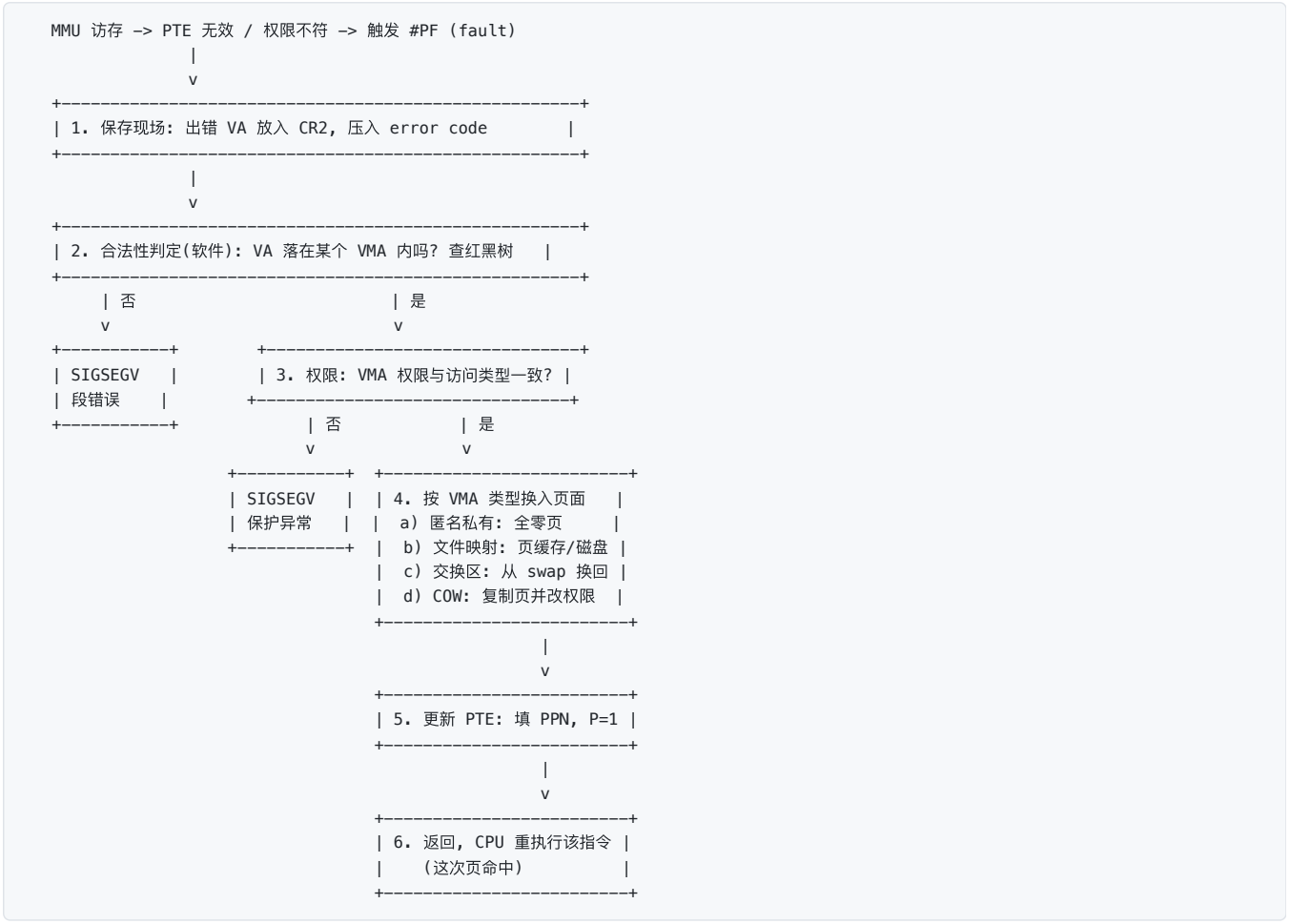
```
 高地址 0xffffffffffffffff
       +--------------------------------------+
       |  内核虚拟内存（用户不可见）              |
0xffff800000000000
       +--------------------------------------+
       |  [vsyscall]  --xp   遗留固定映射        |
       |  [vvar]      r--p   只读内核数据        |
       |  [vdso]      r-xp   内核"共享库"        |
       |  [stack]     rw-p   %rsp 附近, 向下增长 |
       |    ...  未映射空洞（不可访问）  ...     |
 0x7f....  +-------------------------------------+
       |  mmap 匿名区 rw-p   malloc 大块         |
       |  libc.so.6   r-xp/r--p/rw-p  共享库     |
       |  [heap]      rw-p   brk 指针, 向上增长  |
 0x555555554000
       +--------------------------------------+
       |  a.out       r--p  .init/.plt  私有映射 |
       |              r-xp  .text  (代码)       |
       |              r--p  .rodata(只读数据)   |
       |              rw-p  .data/.bss          |
 低地址 0x400000
```

`vm_area_struct` 的核心字段：`vm_start` / `vm_end`、`vm_flags`（`VM_READ` / `VM_WRITE` / `VM_EXEC` / `VM_SHARED`）、`vm_prot`、`vm_file`、`vm_pgoff`。`mm_struct` 持有 `pgd`（一级页表物理地址，即 CR3 的内容）、`mmap`（VMA 链表头）、`start_brk/brk`、`start_stack`。Linux 还用红黑树按地址索引 VMA，使"该地址落在哪个 VMA 里"能在 $O(\log n)$ 内回答。

- **与机器码/硬件的对应**：切换进程时内核把 `mm_struct.pgd` 写进 CR3——这就是"每个进程有自己的地址空间"的硬件实现。VMA 是**纯软件结构**，MMU 完全不知道它存在。

12.2.6 缺页处理程序的完整逻辑（Page Fault Handler）

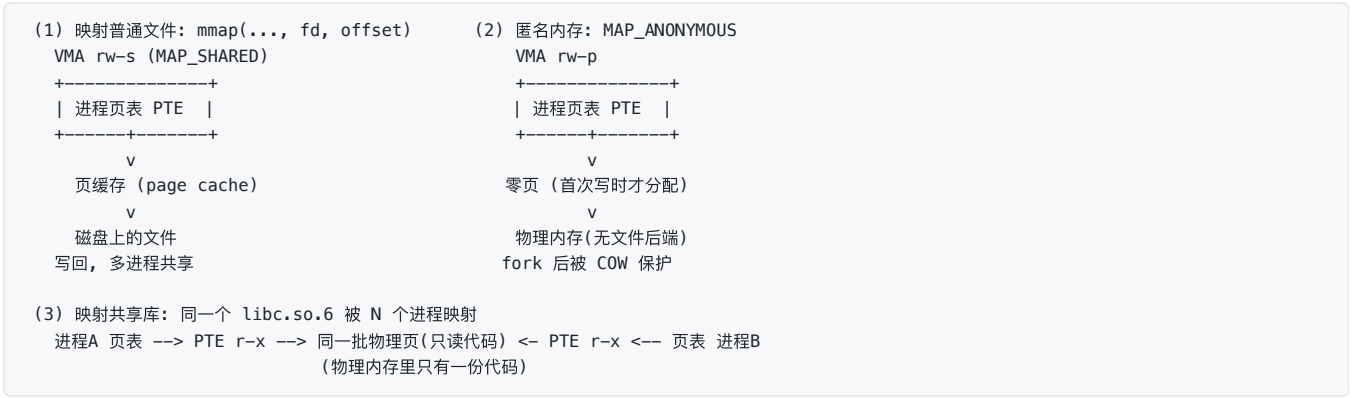
- **定义与目的**：MMU 查页表发现 `P=0` 或权限不符时触发**缺页异常**，控制权交给内核。处理程序要回答：地址合法吗？权限对吗？页从哪来？
- **底层机制图解**：



- **与机器码/硬件的对应**：x86-64 用 `CR2` 保存出错地址，压栈的 error code 区分"页不存在/保护违规、读/写、用户/内核"。`do_page_fault()` 取 `mmap_lock`，用 `find_vma()` 找 VMA，再调 `handle_mm_fault()`。**硬缺页（major fault）**须从磁盘读；**软缺页（minor fault）**页仍在内存（COW 或 page cache 命中）；`dmesg` 的 `segfault at ... error 4` 即 error code 十六进制。

12.2.7 内存映射（Memory Mapping, `mmap` / `munmap`）

- **定义与目的**：`mmap` 把**文件对象**（普通文件、匿名内存、共享库、设备）的区间映射进虚拟地址空间成为一段 VMA，把"磁盘 → 内核缓冲区 → 用户缓冲区"的两次拷贝变成用户直读页缓存的零次拷贝。
- **直观解释**：读文件本像"把字抄到笔记本再读"；`mmap` 是"把书页摊在桌上"——翻到哪页，操作系统才把哪页给你（按需分页，demand paging）。
- **底层机制图解**（三种用途对应三种 VMA）：



- **与机器码/硬件的对应**：`mmap` 系列系统调用：

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
int   munmap(void *addr, size_t length);
int   mprotect(void *addr, size_t len, int prot);
int   msync(void *addr, size_t len, int flags);
```

`prot` 取 `PROT_READ/PROT_WRITE/PROT_EXEC/PROT_NONE`；`flags` 必须含 `MAP_SHARED` 或 `MAP_PRIVATE` 之一，可或上 `MAP_ANONYMOUS`（忽略 `fd`）、`MAP_FIXED`（要求精确地址，危险）、`MAP_POPULATE`（预先驻留）。`offset` 必须是页大小的整数倍。返回值是页对齐的地址；失败返回 `MAP_FAILED`（即 `(void*)-1`）并置 `errno`。

12.2.8 `fork` 与写时复制（Copy-on-Write, COW）

- **定义与目的**：`fork` 语义上要"复制整个地址空间"，实测只要几十微秒。技巧是**只复制页表不复制物理页**：父子 PTE 都改成**只读**并打 COW 标记；谁先写谁触发保护异常，由内核复制那一页。
- **直观解释**：像两人共用一本图书馆的书。管理员宣布"谁都不许在书上写字"；某人真要写时，管理员才影印一页给他——在此之前一本就够两人读。
- **底层机制图解**：

```
fork() 之后(尚未有人写)            进程 A 写 page X 时
父页表              子页表          1) PTE 只读 -> 保护异常 (#PF, error=7)
 +-----+             +-----+          2) 内核发现是 COW 页, 引用计数 > 1
 |RO PTE|-------->|RO PTE|           3) 申请新页, 拷贝原页内容
 +--+--+           +--+--+          4) 写入方 PTE 指向新页, 改回可写
    |                 |             5) 原页留给另一方, 计数减 1
    +--------+--------+
             v
      物理页 P (引用计数=2, 内容相同)
```

```
写后:
父页表              子页表
 +-----+             +-----+
 |RO   |             |RW   |          父仍指 P(只读, 直到它也写)
 +--+--+             +--+--+          子指 P'(私有副本, 可写)
    |                   |
    v                   v
  物理页 P            物理页 P' (拷贝)
```

- **与机器码/硬件的对应**：内核做的是 `copy_mm()` → `dup_mmap()` → `copy_page_range()`：复制页表项并把双方 PTE 清掉 `_PAGE_RW`，异常走 `do_wp_page()`。x86 的 `R/W` 位与 `CR2` 是硬件基础。**`fork` 后立即 `execve`（shell 的常见模式）不复制任何页**，故 shell 能廉价起进程。

12.2.9 `execve` 加载新程序的过程

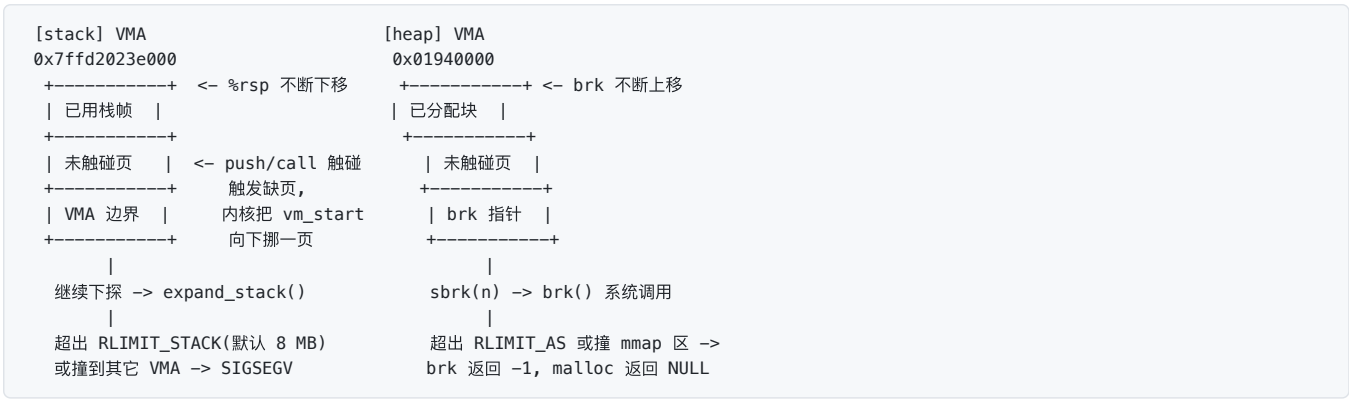
- **定义与目的**：`execve` 在**保留同一进程（同一 PID）**的前提下把地址空间整个换掉。
- **底层机制图解**：

```
execve("./prog", argv, envp)
  1. 校验可执行文件格式 (ELF magic) 与权限
  2. 释放旧地址空间: 遍历 VMA 逐个 munmap/释放物理页
     (打开的文件描述符表保留)
  3. 映射私有区域 (MAP_PRIVATE, 按需分页):
       .text -> r-xp (文件映射)   .rodata -> r--p
       .data -> rw-p (写时复制)   .bss -> rw-p (匿名零页)
  4. 映射共享区域: 把 libc.so.6 等共享库 mmap 进来 (r-xp/rw-p)
  5. 设置入口点: 把 ELF 的 e_entry 写入 PCB 保存的下一条指令地址
     (通常是 _start, 由它调 __libc_start_main 再调 main)
  6. 把 argv/envp 压到新用户栈顶, 设置 %rsp 与 %rdi/%rsi/%rdx
  7. 返回用户态, 从入口点执行, 地址空间已完全更换
```

- **与机器码/硬件的对应**：`.text` 是**文件映射**且 `PROT_EXEC`，同一程序启动 100 次物理内存里只有一份代码；`.bss` 是**匿名映射**，靠"零页 + 缺页"实现"初始化为 0"而不占磁盘空间。

12.2.10 栈与堆的按需增长

- **定义与目的**：栈从高地址向下增长，堆从低地址向上增长（`brk` 指针），两者中间是巨大的未映射空洞。它们**都不是预先分配的**，而是靠缺页驱动扩展。
- **底层机制图解**：



- **与机器码/硬件的对应**：`sbrk(incr)` 是 `brk` 的薄封装，返回**旧的** brk 值；`brk(addr)` 直接把堆顶设为 `addr`。栈的自动增长依赖"缺页地址紧邻 `vm_start` 之下"——所以**跳过一大段直接访问**（`%rsp` 减 1 MB 再写）会越过增长窗口而段错误。`ulimit -s`（8192 KB）限栈，`ulimit -v`（32000000 KB）限虚拟地址空间。

12.3 代码示例与底层机制分析

12.3.1 示例 A：`mmap` 文件映射——共享映射 vs 私有映射

代码 (C)：

```c
/* mmap_demo.c: 文件映射 共享 vs 私有
 * gcc -g -Wall -std=c11 mmap_demo.c -o mmap_demo && ./mmap_demo */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <errno.h>

#define PAGE 4096

static void dump(const char *tag, unsigned char *p, size_t n) {
    printf("%-28s: ", tag);
    for (size_t i = 0; i < n; i++) printf("%02x ", p[i]);
    printf(" | \"%.*s\"\n", (int)n, p);
}

int main(void) {
    const char *path = "/tmp/csapp12/seed.txt";
    /* 1. 造一页(4096B)的文件 */
    int fd = open(path, O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    char buf[PAGE];
    memset(buf, '.', sizeof buf);
    memcpy(buf, "AAA-original-page-content", 25);
    if (write(fd, buf, sizeof buf) != PAGE) { perror("write"); return 1; }

    /* 2. SHARED/PRIVATE 映射同一区间 */
    unsigned char *sh = mmap(NULL, PAGE, PROT_READ | PROT_WRITE, MAP_SHARED,  fd, 0);
    unsigned char *pr = mmap(NULL, PAGE, PROT_READ | PROT_WRITE, MAP_PRIVATE, fd, 0);
    if (sh == MAP_FAILED || pr == MAP_FAILED) { perror("mmap"); return 1; }

    printf("sh (MAP_SHARED ) = %p\npr (MAP_PRIVATE) = %p\n", (void *)sh, (void *)pr);
    printf("sh == pr ? %s   (两个映射不同虚拟地址)\n\n", sh == pr ? "yes" : "NO");

    dump("初始 sh", sh, 4); dump("初始 pr", pr, 4);

    /* 3. 各写首字节 */
    sh[0] = 'S';   /* 共享写: 落到文件 */
    pr[0] = 'P';   /* 私有写: 触发 COW */

    printf("\n-- 写入 sh[0]='S' (SHARED), pr[0]='P' (PRIVATE) --\n");
    dump("写后 sh", sh, 4);
    dump("写后 pr", pr, 4);

    /* 4. 独立读文件验证落盘 */
    int fd2 = open(path, O_RDONLY);
    unsigned char disk[4];
    if (pread(fd2, disk, 4, 0) != 4) { perror("pread"); return 1; }
    dump("文件实际内容(独立读)", disk, 4);
    close(fd2);

    /* 5. 看该文件的 VMA */
    printf("\n-- /proc/self/maps 中 seed.txt 的行 --\n");
    fflush(stdout);
    char cmd[256];
    snprintf(cmd, sizeof cmd, "grep seed.txt /proc/%d/maps", (int)getpid());
    int rc = system(cmd);
    (void)rc;

    munmap(sh, PAGE); munmap(pr, PAGE); close(fd); unlink(path);
    return 0;
}
```

【代码做什么？】

1. 造一个 4096 字节的文件，首 25 字节是 `AAA-original-page-content`，其余填 `'.'`。
2. 对**同一 fd、同一偏移 0** 建立两个映射：`MAP_SHARED` 与 `MAP_PRIVATE`，内核给它们分配不同的虚拟地址区间。
3. 写 `sh[0]='S'`（共享）与 `pr[0]='P'`（私有）。
4. 用 `pread` 绕过映射直接读文件，判断谁真正改了文件。

5. 打印 `/proc/self/maps` 中该文件两行，展示 `rw-s` 与 `rw-p`。

【底层机制透视】

- 初始时两个映射的 PTE 都**指向页缓存里同一批物理页**且只读。所以 `sh == pr` 为假——虚拟地址不同、物理页相同，这正是"同一物理数据出现在多个虚拟地址"的体现。
- 写 `sh[0]`：`MAP_SHARED` 允许写，内核把页缓存该页标脏（置 PTE 的 `D` 位）。**页缓存就是文件内容**，所以文件立刻"变了"（最终由回写线程落盘）。
- 写 `pr[0]`：`MAP_PRIVATE` 的写触发 COW——内核复制页缓存那一页，把本进程 PTE 指向私有副本。此后该进程看到 `'P'`，文件与页缓存仍是 `'S'`。**私有映射的修改不写回文件，与 `fork` 同机制。**
- `mmap` 返回的地址一定页对齐，映射长度向上取整到页的整数倍。

【内存布局 / 数据结构图解】（实测输出的地址）

```
虚拟地址空间                          物理内存 / 页缓存
0x7f158834d000 +---------------+       +----------------------+
 MAP_SHARED    | rw-s seed.txt|------>| 页缓存页（文件第 0 页）|
               +---------------+  +--->|  'S' 'A' 'A' '-' ...  |
0x7f1588317000 +---------------+  |    +----------------------+
 MAP_PRIVATE   | rw-p seed.txt|--+          ^ 写 sh[0]='S' 改这里
               +---------------+             |
                       写 pr[0]='P' -> COW 复制出新物理页
                                  +----------------------+
                                  | 私有副本 'P' 'A' ...  |
                                  +----------------------+
```

【实测验证】（真实运行输出）：

```
$ gcc -g -Wall -std=c11 /tmp/csapp12/mmap_demo.c -o /tmp/csapp12/mmap_demo   # 无警告
$ ./mmap_demo
sh (MAP_SHARED ) = 0x7f158834d000
pr (MAP_PRIVATE) = 0x7f1588317000
sh == pr ? NO   (两个映射不同虚拟地址)

初始 sh                   : 41 41 41 2d  | "AAA-"
初始 pr                   : 41 41 41 2d  | "AAA-"

-- 写入 sh[0]='S' (SHARED), pr[0]='P' (PRIVATE) --
写后 sh                   : 53 41 41 2d  | "SAA-"
写后 pr                   : 50 41 41 2d  | "PAA-"
文件实际内容(独立读): 53 41 41 2d  | "SAA-"

-- /proc/self/maps 中 seed.txt 的行 --
7f1588317000-7f1588318000 rw-p 00000000 08:03 305310360   /tmp/csapp12/seed.txt
7f158834d000-7f158834e000 rw-s 00000000 08:03 305310360   /tmp/csapp12/seed.txt
```

> 地址与 PID 每次运行都变（ASLR），但**权限字段 `rw-p` / `rw-s` 与"共享写改了文件、私有写只改副本"的结论恒定**。

注意 `maps` 行的格式：`起始-结束 权限 文件内偏移 设备 inode 路径`。权限字段的**第 4 个字符**是关键：`p` 表示私有（`MAP_PRIVATE`），`s` 表示共享（`MAP_SHARED`）。

12.3.2 示例 B：`fork` + 写时复制

代码 (C)：

```c
/* cow_demo.c: fork 写时复制 (COW)
 * gcc -g -Wall -std=c11 cow_demo.c -o cow_demo && ./cow_demo */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

static void faults(const char *tag) {
    FILE *f = fopen("/proc/self/stat", "r");
    if (!f) { perror("stat"); return; }
    char line[4096];
    if (!fgets(line, sizeof line, f)) { fclose(f); return; }
    fclose(f);
    char *p = strrchr(line, ')');   /* comm 可能含空格 */
    if (!p) return;
    p++;
    unsigned long minflt = 0, majflt = 0;
    int idx = 3; char *tok = strtok(p, " "); /* 第 3 字段起 */
    while (tok) {
        if (idx == 10) minflt = strtoul(tok, NULL, 10);
        if (idx == 12) { majflt = strtoul(tok, NULL, 10); break; }
        tok = strtok(NULL, " "); idx++;
    }
    printf("%-22s min_flt=%lu  maj_flt=%lu\n", tag, minflt, majflt);
}

int  global_var = 100;          /* .data: 私有可写 */
static int st = 7;              /* .data */

int main(void) {
    int  *heap = malloc(sizeof(int));
    *heap = 42;
    int  local = 5;             /* stack */

    printf("== fork 前 (pid=%d) ==\n", (int)getpid());
    printf("&global_var=%p  &st=%p  &local=%p  heap=%p\n",
           (void *)&global_var, (void *)&st, (void *)&local, (void *)heap);
    printf("global=%d st=%d local=%d *heap=%d\n", global_var, st, local, *heap);
    faults("fork 前");
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("\n[child ] pid=%d  &global_var=%p  &local=%p  heap=%p\n",
               (int)getpid(), (void *)&global_var, (void *)&local, (void *)heap);
        printf("[child ] 改前: global=%d local=%d *heap=%d\n", global_var, local, *heap);
        faults("child fork 后(未写)");
        global_var = 200; local = 50; *heap = 420; /* 写 -> COW */
        printf("[child ] 改后: global=%d local=%d *heap=%d\n", global_var, local, *heap);
        faults("child 写后(COW)");
        fflush(stdout);
        _exit(0);
    }

    int status; waitpid(pid, &status, 0);
    printf("\n[parent] pid=%d  &global_var=%p  &local=%p  heap=%p\n",
           (int)getpid(), (void *)&global_var, (void *)&local, (void *)heap);
    printf("[parent] 子进程改过之后: global=%d local=%d *heap=%d  <- 父进程值没变\n",
           global_var, local, *heap);
    faults("parent 写前");
    global_var = 999;
    faults("parent 写后(COW)");

    free(heap);
    return 0;
}
```

【代码做什么？】

1. 在 `.data`（`global_var`、`st`）、堆（`*heap`）、栈（`local`）各放一个变量，打印地址与值。

2. 从 `/proc/self/stat` 读出 `min_flt`（次要缺页）与 `maj_flt`（主要缺页）计数。
3. `fork` 出子进程：先打印地址与值（应与父进程完全相同），再写这三个变量，然后重读缺页计数。
4. 父进程 `waitpid` 后重新读值——**地址和子进程一模一样，值却仍是自己的**。

【底层机制透视】

- 子进程打印的 `&global_var`、`&local`、`heap` 与父进程**逐位相同**。这毫不神秘：`fork` 复制了页表，所以**虚拟地址不变**；变化的是 PTE 指向的物理页由"先共享、写时复制"决定。
- `min_flt` 的跳变就是 COW 的指纹：子进程 `fork` 后未写时只有 14；写 3 个变量后变成 17，**恰好多 3 次**——每次写一页触发一次保护异常 → 复制一页 → 更新 PTE。父进程随后写 `global_var`，`min_flt` 从 92 涨到 93，又是一次 COW。
- `fork` 快到什么程度？它不做任何数据拷贝，只复制页表（几 KB）并把所有可写 PTE 改成只读。这就是 shell 里 `ls` 能瞬间返回的原因。
- 注意 `_exit(0)` 而不是 `exit(0)`：`exit` 会刷 stdio 缓冲区，而子进程继承的缓冲区里还有父进程未刷的内容，会造成重复输出。

【内存布局 / 数据结构图解】（实测地址）

```
 父进程虚拟地址               子进程虚拟地址             物理内存
0x7fff16be0920 local    ==   0x7fff16be0920 local  --> 栈页 (COW, 引用计数 2)
0xc772a0       heap     ==   0xc772a0       heap   --> 堆页 (COW, 引用计数 2)
0x4040a0       global_var == 0x4040a0       global_var --> .data 页 (COW, 计数 2)

子进程写之后:
0x7fff16be0920 local    ==   0x7fff16be0920 local  --> 父: 旧栈页 | 子: 新栈页
0xc772a0       heap     ==   0xc772a0       heap   --> 父: 旧堆页 | 子: 新堆页
0x4040a0  global_var=100 == 0x4040a0  global_var=200 --> 各自私有副本
```

【与汇编 / 硬件的对应】：`fork` 用户态只是 `call fork@plt`，魔法在 `sys_fork` → `copy_process()` → `copy_mm()` → `dup_mmap()` → `copy_page_range()`。COW 的写保护依赖 PTE 的 `R/W` 位（清 0 即只读），异常由 `#PF` 的 error code（写 + 用户 + 保护违规）识别。可用 gdb 验证：

```
$ gcc -g -Wall -std=c11 cow_demo.c -o cow_demo && gdb -q ./cow_demo
(gdb) break fork
(gdb) run
(gdb) p &global_var
$1 = (int *) 0x4040a0
(gdb) catch syscall fork
(gdb) continue
```

【实测验证】（真实运行输出）：

```
$ ./cow_demo
== fork 前 (pid=3820916) ==
&global_var=0x4040a0  &st=0x4040a4  &local=0x7ffcdc73f930  heap=0x22bb2a0
global=100 st=7 local=5 *heap=42
fork 前               min_flt=78  maj_flt=0

[child ] pid=3820917  &global_var=0x4040a0  &local=0x7ffcdc73f930  heap=0x22bb2a0
[child ] 修改前: global=100 local=5 *heap=42
child fork 后(未写) min_flt=14  maj_flt=0
[child ] 改后: global=200 local=50 *heap=420
child 写后(COW)      min_flt=17  maj_flt=0

[parent] pid=3820916  &global_var=0x4040a0  &local=0x7ffcdc73f930  heap=0x22bb2a0
[parent] 子进程改过之后: global=100 local=5 *heap=42  <- 父进程值没变
parent 写前          min_flt=92  maj_flt=0
parent 写后(COW)     min_flt=93  maj_flt=0
```

> 地址与缺页计数随 ASLR 与启动路径浮动（另一次运行是 `fork 前 min_flt=388`、`parent 写后 min_flt=403`），但**"写 3 个变量 +3 次缺页"和"父子地址完全相同、值各自独立"这两个结论每次都成立**。

12.3.3 示例 C：用 `malloc` 观察 `brk` 与 `mmap` 的分界

代码 (C)：

```c
/* brk_grow.c: 小请求走 brk, 大请求走 mmap
 * gcc -g -Wall -std=c11 brk_grow.c -o brk_grow */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    printf("起始 brk = %p\n", sbrk(0));
    void *ptrs[64];
    for (int i = 0; i < 16; i++) {   /* 64 KiB < 128 KiB */
        ptrs[i] = malloc(64 * 1024);
        memset(ptrs[i], i, 64 * 1024); /* 触碰才落页 */
        printf("第 %2d 次 malloc(64KiB) = %p  brk = %p\n", i, ptrs[i], sbrk(0));
    }
    void *big = malloc(256 * 1024);  /* > 阈值 -> mmap */
    memset(big, 1, 256 * 1024);
    printf("malloc(256KiB) = %p   brk = %p  <- brk 不动, 走了 mmap\n", big, sbrk(0));

    FILE *f = fopen("/proc/self/maps", "r");
    char line[512];
    printf("\n-- [heap] --\n");
    while (f && fgets(line, sizeof line, f))
        if (strstr(line, "[heap]")) fputs(line, stdout);
    if (f) fclose(f);
    return 0;
}
```

【底层机制透视】：glibc 的 `malloc` 有两条路径。请求 < `M_MMAP_THRESHOLD`（默认 128 KB）走**堆（`brk` / `sbrk`）**，否则用 **`mmap` 匿名映射**、`free` 时 `munmap` 归还。理由很实际：`brk` 只能从堆顶线性伸缩，大块夹在中间会造成堆顶无法回收，而 `mmap` 可从任意位置归还。**这就是 Malloc Lab 里 `sbrk` 是主要工具的原因**——trace 里绝大多数请求都很小，战场就在 `[heap]`。

【实测验证】（真实运行输出）：

```
$ gcc -g -Wall -std=c11 brk_grow.c -o brk_grow && ./brk_grow
起始 brk = 0x1eb5000

第  0 次 malloc(64KiB) = 0x1eb62b0  brk = 0x1ed6000
第  1 次 malloc(64KiB) = 0x1ec62c0  brk = 0x1ef7000
第  2 次 malloc(64KiB) = 0x1ed62d0  brk = 0x1ef7000
第  3 次 malloc(64KiB) = 0x1ee62e0  brk = 0x1ef7000
第  4 次 malloc(64KiB) = 0x1ef62f0  brk = 0x1f27000
...
第 15 次 malloc(64KiB) = 0x1fa63a0  brk = 0x1fb7000
malloc(256KiB) = 0x7f1f0eb5c010   brk = 0x1fb7000  <- brk 不动, 走了 mmap

-- [heap] --
01eb5000-01fb7000 rw-p 00000000 00:00 0                                  [heap]
```

> `brk` 增量为 `0x21000 = 132 KB`（glibc 的 `M_TOP_PAD` 后备增量），说明 `brk` 并非每次分配都增长；换一次运行地址与增量会不同，但**"小请求在 `[heap]`、大请求落在 `0x7f...` 且 `brk` 不动"这一判据恒定**。

观察两点：(1) `brk` 每次跳 **0x21000 = 132 KB**，是 glibc 一次 `sbrk` 的后备增量（`M_TOP_PAD`），所以 `brk` 并不每次增长；(2) 256 KiB 的块地址落在 `0x7fd...`（mmap 区），`brk` 纹丝不动。

用 `strace` 可以直接看到这两条系统调用：

```
$ strace -e trace=brk,mmap,munmap ./strace_demo
brk(NULL)                               = 0x13bd000
brk(0x13de000)                          = 0x13de000     <- malloc(64KiB) 走这
mmap(NULL, 1052672, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f68a1aff000
munmap(0x7f68a1aff000, 1052672)         = 0              <- free 大块立即归还
a=0x13bd2a0 b=0x7f68a1aff010
```

12.3.4 示例 D：三类访存异常与内存工具链

`mmap` 一个区间、只读映射、访问未映射地址，分别对应**合法缺页**、**保护异常**、**段错误**。以下程序故意触发后两种，属演示代码，请勿模仿：

```c
/* fault_demo.c 用法: ./fault_demo <0|1|2>
 * ⚠️ 演示用, 勿模仿: mode 1/2 故意触发 SIGSEGV */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/mman.h>

static void handler(int sig, siginfo_t *si, void *ctx) {
    (void)ctx; (void)sig;
    printf("\n[信号] SIGSEGV, si_addr = %p\n", si->si_addr);
    fflush(stdout);
    _exit(42);
}

int main(int argc, char **argv) {
    int mode = argc > 1 ? atoi(argv[1]) : 0;
    struct sigaction sa = {0};
    sa.sa_sigaction = handler; sa.sa_flags = SA_SIGINFO;
    sigaction(SIGSEGV, &sa, NULL);

    if (mode == 0) {                /* 合法缺页: 页未驻留 */
        char *p = mmap(NULL, 4 * 4096, PROT_READ | PROT_WRITE,
                       MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
        printf("匿名映射 %p .. %p (未驻留)\n", p, p + 4*4096-1);
        p[0] = 1;                   /* 缺页, 内核给零页 */
        printf("首次写 p[0] 成功 -> 缺页处理程序已接入\n");
    } else if (mode == 1) {     /* 权限不对 -> 保护异常 */
        char *p = mmap(NULL, 4096, PROT_READ, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
        printf("只读映射 %p, 尝试写入...\n", p);
        fflush(stdout);
        p[0] = 1;
    } else {                    /* 不在任何 VMA 内 -> 段错误 */
        char *bad = (char *)0x0000000100000000UL;
        printf("写不在任何 VMA 内的 %p ...\n", bad);
        fflush(stdout);
        *bad = 1;
    }
    return 0;
}
```

【实测验证】:

```
$ ./fault_demo 0
匿名映射 0x7f80d29b9000 .. 0x7f80d29bcfff (未驻留)
首次写 p[0] 成功 -> 缺页处理程序已接入

$ ./fault_demo 1
只读映射 0x7f1d35f6c000, 尝试写入...
[信号] SIGSEGV, si_addr = 0x7f1d35f6c000        <- 地址合法, 权限不符

$ ./fault_demo 2
写不在任何 VMA 内的地址 0x100000000 ...
[信号] SIGSEGV, si_addr = 0x100000000           <- 地址根本不在任何 VMA 内
```

关键洞察: mode 1 与 mode 2 都得到 `SIGSEGV`，但**原因完全不同**——前者"地址合法、权限不符"，后者"地址不在任何 VMA 内"。内核用同一信号表达二者，必须看 `si_addr` 与 `/proc/pid/maps` 才能区分。

【VMA 与本机参数实测】:

```
$ getconf PAGE_SIZE
4096
$ ulimit -v                 # 虚拟地址空间上限 (KB)
32000000
$ ulimit -s                 # 栈上限 (KB)
8192
$ grep -E 'MemTotal|MemAvailable|SwapTotal|Committed_AS|PageTables' /proc/meminfo
MemTotal:       527645792 kB
MemAvailable:   310249888 kB
SwapTotal:        4095996 kB
Committed_AS:   270887672 kB
PageTables:      1166892 kB   <- 全系统页表本身占 ~1.1 GB!
```

`PageTables: 1166892 kB` 是最有说服力的数字：即便有四级页表与 TLB，**光页表结构就吃掉 1.1 GB 物理内存**。

要知道每个 VMA 究竟驻留了多少物理内存，看 `/proc/<pid>/smaps`（比 `maps` 多出逐段统计）：

```
$ head -12 /proc/self/smaps
56513fbd3000-56513fbd5000 r--p 00000000 08:05 2013266890   /usr/bin/head
Size:                  8 kB
KernelPageSize:        4 kB
Rss:                   8 kB      <- 实际驻留
Pss:                   8 kB      <- 按共享比例摊分后的份额
Shared_Clean:          0 kB
Private_Dirty:         0 kB
Private_Clean:         8 kB
Anonymous:             0 kB      <- 匿名页(堆/栈)为 0, 说明这是纯文件映射
Swap:                  0 kB
```

`Rss` 与 `Size` 之差就是"已映射但尚未触碰"的部分；`Private_Dirty` 与 `Shared_Dirty` 的对比可直接看出 COW 复制与共享写各占多少。

`/proc/self/maps` 的一个真实片段（运行 `vma_demo` 时）：

```
00400000-00401000 r--p 00000000 08:03 304575903      /tmp/csapp12/vma_demo   <- ELF 头
00401000-00402000 r-xp 00001000 08:03 304575903      /tmp/csapp12/vma_demo   <- .text
00402000-00403000 r--p 00002000 08:03 304575903      /tmp/csapp12/vma_demo   <- .rodata
00403000-00404000 r--p 00002000 08:03 304575903      /tmp/csapp12/vma_demo   <- 重定位页
00404000-00405000 rw-p 00003000 08:03 304575903      /tmp/csapp12/vma_demo   <- .data/.bss
01940000-01961000 rw-p 00000000 00:00 0              [heap]
7fa7d0c00000-7fa7d0c28000 r--p 00000000 08:05 3221225831  /usr/lib64/libc.so.6
7fa7d0c28000-7fa7d0d9d000 r-xp 00028000 08:05 3221225831  /usr/lib64/libc.so.6
7fa7d0d9d000-7fa7d0df5000 r--p 0019d000 08:05 3221225831  /usr/lib64/libc.so.6
7fa7d0df5000-7fa7d0df6000 ---p 001f5000 08:05 3221225831  /usr/lib64/libc.so.6  <- 保护页!
7fa7d0dfa000-7fa7d0dfc000 rw-p 001f9000 08:05 3221225831  /usr/lib64/libc.so.6
7fa7d0aff000-7fa7d0c00000 rw-p 00000000 00:00 0                  <- malloc(1MiB) 匿名区
7ffee41a0000-7ffee41c4000 rw-p 00000000 00:00 0              [stack]
7ffee41e2000-7ffee41e6000 r--p 00000000 00:00 0              [vvar]
7ffee41e6000-7ffee41e8000 r-xp 00000000 00:00 0              [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0      [vsyscall]
```

对照 12.2.5 的布局图：**每一行就是一个 VMA**；`---p` 是 `PROT_NONE` 保护页（让越界一字节立即段错误）；`[heap]` 与 `[stack]` 之间的"空洞"宽约 47 TB；`malloc(1MiB)` 的匿名区**文件名列为空**，正是 12.3.3 所说的 mmap 区。

`pmap` 与 `vmstat` 可直接观察 VMA 与换页：

```
$ pmap -x <pid>
Address           Kbytes     RSS   Dirty Mode  Mapping
000056210bce0000     132      24      24 rw---   [ anon ]     <- 堆
00007f62ece00000    2528     156       0 r---- LC_COLLATE
$ vmstat 1 2
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 9  0 4044000 19040120     24 394830304    0    0     2   20    0    0  3  1 95  0  0
```

`vmstat` 的 `si`/`so` 是换入/换出速率（本机 `swpd` 约 4 GB）；`pmap -x` 的 `Kbytes` 是**虚拟大小**、`RSS` 是**实际驻留**，两者相差一个数量级，正好印证"虚拟内存远大于物理内存"。

12.4 实验关联：L5 Malloc Lab

本讲是 **L5 Malloc Lab** 的直接理论前提，recitation（`F25-rec08_slides.txt`）补上了工程细节：

- **为什么 `sbrk` 是主要工具**：12.3.3 已实测，trace 里绝大多数请求远小于 128 KB 的 `MMAP_THRESHOLD`，走 `brk`/`sbrk` 路径。分配器从 `mem_sbrk()`（`sbrk` 封装）拿货，返回**新扩展区域起始地址**，必须处理失败：返回 `NULL` 时 `mm_malloc` 也返回 `NULL`。

- **堆是连续的虚拟内存**：`brk` 只能线性伸缩，分配器须自己管空闲块（隐式 → 显式 → 分离链表），碎片代价由你承担。
- **契约（contracts）与堆不变量**是调试主线。recitation 的排错流程很典型：

```
$ gcc -g -Wall -std=c11 mm.c mdriver.c -o mdriver
$ ./mdriver -c ./traces/syn-struct-short.rep
ERROR [trace ..., line 16]: block 1 (at 0x8000000a0) has 8 garbled bytes, starting at byte 16

$ gdb --args ./mdriver-dbg1 -c ./traces/syn-struct-short.rep
(gdb) watch *0x8000000a0
(gdb) run
Hardware watchpoint 1: *0x8000000a0
Old value = 129
New value = 32
write_block() at mm.c:333          <- 定位到 write_block() 越界写

$ ./mdriver-dbg2 -c ./traces/syn-struct-short.rep
mdriver-dbg: mm.c:331: void write_block(block_t *, size_t, _Bool):
Assertion `(unsigned long)footerp < ((long)block + size)' failed.
```

- **"Garbled bytes" 的含义**：`mdriver` 写 payload 后校验；被改写说明**分配器在已分配块内部写了东西**——典型原因是块头/脚部算错，或 `find_fit` 返回已占用的块。
- **排错提示**：Valgrind **不能**用（它模拟堆，会与你的 `sbrk` 冲突）；`mdriver -D` 尽早检测乱码，`-V` 定位出错的 trace。只能靠 **gdb 硬件观察点 + 自写堆检查器**。
- **两条铁律（为 Lecture 13–14 铺垫）**：
- **应用不知道也不关心分配器如何管理内存**。它只知道 `malloc` / `free` / `realloc` 的语义契约与对齐要求；内部用隐式、显式还是分离链表，完全是自由的实现细节。
- **分配器不能假定数据的位模式**。返回的内存可能是任意字节（`malloc`）或全零（`calloc`），但**绝不能认为 8 字节对齐就够**：x86-64 ABI 要求 `malloc` 返回**16 字节对齐**（`long double`/SSE 需要 `movaps`）。最小块大小与填充策略都必须围绕 16 字节设计。recitation 的利用率先修计算即由此而来：`malloc(24)` 在带脚部的旧设计下是"块头 8 + 数据 24 + 脚部 8 = 40 → 取整 48"，装不进 size 32 的块；去掉脚部后变"8 + 24 = 32 → 取整 32"，正好装下。

12.5 常见错误与调试技巧

- **把 `MAP_PRIVATE` 当成 `MAP_SHARED` 用**：改了映射内存但文件没变，或用 `MAP_SHARED` 改了别人的数据。**调试**：`grep <文件名> /proc/<pid>/maps` 看权限第 4 字符是 `p` 还是 `s`；`strace -e trace=mmap,msync` 确认 flag。
- **`mmap` 的 offset 没页对齐**：`offset` 必须是 `sysconf(_SC_PAGESIZE)` 的整数倍，否则返回 `MAP_FAILED` 并置 `errno = EINVAL`。**调试**：必须写 `if (p == MAP_FAILED) perror("mmap");`——忘记检查就得到"信号 11，想访问 0xffff...ffff"这种迷惑性错误。
- **`fork` 后重复输出**：父子共享 stdio 缓冲区，`exit()` 会把父进程未刷的内容再刷一遍。**调试**：子进程用 `_exit()`，或 `fork` 前 `fflush(NULL)`；`./a.out | cat` 更容易看出重复。
- **栈溢出 vs 跳过增长窗口**：`ulimit -s` 默认 8 MB（本机 8192 KB）。`%rsp` 一次下降多页再写（如 `char big[2*1024*1024]` 直接写末尾）可能越过内核栈增长窗口而段错误。**调试**：`ulimit -s`；`gdb` 的 `bt` 与 `info registers rsp`。
- **误以为虚拟地址相同就是同一变量**：`fork` 后父子变量地址相同、值独立（见 12.3.2）。**调试**：`gdb` 的 `set follow-fork-mode child` / `parent`；打印 `/proc/self/stat` 的 `min_flt` 看 COW 次数。
- **在 `[heap]` 里找不到大块**：`malloc(1MiB)` 走 `mmap`，不在 `[heap]`。**调试**：`strace -e trace=brk,mmap,munmap`；`mallinfo2().hblkhd` 报告 mmap 段字节数（实测 free 后为 0）。
- **不看 `si_addr` 就乱猜 SIGSEGV 原因**：`si_addr` 在 VMA 内外、权限如何，决定是保护异常还是段错误。**调试**：用 `SA_SIGINFO` 打印 `si_addr`，结合 `/proc/<pid>/maps` 与 `gdb` 的 `x/i $pc`。

12.6 关键要点

- **多级页表用"树"换空间**：48 位 VA、4 KB 页、8 B PTE 的单级页表要 512 GB；四级页表只为真正用到的子树分配，映射一个 4 KB 页最少只需 4 个页表页（16 KB），映射一个 2 MB 区间只要 3 个（12 KB）。
- **TLB 是用时间换空间的那一半**：Core i7 的 L1 d-TLB（64 项/4 路）几乎吸收了全部翻译请求，命中则完全跳过 4 次无局部性的 page walk。
- **一次翻译的完整链路是 `VA → TLB/四级 page walk → PA → L1 d-cache`**，其中 PPO ≡ VPO 不变，CI 位在翻译前后也不变（虚拟索引、物理标记），因此 L1 可并行做索引与翻译。
- **PTE 的低 12 位是权限而非地址**：`P`、`R/W`、`U/S`、`A`、`D`、`PS`、`G`、`XD` —— `P=0` 时高 40 位转而记录页在磁盘的位置。COW、按需分页、写回全建在这几位上。
- **VMA 是纯软件的"地址空间地图"**：`vm_area_struct`（起止、权限、文件）串成链表/红黑树，`mm_struct.pgd` 写进 CR3 完成进程切换；缺页处理程序拿 `CR2` 查这张图，再决定"换页 / SIGSEGV"。

- **`mmap` 是虚拟内存最通用的接口**：文件映射、匿名内存、共享库是同一套机制；`fork` 的写时复制与 `MAP_PRIVATE` 的私有写共用 `do_wp_page()`。

12.7 思考题（带答案）

题 1（计算题）：某 32 位机器使用两级页表，页大小 4 KB，PTE 4 字节，VA 划分成 10/10/12 位。(a) 单级页表需要多少内存？(b) 若某进程只用了最低的 4 MB 和最高的 4 MB 虚拟地址空间，两级页表实际需要多少内存？(c) 若 TLBI 借用 VPN 的低 4 位（16 组），一次 TLB 未命中的最坏访存次数是多少？

答：(a) 页数 $= 2^{32}/2^{12} = 2^{20}$，单级页表 $= 2^{20} \times 4\,\mathrm{B} = 4$ MB。(b) 最高 4 MB 需 1 个一级页表 + 1 个二级页表（各 4 KB）；最低 4 MB 与之共享一级页表，但需自己的二级页表。总计 $3 \times 4\,\mathrm{KB} = 12$ KB，**比 4 MB 缩小约 340 倍**——这就是多级页表的价值。(c) 最坏情况：TLB miss + 2 次页表访存（一级、二级 PTE 各一次）+ 1 次取数据；若还要查 L2 TLB 则更多。对比 TLB 命中：只需 1 次数据访存。

题 2（计算题）：用 12.2.3 的页表，翻译 `VA = 0x00007F4A2B8C9A48`，并指出它落在哪个 VMA、会触发什么操作。

答：按 12.2.3 走查，四级索引依次为 `0x0FE`、`0x128`、`0x15C`、`0x0C9`，VPO = `0xA48`，PPN = `0xA3C`，$\mathbf{PA = 0x0000A3CA48}$。若该 VA 落在 `r--p` 的 `.rodata` VMA 内，这是**合法的读**，缺页处理程序只需换入页面并置 `P=1`；若是**写**，VMA 权限只读、`R/W` 不匹配 → **保护异常 → SIGSEGV**（mode 1）。若该 VA 不在任何 VMA 内（如 `0x0000000100000000`，mode 2），则直接**段错误**。索引 254/296/348/201 都很大，说明这是典型的用户栈或共享库区访问，对应的 PGD/PUD/PMD 条目本来就在。

题 3（辨析题）："既然 `fork` 之后父子进程的变量地址完全一样，那它们一定共享同一个物理内存单元；我在父进程里改了它，子进程应该也看得到。"这句话错在哪？

答：错在把"虚拟地址相同"等同于"物理页相同"。`fork` 复制的是**页表**；父子 PTE 在 `fork` 瞬间确实指向同一物理页，但双方 `R/W` 位被清成只读（COW）。任一方写时触发保护异常，内核复制该页、把写入方 PTE 指向副本并恢复可写，另一方仍指原点。因此**地址相同、值独立**。实测（12.3.2）：子进程写后 `global=200`，父进程读到的仍是 `100`，且父进程 `min_flt` 从 92 涨到 93，证明它后来也复制了一次。唯一例外是显式共享内存（`MAP_SHARED` 的 `mmap` 或 `shmget`）。

题 4（辨析题）：有人说"`malloc` 返回的指针是页对齐的，所以我能按 `mmap` 的 `offset` 规则反推它，甚至直接 `munmap` 它来省内存"。错在哪？

答：错两处。第一，`malloc` 返回的是**分配器内部块的 payload 指针**，位于 `[heap]` 中 16 字节对齐的偏移上（实测 `0x1eb62b0`、`0x1941490`），**不是页对齐**，不能反推页基址；分配器还在它前后放了块头/块尾，擅自 `munmap` 会连元数据一起抹掉。第二，只有**大于等于 `M_MMAP_THRESHOLD`（默认 128 KB）**的请求才由 glibc 用 `mmap` 单独映射（实测 `malloc(256KiB) = 0x7f1f0eb5c010`；而它前面的 `malloc(64KiB)` 都落在 `0x1eb...` 的 `[heap]` 内）。释放内存唯一正确的接口是 `free`；想主动还内存给内核可用 `malloc_trim(0)`，而非自己 `munmap`。

Lecture 13: 动态内存分配：基础 (Dynamic Memory Allocation: Basic)

> **讲义对应**：CMU 15–213 Lecture 13 — Dynamic Memory Allocation: Basic（素材：`F25-13-malloc-basic.txt`） **教材对应**：CS:APP3e 第 9 章 9.9（动态内存分配） **关联 Lab**：**L5a Malloc Lab（checkpoint）**（习题课与 bootcamp：`F25-rec07_slides.txt`、`F25-malloc-bootcamp.txt`）

13.1 概述

第 11、12 讲我们把虚拟内存讲到了页表与缺页处理，知道了进程的地址空间里有一段"运行时堆（run-time heap）"，可由 `brk` 系统调用伸缩。本讲回答一个随之而来的问题：**这段堆由谁管理、怎样管理？** 答案是**动态内存分配器（dynamic memory allocator）**。我们先把 `malloc` / `calloc` / `realloc` / `free` 的接口语义与分配器必须服从的硬性约束钉死，再定义两个彼此冲突的性能目标——**吞吐率（throughput）**与**峰值内存利用率（peak memory utilization）**，接着剖析两类**碎片（fragmentation）**；然后动手实现最经典的数据结构——**隐式空闲链表（implicit free list）**，并用 Knuth 的**边界标记（boundary tags）**把合并做到 $O(1)$。本讲为 **L5a Malloc Lab 的 checkpoint** 提供全部理论装备；下一讲（Lecture 14）在此基础上改进吞吐率与利用率（显式空闲链表、分离空闲链表、延迟合并、footer 优化）。

13.2 核心概念与底层机制图解

13.2.1 为什么需要动态内存分配（Why Dynamic Allocation）

- **定义与目的**：静态/全局变量的大小与偏移必须在**编译期**定死（见 Lecture 7：`.data` / `.bss` 段的大小写死在可执行文件里）；函数内的局部数组虽由栈帧在运行期建立，却受**生命周期约束**——函数一返回，栈帧整个消失（见 Lecture 5）。于是两类需求无法满足：① **长度编译期不可知**（链表节点个数、读文件得到的内容）；② **生命周期要超出创建它的函数**。动态内存分配就是为这两类需求准备的：运行期申请任意大小的内存，用多久由程序员决定，用完显式归还。
- **直观解释**：把地址空间想成一栋楼：`.text` / `.data` 是设计图上画死的承重墙；**栈**是前台的折叠桌，人一走就收；**堆**是一整层毛坯仓库，你随时找管理员（分配器）要地方，走时把钥匙交还。
- **底层机制图解**（进程地址空间与"the break"）：

```
 高地址  0x7fffffffffff
+--------------------------------+
|  内核虚拟内存（用户代码不可见）  |
+--------------------------------+
|  用户栈（运行时创建） %rsp 在此 |
|        |                       |
|        v  向下增长             |
|        ...                     |
|        ^  向上增长             |
|  共享库内存映射区（如 libc）    |
+--------------------------------+
|        ^                       |
|        |  <-- brk 指针("the break") 可由 sbrk 调整
+--------------------------------+  <-- heap_end（堆顶，随 sbrk 上移）
|                                |
|  运行时堆（heap，malloc 创建）  |
|                                |
+--------------------------------+  <-- heap_start（堆起点，固定）
|  读/写段 .data、.bss           |
+--------------------------------+
|  只读段 .init、.text、.rodata  |
+--------------------------------+
|  未使用（从低地址起）          |
+--------------------------------+  0
 低地址
```

- **与机器码/硬件的对应**：`malloc` 系列是**库函数**，不是系统调用；它们最终通过 `brk` / `sbrk`（或大块时 `mmap`）向内核要内存，见 13.4。注意 CS:APP 的一个约定：**每次课里的"1 个方块 = 1 个字 = 8 字节"**（x86-64 LP64 下 `long` 与指针都是 8 字节），本章所有堆示意图都按这个换算。

13.2.2 `malloc` 包的接口语义（The malloc Package）

- **定义与目的**（接口原样照录官方语义）：

函数	语义	失败/边界行为
`void *malloc(size_t size)`	返回指向**至少 `size` 字节**的块的指针；在 x86-64 上**按 16 字节边界对齐**	失败返回 `NULL` 并置 `errno`
`void free(void *p)`	把 `p` 指向的块归还空闲池	`p` 必须来自此前的 `malloc` / `calloc` / `realloc`
`void *calloc(size_t n, size_t size)`	`malloc` 的版本，**把块清零**	同上
`void *realloc(void *p, size_t size)`	改变已分配块的大小	**可能搬移到新地址**，旧内容保留 `min(旧, 新)` 字节
`void *sbrk(intptr_t incr)`	分配器**内部**用它伸缩堆	不是给应用程序用的

- **直观解释**：`malloc` 是"**毛坯交付**"——不擦内存，里面的字节是上次用剩的垃圾；`calloc` 是"**精装修交付**"（清零）；`free` 是"**退钥匙**"，不把内存还给操作系统，只把这块地标为可再用。
- **底层机制图解（关键陷阱）**：`free` 只能接受**分配器自己发出去的载荷指针**：

```
  堆中一个块
  +--------+---------------------------+--------+
  | header | payload                   | footer |
  +--------+---------------------------+--------+
           ^                ^
           |                |
      合法的 p        ✗ 非法的"中部指针" p+8
    (malloc 的返回值)   (free 它是 UB!)

  栈变量:  int x;  free(&x);   ✗ UB!（那不是堆地址）
```

- **补充说明**：讲义幻灯片写"`size == 0` 时返回 `NULL`"。C 标准允许实现返回 `NULL` **或**一个可被 `free` 的唯一指针；glibc 实测返回非 `NULL`（本章 13.3.4 实测输出）。**写代码时两种都要能正确处理**——这也是 Malloc Lab 里 `mm_malloc(0)` 必须自行裁决的地方。

13.2.3 分配器的硬性约束（Constraints）

- **定义与目的**：应用可以发出**任意顺序**的 `malloc` / `free` 请求，`free` 的参数必须是 `malloc` 过的块；分配器不知道未来的大小分布。与此同时：

- **必须立即响应**：**不能重排（reorder）或缓冲（buffer）请求**——`malloc` 一被调用就必须立刻返回可用内存。
- **只能用堆区域**：块只能放在空闲内存里，且**不能移动已分配的块（不允许压缩 compaction）**，因为应用程序持有裸指针，一搬移就全部失效。
- **必须满足对齐与隔离**：x86-64 上返回 16 字节对齐的指针（Malloc Lab 放宽到 8 字节）；已分配块的内容分配器一律不能碰。

13.2.4 两个性能目标：吞吐率与内存利用率（Throughput vs. Utilization）

- **吞吐率**：单位时间内完成的请求数。例：10 秒内完成 5,000 次 `malloc` 与 5,000 次 `free` → 吞吐率 = 1,000 operations/second。
- **聚合载荷（aggregate payload）** P_k：第 k 次请求后**当前所有已分配载荷之和**。设请求 R_i 的载荷为 p_i，则 $P_k = \sum_{\text{当前已分配}} p_i$。**峰值聚合载荷** $\max_{i \le k} P_i$ 是序列到第 k 步为止的最大值。
- **当前堆大小** H_k：假定堆只在分配器调用 `sbrk` 时增长、从不收缩。
- **开销（overhead）**：$O_k = \dfrac{H_k}{\max_{i \le k} P_i} - 1.0$，即"没有用于程序数据的那部分堆空间的占比"。
- **峰值内存利用率（peak utilization）**：

$$U = \frac{\max_{k \le n} P_k}{H_n}$$

- **直观解释**：吞吐率是"仓库管理员发货多快"，利用率是"仓库里有多少空间真的放了货"。**两者天然冲突**：要发货快就选第一个看得过去的空位（留下碎片），要放得满就得把所有空位翻个底朝天（慢）。Malloc Lab 用一个加权性能指数把它们捆在一起：$P = wU + (1-w)\min\left(1, \dfrac{T}{T_{libc}}\right)$，默认 $w = 0.6$，$T_{libc} = 600$ Kops/s——**利用率权重更高，但也不能靠"极慢但极省内存"取胜**。
- **基准示例（`syn-array-short`）**：讲义给出的 trace 共 20 步（`a`=allocate，`f`=free），分配总量峰值出现在第 11 步：`a 7 33856` 之后 Allocated $= 90036$，Peak $= 90036$；第 20 步全部释放后 Allocated $= 0$，Peak 仍是 90036。因此峰值利用率的分母就是这 90036。

13.2.5 碎片（Fragmentation）

- **内部碎片（internal fragmentation）**：**一个块内**，块大小大于载荷请求的部分。

$$\text{内部碎片} = \text{块大小} - \text{请求的载荷大小}$$

三个来源：① 维护堆数据结构的开销（头部/脚部）；② 为对齐而填充（padding）；③ 显式策略决定（例如"为了不产生太小的碎块，把一个大块整个给你"）。**它只取决于过去请求的模式**，所以**容易度量**。讲义给出的该基准上"分配器数据 + 对齐填充"的开销约为 **1.5%**，并强调这是**实践中达不到的**（因为不允许移动已分配块）。 – **外部碎片（external fragmentation）**：**空闲内存总量足够，但没有一个连续空闲块大到能满足当前请求**。它取决于**未来请求的模式**，因此**难以度量**，只能估计。讲义基准上"最佳适配（best fit）"策略的总开销是 **8.3%**。 – **直观解释**：内部碎片像"租了 10 平米仓库，实际只放得下 8 平米的货"；外部碎片像"停车场总共还剩 5 个空位，但每个都被夹在两辆车之间，5 米长的货车一辆都进不去"。 – **底层机制图解**（外部碎片：总量够、单块不够）：

```
  p1=malloc(32)  p2=malloc(40)  p3=malloc(48)
+----+   +----+   +----+   +---------+
| p1 |   | p2 |   | p3 |   | 空闲    |
+----+   +----+   +----+   +---------+
  free(p2)
+----+   +----+   +----+   +---------+
| p1 |   |空闲 |   | p3 |   | 空闲    |
+----+   +----+   +----+   +---------+
  p4 = malloc(64)   --> ✗ 失败:
  空闲合计 = 40 + 一大块, 但 p2 那块只有 40 字节,
  不满足 64 字节的**单个连续**要求。
```

- **与机器码/硬件的对应**：碎片是**纯软件**现象，与硬件无关；但它直接决定**缺页次数**（堆越大、触碰的页越多，见 Lecture 11/12 的按需分页），因此是系统性能问题而非代码风格问题。

13.2.6 隐式空闲链表与块格式（Implicit Free List）

- **定义与目的**：为了知道"给一个指针该释放多少内存""下一个块在哪""这块空不空"，每个块前面放一个**头部（header）**，记录**块大小**与**分配标志**。头部+大小串起来就成了一条**隐式空闲链表（implicit free list）**：不需要显式 next/prev 指针，靠"长度"隐式串联所有块。
- **标准技巧**：与其用两个字分别存大小和状态（浪费），不如利用**对齐**——块大小是 16 的倍数时低 4 位恒为 0，于是把最低位当作 `a`（allocated）标志；读大小时必须屏蔽它（`header & ~0xf`）。
- **块格式图解**（对齐的位域划分，1 word = 8 字节）：

```
                        一个 head word (8 字节 = 64 位)
  63                                                          4  3  2  1  0
+--------------------------------------------------------------+--+--+--+--+
|                        Size (块总大小, 16 的倍数)             |0 |0 |0 | a|
+--------------------------------------------------------------+--+--+--+--+
   a = 1: 已分配块          a = 0: 空闲块
   Size: 含头部与填充的总块大小 (字节)

   已分配块:  [ Size | a ] [ payload ................. ] [ 可选 padding ]
   空闲块  :  [ Size | a ] [ 空闲 (可放 next/prev 指针) ] [ 可选 padding ]
               ^8 字节

   双字对齐(double-word aligned):
   heap 起点 16 字节对齐 => 每个块的头部地址都是 8 (mod 16),
   因此 payload 起点必为 16 的倍数。注意:
   **头部本身不在对齐位置, payload 才对齐。**
```

- **与机器码/硬件的对应**（真实的 `gcc -O2 -S` 输出，读头部只需一条 `and`）：

```
get_size:                       get_alloc:
        movq    (%rdi), %rax            movq    (%rdi), %rax
        andq    $-16, %rax              andl    $1, %eax
        ret                             ret

init_header(b, size, alloc):    find_next(b):   /* b + get_size(b) */
        orq     %rdx, %rsi              movq    (%rdi), %rax
        movq    %rsi, (%rdi)            andq    $-16, %rax
        ret                             addq    %rdi, %rax
                                        ret
```

注意 `andq $-16` 就是 `& ~0xf`——一条指令同时完成"提取大小"与"屏蔽标志位"。

- **讲义给出的数据结构和访问函数**（`offsetof` 与零长数组是关键技巧）：

```
typedef uint64_t word_t;
typedef struct block
{
  word_t header;
  unsigned char payload[0];   /* GNU C 零长数组: sizeof(payload) == 0 */
} block_t;

/* 从块指针得到载荷指针 */
return (void *) (block->payload);
/* 从载荷指针反推块指针 (offsetof 给出成员偏移) */
return (block_t *) ((unsigned char *) bp - offsetof(block_t, payload));
/* 读标志位 / 读大小 / 写头部 */
return header & 0x1;            /* get_alloc */
return header & ~0xfL;          /* get_size  */
block->header = size | alloc;   /* write_header */
```

补充说明：`payload[0]` 是 GCC 扩展（C99 的正式写法是柔性数组成员 `payload[]`）。它的价值在于"结构体固定部分只有头部，载荷长度随块变化"——这正是分配器需要的类型双关技巧（见 `F25-rec07_slides.txt` 与 bootcamp 的"Zero-Length Arrays"）。

13.2.7 隐式空闲链表的整体布局（ASCII 图）

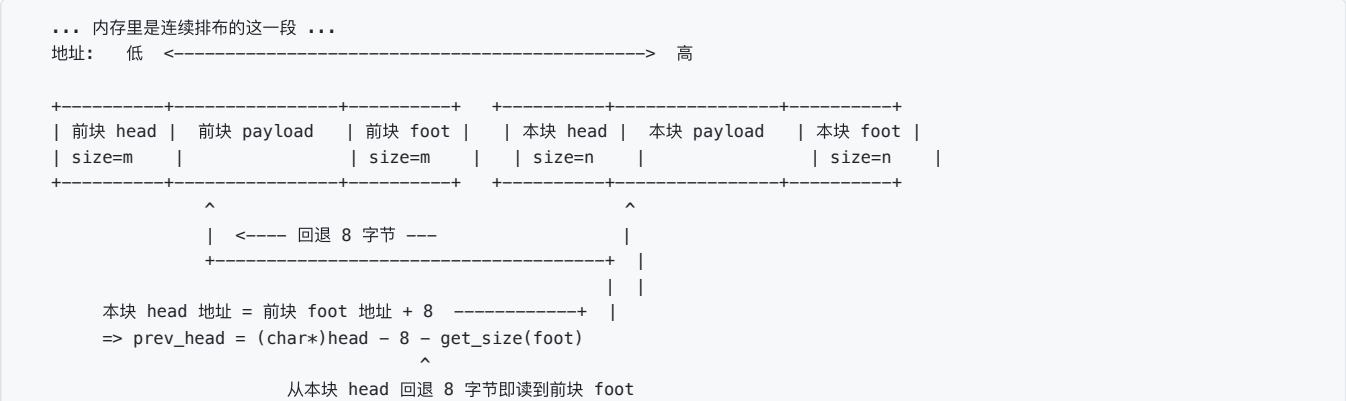
讲义给出的标准示例，头部标注为"**以字为单位的大小 / 分配位**"：

```
   heap_start (双字对齐)
+-------------+-------------+-------------+-------------+----------------+
| 16/0        | 32/1        | 32/1        | 64/0        | 8/1            |
| 空闲块(2字) | 已分配(4字) | 已分配(4字) | 空闲块(8字) | 结尾块(1字)    |
+-------------+-------------+-------------+-------------+----------------+
^                                                                        heap_end
 序言/填充块

  规则:
  * 已分配块: 涂阴影 (shaded); 空闲块: 不涂。
  * 头部写在"非对齐位置" (8 mod 16), 所以 payload 才是 16 字节对齐的。
  * 走链表: next = (char *)block + get_size(block)   ← 唯一的"指针"
  * 结尾块 8/1 (size=8, allocated) 充当哨兵, 使遍历不必判断越界。
```

- **直观解释**：隐式空闲链表就像一本**只有页码、没有目录**的书。要找某段内容（某个大小的空闲块），你必须从第 1 页逐页翻——**这决定了它的分配操作是线性时间**。

13.2.8 边界标记与常量时间的合并（Boundary Tags）

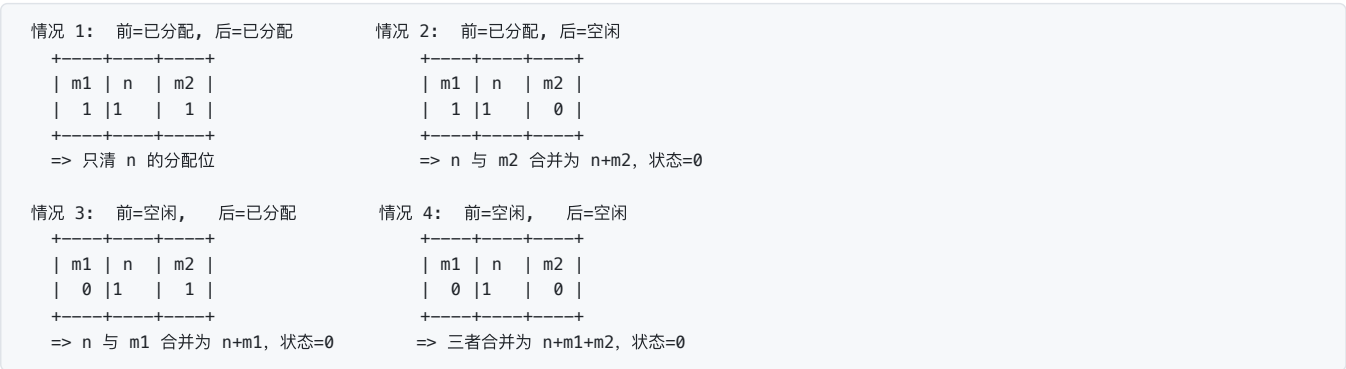
- **定义与目的**：最简单的 `free` 只需清掉分配位，但这会导致**假碎片（false fragmentation）**：明明有两块相邻的空闲内存，分配器却看不到它们是连续的。解决办法是**合并（coalescing）**。与后块合并很容易（`current + size`）；与**前块**合并却有个难题：**怎么知道前一个块的起点和它是否空闲?** 顺着链表走可以从头找到它，但那是 $O(n)$ 的。Knuth 的**边界标记（boundary tags）**技巧【Knuth73】：**在每个块的末尾再复制一份头部信息（脚部 footer）**。于是从当前块的头部**往回退 8 字节**，正好落在前一个块的脚部上，一次性读出前块的**大小**与**状态**——合并变成 $O(1)$。
- **为什么能 $O(1)$ 找到前驱（关键 ASCII 图）**：



- **代价**：**每个块多花 8 字节**，即内部碎片增加。这正是 bootcamp 里"**Eliminate footers in allocated blocks**"优化的动机：脚部只在需要被后一个块回溯时才需要，而只有**空闲块**才需要被合并，所以 13.2.10 会给出只用 2 个标志位（`b = 前块是否已分配`）省掉已分配块脚部的做法。
- **替代方案**：**延迟合并（deferred coalescing）**——`free` 时只标记不合并；等到 `malloc` 找不到合适块时再一次性扫描堆、合并所有相邻空闲块。它把合并成本从"每次 `free`"挪到"偶尔扫描"，代价是两次合并之间可能浪费内存。

13.2.9 合并的四种情况（Constant Time Coalescing）

设当前被释放的块大小为 n，前块 m_1、后块 m_2（记 `1`=已分配，`0`=空闲）：



- **伪代码**（与讲义一致，注意情况 4 中要**事先**保存后块指针）：

```
coalesce(bp):
    prev_alloc = footer(bp - 8) 的分配位      # 回退 8 字节读前块脚部
    next       = bp + size(bp)                # ★ 必须在 bp 被改写前算出来
    next_alloc = header(next) 的分配位
    size       = size(bp)
    if prev_alloc and next_alloc:      # 情况 1
        pass
    elif prev_alloc and not next_alloc:  # 情况 2
        size += size(next)
    elif not prev_alloc and next_alloc:  # 情况 3
        bp = bp - size(footer(bp - 8))   # 前块头部
        size += size(bp)
    else:                                # 情况 4
        bp = bp - size(footer(bp - 8))
        size += size(bp) + size(next)     # next 仍是"原来的后块"
    write_header(bp, size, FREE)
    write_footer(bp, size, FREE)
    return bp
```

- **与机器码/硬件的对应**：整个过程只有几次 `movq` + `and` + `add`，**没有任何循环**——这就是"常量时间合并"的含义；代价是每个块那 8 字节脚部，以及"每块至少 16 字节（head+foot）"的最小块大小。
- ⚠️ **真实踩坑（本章代码实测发现）**：情况 4 里如果先执行 `bp = prev_block(bp)` 再去算 `next_block(bp)`，`next_block` 就会以**前块**为基准走，得到一个完全错误的地址，进而写出损坏的脚部、让堆检查断言失败。这是实现 `coalesce` 时最容易犯的 bug 之一，务必先把 `next` 存成局部变量。

13.2.10 堆结构与序言/结尾块（Prologue & Epilogue）

- **问题**：`coalesce` 要读"前块的脚部"和"后块的头部"，但**堆的第一个块前面没有脚部、最后一个块后面没有头部**，会越界读写。
- **解法**：在 `mm_init` 里放置两个哨兵：

```
   heap_start (16 字节对齐)
+----------------+----------------+----------+   +-------------+----------------+
| 填充 8B        | 序言头 8B      | 序言脚 8B |   | 块0 ...     | 结尾头         |
| (对齐用)       | size=16, a=1   | size=16,1 |   |             | size=0, a=1    |
+----------------+----------------+----------+   +-------------+----------------+
                  ^                  ^                        ^
                  |                  |                        |
      序言块(已分配,永远不释放)   heap_listp 指向此处        遍历终止哨兵
      => 释放第一个真实块时, "前块" 读到的是序言脚部(已分配) -> 情况 1 或 2
      => 结尾头 size=0、alloc=1, 使 find_fit 的循环条件
         (get_size(header) > 0) 自然终止, 且释放最后一个块时 "后块" 是已分配
```

- **直观解释**：序言块是**起跑线前的挡板**，结尾头是**跑道尽头的墙**——`coalesce` 里那两次看似越界的访问因此都读到"合法的、标记为已分配的块"，四种情况的分支不需要任何 `if (bp == heap_start)` 之类的特判。bootcamp 把它表述为"载荷必须 16 字节对齐，但载荷大小不必是 16 的倍数、块大小必须是；所有 `malloc` 出来的东西都要落在序言与结尾之间"。
- **讲义里的等价做法**：幻灯片画的是"Dummy footer before first header（标记为已分配，防止释放第一个块时误合并）"+"Dummy header after last footer（防止释放最后一个块时误合并）"，与序言/结尾是同一件事的两种画法。

13.3 代码示例与底层机制分析

13.3.1 完整可运行的隐式空闲链表分配器（带边界标记与 $O(1)$ 合并）

代码 (C)：堆用一块 16 字节对齐的静态数组模拟（`heap_end` 扮演 `brk`），四个接口与 `mm.c` 同名，另加一个 `dump_heap` 打印每块大小与空闲状态并断言 `header == footer`。

```c
/* c13_alloc_demo.c  编译: gcc -g -Wall -std=c11 c13_alloc_demo.c -o c13_alloc_demo */
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <stddef.h>
#include <assert.h>

typedef uint64_t word_t;
static const size_t WSIZE = sizeof(word_t);      /* 8  : 一个字 */
static const size_t DSIZE = 2 * sizeof(word_t);  /* 16 : 头部+脚部 */
#define HEAP_CAP (1u << 16)

static _Alignas(16) unsigned char heap_area[HEAP_CAP];
static void *heap_start = heap_area;   /* 堆起点(16 对齐) */
static void *heap_end   = heap_area;   /* 当前 brk */

/* ---------------- 头部 / 脚部 访问 ---------------- */
static size_t get_size(void *b)  { return *(word_t *)b & ~0xFUL; }
static size_t get_alloc(void *b) { return *(word_t *)b & 0x1UL; }
static void write_tag(void *b, size_t size, size_t alloc)
{
    *(word_t *)b = (word_t)(size | alloc);
}
static void *next_block(void *b) { return (char *)b + get_size(b); }
/* 前一个块的头部: 从当前头部回退 8 字节正好读到前一个块的脚部 */
static void *prev_block(void *b) { return (char *)b - get_size((char *)b - WSIZE); }
static void *header_of(void *payload) { return (char *)payload - WSIZE; }
static void *payload_of(void *b)      { return (char *)b + WSIZE; }

/* ---------------- 打印整个堆 ---------------- */
static void dump_heap(const char *tag)
{
    printf("---- %s ----\n", tag);
    printf("  %-4s %-12s %-6s %-7s %s\n", "blk", "hdr_addr", "size", "alloc", "note");
    int i = 0;
    for (void *b = (char *)heap_start + WSIZE; b != heap_end; b = next_block(b), i++) {
        size_t sz = get_size(b), a = get_alloc(b);
        printf("  #%-3d %-12p %-6zu %-7s %s\n", i, b, sz, a ? "1" : "0",
               a ? "allocated" : "free(含脚部)");
        if (!a)  /* 堆不变量: 空闲块 header == footer */
            assert(*(word_t *)((char *)b + sz - WSIZE) == *(word_t *)b);
    }
    printf("\n");
}

/* ---------------- 立即合并: 四种情况, 都是 O(1) ---------------- */
static void *coalesce(void *b)
{
    size_t prev_alloc = get_alloc((char *)b - WSIZE); /* 前一个块的脚部 */
    void  *nextb      = next_block(b);                /* ★ 先记下后块! */
    size_t next_alloc = get_alloc(nextb);
    size_t size       = get_size(b);

    if (prev_alloc && next_alloc) {         /* 情况 1 */
        printf("  [coalesce] 情况1 前后都分配 -> 仅清标志, size=%zu\n", size);
    } else if (prev_alloc && !next_alloc) { /* 情况 2 */
        size += get_size(nextb);
        printf("  [coalesce] 情况2 只有后块空闲 -> 合并得 size=%zu\n", size);
    } else if (!prev_alloc && next_alloc) { /* 情况 3 */
        b = prev_block(b);
        size += get_size(b);
        printf("  [coalesce] 情况3 只有前块空闲 -> 合并得 size=%zu\n", size);
    } else {                                /* 情况 4 */
        b = prev_block(b);
        size += get_size(b) + get_size(nextb);
        printf("  [coalesce] 情况4 前后都空闲 -> 三者合并得 size=%zu\n", size);
    }
    write_tag(b, size, 0);
    write_tag((char *)b + size - WSIZE, size, 0); /* 新脚部 */
    return b;
}

/* ---------------- 分割 ---------------- */
static void split_block(void *b, size_t asize)
{
```

```
    size_t bsize = get_size(b);
    if (bsize - asize >= 2 * DSIZE) {          /* 余下部分至少能装一个最小块 */
        write_tag(b, asize, 1);
        write_tag((char *)b + asize - WSIZE, asize, 1);
        void *rest = (char *)b + asize;
        write_tag(rest, bsize - asize, 0);
        write_tag((char *)rest + (bsize - asize) - WSIZE, bsize - asize, 0);
        printf("  [split] %zu -> 分配 %zu + 空闲 %zu\n", bsize, asize, bsize - asize);
    } else {                                   /* 整块给出，产生内部碎片 */
        write_tag(b, bsize, 1);
        write_tag((char *)b + bsize - WSIZE, bsize, 1);
        printf("  [split] 剩余 %zu < 最小块 %zu, 整块分配(内部碎片 %zu 字节)\n",
               bsize - asize, 2 * DSIZE, bsize - asize);
    }
}

/* ---------------- 首次适配 ---------------- */
static void *find_fit(size_t asize)
{
    for (void *b = (char *)heap_start + WSIZE; b != heap_end; b = next_block(b)) {
        if (!get_alloc(b) && asize <= get_size(b)) {
            printf("  [find_fit] 首次适配命中 %p (size=%zu >= %zu)\n", b, get_size(b), asize);
            return b;
        }
    }
    printf("  [find_fit] 遍历整个隐式链表, 无可用空闲块\n");
    return NULL;
}

/* ---------------- 扩展堆 (等价于 mem_sbrk) ---------------- */
static void *extend_heap(size_t bytes)
{
    if ((size_t)((char *)heap_end - (char *)heap_start) + bytes > HEAP_CAP) return NULL;
    void *b = heap_end;
    write_tag(b, bytes, 0);
    write_tag((char *)b + bytes - WSIZE, bytes, 0);
    heap_end = (char *)heap_end + bytes;
    write_tag(heap_end, 8, 1);      /* 新的结尾头部(8/1) */
    printf("  [extend] 扩堆 %zu 字节, 新空闲块 @ %p\n", bytes, b);
    return b;
}

/* ---------------- 四个对外接口 ---------------- */
static int mm_init(void)
{
    heap_end = heap_start;
    write_tag((char *)heap_start, 8, 1);        /* 序言脚部: 8/1 */
    heap_end = (char *)heap_start + WSIZE;
    write_tag(heap_end, 8, 1);                  /* 结尾头部: 8/1 */
    return 0;
}

static void *mm_malloc(size_t size)
{
    if (size == 0) size = 1;
    size_t asize = (size + DSIZE + 15) & ~(size_t)0xF;  /* 头部+脚部, 向上取整到16 */
    printf("  [malloc] 请求 %zu 字节 -> asize=%zu\n", size, asize);
    void *b = find_fit(asize);
    if (b == NULL) {
        size_t need = asize > 4096 ? asize : 4096;      /* 一次多要一点 */
        b = extend_heap((need + 15) & ~(size_t)0xF);
        if (b == NULL) return NULL;
    }
    split_block(b, asize);
    return payload_of(b);
}

static void mm_free(void *p)
{
    if (p == NULL) return;
    void *b = header_of(p);
    size_t size = get_size(b);
    printf("  [free] payload %p -> block %p, size=%zu\n", p, b, size);
    write_tag(b, size, 0);                          /* 1. 清分配位 */
    write_tag((char *)b + size - WSIZE, size, 0);
```

```
    coalesce(b);                                /* 2. 立即合并 */
}
```

最后的 `main` 正是本章的验证程序：它按顺序演示四种合并情况、`malloc(1)` 的内部碎片与 `realloc` 的搬移，每步之后打印整条链表。

```
static void *mm_realloc(void *p, size_t size)
{
    if (p == NULL) return mm_malloc(size);
    if (size == 0) { mm_free(p); return NULL; }
    void *nb = mm_malloc(size);
    if (nb == NULL) return NULL;
    size_t old = get_size(header_of(p)) - DSIZE;
    memcpy(nb, p, old < size ? old : size);
    mm_free(p);
    return nb;
}

/* ---------------- 内部碎片与利用率统计 ---------------- */
static size_t req_now = 0, req_peak = 0;
static void note_alloc(size_t s) { req_now += s; if (req_now > req_peak) req_peak = req_now; }
static void note_free(size_t s)  { req_now -= s; }

static void report(void)
{
    size_t heap_used = 0, payload = 0;
    for (void *b = (char *)heap_start + WSIZE; b != heap_end; b = next_block(b)) {
        heap_used += get_size(b);
        if (get_alloc(b)) payload += get_size(b) - DSIZE;
    }
    printf("  堆已用 %zu 字节; 已分配块的载荷(可用)合计 %zu 字节\n", heap_used, payload);
    printf("  聚合利用率 U_k = 已分配载荷/堆大小 = %zu/%zu = %.3f\n",
           payload, heap_used, (double)payload / (double)heap_used);
    printf("  峰值利用率   = 峰值载荷/堆大小 = %zu/%zu = %.3f\n",
           req_peak, heap_used, (double)req_peak / (double)heap_used);
    printf("  内部碎片合计 = 载荷 %zu - 当前请求 %zu = %zu 字节\n",
           payload, req_now, payload - req_now);
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("=== 隐式空闲链表分配器演示 (word=8B, 块大小为 16B 的倍数) ===\n\n");
    mm_init();

    printf("[1] 连续四次 malloc(32), 块大小 = (32+16) 向上取整 = 48\n");
    void *p1 = mm_malloc(32); note_alloc(32);
    void *p2 = mm_malloc(32); note_alloc(32);
    void *p3 = mm_malloc(32); note_alloc(32);
    void *p4 = mm_malloc(32); note_alloc(32);
    printf("    p1=%p p2=%p p3=%p p4=%p\n\n", p1, p2, p3, p4);
    dump_heap("四次 malloc 之后");

    printf("[2] free(p2): 前(p1)后(p3)都分配 -> 情况 1\n");
    mm_free(p2); note_free(32);
    dump_heap("free(p2) 之后");

    printf("[3] free(p1): 前是序言块(已分配), 后块(p2)空闲 -> 情况 2\n");
    mm_free(p1); note_free(32);
    dump_heap("free(p1) 之后: p1+p2 合成 96B 空闲块?");

    printf("[4] free(p3): 前块(96B 空闲), 后块(p4)已分配 -> 情况 3\n");
    mm_free(p3); note_free(32);
    dump_heap("free(p3) 之后: 合成 144B 空闲块?");

    printf("[5] free(p4): 前块空闲 + 后块(扩堆余量)空闲 -> 情况 4\n");
    mm_free(p4); note_free(32);
    dump_heap("free(p4) 之后: 与堆尾空闲块合并");

    printf("[6] p5 = malloc(100): asize=128, 首次适配命中该空闲块并分割\n");
    void *p5 = mm_malloc(100); note_alloc(100);
    printf("    p5=%p\n\n", p5);
    dump_heap("malloc(100) 之后");

    printf("[7] 演示内部碎片: malloc(1) 得到 32B 的块\n");
    void *p6 = mm_malloc(1); note_alloc(1);
    printf("    p6=%p (请求 1 字节, 块 32 字节 -> 载荷 16, 内部碎片 15)\n\n", p6);
    dump_heap("malloc(1) 之后");

    printf("[8] 内容保留验证: 先用已知模式填满 p5 的 100 字节\n");
    static unsigned char snapshot[100];
```

```
    memset(p5, 0xA5, 100);
    memcpy(snapshot, p5, 100);
    printf("    p7 = mm_realloc(p5, 200): 需要 asize=224 > 128, 必须搬移\n");
    void *p7 = mm_realloc(p5, 200);
    if (p7) { note_alloc(200); note_free(100); }
    printf("    p5=%p -> p7=%p (%s)\n", p5, p7, p5 == p7 ? "原地" : "已搬移");
    printf("    前 100 字节内容是否逐字节保留: %s\n\n",
           memcmp(p7, snapshot, 100) == 0 ? "是" : "否");
    dump_heap("realloc 之后");

    mm_free(p6); note_free(1);
    mm_free(p7); note_free(200);
    printf("[9] 释放 p6/p7 后做统计\n");
    report();
    printf("\n=== 所有 dump_heap 断言通过: 每个空闲块 header == footer ===\n");
    return 0;
}
```

【代码做什么？】

1. `mm_init` 在堆起始处写一个 `8/1` 的**序言脚部**，再写 `8/1` 的**结尾头部**，此时堆里没有任何可用空闲块。
2. 第一次 `mm_malloc(32)`：`asize = (32+16+15) & ~0xF = 48`；`find_fit` 扫不到空闲块 → `extend_heap(4096)` 造出 4096 字节空闲块 → `split_block` 切成"48 已分配 + 4048 空闲"。
3. 后续 `malloc` 直接命中那个大空闲块继续切分：**首次适配**总命中最靠前的空闲块，所以每次切下的位置紧跟前一块之后。
4. `mm_free(p)`：把该块头部与脚部都写成 `size | 0`，然后 `coalesce` 走四种情况之一，最后统一写新头部与新脚部。
5. `dump_heap` 顺序遍历所有块，打印 `size` 与 `alloc`，并断言**每个空闲块的头部与脚部完全一致**。

【底层机制透视】

- **`asize` 的计算是本讲的"块大小公式"**：`asize = (size + DSIZE + 15) & ~0xF`。本设计里已分配块也保留脚部，头部+脚部即 16 字节固定开销，再向上取整到 16 的倍数；公式同时**保证了 16 字节对齐**（块大小是 16 的倍数，且所有块起点都是 16 的倍数）。
- **`prev_block` 是整章的魔法所在**：`(char *)b - get_size((char *)b - WSIZE)`。它先读 `b-8` 处的脚部得到前块大小，再用这个大小回退到前块头部——这就是"边界标记让前驱在 $O(1)$ 内可找"的字面实现。
- **为什么一个 `for` 循环就能走完整个堆**：每个块自带长度，**隐式链表的"指针"就是长度本身**。

【内存布局 / 数据结构图解】（首次适配切分三次之后，真实地址来自 13.3.3 的实测输出）：

```
  heap_area (0x4050a8 起, 假设)
+--------------+--------------+--------------+--------------+--------------+----------+
| 序言脚 8/1   | #0: 48/1     | #1: 48/1     | #2: 48/1     | #3: 48/1     | 3904/0   |
| heap_start   | hdr@0x4050a8 | hdr@0x4050d8 | hdr@0x405108 | hdr@0x405138 | hdr@ ... |
+--------------+--------------+--------------+--------------+--------------+----------+
  ^              ^                                                          ^
  |              payload p1=0x4050b0 (16 对齐!)                              堆尾空闲块
  序言

  每个块内部 (以 #1 为例, 48 字节) :
  +----------+--------------------------+----------+
  | 48 | 1   | payload 32 字节          | 48 | 1   |   <- 已分配块也有脚部
  +----------+--------------------------+----------+
  0x4050d8    0x4050e0                   0x405100   0x405108

  空闲块 #4 (3904 字节) :
  +----------+----------------------------------------+----------+
  | 3904 | 0 | 空闲 (可放 next/prev 指针)             | 3904 | 0 |
  +----------+----------------------------------------+----------+
  0x405168                                            ...  0x4051a8
```

【与汇编 / 硬件的对应】：见 13.2.6 的 `gcc -O2 -S` 片段——`get_size` / `get_alloc` / `find_next` 各编译成 2–3 条指令，`write_header` 是 `orq` + `movq`。整个分配器里没有任何系统调用，唯一的"硬件交互"是写入堆内存引起的**缺页**（若该页尚未映射，由 Lecture 12 的缺页处理程序换入）。

【实测验证】：13.3.3 给出真实编译运行输出。

13.3.2 `mm.c` 风格的伪代码与真实片段（`mm_init` / `extend_heap` / `coalesce` / `mm_free`）

下面是 Malloc Lab 里真正要写的东西的骨架。与 13.3.1 的唯一区别是：用 `mem_sbrk` 扩展堆，并且用**结尾头 `size == 0` 作为遍历终止条件**（不要去和 `brk` 比较地址！）。

```
typedef unsigned long word_t;
#define WSIZE 8
#define DSIZE 16
#define CHUNKSIZE (1 << 12)
#define PACK(size, alloc) ((size) | (alloc))
#define GET(p)        (*(word_t *)(p))
#define PUT(p, val)   (*(word_t *)(p) = (word_t)(val))
#define GET_SIZE(p)   (GET(p) & ~0xFUL)
#define GET_ALLOC(p)  (GET(p) & 0x1UL)
#define HDRP(bp)      ((char *)(bp) - WSIZE)
#define FTRP(bp)      ((char *)(bp) + GET_SIZE(HDRP(bp)) - DSIZE)
#define NEXT_BLKP(bp) ((char *)(bp) + GET_SIZE(HDRP(bp)))
#define PREV_BLKP(bp) ((char *)(bp) - GET_SIZE((char *)(bp) - DSIZE))

static char *heap_listp;   /* 指向序言块的"载荷"（即序言脚部） */

/* 1) 初始化：填充 + 序言头 + 序言脚 + 结尾头，然后扩一次堆 */
int mm_init(void)
{
    if ((heap_listp = mem_sbrk(4 * WSIZE)) == (void *)-1) return -1;
    PUT(heap_listp, 0);                           /* 对齐填充 */
    PUT(heap_listp + WSIZE,     PACK(DSIZE, 1));  /* 序言头部 */
    PUT(heap_listp + 2 * WSIZE, PACK(DSIZE, 1));  /* 序言脚部 */
    PUT(heap_listp + 3 * WSIZE, PACK(0, 1));      /* 结尾头部 */
    heap_listp += DSIZE;                          /* 指向序言载荷 */
    if (extend_heap(CHUNKSIZE / WSIZE) == NULL) return -1;
    return 0;
}

/* 2) 扩堆：用 mem_sbrk 拿 words 个字，造一个新空闲块并合并 */
static void *extend_heap(size_t words)
{
    char *bp;
    size_t size = (words % 2) ? (words + 1) * WSIZE : words * WSIZE;  /* 16 的倍数 */
    if ((long)(bp = mem_sbrk((int)size)) == -1) return NULL;
    PUT(HDRP(bp), PACK(size, 0));            /* 覆盖旧结尾头 -> 新空闲块头 */
    PUT(FTRP(bp), PACK(size, 0));            /* 新空闲块脚部 */
    PUT(HDRP(NEXT_BLKP(bp)), PACK(0, 1));    /* 新的结尾头 */
    return coalesce(bp);                     /* 与前面的空闲块合并 */
}

/* 3) 合并：四种情况，O(1) */
static void *coalesce(void *bp)
{
    size_t prev_alloc = GET_ALLOC(FTRP(PREV_BLKP(bp)));  /* 读前块脚部 */
    size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKP(bp)));  /* 读后块头部 */
    size_t size = GET_SIZE(HDRP(bp));

    if (prev_alloc && next_alloc) {              /* 情况 1: 不合并 */
    } else if (prev_alloc && !next_alloc) {      /* 情况 2: 与后块合并 */
        size += GET_SIZE(HDRP(NEXT_BLKP(bp)));
        PUT(HDRP(bp), PACK(size, 0));
        PUT(FTRP(bp), PACK(size, 0));
    } else if (!prev_alloc && next_alloc) {      /* 情况 3: 与前块合并 */
        size += GET_SIZE(HDRP(PREV_BLKP(bp)));
        PUT(FTRP(bp), PACK(size, 0));
        PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0));
        bp = PREV_BLKP(bp);
    } else {                                     /* 情况 4: 前后都合并 */
        size += GET_SIZE(HDRP(PREV_BLKP(bp))) + GET_SIZE(HDRP(NEXT_BLKP(bp)));
        PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0));
        PUT(FTRP(NEXT_BLKP(bp)), PACK(size, 0));
        bp = PREV_BLKP(bp);
    }
    return bp;
}

/* 4) 释放：清两个标志位 + 立即合并 */
void mm_free(void *bp)
{
    size_t size;
    if (bp == NULL) return;                      /* free(NULL) 必须是空操作 */
    size = GET_SIZE(HDRP(bp));
    PUT(HDRP(bp), PACK(size, 0));                /* 1. 清分配位 */
    PUT(FTRP(bp), PACK(size, 0));                /*    脚部同步 */
```

```
    coalesce(bp);                               /* 2. 立即合并 */
}

/* 5) 首次适配：用"结尾头 size == 0"终止遍历 */
static void *find_fit(size_t asize)
{
    char *bp;
    for (bp = heap_listp; GET_SIZE(HDRP(bp)) > 0; bp = NEXT_BLKP(bp))
        if (!GET_ALLOC(HDRP(bp)) && asize <= GET_SIZE(HDRP(bp))) return bp;
    return NULL;
}
```

【代码做什么？】 这五个函数构成了 checkpoint 的"最小正确骨架"：`mm_init` 铺好哨兵、`extend_heap` 用 `mem_sbrk` 加内存并顺手合并、`find_fit` 线性搜索、`place` 分割、`mm_free` 清标志后合并。

【底层机制透视】 两个必须记住的细节：

- **`mem_sbrk` 而不是 `sbrk`**：lab 的 `memlib.c` 用一块 20 MB 的 `malloc` 区域模拟堆，并把"学生的分配器"与"libc 的 malloc"隔离开。直接用 `sbrk` 会和 libc 自己的堆管理打架。另外 `mem_sbrk` **只接受正数**（`incr < 0` 直接报错返回 `-1`）——所以 **Malloc Lab 里堆不能收缩**。
- **遍历终止条件是结尾头，不是地址比较**：`extend_heap` 每次都会在新堆顶写一个 `size=0, alloc=1` 的结尾头，`find_fit` 靠 `GET_SIZE(HDRP(bp)) > 0` 停下。若改成"`bp != brk`"之类的地址比较，一旦你自己维护的 `brk` 变量与 `mem_sbrk` 的实际返回值不同步（极易发生），就会走飞并死循环。

13.3.3 实测输出：合并全过程与利用率统计

编译与运行命令（真实执行）：

```
$ gcc -g -Wall -std=c11 c13_alloc_demo.c -o c13_alloc_demo && ./c13_alloc_demo
```

真实输出（节选，含四次 `malloc` 后的堆快照、四种合并情况、`malloc(1)` 的内部碎片、`realloc` 搬移、利用率统计）：

```
[1] 连续四次 malloc(32), 块大小 = (32+16) 向上取整 = 48
  [malloc] 请求 32 字节 -> asize=48
  [find_fit] 遍历整个隐式链表, 无可用空闲块
  [extend] 扩堆 4096 字节, 新空闲块 @ 0x4050a8
  [split] 4096 -> 分配 48 + 空闲 4048
  [malloc] 请求 32 字节 -> asize=48
  [find_fit] 首次适配命中 0x4050d8 (size=4048 >= 48)
  [split] 4048 -> 分配 48 + 空闲 4000
  ... (第三、四次类似)
    p1=0x4050b0 p2=0x4050e0 p3=0x405110 p4=0x405140

---- 四次 malloc 之后 ----
  blk  hdr_addr     size   alloc   note
  #0   0x4050a8     48     1       allocated
  #1   0x4050d8     48     1       allocated
  #2   0x405108     48     1       allocated
  #3   0x405138     48     1       allocated
  #4   0x405168     3904   0       free(含脚部)

[2] free(p2): 前(p1)后(p3)都分配 -> 情况 1
  [coalesce] 情况1 前后都分配 -> 仅清标志, size=48

[3] free(p1): 前是序言块(已分配), 后块(p2)空闲 -> 情况 2
  [coalesce] 情况2 只有后块空闲 -> 合并得 size=96
---- free(p1) 之后: p1+p2 合成 96B 空闲块? ----
  #0   0x4050a8     96     0       free(含脚部)
  #1   0x405108     48     1       allocated
  #2   0x405138     48     1       allocated
  #3   0x405168     3904   0       free(含脚部)

[4] free(p3): 前块(96B 空闲), 后块(p4)已分配 -> 情况 3
  [coalesce] 情况3 只有前块空闲 -> 合并得 size=144
---- free(p3) 之后: 合成 144B 空闲块? ----
  #0   0x4050a8     144    0       free(含脚部)

[5] free(p4): 前块空闲 + 后块(扩堆余量)空闲 -> 情况 4
  [coalesce] 情况4 前后都空闲 -> 三者合并得 size=4096
---- free(p4) 之后: 与堆尾空闲块合并 ----
  #0   0x4050a8     4096   0       free(含脚部)     <-- 整个堆合并回一块!

[6] p5 = malloc(100): asize=128, 首次适配命中该空闲块并分割
  [split] 4096 -> 分配 128 + 空闲 3968
    p5=0x4050b0

[7] 演示内部碎片: malloc(1) 得到 32B 的块
  [malloc] 请求 1 字节 -> asize=32
    p6=0x405130 (请求 1 字节, 块 32 字节 -> 载荷 16, 内部碎片 15)

[8] 内容保留验证: 先用已知模式填满 p5 的 100 字节
    p7 = mm_realloc(p5, 200): 需要 asize=224 > 128, 必须搬移
  [malloc] 请求 200 字节 -> asize=224
  [split] 3936 -> 分配 224 + 空闲 3712
  [free] payload 0x4050b0 -> block 0x4050a8, size=128
    p5=0x4050b0 -> p7=0x405150 (已搬移)
    前 100 字节内容是否逐字节保留: 是

[9] 释放 p6/p7 后做统计
  堆已用 4096 字节; 已分配块的载荷(可用)合计 0 字节
  聚合利用率 U_k = 已分配载荷/堆大小 = 0/4096 = 0.000
  峰值利用率    = 峰值载荷/堆大小 = 301/4096 = 0.073
  内部碎片合计 = 载荷 0 - 当前请求 0 = 0 字节

=== 所有 dump_heap 断言通过: 每个空闲块 header == footer ===
```

从输出里能读出的三件事：

1. **四种合并情况各被真实触发了一次**（步骤 2/3/4/5），且步骤 5 把整个堆合并回一个 4096 字节的空闲块——这证明边界标记的 $O(1)$ 双向合并正确。
2. **`malloc(1)` 得到 32 字节的块**，载荷只有 16 字节可用 → 内部碎片 $= 16 - 1 = 15$ 字节。这是"块大小公式 + 最小块约束"的必然结果，也是 Malloc Lab 里 `malloc` 小对象时利用率上不去的主因。
3. **`realloc` 会搬移**：`p5=0x4050b0 → p7=0x405150`，但前 100 字节被逐字节保留（`memcmp` 验证为"是"）。注意输出里 `[coalesce] 情况3 只有前块空闲` 出现在最后释放 `p6` 时——**搬移后留下的 128 字节空洞被后续 `free` 的合并吃掉了**，这正是边界标记带来的"空洞可回收"能力。

13.3.4 `mm.c` 风格实现在真实 `memlib.c` 上的运行结果

把 13.3.2 的代码配上 lab handout 的 `memlib.c` 真正编译运行（**注意 `memlib.c` 里用了 `getpagesize()`，需要 `-D_GNU_SOURCE`**）：

```
$ gcc -g -Wall -std=c11 -D_GNU_SOURCE c13_mm_impl.c memlib.c -o c13_mm && ./c13_mm
```

真实输出：

```
mm_init: 堆大小 = 4128 字节
  块 0 @ 0x7ff05b7ff020 size=16 alloc=1        <-- 序言块
  块 1 @ 0x7ff05b7ff030 size=4096 alloc=0      <-- extend_heap 造的空闲块
  => 2 个块, 其中 1 个空闲; 结尾头 size=0 alloc=1

四次 mm_malloc(32):
  p[0] = 0x7ff05b7ff030    p[1] = 0x7ff05b7ff060
  p[2] = 0x7ff05b7ff090    p[3] = 0x7ff05b7ff0c0
  块 1 @ 0x7ff05b7ff030 size=48 alloc=1
  块 2 @ 0x7ff05b7ff060 size=48 alloc=1
  块 3 @ 0x7ff05b7ff090 size=48 alloc=1
  块 4 @ 0x7ff05b7ff0c0 size=48 alloc=1
  块 5 @ 0x7ff05b7ff0f0 size=3904 alloc=0

mm_free(p[1]); mm_free(p[2]);  -> p[1] 与 p[2] 应合并为 96B 空闲块
  块 2 @ 0x7ff05b7ff060 size=96 alloc=0

mm_free(p[0]); mm_free(p[3]);  -> 最终与堆尾空闲块合成一大块
  块 1 @ 0x7ff05b7ff030 size=4096 alloc=0

q = mm_malloc(100) (asize=128, 首次适配 + 分割):
  q = 0x7ff05b7ff030
  块 1 @ 0x7ff05b7ff030 size=128 alloc=1
  块 2 @ 0x7ff05b7ff0b0 size=3968 alloc=0

堆一致性检查: PASS
堆大小 = 4128 字节, 载荷指针 16 字节对齐: 是
```

几个值得记住的数字：`0x7ff05b7ff030` 是 16 的倍数（16 字节对齐 ✓）；`0x7ff05b7ff060 − 0x7ff05b7ff030 = 0x30 = 48` 字节，与"块大小 48"完全吻合；序言块 16 字节（头+脚）在 `heap_listp` 之前，永远标记为已分配，因此**释放堆中第一个真实块时不会去和"不存在的"前块合并**。

13.3.5 首次适配 vs 最佳适配 vs 下次适配：真实实测对比

讲义给出的基准总开销是：**Perfect Fit 1.6% < Best Fit 8.3% < First Fit 11.9% < Next Fit 21.6%**。下面用同一个隐式空闲链表骨架、**同一条随机 trace**（4000 次 `malloc(8..2007)`，再隔一个 `free` 一个，然后用同分布再 `malloc` 2000 次）实测三种策略：

```
$ gcc -g -Wall -std=c11 c13_fitcompare.c -o c13_fitcompare && ./c13_fitcompare
```

真实输出：

```
first fit (首次适配):
  堆大小 = 4227080 字节, 扩堆次数 = 1032
  最终载荷 = 4059792 字节 -> 最终利用率 = 0.960
  峰值载荷 = 4011660 字节 -> 峰值利用率 = 0.949
  扫描块数合计 = 12795663  (越小吞吐率越高)

best fit (最佳适配):
  堆大小 = 4145160 字节, 扩堆次数 = 1012
  最终载荷 = 4058624 字节 -> 最终利用率 = 0.979
  峰值载荷 = 4011660 字节 -> 峰值利用率 = 0.968
  扫描块数合计 = 15625862  (越小吞吐率越高)

next fit (下次适配):
  堆大小 = 4677640 字节, 扩堆次数 = 1142
  最终载荷 = 4056624 字节 -> 最终利用率 = 0.867
  峰值载荷 = 4011660 字节 -> 峰值利用率 = 0.858
  扫描块数合计 = 4035516  (越小吞吐率越高)
```

结论（与讲义定性一致、并有量化证据）：

策略	复杂度	利用率	扫描量（吞吐率）	碎片倾向
首次适配 first fit	遍历到第一个足够大的块即停；最坏 $O(n)$	0.960	12.80 M	易在链表**开头堆积小碎块（splinters）**
最佳适配 best fit	必须扫完整个堆；最坏 $O(n)$，常数更大	**0.979（最好）**	**15.63 M（最慢）**	留下最小的剩余块 → 碎片小，但会产生**大量极小的不可用碎块**
下次适配 next fit	从上次结束处继续；常比 first fit 快	0.867（最差）	**4.04 M（最快，约 1/3）**	一些研究表明**碎片更严重**（实测堆最大、利用率最低）

为什么 next fit 扫描量只有 first fit 的 1/3，利用率却最差？ 因为它的游标一直在堆里"漂移"，把大空闲块切成零散的小块后就再也不回头用它们，导致堆不断被迫向 `sbrk` 要新内存。这正说明**吞吐率与利用率是一对冲突目标**，也是 Malloc Lab 的性能指数把两者加权（$w = 0.6$）的原因。

13.3.6 接口边界的真实行为（`malloc(0)`、`free(NULL)`、`errno`）

```c
/* c13_m0.c  编译: gcc -g -Wall -std=c11 c13_m0.c -o c13_m0 */
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
int main(void){
  void *p = malloc(0);
  printf("malloc(0) = %p\n", p);
  free(p);                      /* C 标准要求 malloc(0) 的返回值可被 free */
  free(NULL);                   /* 必须是空操作 */
  printf("free(NULL) 正常返回\n");
  errno = 0;
  void *q = malloc((size_t)1<<62);   /* 必然失败 */
  printf("malloc(2^62) = %p, errno=%d\n", q, errno);
  void *r = calloc(4, 8);
  unsigned char *c = r;
  int allz = 1; for (int i=0;i<32;i++) if (c[i]) allz = 0;
  printf("calloc(4,8) 全零: %s\n", allz?"是":"否");
  free(r);
  return 0;
}
```

真实运行输出：

```
malloc(0) = 0x6802a0
free(NULL) 正常返回
malloc(2^62) = (nil), errno=12 (ENOMEM)
calloc(4,8) 全零: 是
```

要点：① glibc 的 `malloc(0)` 返回**非 `NULL` 的可释放指针**（与讲义幻灯片的简化说法不同，属"补充说明"，也是 `mm_malloc(0)` 需要自己裁决的地方）；② `errno=12` 就是 `ENOMEM`，"失败返回 `NULL` **且**置 `errno`"两条都要满足；③ `calloc` 确实清零——注意它清零的是 `n*size` 字节，`n*size` 溢出是经典漏洞（现代 glibc 会检测并返回 `NULL`）。

13.4 实验关联

L5a Malloc Lab（checkpoint）：本讲是 checkpoint 的直接理论来源。

- **要写的四个函数**（`mm.c`，只提交这一个文件）：`mm_init`、`mm_malloc`、`mm_free`、`mm_realloc`，声明在 `mm.h` 里，**不能改接口**。
- **性能指数**：$P = wU + (1-w)\min\left(1, \dfrac{T}{T_{libc}}\right)$，$w = 0.6$，$T_{libc} = 600$ Kops/s（`config.h` 的 `AVG_LIBC_THRUPUT`）。**利用率占 60 分权重**，但吞吐率超过 libc 后不再加分——所以别为了省内存做到慢得离谱。
- **硬性规则（违反直接 0 分）**：不得调用任何内存管理相关的库函数/系统调用（`malloc` / `calloc` / `free` / `realloc` / `sbrk` / `brk` 及其变体）；**不得定义任何全局或静态的复合数据结构**（数组、结构体、树、链表），只允许标量全局变量——这正是 13.2.3 第 ③ 条"只能用堆区域、不能拿别的数据结构作弊"的落实：**所有元数据必须存在堆里**；必须返回 8 字节对齐的指针。
- **必须用 `mem_sbrk`**：`memlib.c` 用一块 `MAX_HEAP = 20 MB` 的区域模拟堆，提供 `mem_sbrk` / `mem_heap_lo` / `mem_heap_hi` / `mem_heapsize` / `mem_pagesize`；`mem_sbrk` 只接受正数，**堆不可缩小**。
- **`mdriver` 用法**（本讲必须会）：

```
$ make                                  # 编译出 mdriver / mdriver-dbg
$ ./mdriver -V                          # 最详细输出：逐 trace 诊断，调试时用
$ ./mdriver -v                          # 每个 trace 的性能表格
$ ./mdriver -V -f short1-bal.rep        # 只跑一个小 trace，开发期首选
$ ./mdriver -t <tracedir>               # 换 trace 目录
$ ./mdriver -l                          # 同时测 libc malloc 作为对照
$ gdb ./mdriver-dbg                     # 断言只在 -dbg 版本里生效（rec07 强调）
```

- **checkpoint 的关键交付物：堆一致性检查器（heap checker）**，`int mm_check(void)`（recitation 里也叫 `mm_checkheap(int verbose)`）。checkpoint 会**按堆检查器的质量给分**，建议的路线图顺序是：**0. 先写堆检查器 → 1. 实现 `coalesce_block()` → 2. 换显式空闲链表 → 3. 再做分离空闲链表 → 4. 进一步优化**（已分配块去脚部、降低最小块大小、压缩头部）。注意：**提交前把所有 `mm_check` 调用去掉**，否则吞吐率会被拖垮。
- **堆不变量（写检查器时逐条查）**：块级——header 与 footer 一致、载荷对齐且大小合法、**不存在相邻空闲块**（除非你用延迟合并）；链表级——next/prev 一致、空闲链表里不能有已分配块、所有空闲块都必须在链表里、无环（可用**龟兔赛跑 hare-and-tortoise** 算法检测）、分离链表中块大小落在正确 size class；堆级——所有块都在堆边界内、哨兵块正确。
- **常见坑（recitation 点名的三类现象）**：**Garbled bytes** = 覆盖了已分配块的数据；**Overlapping payloads** = 两个不同块的载荷在内存上重叠（几乎总是 size 算错或分割越界）；**segfault** = 访问非法地址。调试手段：`break mm.c:213`、`break find_fit if size == 24`、`watch heap_listp`、`backtrace` + `frame 1` 回看 `mm_malloc` 的局部变量、把 `printf` 换成条件编译的 `DBG_PRINTF` / `CHECKHEAP(...)`（用 `#define DEBUG` 一键开关）。
- **建议的下一步**：checkpoint 通过后再读 Lecture 14——显式空闲链表把"遍历所有块"变成"只遍历空闲块"（吞吐率），分离空闲链表让"找一个够大的块"更快、并近似实现最佳适配（利用率），footer 消除与最小块降到 16 字节则是利用率上最后的几个百分点。

本讲无直接关联的其他 lab，但为 L5b 与 Lecture 15（代码优化，分配器吞吐率是绝佳优化对象）提供前提。

13.5 常见错误与调试技巧

- **`free` 非 `malloc` 返回的指针**：`free(p + 8)`（中部指针）、`free(&stack_var)`、二次 `free(p)` 全是 UB——现象是之后某次 `malloc` 返回重叠地址或直接 SIGSEGV。**调试**：`valgrind --leak-check=full ./mdriver -f short1-bal.rep`；gdb 里 `watch *0x...` 盯住被破坏的头部。
- **`mm_malloc(0)` 未处理**：`asize` 算出 0 或 `find_fit(0)` 命中任意空闲块并把它切成 `0`，后续遍历遇到 `size == 0` 的块会**死循环**。**调试**：在 `find_fit` 里加 `assert(GET_SIZE(HDRP(bp)) > 0)`，用 `mdriver-dbg` 跑。
- **忘记在 `split_block` 后写新块的脚部**：头部对了、脚部是垃圾，下一次 `coalesce` 用前块脚部回退时会跳到任意地址。现象是**随机 segfault**，且只在使用首次适配时偶尔出现。**调试**：写堆检查器断言 `GET(HDRP(bp)) == GET(FTRP(bp))`（本章 `dump_heap` 就是这么做的），在每个 `malloc` / `free` 前后调用。
- **`coalesce` 情况 4 的指针次序错**（本章实测踩到的真 bug）：先把 `bp` 改成前块，再去算 `next_block(bp)`，就把前块当成了当前块。现象是合并出一个"大小比堆还大"的块，堆检查器立刻报 `header != footer`。**调试**：`printf("合并: bp=%p prev=%p next=%p\n", ...)`；gdb 里在小 trace 上单步 `coalesce_block`。
- **遍历不终止 / 死循环**：把 `for (bp = heap_listp; bp != brk; ...)` 当成终止条件，而自己维护的 `brk` 与 `mem_sbrk` 不同步。**正确做法**：用结尾头的 `size == 0` 终止。**调试**：`gdb ./mdriver -ex 'b find_fit' -ex run`，然后 `p/x *(unsigned long*)((char*)bp-8)` 看头部值。
- **非法使用 libc 函数**：在 `mm.c` 里 `#include <stdlib.h>` 后用 `malloc` / `memcpy`（`memcpy` 允许），或直接用 `sbrk`。现象是 lab 规则判 0 分或堆互相踩踏。**调试**：`nm mm.o | grep -E 'malloc|free|sbrk'` 检查未定义符号；只在必要处用 `mem_sbrk`。
- **对齐被破坏**：`asize` 忘了向上取整到 16，或用了 `ALIGN(size + 8)`（只加一个头，忘了脚部）。现象是 `mdriver` 报"payload not aligned"。**调试**：在 `mm_malloc` 返回前 `assert(((unsigned long)bp % 16) == 0)`；用 `p/x bp` 直接看低 4 位。
- **在 `free` 里写了已分配块之外的字节**：比如用 `GET_SIZE(HDRP(bp))` 之前 `bp` 已经被改。**调试**：`gdb` 的 `awatch *addr` 在任何访问（读或写）该地址时停下；`rwatch` 只在读时停——定位"谁改了这个头部"极其有效。

13.6 关键要点

- **动态分配的不可替代性来自两条需求**：编译期不知道大小、生命周期超出函数栈帧；分配器只能管理**堆**，`brk` 是它与内核的唯一接口（`sbrk` / `mem_sbrk`）。
- **分配器的五条硬约束中，最不可动摇的是"不能移动已分配块"**——正因为它，碎片（内部与外部）无法通过压缩消除，只能靠放置策略、分割策略与合并策略缓解。
- **吞吐率与峰值利用率天然冲突**：$U = \max_k P_k/H_n$ 度量前者之外的另一半，Malloc Lab 用 $P = wU + (1-w)\min(1, T/T_{libc})$（$w = 0.6$）把它们绑在一起，逼你找平衡点。
- **块头部的精髓是"把对齐留下的空位当标志位用"**：`header = size | a`，读大小时 `& ~0xf`。块大小是 16 的倍数 ⇒ 低 4 位可用于标志（`a`、`b`）。
- **边界标记（footer）是 $O(1)$ 双向合并的钥匙**：从当前头回退 8 字节即可读到前块脚部；代价是每块 8 字节。四种合并情况的伪代码必须背下来。
- **隐式空闲链表简单但分配是线性时间**：除特殊用途外实践中不用它做 `malloc` / `free`，但**分割（splitting）与边界标记合并（boundary tag coalescing）这两个概念对所有分配器都通用**，是 Lecture 14 一切优化的地基。

- **序言块与结尾头不是装饰，是"消边界特判"的工程手段**：`8/1` 的序言脚部让第一个块的前驱"合法且已分配"，`size=0` 的结尾头让遍历与合并有终止条件。

13.7 思考题（带答案）

题 1（计算题：算内部碎片、算利用率、手画链表布局）：某隐式空闲链表分配器（4 字节头、4 字节脚部、块大小 8 的倍数、最小块 16 字节、载荷需 8 字节对齐）依次执行：

```
p1 = malloc(1);  p2 = malloc(20);  p3 = malloc(12);  free(p2);  p4 = malloc(4);
```

(a) 每个块的**总大小**是多少？各块的**内部碎片**是多少？(b) 设堆除了这些块之外没有别的空闲空间浪费，画出 5 步之后堆的 ASCII 布局（标注每块的 `size/alloc`）；(c) 若 `free(p2)` 采用**立即合并**，`p4 = malloc(4)` 能否复用 `p2` 的空间？若采用"只清标志不合并"的简化 `free`，结果是否相同？

答：(a) 块大小 $=$ (请求 $+$ 头 $4+$ 脚 4) 向上取整到 8，且不小于 16 字节：

请求	计算	块大小	载荷可用	内部碎片
`malloc(1)`	$1+8=9\to 16$	16	8	$8-1=7$
`malloc(20)`	$20+8=28\to 32$	32	24	$24-20=4$
`malloc(12)`	$12+8=20\to 24$	24	16	$16-12=4$
`malloc(4)`	$4+8=12\to 16$	16	8	$8-4=4$

(b) `free(p2)` 后 p2 块（32 字节空闲）夹在两个已分配块中间，**情况 1**，只能原地标记为空闲：

```
 堆起点
+----------+----------+----------+----------+----------+
| 16/1 p1  | 32/0 p2  | 24/1 p3  | 16/1 p4  | 结尾 8/1 |
+----------+----------+----------+----------+----------+
  内部碎片 7    空闲        内部碎片4     内部碎片4
```

(c) 能。`malloc(4)` 需要 16 字节，`p2` 的空闲块是 32 字节：$32-16=16\geq$ 最小块 16，于是**分割**成"16/1 的 p4 + 16/0 的空闲块"。若 `free` 只清标志不合并，本小题结果**看起来相同**（因为 p2 前后都是已分配块，情况 1 本来就不合并）——**这正是"假碎片"难以察觉的原因**：只有相邻空闲块出现时差异才暴露（见题 3）。

题 2（计算题：算峰值利用率）：用讲义 `syn-array-short` 基准的数据：峰值聚合载荷 $\max_k P_k=90036$ 字节，最终 `Allocated` 回到 0。若你的分配器为了完成这 20 次请求把堆扩到了 $H_n=110000$ 字节，(a) 峰值利用率 U 是多少？(b) 开销 O_k 是多少？(c) 若每个已分配块平均多出 12 字节的头部+填充，估算这 1.5% 级别的内部碎片贡献了多少利用率损失，并解释为什么"Perfect Fit 1.6%"在实践中达不到。

答：(a) $U=\dfrac{\max_k P_k}{H_n}=\dfrac{90036}{110000}\approx 0.8185$（约 81.9%）。(b) 在峰值处 $\max_k P_k=90036$，开销 $O=\dfrac{H}{P_{\max}}-1=\dfrac{110000}{90036}-1\approx 0.2217$，即 **22.2%** 的堆空间没有装程序数据。(c) 峰值时有若干个已分配块，每个 12 字节 $\Rightarrow$ 若峰值时约 10 个块，则多耗约 120 字节，占 90036 仅 0.13%；**真正的损耗来自外部碎片（空闲但无法使用的洞）**，而不是这 12 字节的头部。讲义指出"Perfect Fit 1.6%（只含内部碎片与对齐）"在实践中达不到，原因正是 compaction **不允许**：已分配块一旦发出就不能移动，碎片只能靠放置/分割/合并策略缓解，永远消不掉；所以任何真实分配器的利用率都低于 Perfect Fit。

题 3（辨析题）："既然 `free(p)` 时把分配位清成 0 就够了，合并（coalescing）只是性能优化，不做也不会错。"这句话错在哪？

答：错在把**正确性**与**最优性**搞混。不合并**不会算错地址**，但会让分配器**在明明有足够连续空闲内存时仍然 `sbrk` 要新内存**——讲义把这叫**假碎片（false fragmentation）**：三个相邻空闲块 `32/0, 32/0, 16/0` 合计 80 字节，而 `malloc(40)`（需 40 字节块）在**首次适配**下逐个检查，每个都不够大，于是返回 `NULL` 或扩堆。对 Malloc Lab 这直接表现为**利用率暴跌**，并且违反堆不变量"不允许存在相邻空闲块"。所以合并不只是"更快"，而是**正确实现该策略的前提**（除非你显式选择延迟合并，并在 `malloc` 前统一清扫）。

题 4（辨析题）："我把堆的元数据（一个空闲块的表）放在 `mm.c` 里一个全局 `static` 结构体数组里，这样 `find_fit` 就是 $O(1)$ 了，比边界标记高明。"这错在哪？

答：错两处，而且都是致命的。**第一，违反 lab 规则**：编程规则明确禁止在 `mm.c` 里定义任何全局/静态**复合**数据结构（数组、结构体、树、链表），只允许标量全局变量——原因正是分配器的**元数据必须放在它所管理的堆里**，否则"用一个外部结构管理堆"就等于作弊，也破坏了"分配器只有堆可用"这条硬约束。**第二，逻辑上循环**：全局数组的大小是编译期固定的，而堆需要处理**任意大小、任意数量**的块；且你要在数组里存"哪些地址空闲"，这些地址本身就需要被分配器管理的内存来记录（否则数组会随块数增长而溢出）——这正是**显式空闲链表（把 `next` / `prev` 指针放进空闲块的载荷区里）** 的设计动机，也是 Lecture 14 的内容。**正确的"$O(1)$ 找块"不是靠外挂表，而是靠改变堆内数据结构**：显式链表只遍历空闲块，分离链表按 size class 分桶。

Lecture 14: 动态内存分配：高级 (Dynamic Memory Allocation: Advanced)

> **讲义对应**：CMU 15–213 Lecture 14 — Dynamic Memory Allocation: Advanced Concepts（素材：`F25-14-malloc-advanced.txt`） **教材对应**：CS:APP3e 第 9 章 9.9.5—9.9.14（显式空闲链表、分离适配、简单分离存储）与 9.10—9.12（垃圾回收、C 语言常见内存错误、小结） **关联 Lab**：L5b Malloc Lab（final）

14.1 概述

Lecture 13 的**隐式空闲链表（implicit free list）**结构极简、`free` 恒定时间，但 `malloc` 必须从头扫到尾（**线性时间**），故现实中无人使用。本讲要回答：**如何在不放弃边界标记合并（boundary tag coalescing）的前提下，把分配从 $O(\text{块数})$ 降到近似 $O(1)$，同时不浪费内存？**

两条正交技术：① **显式空闲链表（explicit free list）**——只把空闲块串成链表，搜索范围从"所有块"缩到"空闲块"；② **分离空闲链表（segregated free list）**——再按**大小类（size class）**切成一组链表，搜索范围进一步缩到一个桶。后半部分转向工程现实：`realloc`、延迟合并（deferred coalescing）、用最低位"偷"回脚部的 8 字节、glibc 的 `malloc`，以及 Malloc Lab 提分路线。

14.2 核心概念与底层机制图解

14.2.1 显式空闲链表（Explicit Free List）

- **定义与目的**：只用 `prev` / `next` 指针把**空闲块**串成双向链表，链表头（root）是全局变量。
- **直观解释**：隐式链表像**按顺序装订的书**，找空白页得逐页翻；显式链表像夹了一叠**便利贴**，直接指向各处空白页。
- **底层机制图解**：关键洞察是"**只跟踪空闲块，所以能用载荷区（payload）存指针**"——已分配块的载荷被应用占用，空闲块的载荷反正没人用。

```
            显式空闲链表的物理布局（a = 已分配位，size 为块总字节数）
低地址                                                                  高地址
+----------------+---------------------------------+----------------+
| size | a  | 载荷与填充（应用数据）                  | size | a  |  ← 已分配块：无指针
+----------------+---------------------------------+----------------+
       ^ payload 返回给应用

+----------------+---------------+---------------+----...----+----------------+
| size | a=0 |      next（可指向堆中任意位置）   |  prev      | 未用 | size | a=0 |
+----------------+---------------+---------------+----...----+----------------+
       ^                                        ^
       +--- 空闲块载荷区的前 16 字节被复用为指针 ---+

逻辑顺序（链表）：  空闲块 C ---> 空闲块 A ---> 空闲块 B ---> NULL
物理顺序（内存）：  A(已分配) B(空闲) C(已分配) D(空闲) ...
                    逻辑与物理顺序无关，这是显式链表的本质特征
```

- **为什么"偏移量"比"指针"省空间（补充说明）**：串联可用**绝对指针**（8 字节堆内地址）或**相对偏移**（4 字节堆基址下标）。教学堆只有几十 MB，4 字节偏移足够，一个空闲块可从 16 字节指针降到 8 字节偏移。代价是**偏移量要求堆基址在整个生命周期内不变**，跨 `mmap` 段时还要额外处理——"省空间但更脆"。
- **代价对比**：分配从"扫描全部块"变"扫描空闲块"，**内存越满加速比越大**；代价是每空闲块多 16 字节开销，以及 `free` 要做"从链表摘除（splice out）"。

14.2.2 插入策略：LIFO 与地址有序（Insertion Policy）

- **定义与目的**：新释放的块插到链表哪里？这**直接决定碎片程度**。
- **LIFO（last-in-first-out）**：插入头部，`block->next = freelist; freelist = block;`。**简单、常数时间**；但研究表明碎片比地址有序严重——刚释放的块被立刻复用，堆的"热区"反复被切碎。
- **FIFO（first-in-first-out）**：插入链表尾部，同样常数时间（维护尾指针），效果与 LIFO 接近，都不如地址有序。
- **地址有序（address-ordered）**：保持 $\text{addr}(\text{prev}) < \text{addr}(\text{curr}) < \text{addr}(\text{next})$。**碎片更低**（相邻块在链表中也相邻）；代价是**插入需搜索** $O(\text{空闲块数})$，但换来一个红利：**可以省掉脚部**（见 14.2.6）。
- **底层机制图解**："splice out"（摘除）与"coalesce"（合并）必须成对出现——被吞并的邻居要先从链表摘掉，否则出现悬空指针。

```
LIFO 释放的四种情况（Root 为链表头；□=已分配 ▨=空闲）

Case 1: 前后皆已分配                    Case 2: 后继空闲
  Before  Root -> ... -> [□]              Before  Root -> [▨] -> [□] -> [□]
  After   Root -> [▨] -> ...              After   Root -> [▨+□] -> [□]
  直接插入头部                             先摘除后继，再合并，插头部

Case 3: 前驱空闲                        Case 4: 前后皆空闲
  Before  Root -> [□] -> [□] -> ...       Before  Root -> [▨] -> [□] -> [▨] -> ...
                  ↑待释放
  After   Root -> [▨+□] -> ...            After   Root -> [▨+□+▨] -> ...
  摘除前驱，合并，插头部                   摘除两邻居，三块合一，插头部
```

- **伪代码（地址有序插入 + 合并，最常用组合）**：

```
/* 把空闲块 bp 按地址顺序插入链表，并完成与前后邻块的合并 */
insert_address_ordered(bp):
    prev = NULL
    curr = free_list_root
    while curr != NULL and curr < bp:          /* 找到第一个地址大于 bp 的节点 */
        prev = curr
        curr = next(curr)
    /* --- 此时有 prev < bp < curr（地址意义上） --- */
    set_next(bp, curr); set_prev(bp, prev)
    if prev != NULL: set_next(prev, bp) else: free_list_root = bp
    if curr != NULL: set_prev(curr, bp)
    coalesce(bp)                               /* 顺手检查物理相邻的 prev/curr 是否空闲 */

coalesce(bp):                                  /* 边界标记法：用 size 而非指针定位邻居 */
    if not allocated(prev_block(bp)):          /* 物理前驱空闲 */
        remove_from_list(prev_block(bp))       /* ← 必须摘除，否则悬空 */
        bp = merge(prev_block(bp), bp)
    if not allocated(next_block(bp)):          /* 物理后继空闲 */
        remove_from_list(next_block(bp))
        bp = merge(bp, next_block(bp))
    return bp
```

14.2.3 分离空闲链表（Segregated Free List）

- **定义与目的**：维护**一组**空闲链表，每条只装某个**大小类（size class）**范围内的空闲块；先在目标类里找，找不到再去更大的类。
- **直观解释**：**图书馆按书脊厚度分书架**——找 200 页的书直接去"150–250 页"那一格，没有才去更厚的一格。**在窄范围里取第一块，天然接近"最佳适配"**。
- **大小类划分示例**（讲义的两种常见选择）：
- 小尺寸**逐档列举**：`{16} {32} {48} {64} {80} {96} {112} {128}`；
- 超过阈值后**按 2 的幂分段**：`{129–256} {257–512} {513–1024} {1025–2048} {2049–4096} {4097–∞}`；
- 最后一个类**必须无上界**（严格说是 2^{64}），因为总可能出现超大请求。

```
分离空闲链表的"大小类桶"结构（首适配 + 分离链表，教材推荐方案）

segs[0..N-1]：N 个链表头（全局变量；lab 只给 128 字节可写全局空间！）
+----------+
| segs[0] |--> [48B ▨] <--> [48B ▨] --> NULL     类 0：size == 48
+----------+
| segs[1] |--> [64B ▨] --> NULL                  类 1：size == 64
+----------+
| segs[2] |--> [96B ▨] <--> [80B ▨] --> NULL     类 2：65..96
+----------+
|  ...    |
+----------+
| segs[k] |--> [512B ▨] --> [300B ▨] --> NULL    类 k：257..512
+----------+
| segs[N-1]|--> [1MB ▨] --> [65536B ▨] --> NULL  无上界类：4097..∞
+----------+

分配（asize = 对齐后的块大小）：
  for c = class(asize) to N-1:            ← 从目标类起，逐类向"更大的类"推进
      for bp in segs[c]:                  ← 类内首次适配
          if size(bp) >= asize: 分割并返回 bp 的载荷
  都没找到 -> sbrk 扩展堆，切出 asize，余量放进"合适的类"
```

- **分割细节**：找到的块 $m \geq n$，余量够放一个最小块就**分割（split）**，把碎片放回**它自己所属的类**（不是原类！）；余量太小则整块给出，接受内碎片。
- **为什么近似 $O(1)$**：按 2 的幂分段时类数量是 $\log_2(\text{max size})$，逐类向后 $O(\log)$；类内首次适配则单类搜索是常数。**极端情况**：每尺寸一类即等价于**最佳适配（best fit）**，利用率最高但链表头数组大到不可接受——"利用率 vs. 空间/时间"的经典权衡。

14.2.4 分离链表的两种组织方式

组织方式	是否分割	是否合并	特点
简单分离存储（simple segregated storage）	**不分割**：请求 n 就整个给出该类的**固定块大小**	**不合并**（块大小相同、位置任意，不需要邻居信息）	分配/释放都 $O(1)$ 且**无需边界标记**；内碎片可达 2×。只用**单向链表**，故省掉 `prev`
分离适配（segregated fit）	**可分割**：从大块切出所需部分，余量回归相应类	**可合并**：保留头部/脚部，与邻居合并	利用率明显更高，是**教材核心推荐方案**；代价是边界标记 + 双向链表

- **如何用单向链表省掉 `prev`**：简单分离存储里每个类中的块**大小完全相同**，所以释放就是压入对应类的单向链表头（`bp->next = segs[c]; segs[c] = bp;`），分配就是弹出链表头。**全程不需要 `prev`**，也不需要脚部（永不合并），每个空闲块只花 8 字节指针。代价是必须**以 chunk 为单位向系统要内存**，且块大小取整带来可观内碎片——所以它适合分配尺寸高度集中的负载（内核对象缓存、固定尺寸网络缓冲区）。

14.2.5 四类分配器总对比

设 F = 空闲块数，B = 总块数，C = 大小类数。下表把本讲与上一讲的技术放在同一标尺上（"内存开销"按每块额外字节计）：

分配器	分配时间	释放时间	合并时间	内存开销（每块）
隐式空闲链表（implicit）	$O(B)$：从头线性扫描**所有块**	$O(1)$：边界标记四类 case	立即合并，$O(1)$	4 字节 header + 4 字节 footer
显式空闲链表（explicit）	$O(F)$：只扫描**空闲块**，堆越满越快	$O(1)$（LIFO/FIFO）或 $O(F)$（地址有序插入）	立即合并，$O(1)$，但需**摘除**被吞并的邻居	8 字节 header + 8 字节 footer + 空闲块内 16 字节 `next` / `prev`
分离适配（segregated fit）	$O(\log C + F_{\text{class}})$：先在目标类内首适配，再逐类向后	找类 $O(\log C)$ + 插入 $O(1)$ 或 $O(F)$	立即合并 + 把结果插入**新的**大小类	同显式链表，外加 N 个链表头（lab 只允许 128 字节全局空间）
简单分离存储（simple segregated storage）	$O(1)$：弹出对应类的链表头	$O(1)$：压入对应类的链表头	**不合并**（无边界标记）	8 字节 header（**无 footer**），空闲块内仅 8 字节 `next`（**无 `prev`**）

读表要点：① 前三者都用**边界标记合并**，因此都能在 $O(1)$ 内完成 `free`；差别全在**分配**的搜索上。② 简单分离存储的"快"是用**不合并 + 块大小取整**换来的——它的内碎片可达 2 倍（请求 33 字节要到 64 字节的类），所以只适合固定尺寸负载。③ 分离适配是唯一在**三个维度上都不差**的方案，这正是教材推荐它、Malloc Lab 也以它为高分基线的原因。

14.2.6 省掉脚部：三种技巧

脚部存在的唯一理由是 `free` 时要**向前**找到前驱块的大小。若"前一块是否已分配"变成 $O(1)$ 可读的 1 bit，脚部就能只在**空闲块**里保留。

- **技巧 A：用块大小的低位存分配位**。16 字节对齐保证块大小与地址的低 3 位恒为 0，header 里可白塞 3 个标志位。

```
一个 8 字节 header 的位域划分（16 字节对齐 → 低 4 位全可用，这里用 2 位）
 63                                    4  3  2  1  0
+---------------------------------------+--+--+--+--+
|           块大小 size（≥16，低 4 位恒 0）|  |  |  |  |
+---------------------------------------+--+--+--+--+
                                        |  |  |
                        bit0 = 本块已分配（ALLOCATED）
                        bit1 = 前一块已分配（PREV_ALLOC）
                        bit2/bit3 = 备用（讲义 bootcamp 提到 4 字节 header 压缩时用）

于是：has_prev_footer(bp)  等价于  !(header(bp) >> 1 & 1)
     prev_block(bp)        = bp − size(footer(bp − 8))   当且仅当 bit1 == 0
```

- **技巧 B：用地址有序性推断前驱状态**。若链表按地址排序，`bp` 的 `prev` 节点就是地址刚好小于 `bp` 的空闲块，于是 `free(bp)` 时 **若 `prev + size(prev) == bp`，前驱物理相邻且必然空闲**，可即刻合并，无需读任何脚部。代价是插入变成 $O(\text{空闲块数})$ 的搜索。
- **技巧 C：CS:APP 的 `PREV_ALLOC` 位（推荐）**。在 header 的 bit1 中记录"**物理前一块是否已分配**"。释放 `bp` 时若 bit1 表明前驱空闲，就用 `prev_block(bp) = bp − size(*(bp − 8))` 读**前驱脚部**（前驱空闲故有脚部）来定位；若前驱已分配，**根本不读那一字节**。于是：

```
已分配块：[ header | payload ... ]                      ← 无脚部，省 8 字节
空闲块  ：[ header | next | prev | ... | footer ]       ← 保留脚部

free(bp) 的位运算：
  if (header(bp) & PREV_ALLOC) == 0:      /* 前驱空闲 */
      pp = bp - size(*(bp - 8))           /* 借前驱的脚部找到它 */
      摘除 pp，合并
  否则：                                  /* 前驱已分配 */
      什么都不读 —— 脚部被彻底省掉

释放后还要顺手更新"后一块"的 PREV_ALLOC 位，这是最容易漏掉的一步！
```

- **收益量化（rec08 的例子）**：`malloc(24)` 带脚部时 $24+8+8=40$ 取整到 16 倍数 → **48 字节**，放不进 32 字节的块；去掉脚部后 $24+8=32$ → **正好放下**。但**去掉脚部不降低最小块**——空闲块仍需 header+next+prev+footer = 32 字节。要真正降低最小块得再砍一个字段（简单分离存储砍 `prev`，或地址有序 + 省脚部砍到 24/16 字节）。

14.2.7 `realloc` 的实现（Reallocation）

- **语义（lab writeup 口径）**：`realloc(NULL,n)` ≡ `malloc(n)`；`realloc(p,0)` ≡ `free(p)`；否则新块内容为旧块前 $\mathbf{min(old, new)}$ 字节，其余未初始化。
- **三条实现路径**：

```
① 原地缩小（shrink in place）：new ≤ old
   [ header | payload(旧) ..................... ]
     └─> [ header | payload(新) | ▨ 余量块 ]          余量 ≥ 最小块才分割，挂回空闲链表
   吞吐率与利用率双赢：不搬数据，立刻回收空间

② 原地扩展（grow in place）：new > old 且物理后继块空闲且 csize + size(next) ≥ asize
   Before: [ header | payload(旧) ][ ▨ next 空闲 ]
   After : [ header | payload(新，地址不变) ][ ▨ 余量 ]（余量还回链表）
   收益：省一次 memcpy，且地址不变（对调用方缓存友好）

③ 搬移（move）：malloc(new) + memcpy(min(old,new)) + free(old)
   必做不可时的兜底；注意 memcpy 的长度必须取 min，绝不能直接 memcpy(new 的大小)
```

- **为什么"独立的 `realloc` "是提分关键**：若写成 `p2 = malloc(n); memcpy; free(p)`，在 `realloc` 密集的 trace（`realloc-bal`、`realloc2-bal`）上会不断搬移，吞吐率低且堆被撑大；实现路径 ① ② 后两个 trace 都大幅改善。

14.2.8 延迟合并（Deferred Coalescing）

- **定义与目的**：`free` 时**只标记空闲并插入链表，不与邻居合并**；等 `malloc` **找不到可用块**时才集中做一轮全堆合并，再试一次。
- **直观解释**：像收拾房间——不是碰掉一张纸就归位，而是一律丢进"待整理箱"，满了再一次性分类。
- **与立即合并的取舍**：

	立即合并	延迟合并
`free` 代价	常数时间，但要做 3 次链表的摘除/插入	更简单（只标记 + 插入一次），但**失去"立刻复用相邻空间"的机会**
碎片	较低，堆更紧凑	略高：合并前可能看不出有空闲大块
实现复杂度	四种 case 都要处理	`free` 极简；但**堆检查器必须允许"相邻空闲块共存"**，否则会误报
适用	通用分配器（glibc 介于两者之间）	对 `free` 吞吐极敏感、分配大小较规律的场景

- **Malloc Lab 的坑**：实现了延迟合并却忘了在 `mm_checkheap` 里**放宽"no contiguous free blocks"**这条不变量，堆检查器会把合法状态报成错误。bootcamp 明确点出："no contiguous free blocks **unless you defer coalescing**"。

14.3 代码示例与底层机制分析

14.3.1 完整实现：分离空闲链表分配器（含 `mm_malloc` / `mm_free` / `mm_realloc`）

代码 (C)：

```c
/* mmseg.c · 分离空闲链表分配器（16B 对齐 / 已分配块无脚部 / PREV_ALLOC 位）
 * 已分配：[header 8B | payload...]   空闲：[header | next | prev | ... | footer]
 * header：bit0 = 本块已分配；bit1 = 前一块已分配 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>

#define WSIZE 8
#define DSIZE 16
#define CHUNK (1 << 12)          /* 一次多要 4 KB，摊薄 sbrk 开销 */
#define ALIGNMENT 16
#define MINBLK 32                /* header+next+prev+footer */
#define NCLASS 16

#define MAX(a, b)       ((a) > (b) ? (a) : (b))
#define ALIGN(s)        ((((size_t)(s)) + (ALIGNMENT-1)) & ~(size_t)(ALIGNMENT-1))
#define PACK(sz, a, pa) ((size_t)(sz) | (size_t)(a) | ((size_t)(pa) << 1))

static char   heap[1 << 20] __attribute__((aligned(16)));
static size_t heap_size = 0;
static char  *heap_end = NULL;   /* 结尾块 */
static void  *segs[NCLASS];      /* 各大小类链表头 */

static void *mem_sbrk(size_t incr) { void *p = heap + heap_size; heap_size += incr; return p; }

/* 块原语 */
static size_t get(void *p)             { return *(size_t *)p; }
static void   put(void *p, size_t v)   { *(size_t *)p = v; }
static size_t blk_size(void *bp)       { return get(bp) & ~(size_t)0xF; }
static int    blk_alloc(void *bp)      { return (int)(get(bp) & 0x1); }
static int    blk_prev_alloc(void *bp) { return (int)((get(bp) >> 1) & 0x1); }
static void   blk_set(void *bp, size_t sz, int a, int pa) { put(bp, PACK(sz,a,pa)); }
static void  *blk_footer(void *bp)     { return (char *)bp + blk_size(bp) - WSIZE; }
static void  *payload(void *bp)        { return (char *)bp + WSIZE; }
static void  *hdr_of(void *p)          { return (char *)p - WSIZE; }
static void  *next_blk(void *bp)       { return (char *)bp + blk_size(bp); }
static void  *NEXTP(void *bp)          { return (char *)bp + WSIZE; }  /* next */
static void  *PREVP(void *bp)          { return (char *)bp + DSIZE; }  /* prev */
static void  *prev_blk(void *bp)       { return (char *)bp - (get((char *)bp - WSIZE) & ~(size_t)0xF); }

/* 大小类：32..128 各一类，其后按 2 的幂分段 */
static int size_class(size_t size)
{
    if (size <= 128) return (int)(size / 16) - 2;      /* 32->0 ... 128->6 */
    size_t s = (size - 1) >> 7; int k = 0;
    while (s > 1) { s >>= 1; k++; }
    int c = 7 + k;
    return c >= NCLASS ? NCLASS - 1 : c;
}
static size_t class_lo(int c)   /* 某大小类的下限，仅用于调试打印 */
{
    if (c <= 6) return (size_t)(c + 2) * 16;
    return ((size_t)1 << (c - 7)) * 128 + 1;
}

/* 空闲链表：LIFO 双向 */
static void list_insert(void *bp)
{
    int   c = size_class(blk_size(bp));
    void *head = segs[c];
    *(void **)NEXTP(bp) = head;
    *(void **)PREVP(bp) = NULL;
    if (head) *(void **)PREVP(head) = bp;
    segs[c] = bp;
}
static void list_remove(void *bp)
{
    int   c  = size_class(blk_size(bp));
    void *pr = *(void **)PREVP(bp);
    void *nx = *(void **)NEXTP(bp);
    if (pr) *(void **)NEXTP(pr) = nx; else segs[c] = nx;
    if (nx) *(void **)PREVP(nx) = pr;
}
```

```
/* 合并：借 PREV_ALLOC 位发现前驱 */
static void *coalesce(void *bp)
{
    size_t size = blk_size(bp);
    if (!blk_prev_alloc(bp)) {              /* 前块空闲：摘除 + 合并 */
        void *pp  = prev_blk(bp);
        int   ppa = blk_prev_alloc(pp);
        list_remove(pp);
        size += blk_size(pp);
        bp = pp;
        blk_set(bp, size, 0, ppa);
    } else {
        blk_set(bp, size, 0, 1);
    }
    void *nb = (char *)bp + size;
    if (blk_size(nb) > 0 && !blk_alloc(nb)) {   /* 后块空闲（排除结尾块） */
        list_remove(nb);
        size += blk_size(nb);
        blk_set(bp, size, 0, blk_prev_alloc(bp));
    }
    put(blk_footer(bp), size);
    void *after = (char *)bp + size;            /* 更新后块的 PREV_ALLOC = 0 */
    blk_set(after, blk_size(after), blk_alloc(after), 0);
    list_insert(bp);
    return bp;
}

static void *extend_heap(size_t bytes)
{
    size_t asize = ALIGN(bytes < MINBLK ? MINBLK : bytes);
    /* 关键：新块 header 落在旧 epilogue 上，把它"顶掉" */
    char  *bp = (char *)mem_sbrk(asize) - WSIZE;
    int    pa = blk_prev_alloc(heap_end);
    blk_set(bp, asize, 0, pa);
    put(blk_footer(bp), asize);
    heap_end = bp + asize;
    blk_set(heap_end, 0, 1, 0);                 /* 新结尾块 */
    return coalesce(bp);
}

/* 首次适配 + 分离链表 */
static void *find_fit(size_t asize)
{
    for (int c = size_class(asize); c < NCLASS; c++)
        for (void *bp = segs[c]; bp; bp = *(void **)NEXTP(bp))
            if (blk_size(bp) >= asize) return bp;
    return NULL;
}

static void split_alloc(void *bp, size_t asize)
{
    size_t csize = blk_size(bp);
    int    pa    = blk_prev_alloc(bp);
    list_remove(bp);
    if (csize - asize >= MINBLK) {              /* 分割，余量放回自己的类 */
        blk_set(bp, asize, 1, pa);
        void *rb = (char *)bp + asize;
        blk_set(rb, csize - asize, 0, 1);
        put(blk_footer(rb), csize - asize);
        void *after = (char *)rb + (csize - asize);
        blk_set(after, blk_size(after), blk_alloc(after), 0);
        list_insert(rb);
    } else {                                    /* 余量太小，整块给出 */
        blk_set(bp, csize, 1, pa);
        void *after = (char *)bp + csize;
        blk_set(after, blk_size(after), blk_alloc(after), 1);
    }
}

static void shrink_alloc(void *bp, size_t asize)
{
    size_t csize = blk_size(bp);
    if (csize - asize < MINBLK) return;
    int pa = blk_prev_alloc(bp);
    blk_set(bp, asize, 1, pa);
```

```
    void *rb = (char *)bp + asize;
    blk_set(rb, csize - asize, 0, 1);
    put(blk_footer(rb), csize - asize);
    void *after = (char *)rb + (csize - asize);
    blk_set(after, blk_size(after), blk_alloc(after), 0);
    coalesce(rb);
}

int mm_init(void)
{
    memset(segs, 0, sizeof(segs));
    heap_size = 0;
    char *p = (char *)mem_sbrk(WSIZE + DSIZE + WSIZE); /* 垫块+序言块+结尾块 */
    if (p == NULL) return -1;
    put(p, 0);                        /* 垫块：首块地址 ≡ 8 (mod 16) */
    blk_set(p + WSIZE, DSIZE, 1, 1); /* 序言块（无脚部） */
    heap_end = p + WSIZE + DSIZE;
    blk_set(heap_end, 0, 1, 1);      /* 结尾块 */
    if (extend_heap(CHUNK) == NULL) return -1;
    return 0;
}

void *mm_malloc(size_t size)
{
    if (size == 0) return NULL;                /* 讲义：返回 NULL */
    size_t asize = ALIGN(size + WSIZE);
    if (asize < MINBLK) asize = MINBLK;
    void *bp = find_fit(asize);
    if (!bp) bp = extend_heap(MAX(asize, CHUNK));
    if (!bp) return NULL;                      /* 堆耗尽 */
    split_alloc(bp, asize);
    return payload(bp);
}

void mm_free(void *p)
{
    if (p == NULL) return;
    void  *bp   = hdr_of(p);
    size_t size = blk_size(bp);
    blk_set(bp, size, 0, blk_prev_alloc(bp));
    put(blk_footer(bp), size);
    coalesce(bp);
}

void *mm_realloc(void *p, size_t size)
{
    if (p == NULL) return mm_malloc(size);      /* == malloc(n) */
    if (size == 0) { mm_free(p); return NULL; } /* == free(p)   */
    void  *bp    = hdr_of(p);
    size_t csize = blk_size(bp);
    size_t asize = ALIGN(size + WSIZE);
    if (asize < MINBLK) asize = MINBLK;

    if (asize <= csize) { shrink_alloc(bp, asize); return p; }   /* ① 缩小 */

    void *nb = next_blk(bp);                                     /* ② 原地扩展 */
    if (blk_size(nb) > 0 && !blk_alloc(nb) && csize + blk_size(nb) >= asize) {
        list_remove(nb);
        size_t nsize = csize + blk_size(nb);
        blk_set(bp, nsize, 1, blk_prev_alloc(bp));
        void *after = (char *)bp + nsize;
        blk_set(after, blk_size(after), blk_alloc(after), 1);
        shrink_alloc(bp, asize);
        return p;
    }
    void *np = mm_malloc(size);                                  /* ③ 搬移 */
    if (!np) return NULL;                      /* 旧块必须保持有效！ */
    size_t copy = csize - WSIZE < size ? csize - WSIZE : size;
    memcpy(np, p, copy);
    mm_free(p);
    return np;
}
/* mm_checkheap 与测试 main 的输出见 14.3.2 */
```

【代码做什么？】

1. `mm_init`：要 24 字节写**垫块**（让首个真实块地址 $\equiv 8 \pmod{16}$，从而 `payload = bp + 8` 是 16 倍数）、**序言块**（16 字节、已分配、无脚部、永不释放，用于消灭 `coalesce` 的边界判断）与**结尾块**（size = 0、已分配）；再用 `extend_heap(4096)` 备好第一块空闲内存。
2. `mm_malloc`：`size + 8` 向 16 取整得 `asize`（< 32 取 32）。`find_fit` 从 `size_class(asize)` **逐类向后**扫（"首次适配 + 分离链表"）。命中后 `split_alloc` 摘除该节点，余量 ≥ `MINBLK` 就切两半并把**余量插入它自己的类**，否则整块给出。
3. `mm_free`：header 的 bit0 清零、补写脚部，然后 `coalesce`。`coalesce` 先看 **bit1（PREV_ALLOC）**：为 0 则前块空闲，用 `bp - *(bp-8)` 读前驱脚部定位、**从链表摘除**、合并；再查物理后继——因为已分配块**没有脚部**，只能靠 `bp + size` 定位，并用 `size(nb) > 0` 排除结尾块。收尾必须把**后一块的 PREV_ALLOC 位清零**，否则下次释放会去读不存在的前驱脚部。
4. `mm_realloc`：先试原地缩小，再试原地扩展，都不行才"分配 + 拷贝 + 释放"；`memcpy` 长度取 $\min(\text{csize} - 8,\ \text{size})$，且 `mm_malloc` 失败时必须**保持旧块有效**。

【底层机制透视】

- **位域复用为何必须靠对齐**：`blk_size` 用 `& ~0xF` 抹掉低 4 位，只有"所有块大小都是 16 的倍数"时才无损；也正因如此 `prev_blk` 才能直接用前驱脚部的大小值做减法。
- **`extend_heap` 的"顶掉 epilogue"技巧**：`mem_sbrk(n)` 返回新内存首字节，减 8 即**旧 epilogue 地址**；新块 header 写在这里正好覆盖它，再在 `bp + n` 写新 epilogue。若直接用 `mem_sbrk(n)` 当新块起点，就会留下 8 字节"黑洞"，破坏"堆内每字节都属于某块"的不变量并让堆遍历走歪（这是我实测中最初踩到的 bug）。
- **搜索复杂度**：类数 $\approx \log_2$，类内长度取决于插入策略。LIFO 下刚释放的块总在头部，**时间局部性好**；地址有序每次插入 $O(\text{空闲块数})$ 但碎片更低。讲义的结论是"**分离链表把首次适配近似成对整个堆的最佳适配**"。
- **与硬件的对应**：`get` / `put` 是 `movq`，`blk_size` 是 `andq $-16, %rax`，`payload()` 是 `leaq 8(%rdi), %rax`。分配器的耗时不在"算术"而在**访存**——`find_fit` 每走一个节点都要读 header 与 `next`，**节点分散 ⇒ cache miss 多 ⇒ 吞吐率低**。这正是"分离链表能提速"的硬件层原因：把搜索限制在小链表里 = **把随机访存变成局部访存**。

【内存布局 / 数据结构图解】（实测 `init` 与 `malloc(24)` 之后的真实堆）

```
地址              内容                      说明
0x405080 ┌──────────────────────┐
         │ 0x0000000000000000   │  垫块（4 字节对齐填充，8 字节占位）
0x405088 ├──────────────────────┤
         │ 0x0000000000000013   │  序言块 header：size=16, alloc=1, prev_alloc=1
0x405090 ├──────────────────────┤  ← 序言块在这一行"结束"，无脚部
         │ 0x0000000000000023   │  malloc(24) 的块 header：size=32, alloc=1, prev_alloc=1
0x405098 ├──────────────────────┤  ← payload（返回给应用，16 字节对齐 ✓）
         │ 24 字节应用数据       │
0x4050b0 ├──────────────────────┤
         │ 0x0000000000000FE3   │  空闲余量 header：size=4064, alloc=0, prev_alloc=1
0x4050b8 ├──────────────────────┤  ← 该空闲块的 next 指针（复用载荷区）
         │ next = NULL          │
0x4050c0 ├──────────────────────┤
         │ prev = NULL          │
         │   ...未使用...       │
0x405fb8 ├──────────────────────┤
         │ 0x0000000000000FE0   │  空闲块脚部：size=4064, alloc=0
0x405fc0 ├──────────────────────┤
         │ 0x0000000000000001   │  结尾块：size=0, alloc=1, prev_alloc=0
0x405fc8 └──────────────────────┘  堆结束（heap_size = 4128）
```

【与汇编 / 硬件的对应】（`gcc -O2 -S` 对 `blk_size` 与 `payload` 的实际产物）

```
# size_t blk_size(void *bp) { return *(size_t*)bp & ~(size_t)0xF; }
blk_size:
        movq    (%rdi), %rax
        andq    $-16, %rax          # 抹掉低 4 位（分配位 + PREV_ALLOC 位）
        ret

# void *payload(void *bp) { return (char*)bp + 8; }
payload:
        leaq    8(%rdi), %rax
        ret

# int blk_prev_alloc(void *bp) { return (*(size_t*)bp >> 1) & 1; }
blk_prev_alloc:
        movq    (%rdi), %rax
        shrq    $1, %rax            # 右移 1 位，把 bit1 降到最低位
        andl    $1, %eax
        ret
```

14.3.2 实测验证：分配 / 释放 / 分割 / 合并 / realloc

编译与运行（真实输出）：

```
$ gcc -g -Wall -std=c11 /tmp/mmseg.c -o /tmp/mmseg && /tmp/mmseg
==== 1) mm_init: 序言块 + 结尾块 + 首个 4KB 空闲块 ====
  堆大小 = 4128 字节
  [init            ] 4096:F/pa=1 | epilogue(0)

==== 2) 分配: 从大空闲块中分割 ====
  malloc(24)   = 0x4050a0  16 字节对齐: 是
  [malloc(24)      ] 32:A/pa=1 4064:F/pa=1 | epilogue(0)
  malloc(1000) = 0x4050c0
  [malloc(1000)    ] 32:A/pa=1 1008:A/pa=1 3056:F/pa=1 | epilogue(0)

==== 3) free 与立即合并 ====
   free(p2) → 与后续空闲余量块合并
  [after free(p2)  ] 32:A/pa=1 4064:F/pa=1 | epilogue(0)
   malloc(64) x2 = 0x4050c0, 0x405110
  [malloc(64) x2   ] 32:A/pa=1 80:A/pa=1 80:A/pa=1 3904:F/pa=1 | epilogue(0)
   free(p3) → 前后皆为已分配块 (case 1), 自成独立空闲块
  [after free(p3)  ] 32:A/pa=1 80:F/pa=1 80:A/pa=0 3904:F/pa=1 | epilogue(0)
   free(p4) → 与前块 (case 3) 合并
  [after free(p4)  ] 32:A/pa=1 4064:F/pa=1 | epilogue(0)

==== 4) realloc: 原地扩展 / 原地缩小 ====
  q = malloc(64) = 0x4050c0
  [malloc(64)=q    ] 32:A/pa=1 80:A/pa=1 3984:F/pa=1 | epilogue(0)
  realloc(q,200)  = 0x4050c0  → 原地扩展 (地址不变)
  原 64 字节数据完整保留 ✓
  [realloc grow    ] 32:A/pa=1 208:A/pa=1 3856:F/pa=1 | epilogue(0)
  realloc(q2,16)  = 0x4050c0  → 原地缩小, 余量挂回空闲链表
  [realloc shrink  ] 32:A/pa=1 32:A/pa=1 4032:F/pa=1 | epilogue(0)

==== 5) realloc: 搬移路径 ====
  r = malloc(64) = 0x4050e0
  [malloc(64)=r    ] 32:A/pa=1 32:A/pa=1 80:A/pa=1 3952:F/pa=1 | epilogue(0)
  realloc(r,4096) = 0x405130  → 搬移 (malloc + memcpy + free)
  原 64 字节数据完整保留 ✓
  [realloc 4096    ] 32:A/pa=1 32:A/pa=1 80:F/pa=1 4112:A/pa=0 3952:F/pa=1 | epilogue(0)

==== 6) 边界语义 ====
  malloc(0)        = (nil)    (讲义: 返回 NULL)
  realloc(NULL, 8) = 0x4050e0    (≡ malloc(8))
  realloc(r2, 0)   = (nil)    (≡ free(r2), 返回 NULL)
  [after realloc(,0)] 32:A/pa=1 32:A/pa=1 32:A/pa=1 8112:F/pa=1 | epilogue(0)

==== 7) 不变量检查 mm_checkheap ====
  mm_checkheap(1) = PASS ✓

==== 8) 利用率 mini-trace (mdriver 口径: 峰值载荷 / 堆大小) ====
  64 次 malloc(900) → 释放全部偶数下标 → 32 次 malloc(400) 填回空档
  峰值载荷 = 57600 字节, 堆大小 = 61472 字节
  peak utilization = 57600/61472 = 0.9370  (93.70%)
  mm_checkheap = PASS ✓

==== 9) 各大小类的空闲块分布 ====
  class  3 (>= 80) : 16 个空闲块
  class  9 (>= 513) : 16 个空闲块
  class 11 (>= 2049) : 1 个空闲块
```

格式：`32:A/pa=1` = 块大小 32 字节、`A` 已分配（`F` 空闲）、`pa` 即 PREV_ALLOC 位。

随机压力测试（同一份源码，把接口当黑盒）：

```
$ gcc -g -Wall -std=c11 /tmp/mmstress.c -o /tmp/mmstress && /tmp/mmstress
200000 步随机操作全部通过, 最终 mm_checkheap = PASS, 堆 = 167968 字节
```

逐条解读：

- **步骤 2 的分割**：4096 的空闲块被切成 `32:A`（服务 `malloc(24)`）与 `4064:F`（落在 class 11，≥2049）。
- **步骤 3 是合并正确性的关键证据**：`free(p3)` 让 80 字节块变空闲，其**后块的 `pa` 立刻从 1 变 0**——正是"更新后块 PREV_ALLOC 位"在起作用；`free(p4)` 因 `pa=0` 正确找到前驱并合并回 4064。
- **步骤 4 是原地路径**：`realloc(q,200)` 后块从 80 涨到 208，**地址完全不变**（`0x4050c0`）；`realloc(q2,16)` 缩回 32 并把 176 字节余量挂回链表。

- **步骤 5 是搬移路径**：4096 > "80 + 后继 3952"，只能新分配并 `memcpy`；旧块 `80:F` 释放后与后面合并成 `8112:F`（步骤 6）。原 64 字节数据完整保留，验证了 `memcpy` 的 `min` 语义。
- **步骤 8**：64 个 900 字节块先撑到峰值，隔一个释放一个再填 400 字节请求，分离链表把小块精确塞进空档，得 **peak utilization = 93.70%**。

14.3.3 利用率计算示例（含内部 / 外部碎片）

模型：教材 9.9 的隐式空闲链表——**4B header + 4B footer，8 字节对齐，最小块 16 字节**，首次适配、立即合并、**按需扩展（不预取）**。堆布局为 `[填充 4B | 序言块 8B | 第一个真实块 ...]`，序言块永不释放但**计入堆大小**。$w(n)$ 为块总大小，故 $w(4) = w(5) = w(6) = 16$。

序列 A：`p1=malloc(4); p2=malloc(5); p3=malloc(6); free(p2); p4=malloc(2);`

```
$ gcc -g -Wall -std=c11 util.c -o util && ./util
== 序列 A: malloc(4); malloc(5); malloc(6); free(p2); malloc(2);  ==
   init          堆= 32 |  8:A 16:F | 请求载荷= 0
   malloc(4)p1   堆= 32 |  8:A 16:A | 请求载荷= 4
   malloc(5)p2   堆= 48 |  8:A 16:A 16:A | 请求载荷= 9
   malloc(6)p3   堆= 64 |  8:A 16:A 16:A 16:A | 请求载荷=15
   free(p2)      堆= 64 |  8:A 16:A 16:F 16:A | 请求载荷=10
   malloc(2)p4   堆= 64 |  8:A 16:A 16:A 16:A | 请求载荷=12

   峰值载荷 = 15, 终态堆 = 64
   peak utilization = 15/64 = 0.2344 (23.44%)
   终态载荷 = 12, 已分配块可用容量 = 24 -> 内部碎片 = 12 B
   终态空闲(外部碎片) = 0 B
```

推导：峰值载荷出现在 `p1+p2+p3 = 15` 字节，此刻堆 = 序言 8B + 三个 16B 块 = **56**；随后 `malloc(2)` 复用了 `free(p2)` 留下的 16 字节空洞，**堆不再增长**，故终态堆仍为 64——U 的分母就是 **64**：

$$U_{\text{peak}} = \frac{15}{64} = 0.2344\ (23.44\%)$$

碎片分解（终态：序言块 + `16:A`(p1) + `16:A`(p4) + `16:A`(p3)）：

- **内碎片（internal fragmentation）**：三个已分配块共 48 字节，扣掉各自头脚（每块 8 字节，共 24 字节）后**可用 24 字节**，而请求只有 $4 + 2 + 6 = 12$ 字节 → **内碎片 = 24 − 12 = 12 字节**，来自把所有小请求抬到最小块**可用 8 字节**的那道取整。
- **外碎片（external fragmentation）**：终态**无**空闲块（`free(p2)` 的洞被 `malloc(2)` 精确填满），故为 **0**。但这是运气：若第 5 个请求是 100 字节，那 16 字节空洞立刻变成外部碎片，并迫使堆再长 112 字节。
- **对照（预取 1 KB）**：同一序列若一次 `sbrk` 预取 1 KB，堆直接到约 1048 字节，利用率暴跌到 **15/1048 ≈ 1.4%**——**利用率直接受 chunk size 影响**：预取提升吞吐率却拖低利用率（见 14.4）。

14.3.4 glibc 的 `malloc`：真实实现概览

glibc 的 `malloc` 不是"一个链表"，而是**brk 与 mmap 双路径 + 多层 bins** 的复合体：

- **分配路径**：小请求走 `brk` 扩展 **主分配区（main arena）**；大请求（默认 ≥ `MMAP_THRESHOLD` = 128 KB）直接用 `mmap`，`free` 时立刻 `munmap` 归还内核——这就是"大块立刻还、小块囤着"的来源。多线程下还有 per-thread arena。
- **多层 bins**：`tcache`（per-thread 快速缓存，每类最多 7 个，**不加锁、不合并**）→ `fastbins`（小块单向链表，也不合并）→ `unsorted bin`（刚释放的块先扔这里，**延迟合并**的体现）→ `smallbins` / `largebins`（按尺寸分桶）。**这就是本讲技术的工业化版本**：tcache = 简单分离存储，small/large bins = 分离适配。
- **调参接口**：`mallopt(M_MMAP_THRESHOLD, ...)`、`M_TRIM_THRESHOLD`、`M_MMAP_MAX`、`M_TOP_PAD`、`M_ARENA_MAX`。注意 `M_MMAP_THRESHOLD` 有**动态自适应**：释放一个大 mmap 块会把阈值往上调，以减少系统调用次数。
- **观察工具**：`mallinfo()` / `mallinfo2()`（返回 `arena`、`hblkhd`、`uordblks`）、`malloc_stats()`、`malloc_usable_size()`、环境变量 `MALLOC_CHECK_=3`。

实测：`malloc_usable_size` 与 `mallinfo2`（真实输出）

```
$ gcc -g -Wall -std=c11 /tmp/gi.c -o /tmp/gi && /tmp/gi
=== malloc_usable_size: 请求大小 vs 实际可用 ===
  malloc(   1) -> usable =   24
  malloc(   8) -> usable =   24
  malloc(  16) -> usable =   24
  malloc(  24) -> usable =   24
  malloc(  25) -> usable =   40
  malloc(  32) -> usable =   40
  malloc(  40) -> usable =   40
  malloc( 100) -> usable =  104
  malloc( 128) -> usable =  136
  malloc(1000) -> usable = 1000

=== malloc(0) 的行为 ===
  malloc(0) = 0xaa62b0  (非 NULL: 返回可 free 的唯一指针)
  malloc(0) = 0xaa67f0  (两次调用不相同)

=== mmap 阈值: 小请求走 brk, 大请求走 mmap ===
  初始: arena = 135168, hblkhd(mmap 区) = 0
  64×malloc(4096) 后: arena = 270336, hblkhd = 0  (仍走 brk)
  8×malloc(256KB) 后: arena = 270336, hblkhd = 2129920  (hblkhd 增长 = 走 mmap)
  释放后:            arena = 270336, hblkhd = 0  (mmap 区立即归还内核)
```

读法：① `malloc(1..24)` 实际给 24 字节（`32 - 8` 头），`malloc(25..40)` 给 40 字节（`48 - 8`）——glibc 的粒度是 16 字节，开销 8 字节；② **`malloc(0)` 在 glibc 上返回一个非 NULL 的唯一指针**，与讲义"F25-13 简明口径：size == 0 返回 NULL"不同，这是**标准允许的实现自由度**（C 标准要求"要么返回 NULL，要么返回一个可 `free` 的唯一指针"）；③ `hblkhd` 从 0 涨到约 2 MB 又归零，**精确证明了 mmap 阈值路径的存在**。

14.3.5 glibc 的两类真实报错（double free 与堆溢出）

⚠️ **仅供演示，请勿模仿——以下两个程序都是故意的未定义行为。**

```c
/* ⚠️ 仅供演示，请勿模仿: double free (同一指针释放两次) */
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    char *p = malloc(32);
    printf("p = %p\n", (void *)p);
    free(p);
    printf("第一次 free(p) 完成\n");
    fflush(stdout);
    free(p);                      /* ← double free: UB, glibc 会中止进程 */
    printf("这行不该被打印\n");
    return 0;
}
```

```
$ gcc -g -Wall -std=c11 /tmp/df.c -o /tmp/df && /tmp/df
p = 0x23e42a0
第一次 free(p) 完成
free(): double free detected in tcache 2
Aborted (core dumped)          # exit code 134 (SIGABRT)
```

`tcache 2` 指的是"第 2 个 tcache 桶": glibc 把刚释放的小块放进 per-thread 的 `tcache`，插入时发现该 chunk 已在桶内，于是直接 `malloc_printerr` 并 `abort`。注意 `-Wall` 同时给出了提示: `warning: pointer 'p' used after 'free' [-Wuse-after-free]`。

```c
/* ⚠️ 仅供演示，请勿模仿：溢出写穿 top chunk 的 size 字段 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <malloc.h>
int main(void) {
    mallopt(M_MMAP_THRESHOLD, 64 * 1024 * 1024); /* 强制大块走 brk 而非 mmap */
    char *big = malloc(1 << 20);
    printf("big = %p (走 brk)\n", (void *)big);
    free(big);                                   /* 归还给 top chunk */
    fflush(stdout);

    char *p = malloc(1000);
    printf("p = %p, usable = %zu\n", (void *)p, malloc_usable_size(p));
    fflush(stdout);

    memset(p, 0xAB, 1000 + 16 + 8);              /* ← 越界 1024 字节，覆盖 top->size */
    char *q = malloc(64);                        /* _int_malloc 校验 top->size 失败 */
    printf("q = %p\n", (void *)q);
    return 0;
}
```

```
$ gcc -g -w -std=c11 /tmp/ovf7.c -o /tmp/ovf7 && /tmp/ovf7
big = 0x7fbd2a467010 (走 brk)
p = 0x15492b0, usable = 1000
准备从 p 越界写 1024 字节，落点覆盖 top->size
malloc(): corrupted top size
Aborted (core dumped)          # exit code 134 (SIGABRT)
```

机理：`p` 的块总长 1008 字节，下一个 chunk 头（即 `top` chunk）的 `size` 字段在 `p + 1008 + 8` 处。越界写入把 `top->size` 改成 `0xABAB…`。下一次 `malloc` 走 `_int_malloc`，在 `use_top` 分支里用 `chunksize(victim) > av->system_mem` 之类的一致性判断识破了它，于是报 `corrupted top size` 并 `abort`。**这正是讲义"堆溢出如何破坏元数据"的现场**——你的分配器里，同样的一幕会表现为 `coalesce` 读到荒谬的块大小、走进错误的地址。这类缺陷的定位手段是 `gdb` 硬件观察点 `watch *addr`、`valgrind`，或写严格的**堆检查器**与**契约断言**。

14.4 实验关联：L5b Malloc Lab（final）

评分公式（writeup 与 `mdriver.c` 一致）：

$$P = wU + (1-w)\min\left(1, \frac{T}{T_{\text{libc}}}\right), \qquad w = 0.6,\ T_{\text{libc}} = 600\ \text{Kops/s}$$

- U = **空间利用率（space utilization）**。`eval_mm_util` 的做法：**模拟**一遍 trace，累计"已分配未释放的字节数"，取峰值 `max_total_size` 除以 `mem_heapsize()`。**它统计"请求字节数"而非"块大小"**，故内碎片与头脚开销都由 U 承担；且分母是**跑完整条 trace 后的堆高水位**——一次失败的 `find_fit` 造成的扩展会永久拉低分数。
- T = **吞吐率（Kops/s）**，每秒完成的 malloc/realloc/free 操作数；超过 600 Kops/s **不再加分**（防止"只会快、不会省"）。
- 结构：**正确性 20 + 性能 35 + 风格 10**；风格里 **5 分给堆检查器**、5 分给结构注释——白送的分，别丢。

优化路线图（checkpoint → final）：

阶段	手段	主要改善
0	隐式链表 + 边界标记合并	正确性（基线）
1	`coalesce_block()` 先做对	正确性
2	**显式空闲链表**	吞吐率（搜索从"所有块"变"空闲块"）
3	**分离空闲链表**（首次适配 + 逐类向后）	吞吐率 + 利用率
4	**去掉已分配块的脚部** + `PREV_ALLOC` 位	利用率（内碎片 ↓）
5	**降低最小块**（32 → 24/16 字节）	利用率（小请求不再浪费）
6	**`realloc` 原地扩展/缩小**	吞吐率（realloc trace）
7	搜索起点优化（rover / next fit）、调整大小类桶、调 `CHUNK`	吞吐率 / 利用率微调

final 的三个硬指标与常见坑：

- **16 字节对齐是硬要求**：`mdriver` 会检查 `ALIGNMENT`，要求**载荷指针** `ptr % 16 == 0`（不是块头）。因块头 + 8 = 载荷，所有块头必须 $\equiv 8 \pmod{16}$；堆基址对齐无法保证，**必须用一个垫块把首个真实块推到 $\equiv 8 \pmod{16}$**。

- **`mem_sbrk` 扩展的序言/结尾块处理**：新块 header 必须**恰好落在旧 epilogue 地址上**（`mem_sbrk(asize) - WSIZE`），否则留 8 字节黑洞；新 epilogue 写在 `bp + asize`，别忘更新它的 `PREV_ALLOC` 位。
- **`mm_realloc` 失败必须保持旧块有效**：`mm_malloc` 返回 `NULL` 时直接返回 `NULL`，**绝不 `free` 旧块**。`memcpy` 长度必须取 $\min$。
- **`find_fit` 的起点**：每次从类 0 起扫（first fit）会让堆头部反复被切碎；**rover / next fit**（从上次成功处继续）常能同时提升两项。bootcamp 还提到 **better fit**（首适配后再往后多看 20 个块取最优）——但每次改动都要用 `mdriver -v` 重新测量。

调试要点（rec08 实录）：

- **`mdriver` 报 "has N garbled bytes"** = 你的分配器覆盖了已分配块里的用户数据。rec08 用 `gdb --args ./mdriver-dbg1 -c ./traces/syn-struct-short.rep` + `watch *0x8000000a0` 硬件观察点，gdb 停在**第一个写入该地址的语句**：`write_block() at mm.c:333, Old value = 129, New value = 32`。改用**写契约（contract）**更快：加 `assert((unsigned long)footerp < ((long)block + size))` 后立刻报 `Assertion ... failed`。
- **`mdriver -D`** 尽早检测 garbled bytes，**`-V`** 定位是哪条 trace 出错。**Valgrind 不能用于 Malloc Lab**（你要替换的就是 `malloc`）。
- **堆检查器要随实现升级**：加了分离链表就检查"块是否在正确的大小类"；用了延迟合并就要允许"相邻空闲块共存"；空闲链表无环可用 **hare & tortoise 算法**；"堆中空闲块数 == 链表中空闲块数"是高价值不变量。

14.5 常见错误与调试技巧

- **忘记更新后块的 `PREV_ALLOC` 位**：`free` 时用错误偏移读前驱脚部，读到垃圾 → 随机崩溃或"头脚不一致"。**调试**：在 `coalesce`、`split_alloc` 末尾加 `CHECKHEAP(1)`，用 `gdb` 的 `x/4gx <bp>` 比对位域。这是去脚部实现中**最高频的错误**。
- **`extend_heap` 没有"顶掉"旧 epilogue**：8 字节黑洞让堆遍历停在错误位置。**调试**：对比 `p mem_heapsize()` 与 `p (char*)mem_heap_hi()-(char*)mem_heap_lo()`，或让 `mm_checkheap` 从 `mem_heap_lo()` 走到 `mem_heap_hi()` 检查每步是否落在块边界。
- **`memcpy` 长度写成请求的新大小**：`realloc` 增大时读出越界数据。**调试**：断在 `mm_realloc`，对照 `p csize`、`p size`、`p copy`。
- **`realloc` 失败时释放了旧块**：数据丢失 + 双重释放。**调试**：在独立测试程序（不是 mdriver）上跑 `valgrind --leak-check=full`。
- **双重释放（double free）**：glibc 报 `free(): double free detected in tcache 2` 并 `SIGABRT`；自写分配器则表现为"同一块被插入链表两次 → 链表成环 → 无限循环"。**调试**：`gdb` 崩溃后 `bt`，再用 **hare & tortoise** 遍历链表查环；或 `setenv MALLOC_CHECK_ 3`。
- **堆溢出破坏元数据**：典型报错 `malloc(): corrupted top size`、`munmap_chunk(): invalid pointer`。**调试**：`gcc -fsanitize=address`（受限环境可能因巨大 shadow 内存无法启动）、`valgrind`，或 `gdb` 硬件观察点 `watch *addr`。
- **`mm_checkheap` 调用时机错误**：在 `coalesce` 中途调用会看到"相邻空闲块未合并"的**假报警**。**调试**：只在 `mm_malloc` 返回前、`mm_free` 返回后调用。
- **`CHUNK` 过大拉低利用率**：预取 1 KB 在小型 trace 上就能把 U 打到 12%。**调试**：用 `mdriver -v` 对比 `util` 列；`MAX_HEAP` 是 20 MB，`CHUNK` 太大会提前 OOM。

14.6 关键要点

1. **显式空闲链表的决定性优势**：搜索范围从"所有块"缩到"空闲块"，且因为只跟踪空闲块，`prev` / `next` 可直接复用载荷区。
2. **插入策略是设计决策**：LIFO 常数时间但碎片高；地址有序需搜索但碎片低，且**是"省掉脚部"的前提之一**。
3. **分离链表 = 大小类分桶 + 首次适配**：先用 $O(\log)$ 个类把范围缩小，换来**近似最佳适配的利用率**与近似 $O(1)$ 的分配时间。
4. **两条分支必须分清**：简单分离存储（不分割、不合并、单向链表、只花 8 字节指针）vs 分离适配（可分割、可合并、需边界标记）——前者快而浪费，后者是教材与 lab 的推荐方案。
5. **去掉脚部不降低最小块**：脚部只影响已分配块的内碎片；最小块仍受空闲块 header+next+prev+footer 约束，要降它必须再砍一个字段。
6. **利用率与吞吐率在 Malloc Lab 里加权对抗**：$P = 0.6U + 0.4\min(1, T/600\text{K})$，任何"多预取"或"多扫几块"都在用一个指标换另一个，必须用 `mdriver -v` 量化后再决定。

14.7 思考题（带答案）

Q1（计算题：分离链表的分段划分） 若采用"小尺寸逐档 16 字节步进到 128，之后按 2 的幂分段"的方案，请列出全部大小类，并指出 **`asize = 1160` 的块应放入第几类**；若该类为空，`find_fit` 接下来会检查哪些类?

答：类划分（16 类，下标 0 起）：

类	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
范围下限	32	48	64	80	96	112	128	129	257	513	1025	2049	4097	8193	16385	32769
范围上限	32	48	64	80	96	112	128	256	512	1024	2048	4096	8192	16384	32768	∞

`asize = 1160`：$1024 < 1160 \le 2048$，落入 **class 10（[1025, 2048]）**。若 class 10 为空，`find_fit` 依次查 **class 11 → 12 → 13 → 14 → 15**（2049 到 ∞），每类内做首次适配。可见：**类越细，跨类步数越多但匹配越准；类越粗，搜索越快但内碎片越大**。

Q2（计算题：判断 `realloc` 能否原地扩展） 某分配器：16 字节对齐，header 8 字节，已分配块无脚部，最小块 32 字节。当前状态为 `[A: 48 字节 | B: 64 字节(空闲) | C: 96 字节]`，指针 `p` 指向 A 的载荷。 分别判断 `realloc(p, 40)`、`realloc(p, 88)`、`realloc(p, 120)` 能否原地扩展？若不能，说明走哪条路径。

答：A 的 `csize = 48`。 – `realloc(p,40)`：`asize = ALIGN(48) = 48`，$48 \le 48$ → **原地缩小**（余量 0 < 32，不分割，直接返回 `p`）。 – `realloc(p,88)`：`asize = ALIGN(96) = 96`；B 空闲且 $48 + 64 = 112 \ge 96$ → **可原地扩展**：合并后块长 112，切 96 给 A，余量 $112 - 96 = 16 < 32$ → **不分割**，A 实占 112 字节（16 字节内碎片）。**地址不变**。 – `realloc(p,120)`：`asize = ALIGN(128) = 128`；$48 + 64 = 112 < 128$ → **不能原地扩展**，走**搬移**：`malloc(120)` + `memcpy(40 字节)` + `free(p)`。拷贝长度是 $48 - 8 = 40$，不是 48 或 120。

Q3（"直观但错误的想法"错在哪） "分离空闲链表比显式空闲链表快，所以给每个可能的块大小都单独建一条链表，就能同时拿到最高的吞吐率和最高的利用率。"

答：错在两处。① **链表头数组本身要空间**。Malloc Lab 只给 128 字节可写全局空间，若按最大块 20 MB、16 字节步进建表需上百万个指针，**数组自己就比堆还大**。② **"每尺寸一类"只优化搜索，不优化碎片**：它把最佳适配变成 $O(1)$，于是**内碎片最大**——最佳适配总选中"刚好装下"的块，把略大的块一直留着，长期产生的"小碎块"最难被后续请求利用；极端情况下（每类恰好剩一个小块）会出现"总空闲很大却无法满足任何请求"的**外部碎片灾难**。工程解是折中：**小尺寸细粒度、大尺寸按 2 的幂粗粒度**，再配 rover 起点与延迟合并。

Q4（计算题：利用率与碎片） 沿用 14.3.3 的隐式链表模型（4 字节 header + 4 字节 footer，8 字节对齐，最小块 16 字节，序言块 8 字节，按需扩展）。序列为 `p1=malloc(3); p2=malloc(12); free(p1); p3=malloc(9);`，求 **peak utilization**，并给出终态的**内部碎片**与**外部碎片**。

答（实测输出见下）：

```
== 序列 B (Q4): malloc(3); malloc(12); free(p1); malloc(9);  ==
   init          堆= 32 |  8:A 16:F | 请求载荷= 0
   malloc(3)p1   堆= 32 |  8:A 16:A | 请求载荷= 3
   malloc(12)p2  堆= 56 |  8:A 16:A 24:A | 请求载荷=15
        -> 峰值载荷 = 15 (此刻堆 = 56)
   free(p1)      堆= 56 |  8:A 16:F 24:A | 请求载荷=12
   malloc(9)p3   堆= 80 |  8:A 16:F 24:A 24:A | 请求载荷=21

   峰值载荷 = 15, 终态堆 = 80
   peak utilization = 15/80 = 0.1875 (18.75%)
   终态载荷 = 21, 已分配块可用容量 = 32 -> 内部碎片 = 11 B
   终态空闲(外部碎片) = 16 B
```

- `w(3)=16`（$3 + 8 = 11 \to 16$），`w(12)=24`（$12 + 8 = 20 \to 24$）。峰值载荷 $= 3 + 12 = 15$，此刻堆 = 序言 8 + 16 + 24 = **56**。
- `free(p1)` 后 `w(9)=24`（$9 + 8 = 17 \to 24$）。空闲块只有 16 字节，$16 < 24$ **放不下** → 必须扩展，堆从 56 涨到 **80**。终态载荷 $= 12 + 9 = 21$。
- **peak utilization** $= 15/80 = 0.1875$（18.75%）——分母是**跑完整个序列后的堆高水位 80**，而不是峰值那一刻的 56。
- **内碎片**：终态已分配块为 `24:A`(p2) 与 `24:A`(p3)，共 48 字节，扣掉各自头脚 8 字节后可用 **32** 字节，请求载荷 21 → **内部碎片 = 32 − 21 = 11 字节**。
- **外碎片**：`free(p1)` 留下的 **16 字节**空闲块无法满足 24 字节请求，且终态无人再申请 → **外部碎片 = 16 字节**。
- 要点：**分母是"跑完全过程后的堆高水位"而非峰值那一刻的堆大小**——这正是 `mdriver` 的 `eval_mm_util` 口径，也是"一次失败的 `find_fit` 造成的扩展会永久拉低分数"的原因。

Lecture 15: 代码优化 (Code Optimization)

> **讲义对应**：CMU 15–213 Lecture 15 — Code Optimization（素材：`F25-15-optimization.txt`） **教材对应**：CS:APP3e 第 5 章 5.1–5.15（优化编译器的能力与局限、程序性能的表达、消除循环低效、减少过程调用、消除不必要的内存引用、理解现代处理器、循环展开、提高并行性、一些限制因素、性能提高技术、确认和消除性能瓶颈） **关联 Lab**：L4 Cache Lab Part B（优化矩阵转置，CPE 评分）/ L5 Malloc Lab（吞吐量与空间利用率）

15.1 概述

前面十四讲一直在回答"计算机**怎么**工作"：位怎么表示、指令怎么执行、缓存怎么命中、虚拟地址怎么翻译、堆怎么分配。本讲换一个方向提问——**知道了这一切之后，怎样让代码跑得更快？** 讲义给出的答案很务实：优化不是玄学，而是"把编译器做不到的常数因子工作手工补上"。同一段求和代码，未优化时 CPE 高达 11，经过消除过程调用、消除内存写回、循环展开、多累加器四步，可降到 0.5 附近——**二十倍的差距，全部来自对汇编、过程调用、存储层次和处理器微架构知识的综合运用**。

本讲承接第 9–10 讲的存储层次与局部性，把"缓存友好"升级为可量化的 **CPE（Cycles Per Element）** 语言；它也是 L4 Cache Lab Part B 的直接理论前提，并为第 24 讲线程级并行埋下"指令级并行（ILP）"的伏笔。

15.2 核心概念与底层机制图解

15.2.1 CPE：用斜率而不是秒来度量性能（Cycles Per Element）

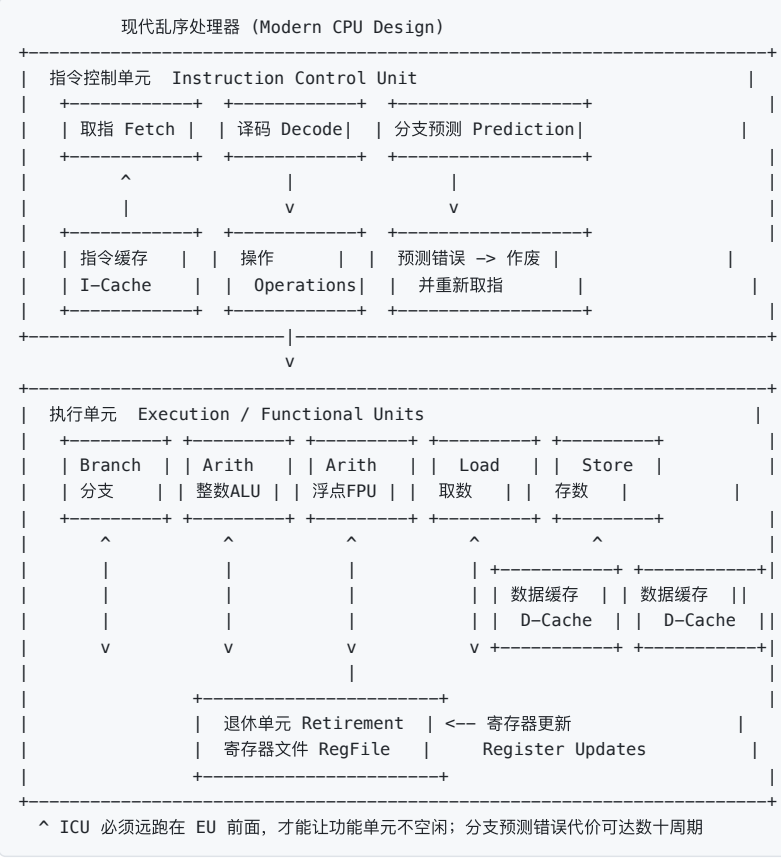
- **定义与目的**：CPE 是"平均每处理一个元素花费的时钟周期数"。不用绝对时间（毫秒）是因为它会被**时钟频率**污染：同一份代码在 2.0 GHz 老机器上跑 10 ms、在 4.0 GHz 新机器上跑 5 ms，但这**不代表**代码变好了。CPE 把频率因子除掉，剩下的才是代码与微架构的真实效率。
- **直观解释（"它是什么？"）**：把程序想成流水线工厂。总耗时 = **启动成本 + 单个零件时间 × 零件数**。CPE 就是"单个零件的时间"，是这条直线的**斜率**；截距（Overhead）无关紧要，n 足够大时斜率才是主导。测 CPE 像测车的油耗：看"每公里耗油"而非"这趟烧了多少油"。
- **底层机制图解**：讲义给出 $Cycles = CPE \times n + Overhead$，CPE 即拟合直线的**斜率**。两条不同实现的直线斜率不同，用足够大的 n 就能把斜率测准。
- **与机器码/硬件的对应**：CPE 的倒数就是硬件的**吞吐量**（每周期处理多少元素）。CPE = 1 意味着"每周期做完一个元素"，CPE = 0.5 意味着"每周期做两个"——这只有在处理器有**多个功能单元并行工作**时才可能，直接指向 15.2.3 的乱序执行。
- **测量方法论（讲义强调的三条纪律）**：① n 必须**足够大**，让截距可忽略；② 每个版本**重复多次**取**最小值或中位数**——调度、中断、频率波动只会让结果变慢不会变快，故最小值最接近真实能力；③ 用 `clock()`（或 `clock_gettime(CLOCK_MONOTONIC, ...)`）计时并扣除循环外的固定开销。讲义的 `fsecs` 工具正是把这些纪律固化成可复用的计时框架。

15.2.2 编译器能做什么、不能做什么（Limits to Compiler Optimization）

- **定义与目的**：优化的**首要原则**是"程序员必须理解编译器能做什么、不能做什么"。它能做的（常数折叠、死代码消除、公共子表达式消除、代码移动、内联）都是**局部**且**保守**的；不能做的才是程序员的机会。
- **直观解释（"它是什么？"）**：编译器像一个**极其守规矩的律师**：它绝不会替你做任何"可能改变合同含义"的修改，哪怕那个条款明显是笔误。你写的 `if (0)`、冗余赋值，它敢删；但一个它看不透的指针，它就绝不敢动。
- **编译器必须保守的三条铁律**：
- **不能改变程序语义**：程序里哪怕一个"边界情况"的行为，程序员可能不在乎，但编译器不知道他不在乎。
- **不能引入原程序没有的副作用**：这直接导致"**不能把函数调用移出循环**"——它无法证明这个函数没有副作用（改全局变量、打印、改文件）。
- **不能处理有歧义的指针别名（aliasing）**：如果 `a` 和 `b` 可能指向同一块内存，对 `*b` 的写就可能改变 `*a` 的值，任何重排都可能是错的。
- **另一条现实约束**：编译器通常**一次只分析一个函数**（全程序分析 LTO 昂贵），跨函数优化主要靠**内联**。这也解释了 15.3 中"把访问函数放进另一个编译单元"为何能忠实复现讲义的低效场景。
- **与机器码/硬件的对应**：讲义把优化分为**局部优化**（单个基本块内：常数折叠、强度削减、死代码消除）与**全局优化**（整个控制流图上：循环变换、代码移动、全局 CSE）。这个划分对应编译器的两套分析框架——块内快而准，跨块必须做不动点迭代。

15.2.3 现代乱序处理器：功能单元、延迟与发射时间（Out-of-Order Execution）

- **定义与目的**：要理解为什么"多累加器"能提速，必须先看清处理器内部。现代 CPU 是**超标量（superscalar）**的：每个周期可以**发射（issue）多条**指令；也是**乱序（out-of-order）**的：指令按数据依赖关系执行，而不按程序书写顺序。
- **直观解释（"它是什么？"）**：把处理器想成**一个大厨房**。指令控制单元（ICU）是**领班**，负责提前读菜谱（取指、译码）并按食材依赖关系安排任务；执行单元（EU）是一排**工位**：几口炒锅（浮点乘法）、几口汤锅（浮点加法）、几个砧板（整数运算）、取料台（load）、出菜台（store）。领班的目标是**让每个工位都不闲着**。
- **底层机制图解**：讲义给出的现代 CPU 机构如下。关键点在于 ICU **必须工作在执行单元前面很多**，才能持续供料；而分支是最大的障碍——在 `cmp` 结果出来之前，ICU 不知道该往哪儿取指，只好**预测**。



- **三个关键微架构参数**（后文所有性能界限都由它们决定）：
- **延迟（latency）**：一条指令从源操作数就绪到结果可用的**周期数**。它决定**依赖链**能跑多快。
- **发射时间（issue time，又叫 reciprocal throughput）**：连续发射两条**同类、互相独立**的指令所需的最小周期数间隔。它的倒数就是**功能单元的条数**。
- **容量（capacity）**：能同时做这条运算的功能单元有几个。
- **与机器码/硬件的对应**：讲义与 CS:APP3e 以 Intel Haswell/Broadwell 为基准，其参数是：

运算	延迟 (latency)	发射时间 (issue)	功能单元数
整数加 `addq`	1	1	4 个整数 ALU
浮点加 `addsd`	3	1	2 个 FP 加法单元
浮点乘 `mulsd`	**5**	**1**	**2 个 FP 乘法单元**

注意：本机（AMD EPYC 7V13，Zen 3）实测 `mulsd` 与 `addsd` 延迟均为 **3** 周期、`addq` 延迟 **1** 周期，`mulsd` 发射时间 **0.5** 周期/条（即 2 条浮点流水线）。参数**依机器而异**，但"**浮点乘延迟远高于发射时间**"这个结构性事实在所有现代 x86 上都成立——这正是多累加器的用武之地。

15.2.4 延迟界限与吞吐量界限（Latency Bound vs Throughput Bound）

- **定义与目的**：任何代码的性能都被两个下界夹住，取二者中较大的那个：
- **延迟界限（latency bound）**：由**关键路径（critical path）**上那条最长的串行依赖链决定。只要运算之间存在真依赖，后一条必须等前一条算完。
- **吞吐量界限（throughput bound）**：由**功能单元的总条数**决定，是原始计算能力的物理极限。
- **直观解释（"它是什么？"）**：把求和想成**一群人往一个桶里丢石头**。如果只有**一个桶**（单一累加器），那么每次丢石头都必须等上一次丢完——瓶颈是"**桶的开口速度**"（延迟）。如果你摆**四个桶**（多累加器），四个人可以同时丢，瓶颈就变成"**场地上能站几个人**"（功能单元数）。桶越多，延迟越被摊薄；但桶多到超过人数之后，再增加也没用。
- **底层机制图解**：单一累加器的依赖链是一条长蛇，每环都必须串行；多个累加器把长蛇切成几条互不相干的短链：

```
一、单一累加器（1 accumulator）：关键路径 = n 次串行加法

    d[0]   d[1]   d[2]   d[3]   d[4]   d[5]   ...
      \      |      |      |      |      |
       +---->+---->+---->+---->+---->+---->  x0

    每一步都必须等上一步算完  ==>  CPE = L  （与功能单元个数无关，纯延迟受限）


二、四个累加器（4 accumulators）：关键路径 = n/4 次串行加法，4 条链并行推进

    d[0]  d[4]  d[8]   ...  -->  x0  --\
    d[1]  d[5]  d[9]   ...  -->  x1  ---+-->  最后合并 x0+x1+x2+x3
    d[2]  d[6]  d[10]  ...  -->  x2  ---+
    d[3]  d[7]  d[11]  ...  -->  x3  --/

    4 条链互不依赖，可同时在多个功能单元上推进
    ==>  CPE = max( L/k , T )     k = 累加器个数, T = 发射时间
         延迟界限被摊薄成 L/k，但最终撞上吞吐量界限 T = 1/C
```

- **与机器码/硬件的对应**：把上面的抽象变成公式。设延迟 L、发射时间 T、功能单元数 $C = 1/T$，用 k 个累加器：
- 延迟界限：$CPE_{\text{lat}} = L/k$
- 吞吐量界限：$CPE_{\text{thr}} = T = 1/C$
- 实际 $CPE \approx \max(L/k,\ T)$

以浮点乘为例（讲义 Haswell 数据 $L = 5, T = 1, C = 2$）：– 1 个累加器：$CPE = \max(5/1, 1) = 5$ —— **纯延迟受限** – 2 个累加器：$CPE = \max(5/2, 1) = 2.5$ – 4 个累加器：$CPE = \max(5/4, 1) = 1.25$ – 5 个及以上：$CPE = \max(5/5, 1) = 1$ —— 此时**降到吞吐量界限**，再加累加器毫无收益

推论：$k^* = L/T$ **是"够用"的累加器个数**。Haswell 上 $k^* = 5/1 = 5$，所以 4 个不够、5 个刚好、8 个纯属浪费。这个不等式是本章所有性能计算的骨架。

15.3 代码示例与底层机制分析

15.3.1 本机实测环境

所有数值均在本机真实编译运行测得，读者请在自己的机器上重跑：

```
CPU      : AMD EPYC 7V13 64-Core (Zen 3), 实测运行主频 3.07 GHz
缓存     : L1d 32 KB / L2 512 KB / L3 32 MB(每 CCX)
编译器   : gcc 12.2.0
计时     : clock_gettime(CLOCK_MONOTONIC)
主频标定 : 16 条串行 addq 依赖链（每迭代恰好 16 周期），最小 5 次
测量纪律 : 每版本重复 15 次，报告 CPE_min 与 CPE_med
```

为什么必须标定主频：标称 3.7 GHz 是最大加速频率，实际持续负载下只有 3.07 GHz。若不标定，CPE 会整体偏差 20%。标定用小段内联汇编：

```
/* 16 条串行 addq（延迟 1 周期/条）+ 1 条无关的 decq/jnz ==> 每迭代恰好 16 个核心周期。
   写成内联汇编，保证 -O1/-O2/-O3 下都不会被优化掉。 */
static double calibrate_ghz(long iters) {
    const int REP = 5, K = 16; double best = 1e30;
    for (int r = 0; r < REP; r++) {
        long c = iters, x = 0; double t0 = now_sec();
        __asm__ __volatile__(
            "1:\n\t"
            "addq $1,%1\n\t addq $1,%1\n\t addq $1,%1\n\t addq $1,%1\n\t"
            "addq $1,%1\n\t addq $1,%1\n\t addq $1,%1\n\t addq $1,%1\n\t"
            "addq $1,%1\n\t addq $1,%1\n\t addq $1,%1\n\t addq $1,%1\n\t"
            "addq $1,%1\n\t addq $1,%1\n\t addq $1,%1\n\t addq $1,%1\n\t"
            "decq %0\n\t jnz 1b\n\t"
            : "+r"(c), "+r"(x) :: "cc");
        double dt = now_sec() - t0;
        if (dt < best) best = dt;
    }
    return (double)iters * K / best / 1e9;      /* GHz */
}
```

踩过的坑（真实教训）：最初用"1 条 addq + 1 条 decq/jnz"（每迭代理论上 1 周期）标定，在 `-O2` 下却稳定给出 **1.537 GHz**——正好是 3.07 的一半。用 `gcc -O2 -S` 反汇编发现循环体被对齐到不同的 16 字节边界，`decq/jnz` 与 `addq` 争抢同一个发射槽，导致每迭代实际 2 周期。改成 16 条串行

`addq` 后，整条依赖链长达 16 周期，彻底淹没了 1 周期的对齐噪声，`-O1/-O2/-O3` 下都稳定在 3.07 GHz。**教训：微基准的标定部分本身也可能受微架构细节影响，必须交叉验证。**

15.3.2 优化阶梯：从 CPE 11 到 0.46

下面五个版本**全部来自同一份源码**，只是写法不同；`vec_length` / `get_vec_start` / `get_vec_element` 用 `__attribute__((noinline))` 强制保留过程调用与边界检查，忠实模拟讲义中"访问函数在另一个编译单元"的情形。

```c
#define _GNU_SOURCE
#include <stddef.h>

typedef struct { size_t len; long *data; } vec;

__attribute__((noinline)) size_t vec_length(vec *v) { return v->len; }
__attribute__((noinline)) long  *get_vec_start(vec *v) { return v->data; }
__attribute__((noinline)) int get_vec_element(vec *v, size_t idx, long *val) {
    if (idx >= v->len)          /* 每次调用都要做的边界检查 */
        return 0;
    *val = v->data[idx];
    return 1;
}

/* ---------- v1: 低效版（循环调用函数 + 每轮写回内存） ---------- */
void combine_v1(vec *v, long *dest) {
    long i;
    *dest = 0;
    for (i = 0; i < (long)vec_length(v); i++) {   /* 每轮调用 vec_length */
        long val;
        get_vec_element(v, i, &val);              /* 每轮调用 + 边界检查 */
        *dest = *dest + val;                      /* 每轮读 + 写内存 */
    }
}

/* ---------- v2: 局部变量累加 + 直接数组访问 ---------- */
void combine_v2(vec *v, long *dest) {
    long i, len = (long)vec_length(v);            /* 长度提到循环外 */
    long *d = get_vec_start(v);                   /* 直接拿数据指针 */
    long t = 0;
    for (i = 0; i < len; i++)
        t += d[i];                                /* 只在寄存器里累加 */
    *dest = t;                                    /* 只写一次内存 */
}

/* ---------- v2u: v2 + 循环展开 2（仍是单一累加器） ---------- */
void combine_v2u(vec *v, long *dest) {
    long i, len = (long)vec_length(v);
    long *d = get_vec_start(v);
    long t = 0;
    for (i = 0; i + 2 <= len; i += 2)
        t = (t + d[i]) + d[i + 1];                /* 展开≠并行：仍是一条依赖链 */
    for (; i < len; i++) t += d[i];
    *dest = t;
}

/* ---------- v3: 4 个累加器，打破加法依赖链 ---------- */
void combine_v3(vec *v, long *dest) {
    long i, len = (long)vec_length(v);
    long *d = get_vec_start(v);
    long x0 = 0, x1 = 0, x2 = 0, x3 = 0;
    for (i = 0; i + 4 <= len; i += 4) {
        x0 += d[i];     x1 += d[i + 1];           /* 4 条互不依赖的链 */
        x2 += d[i + 2]; x3 += d[i + 3];
    }
    for (; i < len; i++) x0 += d[i];
    *dest = (x0 + x1) + (x2 + x3);                /* 最后合并 */
}

/* ---------- v4: 8 个累加器 + restrict ---------- */
long combine_v4(long *restrict d, long len) {
    long i, x0=0,x1=0,x2=0,x3=0,x4=0,x5=0,x6=0,x7=0;
    for (i = 0; i + 8 <= len; i += 8) {
        x0 += d[i  ]; x1 += d[i+1]; x2 += d[i+2]; x3 += d[i+3];
        x4 += d[i+4]; x5 += d[i+5]; x6 += d[i+6]; x7 += d[i+7];
    }
    for (; i < len; i++) x0 += d[i];
    return ((x0+x1)+(x2+x3)) + ((x4+x5)+(x6+x7));
}
```

【代码做什么？】 1. `combine_v1` 完全照搬讲义 `combine1`：把"取长度、取元素、累加"三件事分别写成一次函数调用。 2. `combine_v2` 做三件事：把 `vec_length` **提到循环外**（代码移动），用 `get_vec_start` **一次性拿到数组基址**从而去掉每轮的边界检查，把中间结果放进**局部变量**。 3. `combine_v2u`

额外做**循环展开**：一次处理两个元素，减少分支与索引递增次数——**但它仍只有一个累加器**，这是后面要证伪的"展开就等于快"。 4. `combine_v3` / `combine_v4` 引入 **4/8 个独立累加器**，把一条长依赖链切成多条短链，并用 `restrict` 向编译器承诺不发生别名。

【实测验证】（`n = 10^6`，`long` 数组 8 MB，每版本 15 次取最小）

方法	说明	CPE_min (–O1)	CPE_min (–O2)	CPE_min (–O3 –mavx2)
v1	循环调用 + 每轮写内存	**10.996**	10.000	9.994
v2	局部变量 + 直接指针	**1.094**	1.094	0.529
v2u	展开 2，单一累加器	**0.997**	0.995	0.461
v3	4 个累加器	**0.514**	0.478	0.421
v4	8 个累加器 + restrict	**0.531**	**0.461**	0.424

对比讲义的原始数据（Intel Haswell，CS:APP3e 表 5.1）：

方法	讲义 整数加	本机 整数加	讲义 double 乘	本机 double 乘
Combine1 未优化	22.68	10.996	20.18	10.006
Combine1 –O1	10.12	10.000	11.14	~10.0
Combine1 –O3	4.5	—	7.8	—
Combine4（v2）	1.27	**1.094**	5.01	**3.001**
展开 + 多累加器	0.81	**0.461**	2.51	**0.480**

【底层机制透视】**绝对数值必然不同**：讲义 Haswell 未优化版 CPE 高达 22.68，本机只有 11.0——编译器版本（gcc 12.2 vs 更老）与机器参数（本机 `addq` 延迟仅 1 周期）都不同。但**优化阶梯的形状完全一致**：未优化 → 消调用消内存（跃降 ~10 倍）→ 多累加器（再降 ~2 倍）。**请在自己机器上重测，不要背数字，要理解为什么。**

15.3.3 汇编对比：编译器到底有没有用寄存器

这是本章最关键的证据。用 `gcc –O1 –S demo.c` 反汇编两个版本的**内层循环**：

v1 的内层循环（每轮都在写内存）：

```
.L15:
        leaq    8(%rsp), %rdx        # val 的地址在栈上
        movq    %rbx, %rsi           # idx
        movq    %r12, %rdi           # v
        call    get_vec_element      # <== 每轮一次函数调用（含边界检查）
        movq    8(%rsp), %rax        # <== 从栈上把 val 读回来
        addq    %rax, 0(%rbp)        # <== 读 *dest、加、写回 *dest：内存往返
        addq    $1, %rbx
.L14:
        movq    %r12, %rdi
        call    vec_length           # <== 每轮还要调一次 vec_length!
        cmpq    %rbx, %rax
        jg      .L15
```

v2 的内层循环（累加器锁在寄存器里）：

```
        call    vec_length           # 循环外调用一次
        call    get_vec_start        # 循环外调用一次
.L21:
        addq    (%rdx), %rax         # <== %rax 就是累加器：直接在寄存器里累加
        addq    $8, %rdx             #     指针自己递增，不再做边界检查
        cmpq    %rcx, %rdx
        jne     .L21
        movq    %rax, (%r12)         # <== 整个循环只有这一次内存写
```

【底层机制透视】 逐条数一数：v1 每轮 **2 次函数调用 + 1 次栈访问 + 1 次内存读改写**；v2 每轮 **1 条 `addq` 内存操作数 + 指针递增 + 比较 + 分支**。这解释了 CPE 从 11 到 1.09 的十倍跃降：函数调用要压栈、跳转、返回，而内存读改写的延迟远高于寄存器操作。**注意 `-O1` 已经足够让 GCC 把局部变量 `t` 放进 `%rax`**——因为 `t` 的地址从未逃逸。

对照讲义幻灯片里的经典低效形态（`*dest = *dest OP val`，`dest` 是**外部指针参数**，因此在 `-O1` 下是真正的内存读改写）：

```
        movq    (%rdx), %rax         # 读 b[i]
        addq    %rax, (%rbx)         # 加并写回 b[i]：编译器不敢放进寄存器
        addq    $8, %rdx
        cmpq    %rcx, %rdx
        jne     .L7
```

为什么编译器不敢把它放进寄存器？ 因为 `*dest` 与 `v->data` 可能**别名（alias）**——万一调用者让 `dest` 指向数组内部，把中间结果留在寄存器里就会漏掉每一次"写"的可见效果，程序语义就变了。这是 15.3.4 的主题。

一个值得记录的细节：`-O2` 下 GCC 对**指针写回**版本会做一次激进变换——一边在寄存器 `%rdx` 里累加，一边**每轮仍写回内存**：

```
.L8:
        addq    (%rax), %rdx         # 寄存器累加
        addq    $8, %rax
        movq    %rdx, (%rbx)         # 但仍老老实实每轮写一遍内存
        cmpq    %rax, %rcx
        jne     .L8
```

这是"**遵守语义**"前提下的最优解：它消掉了"读"（从寄存器读），但"写"必须保留。实测这一版 CPE = 1.036，与真正的局部变量版 1.094 **几乎相同**——现代处理器的 store buffer 会把连续写合并掉。**结论："消除内存引用"的收益主要体现在消除"读回"、以及能否把值提升进寄存器上。**

15.3.4 内存别名：`sum_rows1` vs `sum_rows2`

讲义用 `sum_rows` 系列演示**内存别名（Memory Aliasing）**的代价，这是本章最干净的对照实验：

```
/* sum_rows1：每轮都写 b[i]，编译器无法排除 b 与 a 别名 */
void sum_rows1(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += a[i*n + j];      /* 读 b[i]、加、写 b[i]：每轮一次内存往返 */
    }
}

/* sum_rows2：中间结果放局部变量，循环内完全不碰内存 */
void sum_rows2(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        double val = 0;
        for (j = 0; j < n; j++)
            val += a[i*n + j];
        b[i] = val;                  /* 每行只写一次内存 */
    }
}
```

【代码做什么？】 两个函数数学上都是"把 $n \times n$ 矩阵按行求和存进向量 b"。差别只在于**中间结果放在哪里**：`sum_rows1` 放在 `b[i]`（内存），`sum_rows2` 放在 `val`（寄存器）。

【底层机制透视】 编译器**不能**自动完成这个变换，因为它必须考虑 `a` 和 `b` **重叠**的合法情形。讲义给出了反例：设 `double A[9] = {0,1,2,4,8,16,32,64,128}`，令 `B = A + 3`（`B` 与 `A` 的第 3 行重叠），调用 `sum_rows1(A, B, 3)`。逐步跟踪（这段跟踪已在 `/tmp` 下真实运行验证）：

```
i=0: b[0]=0  ==> 把 A[3] 清零
     内层 j=0..2 读 A[0],A[1],A[2] = 0,1,2  ==>  b[0] = 3   (写回，即 A[3]=3)

i=1: b[1]=0  ==> 把 A[4] 清零
     内层读的是 A[3],A[4],A[5]，但注意内层每次 += 都会写回 A[4]，
     而下一次迭代又要读 A[4] —— 自己刚写的值被自己读到！
        j=0: 读 A[3]=3         -> b[1]=3   (写 A[4]=3)
        j=1: 读 A[4]=3(刚写的) -> b[1]=6   (写 A[4]=6)   <== 别名在"内层"就咬人了
        j=2: 读 A[5]=16        -> b[1]=22  (写 A[4]=22)

i=2: b[2]=0  ==> 把 A[5] 清零；内层读 A[6],A[7],A[8] = 32,64,128  ==>  b[2] = 224
```

最终 `B = [3, 22, 224]`，而**不重叠**时正确答案是 `B = [3, 28, 224]`（各行真和：$0+1+2=3$，$4+8+16=28$，$32+64+128=224$）。**关键洞察**：别名不只在"外层迭代之间"起作用，它在**内层循环内部**就改变了结果——因为每轮的写会被下一轮的读看到。这正是编译器**必须把 store 留在循环里**、无法提升到循环外的根本原因。

【实测验证】（`gcc -O1`，`double` 元素，CPE = 周期/元素）

矩阵维数 n	`sum_rows1`（写内存）	`sum_rows2`（局部变量）	加速比
512	11.475	2.784	**4.12×**
1024	11.708	2.899	**4.04×**
2048	11.866	2.976	**3.99×**

同一实验在 `-O2` 下 `sum_rows1` 变成 2.785 / 2.899 / 2.978，加速比降到 **1.00×**——因为 GCC 在 `-O2` 上能证明"每轮写的 `b[i]` 与读的 `a[...]` 不冲突"并做了变换（反汇编可见它把内层循环展开、用 `addsd` 累加后 `movsd` 写回）。**这正是"编译器能力边界"的完美例证**：同一个变换，在 `-O1` 的 `sum_rows1` 上编译器拒绝做（保守），在 `-O2` 上它自己就会做（安全）。**所以程序员最该做的是把代码写成编译器敢优化的形式**，而不是手写汇编。

`restrict` **是给编译器的承诺**：`void sum_rows2r(double *restrict a, double *restrict b, long n)` 告诉编译器"这一对指针访问的内存块**绝不重叠**"。有了这个承诺，编译器就敢做强得多的重排与向量化。**代价是：如果违反承诺（两块内存确实重叠），行为是未定义行为（UB）**——不是"结果不对"这么轻，而是编译器可以生成任何东西。本机实测 `sum_rows2` 与 `sum_rows2r` 在 `-O1` 下 CPE 完全一致（2.78 / 2.90 / 2.98），说明对这段代码 `restrict` 没带来额外收益；但在 15.3.6 的逐元素运算上它决定了能否向量化。

15.3.5 延迟界限与吞吐量界限的计算过程

现在用**真实测量**的参数验证 15.2.4 的公式。先用内联汇编直接测本机的延迟与发射时间（8 条独立依赖链法）：

运算	延迟（单链）	发射时间（8 条独立链）	反推功能单元数
`mulsd`（double 乘）	**2.998 ≈ 3 周期**	**0.500 周期/条**	**2.00 个**
`addsd`（double 加）	**2.998 ≈ 3 周期**	**0.502 周期/条**	**1.99 个**
`addq`（整数加）	**0.998 ≈ 1 周期**	**0.282 周期/条**	**3.55 个**（4 个 ALU 的测量上限）

为什么"8 条独立链"能测出发射时间：k 条互不依赖的链可以并行推进，只要 k 足够大把延迟完全掩盖，每迭代的时间就由发射带宽决定，即 $k \times T$；所以 $T = \text{cycles}/\text{迭代}/k$。取 $k = 8$ 是因为 $8 > L/T = 6$，肯定够掩盖延迟。**这里有一个方法论陷阱**：不能把空循环（`decq/jnz`，1 周期/迭代）的 1 周期从测量值里直接减掉——因为 `decq/jnz` 与运算指令是**并行**发射的，减掉会重复扣除。正确做法是**直接读原始 cycles/迭代**，再按链数换算。

（一）浮点乘 `combine_d`（`OP = *`，`IDENT = 1.0`）的界限验证

本机参数 $L = 3$，$T = 0.5$，单元数 $C = 2$，代入 $CPE = \max(L/k,\ T)$：

累加器数 k	延迟界限 L/k	吞吐量界限 T	理论 CPE	实测 CPE（`-O2`，$n = 10^6$）
1（v2）	$3/1 = 3.00$	0.5	3.00	3.001 ✔
4（v3）	$3/4 = 0.75$	0.5	0.75	0.748 ✔
8（v4）	$3/8 = 0.375$	0.5	0.50	0.480 ✔

三行实测值与理论值几乎完全吻合，这是本章最有说服力的证据：**v2 是纯延迟受限（3.00），v3 仍是延迟受限（0.75），v4 已经撞上吞吐量天花板（0.48 ≈ 0.5）**。由此也可算出"够用"的累加器数 $k^* = L/T = 3/0.5 = 6$——**6 个累加器就能达到吞吐量界限，8 个纯属保险**。

对照讲义 Haswell 的同一计算：$L = 5$，$T = 1$，$C = 2$，则 $k^* = 5$，理论阶梯是 5 → 2.5 → 1.25 → 1.0。讲义表格中"Combine4 浮点乘 = 5.01"**精确等于延迟界限 5**，"展开+多累加器 = 2.51"**精确等于 $5/2 = 2.5$**。两个平台、两套参数、同一个公式，结论完全一致。

（二）整数加 `combine`（`OP = +`，`IDENT = 0`）为什么"看起来超越了界限"

本机 `addq` 延迟 1、发射时间 0.282，理论延迟界限 1、吞吐量界限 0.282。但实测 v2（1 个累加器）= **1.094**，v4（8 个累加器）= **0.461**——都**高于** 0.282。原因在于整数运算太便宜了，瓶颈转移到了**加载单元（load unit）**：每次迭代必须从内存取一个 `long`，而加载端口的吞吐量是有限制的：

$$CPE_{\text{load}} = \frac{1}{\text{每周期可发射的 load 数}} = \frac{1}{2} = 0.5$$

实测 0.461–0.478 与 0.5 非常接近（略低，说明实际可维持的加载速率略高于每周期 2 次）。**这个例子极其重要**：它说明"延迟界限 / 吞吐量界限"不是只有两条，而是**所有资源下界中的最大值**——运算单元、加载单元、存储单元、分支单元各有一条界限，真正的 CPE 是它们的上包络。**所以当运算变得足够便宜时，瓶颈一定会转移到访存上**，这也是 L4 Cache Lab 要考的核心。再往上加累加器完全无用：整数加的实际天花板由 LSU 决定，而不是由 ALU 决定。

（三）数据超出缓存后的第三个界限：内存带宽

把 n 从 10^6（8 MB，L3 驻留）增加到 10^7（80 MB，DRAM），所有版本都被拉平：

方法	$n = 10^6$ (8 MB) CPE	$n = 10^7$ (80 MB) CPE
v2（1 累加器）	1.094	1.340
v4（8 累加器）	0.461	**1.000**

$n = 10^7$ 时 v4 的 CPE = 1.000 对应 $80\,\text{MB}/3.25\,\text{ms} \approx 24.6$ GB/s 的单核 DRAM 带宽。**无论怎么优化计算，都突破不了这堵墙**——此时唯一的出路是改善局部性（详见 Lecture 9—10 的存储器山与分块）。这也解释了为什么优化阶梯在高 n 下会"塌缩"：8 MB 时 v2 是 1.094、v4 是 0.461（差 2.4 倍），80 MB 时变成 1.340 vs 1.000（只差 1.3 倍）。

15.3.6 突破：AVX2 / FMA 自动向量化

前面所有优化都还在"每周期处理一个元素"的量级。真正再上一个数量级要靠 **SIMD（Single Instruction Multiple Data）**。用 `-O3 -mavx2 -mfma` 编译后，GCC 能自动把循环向量化：

```
# gcc -O3 -mavx2 -mfma 对 combine_v2 的自动向量化
        vpxor   %xmm0, %xmm0, %xmm0          # 向量累加器清零
.L40:
        vpaddq  (%rdx), %ymm0, %ymm0         # <== 一条指令加 4 个 long (256 位)!
        addq    $32, %rdx
        cmpq    %rdi, %rcx
        jb      .L40
        vextracti128 $0x1, %ymm0, %xmm0      # 水平归约：把 4 条 lane 加起来
        vpsrldq $8, %xmm0, %xmm1
        vpaddq  %xmm1, %xmm0, %xmm0
        vmovq   %xmm0, %rdx
```

逐元素运算 `a[i] = b[i]*k + c[i]`（讲义 Scheduling 一节）则被编译成 **FMA 指令**：

```
# gcc -O3 -mavx2 -mfma 对 scale4 的自动向量化
        vmovupd         (%r9,%rax), %ymm1            # 一次载入 4 个 double
        vfmadd213pd     (%r8,%rax), %ymm2, %ymm1     # <== FMA: a = b*k + c, 一条指令算 4 个
        vmovupd         %ymm1, (%rdi,%rax)
```

【底层机制透视】 `vfmadd` 把"乘"和"加"融合成一条指令，**既省指令又省一次舍入**（中间结果不截断，精度更高）。相比 15.3.2 中 v2 的 0.529 CPE，向量化后每周期能处理 2 个 double 甚至更多。

GCC 自动向量化的三个前提条件（缺一不可）：1. **无别名（no aliasing）**：编译器必须能证明读的数组与写的数组不重叠。这就是 `restrict` 的作用；缺了它，GCC 会保守地拒绝向量化。 2. **无数据依赖（no loop-carried dependency）**：如果第 i 次迭代要读第 $i-1$ 次写的结果（如 `a[i] = a[i-1] + b[i]`），无法并行。 3. **连续访问（unit stride）**：步长必须为 1，才能用 `vmovupd` 一次载入连续 4 个元素。跨步访问（如按列遍历）会让向量化失败。

注意：本机实测 `-O3 -mavx2 -mfma` 下 `sum_rows1` vs `sum_rows2` 加速比只有 1.00—1.07×，`scale4` 在 $n = 10^7$ 时有效带宽只有 31.7 GB/s——**向量化不是万能药**：瓶颈是内存带宽时 SIMD 无法突破。**先判断瓶颈在哪，再决定优化手段。**

15.4 实验关联

L4 Cache Lab Part B（优化矩阵转置） 是本讲技巧的直接战场。`trans.c` 要在 `-O1` 下把 $N \times N$ 矩阵转置的 CPE 压到阈值以下，本质就是本讲四板斧的叠加： – **消除内存引用**：块内转置时用**局部变量**暂存一整行（而非逐个 `B[j][i] = A[i][j]`），避免对 `B` 的反复读改写——这正是 `sum_rows1` → `sum_rows2` 的同一招。 – **循环展开 + 分块（blocking）**：32×32 块让每次载入的缓存行被充分利用；64×64 需要 8×8 分块加"半行暂存"来同时避免 A、B 的对角线冲突。 – **代码移动**：把 `i*n`、基址加法的重复计算提出循环。 – **常见坑**：`-O1` 下 GCC 不做自动向量化，**必须手工做**；测试务必覆盖 $N = 32, 64, 128, 256, 512, 1024$，不能只针对 $32/64/128$ 调参。

L5 Malloc Lab（吞吐量部分） 用同一个思想：分配器 `find_fit` 每轮都调用辅助函数、反复读堆头部，CPE 极差；改成**内联 + 局部变量缓存堆指针与边界**，吞吐量能提升数倍。讲义 `LAB-perflab.txt`（早年以 `rotate` / `smooth` 两个图像算子评分的 Code Optimization Lab）给出历史基线可作对照：naive `rotate` CPE 14.7→94.5（随 N 恶化说明缓存失效），优化后 8.0→25.3，几何平均加速 3.1×；naive `smooth` CPE 约 700（计算密集），优化后约 41，加速 15.2×——**完美体现了"访存密集 vs 计算密集需要不同手法"**。

15.5 常见错误与调试技巧

- **迷信 `-O3`**：现象是加了 `-O3` 反而更慢。原因是向量化带来的对齐检查、代码膨胀挤爆 i-cache、或非临时存储在数据仍在缓存中时反而亏。**调试**：`gcc -O2 -S -o - file.c | less` 对比 `-O2` / `-O3` 的汇编差异；用 `perf stat -e cache-misses,cache-references,instructions,cycles ./prog` 看指令数与 miss 率是否同步改善。
- **用 UB 换性能**：现象是 `restrict` 化或指针强转后结果偶发错误。原因是违反 `restrict` 承诺、或 `-O3` 下依赖了未定义的有符号溢出。**调试**：`gcc -O1 -fsanitize=undefined,address -g file.c` 跑一遍；用 `-fno-strict-aliasing` 对比结果是否变化即可确认是否为别名 UB。
- **微基准测不准**：现象是同一份代码两次测得 CPE 差两三倍。原因是频率波动、CPU 迁移、其他进程干扰、n 太小截距主导。**调试**：`taskset -c 5 ./prog` 绑核；`for i in $(seq 20); do taskset -c 5 ./prog; done | sort -n | head` 取多轮最小值；用本讲的依赖链法先标定真实频率。
- **以为"展开就会快"**：现象是手工展开 4 倍后 CPE 纹丝不动。原因是只展开了、没增加累加器，依赖链长度没变。**调试**：看 CPE 是否卡在延迟界限上（用 15.3.5 的公式算一下预期界限），若等于 L/k 且 k 没变，就必须加累加器。
- **累加器加太多**：现象是加到 16 个累加器后反而变慢。原因是寄存器不够用导致**寄存器溢出（register spilling）**，把值写到栈上，每次访问又多一次内存往返。**调试**：`gcc -O2 -S -o - file.c | grep -c 'rsp'` 数栈访问；或 `perf stat -e ld_blocks.store_forward ./prog`。

- **混淆"测量"与"优化"**：现象是改进了 CPE 但程序总时间没变。原因是这段代码在总时间中占比很小（见 15.7 的 Amdahl 计算题）。**调试**：先用 `perf record -g ./prog && perf report` 定位热点函数，再优化——**不要猜热点**。
- **把计时逻辑和被测代码混在一起**：现象是 CPE 数值异常稳定地偏大。原因是把初始化、内存分配、`printf` 都算进了计时区间。**调试**：严格只对目标循环计时，重复 15 次以上，先跑一遍预热。

15.6 关键要点

- **CPE 是代码效率的语言，不是时间**：$Cycles = CPE \times n + Overhead$，用足够大的 n、多次重复取最小值，才能测出稳定斜率。
- **编译器的三条铁律决定了优化机会**：不改语义、不加副作用、不猜别名——"不能把函数调用移出循环"和"不敢把 `*dest` 放进寄存器"都是这三条的直接后果。
- **优化阶梯的形状是普适的**：消除过程调用（~10×）→ 消除内存引用（~1.3×）→ 循环展开与多累加器（~2×）→ SIMD（再 ~2×）；绝对数值随机器而变，但**顺序与瓶颈迁移规律不变**。
- **$CPE \approx \max(L/k,\ T)$ 是本章的骨架公式**：k 个累加器把延迟界限摊成 L/k，但永远打不穿吞吐量界限 $T = 1/C$；"够用"的累加器数是 $k^{*} = L/T$。本机实测浮点乘 $L = 3, T = 0.5$ 给出 $k^{*} = 6$，实测 1/4/8 累加器分别为 3.00/0.748/0.480 CPE，与理论 3.00/0.75/0.50 完全吻合。
- **瓶颈是多条下界的上包络**：运算单元、加载单元、存储单元、DRAM 带宽各有一条界限。整数加太便宜，瓶颈转移到 2 load/cycle（CPE 0.5）；数据超出缓存后，瓶颈是 ~25 GB/s 的单核带宽。
- **先测量，再优化，最后验证**：`perf` 找热点 → 改代码写成编译器敢优化的形式（局部变量、`restrict`、连续访问）→ 重新测量确认。

15.7 思考题（带答案）

题 1（计算题：延迟/吞吐量界限与累加器个数） 某处理器的浮点乘 `mulsd` 延迟 $L = 4$ 周期、发射时间 $T = 0.5$ 周期/条。求：(a) 功能单元数 C；(b) 用 1、2、4、8 个累加器做点积时各自的 CPE 下界；(c) "够用"的最少累加器个数 k^{*}；(d) 若实测 8 个累加器得到 CPE = 1.2，瓶颈最可能是什么？

答：(a) $C = 1/T = 1/0.5 = 2$ 个浮点乘法单元。(b) $CPE = \max(L/k, T) = \max(4/k, 0.5)$：– $k = 1$：$\max(4, 0.5) = 4.00$ – $k = 2$：$\max(2, 0.5) = 2.00$ – $k = 4$：$\max(1, 0.5) = 1.00$ – $k = 8$：$\max(0.5, 0.5) = 0.50$（已到吞吐量界限）(c) $k^{*} = L/T = 4/0.5 = 8$。所以 $k \geq 8$ 之后再加累加器毫无收益。(d) 实测 1.2 远高于理论 0.5，说明真正瓶颈不在乘法单元上——最可能是**加载延迟/带宽**（每次迭代要取一个元素）、**寄存器溢出到栈**（8 个累加器 + 指针 + 计数器超过可用寄存器），或**数据不在 L1**。排查顺序：先看汇编有无 `movsd ...(%rsp)` 这类栈访问，再看数据规模是否超出 L1/L2。

题 2（计算题：Amdahl 定律） 某程序中 `combine` 占运行时间的 60%，其余 40% 与它无关。你把 `combine` 从 CPE 1.094 优化到 0.478（本机 `-O2` 实测的 v2 → v4）。(a) `combine` 自身的加速比是多少？(b) 整个程序的加速比是多少？(c) 若要把整体加速比做到 2.0，`combine` 需要再快多少倍？

答：(a) $S_{\text{combine}} = 1.094/0.478 = 2.29\times$。(b) 设原总时间为 1，则 `combine` 从 0.60 降到 $0.60/2.29 = 0.262$，其余仍为 0.40，新总时间 $= 0.662$。整体加速比 $= 1/0.662 = 1.51\times$。(c) 设 `combine` 需要加速 S 倍，则 $\dfrac{1}{0.4 + 0.6/S} \geq 2 \Rightarrow 0.4 + 0.6/S \leq 0.5 \Rightarrow 0.6/S \leq 0.1 \Rightarrow S \geq 6\times$。注意 Amdahl 定律的硬上限：**即使 `combine` 快到耗时为零，整体加速比也只有 $1/0.4 = 2.5\times$**——所以超过 6 倍的努力已经没有意义，应该去优化那 40%。

题 3（"直观但错误的想法"） 同学 A 说："`-O3` 比 `-O2` 优化级别高，所以任何代码用 `-O3` 编译一定更快。"同学 B 说："既然循环展开能减少分支开销，我把循环展开 16 倍，性能一定最好。"同学 C 说："`restrict` 只是注释，加了没坏处，加满就行。"同学 D 说："既然 `sum_rows1` 慢是因为每轮写 `b[i]`，编译器只要聪明一点，把结尾时的值写一次就行了。"请分别指出错在哪。

答：– **同学 A 忽略代码大小与向量化的代价**。`-O3` 会更激进地展开与向量化，导致代码膨胀、i-cache 失效、以及为对齐插入的运行时检查；实测 `-O3 -mavx2` 下 `sum_rows1` vs `sum_rows2` 的收益甚至只有 1.00–1.07×（`-O1` 下反而是 4.0×），因为向量化对已访存受限的代码无能为力。**正确做法是两种都测**。 – **同学 B 混淆了"减少循环开销"与"打破依赖链"**。展开只减少分支/索引开销；若不增加**累加器**，依赖链长度不变，CPE 依然被延迟界限 L 卡住——实测 `v2u`（展开 2、单一累加器）= 0.997，与不展开的 v2 = 1.094 几乎相同，而 `v3`（4 累加器）= 0.478。过度展开还会导致**寄存器溢出**，中间值被挤到栈上，反而更慢。 – **同学 C 错在 `restrict` 是可被违反的承诺，违反即 UB**。它向编译器保证两块内存不重叠；一旦实际重叠（如 `f(a, a+3)`），编译器可基于该承诺做任意重排，产生"看起来完全莫名"的错误结果，而且**在 `-O1` 下可能正常、`-O3` 下崩溃**。正确用法是只在确实不重叠的接口上加，并当作 API 契约维护。 – **同学 D 错在这个变换并非总是等价**。讲义的反例是 `double A[9]={0,1,2,4,8,16,32,64,128}; double B[3]=A+3;` 后调用 `sum_rows1(A,B,3)` —— `b` 指向 `a` 的第 3 行，**写 `B[0]` 就是写 `A[3]`**。实测最终 `B = [3, 22, 224]`，而**不重叠**时的正确答案是 `[3, 28, 224]`。更关键的是：别名在**内层循环内部**就改变了结果——`i=1` 时内层先把 `A[4]` 清零、之后每轮 `+=` 都写回 `A[4]` 而下一轮又读回 `A[4]`，把 `4` 变成了 `3`。所以编译器**无法排除** `a` 与 `b` 重叠（15.2.2 第三条铁律），只能保守地每轮读写内存；要让它优化，程序员必须**主动消除歧义**（改用局部变量或加 `restrict`）。

Lecture 16: 进程与多任务 (Processes and Multitasking)

> **讲义对应**：CMU 15–213 Lecture 16 — Processes and Multitasking（素材：`F25-16-processes.txt`；配套 recitation：`F25-rec09_slides.txt`） **教材对应**：CS:APP3e 第 8 章 8.1–8.4（异常、进程、系统调用与进程控制） **关联 Lab**：L6 Shell Lab（`tsh`）——本讲是 Shell Lab 的全部基础

16.1 概述

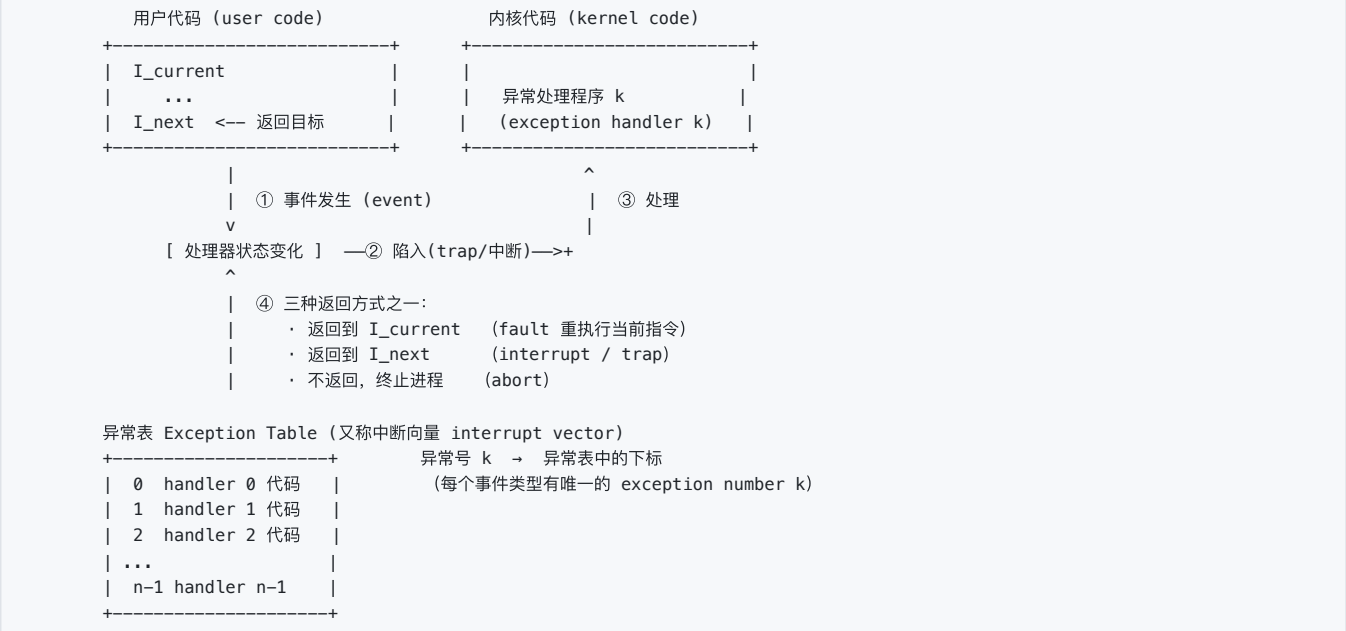
从 1957 年 NASA 的 IBM 704 一次只能跑一个批处理作业，到今天一台笔记本上同时有几百个"任务"，中间隔着一整套操作系统机制。本讲的核心问题是：**一台只有有限 CPU 核的机器，如何让许多程序"同时"运行，并且彼此互不干扰?**

答案是操作系统提供的两个虚假承诺："每个程序独占 CPU"（由**上下文切换**实现）与"每个程序独占内存"（由第 11–12 讲的**虚拟内存**实现）。支撑它们的底层机制是**异常控制流（Exceptional Control Flow, ECF）**——程序计数器从 a_k 突然跳到 b_k 的那种"突变"。承接 Lecture 15 的代码优化，本讲把视角从"一条指令怎么快"提升到"整个系统怎么调度"；通向 Lecture 17 的信号与非局部跳转，并直接为 L6 Shell Lab 铺路。

16.2 核心概念与底层机制图解

16.2.1 异常控制流（Exceptional Control Flow, ECF）

- **定义与目的**：ECF 是"处理器状态发生变化时，控制权从当前指令流转移到内核"的机制，存在于计算机系统的**所有层次**：低层是硬件 + 操作系统共同实现的**异常（exception）**；高层依次是**进程上下文切换**（OS + 硬件定时器）、**信号**（OS 软件）、**非局部跳转** `setjmp` / `longjmp`（C 运行时库）。本讲聚焦前两者，信号留到 Lecture 17。
- **直观解释（"它是什么？"）**：普通控制流像一列按时刻表行驶的火车，只能沿轨道一站一站走（`jmp` / `call` / `ret` 还在轨道内）。ECF 则像车上的**紧急制动 + 调度中心**：定时器一跳闸（中断），火车必须立刻靠边，由调度中心决定下一趟走哪辆车。程序员看不到这个切换，但它的后果（并发、竞态）无处不在。
- **底层机制图解**：



- **与机器码/硬件的对应**：异常号 k 加在**异常表基址寄存器**上，得到处理程序入口。用户程序请求内核服务时执行 `syscall` 指令（历史上是 `int 0x80`）：系统调用号放在 `%rax`，参数依次放 `%rdi, %rsi, %rdx, %r10, %r8, %r9`，返回值放 `%rax`。真实的 `open` 封装：

```
00000000000e5d70 <__open>:
   ...
   e5d79:  b8 02 00 00 00   mov  $0x2,%eax   # open 是 2 号系统调用
   e5d7e:  0f 05            syscall          # 返回值在 %rax
   e5d80:  48 3d 01 f0 ff ff cmp $0xfffffffffffff001,%rax
   ...
   e5dfa:  c3               retq
```

它"几乎像一次函数调用"：同样有控制转移、同样用调用约定传参、同样在 `%rax` 拿结果，返回后执行下一条指令。**但有一处根本不同——它由内核以更高特权执行**，且失败时返回一个 -1 到 -4095 之间的负值（对应 `errno` 的负值）。

16.2.2 四类异常（Interrupt / Trap / Fault / Abort）

- **定义与目的**：按"事件来自处理器外部还是当前指令"以及"返回行为"划分，硬件 ECF 分成四类。**异步异常（asynchronous exception）** 由处理器外部事件引起，通过置位处理器的**中断引脚（interrupt pin）** 通知 CPU；**同步异常（synchronous exception）** 由正在执行的指令直接引起。
- **直观解释**：异步就像手机来电——不管你正在做什么都会打断你；同步就像你写字时笔断了——是你手上这个动作直接导致的后果。
- **底层机制图解（对照表）**：

```
+----------+--------+----------+---------------------+---------------------------+
|  类别    | 原因   | 异步/同步 | 返回行为            | 典型例子                  |
+----------+--------+----------+---------------------+---------------------------+
| 中断     | 来自处理器|          | 返回到 I_next       | 定时器中断(每几毫秒)、    |
| Interrupt| 外部   | 异步     | (下一条指令)        | I/O 中断、Ctrl-C、网络包到达|
+----------+--------+----------+---------------------+---------------------------+
| 陷阱     | 有意为之 |          | 返回到 I_next       | 系统调用(syscall)、       |
| Trap     | 的指令  | 同步     | (下一条指令)        | gdb 断点                  |
+----------+--------+----------+---------------------+---------------------------+
| 故障     | 指令引起的|          | 重执行 I_current    | 缺页(page fault, 可恢复)、 |
| Fault    | 意外   | 同步     | 或终止进程          | 保护故障、浮点异常        |
+----------+--------+----------+---------------------+---------------------------+
| 终止     | 意外且  |          | 直接终止当前程序    | 非法指令、奇偶校验错、    |
| Abort    | 不可恢复 | 同步     | (不返回)            | 机器检查(machine check)   |
+----------+--------+----------+---------------------+---------------------------+
```

- **与机器码/硬件的对应**：x86-64 常见异常号（**补充说明**，取自教材 Figure 8.9）：0 = 除法错误，13 = 一般保护故障，14 = 缺页，18 = 机器检查，32–255 由操作系统自定义（Linux 用 128 号即 `0x80`）。缺页最能说明 fault 的语义：

```
80483b7:  c7 05 10 9d 04 08 0d  movl  $0xd,0x8049d10   # a[500] = 13;
```

这条指令触发缺页 → 内核把页从磁盘调入内存 → **重新执行同一条** **`movl`**，这次命中。若访问的是非法地址（如 `a[5000]`），内核无法恢复，改向进程发送 `SIGSEGV`，进程以 "segmentation fault" 退出——同一个 fault 机制，两种结局。

16.2.3 用户模式与内核模式（User Mode / Kernel Mode）

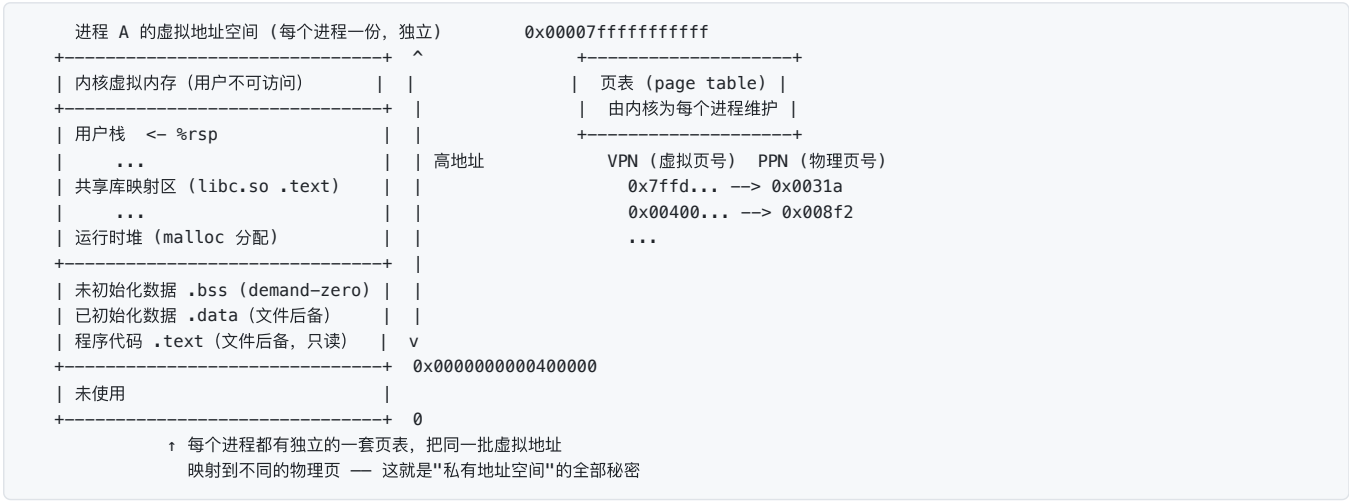
- **定义与目的**：处理器用**模式位（mode bit）** 标记当前特权级。置位（内核模式）时进程可执行**特权指令（privileged instructions）**、访问全部内存与 I/O 端口；清零（用户模式）时受限。**为什么需要它**：若用户程序能随意改写页表、关中断、直接操作磁盘控制器，任何一个 bug 或恶意程序都能毁掉整台机器。模式位是"内核受保护、进程之间相互隔离"的最后一道物理防线。
- **直观解释（"它是什么？"）**：普通员工（用户模式）只能进自己办公室和公共走廊；物业管理员（内核模式）持万能钥匙，能进配电间。唯一合法的"召唤管理员"方式，就是按下墙上的对讲机——这正好就是 `syscall` 指令所做的事。
- **底层机制图解**：

```
    用户模式 (mode bit = 0)                内核模式 (mode bit = 1)
  +-------------------------+            +---------------------------+
  | 自己的代码/数据          |            | 页表、中断描述符表        |
  | 自己的栈、堆            |  syscall   | 设备寄存器、I/O 端口      |
  | 只能通过 /proc /sys 观察 | ---------> | 特权指令 (cli/hlt/lgdt…)  |
  | 内核的"只读窗口"         | <--------- | 全部物理内存              |
  +-------------------------+    ret     +---------------------------+
        /proc/PID/maps, /proc/PID/status, /sys/... 是用户态读内核状态的接口
```

- **与机器码/硬件的对应**：`syscall` 之外的任何"越权"动作都会产生**一般保护故障**（异常号 13），内核据此终止进程。Linux 把 `/proc` 与 `/sys` 实现为**伪文件系统**：读 `/proc/self/maps`（虚拟内存各区间映射）、`/proc/self/status`（任务状态）时，内核在 `read` 系统调用里现算现返回，磁盘上并不存在这些数据。

16.2.4 进程与它的两个关键抽象

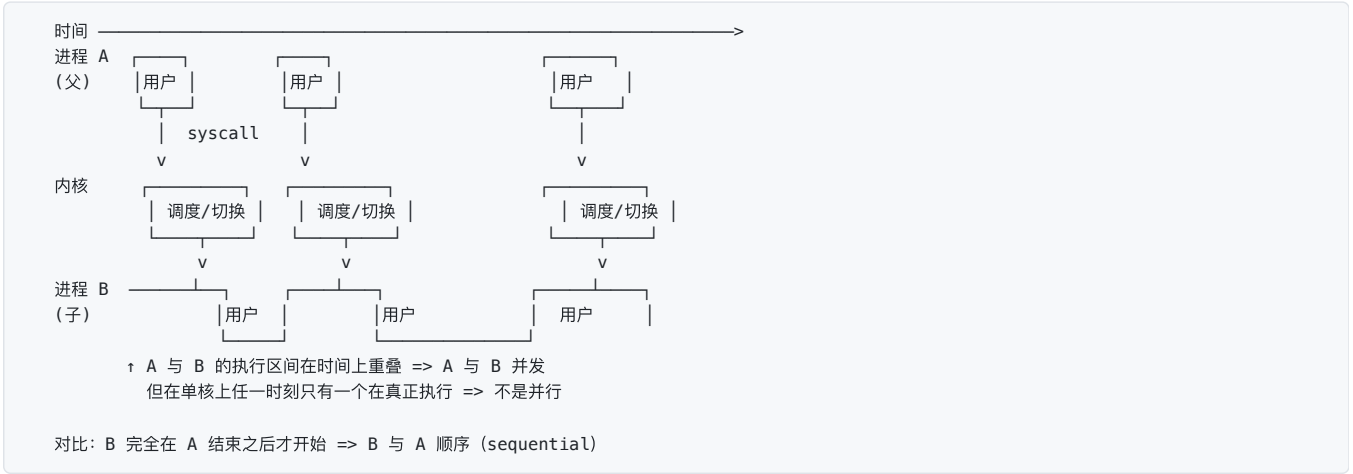
- **定义与目的**：**进程（process）是一个执行中程序的实例**。注意它与"程序"（磁盘上的静态文件）和"处理器"（执行单元）都不同。进程提供两个关键抽象：① **私有地址空间（private address space）**——每个程序看起来独占主存，由虚拟内存实现；② **逻辑控制流（logical control flow）**——每个程序看起来独占 CPU，由上下文切换实现。
- **直观解释**：进程就像**独立公寓**：墙（地址空间隔离）让你看不到也碰不到邻居的家具（其他进程的内存），水电闸（CPU、寄存器）也都是"你一个人的"——实际上整栋楼是分时共享的。
- **底层机制图解（进程地址空间与页表）**：



- **与机器码/硬件的对应**：每次上下文切换都要**换页表基址寄存器**（x86-64 的 `%cr3`），这使得 TLB 失效、地址翻译重新开始——这也是上下文切换比普通函数调用昂贵得多的原因之一。切换的触发者可以是 `syscall`（进程主动陷入内核，如 `read` 阻塞），也可以是**定时器中断**（内核借此夺回控制权，实现抢占式调度）。内核不是独立进程，它"跑在某个既有进程的上下文里"。

16.2.5 并发流、多任务与上下文切换

- **定义与目的**：让单个 CPU 核交错执行多个进程称为**多任务（multitasking）**，分配的时间段称为**时间片（time slice）**。如果两个进程的执行在时间上**重叠**，则称它们是**并发（concurrent）的**；否则是**顺序（sequential）的**。保存当前进程寄存器、恢复下一个进程寄存器并切换地址空间的过程，就是**上下文切换（context switch）**。
- **关键区分**：**并发**只要求时间上重叠，**并行（parallel）** 要求物理上真正同时执行。单核上 A、B 只能并发（交错）而非并行；多核上才可能并行。多核共享主存（以及部分缓存），每个核可执行不同进程，**把处理器调度到核上由内核完成**。
- **底层机制图解（时间轴：父/子进程的逻辑控制流）**：



- **与机器码/硬件的对应**：上下文切换保存的是**寄存器状态 + 地址空间标识**；单核实现"保存当前寄存器到内存 → 调度下一个进程 → 装载被保存的寄存器并切换地址空间"三步。

16.2.6 进程组与作业（Process Group / Job）

- **定义与目的**：**进程组（process group）** 是一组进程的集合，由正整数 PGID 标识；`setpgid(pid, pgid)` 设置、`getpgrp()` 查询。**作业（job）**是一次命令行解析所产生的全部子进程的总称。Shell 用进程组组织作业，从而支持**作业控制（job control）**。
- **直观解释**：进程组就像"一个施工队"，队长的 PID 就是队号。要通知整个队停工，不必挨个打电话——冲着队号喊即可：`kill(-pgid, sig)` 中的负号正是"发给整组"。
- **底层机制图解**：

```
终端 (terminal driver)
  |  用户按 Ctrl-C / Ctrl-Z
  |  终端驱动【不知道】shell 的作业表，它只认一件事：
  v  把信号发给【前台进程组】(foreground process group)
+-------------------------------------------------------+
| 前台进程组 PGID = 100                                  |
|   [100] tsh 的子进程 (job 1)  ← 收到 SIGINT/SIGTSTP   |
+-------------------------------------------------------+
| 后台进程组 PGID = 105   (job 2)  —— 收不到，继续跑     |
| 后台进程组 PGID = 110   (job 3)  —— 收不到，继续跑     |
+-------------------------------------------------------+
关键推论：子进程 fork 后必须 setpgid(0, 0) 把自己放进【新】进程组，
         否则它会留在 shell 的前台进程组里，Ctrl-C 会连带打死 shell 自己。
```

- **与机器码/硬件的对应**：`kill` 系统调用（编号 62）的 `pid` 参数为负时表示"向 PGID = |pid| 的进程组发送信号"。这正是 Shell Lab 里 `sigint_handler` / `sigtstp_handler` 必须写 `kill(-pid, SIGINT)` 的原因——`sdriver.pl` 会专门测试这个错误。

16.3 代码示例与底层机制分析

16.3.1 示例一：fork —— 调用一次，返回两次

代码 (C)：

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    int x = 1;
    pid_t pid;

    pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {
        /* 子进程：fork 在这里返回 0；它拥有 x 的独立副本 */
        x++;                    /* 只影响子进程的 x */
        printf("[child ] pid=%d ppid=%d | x=%d\n",
               (int)getpid(), (int)getppid(), x);
        return 0;               /* 或者 exit(0); 二者等价 */
    }
    /* 父进程：fork 在这里返回子进程 PID；它拥有 x 的另一份副本 */
    x--;                        /* 只影响父进程的 x */
    printf("[parent] pid=%d ppid=%d | x=%d\n",
           (int)getpid(), (int)getppid(), x);
    waitpid(pid, NULL, 0);      /* 回收子进程，避免僵尸 */
    return 0;
}
```

【代码做什么？】 ① 主进程调用 `fork()`；② 内核复制出一个几乎相同的进程，把同一个 `fork()` 调用点变成两个返回点——父进程拿到子进程 PID，子进程拿到 0；③ 父子各自独立修改自己的 `x`（两边初值都是 1，之后互不影响）；④ 父进程 `waitpid` 回收子进程。

【底层机制透视】 `fork` 的概念模型是"完整复制执行状态：寄存器、内存、被保存的寄存器，然后指定一个为父、一个为子"。逐字节复制内存极其昂贵，实现上采用**写时复制（copy-on-write）**：父子先共享同一批只读物理页，任一进程试图写入时触发保护故障，内核才复制那一页。子进程还**继承父进程的文件描述符**（`stdin`/`stdout`/`stderr` 指向同一批打开文件），所以父子输出会交错到同一终端上。

【内存布局 / 数据结构图解】 关键点是"同一个虚拟地址，两份不同的物理内容"：

```
父进程虚拟地址空间                子进程虚拟地址空间
+------------------+           +------------------+
|  ...             |           |  ...             |    虚拟地址相同：
|  x @ 0x7ffd1234  |           |  x @ 0x7ffd1234  |    x 都在 0x7ffd1234
|    值 = 0        |           |    值 = 2        |    物理页不同！
|  ...             |           |  ...             |
+------------------+           +------------------+
父进程 PID = 3575538            子进程 PID = 3575540
父进程 PPID = 2126633 (shell)   子进程 PPID = 3575538 (父进程)
```

【与汇编 / 硬件的对应】 栈上的 `x` 由 `-4(%rbp)` 之类的地址访问。`fork` 返回后，glibc 通过比较 `%rax`（系统调用返回值）区分父子：`testq %rax,%rax; je .Lchild`。x86-64 上 `fork` 实际由 `clone(CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD)` 实现。

【实测验证】 真实运行（`gcc -g -Wall -std=c11 forkdemo.c -o forkdemo`，本机 Linux x86-64）：

```
$ ./forkdemo
[child ] pid=3575540 ppid=3575538 | x=2
[parent] pid=3575538 ppid=2126633 | x=0
```

无论跑多少次，**每一行的内容都是确定的**（子必打印 `x=2` 且 `ppid` 等于父的 `pid`，父必打印 `x=0`），但**两行的先后顺序不确定**——讲义给出的三次运行结果中就有一次是子在前。`strace -f -e trace=clone,execve,wait4 ./forkdemo` 证实了这条链路：

```
execve("./forkdemo", ["./forkdemo"], ...) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, ...) = 3578767
[pid 3578763] wait4(3578767, <unfinished ...>
[pid 3578767] exit_group(0)             = ?
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=3578767, si_status=0} ---
<... wait4 resumed>NULL, 0, NULL)       = 3578767
```

16.3.2 示例二：真实的 fork 陷阱——输出重复

代码 (C)（⚠️ 仅供演示，请勿模仿）：

```c
/* forktrap_bad.c —— 故意制造"同一行打印两遍" */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    printf("BUFFERED-LINE");        /* 无换行、无 fflush：仍在用户态缓冲区 */
    if (fork() == 0) {
        exit(0);                    /* 子进程用 exit()，会冲刷 stdio 缓冲区 */
    }
    exit(0);                        /* 父进程也冲刷一次 */
}
```

【代码做什么？】 `printf` 的内容因为**没有换行且 stdout 被重定向**（全缓冲模式），停留在 glibc 用户态的 `FILE` 缓冲区里，一个字节都没写到内核。随后 `fork()` 复制整个地址空间——**包括这个缓冲区**。父子各自 `exit()` 时各冲刷一次，同一行于是被写两遍。

【底层机制透视】 这是"fork 复制地址空间"的直接后果，也是讲义"Shared open files"那句话的反面：**打开文件表共享，stdio 缓冲区私有**。修复只需两步（`fork` 前 `fflush`，子进程用 `_exit`）：

```c
/* forktrap_good.c —— 正确做法 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    printf("BUFFERED-LINE");
    fflush(stdout);                 /* 关键：fork 前把用户态缓冲区排空 */
    if (fork() == 0) {
        _exit(0);                   /* 关键：_exit 不触碰 stdio 缓冲区 */
    }
    _exit(0);
}
```

【实测验证】 真实运行结果对比（两者都重定向到文件，即触发全缓冲）：

```
$ ./forktrap_bad  > bad.out ; wc -c bad.out
26 bad.out

$ cat -A bad.out
BUFFERED-LINEBUFFERED-LINE        <- 同一行出现两次（26 = 13 × 2 字节）

$ ./forktrap_good > good.out ; wc -c good.out
13 good.out

$ cat good.out
BUFFERED-LINE                     <- 只出现一次（13 字节）
```

在终端上直接运行 `forktrap_bad` 却只能看到一行——stdout 连着 tty 时是**行缓冲**，而缓冲区里没有换行符。所以这个 bug 只在"输出重定向 + 未 fflush"时暴露，这也正是 Shell Lab 里到处要写 `fflush(stdout)` 的原因。

16.3.3 示例三：waitpid 与 status 解析

代码 (C):

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

static pid_t spawn(int code, int sig, const char *tag)
{
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {
        if (sig != 0) {
            signal(sig, SIG_DFL);       /* 恢复默认动作，再自杀 */
            raise(sig);
            _exit(99);                  /* 若信号被忽略才会到这里 */
        }
        _exit(code & 0xff);             /* 退出码只保留低 8 位 */
    }
    printf("spawn %-10s -> pid=%d\n", tag, (int)pid);
    return pid;
}

static void report(pid_t pid)
{
    int status = -1;
    pid_t w = waitpid(pid, &status, 0);
    if (w < 0) { perror("waitpid"); return; }
    printf("  waitpid returned %d ; raw status = 0x%04x\n", (int)w, status);
    if (WIFEXITED(status))
        printf("  WIFEXITED=1  WEXITSTATUS=%d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("  WIFSIGNALED=1 WTERMSIG=%d (%s)  WCOREDUMP=%d\n",
               WTERMSIG(status), strsignal(WTERMSIG(status)), WCOREDUMP(status));
    else if (WIFSTOPPED(status))
        printf("  WIFSTOPPED=1 WSTOPSIG=%d\n", WSTOPSIG(status));
    printf("\n");
}

int main(void)
{
    printf("=== (a) 正常 exit(42) ===\n");      report(spawn(42, 0, "exit(42)"));
    printf("=== (b) 正常 exit(0) ===\n");       report(spawn(0, 0, "exit(0)"));
    printf("=== (c) 段错误 -> SIGSEGV(11) ===\n");
    {
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); exit(1); }
        if (pid == 0) {
            volatile int *p = (int *)0;   /* ⚠ 仅供演示：故意解引用空指针 */
            *p = 13;
            _exit(1);
        }
        printf("spawn %-10s -> pid=%d\n", "segv", (int)pid);
        report(pid);
    }
    printf("=== (d) 被 SIGINT(2) 杀死 ===\n");  report(spawn(0, SIGINT, "SIGINT"));
    printf("=== (e) exit(0x213) 只保留低 8 位 ===\n"); report(spawn(0x213, 0, "exit(0x213)"));
    return 0;
}
```

【实测验证】 真实运行输出：

```
$ ./waitsdemo
=== (a) 子进程正常 exit(42) ===
spawn exit(42)   -> pid=3576150
  waitpid returned 3576150 ; raw status = 0x2a00
  WIFEXITED=1  WEXITSTATUS=42

=== (b) 子进程正常 exit(0) ===
  waitpid returned 3576151 ; raw status = 0x0000
  WIFEXITED=1  WEXITSTATUS=0

=== (c) 子进程段错误 -> SIGSEGV(11) ===
  waitpid returned 3576152 ; raw status = 0x008b
  WIFSIGNALED=1 WTERMSIG=11 (Segmentation fault)  WCOREDUMP=128

=== (d) 子进程被 SIGINT(2) 杀死 ===
  waitpid returned 3576229 ; raw status = 0x0002
  WIFSIGNALED=1 WTERMSIG=2 (Interrupt)  WCOREDUMP=0

=== (e) 退出码 0x213 只保留低 8 位 ===
  waitpid returned 3576230 ; raw status = 0x1300
  WIFEXITED=1  WEXITSTATUS=19
```

【底层机制透视】 关键数字：`0x2a00` = 42 << 8，说明**正常退出的状态码放在 status 高字节**；`0x8b` = 139 = 128（core dump 标志）+ 11（SIGSEGV），说明**信号号在低 7 位，第 8 位是 core dump 标志**。recitation 的 `exit(0x213)` 得到 `0x1300`，因为 `WEXITSTATUS` **只返回 1 个字节**——$0x213 \bmod 256 = 0x13 = 19$。

【速查表：`waitpid` 选项与 `status` 宏】

```
  pid_t waitpid(pid_t pid, int *status, int options)
  ┌──── pid 取值 ──────┬──────────────────────────────────────────┐
  │ pid > 0            │ 只等 PID == pid 的那一个子进程           │
  │ pid == -1          │ 等【任意】一个子进程（等价于 wait）      │
  │ pid == 0           │ 等与调用者同一进程组中的任意子进程       │
  │ pid < -1           │ 等进程组 |pid| 中的任意子进程            │
  └────────────────────┴──────────────────────────────────────────┘
  ┌──── options 位 ────┬──────────────────────────────────────────┐
  │ 0                  │ 阻塞直到目标子进程终止/停止              │
  │ WNOHANG            │ 非阻塞：无子进程可回收时【立即】返回 0   │
  │ WUNTRACED          │ 除终止外，也报告【被停止】的子进程       │
  │ WCONTINUED         │ 也报告收到 SIGCONT 而恢复运行的子进程    │
  └────────────────────┴──────────────────────────────────────────┘
  返回值：> 0 = 被回收子进程的 PID ；0 = 仅 WNOHANG 且有子进程但都未终止 ；
          -1 = 出错（无子进程时 errno == ECHILD）

  ┌──── status 位域 ──────────────────────────────────────────────┐
  │  bit15..8          | bit7 | bit6..0                          │
  │  退出状态 (exit code)| coredump| 信号号 或 停止信号号 或 0x7f(SIGCONT) │
  └──────────────────────────────────────────────────────────────┘
  ┌──── 判定宏 ──────────┬──── 取值宏 ──────┬──────────────────────┐
  │ WIFEXITED(st)        │ WEXITSTATUS(st)  │ 正常 exit / main 返回 │
  │ WIFSIGNALED(st)      │ WTERMSIG(st)     │ 被未捕获信号杀死      │
  │                      │ WCOREDUMP(st)    │ 是否产生了 core 文件  │
  │ WIFSTOPPED(st)       │ WSTOPSIG(st)     │ 被信号停止（需 WUNTRACED）  │
  │ WIFCONTINUED(st)     │ —                │ 被 SIGCONT 恢复（需 WCONTINUED）│
  └──────────────────────┴──────────────────┴──────────────────────┘
  铁律：这些宏【必须】配合使用——先 WIFEXITED 判断，再取 WEXITSTATUS。
        直接读 WEXITSTATUS 一个被信号杀死的子进程，读出的是垃圾。
```

16.3.4 示例四：fork + execve 迷你 shell

代码 (C)：

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAXLINE 1024
#define MAXARGS 64

int main(void)
{
    char  buf[MAXLINE];
    char *argv[MAXARGS];
    extern char **environ;

    while (1) {
        printf("minish> ");
        fflush(stdout);
        if (fgets(buf, MAXLINE, stdin) == NULL)
            break;                                  /* Ctrl-D / EOF */

        int argc = 0;
        char *tok = strtok(buf, " \t\n");
        while (tok != NULL && argc < MAXARGS - 1) {
            argv[argc++] = tok;
            tok = strtok(NULL, " \t\n");
        }
        argv[argc] = NULL;
        if (argc == 0) continue;                    /* 空行忽略 */

        if (strcmp(argv[0], "quit") == 0)           /* 内建命令 */
            break;

        pid_t pid = fork();
        if (pid < 0) { perror("fork"); exit(1); }
        if (pid == 0) {
            execve(argv[0], argv, environ);         /* 成功则永不返回 */
            fprintf(stderr, "minish: %s: %s\n", argv[0], strerror(errno));
            _exit(127);
        }
        int status;
        if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); exit(1); }
        if (WIFEXITED(status))
            printf("[exit status %d]\n", WEXITSTATUS(status));
        else if (WIFSIGNALED(status))
            printf("[killed by signal %d]\n", WTERMSIG(status));
    }
    printf("bye\n");
    return 0;
}
```

【代码做什么？】 读一行 → 按空白切词成 `argv` → 内建命令就地执行 → 否则 `fork`，子进程 `execve` 加载目标程序，父进程 `waitpid` 等前台作业结束并报告死因。这就是讲义 `shellex.c` 的 `eval` 主干。

【底层机制透视】 `execve(filename, argv, envp)` 做四件事：① 用 `filename` 的内容覆盖当前进程的**代码、数据、栈**；② 按 `argv` 布置新用户栈（含 `argc`、`argv[]`、`envp[]`、`environ`）；③ 保留 **PID、已打开的文件描述符、信号上下文**；④ 把 PC 设为 `.text` 入口点。Linux 的实现是：释放旧 `vm_area_struct` 与页表 → 为新区域建 `vm_area_struct` 与页表（代码/已初始化数据由**目标文件后备**，`.bss` 与栈由**匿名文件后备**）→ 设置 PC；代码页与数据页随后由**缺页故障按需调入**。"调用一次，永不返回"——除非出错（此时返回 **−1** 并设 `errno`）。

【内存布局图解：新程序启动时的栈】

```
高地址 (Bottom of stack)
+---------------------------------+
| 环境变量字符串 "USER=..."        |
| ...                             |
| 命令行参数字符串 "/bin/ls","-lt"  |
+---------------------------------+
| envp[n] == NULL                 |  <- %rdx 指向 envp[]
| ...                             |
| envp[0] --+                     |
| argv[argc] = NULL               |  <- %rsi 指向 argv[]
| ...       |                     |
| argv[0] --|-------------------+---> "/bin/ls"
+---------------------------------+
| 未来 main 的栈帧                 |
+---------------------------------+
| libc_start_main 的栈帧           |  <- argc 在 %rdi
低地址 (Top of stack)
```

【实测验证】 真实的交互与输出（命令通过管道喂入）：

```
$ printf '/bin/echo hello from execve\n/bin/ls -d /tmp\n/tmp/c16/selfsig 2\n/tmp/c16/nosuchprog\n/bin/sleep 2\nquit\n' | ./minish
minish> hello from execve
[exit status 0]
minish> /tmp
[exit status 0]
minish> selfsig: pid=3578370 raising signal 2
[killed by signal 2]
minish> minish: /tmp/c16/nosuchprog: No such file or directory
[exit status 127]
minish> [exit status 0]
minish> bye
```

三种结局都覆盖到了：`/bin/echo` 正常退出（status 0）、`selfsig 2` 被 SIGINT 杀死（`WIFSIGNALED` 路径）、不存在的程序走 `execve` 失败路径（127，与真实 shell 的"命令未找到"约定一致）。注意：**execve 不搜索 `PATH`**，必须给出完整路径——这也解释了讲义示例里"必须写 `/bin/ls` 而不是 `ls`"。若要搜索 `PATH`，应使用包装函数 `execl` / `execv` / `execvp`（`execvp` 才会搜索 `PATH`）。

16.3.5 示例五：SIGCHLD 回收循环与竞态防护

代码 (C)（节选，Shell Lab 的核心骨架）：

```
/* 信号处理程序：一次 waitpid 调用只能回收一个孩子，必须循环 */
static void sigchld_handler(int sig)
{
    int olderrno = errno;                       /* 保存/恢复 errno */
    pid_t pid;
    int status;

    while ((pid = waitpid(-1, &status, WNOHANG | WUNTRACED)) > 0) {
        if (WIFEXITED(status)) {
            deletejob(pid);                     /* 正常终止：出队 */
        } else if (WIFSIGNALED(status)) {
            deletejob(pid);                     /* 被信号杀死：出队 */
        } else if (WIFSTOPPED(status)) {
            getjob(pid)->state = ST;            /* 被停止：【留在】作业表中 */
        }
    }
    errno = olderrno;
}

/* eval 中的经典竞态防护：先阻塞 SIGCHLD，fork，addjob，再解阻 */
sigset_t mask, prev;
sigemptyset(&mask);
sigaddset(&mask, SIGCHLD);
sigprocmask(SIG_BLOCK, &mask, &prev);           /* ① 阻塞 */
pid_t pid = fork();
if (pid == 0) {
    sigprocmask(SIG_SETMASK, &prev, NULL);      /* 子进程必须解除继承的屏蔽 */
    execve(argv[0], argv, environ);
    _exit(127);
}
addjob(jobs, pid, bg ? BG : FG, cmdline);       /* ② 登记 */
sigprocmask(SIG_SETMASK, &prev, NULL);          /* ③ 解阻 */
if (!bg) waitfg(pid);                           /* ④ 前台作业：等它结束 */
else     printf("[%d] (%d) %s", pid2jid(pid), pid, cmdline);
```

【代码做什么？】 ① 阻塞 `SIGCHLD`；② `fork`；③ 父进程 `addjob` 登记；④ 解阻。此后每当有子进程终止或停止，内核递达 `SIGCHLD`，处理程序用 `while ((pid = waitpid(-1, &status, WNOHANG|WUNTRACED)) > 0)` **循环回收所有已终止/停止的子进程**——因为信号**不排队**，多个孩子同时死掉只会递达一次 `SIGCHLD`，只调一次 `waitpid` 会漏掉其余的僵尸。

【底层机制透视：为什么必须先阻塞】 若不加阻塞，存在这样一个窗口：

```
   父进程                                  子进程
   fork() ──────────────────────────────────> 创建
     │  ← 此处被内核抢占，父进程还没跑 addjob
     │                                     exit(0) → 变成僵尸
     │                                     内核向父进程递达 SIGCHLD
     │  sigchld_handler 运行：
     │     waitpid(-1, ...) 回收了子进程
     │     deletejob(pid)  ── 但作业表里【根本没有这个 pid】！
     │                                       (deletejob 找不到，静默失败)
     │  addjob(pid, ...)  ← 迟到的登记，把一个【已死】的 pid 写进作业表
     v
   结果：作业表里留下一个永远无法回收的幽灵条目，jobs 命令谎报"运行中"。
```

阻塞 `SIGCHLD` 让"创建–登记"对信号处理程序而言成为**原子段**，从而封死这个窗口。另两个细节同样关键：**子进程要解除继承的屏蔽**（否则 `exec` 后的程序收不到 `SIGCHLD`），**处理程序要保存并恢复 `errno`**。

【实测验证】 用三个"fork 完立刻退出"的子进程 + 一个自我停止的子进程验证，输出为：

```
$ ./reapdemo
  [handler] pid 3576245 正常退出, exit status 10
  [handler] pid 3576246 正常退出, exit status 11
  [handler] pid 3576247 正常退出, exit status 12
[main] 等待中... nreaped=3
  [handler] pid 3576248 被信号 20 停止(作业仍在表中)
[main] 全部回收完毕, nreaped=4
```

`while` 循环在一次 `SIGCHLD` 里连续回收了 3 个已终止的子进程；`WUNTRACED` 则让第 4 个**被 `SIGTSTP`（20）停止**的子进程也被报告出来——这正是 `tsh` 需要区分 `ST`（停止）与 `BG`（后台运行）状态的原因。

16.4 实验关联

L6 Shell Lab（`tsh`） 是本讲的直接落地。任务是在 `tsh.c` 中补全七个函数：`eval`（解析并执行命令行，约 70 行）、`builtin_cmd`（识别 `quit` / `jobs` / `bg` / `fg`，25 行）、`do_bgfg`（50 行）、`waitfg`（等前台作业完成，20 行）、`sigchld_handler`（约 80 行）、`sigint_handler`、`sigtstp_handler`（各 15 行）。评分：16 个 trace × 5 分 = 80 分正确性 + 10 分风格（注释 5 分，**检查每一个系统调用的返回值** 5 分）。

必须记住的三个实验要点：① `sigint_handler` / `sigtstp_handler` 里用 `kill(-pid, sig)` 发给**整个前台进程组**，`sdriver.pl` 专门测试这个错误；② `waitfg` 用忙等循环（`while (fgpid(jobs) == pid) sleep(...)`），所有回收集中在 `sigchld_handler`，不要两边都调 `waitpid`；③ 子进程在 `fork` 之后、`execve` 之前调用 `setpgid(0, 0)` 自立进程组，否则 Ctrl–C 会连同 shell 一起打死。讲义还提醒：**不要**在 `tsh` 里运行 `more` / `less` / `vi` / `emacs`，它们会乱改终端设置；请用 `/bin/ls`、`/bin/ps`、`/bin/echo` 测试。背景知识按顺序读 CS:APP 第 8 章即可，本章已覆盖 8.1–8.4，信号部分见 Lecture 17。

16.5 常见错误与调试技巧

- **僵尸进程堆积（zombie / defunct）**：子进程已终止但父进程未 `wait`，`ps` 中显示 `<defunct>`，占用 PID 与内核表项。**调试**：`ps -o pid,ppid,stat,cmd -e | awk '$3 ~ /Z/'`；修法是父进程显式回收（shell/服务器这类长命进程必须回收）。
- **`fork` 后忘记 `exit`，子进程继续跑父进程代码**：同一段逻辑执行两遍、输出翻倍，甚至递归 fork 出进程树。**调试**：`pstree -p <pid>`、`strace -f -e trace=clone`。
- **输出重复（stdio 缓冲区被 fork 复制）**：见 16.3.2。**调试**：把 stdout 重定向复现；`strace -f -e trace=write` 数 write 次数；修法是 `fflush` + `_exit`。
- **`execve` 不搜索 `PATH`**：写 `ls` 而不是 `/bin/ls` 会得到 `No such file or directory`。**调试**：用 `execvp`；或 `printenv PATH` 确认目录列表。
- **只调一次 `waitpid` 就以为收完了**：信号不排队，多个子进程同时终止只会得到一次 `SIGCHLD`。**调试**：处理程序里打日志计数，对照 `ps --ppid <shell_pid>` 剩余僵尸数；必须用 `while (...) > 0`。
- **`status` 未解码**：直接读 `status` 得到 `0x2a00` 而不是 42。**调试**：始终用 `WIFEXITED` / `WEXITSTATUS`；gdb 里 `p/x status` 看原始位。
- **Ctrl–C 连 shell 一起打死**：子进程没 `setpgid(0, 0)`，留在 shell 的前台进程组里。**调试**：`ps -o pid,pgid,cmd -e | grep tsh` 对比 PGID；handler 必须用 `kill(-pid, sig)`，并保存/恢复 `errno`。
- **Ctrl–C 连 shell 一起打死**：子进程没 `setpgid(0, 0)`，留在 shell 的前台进程组里。**调试**：`ps -o pid,pgid,cmd -e | grep tsh` 对比 PGID；handler 里必须用 `kill(-pid, sig)`。

16.6 关键要点

- **进程 = 执行中程序的实例**，它靠两个虚构抽象活得好：私有地址空间（虚拟内存）与逻辑控制流（上下文切换）。
- **异常是"控制流的突变"，四类各有返回纪律**：中断与陷阱返回到 I_{next}，故障重执行 $I_{current}$ 或终止，终止永不返回。`syscall` 是"有意的陷阱"，是用户程序接触外部世界的唯一合法通道。
- **充分的隔离需要硬件特权级**：模式位把用户进程挡在页表、设备与特权指令之外，`/proc` 与 `/sys` 是留给用户态的只读窗口。
- **`fork` 复制地址空间（写时复制），`execve` 覆盖地址空间（保留 PID）**。二者组合是所有 shell 的全部秘密：`fork` + `execve` + `waitpid`。
- **"创建资源 + 登记资源"的两步操作都要考虑信号竞态**：阻塞信号 → `fork` → `addjob` → 解阻，顺序不能颠倒。同一模式在并发编程里会以"锁的顺序"再次出现（见 Lecture 21–22）。
- **长命进程必须回收子进程**，否则僵尸会把 PID 耗尽——shell 与服务器是仅有的两类"必须显式回收"的程序。

16.7 思考题（带答案）

题 1（推演题） 下面是 recitation 中的一段代码。请写出所有可能的输出行数与内容，并指出哪些输出顺序是**确定的**：

```
int main(void) {
    int status;
    if (fork() == 0) {
        pid_t pid = fork();
        printf("Child: %d\n", getpid());
        if (pid == 0) exit(0);
    }
    pid_t pid = wait(&status);
    printf("Parent: %d\n", pid);
    exit(0);
}
```

答：进程图有三个叶子（原始父 P、子 C、孙 G）。`printf("Child")` 会被 **C 和 G 各执行一次**，`printf("Parent")` 会被 P 和 C 各执行一次——因为 C 在 `fork` 之后没有 `exit`，会继续落到下面的 `wait` / `printf`。所以**恰好 4 行输出**：两行 `Child:`、两行 `Parent:`。确定的约束是：① `Child: <C的pid>` 一定在 C 的 `Parent: <G的pid>` **之前**（C 先 printf 再 wait）；② P 的 `Parent: <C的pid>` 一定在 `Child: <C的pid>` **之后**（P 通过 wait 等到 C 终止）；③ 两个 `Child` 行之间、两个 `Parent` 行之间的相对次序都自由。两种可行交错是：`Child(C), Child(G), Parent(C), Parent(P)` 与 `Child(C), Parent(C), Child(G), Parent(P)`；而 `Child(G)` 出现在 `Child(C)` 之前**不可行**（recitation 的答案正是这两种）。

题 2 小明说："我按 Ctrl-C 之后 shell 也退出了，所以 `tsh` 应该把 SIGINT 的默认动作设成 `SIG_IGN` 来避免被打死。" 这个想法错在哪?

答：错在根因判断。默认情况下 shell 创建的子进程与 shell 同属一个**前台进程组**，Ctrl-C 由**终端驱动**发给整个前台进程组，因此 shell 与子进程都被打到——问题出在**进程组划分**，不在信号处理。正确做法是子进程在 `fork` 之后、`execve` 之前 `setpgid(0, 0)` 自立门户，shell 自己安装 handler 捕获 SIGINT，再 `kill(-pid, SIGINT)` 转发给前台作业。若改成 `SIG_IGN`，shell 确实不死了，但 Ctrl-C 会彻底失效，且 `SIG_IGN` 会被 `execve` 后的程序继承，行为完全失控。

题 3（计算题） 某长命服务器每秒 fork 一个子进程，子进程 10 ms 后退出，而服务器从不调用 `wait`。假设 PID 空间为 **32768** 个且被僵尸占满后无法再创建新进程。问到系统无法 `fork` 需要多久？（忽略其他进程）

答：每秒产生一个僵尸且永不复用，僵尸数随时间线性增长。需要 **32768** 个僵尸，即约 **32768** 秒 ≈ **9.1 小时**。注意它不取决于子进程的执行时间（10 ms），只取决于创建速率与 PID 空间大小；实际 Linux 的 `pid_max` 默认是 **4194304**，对应约 48.5 天——但只要有别的进程也在消耗 PID，耗尽会快得多。结论：长命进程必须回收子进程。

Lecture 17: 异常控制流——信号与非本地跳转 (Exceptional Control Flow: Signals and Nonlocal Jumps)

> **讲义对应**：CMU 15–213 Lecture 17 — Exceptional Control Flow（素材：`F25-17-ecf.txt`） **教材对应**：CS:APP3e 第 8 章 8.5—8.8（信号、非本地跳转、进程控制工具） **关联 Lab**：L6 Shell Lab（tshlab）

17.1 概述

上一讲用 `fork` / `execve` / `waitpid` 建立了"进程"抽象，并写出一个能跑前台任务的简单 shell——但它有致命缺陷：shell 设计寿命**无限长**，却对后台作业（background job）束手无策。后台子进程终止后变成**僵尸（zombie）**，永久占用内核进程表项；shell 从不停下来 `wait`，PID 被一个接一个耗尽。

本讲引入**信号（signal）**：一条由内核或进程发出的小消息，通知目标进程系统中发生了某类事件。它让进程无须轮询就能感知**系统状态变化**（子进程终止、定时器到期、用户按下 ctrl-c）。在此之上还引入**非本地跳转（nonlocal jump）**——`setjmp` / `longjmp`，让 C 拥有类似 C++ `throw` / `catch` 的跨栈帧错误传播。

信号最"反直觉"之处在于：处理程序与主程序**并发**运行、共享全局数据，随时可能插进一条赋值语句中间；而内核的态度是"**至多记一位**"，从不排队。这是 L6 Shell Lab 的全部难点，也为 Lecture 21 的并发编程埋伏笔。

17.2 核心概念与底层机制图解

17.2.1 信号是什么（What a Signal Is）

- **定义与目的**：一条**小消息**，通知进程系统中发生了某类事件。它是异常与中断在用户态的对应物：异常由硬件与内核处理，信号则让**应用软件**也能参与异常控制流。
- **直观解释**：像办公室的**火警铃**——铃声不携带信息，你只知道"响了"，不知道谁按的、为什么按；能区分的只有"哪一种铃"（信号类型，整数 ID，Linux 上 1–30）和"它响了"这一事实。信号**没有数据载荷**。
- **底层机制图解**（综合讲义 8.5 的时间轴）：内核并不立即打断目标进程，而是先在位向量里记一笔；下次准备把控制权交还该进程时，才顺便让它"收到"信号并执行处理程序。

- **与机器码/硬件的对应**：发送信号只是"内核修改目标进程上下文中的某个状态位"，没有专门的机器指令；真正把控制流转向处理程序的是内核在返回用户态前对进程栈与 `%rip` 的改写。

17.2.2 发送信号：`kill`、进程组与 `alarm`

- **定义与目的**：内核以两种方式发送信号：**自己检测到系统事件**（除零→`SIGFPE`、子进程终止→`SIGCHLD`、定时器到期→`SIGALRM`），或**某进程调用 `kill` 显式请求**。
- **进程组（process group）**：每个进程**恰好属于一个**进程组（正整数 `pgid`），它是信号广播的单位。shell 借它实现作业控制：一个作业就是一个进程组。

```
                      Shell (pid=10, pgid=10)
                           |  fork + setpgid
     +---------------------+----------------------+
     |                     |                      |
 前台作业              后台作业 #1             后台作业 #2
pgid=20               pgid=32                 pgid=40
  |    |                |                       |
pid=20 pid=21,22      pid=32                  pid=40
  ↑
ctrl-c / ctrl-z 只发给【前台进程组 20】的所有成员
```

- **`kill` 的语义**：`int kill(pid_t pid, int sig)`，成功返回 0，失败返回 −1 并置 `errno`。`pid` 的符号决定目标集合：

`pid` 取值	含义
`pid > 0`	只发给进程 `pid`
`pid == 0`	发给**调用者所在进程组**的每个进程（含自己）
`pid < 0`	发给**进程组 `|pid|`** 中的每个进程
`pid == -1`	发给调用者有权发送信号的**所有进程**

- **`alarm(n)`**：请求内核在 n 秒后发 `SIGALRM`。两个必须记住的细节：① **只触发一次**，不周期重复；② **再次调用会覆盖旧闹钟**，并返回**上一个闹钟的剩余秒数**（实测：`alarm(3)` 后立刻 `alarm(1)` 返回 3）。
- **与机器码/硬件的对应**：内核侧定时由**硬件定时器芯片**周期性中断 CPU（约每几毫秒一次）驱动；`kill` 是系统调用号 62 的 `syscall`，参数走 `%rdi`(pid) 与 `%rsi`(sig)。

17.2.3 接收信号：内核递送的时机与 `pnb`

- **定义与目的**：进程**接收**信号，指它被内核强制做出反应。反应有三种：**忽略**、**终止**（可带 core dump）、**捕获**（执行用户级**信号处理程序 signal handler**）。
- **底层机制图解**：讲义给出的判定规则极简单，却是一切推理的出发点——注意内核把 pnb 中**所有**信号递送完才回到用户态，故多个不同信号会按编号从小到大依次处理。

```
内核准备把控制权交给进程 p 时：
    pnb = pending & ~blocked        /* pending 且未被阻塞的信号集合 */
    if (pnb == 0)
        按逻辑流继续执行 p 的下一条指令
    else
        取 pnb 中【编号最小】的非零位 k，强制 p 接收信号 k
        对 pnb 中所有非零位重复上述过程
        再按逻辑流继续执行 p 的下一条指令
```

- **处理程序是独立的逻辑流**：它与主程序**并发**运行，但只是同一进程内的一条**逻辑流（logical flow）**，不是新进程、也没有独立地址空间。它**可以被打断**——处理 `SIGINT` 时若又来 `SIGTSTP`，内核会再插入一层处理程序（嵌套）。内核还会**自动阻塞正在被处理的那个信号类型**（隐式阻塞），所以 `SIGINT` 的处理程序不会被另一个 `SIGINT` 打断。

```
主程序 Icurr ──► 收到信号 s ──► Handler S 运行 ──► 收到信号 t ──► Handler T 运行
                                  ▲                                  |
                                  └──────── T 返回，S 继续 ◄─────────┘
                                                  S 返回 ──► 主程序 Inext 继续
```

17.2.4 待处理与阻塞：内核中的两个位向量

- **定义与目的**：`pending` = 已发送但尚未接收；`blocked`（又名**信号掩码 signal mask**）= 可以屏蔽其接收。二者是内核在**每个进程上下文**中维护的两个位向量。
- **直观解释**：`pending` 像邮箱里"有件快递到了"的小纸条，`blocked` 像门上挂的"免打扰"牌子。挂了牌子快递员照样塞纸条（pending 置位），只是你暂不去取；牌子一摘立刻去取——**而不管纸条被塞了几张，你只看到一条消息**。

- **底层机制图解**：

```
       进程 p 的内核上下文（每个进程各一份）
       pending 位向量（已发送、未接收）
 15 14 13 12 11 10  9  8  7  6  5  4  3  2  1  0
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| 0| 0| 0| 0| 0| 0| 0| 0| 0| 0| 0| 0| 1| 0| 0|   ← SIGALRM(14) 待处理
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
       blocked 位向量（信号掩码，可由 sigprocmask 修改）
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| 0| 0| 0| 0| 0| 0| 0| 0| 0| 0| 0| 0| 1| 0| 1|   ← 阻塞了 SIGALRM(14) 与 SIGINT(2)
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
       pnb = pending & ~blocked = 0  →  本次不递送，先执行用户指令
        （SIGINT 此时若到达，只会把 pending 的第 2 位置 1，仍需等待解除阻塞）
```

- **最重要的事实：信号不排队（signals are not queued）**。每种信号在 `pending` 中**只占一位**，故同一时刻**至多一个**该类型的待处理信号；在它被接收前，后续同类型信号**被直接丢弃**。推论：**不能用信号计数事件**。本地实测：阻塞 `SIGUSR1` 后连续 `kill` 自己 100 次，`sigpending` 查到的集合里 `SIGUSR1` 仍只是一个位。
- **不可阻塞的信号**：`SIGKILL`(9) 与 `SIGSTOP`(19) 不可捕获、不可忽略、不可阻塞——内核为管理员保留的"最后手段"。实测：对二者调用 `sigaction` 一律失败并返回 `EINVAL`(22)。

17.2.5 安装处理程序：`signal` 与 `sigaction`

- **定义与目的**：安装（install）处理程序即修改某个信号号的默认动作。`signal(int, handler_t *)` 是历史接口；`sigaction(int signum, const struct sigaction *act, struct sigaction *oldact)` 是现代接口，可精确控制标志，`oldact` 非空时写回**原先**的处理方式。三个特殊常量：`SIG_IGN`（忽略）、`SIG_DFL`（恢复默认）、用户函数地址（捕获）。
- **`sa_flags` 常用位**：

标志	含义
`SA_RESTART`	自动重启被该信号打断的**慢速系统调用**（如 `read`、`wait`）
`SA_NODEFER`	处理程序运行期间**不**自动阻塞自身（默认阻塞，见 17.2.3）
`SA_RESETHAND`	进入处理程序时把动作**重置为 `SIG_DFL`**（只生效一次）
`SA_SIGINFO`	改用三参数 `sa_sigaction`，可取得 `siginfo_t` 与上下文
`SA_ONSTACK`	在备用信号栈（`sigaltstack`）上运行处理程序

实测 `SA_RESTART` 数值为 `0x10000000`。**`signal` 不可移植**：不同系统语义不同——有的在一次触发后把处理程序重置为默认（System V 风格），有的自动重启系统调用；可移植代码**一律使用 `sigaction`**。三个可移植性差异点——① 触发后是否重置处理程序；② 运行期间是否自动阻塞自身；③ 被中断的系统调用是否自动重启——分别由 `SA_RESETHAND`、`SA_NODEFER`、`SA_RESTART` 显式决定。

17.2.6 阻塞信号与 `sigprocmask`

- **接口**：`int sigprocmask(int how, const sigset_t *set, sigset_t *oldset)`：

`how`	含义
`SIG_BLOCK`	把 `set` 中的信号**加入**当前掩码（屏蔽）
`SIG_UNBLOCK`	把 `set` 中的信号从掩码中**移除**
`SIG_SETMASK`	把当前掩码**替换**为 `set`

- **集合操作**：`sigemptyset`、`sigfillset`、`sigaddset`、`sigdelset`、`sigismember`；另有 `sigpending(&set)` 查询待处理集合。
- **推荐的配对写法**：不要用 `SIG_UNBLOCK` 解除，而用 `SIG_SETMASK` + 保存的 `prev` 恢复——`SIG_UNBLOCK` 会**无条件解除**，即使该信号进入本段代码前本就阻塞，从而破坏嵌套的正确性。讲义与 recitation 均强调这一点：

```
sigset_t mask, prev_mask;
sigemptyset(&mask);
sigaddset(&mask, SIGINT);
/* 屏蔽 SIGINT，同时把"原掩码"保存到 prev_mask */
sigprocmask(SIG_BLOCK, &mask, &prev_mask);
    /* 这段临界区不会被 SIGINT 打断 */
/* 恢复原掩码 —— 等价于解除 SIGINT，且对嵌套安全 */
sigprocmask(SIG_SETMASK, &prev_mask, NULL);
```

17.2.7 非本地跳转（Nonlocal Jumps）

- **定义与目的**：C 的控制流原语只能在同一栈帧内跳转（`goto`）或按栈纪律返回（`return`）。`setjmp(env)` 把**当前寄存器上下文与栈指针**存入 `jmp_buf`，`longjmp(env, retval)` 恢复该上下文——效果是**直接掀掉中间所有栈帧**跳回 `setjmp` 处。直观上像夹一张书签后决定"直接翻回那一页、跳过中间所有内容"。
- **返回值语义**：`setjmp` **直接**返回 0；从 `longjmp` 返回时"返回"的是其第二个参数 `retval`，故 `retval` **必须非 0**。`setjmp` 只能出现在 `if`、`switch`、循环条件或比较表达式中，**不能**写成 `x = setjmp(env)`。
- **底层机制图解**（`%rsp` 一次性弹回 `main` 栈顶，中间 4 层栈帧被凭空抹去）：

```
高地址
  +--------------------------+  <-- main 的栈帧（setjmp 在此保存上下文）
  |   main(argc, argv)       |
  |   jmp_buf env ------------|--> 保存 %rsp, %rbp, %rbx, %r12-%r15, %rip
  +--------------------------+  |
  |   level1 的栈帧           |  |
  +--------------------------+  |
  |   level2 的栈帧           |  |  longjmp(env, 0x213) 让 %rsp 一次性
  +--------------------------+  |  弹回 main 的栈顶，并令 setjmp 返回 0x213
  |   level3 的栈帧           |  |  ★ 中间 4 个栈帧被"凭空抹去"，
  +--------------------------+  |     它们占用的资源不会被释放
  |   level4 的栈帧 ← 出错点 |  |
  +--------------------------+  |
  |        ...               |  |
  +--------------------------+  |
低地址                          +--> 跳回此处（setjmp 返回非 0）
```

- **两个必须记住的陷阱**：
- **资源泄漏**：`longjmp` **不释放**中间栈帧——像撕掉书中页码，夹在里面的借书卡却还在。中间层 `malloc` 的内存、打开的 fd 都不被清理。
- **变量值不确定（UB）**：在 `setjmp` 与 `longjmp` 之间被修改的**局部变量**，若被分配到**寄存器**（而 `longjmp` 恢复了该寄存器的旧值），之后读到的值就"回到了过去"。标准只保证 `volatile` 变量一定正确。
- **信号版本**：`sigsetjmp(env, savesigs)` / `siglongjmp(env, retval)`；`savesigs` 非 0 时同时保存**信号掩码**并在跳回时恢复，从处理程序里跳出应当用这一对。C++ 的 `throw`/`catch` 与 Rust 的 `panic` 语义相同，但其实现（栈展开）会**逐个调用中间栈帧的析构函数**，故不泄漏——这是与 `longjmp` 最本质的区别。

17.3 代码示例与底层机制分析

17.3.1 示例 (a)：用 `sigaction` 捕获 `SIGINT` / `SIGQUIT` 并优雅退出

代码（C）：完整源码见 `/tmp/csapp17/ecf_a_sigaction.c`。核心部分：

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <signal.h>
#include <unistd.h>

/* G4/G5：全局标志用 volatile sig_atomic_t */
static volatile sig_atomic_t sigint_count = 0;
static volatile sig_atomic_t sigquit_seen = 0;

/* G1：处理程序里只用异步信号安全函数 —— write 是唯一安全的输出函数 */
static void safe_puts(const char *s)
{
    size_t n = strlen(s);                /* strlen 在 POSIX 安全清单中 */
    ssize_t r = write(STDOUT_FILENO, s, n);
    (void)r;
}

static void sigint_handler(int signum)
{
    int saved_errno = errno;             /* G2：进场先存 errno */
    sigint_count++;                      /* G5：只做赋值 */
    safe_puts("[handler] caught SIGINT (ctrl-c), ignoring it\n");
    sleep(1);                            /* sleep 也是异步信号安全的 */
    errno = saved_errno;                 /* G2：退场恢复 errno */
    (void)signum;
}

static void sigquit_handler(int signum)
{
    int saved_errno = errno;
    sigquit_seen = 1;                    /* 只置位，收尾交给主程序 */
    safe_puts("[handler] caught SIGQUIT (ctrl-\\), asking main to quit\n");
    errno = saved_errno;
    (void)signum;
}

static void install(int signum, void (*handler)(int), int flags)
{
    struct sigaction act;
    memset(&act, 0, sizeof(act));
    act.sa_handler = handler;
    sigemptyset(&act.sa_mask);           /* 运行期间不额外屏蔽别的信号 */
    act.sa_flags = flags;
    if (sigaction(signum, &act, NULL) < 0) { perror("sigaction"); exit(EXIT_FAILURE); }
}

int main(void)
{
    install(SIGINT,  sigint_handler,  SA_RESTART);
    install(SIGQUIT, sigquit_handler, 0);
    for (int i = 0; i < 3; i++) {
        kill(getpid(), SIGINT);                    /* 立刻被递送并处理 */
        printf("[main] back from SIGINT, sigint_count = %d\n", (int)sigint_count);
    }
    kill(getpid(), SIGQUIT);
    while (!sigquit_seen) pause();
    printf("[main] graceful exit. SIGINT handled %d times, SIGQUIT seen %d\n",
           (int)sigint_count, (int)sigquit_seen);
    return 0;
}
```

【代码做什么？】 ① `install` 用 `sigaction` 给 `SIGINT` 装"忽略但计数"、给 `SIGQUIT` 装"置位请求退出"的处理程序；② 主程序三次 `kill(getpid(), SIGINT)`，每次 `kill` 返回时处理程序**已执行完毕**；③ 发 `SIGQUIT` 后进入 `pause()` 挂起等待标志置位；④ 主程序自己收尾并打印统计。

【底层机制透视】 关键是"`kill` 返回的时机"。`kill` 是系统调用，返回路径必经内核的"递送信号"检查点，于是 `pending & ~blocked` 立刻非零，内核当场把 `%rip` 指向 `sigint_handler` 并为它构造临时栈帧；处理程序 `return` 后恢复主程序上下文。因为处理程序运行期间内核**隐式阻塞** `SIGINT`，处理程序内部再来的 `SIGINT` 会留在 pending 里等返回后再递送——这正是 `sigint_count` 能精确加到 3 的原因。

实测输出（`gcc -g -Wall -std=c11 ecf_a_sigaction.c -o ecf_a_sigaction`，用 `stdbuf -oL` 保证行缓冲）：

```
[main] pid = 3570419
[main] installed handler for signal 2 (flags=0x10000000)
[main] installed handler for signal 3 (flags=0x0)
[main] SIGINT=2 SIGQUIT=3 SIGCHLD=17 SIGKILL=9 (不可捕获)
[main] sending SIGINT to self (round 1)
[handler] caught SIGINT (ctrl-c), ignoring it
[main] back from SIGINT, sigint_count = 1
[main] sending SIGINT to self (round 2)
[handler] caught SIGINT (ctrl-c), ignoring it
[main] back from SIGINT, sigint_count = 2
[main] sending SIGINT to self (round 3)
[handler] caught SIGINT (ctrl-c), ignoring it
[main] back from SIGINT, sigint_count = 3
[main] sending SIGQUIT to self
[handler] caught SIGQUIT (ctrl-\), asking main to quit
[main] back from SIGQUIT
[main] graceful exit. SIGINT handled 3 times, SIGQUIT seen 1
```

`flags=0x10000000` 即 `SA_RESTART` 的真实数值；`SIGQUIT` 的 `flags=0x0` 不重启系统调用。

17.3.2 示例 (b)：`SIGCHLD` 的两个经典竞态——正确版与错误版对比

这是 **Shell Lab 的核心考点**。完整源码见 `/tmp/csapp17/ecf_b_sigchld.c`，用 `./ecf_b_sigchld <race|racefix|once|loop>` 切换四种模式。

竞态一：`fork` 与 `addjob` 之间的窗口。 讲义 `procmask1.c` 写的是"先 `fork`，再 `addjob`"，隐含假设**父进程先跑**。但 `fork` 后父子进程并发：若子进程在 `addjob` 前终止，`SIGCHLD` 处理程序会先运行，去 `deletejob(pid)` 而那个 job 还不在列表里——删除失败，父进程随后又把它加进去，列表中就留下一个**永远无法回收的幽灵 job（ghost job）**。正确版本必须在 `fork` **之前**屏蔽 `SIGCHLD`：

```
/* ---- 正确版（等价于讲义 procmask2.c）：阻塞 / 解除阻塞的配对 ---- */
int main(int argc, char **argv)
{
    int pid;
    sigset_t mask_all, mask_one, prev_one;
    int n = 5;
    sigfillset(&mask_all);
    sigemptyset(&mask_one);
    sigaddset(&mask_one, SIGCHLD);          /* 只管 SIGCHLD 一个 */
    signal(SIGCHLD, handler);
    initjobs();

    while (n--) {
        /* ① 屏蔽 SIGCHLD —— 必须在 fork 之前，否则窗口仍存在 */
        sigprocmask(SIG_BLOCK, &mask_one, &prev_one);
        if ((pid = fork()) == 0) {          /* 子进程 */
            /* 子进程继承父进程的屏蔽字，execve 之前必须解除 */
            sigprocmask(SIG_SETMASK, &prev_one, NULL);
            setpgid(0, 0);                  /* 自建进程组，隔离 ctrl-c */
            execve("/bin/date", argv, NULL);
            _exit(127);
        }
        /* ② 父进程：屏蔽所有信号，保护 job 列表这一共享数据结构（G3） */
        sigprocmask(SIG_BLOCK, &mask_all, NULL);
        addjob(pid);                        /* 此刻子进程即使已死，信号也进不来 */
        /* ③ 解除屏蔽 —— 用 SIG_SETMASK 恢复 prev_one，而不是 SIG_UNBLOCK */
        sigprocmask(SIG_SETMASK, &prev_one, NULL);
    }
    exit(0);
}
```

★ 三点必须同时做到：**(i) 屏蔽发生在 `fork` 之前**；**(ii) `addjob` 时屏蔽所有信号**以保护 job 列表；**(iii) 解除用 `SIG_SETMASK` + `prev_one` 恢复**而非 `SIG_UNBLOCK`，这样可安全嵌套。子进程侧那次 `SIG_SETMASK` 也不可省——子进程**继承**父进程的信号掩码，不解除则 `execve` 出来的程序会带着 `SIGCHLD` 被永久屏蔽的状态运行。

竞态二：信号合并导致的漏收。 即使修好上面的问题，若处理程序只调用**一次** `waitpid` 仍会出错：内核对待 `SIGCHLD` 只有一位，多个子进程短时间陆续终止时多次 `SIGCHLD` 会**合并成一次递送**，处理程序只回收一个僵尸，其余永远留在进程表里。修复办法是**用 `while` 循环 + `WNOHANG` 收干净**：

```
/* ---- 正确版处理程序：一次递送，回收全部已终止子进程 ---- */
void sigchld_handler(int sig)
{
    int olderrno = errno;                    /* G2 */
    sigset_t mask_all, prev_all;
    pid_t pid;
    sigfillset(&mask_all);
    /* WNOHANG：还有活着的子进程时立即返回 0，绝不阻塞等待 */
    while ((pid = waitpid(-1, NULL, WNOHANG)) > 0) {
        sigprocmask(SIG_BLOCK, &mask_all, &prev_all);   /* G3：保护 job 列表 */
        deletejob(pid);
        sigprocmask(SIG_SETMASK, &prev_all, NULL);
    }
    if (pid != 0 && errno != ECHILD)         /* 0=还有活子进程；<0 且 ECHILD=无子进程 */
        sio_error("waitpid error");
    errno = olderrno;                        /* G2 */
}
```

实测输出（四种模式各跑一次，真实结果）：

```
$ ./ecf_b_sigchld race
=== mode = race (N = 5 children) ===
  [handler] deletejob: pid 3579688 NOT in job list -- 幽灵 job 即将产生!
  [handler] deletejob: pid 3579696 NOT in job list -- 幽灵 job 即将产生!
   (同样的告警共出现 5 次，此处略去 3 行)
handler_calls    = 5
reaped_in_handler= 5
reaped_in_main   = 0
最终 job 列表: njobs_inuse = 5  [ (jid=1,pid=3579688) (jid=2,pid=3579696) ... ]
结论: FAIL —— 存在幽灵 job 或残留僵尸

$ ./ecf_b_sigchld racefix
handler_calls    = 5
reaped_in_handler= 5
reaped_in_main   = 0
最终 job 列表: njobs_inuse = 0  [ ]
结论: PASS —— job 列表干净、僵尸收干净

$ ./ecf_b_sigchld once
[main] 5 个子进程已全部死亡，此刻 pending 位向量里 SIGCHLD 只有 1 位
handler_calls    = 1  (SIGCHLD 处理程序实际被调用次数)
reaped_in_handler= 1
reaped_in_main   = 4  (处理程序漏收、主程序补收的僵尸数)
最终 job 列表: njobs_inuse = 4  [ (jid=2,pid=3579812) ... ]
结论: FAIL —— 存在幽灵 job 或残留僵尸

$ ./ecf_b_sigchld loop
[main] 5 个子进程已全部死亡，此刻 pending 位向量里 SIGCHLD 只有 1 位
handler_calls    = 1
reaped_in_handler= 5
reaped_in_main   = 0
最终 job 列表: njobs_inuse = 0  [ ]
结论: PASS —— job 列表干净、僵尸收干净
```

`once` 与 `loop` 的对比把"信号不排队"体现得淋漓尽致：**5 个子进程全部死亡，处理程序只被调用 1 次**，却因 `while` 循环回收了全部 5 个。这正是 Shell Lab 必须采用 `while (waitpid(-1, &status, WNOHANG | WUNTRACED) > 0)` 的原因——`WUNTRACED` 让它同时捕获"被停止"的子进程。

【数据结构图解】 幽灵 job 的产生过程：

```
  时刻 t0  父进程：fork() 返回 pid=X            子进程 X 开始运行
  时刻 t1  父进程：被调度走的瞬间（还没 addjob）  子进程 X：_exit(0) → 变僵尸，内核发 SIGCHLD
  时刻 t2  内核把 SIGCHLD 递给父进程 → 处理程序运行
           handler：waitpid 收走 X；deletejob(X) → 查无此 job，删除失败
  时刻 t3  父进程回到用户态，执行 addjob(X)     ★ 把一个已死进程登记进 job 列表
  结果：jobs 表里留下 (jid=1, pid=X)，而 X 早已不存在 → jobs 命令永远显示它
```

17.3.3 示例 (c)：`setjmp` / `longjmp` 的深层跳转与 `volatile` 陷阱

完整源码见 `/tmp/csapp17/ecf_c_setjmp.c`，用 `./ecf_c_O2 <deep|volatile|sigjmp>` 切换模式。

深层跳转：`main` 用 `setjmp` 设置落点，依次调用 `level1 → level2 → level3 → level4`，在 `level4` 里 `longjmp(deep_env, 0x213)`。

```
static jmp_buf deep_env;

static void level4(int errcode)
{
    printf("    level4: 发现错误 errcode=%d，调用 longjmp 直接跳回 main 的 setjmp 点\n", errcode);
    longjmp(deep_env, errcode);       /* 第二个参数必须非 0！ */
    printf("    level4: 这一行永远不会执行\n");
}
static void level3(int e) { level4(e); }
static void level2(int e) { level3(e); }
static void level1(int e) { level2(e); }

int mode_deep(void)
{
    int code = setjmp(deep_env);      /* 只能出现在 if / switch / 比较中 */
    if (code == 0) {
        level1(0x213);                /* 直接调用时 setjmp 返回 0 */
    } else {
        printf("[deep] 从 longjmp 返回，setjmp 这次返回 %d (0x%x)\n", code, code);
    }
    return 0;
}
```

实测输出（`-O0` 与 `-O2` 完全一致）：

```
[deep] 调用点地址附近的 setjmp 返回了 0（直接调用）
[deep] 开始向下钻：level1 -> level2 -> level3 -> level4
    level1: 进入
    level2: 进入
    level3: 进入
    level4: 发现错误 errcode=531，调用 longjmp 直接跳回 main 的 setjmp 点
[deep] 从 longjmp 返回，setjmp 这次返回 531 (0x213)
[deep] 中间 4 层栈帧已被一次性丢弃
```

【底层机制透视】 `setjmp` 把 `%rsp`、`%rbp`、`%rbx`、`%r12`—`%r15` 与返回地址写进 `jmp_buf`；`longjmp` 把这些寄存器原样写回，于是 `%rsp` 一下子弹回 `main` 的栈顶，`%rip` 指向 `setjmp` 之后。中间 4 层栈帧里 `printf` 之后的语句被彻底跳过——**没有执行任何清理代码**。

`volatile` 陷阱演示（⚠ 故意踩 UB，请勿模仿）：

```
/* ⚠ 仅供演示：这正是 CS:APP3e 8.5 节指出的未定义行为 */
int mode_volatile(void)
{
    int          i = 1;    /* 非 volatile：可能被放进寄存器 */
    volatile int j = 1;    /* volatile：强制每次访问都去内存 */
    register int k = 1;

    if (setjmp(vol_env) == 0) {
        i = 2; j = 2; k = 2;      /* setjmp 之后修改 */
        vol_inner();              /* 里面 longjmp(vol_env, 7) */
    }
    printf("i(非volatile)=%d  j(volatile)=%d  k(register)=%d\n", i, j, k);
    return 0;
}
```

实测输出（同一份源码，只改优化级别）：

```
$ ./ecf_c_O0 volatile
[volatile] longjmp 之后: i(非volatile)=2  j(volatile)=2  k(register)=2

$ ./ecf_c_O2 volatile
[volatile] longjmp 之后: i(非volatile)=1  j(volatile)=2  k(register)=1
```

这行输出最值得凝视：`-O0` 下 `i` 与 `k` 都是 2，一切"看起来正确"；`-O2` 下它们**倒退回了 `setjmp` 之前的值 1**，只有 `volatile` 的 `j` 始终是 2。对应的真实汇编（`gcc -S -O2`）给出了常量折叠的直接证据：

```
mode_volatile:
        subq    $24, %rsp
        movl    $vol_env, %edi
        movl    $1, 12(%rsp)        # j = 1，并且【只】存在内存里
        call    _setjmp
        testl   %eax, %eax
        je      .L6
        movl    12(%rsp), %edx      # 从内存读回 j —— 一定是 2
        movl    $1, %ecx            # k 直接折叠成常量 1
        movl    $1, %esi            # i 直接折叠成常量 1
        ...
.L6:
        movl    $7, %esi
        movl    $vol_env, %edi
        movl    $2, 12(%rsp)        # 只写回 j；i 与 k 根本没进内存
        call    longjmp
```

【与汇编 / 硬件的对应】 `i` 与 `k` 之所以"倒退"，是因为编译器认定其值恒为 1（`setjmp` 返回 0 的路径上 `i=2` 之后必然 `longjmp` 离开），于是把两者**折叠成立即数**，不分配内存位置。`volatile` 的 `j` 被强制放到 `12(%rsp)`，而 `longjmp` 恰好恢复了 `%rsp`，故它读到的仍是 2。**结论：跨越 `setjmp` / `longjmp` 的局部变量必须声明为 `volatile`。**

`sigsetjmp` / `siglongjmp`：第三个模式从 `SIGALRM` 处理程序里跳出。实测：

```
[sigjmp] 安装好 SIGALRM 处理程序，alarm(1) 后进入 pause()
[sigjmp] 从信号处理程序 siglongjmp 回来，jumped=1
[sigjmp] sigsetjmp 的第二参数为 1，故信号屏蔽字已被恢复
```

17.3.4 信号速查表

ID	名称	默认动作	对应事件	可否捕获/忽略/阻塞
1	`SIGHUP`	Terminate	终端挂断	可
2	`SIGINT`	Terminate	用户按 ctrl–c	可
3	`SIGQUIT`	Terminate + core	用户按 ctrl–\	可
4	`SIGILL`	Terminate + core	非法指令	可
5	`SIGTRAP`	Terminate + core	断点/陷阱	可
6	`SIGABRT`	Terminate + core	`abort()`	可
7	`SIGBUS`	Terminate + core	总线错误	可
8	`SIGFPE`	Terminate + core	除零/算术错误	可
9	`SIGKILL`	Terminate	强杀	**不可**（`EINVAL`）
10	`SIGUSR1`	Terminate	用户自定义	可
11	`SIGSEGV`	Terminate + core	段错误	可
12	`SIGUSR2`	Terminate	用户自定义	可
13	`SIGPIPE`	Terminate	写已关闭的管道	可
14	`SIGALRM`	Terminate	`alarm()` 定时器到期	可
15	`SIGTERM`	Terminate	温和终止请求	可
16	`SIGSTKFLT`	Terminate	协处理器栈错误	可
17	`SIGCHLD`	Ignore	子进程停止或终止	可
18	`SIGCONT`	Continue	恢复被停止的进程	可
19	`SIGSTOP`	Stop	强制停止	**不可**（`EINVAL`）
20	`SIGTSTP`	Stop	用户按 ctrl–z	可
21	`SIGTTIN`	Stop	后台进程读终端	可
22	`SIGTTOU`	Stop	后台进程写终端	可
23	`SIGURG`	Ignore	socket 紧急数据	可

ID	名称	默认动作	对应事件	可否捕获/忽略/阻塞
24	`SIGXCPU`	Terminate + core	CPU 时间超限	可
25	`SIGXFSZ`	Terminate + core	文件大小超限	可
26	`SIGVTALRM`	Terminate	虚拟定时器	可
27	`SIGPROF`	Terminate	性能剖析定时器	可
28	`SIGWINCH`	Ignore	终端窗口大小改变	可
29	`SIGIO`	Terminate	异步 I/O 就绪	可
30	`SIGPWR`	Terminate	电源故障	可

实测确认：`sigaction(SIGKILL)` 与 `sigaction(SIGSTOP)` 均返回 −1 且 `errno = 22 (EINVAL)`；`SIGINT=2, SIGQUIT=3, SIGALRM=14, SIGCHLD=17, SIGCONT=18, SIGTSTP=20` 与表格一致。

17.3.5 异步信号安全函数清单

定义：函数**异步信号安全（async-signal-safe）**，当且仅当它要么**可重入**（所有变量都在栈帧上，CS:APP3e 12.7.2），要么**不可被信号打断**。POSIX 保证 **117 个**函数满足该性质（`man 7 signal-safety`）。

✅ 安全（可在处理程序中调用）	❌ 不安全（严禁在处理程序中调用）
`write`（**唯一安全的输出函数**）	`printf` / `fprintf`（stdio 缓冲区非可重入）
`_exit` / `_Exit`	`exit`（会跑 `atexit` 与 stdio 清理）
`read`、`open`、`close`、`dup2`、`lseek`	`malloc` / `free`（堆状态可能被撕裂）
`wait`、`waitpid`、`sleep`、`alarm`、`pause`	`sprintf` / `snprintf`（依赖 locale 与缓冲）
`kill`、`raise`、`sigqueue`	`strerror`、`getpwnam`（静态缓冲区）
`sigprocmask`、`sigpending`、`sigsuspend`	`readdir`、`localtime`、`asctime`（静态缓冲区）
`sigaction`、`signal` 与 `sigemptyset/addset/fillset/delset/ismember`	`setenv` / `putenv`（修改全局环境）
`fork`、`execve`、`setpgid`、`getpid`、`getpgrp`	`rand` / `srand`（全局种子状态）
`strlen`、`strcmp`、`strcpy`、`memcpy`、`memset`、`memmove`	`stdout` / `stderr` 的任何 `f*` 系列
`longjmp`、`siglongjmp`（见注释）	`pthread_*` 中未列入清单者、`dlopen`

注意：`longjmp` / `siglongjmp` 自 POSIX.1-2008 TC2 起列入安全清单，但有重要限定——若处理程序中断的正是某个**非**异步信号安全函数，从处理程序 `longjmp` 出去行为未定义；`setjmp` 本身**不在**清单里。**实用替代**：CS:APP 的 `csapp.c` 提供可重入安全 I/O 库（SIO）：`sio_puts`、`sio_putl`、`sio_error`，以及支持受限格式串（`%c %s %d %u %x %%`）的 `sio_printf`；它们内部只用 `write`。

17.3.6 显式等待信号：`pause`、`sigsuspend`

讲义用四种写法对比"如何等一个信号"，这段对照是本章精华：

写法	是否浪费 CPU	是否有竞态	响应延迟
`while (!pid) ;`	★ 忙等待，疯狂占用 CPU	无	立即
`while (!pid) sleep(1);`	不忙等	无	最坏 **1 秒**（太慢）
`while (!pid) pause();`	不忙等（挂起）	**有竞态！**	立即
`while (!pid) sigsuspend(&prev);`	不忙等	**无**	立即

`pause` 的竞态在于：检查 `!pid` 为真之后、调用 `pause()` 之前信号可能恰好到达——处理程序已把 `pid` 置为非零，但 `pause()` 仍会挂起，程序**永远醒不过来**。`int sigsuspend(const sigset_t *mask)` 是下述序列的**原子（不可中断）**版本："挂起之前先解除目标信号的阻塞，且两步之间不存在可被信号插入的缝隙"：

```
sigprocmask(SIG_SETMASK, &mask, &prev);   /* 换掩码 */
pause();                                  /* 挂起 */
sigprocmask(SIG_SETMASK, &prev, NULL);    /* 恢复掩码 */
```

正确用法：**主流程先阻塞 `SIGCHLD`，等待循环里用 `sigsuspend(&prev)` 临时解除它**——信号只可能在 `sigsuspend` 内部到达，绝不会丢失。

```
主流程:  sigprocmask(SIG_BLOCK, &mask, &prev)   ← SIGCHLD 进不了
         fork()
         pid = 0
         while (!pid)
             sigsuspend(&prev)                    ← 原子地:解除阻塞 + 挂起
         (信号在此之内到达 → 处理程序置 pid → 返回 → sigsuspend 返回 → 循环退出)
         sigprocmask(SIG_SETMASK, &prev, NULL)   ← 恢复
```

17.4 实验关联

本讲与 **L6 Shell Lab（tshlab）** 一一对应：要写出 `tsh` 的 7 个函数 `eval`、`builtin_cmd`、`do_bgfg`、`waitfg`、`sigchld_handler`、`sigint_handler`、`sigtstp_handler`。评分按 **16 个 trace** 各 5 分计 80 分正确性 + 10 分风格分（注释 5 分 + 每个系统调用都查返回值 5 分）。

必须做对的四件事：

1. **`sigchld_handler` 的正确写法**：用 `while` 循环 + `WNOHANG | WUNTRACED` 收干净，循环体内用 `sigprocmask` / `SIG_SETMASK` 保护 job 列表，进出各存/恢复一次 `errno`（见 17.3.2）。**一个 trace 里可能只有一次 `SIGCHLD` 递送却对应多个退出/停止的子进程**——这正是 trace16 与 `mysplit` 系列要考的。
2. **`SIGINT` / `SIGTSTP` 处理程序向前台*进程组*转发**：必须写 `kill(-fg_pgid, SIGINT)`，**用负号**；后台作业在 `fork` 后 `execve` 前必须 `setpgid(0, 0)`，否则 ctrl-c 会同时打到 shell 自己（`sdriver.pl` 专门检测这一错误）。无前台作业时信号应**无效果**。
3. **`waitfg` 用 `sigsuspend` 等待**：当下 recitation 明确把"紧密循环"列为 **禁止** 做法。历史版 writeup 曾建议"`waitfg` 忙循环 `sleep`、`sigchld_handler` 里只调一次 `waitpid`"，那与竞态二直接冲突——**以 `sigsuspend` + `while` 循环为准**。
4. **`jobs` 输出顺序**：参考输出 `tshref.out` 中按 JID 升序列出，格式 `[jid] (pid) Running ./cmd &` / `[jid] (pid) Stopped ./cmd &`；`fg` 转前台先打印 `[jid] (pid) cmd`；终止打印 `Job [jid] (pid) terminated by signal 2`，停止打印 `Job [jid] (pid) stopped by signal 20`。**字符串必须逐字符与 `tshref` 一致**。

从 `trace01.txt` 起逐个推进，每步用 `make test01` 与 `make rtest01` 对比。不要用 `more` / `less` / `vi` / `emacs` 测试（会改终端设置）；用 `/bin/ls`、`/bin/ps`、`/bin/echo` 与 `./myspin`、`./mysplit`、`./mystop`、`./myint`。

17.5 常见错误与调试技巧

- **`SIGCHLD` 被"忽略"或一直阻塞**：`waitpid` 收不到通知，僵尸堆积、`jobs` 列出早已消失的作业。**调试**：`ps -o pid,ppid,stat,comm -l` 看 `Z` 状态与 `<defunct>`；`strace -f -e trace=wait4,kill,rt_sigprocmask,rt_sigaction ./tsh`。
- **`fork` 与 `addjob` 之间的竞态（幽灵 job）**：`jobs` 里有 PID 早已死亡的条目；复现见 17.3.2 的 `race` 模式。**调试**：在 `deletejob` 失败分支用 `write` 打印 PID，观察"NOT in job list"。
- **处理程序里调用 `printf`**：偶发的输出错乱、死锁或崩溃——stdio 缓冲区被重入。**调试**：`gdb` 在 `sigchld_handler` 打断点，`bt` 看是否落在 `_IO_file_xsputn` 内部；改用 `sio_puts` / `write`。
- **忘记 `volatile sig_atomic_t`**：`-O2` 下 `while (!pid) ;` 变成死循环（`pid` 被缓存在寄存器）。**调试**：`objdump -d -M intel ./tsh | grep -A20 '<waitfg>'` 看循环体是否每次从内存重载。
- **`longjmp` 后的局部变量"倒退"**：错误码丢失、循环计数复位。**调试**：`-O0` 与 `-O2` 各编一份对比（本例即如此暴露），用 `volatile` 修正。
- **`alarm` 覆盖或只响一次**：周期性定时只触发一次，或返回值被丢掉导致计时混乱。**调试**：`strace -e trace=alarm,rt_sigaction,rt_sigreturn ./prog`。
- **ctrl-c 打到了 shell 自己**：子进程没 `setpgid(0, 0)`，与 shell 同组。**调试**：`ps -o pid,pgid,tpgid,comm` 看 STAT 的 `+`（前台进程组）标记。
- **`SIG_UNBLOCK` 破坏嵌套**：解除后外层原本的屏蔽状态被永久丢失。**调试**：打印 `sigprocmask` 的 `oldset` 比对；统一改用 `SIG_SETMASK` + 保存的 `prev`。

17.6 关键要点

1. **信号只是一条无载荷的小消息**，全部信息就是"哪个信号号 + 它到了"；发送只是内核在目标进程的 `pending` 位向量里置一位，递送时机由内核在返回用户态前用 `pnb = pending & ~blocked` 判定。
2. **信号不排队，每种至多一位**：同类型多次到达会被合并且丢弃——**绝不能用信号计数事件**，必须用 `while (waitpid(..., WNOHANG))` 把状态清干净。
3. **`SIGKILL` 与 `SIGSTOP` 是内核的底线**：不可捕获、不可忽略、不可阻塞；其余 28 种信号都可被程序接管。
4. **处理程序是与主程序并发的独立逻辑流**，必须遵守 G0–G5：尽量简单、只调用异步信号安全函数（`write` 是唯一安全的输出）、保存恢复 `errno`、用 `sigprocmask` 保护共享数据、全局变量加 `volatile sig_atomic_t`。
5. **凡是"检查标志再阻塞"的写法都有竞态**；正确姿势是"先阻塞信号 → 做临界区工作 → 用 `sigsuspend` 原子地解除并等待"，Shell Lab 的 `fork` / `addjob` 与 `waitfg` 都遵循这一模式。
6. **`longjmp` 只恢复寄存器，不做任何清理**：中间栈帧的资源会泄漏，跨越 `setjmp` 的局部变量必须 `volatile`；从处理程序跳出要用 `sigsetjmp` / `siglongjmp` 以正确恢复信号掩码。

17.7 思考题（带答案）

Q1（推演题） 执行顺序：① `sigprocmask(SIG_BLOCK, {SIGCHLD}, &prev)`；② `fork` 出 C；③ C 立刻 `_exit(0)`；④ 父进程 `addjob(C)`；⑤ 再 `fork` 出 D，D 也立刻 `_exit(0)`；⑥ `sigprocmask(SIG_SETMASK, &prev, NULL)` 解除阻塞；⑦ 进入 `while (!done) sigsuspend(&prev);`。`SIGCHLD` 处理程序运行几次？共回收几个？若只调用一次 `waitpid`，还剩几个僵尸？

答：C 与 D 的 `SIGCHLD` 在阻塞期间只是把 `pending` 第 17 位置 1，故**处理程序只运行 1 次**（两次事件合并）。用 `while (waitpid(-1, NULL, WNOHANG) > 0)` 这一次运行即可**回收两个，剩余 0 个僵尸**；只调用一次 `waitpid` 则只回收一个，另一个**永远成为僵尸**。第三个陷阱：若第 ⑥ 步在第 ⑦ 步判断**之后**，解除阻塞与 `sigsuspend` 之间就有窗口；`sigsuspend(&prev)` 原子地解除并挂起才能消除它。**另需注意**：若 `done` 靠"处理程序调用次数"累加，合并会使它永远达不到 2 而挂死——必须改用"是否还有未回收的子进程"这类状态量。

Q2（"直观但错误"题） 有同学说："`volatile int counter = 0;` 配上 `void handler(int s) { counter++; }`，`fork` 10 个子进程后 `while (counter < 10);` 等待，程序一定能正常结束。"错在哪？

答：**一是计数假设错了**：`counter` 要到 10 必须处理程序被调用 10 次，而 `SIGCHLD` 不排队——10 个几乎同时退出的子进程可能只引起 1 次递送，主循环死等。**二是 `counter++` 不是原子操作**。正确做法：用 `volatile sig_atomic_t` 并把子进程收干净，主程序用 `sigsuspend` 而非忙等。

Q3（计算/推演题） shell 启动 3 个后台作业，分别在 1s、2s、2.05s 退出；主程序全程未阻塞 `SIGCHLD`，处理程序用 `while (waitpid(-1, NULL, WNOHANG) > 0)`。处理程序被调用几次？每次回收几个？

答：**3 次**，每次回收 1 个。三者间隔远大于"内核从 `_exit` 到父进程被调度回来处理信号"的延迟，`pending` 位每次都被及时消费并重新置位，**没有合并**。对比 `loop` 模式：5 个子进程在 60ms 内密集退出且父进程尚未获得 CPU，`pending` 位一直未消费，5 次事件塌缩成 1 次递送。**处理程序调用次数 = 时间上足够分散的事件数；回收数 = 真实的终止子进程数。**

Q4 `setjmp` / `longjmp` 与 C++ 的 `throw` / `catch` 都能跨栈帧传递控制，为何后者不会泄漏资源？

答：C++ 异常会执行**栈展开（stack unwinding）**，逐个销毁中间栈帧的自动对象；`longjmp` 则只写回 `%rsp` / `%rip` 与若干寄存器，中间栈帧**直接作废**。补救：把待释放资源集中登记在**跳转目标那一层**，返回后统一释放。**`longjmp` 只搬控制流，不搬所有权。**

Lecture 18: 系统级 I/O 与文件系统 (System-Level I/O and File Systems)

> **讲义对应**：CMU 15–213 Lecture 18 — System Level I/O and File Systems（素材：`F25-18-system-io.txt`） **教材对应**：CS:APP3e 第 10 章 系统级 I/O（10.1–10.12） **关联 Lab**：L6 Shell Lab（tsh）/ **L7 Proxy Lab**（缓存 Web 代理）

18.1 概述

本讲回答一个承前启后的核心问题：**当你在 C 里写下 `printf("hi")` 时，内核究竟做了什么？** 讲义从最底层的 Unix I/O（`open` / `read` / `write` / `close` / `stat`）出发，向上解释 C 标准库 `FILE *` 流的缓冲模型，横向讲清"什么时候该用哪一层"，再揭示内核用**描述符表 → 打开文件表 → v-node 表**三层结构支撑文件共享与重定向。它承接 Lecture 16–17 的进程与信号，是 Lecture 19–20 文件系统与网络编程的前提，也是 Shell Lab 的 `>` **重定向**与 Proxy Lab 的 `rio_readlineb` / `rio_writen` 的技术底座。讲义始终强调：**短计数（short count）不是错误而是常态，你必须处理它。**

18.2 核心概念与底层机制图解

18.2.1 一切皆文件：字节序列模型（A File Is a Sequence of Bytes）

- **定义与目的**：文件就是一个 m 字节的序列 $B_0, B_1, \ldots, B_k, \ldots, B_{m-1}$。内核为**所有**文件提供同一套操作：打开/关闭、读写、查询元数据、改变当前位置（seek）。"当前文件位置 k"位于字节 B_{k-1} 与 B_k 之间。
- **直观解释**：把文件想象成一条**超长磁带**，磁头停在某处；`read` 是"从磁头处复制一段到内存并把磁头右移"，`write` 是"把内存里的一段覆写到磁头处并右移"，`lseek` 则是用手把磁头挪到别处。
- **底层机制图解**：讲义的"酷事实"是——所有 I/O 设备都是文件（`/dev/sda2` 磁盘分区、`/dev/tty2` 终端、`/dev/null` 丢弃写入），连内核数据结构也被暴露成文件（`/proc/$$/status`、`/proc/$$/fd/`）。**统一抽象换来的可组合性**是 Unix 最锋利的设计。
- **与硬件/机器码的对应**：每次 `read` / `write` 都是一条 `syscall` 指令（调用号放 `%eax`）。讲义的 `close` 实现堪称最小内核入口：

```
.globl close
close:
    mov $3, %eax           # 系统调用号 3 = close
    syscall                # 陷入内核（用户态 → 内核态）
    cmp $-4096, %rax       # 返回值落在 [-4095, -1] 即为 -errno
    jae __syscall_error
    ret
```

一次 `read` / `write` 系统调用要花**超过 10,000 个时钟周期**（讲义原话），这正是标准 I/O 缓冲存在的全部理由。

18.2.2 文件类型（File Types）

- **普通文件（regular file）**：存放任意数据。应用层常分**文本文件**与**二进制文件**，但**内核不在乎，全是字节**。文本文件是"文本行的序列"，文本行以行结束符**终止**（不是分隔！）；字符由编码定义（ASCII、UTF-8、EUC-JP…）。行结束符（EOL）：Unix 系是单字节 `0x0A`（LF）；DOS/Windows 是两字节 `0x0D 0x0A`（CR+LF），许多 Internet 协议也如此，C 库自动翻译成 `'\n'`。
- **目录（directory）**：由一组**目录项（entry，也叫 link）**构成的数组，每项把**文件名映射到文件**。每个目录至少含两项：`.` 指向自身、`..` 指向父目录；`mkdir` 建空目录、`ls` 查看、`rmdir` 删空目录。
- **套接字（socket）**：与另一个进程（可能跨机器）通信的文件——Proxy Lab 的核心对象。
- **超出范围的类型**：命名管道（FIFO）、符号链接、字符/块设备。

18.2.3 打开文件与文件描述符（Opening Files and File Descriptors）

- **定义与目的**：`open` 告知内核"我准备访问这个文件"，返回一个小整数 **文件描述符（file descriptor, fd）。每个进程出生时有三个打开的文件：0 = 标准输入 stdin、1 = 标准输出 stdout、2 = 标准错误 stderr**——它们可能是文件、管道、终端，甚至网络连接。
- **直观解释**：fd 像**医院挂号单的号码**：不必知道诊室在哪，报号码即可，系统替你找到对应表项。而"**返回最小可用 fd**"意味着关掉 3 号窗口后下次就会拿到 3 号——这是 `dup2` 与重定向能工作的前提。
- **出错处理**：`fd == -1` 表示出错，用 `errno` + `perror` / `strerror` 报告。

`open` 的 flags 完整表（讲义口径；`flags` 中恰好取一个访问模式，其余用 `|` 组合）：

flags	含义	出现场合
`O_RDONLY`	只读	打开已有文件时**恰选一个**
`O_WRONLY`	只写	打开已有文件时**恰选一个**；创建文件时也可选
`O_RDWR`	读写皆可	打开已有文件时**恰选一个**；创建文件时也可选
`O_CREAT`	文件不存在则创建（**必须**提供第三参数 `mode`）	创建文件时必须有
`O_EXCL`	文件已存在则失败（与 `O_CREAT` 连用实现原子创建）	可选附加位
`O_TRUNC`	存在则先删除原有内容	可选附加位
`O_APPEND`	所有写入都追加到文件末尾	可选附加位
`O_CLOEXEC`	调用 `execve()` 时自动关闭该 fd	可选附加位

```
int fd;
if ((fd = open("/etc/hosts", O_RDONLY)) < 0) {   /* 两参数形式：打开已有文件 */
    perror("open");
    exit(1);
}
```

神秘的第三个参数：`flags` 含 `O_CREAT` 时第三参数必须存在，否则被忽略。`mode` 是新建文件的默认权限，会被 `umask` 修正；讲义建议无特殊理由就用 `sys/stat.h` 的 `DEFFILEMODE`（`0666`）。

关闭文件：**不要关闭同一个 fd 两次**（等价于对同一指针 `free()` 两次）；**`close` 可能"失败"**——文件其实已关，但 OS 借此上报此前某次 `write` 的**延迟错误**，不检查返回值可能**悄悄丢数据**：

```
if (close(fd) < 0) {
    fprintf(stderr, "%s: write error: %s", filename, strerror(errno));
    exit(1);
}
```

18.2.4 读写文件与短计数（Reading/Writing Files, Short Counts）

```
ssize_t read(int fd, void *buf, size_t n);        /* 从当前位置复制到 buf 并更新位置 */
ssize_t write(int fd, const void *buf, size_t n); /* 从 buf 复制到当前位置并更新位置 */
```

返回类型 `ssize_t` 是**有符号**整数：`nbytes < 0` 才是错误；**短计数（`nbytes < n`）完全可能，且不是错误**；`read` 返回 0 **表示 EOF**（`write` 没有 EOF 语义）。

短计数何时发生（讲义列举）：读到 EOF 时、从终端读文本行时、读写网络套接字与管道时。**何时不会发生**：从磁盘文件读（除 EOF 外）、写磁盘文件。还有 `EINTR`：被信号中断时返回 −1 且 `errno == EINTR`，正确做法是**重试**而非报错退出。**最佳实践：永远为短计数留出处理逻辑。**

为什么必须用 rio：若直接写 `while ((n = read(fd, buf, n)) > 0) write(out, buf, n);`，当底层是套接字或管道时 `write` 很可能只写出一部分，剩下字节**静默丢失**——Proxy Lab 的批量大对象测试会因此损坏整个响应体。rio 包就是把"循环直到写满/读满"封装成函数。

rio 函数	缓冲	语义	停止条件	典型用途
`rio_readn(fd, buf, n)`	无缓冲	读**恰好** `n` 字节	只在 EOF 时返回短计数	已知长度的二进制体读取
`rio_writen(fd, buf, n)`	无缓冲	写**恰好** `n` 字节	**永不返回短计数**	写 HTTP 响应、转发请求
`rio_readinitb(&rp, fd)`	—	把 `rio_t` 与 fd 绑定并清空缓冲区	—	每个 fd 一个 `rio_t`
`rio_readlineb(&rp, buf, maxlen)`	有缓冲	读一行文本	满 `maxlen` / EOF / 遇 `'\n'`	解析 HTTP 请求行与头部
`rio_readnb(&rp, buf, n)`	有缓冲	读最多 `n` 字节二进制	满 `n` / EOF	读 `Content-length` 指定长度的响应体

关键约束：**`rio_readlineb` 与 `rio_readnb` 可在同一描述符上任意交错且线程安全**（每个 fd 一个 `rio_t`）；但**不要与无缓冲的 `rio_readn` 混用**（缓冲区里的预读字节会被越过）。而 `rio_readn` 与 `rio_writen` 之间可任意交错。

二进制警告：处理二进制文件（图像、视频）时**绝不能**用文本导向函数（`fgets`、`scanf`、`rio_readlineb`，会解释 EOL 字符），也**绝不能**用字符串函数（`strlen`、`strcpy`，把字节 0 当特殊字符）。请改用 `rio_readn` / `rio_readnb`。

18.2.5 文件元数据与 `struct stat`（File Metadata）

元数据是"关于数据的数据"，由内核按文件维护，通过 `stat` / `fstat` / `lstat` 访问。讲义给出的结构体：

```
/* Metadata returned by the stat and fstat functions */
struct stat {
    dev_t         st_dev;      /* Device */
    ino_t         st_ino;      /* inode */
    mode_t        st_mode;     /* Protection and file type */
    nlink_t       st_nlink;    /* Number of hard links */
    uid_t         st_uid;      /* User ID of owner */
    gid_t         st_gid;      /* Group ID of owner */
    dev_t         st_rdev;     /* Device type (if inode device) */
    off_t         st_size;     /* Total size, in bytes */
    unsigned long st_blksize;  /* Blocksize for filesystem I/O */
    unsigned long st_blocks;   /* Number of blocks allocated */
    time_t        st_atime;    /* Time of last access */
    time_t        st_mtime;    /* Time of last modification */
    time_t        st_ctime;    /* Time of last change */
};
```

三函数区别：`stat` 跟随符号链接、`lstat` **不**跟随（拿链接自身信息）、`fstat` 用已打开 fd 查询。字段要点：`st_mode` 同时编码**类型**与**权限**；`st_ino` 是 i-node 号（**硬链接共享它**）；`st_nlink` 减到 0 才真正删除数据；时间戳区分 atime（访问）/mtime（改内容）/ctime（改 i-node）。判断类型必须用宏：`S_ISREG`（`0100000`）、`S_ISDIR`（`0040000`）、`S_ISSOCK`（`0140000`）、`S_ISLNK`（`0120000`）、`S_ISFIFO`、`S_ISCHR`、`S_ISBLK`。

18.2.6 内核如何表示打开的文件：三层结构

这是本讲**最重要的图**：

ASCII 图 1：描述符表 → 打开文件表 → v-node 表

```
  进程 A 的描述符表          打开文件表（全系统共享）        v-node 表（全系统共享）
  [每进程一张]               [含文件位置、引用计数]          [含 stat 信息]
+------------------+       +-------------------------+      +-------------------------+
| fd 0  |    *-----|------>| File pos = 12           |      | v-node for File A       |
+------------------+       | refcnt   = 1            |      |  File access            |
| fd 1  |    *-----|--+    | v-node ptr -------------|----->|  File size              |
+------------------+  |    +-------------------------+      |  File type              |
| fd 2  |    *-----|--|--->| File pos = 0            |      +-------------------------+
+------------------+  |    | refcnt   = 1            |               ^
| fd 3  |  (free)  |  |    | v-node ptr -------------|----+          |
+------------------+  |    +-------------------------+    |  +-------------------------+
| fd 4  |    *-----|--|-----------------------------------+->| v-node for File B       |
+------------------+  |                                      |  File access            |
                      |                                      |  File size              |
                      +---> 终端（File A）                    |  File type              |
                                                             +-------------------------+

  fd 0 = stdin    fd 1 = stdout    fd 2 = stderr
  "File pos 按打开文件维护（per open file），不是按描述符！"
```

三层职责：**描述符表**每进程一张、表项只是指针；**打开文件表**全系统共享，含**文件位置** `k`、**引用计数** `refcnt`、**v-node 指针**；**v-node 表**全系统共享，含 `stat` 信息。"位置在中间层"是理解此后所有共享行为的关键。

两次 `open` 同一文件：得到**两个不同的打开文件表项**（各自 `refcnt=1`、各自独立的 File pos），但 v-node 指向同一个——"逻辑上不同、物理上相同的文件"。

`fork` 后的共享：子进程**继承父进程的描述符表**（`exec` 也不改变，可用 `fcntl` 修改），父子表内容相同，每个打开文件表项 `refcnt` **加 1**。由于**位置在打开文件表里**，父子**共享同一偏移量**——这正是讲义 `ffiles2.c` 中"父读 `c`、子读 `b`"的原因。

18.2.7 `dup2` 与 I/O 重定向（dup2 and I/O Redirection）

`linux> ls > foo.txt` 如何实现？shell 在子进程里调用 `dup2(oldfd, newfd)`，**把（每进程的）描述符表项 `oldfd` 复制到表项 `newfd`**。

ASCII 图 2：`dup2(4, 1)` 前后的结构对比

```
          dup2(4, 1) 之前                                dup2(4, 1) 之后
  描述符表                                          描述符表
+---------+                                         +---------+
| fd 0 --|--> [打开文件表项: 终端   refcnt=1]        | fd 0 --|--> [打开文件表项: 终端  refcnt=0] <-- 被回收
+---------+                                         +---------+
| fd 1 --|--> [打开文件表项: 终端   refcnt=1]        | fd 1 --|--+
+---------+                                         +---------+  |
| fd 2 --|--> [打开文件表项: 终端   refcnt=1]        | fd 2 --|--> [打开文件表项: 终端  refcnt=1]
+---------+                                         +---------+  |
| fd 3   |   (free)                                  | fd 3   |  |  (free)
+---------+                                         +---------+  |
| fd 4 --|--> [打开文件表项: foo.txt refcnt=1]      | fd 4 --|--+
+---------+      File pos = 0                        +---------+  |
                     ^                                           |
                     |                +--------------------------+
                     +----------------v
                      [打开文件表项: foo.txt  refcnt=2]  <-- 两个描述符指向同一个打开文件表项
                          File pos = 0    （共享同一个偏移量）
```

标准重定向三步曲（讲义"Step #1 / Step #2"）：

```
int fd = open("foo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  /* Step 1 */
dup2(fd, 1);                                                    /* Step 2: 让 fd 1 指向该文件 */
close(fd);                                                      /* 消除多余描述符 */
```

为什么要 `close(fd)`？`dup2` 之后 fd 1 与 fd 已指向**同一打开文件表项**，`refcnt` 因 `dup2` 变成 2（同时原终端表项跌到 0 被回收）；关掉多余 fd 让 `refcnt` 回到 1，避免**文件描述符泄漏**——Proxy Lab 明确把"无 fd 泄漏"列为鲁棒性要求。

18.2.8 标准 I/O 与缓冲（Standard I/O and Buffering）

C 标准库（`libc.so`）提供**高层**函数（K&R 附录 B）：`fopen` / `fclose`、`fread` / `fwrite`、`fgets` / `fputs`、`fscanf` / `fprintf`。**流（stream）= 一个文件描述符 + 一块内存缓冲区**。程序出生时就打开三个流（`stdio.h` 定义）：

```
extern FILE *stdin;   /* standard input  (descriptor 0) */
extern FILE *stdout;  /* standard output (descriptor 1) */
extern FILE *stderr;  /* standard error  (descriptor 2) */
```

`FILE *` 与 fd 的关系：讲义揭开了实现——`fopen` 内部是 `open(...)` 后 `fdopen(fd, mode)`；`fputs` 内部循环 `write(fp->fd, s, n)` 直到写完（**它自己就在处理短计数**）；`fclose` 内部是 `close(fp->fd)`。**从 `FILE *` 拿回 fd 用 `fileno(FILE*)`，反向用 `fdopen`。**

为什么需要缓冲：应用常一次读写一个字符（`getc`/`putc`、`fgets` 逐字符读到换行），而每次 `read`/`write` 要 **> 10,000 时钟周期**。于是标准 I/O 用 `read` 一次抓一大块进缓冲区，用户函数逐字节取用，空了再补。**缓冲区在遇到 `"\n"`、`fflush`、`exit`、或从 `main` 返回时刷到输出 fd。** 讲义的 `strace` 证据（六个 `printf` 只产生一次 `write`）：

```
linux> strace ./hello
write(1, "hello\n", 6)              = 6
exit_group(0)                       = ?
```

`stderr` **无缓冲**（保证错误立刻可见）；`stdout` 连终端时**行缓冲**、重定向到文件时**全缓冲**——这个差别正是 18.3.3 陷阱的根源。也可用 `setvbuf(stream, NULL, _IOFBF|_IOLBF|_IONBF, size)` 显式设定。

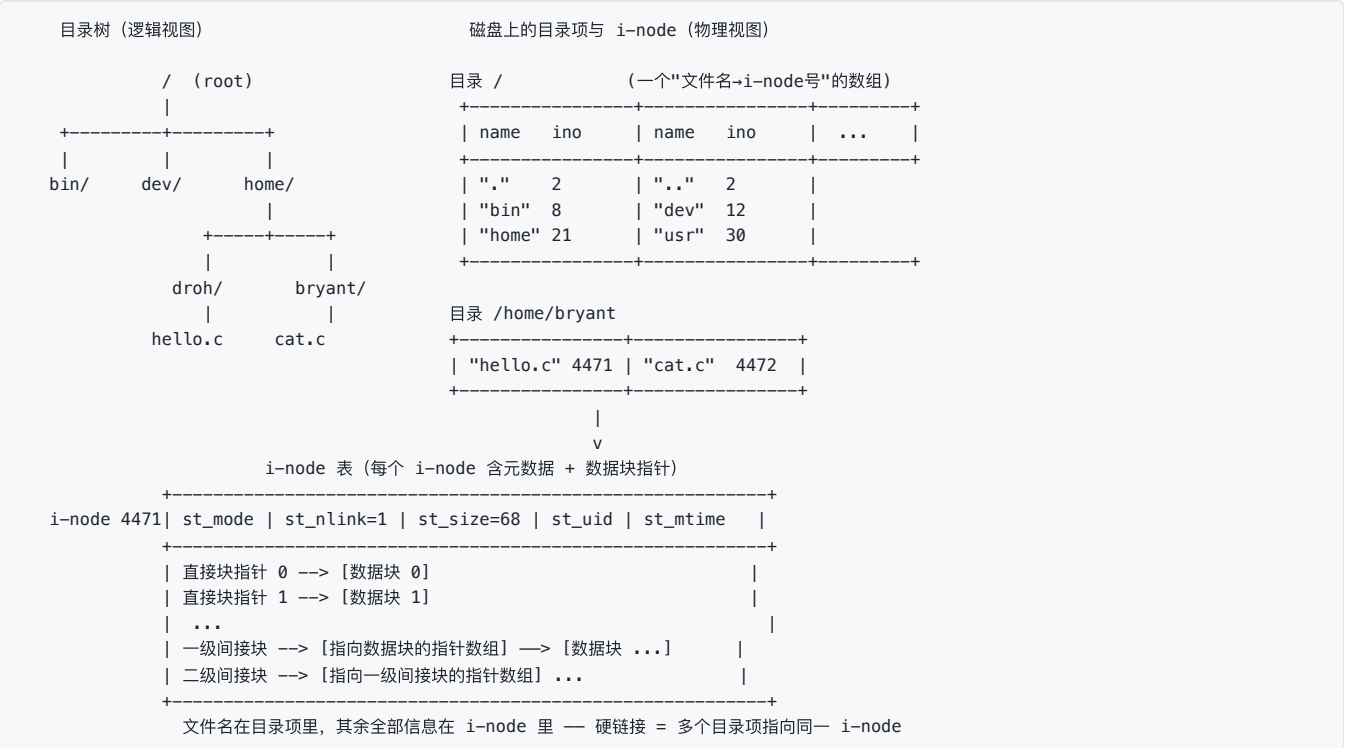
18.2.9 该用哪一层 I/O（Choosing I/O Functions）

	Unix I/O	标准 I/O
控制力 / 便利性	**最大** / 最小	较小 / 较大
元数据访问	**有**（`stat`/`fstat`/`lstat`）	**没有**
async-signal-safe	**是**，可在信号处理函数里用	**否**
网络套接字	**可以**	不合适（流与套接字的限制交互恶劣）
短计数	需自己处理（易错）	**自动处理**
文本行效率	需自己缓冲（易错）	缓冲带来效率

通用规则：能用高层就用高层。具体建议：**用标准 I/O**——处理"普通"文件；**用裸 Unix I/O**——① 在**信号处理函数**里（async-signal-safe）；② 读写**网络套接字**时（此时请用专门的带缓冲网络 I/O 库，如 `rio_*`、`libevent`、`libuv`）；③ 极少数需要绝对最高性能的场合。

18.2.10 文件系统概述：目录树与 i-node

ASCII 图 3：磁盘目录树与 i-node 的关系



- **目录层次与挂载**：所有文件组织成以**根目录** `/` 为锚的层次结构；磁盘**分区（partition）**挂载（mounting）到目录树某点后内容才可见；内核为**每个进程**维护**当前工作目录（cwd）**，用 `cd` 修改。
- **路径名**：**绝对路径**以 `/` 开头（`/home/droh/hello.c`），**相对路径**从 cwd 出发（`../droh/hello.c`）。

- **i-node**：**目录项**只管"文件名 → i-node 号"的映射；**i-node** 存元数据与**数据块指针**（直接块 + 一级/二级间接块），支持任意大的文件。目录项里**没有**大小、权限等信息——这正是硬链接能共享它们的原因。
- **硬链接 vs 符号链接**：`ln a b` 让两个目录项指向**同一 i-node**（`st_ino` 相同、`st_nlink` 变为 2），删掉任一名字数据仍在；`ln -s a c` 创建**新文件**，内容是目标路径串（`st_ino` 不同、`st_nlink` 为 1、`st_size` 等于串长），目标被删即成悬空链接。`unlink` 删除目录项，`st_nlink` 减到 0 时数据块才回收。
- **目录操作 API**：`opendir` / `readdir` / `closedir` 遍历目录，`struct dirent` 的 `d_name` 是文件名、`d_ino` 是 i-node 号；`mkdir` / `rmdir` / `chdir` / `getcwd` 增删改查。

18.3 代码示例与底层机制分析

以下三个程序均已在 `/tmp/csapp18/` 下用 `gcc -g -Wall -std=c11` **实际编译并运行**，输出为真实结果。

18.3.1 示例 A：正确处理短计数的文件复制程序

代码 (C)：

```c
/* mycp.c — 用 Unix I/O 复制文件，正确处理短计数（short count） */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define BUFSIZE 4096

/* rio_writen 风格的健壮写：保证写满 n 字节，除非出错 */
static ssize_t writen(int fd, const char *buf, size_t n)
{
    size_t nleft = n;
    ssize_t nwritten;
    const char *ptr = buf;

    while (nleft > 0) {
        if ((nwritten = write(fd, ptr, nleft)) < 0) {
            if (errno == EINTR) {   /* 被信号中断：重试，不算错误 */
                nwritten = 0;
            } else {
                return -1;          /* 真错误 */
            }
        } else if (nwritten == 0) {
            break;                  /* 不应该发生在磁盘文件上 */
        }
        nleft -= (size_t)nwritten;
        ptr   += nwritten;
    }
    return (ssize_t)(n - nleft);
}

int main(int argc, char *argv[])
{
    if (argc != 3) {
        fprintf(stderr, "usage: %s <src> <dst>\n", argv[0]);
        return 1;
    }

    int srcfd = open(argv[1], O_RDONLY);
    if (srcfd < 0) {
        fprintf(stderr, "open %s: %s\n", argv[1], strerror(errno));
        return 1;
    }

    /* O_CREAT 必须给第三个参数 mode；受 umask 修正 */
    int dstfd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, DEFFILEMODE);
    if (dstfd < 0) {
        fprintf(stderr, "open %s: %s\n", argv[2], strerror(errno));
        close(srcfd);
        return 1;
    }

    char buf[BUFSIZE];
    ssize_t nread;
    long total = 0;

    /* read 返回 0 才是 EOF；短计数 nread < BUFSIZE 完全正常 */
    while ((nread = read(srcfd, buf, sizeof(buf))) > 0) {
        if (writen(dstfd, buf, (size_t)nread) != nread) {
            fprintf(stderr, "write %s: %s\n", argv[2], strerror(errno));
            close(srcfd);
            close(dstfd);
            return 1;
        }
        total += nread;
    }
    if (nread < 0) {
        fprintf(stderr, "read %s: %s\n", argv[1], strerror(errno));
        close(srcfd);
        close(dstfd);
        return 1;
```

```
    }

    /* close 也可能"失败"：它在上报此前 write 延迟的错误 */
    if (close(dstfd) < 0) {
        fprintf(stderr, "close %s: %s\n", argv[2], strerror(errno));
        close(srcfd);
        return 1;
    }
    if (close(srcfd) < 0) {
        fprintf(stderr, "close %s: %s\n", argv[1], strerror(errno));
        return 1;
    }

    fprintf(stderr, "[mycp] copied %ld bytes from %s to %s\n",
            total, argv[1], argv[2]);
    return 0;
}
```

【代码做什么？】 ① `O_RDONLY` 打开源文件（本机常得 fd 3）。② `O_WRONLY|O_CREAT|O_TRUNC` + `DEFFILEMODE` 打开/截断目标文件，`O_CREAT` 触发三参数 `open`。③ 循环 `read(fd, buf, 4096)`：**> 0 就继续**（小于 4096 是短计数，正常），**0 是 EOF**，**< 0 才是错误**。④ 每读到一块就调用自制的 `writen`，**保证每个字节都写出去**，`EINTR` 时重试。⑤ 先 `close(dstfd)` 并检查返回值（捕获延迟写错误）。⑥ 统计信息走 `stderr`，不污染数据流。

【底层机制透视】 `read` / `write` 各是一次 `syscall`：内核按 fd 查打开文件表项、取出 `File pos`，再去 v-node 找数据块。`read` 把数据从**页缓存**拷进用户缓冲区（未命中则触发缺页与磁盘 I/O，详见 Lecture 11—12），再把 `File pos` 前移 `nread`——位置**按打开文件表项**维护，故同一文件 `open` 两次得到两个独立偏移量。`write` 只把数据放进页缓存并标脏，落盘由回写（write-back）完成：`write` 成功不代表数据在盘上，`close` 报出的错误往往来自更早的写入。

【内存布局 / 数据结构图解】（用户栈放缓冲区；内核侧 fd 3/fd 4 各自一个打开文件表项，v-node 才共享）

```
        用户栈（高地址）
   0x7ffd_xxxx  +-------------------------+
                | buf[4096]               |   <-- read/write 的用户缓冲区
                +-------------------------+
                | srcfd=3, dstfd=4        |
                | nread, total, ptr       |
                +-------------------------+   <- %rsp
        描述符表（内核，按进程）
        fd 3 -> [打开文件表项: redirect.c  File pos=0  refcnt=1]  --\
        fd 4 -> [打开文件表项: out.txt    File pos=0  refcnt=1]  --+--> v-node (st_size, 数据块指针)
```

【与汇编 / 硬件的对应】（`gcc -S` 风格片段，可见 `syscall` 前的参数搬运）

```
    movl    %ebx, %edi          # fd   -> %rdi (第 1 参数)
    movq    %r12, %rsi          # buf  -> %rsi (第 2 参数)
    movl    $4096, %edx         # n    -> %rdx (第 3 参数)
    call    read@plt            # 进入 glibc 包装，内部 movl $0, %eax; syscall
    testq   %rax, %rax
    jle     .Ldone              # <= 0 退出循环 (0 = EOF, < 0 = 错误)
    movq    %rax, %rbp          # 短计数：实际字节数留在 %rax
```

【实测验证】（真实输出）

```
$ ./mycp redirect.c copy-of-redirect.c
[mycp] copied 2079 bytes from redirect.c to copy-of-redirect.c
$ cmp redirect.c copy-of-redirect.c && echo identical
identical
```

用 `strace` 统计缓冲区大小对系统调用次数的影响：

```
BUFSIZE=1    -> read/write 系统调用次数 = 4161
BUFSIZE=512  -> read/write 系统调用次数 = 13
BUFSIZE=4096 -> read/write 系统调用次数 = 5
```

缓冲区从 1 字节调到 4096 字节，系统调用次数从 4161 降到 5——**这就是标准 I/O 缓冲存在的理由**。

18.3.2 示例 B：`dup2` 实现 stdout 重定向

代码 (C)：

```c
/* redirect.c — 用 dup2 把 stdout 重定向到文件，再恢复原 stdout */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>

/* 查看 fd 当前指向的对象（Linux: /proc/self/fd/N 是符号链接） */
static void show_fd(const char *tag, int fd)
{
    char link[64], target[512];
    snprintf(link, sizeof(link), "/proc/self/fd/%d", fd);
    ssize_t n = readlink(link, target, sizeof(target) - 1);
    if (n < 0) { target[0] = '?'; n = 1; }
    target[n] = '\0';
    fprintf(stderr, "  [%-14s] fd %d -> %s\n", tag, fd, target);
}

int main(void)
{
    if (setvbuf(stdout, NULL, _IOLBF, 0) != 0) {   /* 行缓冲，顺序可预期 */
        fprintf(stderr, "setvbuf failed\n");
        return 1;
    }

    show_fd("before", STDOUT_FILENO);
    printf("A: 这一行去终端（fd 1 指向终端/socket）\n");
    fflush(stdout);

    /* 备份原来的 stdout：dup 返回最小的可用 fd，这里应为 3 */
    int saved_stdout = dup(STDOUT_FILENO);
    if (saved_stdout < 0) { perror("dup"); return 1; }
    show_fd("saved_stdout", saved_stdout);

    int fd = open("redirect-out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    fprintf(stderr, "  opened redirect-out.txt -> fd %d\n", fd);

    /* 关键一步：dup2(oldfd=fd, newfd=1) */
    if (dup2(fd, STDOUT_FILENO) < 0) { perror("dup2"); return 1; }
    if (close(fd) < 0) { perror("close"); return 1; }   /* refcnt 回到 1 */

    show_fd("after dup2", STDOUT_FILENO);
    printf("B: 这一行去 redirect-out.txt\n");
    fflush(stdout);              /* 确认 B 已经写进文件 */

    /* 恢复：把备份的描述符拷回 fd 1 */
    if (dup2(saved_stdout, STDOUT_FILENO) < 0) { perror("dup2 restore"); return 1; }
    if (close(saved_stdout) < 0) { perror("close saved"); return 1; }
    show_fd("restored", STDOUT_FILENO);
    printf("C: 这一行又回到终端\n");
    return 0;
}
```

【代码做什么？】 ① `dup(1)` 拿到最小的可用 fd 作为**备份**（实测 3），此时表项 3 与 1 指向**同一打开文件表项**，`refcnt = 2`。② `open` 目标文件得 fd 4（3 已被占用）。③ `dup2(4, 1)`：**只改描述符表项 1**，使其指向 fd 4 所指的表项——原终端表项 `refcnt` 从 2 降到 1（`saved_stdout` 撑着）。④ `printf("B")` 经 `FILE *stdout` 缓冲，最终 `write(1,...)` **落进文件**。⑤ `dup2(saved_stdout, 1)` 把终端表项拷回 fd 1，`printf("C")` 回到终端。

【底层机制透视】 `dup2` **不复制打开文件表项**，只复制描述符表里的**指针**——重定向前后两 fd **共享同一个** **`File pos`**。`ls > foo.txt` 之所以从文件头写起，正因 shell 在 `fork` 之后、`execve` 之前完成 `dup2`，而 `execve` **保留**已打开描述符与 `File pos`。

【实测验证】（真实输出；本环境 stdout 是 socket，注意 `fd 1` 的指向变化）

```
$ rm -f redirect-out.txt && ./redirect
A: 这一行去终端（fd 1 指向终端/socket）
C: 这一行又回到终端
  [before        ] fd 1 -> socket:[3420704682]
  [saved_stdout  ] fd 3 -> socket:[3420704682]     <-- dup 返回最小可用 fd = 3
  opened redirect-out.txt -> fd 4                  <-- open 拿到下一个可用 fd = 4
  [after dup2    ] fd 1 -> /tmp/csapp18/redirect-out.txt
  [restored      ] fd 1 -> socket:[3420704682]

$ cat redirect-out.txt
B: 这一行去 redirect-out.txt
$ ls -l redirect-out.txt
-rw-r--r-- 1 runguoli mzhang 33 Sep 23 19:33 redirect-out.txt
```

（stdout 与 stderr 在此环境指向同一 socket，故终端行与 `[...]` 诊断行交错；内容归属由 `redirect-out.txt` 里**只有 B** 这一事实确证。）

18.3.3 示例 C：`fork` + 标准 I/O 缓冲区导致输出重复的陷阱

代码 (C)：

```c
/* forkbuf.c - 演示 fork 复制标准 I/O 缓冲区导致的重复输出 */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    int use_fflush = (argc > 1 && strcmp(argv[1], "flush") == 0);
    /* 让 stdout 变成全缓冲：重定向到文件时默认就是全缓冲 */
    setvbuf(stdout, NULL, _IOFBF, 4096);

    printf("PARENT-PREFIX: pid=%d\n", (int)getpid());

    if (use_fflush) {
        fflush(stdout);          /* 修复：fork 前清空缓冲区 */
    }

    pid_t pid = fork();
    if (pid < 0) {
        fprintf(stderr, "fork: %s\n", strerror(errno));
        return 1;
    }
    if (pid == 0) {
        printf("CHILD: pid=%d\n", (int)getpid());
        return 0;                /* child 自己退出，也会 flush */
    }
    printf("PARENT: pid=%d\n", (int)getpid());
    waitpid(pid, NULL, 0);
    return 0;                    /* return from main 会 flush */
}
```

【代码做什么？】 ① `setvbuf` 把 `stdout` 强制设为**全缓冲**（重定向到文件时这是默认行为）。② `printf("PARENT-PREFIX...")` 只把字节写进**用户态缓冲区**，尚无任何 `write` 调用。③ `fork()`：**子进程复制整个地址空间，包括这块缓冲区**。④ 子进程追加 `CHILD`、父进程追加 `PARENT`，各自在退出时刷缓冲区。⑤ 带 `flush` 参数时在 `fork` 前先 `fflush`，缓冲区为空，重复消失。

【底层机制透视】 关键在于**缓冲区位于用户地址空间**，而 `fork` 复制地址空间；`File pos`、页缓存等内核态资源不复制（只增加 `refcnt`）。所以"重复的是**用户态缓冲的内容**，共享的是**内核态的文件位置**"。`stdout` 连着终端时是**行缓冲**，每条 `\n` 立刻触发 `write` 清空缓冲区，`fork` 时缓冲区恰为空，**陷阱不出现**——这正是该 bug 只在重定向到文件或管道时冒出来的原因。

【内存布局 / 数据结构图解】

```
      fork 之前                          fork 之后
   父进程地址空间                      父进程                        子进程（副本）
  +----------------------+          +----------------------+   +----------------------+
  | stdout FILE 结构     |          | stdout FILE 结构     |   | stdout FILE 结构     |  <- 独立副本
  |   buf: "PARENT-      |          |   buf: "PARENT-      |   |   buf: "PARENT-      |
  |         PREFIX...\n"|          |         PREFIX...\n"|   |         PREFIX...\n"|
  |   fd = 1             |          |         + "PARENT\n"|   |         + "CHILD\n" |
  +----------------------+          +----------|-----------+   +----------|-----------+
                                               | write(1,...)             | write(1,...)
                                               v                          v
                                    [打开文件表项: 终端/文件   File pos 共享, refcnt=2]
                                    两个进程的 write 都追加到同一个 File pos 之后
```

【实测验证】（真实输出）

```
$ ./forkbuf > noflush.out 2>&1 ; cat -A noflush.out
PARENT-PREFIX: pid=3591877$
CHILD: pid=3591878$
PARENT-PREFIX: pid=3591877$        <-- 重复! 子进程刷出了 fork 前就存在的缓冲内容
PARENT: pid=3591877$
$ wc -l < noflush.out
4

$ ./forkbuf flush > flush.out 2>&1 ; cat flush.out
PARENT-PREFIX: pid=3591884
CHILD: pid=3591885
PARENT: pid=3591884
$ wc -l < flush.out
3
```

未 flush：4 行（`PARENT-PREFIX` 两次）；fork 前 `fflush`：3 行。 修复手段优先级：① `fork` 前 `fflush(NULL)`；② `setvbuf(stdout, NULL, _IONBF, 0)` 关掉缓冲；③ 只用 Unix I/O 的 `write` 做输出——这也正是 Shell Lab 中信号处理函数不能调 `printf` 的原因（既非 async-signal-safe，又引入缓冲困惑）。

18.3.4 补充实测：短计数、元数据、链接与目录

管道上的短计数（请求 4096 字节，实际只得到 5 字节）：

```
$ ./shortread
requested 4096 bytes, got 5  -> short count? YES
second read returned 0 (0 means EOF, not error)
```

底层是管道时 `read` 给多少算多少，剩下的**必须**靠循环补齐；写端关闭后返回 **0**（EOF），**不是** -1。

`stat` 元数据与文件类型（`lstat` + `S_IS*` 宏）：

```
$ ./statinfo redirect.c
type     : regular file  (st_mode = 0100000)
perms    : 644
st_size  : 2429 bytes
st_ino   : 504007666
st_nlink : 1
st_blksize/st_blocks: 4096 / 8
st_mtime : 2026-09-23 19:33:23

$ ./statinfo link-to-redirect.c        # lstat 不跟随符号链接
type     : symbolic link  (st_mode = 0120000)
st_size  : 10 bytes                    # 恰好是 "redirect.c" 的 10 个字符
```

硬链接 vs 符号链接（`ln` 与 `ln -s`）：

```
$ stat -c '%n ino=%i nlink=%h' hard.txt hard2.txt sym.txt
hard.txt  ino=504007647 nlink=2      <-- 同一 i-node, nlink 递增
hard2.txt ino=504007647 nlink=2      <-- 同一 i-node
sym.txt   ino=504007648 nlink=1      <-- 独立的新文件，内容是路径串
```

目录遍历（`opendir` / `readdir` / `closedir` + `mkdir` / `chdir` / `getcwd` / `rmdir`）：

```
$ ./listdir demo-dir
cwd = /tmp/csapp18
name                  d_ino            size  type
------------------------------------------------------
.                     504007644          32  dir      <-- "." 指向自身
..                    504007660        4096  dir      <-- ".." 指向父目录
a.txt                 504007645           0  file
b.bin                 504007646           0  file
after chdir: cwd = /tmp/csapp18/demo-subdir
```

讲义 `ffiles1.c` 的复现（三次 `open` + `dup2`）：

```
$ ./ffiles1
fd1=3 fd2=4 fd3=5  c1=a c2=a c3=b
```

三个独立 `open` 给出三个**独立的 `File pos`**；`dup2(fd2, fd3)` 让表项 3 与 4 指向同一打开文件表项，**共享位置**。于是 `read(fd1)` 从位置 0 读走 `a`，`read(fd2)` 从它自己的位置 0 读走 `a`，`read(fd3)` 与 fd2 共享位置故读走 `b`。**与讲义答案 `c1 = a, c2 = a, c3 = b` 完全一致。**

`fork` 共享偏移量（讲义 `ffiles2.c`，`sleep` 使某进程必定先读）：

```
$ for i in 1 2 3; do ./ffiles2; done
Child: c1 = a, c2 = b        <-- 子进程先读走 b
Parent: c1 = a, c2 = c       <-- 父进程随后读走 c
Parent: c1 = a, c2 = b
Child: c1 = a, c2 = c
Child: c1 = a, c2 = b
Parent: c1 = a, c2 = c
```

两种次序都覆盖了讲义给出的答案；`c1` **总是** `a`（它在 `fork` 前就读好，被复制进两个地址空间），而 `c2` 在 `b` / `c` 之间取决于调度——**共享 `File pos` 的铁证**。

18.4 实验关联

L6 Shell Lab（tsh）： – tsh 官方规格写着"**无需支持管道 `|` 或 I/O 重定向 `<` `>`**"，但**重定向的机制**正是本讲的 `dup2`：真实 shell 在 `fork` 之后、`execve` 之前完成 `open` + `dup2` + `close`，因为 `execve` 会保留已打开的描述符。理解了"位置在打开文件表项里"，就能解释 `ls > foo.txt` 为何从文件头写起。 – Style 分要求**检查每一个系统调用的返回值**；**信号处理函数里只能用 async-signal-safe 函数**，`printf` 不安全，请用 `write`。 – 隐性关联：`fork` 的**输出重复**（18.3.3）——父进程在 `fork` 前若留下未刷新的标准 I/O 缓冲，子进程会把它一起刷出来。

L7 Proxy Lab：本讲是它的直接技术前提。 – **解析 HTTP 请求**：用 `rio_readlineb(&rio, buf, MAXLINE)` 逐行读请求行与头部，因为 socket 上的 `read` **必然**短计数，而"一行"多长事先未知。注意讲义的 `\r\n`：HTTP 每行以 CR+LF 结束，整个请求以空行 `\r\n` 终止。 – **转发请求与响应**：用 `rio_writen`（**永不返回短计数**），绝不要用 `fputs` / `fwrite`（标准 I/O 不适合 socket 且非 async-signal-safe）。 – **判断本地文件是否存在、是否可读**：用 `stat` + `S_ISREG`（Tiny 服务器判断静态文件），**标准 I/O 根本不提供元数据接口**。 – **响应体必须按 `Content-length` 精确转发**：这是 proxy **最容易出 bug 的地方**。拿到 `Content-length: N` 后用 `rio_readnb`（或循环 `rio_readn`）**恰好**转发 N 字节：少一字节客户端挂住或截断，多一字节污染下一个响应。 – **二进制内容**：讲义警告"网上并非都是 ASCII 文本"，图片与视频都是二进制，**不要**用 `rio_readlineb`、`strlen`、`strcpy` 处理响应体。 – **鲁棒性**：忽略 `SIGPIPE`、优雅处理 `EPIPE` 与 `ECONNRESET`；**不要**用 `csapp.c` 那套"出错即退出"的包装——代理一旦开始接受连接就不该终止（讲义："Stevens 包装让代码变短，但不给你从错误中恢复的机会"）。 – **fd 泄漏**：`dup2` 后一定 `close` 多余 fd；每个连接处理完要关闭 clientfd 与 serverfd。

18.5 常见错误与调试技巧

- **忽略短计数**：以为 `write(fd, buf, n)` 一定写满 `n` 字节；磁盘上"碰巧"不出错，一上 socket/管道就丢数据。**调试**：`strace -e trace=read,write -s 64 ./prog`。
- **把 EOF 当成错误**：写 `while (read(fd, buf, n) != n)` 导致读到文件尾就报错退出。**调试**：记住三值语义——`> 0` 正常（可能短）、`= 0` EOF（仅 `read`）、`< 0` 错误。
- **`EINTR` 未重试**：信号递达打断 `read` / `write`，返回 −1 且 `errno == EINTR`，被误判为致命错误。**调试**：加 `if (errno == EINTR) continue;`；用 `strace -f -e trace=signal,read,write` 复现。
- **双重 close / fd 泄漏**：第二次 `close` 可能关掉**别的线程刚打开的**同名 fd；`dup2` 后忘 `close` 多余描述符则长运行进程耗尽 fd（`ulimit -n` 通常 1024）。**调试**：`valgrind --track-fds=yes ./prog`、`ls -l /proc/<pid>/fd/ | wc -l` 观察是否随请求数单调增长。
- **混用标准 I/O 与 Unix I/O 导致错序**：`printf` 的字节还在 `FILE*` 缓冲区，紧接的 `write` 已进内核，`exit` 刷缓冲时前者反而后到。本讲实测顺序为 `2 → 1 → 3`，每次 `printf` 后加 `fflush(stdout)` 即恢复 `1 → 2 → 3`。**调试**：`strace -e trace=write ./prog`。
- **`fork` 后标准 I/O 缓冲区被复制**：只在重定向到文件时出现的"重复输出"。**调试**：`fork` 前 `fflush(NULL)`；`strace -f -e trace=write` 会看到同一段字节被写两次。

- **`open` 忘记第三参数 / `mode` 错误**：`O_CREAT` 缺 `mode` 时新文件权限取栈上垃圾值（UB，实际结果依编译器/机器而异）。**调试**：`stat -c %a` 检查权限；统一用 `DEFFILEMODE` 并注意 `umask`。

18.6 关键要点

- **一切皆文件**：文件就是字节序列 $B_0 \dots B_{m-1}$，内核用同一套 `open` / `read` / `write` / `close` / `stat` / `lseek` 服务磁盘、终端、管道、套接字乃至 `/proc`。
- **短计数是常态而非错误**：`read` 返回 0 才是 EOF，返回 –1（非 `EINTR`）才是错误；`rio_*` 四个函数就是为封装这些循环而生，**网络 I/O 必须用它们**。
- **文件位置在"打开文件表"里，不在描述符表里**：因此 `open` 两次得到两个独立偏移量，而 `fork` 与 `dup2` 让多个描述符**共享**同一偏移量——这一条同时解释 `ffiles1` / `ffiles2` 的输出与 `>` 重定向的语义。
- **`dup2(oldfd, newfd)` 是重定向的全部秘密**：它只改描述符表项，配合"`open` 返回最小可用 fd"与"`execve` 保留已打开 fd"，shell 才能实现 `ls > foo.txt`。
- **标准 I/O = fd + 用户态缓冲区**：缓冲把上千次系统调用压成几次（实测 4161 → 5），代价是**非 async-signal-safe、无元数据接口、不适合套接字**，以及 `fork` 后缓冲区被复制的重复输出。
- **`FILE *` 与 fd 用 `fileno` / `fdopen` 互转**；`st_mode` 类型判断必须用 `S_ISREG` / `S_ISDIR` / `S_ISSOCK` 宏；硬链接共享 i-node，符号链接是独立的新文件。

18.7 思考题（带答案）

Q1（推演题）：给定下面的程序，`f1.txt`、`f2.txt` 均初始不存在。请判断：两次 `open` 各返回什么 fd？最终哪些 fd 指向哪个文件？程序共输出几行、写到哪里？为什么？

```
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    int fd1 = open("f1.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);  /* 记为 a */
    printf("hello from parent\n");        /* 未 fflush */
    int fd2 = open("f2.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);  /* 记为 b */
    dup2(fd2, STDOUT_FILENO);
    close(fd2);
    if (fork() == 0) {
        printf("child\n");
        return 0;
    }
    printf("parent\n");
    wait(NULL);
    return 0;
}
```

答：fd 从 3 开始分配，故 **a = 3、b = 4**。`dup2(fd2, 1)` 让**描述符表项 1 改指向 `f2.txt` 的打开文件表项**（`refcnt` 变 2），随后 `close(fd2)` 让 `refcnt` 回到 1；此时**只有 fd 1 指向 `f2.txt`**，fd 3 仍指向 `f1.txt`，但该文件上**没有任何 `write`**，故只是一具 0 字节空壳。`printf("hello from parent\n")` 的字节此时还在 `stdout` 用户态缓冲区里（`fork` 前未 `fflush`，stdout 指向磁盘文件故为全缓冲），而 `fork` **复制了这块缓冲区**：子进程退出时刷出 `hello from parent` + `child`，父进程刷出 `hello from parent` + `parent`——**共 4 行**（`hello` 出现**两次**），全部写入 `f2.txt`，终端**一行都没有**。若在 `fork` 前加 `fflush(NULL)` 则只剩 **3 行**。（本机实测：终端无输出、`f1.txt` 恰 0 字节、`f2.txt` 恰上述 4 行。）

Q2（"直观但错误的想法"）：有同学说："我用 `write(fd, buf, n)` 写磁盘文件，返回值永远是 `n`，所以 Proxy Lab 里也可以直接对 socket 这么写，代码更简洁。"错在哪？

答：错在**把"磁盘文件的实现特性"当成"`write` 的语义保证"**。讲义明确区分：短计数**不会**发生在"写磁盘文件"和"读磁盘文件（除 EOF 外）"上，但**会**发生在"读写网络套接字、管道"以及"从终端读文本行、遇到 EOF"时。所以磁盘上"碰巧总是写满"是**环境特性**而非**接口契约**。转发大对象时 socket 发送缓冲区一满，`write` 就返回小于 `n` 的值，剩下字节**静默丢失**，响应体短于 `Content-length`。正确做法是 `rio_writen`（讲义保证它**永不返回短计数**）。

Q3（推演题）：为什么讲义 `ffiles2.c` 中**一定有**一个进程读到 `b`、另一个读到 `c`，而 `c1` 却**总是** `a`？

答：`fd1` 在 `fork` **之前** `open`，父子描述符表项指向**同一个打开文件表项，共享同一个 `File pos`**。`read(fd1, &c1, 1)` 也在 `fork` **之前**，所以 `c1 = 'a'` 是被复制进两个地址空间的普通变量，必为 `a`。第二个 `read` 在 `fork` **之后**：先执行者从共享位置 1 读走 `b` 并把位置推到 2，后执行者从位置 2 读走 `c`；`sleep(s)` 只决定先后。本机三次运行得到 `Child=b/Parent=c`、`Parent=b/Child=c`、`Child=b/Parent=c`，与讲义两种次序答案吻合。

Q4（计算题）：某程序用 `read(fd, buf, 1)` 逐字节把 1 MiB（2^{20} 字节）文件复制到另一个磁盘文件，每个系统调用耗时 10,000 个时钟周期。改用 4096 字节缓冲区后，系统调用次数与耗时各降低多少倍？

答：逐字节方案共 2^{20} 次 `read` 加 2^{20} 次 `write` = $2{,}097{,}152$ 次系统调用，耗时约 $2^{21} \times 10^4 \approx 2.1 \times 10^{10}$ 个周期。4096 字节方案需 $2^{20}/2^{12} = 256$ 轮即 512 次调用，耗时约 5.12×10^6 个周期。两者都降低 $\mathbf{2^{12} = 4096}$ 倍（即缓冲区增大的倍数）。18.3.1 的 `strace` 实测（4161 →

5）方向一致，只是文件仅 2079 字节，绝对次数远小于理论值。

Lecture 19: 文件系统与网络编程 I (File Systems; Network Programming Part I)

> **讲义对应**：CMU 15–213 Lecture 19 — File Systems / Network Programming (Part I)（素材：`F25-19-fs_netprog1.txt`） **教材对应**：CS:APP3e 第 11 章 11.1–11.4（网络编程：客户端–服务器模型、网络、IP 与域名、套接字接口） **关联 Lab**：**L7 Proxy Lab**（Web 代理：套接字 + HTTP + 并发 + 缓存）

19.1 概述

本讲前半部分用 SFS（Shark File System，L8 的骨架文件系统）快速串起"文件系统如何把盘块抽象成文件"，后半部分进入**网络编程**。核心问题是：**两个位于不同主机上的进程，如何通过一条"看不见的线"可靠地交换字节？** 答案是 TCP/IP 协议族 + **套接字接口（sockets interface）**——它把网络访问统一成**文件描述符**，于是上一讲（L18）学过的 Unix I/O 与 RIO 包可以原封不动地复用到网络上。本讲是 L20（并发服务器）与 **Proxy Lab** 的直接前提：Proxy Lab 的骨架代码就是这里的 `open_listenfd` / `accept` / `rio_readlineb` 循环。

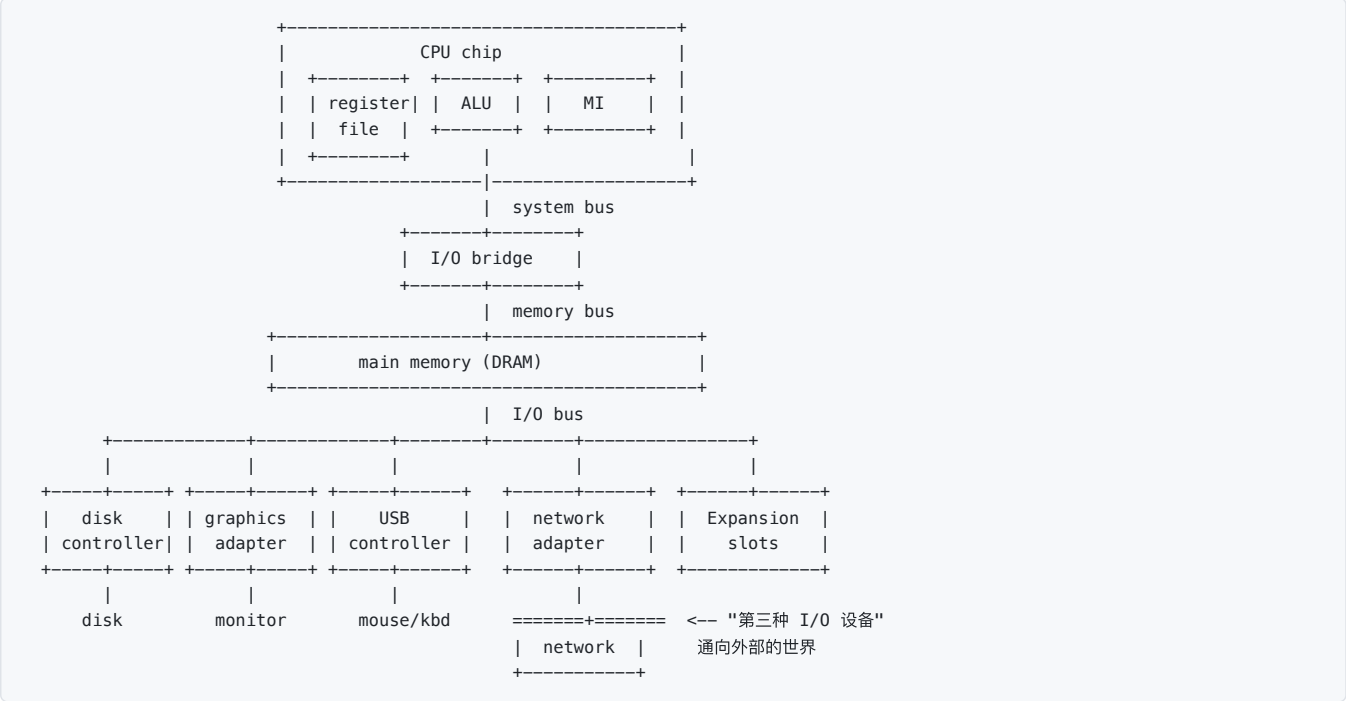
19.2 核心概念与底层机制图解

19.2.1 文件系统速览（略，为 L8 铺垫）

文件系统**管理磁盘块以提供文件抽象**：磁盘表面划分为**磁道（track）**，磁道划分为**扇区（sector）**，文件系统以**块（block）**为单位管理。创建文件系统要做**格式化**：指定一个（或多个）**超级块（super block）**记录类型/大小/根目录/空闲块，其余块标记为空闲。**目录本身是一种特殊文件**，把字符串映射到文件（还可能映射到目录）。打开文件做三件事：找空闲 fd、分配打开文件表项（pos、权限）、把文件信息载入内存。删除文件分两步：**移除映射** + **把块放回空闲链表**（"像 `free()`，但要先回答：打开中的文件能被删除吗？"）。SFS 的特点：用 `mmap` 把整个"磁盘"文件映射进内存、块大小 512 字节、块 0 是超级块（其他引用 0 表示 NULL）、**扁平目录结构**。SFS 的难点不在代码量（平均约 200 行），而在**识别临界区**——临界区由共享变量/资源共享定义，可能是两个线程调用相同**或不同**的函数。

19.2.2 网络作为第三种 I/O 设备（Network as an I/O Device）

- **定义与目的**：网络是主机上又一种 I/O 设备，但它比磁盘**更深、更慢**——数据要经过网卡 → 交换机 → 路由器 → 骨干网 → …… 才能到达对端。
- **直观解释**：把网络想成"主机之间的一条 I/O 总线"。本地 I/O 总线连 CPU 与磁盘/显卡；而网络适配器（network adapter）把这条总线延伸到了**全世界**。访问 `www.google.com` 就像访问一块"别人家的磁盘"，只是延迟高几个数量级、还可能丢数据。
- **底层机制图解**：教材名言——*"网络是一种 I/O 设备"*。



- **与机器码/硬件的对应**：应用进程通过**系统调用**陷入内核，内核的网络协议栈把数据交给网卡；网卡发完后通过**中断（interrupt）**通知 CPU 数据到达/发送完成。所以网络 I/O 与磁盘 I/O 在硬件层面是同一套模式：`DMA + 中断`。

19.2.3 网络的分层与协议（Protocol Layering）

- **定义与目的**：**协议（protocol）** 是一组规则，规定主机与路由器在跨网络传输数据时如何协作；它"抹平"了不同 LAN/WAN 之间的差异。
- **直观解释**：寄国际快递。你只写"寄到巴黎"，剩下的交给邮政系统：本地区域中心 → 国家交换中心 → 目的国 → 收件人。每一层只关心自己的信封，不关心里面装什么。
- **底层机制图解（协议分层）**：

```
应用层   | HTTP / FTP / SMTP / SSH / DNS ...                          | 进程间报文
--------+-----------------------------------------------------------+----------
传输层   | UDP（不可靠数据报）   |  TCP（可靠、面向连接、全双工字节流）| 端到端(端口)
--------+----------------------+-----------------------------------+----------
网络层   | IP（Internet Protocol）—— 主机到主机、尽力而为的数据报      | 主机到主机(IP)
--------+-----------------------------------------------------------+----------
链路层   | Ethernet / WiFi / Fibre Channel / T1 / DSL ...            | 一跳(帧)
--------+-----------------------------------------------------------+----------
物理层   | 双绞线 / 光纤 / 无线电                                      | 比特
```

封装（encapsulation）：数据自上而下逐层加头，到对端再逐层剥头。

```
 Host A                                                    Host B
 +---------+      +---------+---------+      +---------+---------+---------+
 |  data   | ---> | FH1 |  data+PH | ---> | FH2 |  data+PH        | ---> |  data   |
 +---------+      +---------+---------+      +---------+---------+---------+
    应用数据       LAN1 帧                    LAN2 帧
                    |  Router 拆 FH1、装 FH2  |
 PH = internet packet header (IP 头：目的地址、大小…)
 FH = LAN frame header       (链路层帧头：MAC 地址…)
```

- **与机器码/硬件的对应**：分层是**软件**实现的（内核协议栈 + 用户态库），只有最底层落到网卡固件。这也解释了"包可能走不同路径"：每一跳的路由器独立决策，IP 层不保证路径一致。

19.2.4 IP 地址、域名与 DNS

一个程序员眼中的 Internet 有三件事：**32 位 IP 地址**（如 `128.2.203.179`，`127.0.0.1` 永远是本机）、**域名系统（DNS）** 把人类可读的域名映射到 IP（`www.cs.cmu.edu` → `128.2.217.3`）、**连接**让一个进程与另一主机上的进程通信。

- **IPv4 vs IPv6**：IPv4 于 1981 年规定，**32 位**地址；从 1990 年代起就知道地址不够用。IPv6 于 1996 年规定，**128 位**地址（`2001:0db8:0:0:0:0:cafe:1a7e`），但**采用极慢**，因为要更换路由器（CMU 的网络完全不支持 IPv6！）。好消息是**应用程序员基本不用在意**——套接字 API 让同一份代码能无缝使用两者。
- **点分十进制（dotted decimal notation）**：32 位地址按字节写成十进制、用点分隔：`0x8002C2F2` = `128.2.194.242`。
- **DNS 映射的性质**（用 `nslookup` 可观察）：

```
一对一的映射:      whaleshark.ics.cs.cmu.edu -> 128.2.210.175
多名映射同一 IP:   cs.mit.edu   -> 18.25.0.23
                    eecs.mit.edu -> 18.25.0.23      (同一等价类)
反向查询:          nslookup 18.25.0.23
                    -> 23.0.25.18.in-addr.arpa   name = eecs.mit.edu.
多名映射多 IP:     www.twitter.com -> 104.244.42.65
                                      104.244.42.129
                                      104.244.42.193
                                      104.244.42.1     (每次顺序可能不同!)
有效但无地址:      ics.cs.cmu.edu -> (No Address given)
本机固定:          localhost -> 127.0.0.1   (环回地址 loopback)
```

概念上，DNS 是一个全球分布式数据库；每个**主机条目（host entry）** 定义了域名与 IP 的映射，数学上是一个**域名与 IP 的等价类**。

19.2.5 客户端–服务器模型与连接剖析

- **定义与目的**：绝大多数网络应用基于**客户端–服务器模型（client-server model）**：服务器进程管理某种**资源**，客户端发请求，服务器操纵资源产生响应。
- **直观解释**：**自动售货机**。售货机（服务器）守着货物（资源）；你（客户端）投币按钮（请求），它掉出商品（响应）；没有请求就一直待机。
- **底层机制图解**：

```
Client process                                  Server process
     |                                                |
     |  1. 客户端发请求  ------------------------------>   |
     |                                                |  2. 服务器处理请求
     |  3. 服务器发响应  <------------------------------   |     (操纵资源)
     |                                                |
4. 客户端处理响应                                  [ Resource ]

注意：客户端与服务器都是"运行在主机上的进程"，可同机也可异机。
```

连接（connection）的三个性质：**点对点（point-to-point）**——连接一对进程；**全双工（full-duplex）**——数据可同时双向流动；**可靠（reliable）**——源端发出的字节流最终按原顺序到达目的端。**套接字（socket）是连接的端点**，套接字地址是 `IP:端口` 对。**端口是 16 位整数**：**临时端口（ephemeral port）** 由客户端内核自动分配；**知名端口（well-known port）** 与服务绑定。

```
Connection socket pair
(128.2.194.242:51213 , 208.216.181.15:80)
 \_____client_____/    \_____server_____/

Client socket address       128.2.194.242 : 51213  <- 临时端口(内核分配)
Server socket address       208.216.181.15 :   80  <- 知名端口(Web)

内核靠"目的端口"把到达的包分派给正确的服务进程：
   目的 = 128.2.194.242:80  --> Web server (port 80)
   目的 = 128.2.194.242:7   --> Echo server (port 7)
```

常见知名端口：`echo 7`、`ftp 21`、`ssh 22`、`smtp 25`、`http 80`、`https 443`；映射表在 `/etc/services`。

19.2.6 字节序：网络字节序 vs 主机字节序

- **定义与目的**：**IP 地址与端口号在内存中一律以大端字节序（big-endian / network byte order）存放**。这对任何在包头里跨机器传输的整数都成立。
- **直观解释**：两个国家的人约定"先说高位"，但本国人习惯"先说低位"。打电话时必须按约定说，否则对方听到的是反的。
- **底层机制图解**：

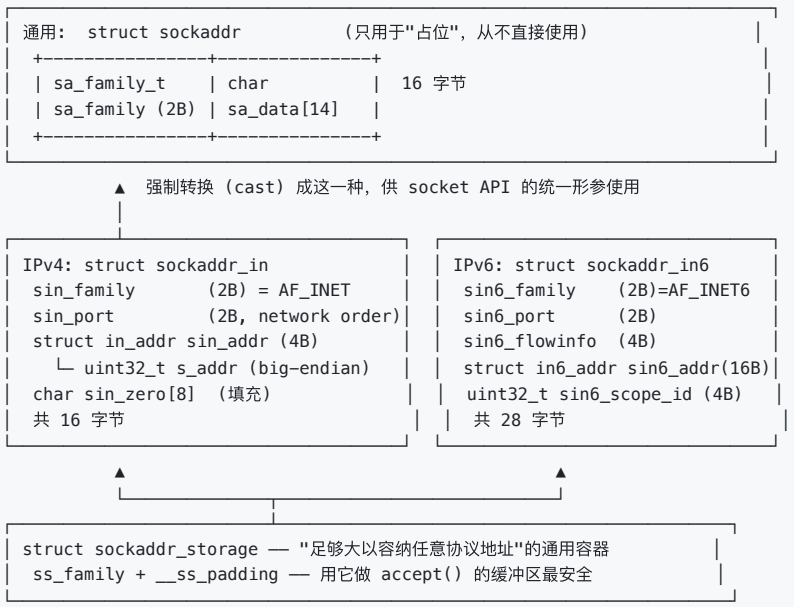
```
struct in_addr { uint32_t s_addr; /* network byte order (big-endian) */ };

值 128.2.194.242 = 0x8002C2F2

小端主机内存:  低地址 -> 高地址
     +------+------+------+------+
     | 0xF2 | 0xC2 | 0x02 | 0x80 |   把 0x8002C2F2 直接存进去（错！）
     +------+------+------+------+
网络序内存(大端):
     +------+------+------+------+
     | 0x80 | 0x02 | 0xC2 | 0xF2 |   htonl(0x8002C2F2) 之后（对）
     +------+------+------+------+
```

- **与机器码/硬件的对应**：x86-64 是**小端**机器，所以必须显式转换。四个转换函数（`arpa/inet.h`）：`htonl` / `htons`（host→network，用于 long/short）、`ntohl` / `ntohs`（network→host）。**这是网络编程最常见的 bug 来源之一。**

19.2.7 套接字地址结构与全部函数（Sockets Interface）

套接字接口是 1980 年代初随 Berkeley Unix 诞生的系统级函数组，与 Unix I/O 配合使用，如今在所有现代系统上都有。对内核而言**套接字是通信端点**；对应用而言**套接字就是一个文件描述符**——正因如此才能复用 `read` / `write` 与 RIO。**套接字 I/O 与普通文件 I/O 的唯一区别只在于"如何打开描述符"。**

套接字地址结构（ASCII 图）：



经典错误示例：`struct sockaddr_in` 是 IPv4 专用的；写协议无关代码时应使用 `struct sockaddr_storage`，否则一旦客户端用 IPv6 连接就会缓冲区溢出。

完整套接字函数速查表：

函数	作用	谁调用	关键参数	常见错误
`getaddrinfo(node, service, hints, &res)`	主机名+服务名 → 地址链表（协议无关，线程安全）	客户端+服务器	`hints.ai_socktype=SOCK_STREAM`；服务器加 `AI_PASSIVE`	忘记检查返回值；忘记 `freeaddrinfo`；忘记 `memset(&hints,0,...)`
`getnameinfo(sa, salen, host, hl, serv, sl, flags)`	套接字地址 → 主机名/服务名字符串（反向查询）	服务器（`accept` 之后）	`NI_NUMERICHOST` / `NI_NUMERICSERV` 避免阻塞式反查	缓冲区长度搞错；默认 flags 会做反向 DNS，可能阻塞
`freeaddrinfo(res)`	释放 `getaddrinfo` 返回的链表	客户端+服务器	—	内存泄漏
`socket(domain, type, protocol)`	创建套接字，返回描述符	客户端+服务器	`AF_INET` / `AF_INET6`，`SOCK_STREAM`，`0`	忘记检查 `-1`；类型与用途不符
`connect(sockfd, addr, addrlen)`	客户端发起三次握手，建立连接	仅客户端	目标服务器地址	用 `listenfd` 去 `connect`；未处理 `ECONNREFUSED`
`bind(sockfd, addr, addrlen)`	把套接字绑定到本地地址/端口	仅服务器	端口须 `htons()`	**忘记 `htons`**；端口被占（`EADDRINUSE`）
`listen(sockfd, backlog)`	把主动套接字转为**监听**套接字	仅服务器	`backlog`（CS:APP 用 `LISTENQ=1024`）	顺序颠倒（`listen` 在 `bind` 前）
`accept(listenfd, &addr, &addrlen)`	从已完成队列取一个连接，**返回新的 connfd**	仅服务器	`addr` 传 `sockaddr_storage*`，`addrlen` 是**值-结果**参数	用 `listenfd` 读写数据；`addrlen` 未初始化为缓冲区长度
`setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &1, sizeof(int))`	允许立即重用处于 `TIME_WAIT` 的地址	仅服务器	必须在 `bind` **之前**	忘记设置 → 重启服务器报 "Address already in use"
`close(fd)`	关闭描述符，发送 FIN	客户端+服务器	—	忘记关闭 `connfd` → **描述符泄漏**（长跑服务器致命）
`htonl` / `htons` / `ntohl` / `ntohs`	主机序 ↔ 网络序转换	客户端+服务器	16 位用 s，32 位用 l	漏掉转换 → 端口/地址错乱

函数	作用	谁调用	关键参数	常见错误
`inet_pton` / `inet_ntop`	文本 ↔ 二进制 IP 地址（协议无关，可重入）	客户端+服务器	`AF_INET` / `AF_INET6`	再用已废弃的 `inet_ntoa`（不可重入）
`gethostbyname`	主机名 → IP（**旧接口**）	不推荐	—	**不可重入、非线程安全**；已被 `getaddrinfo` 取代
`rio_readlineb` / `rio_writen`	带缓冲的健壮读写（L18）	客户端+服务器	—	在套接字上用标准 I/O（`fgets` / `fputs`）导致缓冲问题

客户端–服务器套接字函数调用流程图：

```
          CLIENT                                      SERVER
            |                                           |
            |              (等待连接请求)          socket() <- 创建监听套接字
            |                                       setsockopt(SO_REUSEADDR)
            |                                       bind()   <- 绑定知名端口
            |                                       listen() <- 转为监听套接字
            |                                           |
      getaddrinfo()                                   accept() <- 阻塞, 返回 connfd
        socket()|  <---- 三次握手 ----                    |
       connect()|------------->----------------------      |
            |                                           |
       rio_writen() -------- HTTP 请求 ---------------> rio_readlineb()
       rio_readlineb() <----- HTTP 响应 -------------- rio_writen()
            |                                           |
         close() -------- FIN ----------------->  rio_readlineb() 返回 0 (EOF)
            |                                           |
            |                                       close(connfd) <- 只关 connfd!
            |                                       (回到 accept 等待下一个)
            v                                           v
       注意: clientfd 一个进程一个          注意: listenfd 只创建一次且永不关闭
                                                  connfd 每次连接一个、用完就关
```

`listenfd` vs `connfd`——必须分清的两个描述符：

	`listenfd`（监听描述符）	`connfd`（连接描述符）
创建次数	服务器生命周期内**只创建一次**	**每个客户端连接创建一个**
创建者	`open_listenfd`（内部 `socket` + `bind` + `listen`）	`accept` 的返回值
作用	作为"总机"，只负责接收连接请求	作为"分机"，负责与这个客户端收发数据
I/O	**绝不能**对它 `read` / `write`	`read` / `write` 数据只能用它
关闭时机	服务器退出时	处理完该客户端**立刻关闭**

19.3 代码示例与底层机制分析

19.3.1 字节序演示：`htonl` 前后对比

```c
/* byteorder.c —— 演示网络字节序(大端)与主机字节序的差异 */
#include <stdio.h>
#include <stdint.h>
#include <arpa/inet.h>

static void dump(const char *tag, uint32_t v)
{
    unsigned char *p = (unsigned char *)&v;
    printf("%-22s = 0x%08X   内存字节: %02x %02x %02x %02x\n",
           tag, v, p[0], p[1], p[2], p[3]);
}

int main(void)
{
    uint32_t one = 1;
    printf("本机字节序: %s\n", (*(unsigned char *)&one == 1) ? "小端 (little-endian)"
                                                            : "大端 (big-endian)");

    struct in_addr a;                           /* IPv4 地址结构 */
    inet_pton(AF_INET, "128.2.194.242", &a);    /* 文本 -> 二进制(网络序) */
    dump("inet_pton 之后 s_addr", a.s_addr);
    char buf[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &a, buf, sizeof buf);    /* 二进制(网络序) -> 文本 */
    printf("inet_ntop 还原        = %s\n", buf);

    uint16_t port_host = 8080;
    uint16_t port_net  = htons(port_host);      /* 16 位: host -> network */
    printf("port host order       = 0x%04X  (%u)\n", port_host, port_host);
    printf("port network order    = 0x%04X  (%u)\n", port_net, port_net);
    printf("ntohs(htons(8080))    = %u\n", ntohs(port_net));

    uint32_t ip_host = 0x8002C2F2u;             /* 128.2.194.242 的数值 */
    uint32_t ip_net  = htonl(ip_host);          /* 32 位: host -> network */
    dump("htonl 之前 (主机序)", ip_host);
    dump("htonl 之后 (网络序)", ip_net);
    printf("主机序按字节打印 = %u.%u.%u.%u\n",
           (ip_host >> 24) & 0xFF, (ip_host >> 16) & 0xFF,
           (ip_host >> 8) & 0xFF,  ip_host & 0xFF);
    return 0;
}
```

【代码做什么？】 ① 取整数 `1` 看它的最低地址字节判断本机字节序；② 用 `inet_pton` 把点分十进制地址转成网络序二进制，并打印其内存字节；③ 演示 16 位 `htons` 与 32 位 `htonl` 的对称性（`ntohs(htons(x)) == x`）；④ 打印 `htonl` 前后同一数值在内存中的字节排列差异。

【底层机制透视】 `inet_pton` 内部会把地址按**大端**写入 `s_addr`，因此在小端机器上 `s_addr` 的数值变成 `0xF2C20280`——这不是"值错了"，而是"同一个 32 位里字节的排列被有意反转"。`htonl` 的实现是字节交换（`bswap`），在 x86-64 上编译器会直接生成一条 `bswap %eax`；在本来就是大端的机器上则是空操作。**这条规则对包头中的所有整数都成立**，所以端口号也必须转换。

【实测验证】（本机 GCC 12.2.0 / x86-64 Linux，真实输出）

```
$ gcc -g -Wall -std=c11 byteorder.c -o byteorder && ./byteorder
本机字节序: 小端 (little-endian)
----------------------------------------------------------------
inet_pton 之后 s_addr = 0xF2C20280   内存字节: 80 02 c2 f2
inet_ntop 还原        = 128.2.194.242
----------------------------------------------------------------
port host order       = 0x1F90  (8080)
port network order    = 0x901F  (36895)
ntohs(htons(8080))    = 8080
----------------------------------------------------------------
htonl 之前 (主机序) = 0x8002C2F2   内存字节: f2 c2 02 80
htonl 之后 (网络序) = 0xF2C20280   内存字节: 80 02 c2 f2
主机序按字节打印 = 128.2.194.242
```

注意 `htonl 之前` 的内存是 `f2 c2 02 80`（小端），`之后` 是 `80 02 c2 f2`（大端）——**同一串字节恰好完全反序**。

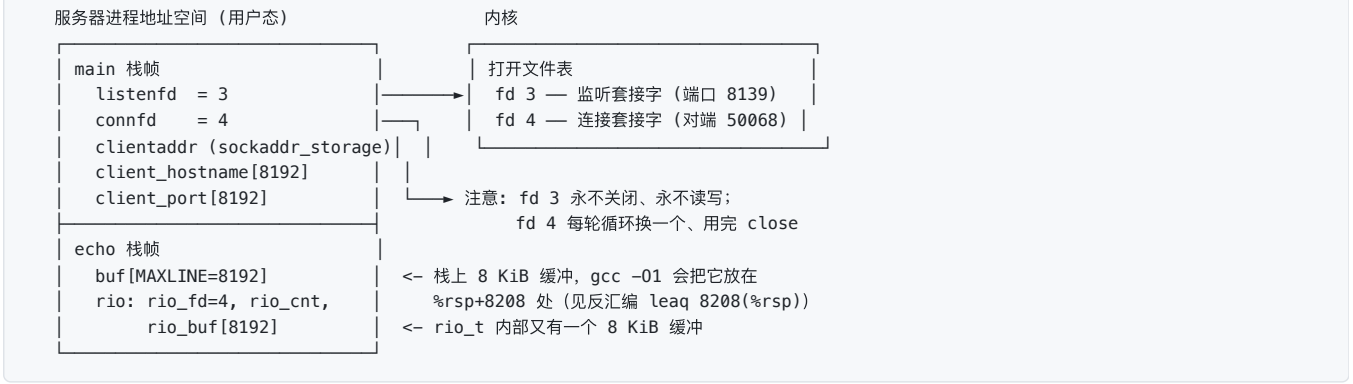
19.3.2 回显服务器 `echoserveri.c`

```c
/* echoserveri.c - 迭代式(iterative) echo 服务器, CS:APP3e 图 11-16 风格 */
#include "csapp.h"

void echo(int connfd);

int main(int argc, char **argv)
{
    int listenfd, connfd;
    socklen_t clientlen;
    struct sockaddr_storage clientaddr;   /* 足以容纳任意协议地址 */
    char client_hostname[MAXLINE], client_port[MAXLINE];

    if (argc != 2) {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(0);
    }

    listenfd = Open_listenfd(argv[1]);    /* 只创建一次的监听描述符 */
    printf("listening on port %s, listenfd = %d\n", argv[1], listenfd);
    fflush(stdout);

    while (1) {
        clientlen = sizeof(struct sockaddr_storage);
        connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
        Getnameinfo((SA *)&clientaddr, clientlen, client_hostname, MAXLINE,
                    client_port, MAXLINE, 0);
        printf("Connected to (%s, %s)  connfd = %d\n",
               client_hostname, client_port, connfd);
        fflush(stdout);
        echo(connfd);                     /* 每次连接一个连接描述符 */
        Close(connfd);
    }
    exit(0);
}

void echo(int connfd)
{
    size_t n;
    char buf[MAXLINE];
    rio_t rio;

    Rio_readinitb(&rio, connfd);
    while ((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
        printf("server received %d bytes\n", (int)n);
        fflush(stdout);
        Rio_writen(connfd, buf, n);       /* 原样回显 */
    }
}
```

【代码做什么？】 ① `Open_listenfd` 创建**唯一**的 `listenfd`；② 进入无限循环，`Accept` **阻塞等待**连接，返回一个新的 `connfd`；③ 用 `Getnameinfo` 把客户端地址反查成主机名/端口并打印；④ 调用 `echo` 用 RIO 逐行读取并原样回写，直到读到 **EOF**（`Rio_readlineb` 返回 0）；⑤ `Close(connfd)`，回到 ②。

【底层机制透视】 `Accept` 是从内核的**已完成连接队列**中取出一个已握手完成的连接——真正的三次握手由内核完成，`accept` 只是"取号"。`Rio_readinitb` 在用户态分配 8192 字节缓冲（`rio_t` 结构），于是 `Rio_readlineb` 一次 `read` 就能喂饱多行，避免**每字节一次系统调用**。`Rio_readlineb` 返回 0 不是错误，而是对端 `close` 触发的 **EOF**（收到 FIN）——这是"服务器只处理一个客户端、处理完继续接受下一个"的迭代式（iterative）服务器的关键。

【内存布局图解】（服务器主循环的栈帧与内核对象）



【与汇编 / 硬件的对应】 `gcc -S -O1` 生成的客户端主循环（真实片段，AT&T 语法）：

```
        call    Open_clientfd           # 返回 fd 在 %eax
        movl    %eax, %ebx              # clientfd 存入 callee-saved 的 %ebx
.L4:
        leaq    8208(%rsp), %rdi        # buf 位于栈顶上方 8208 字节处
        call    strlen
        movq    %rax, %rdx              # n = strlen(buf)
        leaq    8208(%rsp), %rsi
        movl    %ebx, %edi              # fd = clientfd
        call    Rio_writen
        movl    $8192, %edx
        leaq    8208(%rsp), %rsi
        movq    %rsp, %rdi              # rio_t 在栈顶
        call    Rio_readlineb
        movq    stdout(%rip), %rsi
        leaq    8208(%rsp), %rdi
        call    Fputs
.L3:
        movq    stdin(%rip), %rdx
        movl    $8192, %esi
        leaq    8208(%rsp), %rdi
        call    Fgets                   # 读不到行返回 NULL -> 退出循环
        testq   %rax, %rax
        jne     .L4
        movl    %ebx, %edi
        call    Close
```

注意 `clientfd` 存放在 `%ebx`——它必须跨 `call` 存活，所以编译器选了**被调用者保存（callee-saved）** 寄存器，这直接复用了 L5 学的调用约定。

19.3.3 回显客户端 `echoclient.c`

```c
/* echoclient.c — echo 客户端，CS:APP3e 图 11-17 风格 */
#include "csapp.h"

int main(int argc, char **argv)
{
    int clientfd;
    char *host, *port, buf[MAXLINE];
    rio_t rio;

    if (argc != 3) {
        fprintf(stderr, "usage: %s <host> <port>\n", argv[0]);
        exit(0);
    }
    host = argv[1];
    port = argv[2];

    clientfd = Open_clientfd(host, port);   /* getaddrinfo+socket+connect */
    Rio_readinitb(&rio, clientfd);
    printf("clientfd = %d\n", clientfd);
    fflush(stdout);

    while (Fgets(buf, MAXLINE, stdin) != NULL) {
        Rio_writen(clientfd, buf, strlen(buf));
        Rio_readlineb(&rio, buf, MAXLINE);
        Fputs(buf, stdout);
    }
    Close(clientfd);
    exit(0);
}
```

【代码做什么？】 ① `Open_clientfd` 一步完成 `getaddrinfo` + `socket` + `connect`；② 用 `Fgets` 从**终端**读一行；③ `Rio_writen` 发给服务器；④ `Rio_readlineb` 读回显；⑤ `Fputs` 打印到**终端**；⑥ 终端 EOF（`Ctrl-D`）时 `Fgets` 返回 `NULL`，关闭连接退出。

【底层机制透视】 客户端**不需要** **`bind`**：内核在 `connect` 时自动分配一个**临时端口**并绑定本地 IP。客户端也**不需要** **`listen`** / **`accept`** ——它是主动方。`Fgets` 与 `Rio_writen` 混用是安全的，因为前者操作 `FILE*`（终端）、后者操作文件描述符（套接字）；**危险的是在同一个套接字上混用** **`fgets`** **和** **`rio_readlineb`**（L18 讲的缓冲冲突）。

【实测验证：两个终端真实跑通】 编译并运行（注意：`csapp.c` 用了 `SA_RESTART`、`AI_ADDRCONFIG` 等，需 `-D_POSIX_C_SOURCE=200809L`，否则 `-std=c11` 下会因特性宏屏蔽而报 `struct addrinfo` 不完整）：

```
$ gcc -g -Wall -std=c11 -D_POSIX_C_SOURCE=200809L echoserveri.c csapp.c -o echoserveri -lpthread
$ gcc -g -Wall -std=c11 -D_POSIX_C_SOURCE=200809L echoclient.c  csapp.c -o echoclient  -lpthread
```

终端 A（终端 1）：启动服务器

```
$ ./echoserveri 8139
listening on port 8139, listenfd = 3
Connected to (localhost, 50068)  connfd = 4
server received 26 bytes
server received 17 bytes
Connected to (localhost, 50072)  connfd = 4
server received 29 bytes
```

终端 B（终端 2）用 `nc` 连接（第 1 次会话）

```
$ printf 'This line is being echoed\nThis one is, too\n' | nc 127.0.0.1 8139
This line is being echoed
This one is, too
```

终端 B 用自己写的 `echoclient` 连接（第 2 次会话，新连接）

```
$ printf 'This one is a new connection\n' | ./echoclient 127.0.0.1 8139
clientfd = 3
This one is a new connection
```

这与讲义上的会话示例逐行对应：服务器打印 `Connected to (...)` 与 `server received N bytes`，客户端回显同样的行。**注意** **`connfd`** **两次都是 4** ——第一个连接关闭后 fd 4 被释放，`accept` 立刻复用了它；这正是"`connfd` 是每连接消耗品"的直接证据。（括号里的客户端端口是内核分配的临时端口，每次运行都不同，此处为实测值。）

【实测验证：用 strace 看内核真实调用序列】

```
$ strace -f -e trace=socket,bind,listen,accept,close -o trace.log ./echoserveri 8139
（另一终端执行 printf 'hello\n' | nc 127.0.0.1 8139）

socket(AF_INET, SOCK_STREAM, IPPROTO_TCP)                       = 3
bind(3, {sa_family=AF_INET, sin_port=htons(8139),
         sin_addr=inet_addr("0.0.0.0")}, 16)                    = 0
listen(3, 1024)                                                 = 0
accept(3, {sa_family=AF_INET, sin_port=htons(56540),
           sin_addr=inet_addr("127.0.0.1")}, [128 => 16])        = 4
close(4)                                                        = 0
accept(3, 0x7ffd36d694d0, [128])                  = ? ERESTARTSYS
```

三件事被证实：① `open_listenfd` 内部确实就是 `socket` → `bind` → `listen`，且 `bind` 的端口显示为 `htons(8139)`（**网络序**）；② `accept` 返回 **4**，是从 `listenfd=3` 派生的**新**描述符；③ 处理完只 `close(4)`，`listenfd=3` 依然活着并回到 `accept` 阻塞（`ERESTARTSYS` 是信号中断后重启，说明它确实在等）。

【实测验证：`/proc` 里的 fd 表】 保持一个连接打开时观察服务器进程：

```
$ ls -l /proc/$(pgrep -x echoserveri)/fd | grep socket
lrwx------ 1 runguoli mzhang 64 ... 3 -> socket:[3421151768]   <- listenfd
lrwx------ 1 runguoli mzhang 64 ... 4 -> socket:[3421151769]   <- connfd
lrwx------ 1 runguoli mzhang 64 ... 5 -> socket:[3421170446]   <- (csapp.c 内部临时)
$ ss -tn | grep 8139
ESTAB  0  0  127.0.0.1:8139   127.0.0.1:59842
```

19.3.4 `open_clientfd` / `open_listenfd`（教材风格完整实现）

```c
/* csapp.c 摘录：open_clientfd / open_listenfd 的教材完整实现 */
#include "csapp.h"

/* open_clientfd - 连接到 <hostname, port> 的服务器，返回可读写的套接字描述符。
 * 可重入（reentrant）且协议无关（protocol-independent）。
 * 出错返回 -2（getaddrinfo 失败）或 -1（其他错误，errno 已设置）。 */
int open_clientfd(char *hostname, char *port) {
    int clientfd, rc;
    struct addrinfo hints, *listp, *p;

    /* Get a list of potential server addresses */
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_socktype = SOCK_STREAM;  /* Open a connection */
    hints.ai_flags = AI_NUMERICSERV;  /* ... using a numeric port arg. */
    hints.ai_flags |= AI_ADDRCONFIG;  /* Recommended for connections */
    if ((rc = getaddrinfo(hostname, port, &hints, &listp)) != 0) {
        fprintf(stderr, "getaddrinfo failed (%s:%s): %s\n",
                hostname, port, gai_strerror(rc));
        return -2;
    }

    /* Walk the list for one that we can successfully connect to */
    for (p = listp; p; p = p->ai_next) {
        /* Create a socket descriptor */
        if ((clientfd = socket(p->ai_family, p->ai_socktype,
                               p->ai_protocol)) < 0)
            continue; /* Socket failed, try the next */

        /* Connect to the server */
        if (connect(clientfd, p->ai_addr, p->ai_addrlen) != -1)
            break; /* Success */
        if (close(clientfd) < 0) { /* Connect failed, try another */
            fprintf(stderr, "open_clientfd: close failed: %s\n", strerror(errno));
            return -1;
        }
    }

    /* Clean up */
    freeaddrinfo(listp);
    if (!p) /* All connects failed */
        return -1;
    else    /* The last connect succeeded */
        return clientfd;
}

/* open_listenfd - 在 port 上打开并返回一个监听套接字。
 * 同样可重入且协议无关。出错返回 -2 或 -1。 */
int open_listenfd(char *port)
{
    struct addrinfo hints, *listp, *p;
    int listenfd, rc, optval=1;

    /* Get a list of potential server addresses */
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_socktype = SOCK_STREAM;             /* Accept connections */
    hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG; /* ... on any IP address */
    hints.ai_flags |= AI_NUMERICSERV;            /* ... using port number */
    if ((rc = getaddrinfo(NULL, port, &hints, &listp)) != 0) {
        fprintf(stderr, "getaddrinfo failed (port %s): %s\n",
                port, gai_strerror(rc));
        return -2;
    }

    /* Walk the list for one that we can bind to */
    for (p = listp; p; p = p->ai_next) {
        /* Create a socket descriptor */
        if ((listenfd = socket(p->ai_family, p->ai_socktype,
                               p->ai_protocol)) < 0)
            continue;  /* Socket failed, try the next */

        /* Eliminates "Address already in use" error from bind */
        setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR,
                   (const void *)&optval, sizeof(int));

        /* Bind the descriptor to the address */
```

```
        if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0)
            break; /* Success */
        if (close(listenfd) < 0) { /* Bind failed, try the next */
            fprintf(stderr, "open_listenfd close failed: %s\n", strerror(errno));
            return -1;
        }
    }

    /* Clean up */
    freeaddrinfo(listp);
    if (!p) /* No address worked */
        return -1;

    /* Make it a listening socket ready to accept connection requests */
    if (listen(listenfd, LISTENQ) < 0) {
        close(listenfd);
        return -1;
    }
    return listenfd;
}
```

【代码做什么？】 两个函数都是同一个"**三件套模板**"：① `memset(&hints,0,...)` 清零并设置 `ai_socktype`/`ai_flags`；② `getaddrinfo` 拿到**候选地址链表**；③ `for (p = listp; p; p = p->ai_next)` 逐个尝试，失败就 `continue` 试下一个。

【底层机制透视】 为什么要遍历链表？因为 `getaddrinfo` 对同一个名字可能返回**多条**记录（IPv4/IPv6、多个 IP），只有逐个尝试才知道哪条真正可达。我实测 `./gai localhost 8139` 的输出就是证据：

```
getaddrinfo("localhost", "8139") 返回的链表:
  [0] family=AF_INET   socktype=SOCK_STREAM protocol=6    addrlen=16  addr=127.0.0.1:8139
  [1] family=AF_INET   socktype=SOCK_STREAM protocol=6    addrlen=16  addr=127.0.0.1:8139
getnameinfo(第一个地址, flags=0) = "localhost"  (反向 DNS)
```

`AI_PASSIVE` 让 `getaddrinfo(NULL, port, ...)` 返回**通配地址**（`0.0.0.0` / `::`）供 `bind` 使用——这正是服务器能接受任意网卡上连接的原因。`SO_REUSEADDR` 解决的是 `TIME_WAIT` 问题：服务器重启时旧连接仍处于 `TIME_WAIT`，没有它 `bind` 会返回 `EADDRINUSE`。

关键区别：`open_clientfd` 的循环里调 `connect`，`open_listenfd` 的循环里调 `bind`，**两者都以 `freeaddrinfo(listp)` 收尾**（否则每次调用泄漏一条链表——长跑服务器上就是慢性内存泄漏）。另外 `open_listenfd` 把 `listen` 放在循环**外面**：只有 `bind` 成功后 `p` 才非空，循环结束意味着"绑定已完成"，此时才 `listen`。

19.3.5 HTTP 协议概览（为 L20 与 Proxy Lab 铺垫）

HTTP 请求格式：`<method> <uri> <version>\r\n<headers>\r\n\r\n`

```
    GET /hub/index.html HTTP/1.0\r\n     <- 请求行: 方法 URI 版本
    Host: www.cmu.edu\r\n                <- 头部: 名字: 值
    User-Agent: Mozilla/5.0 ...\r\n
    Connection: close\r\n
    Proxy-Connection: close\r\n
    \r\n                                 <- 空行: 请求结束 (CRLF, 即 \r\n)
```

HTTP 响应格式：`<version> <status-code> <status-message>\r\n<headers>\r\n\r\n<body>`

```
    HTTP/1.0 200 OK\r\n                  <- 状态行
    Server: Tiny Web Server\r\n
    Content-length: 120\r\n
    Content-type: text/html\r\n
    \r\n                                 <- 空行: 头部结束
    <html>...\r\n                        <- 响应体 (可为二进制!)
```

常见方法 `GET` / `POST`；常见状态码 `200 OK`、`301 Moved Permanently`、`403 Forbidden`、`404 Not Found`、`501 Not Implemented`；常见头部 `Host`、`User-Agent`、`Content-Length`、`Content-Type`、`Connection`、`Proxy-Connection`。**注意每一行都以 `\r\n` 结束，整个请求/头部以空行 `\r\n` 结束**——这是 Proxy Lab 解析逻辑的全部依据。

我用 Proxy Lab 附带的 CS:APP **Tiny 服务器**做了真实验证（`gcc -g -Wall -std=c11 -D_POSIX_C_SOURCE=200809L tiny.c csapp.c -o tiny`，`./tiny 8139`）：

```
$ printf 'GET /home.html HTTP/1.0\r\nHost: localhost\r\n\r\n' | nc 127.0.0.1 8139
HTTP/1.0 200 OK
Server: Tiny Web Server
Content-length: 120
Content-type: text/html

<html>
<head><title>test</title></head>
<body>
<img align="middle" src="godzilla.gif">
Dave O'Hallaron
</body>
</html>
```

19.4 实验关联：L7 Proxy Lab

Proxy Lab 要求写一个**缓存 Web 代理**：浏览器把请求发给代理，代理转发给真正的服务器，再把响应回送给浏览器。三个阶段——**Part I 顺序代理**（40 分）、**Part II 并发**（15 分）、**Part III 缓存**（15 分，共 70 分）。

本讲内容直接对应 Part I 的骨架：

```
    int listenfd = open_listenfd(argv[1]);   // 命令行指定的监听端口
    while (1) {
        int connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
        doit(connfd);                        // 读请求 -> 解析 -> 转发 -> 回写
        Close(connfd);
    }
```

`doit` 内部用的正是本讲的工具：`rio_readlineb` 读请求行与头部、`open_clientfd(host, port)` 连上游服务器、`rio_writen` 回写响应。

Part I 的三个关键坑（都源于本讲的知识点）：

1. **URI 有两种形式**。浏览器发给代理的是**绝对 URL**（`GET http://www.cmu.edu/hub/index.html HTTP/1.1`），代理转发给服务器时必须改写成**相对路径**（`GET /hub/index.html HTTP/1.0`）。我用真实 Tiny 服务器验证了这个差异：`$ printf 'GET http://localhost:8139/home.html HTTP/1.1\r\nHost: localhost:8139\r\n\r\n' | nc 127.0.0.1 8139 HTTP/1.0 404 Not found ... 404: Not found <p>Tiny couldn't find this file: .http://localhost:8139/home.html <- 把整个 URL 当文件名了! $ printf 'GET /home.html HTTP/1.0\r\nHost: localhost:8139\r\n\r\n' | nc 127.0.0.1 8139 HTTP/1.0 200 OK <- 正确` 所以解析器必须同时处理 `http://` 全 URL 与相对路径：从全 URL 里切出 **hostname**（可选**端口**，如 `http://www.cmu.edu:8080/...` 要连 8080 而不是默认 80）与 **path+query**。
2. **必须转发 `Host` 头**，并总是加上 `Connection: close` 与 `Proxy-Connection: close`（代理每个请求新开一条连接即可）。虽然 HTTP/1.0 规范不强制 `Host`，但虚拟主机服务器不认没有 `Host` 的请求。
3. **一律转发为 HTTP/1.0**，即使原请求是 HTTP/1.1。

其他必须记住的坑（writeup 明确列出）：

- `csapp.c` 的错误处理函数在代理里**不能照用**——服务器一旦开始 `accept` 就不该退出，`unix_error` 那种 `exit(0)` 会让代理崩掉。
- **必须忽略 `SIGPIPE`** 并优雅处理 `EPIPE`——客户端提前关闭时 `write` 会触发 `SIGPIPE`，默认动作是**终止进程**；`read` 也可能返回 `-1` 且 `errno == ECONNRESET`，同样不能因此退出。
- 网上内容是**二进制**（图片/视频），必须用 `rio_readnb` / `rio_writen` 这类**字节流**函数，绝不能用 `strlen` 或按行处理 body。
- Part II 推荐**每连接一线程**且置为 **detached**（避免内存泄漏）；`open_clientfd` / `open_listenfd` 基于 `getaddrinfo`，**是线程安全的**（对比：`gethostbyname` 不是）。
- Part III 缓存：`MAX_CACHE_SIZE = 1 MiB`、`MAX_OBJECT_SIZE = 100 KiB`，只统计**对象本身**的字节；淘汰近似 **LRU**（读和写都算"使用"）。同步要求**多个读者可同时读、写者独占——一把大互斥锁不可接受**（为 L22–L23 的读者-写者问题埋下伏笔）。

调试工具链：`./driver.sh`（自动评分）、CS:APP **Tiny** 服务器（driver 用它取页面）、`telnet` / `nc` 手动构造请求（`nc -l 12345` 当"假服务器"可看清代理**实际发出的请求**）、`curl -v --proxy http://localhost:15214 http://localhost:15213/home.html`（打印完整代理交互）、Firefox（测试前关掉浏览器自己的缓存）。端口用 `./port-for-user.pl <userID>` 生成，**不要随便挑**。

19.5 常见错误与调试技巧

- **忘记 `htons` / `htonl`**：服务器"以为"自己在 8080，内核实际把端口解释成字节翻转后的 `36895`（0x901F 反过来）。现象是**连接被拒绝（Connection refused）**，日志里 `ntohs(sin_port)` 打印出莫名其妙的数字。**调试**：`ss -tlnp | grep <pid>` 看内核真实监听端口；用 `inet_ntop` / `ntohs` 把地址"翻译"回可读形式打印；用 `strace -e trace=bind ./server <port>` 直接看 `bind` 的实参——本讲实测：`$ ./badport # sin_port = 8900 而漏了 htons 程序想绑 8900，内核实际分配的端口 = 50210 (0xC422) $ ./badserver & # 服务器以为自己在 8900 server: "I am on port 8900" $ nc 127.0.0.1 8900 Ncat: Connection refused. <- 客户端连"服务器以为的端口"失败 $ nc 127.0.0.1 50210 hi <- 连内核真实端口才成功`

- **`listenfd` / `connfd` 混用**：在监听套接字上 `read` 得到 `errno = 107 (ENOTCONN)`；在连接套接字上 `accept` 得到 `errno = 22 (EINVAL)`。**调试**：在每次 `socket` / `accept` 之后立刻 `printf("listenfd=%d connfd=%d\n", ...)`，并对照 `/proc/<pid>/fd` 与 `ss -tn`；`gdb -p <pid>` 后 `p listenfd` / `p connfd`。
- **忘记 `close(connfd)`**：长跑服务器会**描述符泄漏**，最终 `accept` 返回 `EMFILE`（Too many open files），服务器彻底停止服务。**调试**：`ls -l /proc/<pid>/fd | wc -l` 观察增长；`valgrind --track-fds=yes ./server <port>`；`lsof -p <pid> | grep socket | wc -l`。
- **`bind` 报 "Address already in use"**：上次运行留下的连接处于 `TIME_WAIT`。**调试**：`ss -tn state time-wait | grep <port>`；修复方式是 `setsockopt(..., SO_REUSEADDR, ...)` **且在 `bind` 之前**调用（`open_listenfd` 已经替你做了）。
- **在套接字上用标准 I/O（`fgets` / `fputs` / `fprintf`）**：`FILE*` 会在用户态缓冲，与直接 `read` 抢同一批字节，造成**丢行/粘包**；且 `fgets` 遇到二进制数据（含 `\0`）会截断。**调试**：`strace -e trace=read,write -f ./server` 观察 `read` 返回的字节数与你的 `printf` 是否对得上；统一改用 RIO（`rio_readlineb` 读文本行 / `rio_readnb` 读二进制）。
- **不检查 `rio_writen` 返回值 / 不处理 `SIGPIPE`**：客户端提前 `Ctrl-C`，服务器在下一次 `write` 时**直接被信号杀死**，日志停在半句话。**调试**：`signal(SIGPIPE, SIG_IGN)`；检查 `write` 返回 `EPIPE` / `ECONNRESET` 时只关闭该 `connfd` 并继续循环。
- **用 `struct sockaddr_in` 接 `accept` 的地址**：IPv6 客户端连接时会**写坏栈上内存**（`sockaddr_in6` 有 28 字节）。**调试**：改用 `struct sockaddr_storage` + `clientlen = sizeof(struct sockaddr_storage)`；用 `valgrind` 或 `-fsanitize=address` 编译能立刻抓到。
- **`getaddrinfo` 返回值用 `errno` 检查**：`getaddrinfo` 失败时返回的是**自己的错误码**（要用 `gai_strerror(rc)`），**不设置 `errno`**，用 `perror` 会打印出误导性的 "Success"。**调试**：`fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rc))`。

19.6 关键要点

- **网络是一种 I/O 设备**，而且是存储层次中最深、最慢的一层——应用程序员看到的是"套接字就是文件描述符"，于是 L18 的 Unix I/O 与 RIO 可以无缝复用。
- **分层是互联网可扩展的根本**：物理层 → 链路层（以太网/WiFi）→ 网络层（IP，不可靠数据报）→ 传输层（TCP 可靠字节流 / UDP 不可靠数据报）→ 应用层（HTTP/FTP）；每层只依赖下一层提供的抽象，封装时逐层加头。
- **主机 = 32 位 IPv4 地址（或 128 位 IPv6）+ 16 位端口**；`IP:port` 唯一标识一条连接的端点，内核靠**目的端口**把包分派给正确的服务进程。
- **包头里的一切整数都是大端（network byte order）**，x86-64 是小端，所以必须 `htonl` / `htons` / `ntohl` / `ntohs`——这是网络编程最常见的 bug 来源。
- **`listenfd` 只创建一次、永不读写；`connfd` 每个连接一个、用完即关**。这条区分是理解所有服务器代码（迭代式、并发式、预线程式）的钥匙。
- **`open_clientfd` / `open_listenfd` 把 `getaddrinfo` + `socket` + `connect` / `bind` / `listen` 的样板封装起来，并且因为基于 `getaddrinfo` 而线程安全**——Proxy Lab 与后续所有服务器代码都直接用它们。

19.7 思考题（带答案）

题 1（推演题：字节序） 某同学写了一个服务器，用 `struct sockaddr_in` 时写成 `sin_port = 8000;`（漏了 `htons`），其余代码完全正确。请推演：(a) 客户端执行 `nc 127.0.0.1 8000` 会发生什么？(b) 客户端要在哪个端口上才能连上？(c) 如果客户端用 `./echoclient 127.0.0.1 8000`，会打印什么错误？(d) 如果这台机器是**大端**机器，还会有 bug 吗？

答：(a) 连不上，`nc` 报 `Connection refused`——内核把 `8000 = 0x1F40` 当成**已是网络序**，在小端机上解释为 `ntohs(0x1F40) = 0x401F = 16415`。(b) **16415** 端口。(c) `connect` 返回 `-1` 且 `errno = ECONNREFUSED`；因 `close(clientfd)` 成功、`p` 走到 `NULL`，函数返回 **–1**；若用 `Open_clientfd` 包装则打印类似 `Open_clientfd error: Connection refused` 后 `exit(0)`。(d) **不会**——大端机上 `htons` 是恒等操作，主机序恰好等于网络序。**这正是该类 bug 在 x86 上必现、在别处却"看起来没事"的原因**。实测（`bind` 8900 漏 `htons`）：

```
程序想绑 8900，内核实际分配的端口 = 50210 (0xC422)
$ nc 127.0.0.1 8900   -> Ncat: Connection refused.
$ nc 127.0.0.1 50210  -> hi          (成功)
```

题 2（推演题：`listenfd` / `connfd`） 下面这段"服务器"能工作吗？如果 `accept` 返回的 `connfd` 在某一轮恰好也是 `3`（与 `listenfd` 相同），会发生什么？

```
int listenfd = open_listenfd(port);
while (1) {
    int connfd = accept(listenfd, NULL, NULL);
    rio_readinitb(&rio, listenfd);        /* ← 本行可疑 */
    while (rio_readlineb(&rio, buf, MAXLINE) != 0)
        rio_writen(listenfd, buf, strlen(buf));   /* ← 本行可疑 */
}
```

答：**不能工作**。① `rio_readinitb(&rio, listenfd)` 让 RIO 去读**监听套接字**，其上没有数据流，`read` 失败返回 `errno = 107 (ENOTCONN)`；实测：

```
[错1] read(listenfd, ...) = -1, errno = 107 (Transport endpoint is not connected)
[错2] accept(connfd, ...) = -1, errno = 22 (Invalid argument)
[对 ] read(connfd, ...)  = 5, errno = 0 (Success)
```

② `rio_writen(listenfd, ...)` 同样失败。③ 更严重的是 `accept` 的返回值被**完全忽略**：每个客户端都得不到服务，且 `connfd` **永不** `close`——每连接泄漏一个描述符，最终撞上 `EMFILE`。至于"`connfd` 恰好等于 `listenfd = 3`"：**同一进程内不可能**，fd 3 已被占用，`accept` 只能返回下一个空闲号（通常是 4）。真正危险的场景是**把 `listenfd` 传给了本该接收 `connfd` 的函数**——编译器不会报错（都是 `int`），这正是"类型系统帮不了你"的陷阱，只能靠命名规范与日志（每次 `accept` 后打印两个值）自查。

题 3（计算题：fd 与缓存上界） Proxy Lab 的最大并发连接数 $T = 64$，每个连接缓冲区按 `MAX_OBJECT_SIZE` 计。请计算：(a) 代理用于 Web 对象的**上界**内存是多少 MB? (b) 如果缓存里已有 N 个**恰好 10 KiB** 的对象，N 最大是多少? (c) 若每个连接还额外占 3 个文件描述符（clientfd、connfd、日志），加上默认 `ulimit -n = 1024`，还有多少余量?

答：(a) 上界
$= \mathbf{MAX_CACHE_SIZE} + T \times \mathbf{MAX_OBJECT_SIZE} = 1\ \mathbf{MiB} + 64 \times 100\ \mathbf{KiB} = 1024\ \mathbf{KiB} + 6400\ \mathbf{KiB} = 7424\ \mathbf{KiB} \approx \mathbf{7.25\ MiB}$
。(b) 缓存只统计对象本身字节，$\mathbf{1024/10 = 102.4}$，故 $N_{\max} = \mathbf{102}$（第 103 个超出 1 MiB，须先按近似 LRU 淘汰一个）。(c) $\mathbf{64 \times 3 = 192}$，$\mathbf{1024 - 192 = 832}$，余量 832 个；但 `connfd` 若忘记关闭会持续泄漏，再大的余量终将耗尽——**"余量"不是不 `close` 的理由**。

题 4（"直观但错误的想法"） 有同学认为："既然 `accept` 返回的 `connfd` 就是用来收发数据的，那我能不能**跳过 `accept` 这一步**，直接对 `listenfd` 用 `rio_readlineb`？反正 `listenfd` 也是个套接字描述符。" 这个想法错在哪？如果这个逻辑成立，服务器会有什么后果?

答：错在**把"端点工厂"当成了"端点"**。`listen` 之后 `listenfd` 的语义已变为"监听套接字"——内核只允许对它 `accept`。它背后是**两个握手队列**，而不是一条字节流，所以"用 `listenfd` 读数据"在内核里没有对应对象；实测证据即 `read(listenfd, ...) = -1, errno = 107 (ENOTCONN)`。若这个逻辑成立，后果是**多条连接的数据混在同一条"流"里无法区分**：你失去了 TCP "每条连接一个独立字节流"的核心抽象，任何"一个客户端一个处理单元"的并发模型（迭代 / 每连接一线程 / 预线程）都无法实现。`accept` 的本质是**从已完成队列认领一条连接、把它物化为新描述符**，使应用能按连接隔离状态——这是服务器设计的基石。

Lecture 20: 网络编程 II 与 Web 服务器 (Network Programming II and Web Servers)

> **讲义对应**：CMU 15–213 Lecture 20 — Network Programming (Part II)（素材：`F25-20-netprog2.txt`；习题课 `F25-rec11_slides.txt`） **教材对应**：CS:APP3e 第 11 章 11.4–11.6（HTTP 事务、HTTP 报文、Tiny Web 服务器、CGI 动态内容） **关联 Lab**：L7 Proxy Lab（迭代式 → 并发式 → 缓存式代理）

20.1 概述

上一讲（Lecture 19）把 socket 抽象成"文件描述符"，学会了 `open_listenfd` / `accept` / `open_clientfd` 这条建立 TCP 连接的通路。但连接只是管道，管道里流的是什么？本讲回答这个问题：流的是一种**文本协议**——超文本传输协议（HyperText Transfer Protocol, HTTP）。我们先把 HTTP 报文格式精确到字节（每个 `\r\n` 都算数），再剖析教材里那个 200 行却五脏俱全的 **Tiny Web 服务器**，最后引入 **代理（proxy）**——它同时是服务器又是客户端，是 Proxy Lab 的全部内容。这也是 Lecture 21–23（并发与同步）的直接动机：一旦代理要同时服务多个客户端并共享缓存，就必须面对线程与读者–写者锁。

20.2 核心概念与底层机制图解

20.2.1 Web 基础与 Web 内容（Web Basics and MIME Content）

- **定义与目的**：Web 客户端（浏览器）与 Web 服务器（Apache、nginx，或教材的 Tiny）通过 HTTP 通信。客户端与服务器建立 TCP 连接 → 客户端请求内容 → 服务器回送内容 → 最终关闭连接。当前版本已是 HTTP/2.0，但 **HTTP/1.1（RFC 2616）仍被广泛使用**，也是本课程与 Proxy Lab 的工作对象。
- **直观解释（"它是什么？"）**：把 Web 想成**点菜**。HTTP 就是"服务员与厨房的对话规范"：先说点哪道菜（请求行），再说特殊要求（请求头），最后用一声"就这些"（空行）结束点单；厨房先回一句"好嘞/没了"（状态行），附上菜品说明（响应头），再端上菜（响应体）。**内容（content）就是一串字节**，其身份由 MIME 类型标签说明。
- **MIME 类型（Multipurpose Internet Mail Extensions）**：HTTP 响应体是二进制安全的，服务器必须告诉客户端这串字节是什么。常见类型：`text/html`、`text/plain`、`image/gif`、`image/png`、`image/jpeg`；完整清单见 IANA 的 media-types 列表。**关键工程含义**：响应体可能是任意二进制，因此绝不能用 `strlen` / `strcpy` 处理它，必须按字节数（`Content-Length`）读写——这正是 Proxy Lab 反复强调"内容不全是 ASCII"的原因。
- **静态与动态内容**：静态内容（static content）是磁盘上的文件，请求标识"要哪个文件"；动态内容（dynamic content）是**服务器当场运行一个程序**产生的输出，请求标识"运行哪个可执行文件"。讲义约定：可执行程序放在 `cgi-bin` 目录下。

- **与机器码/硬件的对应**：HTTP 的"文本协议"只是约定，落到内核仍是 `write()` 写入 TCP 发送缓冲区、由网卡 DMA 发出；TCP 负责切分成报文段、重传丢失段、按序重组。HTTP 能只用文本，是因为 TCP 已提供**可靠字节流**。

20.2.2 URL 与代理的位置（URL and the Proxy）

- **URL（Uniform Resource Locator）结构**：`http://<host>:<port>/<path>?<query>`。以 `http://www.cmu.edu:80/index.html` 为例：客户端用**前缀** `http://www.cmu.edu:80` 推断协议（HTTP）、服务器位置（`www.cmu.edu`）、端口（80）；服务器用**后缀** `/index.html` 判断静态还是动态、定位文件系统上的文件，其中初始 `/` 表示内容的"家目录"。
- **最小后缀是** `/`：服务器会把它扩展为配置好的默认文件名（一般约定 `index.html`；教材 handout 的 Tiny 配置为 `home.html`）。
- **代理（proxy）的位置与作用**：代理是客户端与**源服务器（origin server）**之间的中间人：**对客户端它像服务器，对服务器它像客户端**。它的价值在于"请求与响应都从它手里过"：能缓存（caching）、记日志（logging）、匿名化（anonymization）、过滤（filtering）。经典场景：客户端 A 与 B 同处便宜的本地网，A 先取 `foo.html` 并缓存；B 再要同一文件时代理直接从内存返回，省掉一次昂贵的跨广域网往返。
- **底层机制图解**：见 20.2.4 代理数据流图。代理在 TCP 层面**终止**一条连接并**另起**一条，因此必须自己拼装下一跳的请求报文。

20.2.3 HTTP 事务与报文格式（HTTP Transaction and Message Format）

- **定义与目的**：一次 HTTP 事务 = 建立 TCP 连接 → 客户端发请求 → 服务器发响应 → 关闭连接。`Connection: close` 表示一次事务后关闭；**持久连接（persistent connection）** 则复用同一条 TCP 连接发多次请求/响应，省去反复三次握手。
- **直观解释（"它是什么？"）**：HTTP 报文像**电报**：一行一行写，每行以 `\r\n`（0x0D 0x0A）结束；一个**空行**（单独一个 `\r\n`）表示"正文开始"。这就是为什么 Proxy Lab 的解析器核心动作只有一句：**一直读行，直到读到内容恰为 `"\r\n"` 的那一行**。
- **底层机制图解（字节级报文格式，真实抓取的请求/响应）**：

```
 请求报文 = 请求行 + 零或多个请求头 + 空行 + 可选请求体
=== HTTP REQUEST, 89 bytes, real capture ===
 offset  hex bytes (16 per row)                          |ASCII           |  meaning
 0000    47 45 54 20 2f 69 6e 64 65 78 2e 68 74 6d 6c 20  |GET /index.html |  <-- REQUEST LINE
 0016    48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74 3a 20  |HTTP/1.1..Host: |
 0032    31 32 37 2e 30 2e 30 2e 31 3a 31 38 32 31 33 0d  |127.0.0.1:18213.|
 0048    0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 63 75 72  |.User-Agent: cur|
 0064    6c 2f 37 2e 37 36 2e 31 0d 0a 41 63 63 65 70 74  |l/7.76.1..Accept|
 0080    3a 20 2a 2f 2a 0d 0a 0d 0a                       |: */*....       |
 (headers end, body starts at offset 89)
 关键: 0d 0a 依次出现在偏移 24(请求行末), 47(Host 末), 72(User-Agent 末),
       85(Accept 末); 紧随其后的 87 处那个"孤立"的 0d 0a 就是终止全部头部的空行。
       请求行内 "GET /index.html HTTP/1.1" 三个字段以 0x20(空格) 分隔。

 响应报文 = 状态行 + 零或多个响应头 + 空行 + 响应体
=== HTTP RESPONSE, 228 bytes, real capture ===
 offset  hex bytes (16 per row)                          |ASCII           |  meaning
 0000    48 54 54 50 2f 31 2e 30 20 32 30 30 20 4f 4b 0d  |HTTP/1.0 200 OK.|  <-- STATUS LINE
 0016    0a 53 65 72 76 65 72 3a 20 54 69 6e 79 2d 4c 69  |.Server: Tiny-Li|
 0032    74 65 20 57 65 62 20 53 65 72 76 65 72 0d 0a 43  |te Web Server..C|
 0048    6f 6e 74 65 6e 74 2d 6c 65 6e 67 74 68 3a 20 31  |ontent-length: 1|
 0064    33 33 0d 0a 43 6f 6e 74 65 6e 74 2d 74 79 70 65  |33..Content-type|
 0080    3a 20 74 65 78 74 2f 68 74 6d 6c 0d 0a 0d 0a 3c  |: text/html....<|
 0096    68 74 6d 6c 3e 0a 3c 68 65 61 64 3e 3c 74 69 74  |html>.<head><tit|  <-- BODY begins
 0112    6c 65 3e 43 53 41 50 50 20 54 69 6e 79 20 54 65  |le>CSAPP Tiny Te|
 0128    73 74 3c 2f 74 69 74 6c 65 3e 3c 2f 68 65 61 64  |st</title></head|
 0144    3e 0a 3c 62 6f 64 79 3e 0a 3c 68 31 3e 48 65 6c  |>.<body>.<h1>Hel|
 0160    6c 6f 20 66 72 6f 6d 20 54 69 6e 79 2d 4c 69 74  |lo from Tiny-Lit|
 0176    65 3c 2f 68 31 3e 0a 3c 70 3e 4c 65 63 74 75 72  |e</h1>.<p>Lectur|
 0192    65 20 32 30 20 64 65 6d 6f 20 70 61 67 65 2e 3c  |e 20 demo page.<|
 0208    2f 70 3e 0a 3c 2f 62 6f 64 79 3e 0a 3c 2f 68 74  |/p>.</body>.</ht|
 0224    6d 6c 3e 0a                                      |ml>.            |
 (headers end, body starts at offset 95; 正文恰为 133 字节, 二进制安全)
```

- **请求行** `<method> <uri> <version>`：方法可为 `GET` / `POST` / `OPTIONS` / `HEAD` / `PUT` / `DELETE` / `TRACE`；`<uri>` **对代理是全 URL，对源服务器是 URL 后缀**（这是代理与服务器解析逻辑的第一道分水岭）；版本为 `HTTP/1.0` 或 `HTTP/1.1`。注意 URL 是 URI 的一个子类。
- **常见请求头**：`Host`（**HTTP/1.1 必需**，用于虚拟主机）、`User-Agent`（客户端标识）、`Connection`、`Proxy-Connection`（是否保持连接）、`Content-Length`、`Content-Type`、`Cookie`。
- **响应行** `<version> <status-code> <status-message>`，状态码分五类：

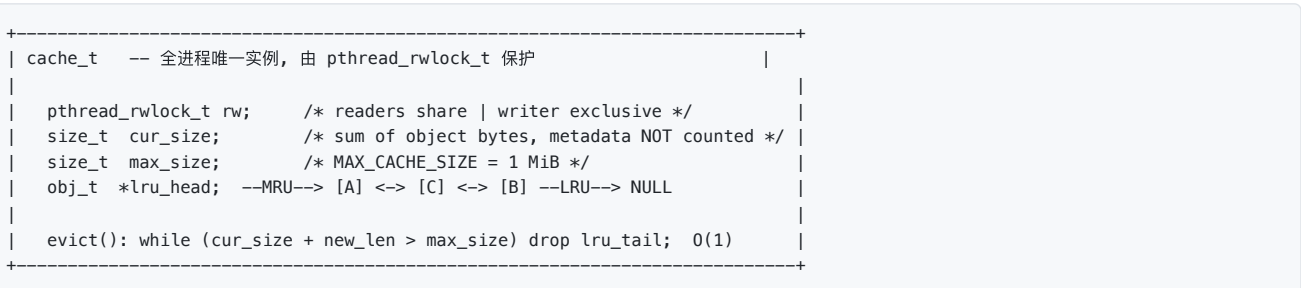
类别	含义	本课程常见实例
`1xx`	信息	继续处理
`2xx`	成功	`200 OK`

类别	含义	本课程常见实例
`3xx`	重定向	`301 Moved Permanently`、`302 Found`
`4xx`	客户端错误	`400 Bad Request`、`403 Forbidden`、`404 Not Found`
`5xx`	服务器错误	`500 Internal Server Error`、`501 Not Implemented`、`502 Bad Gateway`、`505 HTTP Version Not Supported`

常见响应头：`Content-Length`、`Content-Type`、`Date`、`Last-Modified`、`Server`、`Connection`。 – **与机器码/硬件的对应**：`rio_readlineb` 的实现是**逐字节调用 `rio_read` 直到见到 `\n`**，所以解析 HTTP 头是 O(报文长度) 次函数调用——这正是它比大块 `read` "慢"却仍被选用的原因：**正确性优先**，且头只有几十到几百字节；响应体才用 `rio_readn` / `mmap` 批量搬运。

20.2.4 代理的数据流与缓存结构（Proxy Data Flow and Cache Structure）

- **定义与目的**：代理必须同时扮演两个角色：**对客户端是服务器**（`open_listenfd` + `accept`），**对源服务器是客户端**（`open_clientfd` + 转发）。缓存则是"命中即免去一次网络往返"的性能机制。
- **直观解释（"它是什么？"）**：代理像**图书馆的借阅台**。你要某本书（对象），管理员先翻**近旁的书架（缓存）**：有就直接给你（快、免费），没有就打电话去总馆调（慢、贵），书到手再顺手在书架上放一本。书架面积有限（`MAX_CACHE_SIZE = 1 MiB`），太厚的书不收藏（`MAX_OBJECT_SIZE = 100 KiB`），书架满了就淘汰**最久没人翻的**那本（LRU）。
- **底层机制图解（数据流 + 缓存结构）**：



- **为什么必须是读者–写者锁而不是互斥锁**：缓存**读多写少**。用互斥锁（mutex）时，多个线程即使只读互不相同的对象也必须排队，吞吐被串行化；读者–写者锁允许**任意多个读者并发**，只有写者独占。Proxy Lab 明确规定"用一把大独占锁保护缓存是不可接受的方案"，并要求**多线程必须能同时读缓存**。
- **与机器码/硬件的对应**：`pthread_create` 最终调用 `clone()` 创建共享地址空间的执行流；读者–写者锁的读者计数是 `pthread_rwlock_t` 内的原子计数器，更新它编译成 `lock xadd`（带 `lock` 前缀的原子读–改–写），由缓存一致性协议（MESI）保证跨核可见。

20.3 代码示例与底层机制分析

20.3.1 简化版 Tiny Web 服务器（支持静态 GET）

以下是自包含的完整 Tiny 精简版（内联了 RIO 包，不需要 `csapp.c`），支持 `GET` 静态文件、目录追加默认页、`200` / `403` / `404` / `501`，并用 `mmap` 发送文件体。

```c
/* tiny_srv.c — 简化版 Tiny Web 服务器：gcc -g -Wall -Wextra -std=gnu11 tiny_srv.c -o tiny_srv
 * 运行：cd docroot && ./tiny_srv 18213      (以 docroot 为工作目录)
 * 验证：curl -v http://127.0.0.1:18213/index.html                              */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <unistd.h>
#include <errno.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <sys/socket.h>
#include <netdb.h>
#define MAXLINE 8192
#define MAXBUF  (MAXLINE * 2)
#define LISTENQ 1024
typedef struct sockaddr SA;

/* ---------- RIO: 缓冲读一行(rio_readlineb) + 无缓冲写满(rio_writen) ---------- */
typedef struct { int rio_fd; ssize_t rio_cnt; char *rio_bufptr; char rio_buf[MAXLINE]; } rio_t;

ssize_t rio_readn(int fd, void *usrbuf, size_t n) {          /* 读满 n 字节，仅 EOF 才短读 */
    size_t nleft = n; ssize_t nread; char *bufp = usrbuf;
    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) nread = 0; else return -1;
        } else if (nread == 0) break;                        /* EOF */
        nleft -= nread; bufp += nread;
    }
    return (ssize_t)(n - nleft);
}
static ssize_t rio_writen(int fd, const void *usrbuf, size_t n) { /* 绝不短写 */
    size_t nleft = n; ssize_t nwritten; const char *bufp = usrbuf;
    while (nleft > 0) {
        if ((nwritten = write(fd, bufp, nleft)) <= 0) {
            if (errno == EINTR) nwritten = 0; else return -1;
        }
        nleft -= nwritten; bufp += nwritten;
    }
    return (ssize_t)n;
}
static void rio_readinitb(rio_t *rp, int fd)
{ rp->rio_fd = fd; rp->rio_cnt = 0; rp->rio_bufptr = rp->rio_buf; }

static ssize_t rio_read(rio_t *rp, char *usrbuf, size_t n) {  /* 内部缓冲读 */
    int cnt;
    while (rp->rio_cnt <= 0) {                               /* 缓冲空则填满 */
        rp->rio_cnt = read(rp->rio_fd, rp->rio_buf, sizeof(rp->rio_buf));
        if (rp->rio_cnt < 0) { if (errno != EINTR) return -1; }
        else if (rp->rio_cnt == 0) return 0;                 /* EOF */
        else rp->rio_bufptr = rp->rio_buf;
    }
    cnt = (rp->rio_cnt < (ssize_t)n) ? (int)rp->rio_cnt : (int)n;
    memcpy(usrbuf, rp->rio_bufptr, cnt);
    rp->rio_bufptr += cnt; rp->rio_cnt -= cnt;
    return cnt;
}
static ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen) {
    ssize_t n, rc; char c, *bufp = usrbuf;
    for (n = 1; n < (ssize_t)maxlen; n++) {
        if ((rc = rio_read(rp, &c, 1)) == 1) { *bufp++ = c; if (c == '\n') break; }
        else if (rc == 0) { if (n == 1) return 0; else break; }
        else return -1;
    }
    *bufp = '\0';
    return n;
}
static void unix_error(const char *m){ fprintf(stderr,"%s: %s\n",m,strerror(errno)); exit(1); }

static int Open_listenfd(const char *port) {                 /* getaddrinfo 版: 协议无关+线程安全 */
    struct addrinfo hints, *listp, *p; int listenfd, optval = 1;
    memset(&hints, 0, sizeof(hints));
```

```
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG | AI_NUMERICSERV;
    if (getaddrinfo(NULL, port, &hints, &listp) != 0) unix_error("getaddrinfo");
    for (p = listp; p; p = p->ai_next) {
        if ((listenfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0) continue;
        setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(int));
        if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0) break;
        close(listenfd);
    }
    freeaddrinfo(listp);
    if (!p) return -1;
    if (listen(listenfd, LISTENQ) < 0) { close(listenfd); return -1; }
    return listenfd;
}

/* ------------------------------ Tiny 主体 ------------------------------ */
static void serve_static(int fd, const char *filename, int filesize) {
    int srcfd; char *srcp, filetype[MAXLINE], buf[MAXBUF];
    if      (strstr(filename, ".html")) strcpy(filetype, "text/html");
    else if (strstr(filename, ".gif"))  strcpy(filetype, "image/gif");
    else if (strstr(filename, ".png"))  strcpy(filetype, "image/png");
    else if (strstr(filename, ".jpg"))  strcpy(filetype, "image/jpeg");
    else                                strcpy(filetype, "text/plain");

    sprintf(buf, "HTTP/1.0 200 OK\r\n");               rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Server: Tiny-Lite Web Server\r\n");   rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Content-length: %d\r\n", filesize);   rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Content-type: %s\r\n\r\n", filetype); rio_writen(fd, buf, strlen(buf));

    srcfd = open(filename, O_RDONLY, 0);
    if (srcfd < 0) return;
    srcp = mmap(0, filesize, PROT_READ, MAP_PRIVATE, srcfd, 0);  /* 文件页按需装入 */
    close(srcfd);                                                /* 映射后即可关闭 fd */
    if (srcp != MAP_FAILED) { rio_writen(fd, srcp, filesize); munmap(srcp, filesize); }
}

static void clienterror(int fd, const char *cause, const char *errnum,
                        const char *shortmsg, const char *longmsg) {
    char buf[MAXLINE], body[MAXLINE];
    sprintf(body, "<html><title>Tiny Error</title>");
    sprintf(body + strlen(body), "<body bgcolor=\"ffffff\">\r\n");
    sprintf(body + strlen(body), "%s: %s\r\n", errnum, shortmsg);
    sprintf(body + strlen(body), "<p>%s: %s\r\n", longmsg, cause);
    sprintf(body + strlen(body), "<hr><em>The Tiny Web server</em>\r\n");
    sprintf(buf, "HTTP/1.0 %s %s\r\n", errnum, shortmsg);   rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Content-type: text/html\r\n");            rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Content-length: %d\r\n\r\n",(int)strlen(body)); rio_writen(fd, buf, strlen(buf));
    rio_writen(fd, body, strlen(body));
}

static void doit(int fd) {
    int is_static; struct stat sbuf;
    char buf[MAXLINE], method[MAXLINE], uri[MAXLINE], version[MAXLINE];
    char filename[MAXLINE]; rio_t rio;

    rio_readinitb(&rio, fd);
    if (!rio_readlineb(&rio, buf, MAXLINE)) return;            /* 客户端未发请求就关闭 */
    printf("[server] request line: %s", buf);
    sscanf(buf, "%s %s %s", method, uri, version);             /* 解析请求行 */
    if (strcasecmp(method, "GET")) {                           /* 非 GET -> 501 */
        clienterror(fd, method, "501", "Not Implemented", "Tiny does not implement this method");
        return;
    }
    while (strcmp(buf, "\r\n")) {                              /* 读头直到空行 */
        if (!rio_readlineb(&rio, buf, MAXLINE)) break;
        printf("[server] header: %s", buf);
    }
    is_static = (strstr(uri, "cgi-bin") == NULL);              /* cgi-bin -> 动态 */
    strcpy(filename, "."); strcat(filename, uri);              /* URI 映射到 "./"+uri */
    if (uri[strlen(uri) - 1] == '/') strcat(filename, "index.html");  /* 目录默认页 */
    if (!is_static) {
        clienterror(fd, filename, "501", "Not Implemented", "CGI not shown in this demo");
        return;
    }
    if (stat(filename, &sbuf) < 0) {                           /* 不存在 -> 404 */
```

```
        clienterror(fd, filename, "404", "Not found", "Tiny couldn't find this file");
        return;
    }
    if (!(S_ISREG(sbuf.st_mode)) || !(S_IRUSR & sbuf.st_mode)) { /* 非常规文件/不可读 -> 403 */
        clienterror(fd, filename, "403", "Forbidden", "Tiny couldn't read the file");
        return;
    }
    serve_static(fd, filename, (int)sbuf.st_size);
}

int main(int argc, char **argv) {
    int listenfd, connfd; char hostname[MAXLINE], port[MAXLINE];
    socklen_t clientlen; struct sockaddr_storage clientaddr;
    if (argc != 2) { fprintf(stderr, "usage: %s <port>\n", argv[0]); exit(1); }
    listenfd = Open_listenfd(argv[1]);
    if (listenfd < 0) unix_error("Open_listenfd");
    printf("[server] listening on port %s\n", argv[1]); fflush(stdout);
    while (1) {                                              /* 迭代式: 一次一个连接 */
        clientlen = sizeof(clientaddr);
        connfd = accept(listenfd, (SA *)&clientaddr, &clientlen);
        if (connfd < 0) continue;
        getnameinfo((SA *)&clientaddr, clientlen, hostname, MAXLINE, port, MAXLINE, 0);
        printf("[server] accepted connection from (%s, %s)\n", hostname, port); fflush(stdout);
        doit(connfd);
        close(connfd);
    }
    return 0;
}
```

【代码做什么？】

1. `main` 用 `Open_listenfd` 建立监听描述符（`getaddrinfo` → `socket` → `setsockopt(SO_REUSEADDR)` → `bind` → `listen`），随后进入无限循环。
2. `accept` 阻塞等待连接请求，返回**已连接描述符** `connfd`（每次 `accept` 都是新的 fd，只服务这一个客户端）。
3. `doit` 用 `rio_readlineb` 读**请求行**，`sscanf` 拆成 `method` / `uri` / `version`；非 `GET` 立即 `clienterror(…, "501", …)`。
4. 循环读头直到读到 `"\r\n"`——**这是 HTTP 解析的锚点**。注意 `rio_readlineb` 把 `\r\n` 一起返回，所以比较 `strcmp(buf, "\r\n") == 0` 成立。
5. `parse_uri` 的简化版：URI 含 `cgi-bin` 即动态；否则把 `"."+uri` 作为文件名，若 URI 以 `/` 结尾则追加默认页。
6. `stat` 判断文件存在性（失败 → `404`）、`S_ISREG` 判断是否普通文件、`S_IRUSR` 判断用户可读位（失败 → `403`）。
7. `serve_static` 先写 4 行响应头（**顺序无关，但空行必须在最后**），再 `open` + `mmap` + `rio_writen` 发文件体。

【底层机制透视】

- **RIO 两层设计的理由**：`rio_read` 一次 `read` 填满 8 KB 内部缓冲，之后 `rio_readlineb` 逐字节取用——**系统调用次数从 O(行数) 降到 O(缓冲区数)**，同时保留了"读到 `\n` 为止"的行语义。注意 `rio_readn`（无缓冲）与 `rio_readlineb`（有缓冲）**绝不能混用在同一个 fd 上**，否则缓冲区的预读字节会被无缓冲读"跳过"。
- **`mmap` 优于 `malloc` + `read` 之处**：`malloc` + `rio_readn` 需要"内核页缓存 → 用户缓冲区 → socket 缓冲区"两次拷贝；`mmap` 让文件页直接映射进进程地址空间，`rio_writen` 时**直接从页缓存拷到 socket 缓冲区**，省掉一次拷贝和一次显式分配。缺点是对小文件而言 `mmap` / `munmap` 的建页表开销可能反超收益——Tiny 服务的就是小页面，`mmap` 属于"优雅但不一定最快"；而 Proxy Lab 的响应体是流式的，只能用缓冲区累积。
- **`close(srcfd)` 早于 `munmap`**：映射建立后，文件描述符的使命就结束了；`mmap` 持有的是对 **inode** 的引用，即使文件被 `unlink`，映射依然有效直到 `munmap`。

【内存布局 / 数据结构图解】

```
进程虚拟地址空间（x86-64, 文件 133 字节, 真实地址来自 strace 输出）
0x7f7c85ed3000  +---------------------------+  <- mmap(NULL,133,PROT_READ,
                | page holding file content |     MAP_PRIVATE,fd,0)
                | "<html>...\n" (133 bytes) |     返回地址页对齐; 页内其余为填充
                +---------------------------+
0x7ffff...      +---------------------------+  高地址
                | argv / envp / 辅助向量      |
                +---------------------------+
                | rio_t rio                 |  rio_buf[8192] 在栈上
                |   .rio_cnt / .rio_bufptr  |  rio_cnt 记录缓冲区内剩余字节数
                | buf[MAXLINE]    = 8192 B  |  请求行/头部就地解析
                | filetype[MAXLINE]         |
                | buf[MAXBUF]     = 16384 B |  serve_static 拼响应头
                +---------------------------+
堆（低地址）      |（本程序几乎不用堆）         |
                +---------------------------+
                | .data / .bss              |  低地址
                +---------------------------+
注意: 栈向下增长, 且 rio_t 结构体中 8 KB 的 rio_buf 是"值"而非指针,
      因此每层函数调用都在栈上真实占用 8 KB 栈空间。
```

【与汇编 / 硬件的对应】（`gcc -O1 -S` 真实产物，`rio_writen` 的核心循环）

```
rio_writen:
        pushq   %r14 ; movq %rdx,%r13 ; testq %rdx,%rdx ; je .L23   # n==0 直接返回
        movl    %edi, %r12d      # fd   -> %r12d
        movq    %rsi, %rbp       # buf  -> %rbp   (循环指针)
        movq    %rdx, %rbx       # n    -> %rbx   (剩余字节)
.L26:
        movq    %rbx, %rdx       # 第 3 参数 nleft
        movq    %rbp, %rsi       # 第 2 参数 bufp
        movl    %r12d, %edi      # 第 1 参数 fd
        call    write            # 系统调用
        testq   %rax, %rax ; jg .L24
        call    __errno_location ; cmpl $4,(%rax) ; jne .L27    # errno==4(EINTR)? 重试 : 报错
.L24:
        addq    %rax, %rbp       # bufp += nwritten
        subq    %rax, %rbx       # nleft -= nwritten
        je      .L23             # nleft==0 退出
        jmp     .L26
```

这段汇编体现了三条契约：`%rdi/%rsi/%rdx` 依次是 `fd/buf/n`（x86-64 整数参数寄存器顺序），返回值在 `%rax`；`EINTR`（值为 4）必须重试；循环变量放在被调用者保存寄存器（`%rbx/%rbp/%r12`）中，因为 `write` 会破坏调用者保存寄存器。

【实测验证】（真实运行结果，工作目录 `docroot`，服务端口 18213）

```
$ curl -sv http://127.0.0.1:18213/index.html 2>&1
> GET /index.html HTTP/1.1
> Host: 127.0.0.1:18213
> User-Agent: curl/7.76.1
> Accept: */*
>
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: Tiny-Lite Web Server
< Content-length: 133
< Content-type: text/html
<
{ [133 bytes data]
<html>
<head><title>CSAPP Tiny Test</title></head>
<body>
<h1>Hello from Tiny-Lite</h1>
<p>Lecture 20 demo page.</p>
</body>
</html>
* Closing connection 0
```

服务器端 `strace` 记录的真实系统调用序列（验证了"先写头、再 `mmap`、最后写体"的顺序）：

```
read(4, "GET /index.html HTTP/1.1\r\nHost: "..., 8192) = 89
write(4, "HTTP/1.0 200 OK\r\n", 17) = 17
write(4, "Server: Tiny-Lite Web Server\r\n", 30) = 30
write(4, "Content-length: 133\r\n", 21) = 21
write(4, "Content-type: text/html\r\n\r\n", 27) = 27
openat(AT_FDCWD, "./index.html", O_RDONLY) = 6
mmap(NULL, 133, PROT_READ, MAP_PRIVATE, 6, 0) = 0x7f7c85ed3000
write(4, "<html>\n<head><title>CSAPP Tiny T"..., 133) = 133
munmap(0x7f7c85ed3000, 133)             = 0
```

`404` 分支同样实测通过（`curl` 得到 `HTTP/1.0 404 Not found`，`Content-length: 152`，错误页正文明确含 `./nope.html`）。**注意 `Content-Length` 与 `Content-type` 的顺序与讲义不同但完全合法**——头部顺序无语义，只有空行的位置有意义。

20.3.2 最小可用代理（转发 `GET http://host:port/path`）

```c
/* miniproxy.c — 最小可用代理: gcc -g -Wall -Wextra -std=gnu11 miniproxy.c -o miniproxy
 * 运行: ./miniproxy 18214
 * 验证: curl -x http://127.0.0.1:18214 http://127.0.0.1:18213/index.html */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#define MAXLINE 8192
#define LISTENQ 1024
typedef struct sockaddr SA;

/* 逐字节读到 '\r\n'：头部长度不可预知，这样最稳 */
static ssize_t read_until_crlf(int fd, char *buf, size_t max) {
    size_t i = 0; char c; ssize_t n;
    while (i + 1 < max) {
        if ((n = read(fd, &c, 1)) <= 0) return n == 0 ? (ssize_t)i : -1;
        buf[i++] = c;
        if (i >= 2 && buf[i-2] == '\r' && buf[i-1] == '\n') break;
    }
    buf[i] = '\0';
    return (ssize_t)i;
}
static ssize_t writen(int fd, const char *buf, size_t n) {
    size_t left = n; ssize_t w;
    while (left > 0) {
        if ((w = write(fd, buf, left)) <= 0) {
            if (errno == EINTR) continue;
            return -1;                      /* 常见: EPIPE —— 客户端提前断开 */
        }
        left -= (size_t)w; buf += w;
    }
    return (ssize_t)n;
}
static ssize_t readn(int fd, char *buf, size_t n) {   /* 严格读满 n 字节 */
    size_t left = n; ssize_t r;
    while (left > 0) {
        if ((r = read(fd, buf, left)) <= 0) {
            if (r < 0 && errno == EINTR) continue;
            return r < 0 ? -1 : (ssize_t)(n - left);
        }
        left -= (size_t)r; buf += r;
    }
    return (ssize_t)n;
}
static void clienterror(int fd, const char *num, const char *msg, const char *longmsg) {
    char buf[MAXLINE], body[MAXLINE];
    snprintf(body, sizeof(body),
             "<html><title>Proxy Error</title>\r\n%s: %s\r\n<p>%s\r\n", num, msg, longmsg);
    snprintf(buf, sizeof(buf), "HTTP/1.0 %s %s\r\n", num, msg);   writen(fd, buf, strlen(buf));
    snprintf(buf, sizeof(buf), "Content-type: text/html\r\n");    writen(fd, buf, strlen(buf));
    snprintf(buf, sizeof(buf), "Content-length: %d\r\n\r\n", (int)strlen(body));
    writen(fd, buf, strlen(buf));
    writen(fd, body, strlen(body));
}
static int open_clientfd(const char *host, const char *port) {
    struct addrinfo hints, *listp, *p; int fd;
    memset(&hints, 0, sizeof(hints));
    hints.ai_socktype = SOCK_STREAM; hints.ai_flags = AI_NUMERICSERV;
    if (getaddrinfo(host, port, &hints, &listp) != 0) return -1;
    for (p = listp; p; p = p->ai_next) {
        if ((fd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0) continue;
        if (connect(fd, p->ai_addr, p->ai_addrlen) == 0) break;
        close(fd);
    }
    freeaddrinfo(listp);
    return p ? fd : -1;
}
/* 从 "http://host[:port]/path" 拆出 host / port / path（含查询串） */
```

```
static int parse_url(const char *url, char *host, char *port, char *path) {
    const char *p = url, *slash, *colon; size_t hl, pl;
    if (strncasecmp(p, "http://", 7) != 0) return -1;
    p += 7; slash = strchr(p, '/');
    if (!slash) return -1;
    colon = memchr(p, ':', (size_t)(slash - p));
    if (colon) {
        hl = (size_t)(colon - p); pl = (size_t)(slash - colon - 1);
        if (hl == 0 || hl >= MAXLINE || pl == 0 || pl >= 16) return -1;
        memcpy(host, p, hl); host[hl] = '\0';
        memcpy(port, colon + 1, pl); port[pl] = '\0';
    } else {
        hl = (size_t)(slash - p);
        if (hl == 0 || hl >= MAXLINE) return -1;
        memcpy(host, p, hl); host[hl] = '\0';
        strcpy(port, "80");                             /* 缺省端口 80 */
    }
    snprintf(path, MAXLINE, "%s", slash);               /* 保留 ?query */
    return 0;
}
static void handle(int connfd) {
    char buf[MAXLINE], method[MAXLINE], url[MAXLINE], version[MAXLINE];
    char host[MAXLINE], port[16], path[MAXLINE], server_buf[MAXLINE * 4];
    int serverfd, bad_line = 0; ssize_t n;
    long content_length = -1; int saw_cl = 0;

    n = read_until_crlf(connfd, buf, MAXLINE);          /* 1. 请求行 */
    if (n <= 0) return;
    if (sscanf(buf, "%s %s %s", method, url, version) != 3) bad_line = 1;

    /* 2. 无论如何先把客户端请求头读干净(直到空行)!
     *    若带着未读数据 close(), 内核发送 RST 而非 FIN, 客户端读不到错误响应。 */
    while ((n = read_until_crlf(connfd, buf, MAXLINE)) > 0) {
        if (strcmp(buf, "\r\n") == 0) break;
        if (strncasecmp(buf, "Content-Length:", 15) == 0) {
            content_length = strtol(buf + 15, NULL, 10); saw_cl = 1;
        }
    }
    if (saw_cl && content_length > 0) {                 /* 有 body 一并读掉 */
        char sink[MAXLINE]; long left = content_length;
        while (left > 0) {
            size_t chunk = left < MAXLINE ? (size_t)left : MAXLINE;
            if (readn(connfd, sink, chunk) <= 0) break;
            left -= (long)chunk;
        }
    }
    if (bad_line) { clienterror(connfd,"400","Bad Request","Malformed request line"); return; }
    if (strcasecmp(method, "GET")) {
        clienterror(connfd, "501", "Not Implemented", "Proxy only implements GET"); return; }
    if (strcasecmp(version,"HTTP/1.0") && strcasecmp(version,"HTTP/1.1")) {
        clienterror(connfd,"505","HTTP Version Not Supported","Only HTTP/1.0 and 1.1"); return; }
    if (url[0] == '/') {   /* 相对路径 = 源服务器请求, Proxy Lab 要求 400 */
        clienterror(connfd,"400","Bad Request","Origin-server requests not supported"); return; }
    if (parse_url(url, host, port, path) < 0) {
        clienterror(connfd,"400","Bad Request","Proxy expects absolute URL"); return; }

    /* 3. 连接源服务器 (必须在读完请求头之后) */
    if ((serverfd = open_clientfd(host, port)) < 0) {
        clienterror(connfd,"502","Bad Gateway","Could not connect to origin server"); return; }
    /* 4. 转发: 降级为 HTTP/1.0, 补 Host/Connection/Proxy-Connection */
    snprintf(server_buf, sizeof(server_buf),
             "GET %s HTTP/1.0\r\nHost: %s\r\nUser-Agent: miniproxy/1.0\r\n"
             "Connection: close\r\nProxy-Connection: close\r\n\r\n", path, host);
    if (writen(serverfd, server_buf, strlen(server_buf)) < 0) { close(serverfd); return; }

    /* 5. 转发响应头, 同时记下 Content-Length */
    content_length = -1; saw_cl = 0;
    while ((n = read_until_crlf(serverfd, buf, MAXLINE)) > 0) {
        if (writen(connfd, buf, (size_t)n) < 0) { close(serverfd); return; }
        if (strncasecmp(buf, "Content-Length:", 15) == 0) {
            content_length = strtol(buf + 15, NULL, 10); saw_cl = 1;
        }
        if (strcmp(buf, "\r\n") == 0) break;
    }
    /* 6. 转发响应体: 有长度就"恰好"读这么多, 否则读到 EOF */
```

```
    if (saw_cl && content_length >= 0) {
        char body[MAXLINE]; long left = content_length;
        while (left > 0) {
            size_t chunk = left < MAXLINE ? (size_t)left : MAXLINE;
            ssize_t got = readn(serverfd, body, chunk);
            if (got <= 0) break;
            if (writen(connfd, body, (size_t)got) < 0) break;
            left -= got;
        }
        printf("[proxy] relayed %ld bytes of body (Content-Length exact)\n", content_length - left);
    } else {
        while ((n = read(serverfd, buf, MAXLINE)) > 0)
            if (writen(connfd, buf, (size_t)n) < 0) break;
    }
    close(serverfd);
}
int main(int argc, char **argv) {
    int listenfd, connfd, optval = 1;
    struct addrinfo hints, *listp, *p;
    struct sockaddr_storage clientaddr; socklen_t clientlen;
    if (argc != 2) { fprintf(stderr, "usage: %s <port>\n", argv[0]); exit(1); }
    signal(SIGPIPE, SIG_IGN);                           /* 写已关闭 socket 不要杀死进程 */
    memset(&hints, 0, sizeof(hints));
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG | AI_NUMERICSERV;
    if (getaddrinfo(NULL, argv[1], &hints, &listp) != 0) { perror("getaddrinfo"); exit(1); }
    for (p = listp; p; p = p->ai_next) {
        if ((listenfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0) continue;
        setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(int));
        if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0) break;
        close(listenfd);
    }
    freeaddrinfo(listp);
    if (!p || listen(listenfd, LISTENQ) < 0) { perror("listen"); exit(1); }
    printf("[proxy] listening on port %s\n", argv[1]); fflush(stdout);
    while (1) {                                         /* 迭代式：一次一个客户端 */
        clientlen = sizeof(clientaddr);
        if ((connfd = accept(listenfd, (SA *)&clientaddr, &clientlen)) < 0) continue;
        handle(connfd);
        close(connfd);
    }
    return 0;
}
```

【代码做什么？】

1. 第 1 步读请求行并 `sscanf` 拆字段，但**先不判断合法性**——因为必须先读完请求头（第 2 步）才能关闭连接。
2. 第 2 步把客户端头部读到空行为止，顺带记下 `Content-Length` 并把请求体一并读走。
3. 校验：方法非 `GET` → `501`；版本非 1.0/1.1 → `505`；URI 以 `/` 开头（源服务器请求）→ `400`；URL 拆分失败 → `400`。
4. `open_clientfd` 连源服务器，失败 → `502`。
5. 重新拼装请求：**路径只取后缀**、版本**降级为 HTTP/1.0**、补 `Host` / `User-Agent` / `Connection: close` / `Proxy-Connection: close`。
6. 边读边转发响应头，记录 `Content-Length`；再按长度**精确**读满响应体转发。

【底层机制透视】

- **为什么"先读干净再报错"**：TCP 的 `close()` 只在**接收缓冲区已空**时才发 FIN；若有未读数据，内核改发 **RST**。客户端收到 RST 会丢弃尚未交付的数据，于是"错误响应"根本读不到，`curl` 只报 `Connection reset by peer`——最容易被误判成逻辑 bug 的现象。
- **`Content-Length` 必须精确**：多读一个字节，那个字节就是**下一条响应的开头**，持久连接上会直接串包；少读则客户端永远等不到剩余字节而挂住。讲义与本 lab 都把它列为头号 bug 来源。因此这里用 `readn`（循环读满）而不是 `read`。
- **`SIGPIPE` 必须忽略**：向已关闭的 socket 写入会触发 `SIGPIPE`，默认动作是**终止进程**——对一个长跑服务器是灾难。忽略后 `write` 返回 -1/`EPIPE`，交给代码优雅处理。

【内存布局 / 数据结构图解】

```
代理进程一次事务的缓冲区占用（全部在栈上，默认 8 MB 栈足够）
+---------------------------+   连接源服务器前 connfd 必须已读空
| buf[MAXLINE]       8 KB   |   读请求行/头部/响应头（逐行复用同一块）
| method[MAXLINE]    8 KB   |   请求行三字段
| url[MAXLINE]       8 KB   |
| version[MAXLINE]   8 KB   |
| host[MAXLINE]      8 KB   |   parse_url 输出: "127.0.0.1"
| port[16]            16 B  |   parse_url 输出: "18213"（缺省 "80"）
| path[MAXLINE]      8 KB   |   "/some/path.html"（含 ?query）
| server_buf[4*MAXLINE] 32 KB |  重组的转发请求（Host/UA/Connection 必填）
| body[MAXLINE]      8 KB   |   Content-Length 精确转发的中转块
+---------------------------+
缓存版再加: obj_t* -> malloc(len) 以及全局 cache_t（堆/数据段，加锁保护）
```

【与汇编 / 硬件的对应】：`signal(SIGPIPE, SIG_IGN)` 修改内核中该进程的**信号处置表**；`accept` 在无连接时让进程进入可中断睡眠，由网卡中断唤醒完成三次握手；`dup2`（动态内容用）在 fd 表中做一次指针复制。

【实测验证】（真实运行结果：Tiny 在 18213，代理在 18214）

```
$ curl -sv -x http://127.0.0.1:18214 http://127.0.0.1:18213/index.html
*   Trying 127.0.0.1:18214...
* Connected to 127.0.0.1 (127.0.0.1) port 18214 (#0)
> GET http://127.0.0.1:18213/index.html HTTP/1.1     <- 客户端对代理说"全 URL"
> Host: 127.0.0.1:18213
> User-Agent: curl/7.76.1
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK                                    <- 代理把源服务器响应原样送回
< Server: Tiny-Lite Web Server
< Content-length: 133
< Content-type: text/html
<
{ [133 bytes data]
* Closing connection 0
```

用一个假源服务器（Python 原始 socket）抓下**代理实际发出的字节**，验证"降级为 HTTP/1.0 + 补 Host"：

```
b'GET /some/path.html HTTP/1.0\r\nHost: 127.0.0.1\r\nUser-Agent: miniproxy/1.0\r\n'
b'Connection: close\r\nProxy-Connection: close\r\n\r\n'
```

代理自身日志（证明按长度精确转发）：

```
[proxy] client request line: GET http://127.0.0.1:18213/hello.txt HTTP/1.1
[proxy] forwarding:
GET /hello.txt HTTP/1.0
Host: 127.0.0.1
User-Agent: miniproxy/1.0
Connection: close
Proxy-Connection: close

[proxy] relayed 29 bytes of body (Content-Length exact)
```

四条错误路径的原始响应字节（**全部实测通过**）：

```
b'HTTP/1.0 501 Not Implemented\r\n...\r\n501: Not Implemented\r\n<p>Proxy only implements GET\r\n'
b'HTTP/1.0 400 Bad Request\r\n...\r\n400: Bad Request\r\n<p>Origin-server requests not supported\r\n'
b'HTTP/1.0 505 HTTP Version Not Supported\r\n...\r\n505: HTTP Version Not Supported\r\n<p>Only HTTP/1.0 and 1.1\r\n'
b'HTTP/1.0 502 Bad Gateway\r\n...\r\n502: Bad Gateway\r\n<p>Could not connect to origin server\r\n'
```

20.3.3 服务动态内容：CGI 四问

讲义用四个问题概括 CGI（Common Gateway Interface）：**客户端如何把参数传给服务器？服务器如何把参数传给子进程？其他请求信息如何传递？服务器如何捕获子进程的输出？** 答案分别是：

1. **参数附加在 URI 上**：`http://add.com/cgi-bin/adder?15213&18213`；参数表以 `?` 开始、以 `&` 分隔，空格用 `+` 或 `%20` 表示。
2. **服务器通过环境变量传递**：`?` 之后的整串放进 `QUERY_STRING`（上例为 `"15213&18213"`）；另有 `SERVER_PORT`、`REQUEST_METHOD` 等。
3. **子进程用 `getenv("QUERY_STRING")` 读取**，`adder.c` 用 `strchr(buf,'&')` 切开参数再 `atoi`。

4. **服务器用 `dup2` 把子进程的 stdout 接到客户端 socket**：

```
void serve_dynamic(int fd, char *filename, char *cgiargs) {
    char buf[MAXLINE], *emptylist[] = { NULL };
    sprintf(buf, "HTTP/1.0 200 OK\r\n");        Rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Server: Tiny Web Server\r\n"); Rio_writen(fd, buf, strlen(buf));
    if (Fork() == 0) {                          /* 子进程 */
        setenv("QUERY_STRING", cgiargs, 1);     /* 第 3 参数 1 = 覆盖已有值 */
        Dup2(fd, STDOUT_FILENO);                /* stdout 重定向到客户端连接 */
        Execve(filename, emptylist, environ);   /* 执行 CGI 程序 */
    }
    Wait(NULL);                                 /* 父进程回收子进程，避免僵尸 */
}
```

关键点：只有 CGI 子进程知道内容的类型和长度，所以 **`Content-length` / `Content-type` 必须由 CGI 程序自己打印**（真实会话里 `Connection: close`、`Content-length: 117`、`Content-type: text/html` 都来自 CGI，而状态行与 `Server:` 来自 Tiny）。CGI 因"每请求都 `fork` + `exec`"代价高，已被 fastCGI、Apache 模块、Servlet 等取代。

20.4 实验关联：L7 Proxy Lab 分阶段实现路线图

Proxy Lab 占课程成绩 4%，评分 70 分 = BasicCorrectness 40 + Concurrency 15 + Cache 15，用 `./driver.sh` 自动评分。**先把迭代式做对，再加并发，最后加缓存**——顺序不可颠倒，否则 bug 定位成本会指数上升。

阶段	目标与关键动作	测试方法
阶段 0：跑通基线	编译 handout 的 `tiny`，用 `./free-port.sh` 或 `port-for-user.pl <userID>` 取端口；同时运行 Tiny 与代理	`curl -v --proxy http://localhost:<p+1> http://localhost:<p>/home.html`
阶段 I：迭代式代理	`open_listenfd` + `accept` + 解析全 URL 请求行；转发 `GET path HTTP/1.0`；**必须补 `Host` / `User-Agent` / `Connection: close` / `Proxy-Connection: close`**；按 `Content-Length` 精确转发响应体；实现 `400/501/505/502`	① `nc -l 12345` 当假服务器，用浏览器/curl 请求它，**肉眼检查代理发出的每一个头**；② `telnet <proxy> <port>` 手工发 `GET http://localhost:<p>/home.html HTTP/1.0`；③ `curl -x` 取普通页与二进制页（如 `godzilla.gif`）并 `cmp` 比对
阶段 II：并发代理	`pthread_create` 每连接一线程，并 `pthread_detach` 避免内存泄漏；**不共享可变状态**是关键（所有缓冲区放线程栈上）；用 `open_clientfd`（基于 `getaddrinfo`，线程安全）而非过时的 `gethostbyname`	① 同时开多个 `curl` 或用 `driver.sh` 的并发用例；② `valgrind --leak-check=full` 确认无线程栈/描述符泄漏；③ 用 `awk` 按线程 ID 拆分日志
阶段 III：缓存代理	全局有界缓存：`MAX_CACHE_SIZE = 1 MiB`、`MAX_OBJECT_SIZE = 100 KiB`（只计对象字节）；`pthread_rwlock_t` 保护；LRU 淘汰；每对象带时间戳；**边转发边累积**，超限即丢弃	① 同一对象请求两次，第二次日志须显示"命中且未连源服务器"；② 用 `nop-server.py` 与 `driver.sh` 跑 Cache 用例；③ 验证"读也更新 LRU"：反复读 A 再插入 C，被淘汰的应是 B

本 lab 的六大坑（讲义与 recitation 反复强调）：

1. **`Content-Length` 多读/少读**——第一号 bug。
2. **用标准 I/O（`fprintf` / `fgets`）操作 socket**——stdio 缓冲与 fd 语义冲突，必须用 RIO。
3. **照抄 `csapp.c` 的错误处理**——`unix_error` 会 `exit(1)`，而代理一旦开始服务就**不允许退出**，必须自己写"记录并继续"的错误处理。
4. **忽略 `SIGPIPE`** 与 `ECONNRESET`——进程会被信号杀死或误判为致命错误。
5. **假定内容是 ASCII**——图像/视频是二进制，只能用按字节数的 I/O。
6. **转发时把浏览器加的 `Connection: Keep-Alive` 原样透传**——必须改写为 `close`，否则服务器保持连接而客户端在等 FIN。

调试工具链：`telnet` / `nc` 手工构造报文、`curl -v` 看完整事务、`pxydrive.py` 复现并 trace 事务（`generate` / `serve` / `fetch` / `trace` / `check`）、`gdb` 多窗口调试（`./port-for-user.pl` 取独占端口）。

20.5 常见错误与调试技巧

- **`Content-Length` 与实际正文不符**：客户端挂住或读到垃圾。**调试**：`curl -v` 看 `Content-length`，再 `curl -s ... | wc -c` 数实际字节；`strace -e trace=write` 核对写入总量。
- **忘记用空行结束头部**：服务器一直等更多头部直到超时。**调试**：`nc -l 12345` 收下代理发出的原始字节，用 `xxd` 确认末尾是 `0d 0a 0d 0a`。
- **`rio_readn` 与 `rio_readlineb` 混用同一 fd**：预读进缓冲的字节被无缓冲读跳过，表现为"随机丢数据"。**调试**：`gdb -tui` 断在 `rio_read`，`p rp->rio_cnt`、`p rp->rio_bufptr`。
- **向已关闭 socket 写入导致进程消失**：`SIGPIPE`。**调试**：`gdb` 里 `info signal SIGPIPE`；或 `signal(SIGPIPE, SIG_IGN)` 后观察 `errno == EPIPE`。

- **带着未读数据 `close()`**：客户端报 `Connection reset by peer`。**调试**：`tcpdump -i lo -nn port <p> -X` 看是否发出 `RST`；`strace -f -e trace=close,read` 确认 `close` 前是否还有可读数据。
- **缓冲区溢出**：把 8 KB 的 `buf` 当无限容量拼接 URI。**调试**：`valgrind` 查堆越界；**栈越界须用 `gcc -fsanitize=address -g`**。
- **线程未 detach 导致内存泄漏**：长跑后 RSS 持续上涨。**调试**：`valgrind --leak-check=full --show-leak-kinds=all`。
- **缓存被单把互斥锁串行化**：并发吞吐不达标。**调试**：`perf stat -e cache-misses,context-switches` 对比 RW 锁与 mutex 的上下文切换次数。

20.6 关键要点

- **HTTP 的一切解析都围绕 `\r\n`**：每行以 `\r\n` 结束，一个**单独的空行**标志头部结束；记住这一条，Proxy Lab 解析器就有了一半。
- **代理的身份是双重的**：对客户端是服务器（`listenfd` / `accept`），对源服务器是客户端（`open_clientfd` / `connect`）；它**终止一条连接、另起一条**。
- **Tiny 的价值在于骨架而非性能**：`main` / `doit` / `clienterror` / `read_requesthdrs` / `parse_uri` / `serve_static` / `serve_dynamic` 这七个函数是 Web 服务器的最小完备集合。
- **正文长度必须精确**：`Content-Length` 多读一字节就会破坏下一条响应，这既是代理头号 bug 来源，也是持久连接能工作的前提。
- **缓存必须并发友好**：全局、有界（1 MiB / 100 KiB）、LRU、带时间戳，且**用读者-写者锁而非互斥锁**——读多写少，读者必须能并发。
- **长跑进程的错误处理原则**：忽略 `SIGPIPE`、容忍 `ECONNRESET`、绝不 `exit`、绝不泄漏 fd 与内存。

20.7 思考题（带答案）

题 1（推演题）：代理按 `Content-Length: 100` 读取响应体时多读了 50 字节，会发生什么？如果少读 50 字节呢？为什么"读多"比"读少"更危险？

> **答**：多读的 50 字节是**源服务器为下一次请求准备的下一条响应的开头**（在持久连接下）。它们会被代理当作本次响应体转发给客户端，于是客户端拿到的正文前面正确、尾部混入垃圾；更糟的是代理随后转发"下一条响应头"时会从被截断的中间位置开始，导致**整条连接上的报文全面错位**，且客户端可能把 `HTTP/1.0 200` 之类的文本当普通数据吞掉而不报错——错误隐蔽。 少读 50 字节时，客户端还在等剩余 50 字节（因为头里声明了 100），于是**挂住直到超时**，服务器也可能一直等待客户端读完而不关闭连接。现象明显、容易定位。 因此"读多"造成**静默的数据损坏**（对应 recitation 里 `proxy-corrupt` 的报错：`'F' (hex 0x46) != 'G' (hex 0x47)`），"读少"造成**可观测的挂起**。工程上宁可挂起也不要静默损坏。

题 2（推演题）：为什么 Proxy Lab 明确要求缓存用**读者-写者锁**而不是一把互斥锁？请从"缓存访问的读写比例"和"临界区长度"两个角度推演，并说明 LRU 与该锁之间为什么存在张力。

> **答**：缓存是典型的**读多写少**结构（同一热对象被反复请求，只有首次未命中才写）。用互斥锁时，任意两个线程即使只读互不相同的对象也必须排队，吞吐量退化为串行；而读者-写者锁允许**任意多个读者并发**，只有写者（插入/淘汰）独占，因此并发读的扩展性显著更好——这正是 lab 写明的"多个线程必须能同时读缓存"的要求。 第二个角度是**临界区长度**：读缓存只是查表并复制数据指针（微秒级），而"未命中走网络"这一段**绝不能持锁**——否则一个慢服务器会阻塞全部读者。所以正确做法是：读锁内完成查找与引用计数增加，立刻放锁，再在锁外发送数据。 **张力在于 LRU**：严格 LRU 要求"读操作也更新链表中对象的位置（把命中的对象移到 MRU 头）"，这**本质上是写操作**，会破坏"多读者并发"的前提。这正是 lab 允许"不必严格 LRU，只要接近"的原因：可以只让**插入**时更新顺序，读命中时仅更新时间戳（时间戳更新用原子写或放宽精度），从而让读者路径保持真正只读。这也是"不用严格 LRU 换取并发度"的经典工程权衡。

题 3（计算题）：Tiny 用 `mmap` 而非 `malloc` + `rio_readn` 发送一个 133 字节文件。若一次 `mmap` / `munmap` 的系统调用与缺页开销合计约 5 μs，而 `malloc` + `read`（两次拷贝）对 133 字节开销约 1.5 μs，那么文件要多大时 `mmap` 才开始占优？本 lab 的缓存对象上限（100 KiB）下应选哪种？

> **答**：设文件大小为 F 字节。`mmap` 路径的开销近似 $5\ \mu s + c \cdot F$（c 为单字节搬运代价），`malloc` + `read` 路径近似 $1.5\ \mu s + 2c \cdot F$（多一次拷贝）。令两者相等：$5 + cF = 1.5 + 2cF$，得 $cF = 3.5\ \mu s$。 关键在 c：内存带宽约 10 GB/s 时 $c \approx 0.1\ ns/B = 10^{-4}\ \mu s/B$，于是 $F \approx 3.5/10^{-4} = 35000$ 字节 ≈ 35 KB。也就是说 **35 KB 以上的文件 `mmap` 才划算**，小文件用 `mmap` 反而更慢——这与 Tiny 的行为一致：它服务的是几十字节到几 KB 的小页面，`mmap` 其实是"优雅但不一定最快"的选择。 在 Proxy Lab 中，响应体是**流式**的（必须先读来才能知道长度并决定是否入缓存），无法直接 `mmap` 源服务器 socket 上的数据，因此只能用"累积缓冲区"方案；`MAX_CACHE_SIZE + T × MAX_OBJECT_SIZE` 才是内存上界（T 为并发连接数）。

题 4（"直观但错误的想法"错在哪）：有同学认为"代理只要把 `connfd` 里读到的字节原样写进 `serverfd`、再把 `serverfd` 读到的字节原样写回 `connfd`，就实现了代理，根本不需要解析 HTTP"。这个想法错在哪？

> **答**：错在三处。① **必须改写请求行**：客户端发的是 `GET http://host:port/path HTTP/1.1`（全 URL），而源服务器期望的是 `GET /path HTTP/1.0`（后缀），照抄会让服务器返回 `400`；同时版本必须降级为 HTTP/1.0。② **必须增删头部**：`Host` 在 HTTP/1.0 下不强制但虚拟主机必需；浏览器的 `Connection: Keep-Alive` / `Proxy-Connection: Keep-Alive` 必须改写为 `close`，否则连接语义与代理"一次事务一条连接"的假设冲突；`Proxy-Connection` 是**代理专用头，不应转发给源服务器**。③ **必须先读完请求头再连接源服务器**：若边读边转发，代理会在解析出 `Host` 之前就试图 `connect`，而它此刻还不知道目标是谁。此外还有端口缺省补 80、`Content-Length` 精确转发、错误码映射（`502/400/501/505`）等——**代理的全部工作量恰恰集中在"解析并重写头部"这一层**，这也正是 Proxy Lab 要考的东西。

Lecture 21: 并发编程 (Concurrent Programming)

> **讲义对应**：CMU 15–213 Lecture 21 — Concurrent Programming（素材：`F25-21-concprog.txt`，2025 年 11 月 18 日） **教材对应**：CS:APP3e 第 12 章 12.1–12.3（基于进程的并发、基于 I/O 多路复用的并发、基于线程的并发） **关联 Lab**：**L7 Proxy Lab**（并发代理 + 缓存同步） / **L8 SFS Lab**（文件系统层面的并行与性能）

21.1 概述

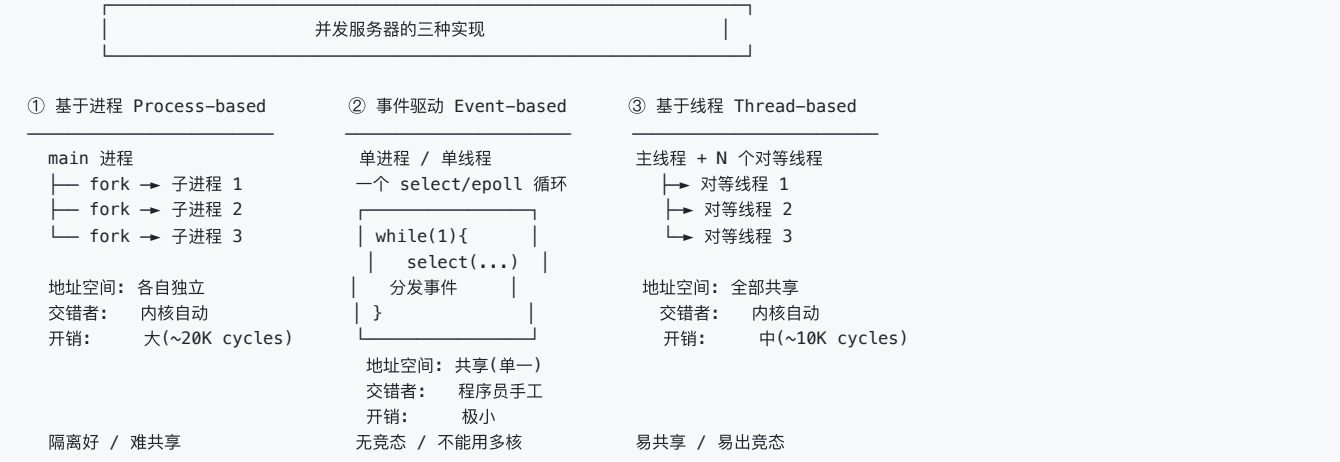
第 20 讲结束时的 echo 服务器是**迭代式（iterative）**的：`accept` 一个连接，服务到对端关闭，再回去 `accept` 下一个。它有一个致命缺陷——只要一个客户端连上后发呆不输入，服务器就永远阻塞在 `read` 上；第二个客户端虽然 `connect` 成功（TCP 监听队列替他排队），却在 `read` 上无限等待。本讲回答的就是：**如何让一个服务器同时服务多个客户端?**

课程给出的答案是三条路线：**基于进程（process-based）**、**基于 I/O 多路复用的事件驱动（event-based）**、**基于线程（thread-based）**。三者共享同一个抽象——**逻辑控制流（logical flow）**：进程由内核自动交错，事件驱动由程序员手工交错，线程则是两者的混合体。本讲同时抛出并发编程的三大经典病：**竞态（race）**、**死锁（deadlock）**、**活锁 / 饥饿（livelock / starvation）**，并用 `badcnt.c` 的"两亿次自增却数不到两亿"把竞态钉在案发现场。本讲为 Lecture 22–23 的同步机制（互斥锁、信号量、条件变量）铺路，也直接决定 Proxy Lab 并发与缓存两部分的成败。

21.2 核心概念与底层机制图解

21.2.1 三种并发模型（Three Approaches to Concurrency）

- **定义与目的**：并发服务器用多个**并发流**同时服务多个客户端。区别只在于两件事——**谁来交错这些流**（内核还是程序员），以及**这些流是否共享地址空间**。
- **直观解释**：进程法像"给每位客人单开一间包厢"（互不打扰，但开店关店成本高）；事件驱动法像"一位服务员端着托盘在大厅里轮转"（一人搞定所有桌，但忙不过来时全场都等）；线程法像"多个服务员共用同一个厨房和仓库"（协作快，可抢同一个锅就出事）。
- **底层机制图解**：



- **与机器码/硬件的对应**：三者的差别最终都落到**地址空间**上。`fork` 会复制页表并触发写时复制（copy-on-write），所以父子进程的 `connfd` 是同一张打开文件表项的两个引用（内核维护引用计数）；线程由 `clone(CLONE_VM|CLONE_FS|...)` 创建，**共享同一张页表**，只是各自持有一套寄存器上下文（PC、SP、通用寄存器、条件码）和一块栈。

21.2.2 基于进程的并发服务器（Process-based Servers，教材 12.1）

- **定义与目的**：父进程 `accept` 得到 `connfd`，`fork` 一个子进程去服务该客户端，父进程立刻回头 `accept`。每个客户端由**独立的子进程**处理。
- **直观解释**：这是"复制一家分店"——每家分店有自己的账本、自己的钥匙，谁也改不到别人；代价是开分店要真金白银（复制地址空间、页表）。

- **底层机制图解（描述符引用计数的变化）**：

```
时刻 T0：父进程 accept 返回 connfd=4
父进程描述符表：[0][1][2][3]=listenfd   [4]=connfd ──► refcnt(connfd)=1

时刻 T1：fork 之后（父子各持一份描述符表副本）
  父进程：[3]=listenfd ─┐          ┌─ [3]=listenfd :子进程
          [4]=connfd ──┼──────────┼─ [4]=connfd   :子进程
                       ▼          ▼
     两个描述符指向内核里同一张打开文件表项
     refcnt(listenfd)=2      refcnt(connfd)=2      ← 内核引用计数

时刻 T2：子进程 close(listenfd)，父进程 close(connfd)
     refcnt(listenfd)=1      refcnt(connfd)=1
  ⇒ 连接只有在 refcnt(connfd) 降到 0 时才真正关闭（发 FIN）
```

- **与机器码/硬件的对应**：`fork` 之后两个进程的 `connfd` 是**同一个整数 4**，但它们指向内核里**同一张打开文件表项**。父进程若不 `close(connfd)`，那张表项的引用计数永远是 2，客户端关闭连接后服务器也无法感知 EOF——连接被永久泄漏。子进程若不 `close(listenfd)`，则每个子进程都占着一个监听描述符，且监听套接字在最后一个引用关闭前不会释放。

21.2.3 I/O 多路复用与 `select`（I/O Multiplexing with `select`，教材 12.2）

- **定义与目的**：I/O 多路复用（I/O multiplexing） 让一个进程**同时等待多个描述符**上的事件。`select` 阻塞直到集合中至少一个描述符可读 / 可写 / 异常，然后告诉你是哪些。
- **直观解释**：`select` 是"总机接线员"：你不用为每通电话配一个专人，接线员盯着所有线路，哪条线亮了就转到对应分机。
- **接口速查**：

```
#include <sys/select.h>
int select(int nfds, fd_set *readset, fd_set *writeset,
           fd_set *exceptset, struct timeval *timeout);

void FD_ZERO(fd_set *set);          /* 清空集合：必须最先做 */
void FD_SET(int fd, fd_set *set);   /* 加入 fd */
void FD_CLR(int fd, fd_set *set);   /* 移除 fd */
int  FD_ISSET(int fd, fd_set *set); /* 判断 fd 是否就绪 */
```

- **底层机制图解（`fd_set` 位向量与内核改写）**：

```
用户态 fd_set（128 字节 = 1024 位）── FD_SETSIZE = 1024
位号：  0  1  2  3  4  5 ... 1023
      ┌──┬──┬──┬──┬──┬──┬──┬──┐
调用前│ 0│ 0│ 0│ 1│ 1│ 1│ 0 │ 0 │   FD_SET(3,&rs); FD_SET(4,&rs); FD_SET(5,&rs);
      └──┴──┴──┴──┴──┴──┴──┴──┘
        │       │  │  │
        │       └──┴──┴──► 只有 4、5 有输入挂起
        ▼
      ┌──┬──┬──┬──┬──┬──┬──┬──┐
返回后│ 0│ 0│ 0│ 0│ 1│ 1│ 0 │ 0 │   ← 内核"就地"清除了未就绪的位！
      └──┴──┴──┴──┴──┴──┴──┴──┘
                 ▲  ▲
                 │  └─ FD_ISSET(4,&rs) == 1  ⇒ 处理 connfd=4
                 └──── FD_ISSET(3,&rs) == 0  ⇒ listenfd 无事件

nfds = maxfd + 1（最大描述符 + 1，不是描述符个数！）
返回值 = 就绪描述符的个数（0 = 超时；-1 = 出错，errno=EINTR 表示被信号打断）
```

- **陷阱**：`select` 把 `readset` / `writeset` / `exceptset` 当作**输入输出参数**——传入时表示"我关心这些 fd"，返回时被改写成"这些 fd 就绪了"。所以下一次调用前必须 `FD_ZERO` + 重新 `FD_SET`。这一步漏掉的程序会表现得像"随机漏事件"，是最经典的 `select` bug。
- **与机器码/硬件的对应**：`select` 在 Linux 上的复杂度是 $O(n)$，内核要线性扫描位向量；`poll` 用 `struct pollfd` 数组去掉了 `FD_SETSIZE` 限制但仍需 $O(n)$；`epoll` 则在内核中维护红黑树 + 就绪链表，$O(1)$ 拿到就绪集合，适合十万级连接。三者语义相通，Proxy Lab 与教材用的是 `select`。

21.2.4 基于线程的并发服务器（Thread-based Servers，教材 12.3）

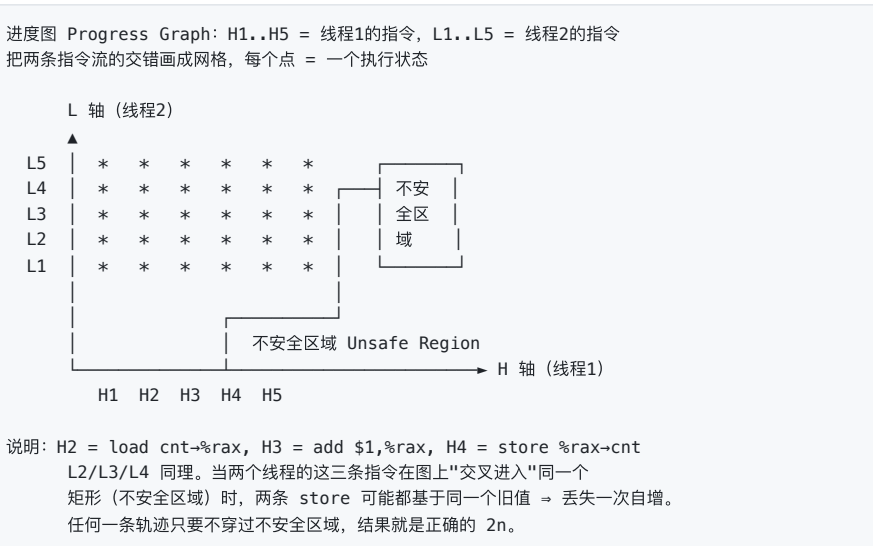
- **定义与目的**：主线程 `accept` 得到 `connfd`，把它**复制到堆上**，`pthread_create` 一个对等线程去服务该客户端。所有线程共享同一个地址空间（代码、全局数据、堆、内核上下文），各自只有独立的线程上下文与栈。

- **直观解释**：进程法像"每来一桌客人就新开一家餐厅"，线程法像"同一家餐厅里多雇一个服务员"——共用厨房（数据）所以上菜快，可同一个账本被两人同时改就乱套。
- **底层机制图解（进程地址空间 vs 多线程地址空间）**：

- **与机器码/硬件的对应**：多核机器上线程可以**真并行**（时间轴上重叠），单核上则是**时间片（time slicing）**模拟并行。讲义给出的 Linux 实测数量级：创建并回收一个进程约 20K 个周期，一个线程约 10K 个周期（或更少）——进程控制的开销大约是线程的两倍。

21.2.5 共享变量与竞态（Shared Variables and Races）

- **定义与目的**：若一个变量被多个线程引用，且**至少有一个线程在写它**，则它是**共享变量（shared variable）**。若两个线程对同一共享变量的访问顺序会影响结果，就发生了**竞态（race）**。
- **直观解释**：合租公寓的公用洗衣机旁挂着一块记账板——两人同时去写"洗衣次数"，谁最后写谁的数字留下，另一次就凭空消失了。
- **底层机制图解（进度图与不安全区域）**：



- **与机器码/硬件的对应**：`cnt++` 不是一条指令！即使 `cnt` 只占 8 字节且 8 字节对齐（x86-64 保证单条对齐的 8 字节访存本身是原子的），**读—改—写三步之间随时可能被切走**。教材 12.4 节用 `objdump` 展示的正是这个事实（见 21.3.1 的真实反汇编）。

21.3 代码示例与底层机制分析

21.3.1 示例 (a)：竞态的真实面孔 —— `badcnt.c`

代码 (C)：

```c
/*
 * badcnt.c - CS:APP3e Figure 12.4/12.5 风格的竞态演示
 * 两个线程各对全局共享变量 cnt 自增 niters 次。
 * 正确结果是 2*niters，但因为没有同步，实际结果往往远小于它。
 * 编译: gcc -g -Wall -O1 -std=c11 badcnt.c -o badcnt -lpthread
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

volatile long cnt = 0;   /* 全局共享计数器：被两个线程同时读写 */

void *thread(void *vargp)
{
    long i, niters = *((long *)vargp);
    for (i = 0; i < niters; i++)
        cnt++;           /* 非原子！编译成 load/add/store 三条指令 */
    return NULL;
}

int main(int argc, char **argv)
{
    long niters;
    pthread_t tid1, tid2;

    if (argc != 2) { fprintf(stderr, "usage: %s <niters>\n", argv[0]); exit(1); }
    niters = atol(argv[1]);

    pthread_create(&tid1, NULL, thread, &niters);
    pthread_create(&tid2, NULL, thread, &niters);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    if (cnt != 2 * niters)
        printf("BOOM! cnt=%ld (期望 %ld, 丢失 %ld, 完成率 %.2f%%)\n",
               cnt, 2 * niters, 2 * niters - cnt,
               100.0 * cnt / (2.0 * niters));
    else
        printf("OK!   cnt=%ld\n", cnt);
    return 0;
}
```

【代码做什么？】

1. 主线程把 `niters` 的**地址**（不是值）传给两个线程——两个线程读同一个 `long`，各自循环 `niters` 次做 `cnt++`。
2. 两个 `pthread_create` 之后，主线程 `pthread_join` 等两个线程都结束，此时理论上 `cnt == 2*niters`。
3. 若不等，打印实际值、丢失量与完成率。

【底层机制透视】：`volatile` 只禁止编译器把 `cnt` 缓存在寄存器里（讲义用它把"编译器优化掉整个循环"这种干扰排除掉），它**完全不能保证原子性**。真正的祸根是 `cnt++` 在机器层是三条指令，而线程切换可能发生在任意两条之间。

【与汇编 / 硬件的对应】（本机 GCC 12.2.0 `-O1` 的真实 `objdump -d -M intel` 输出）：

```
0000000000401176 <thread>:
  401183:  48 8b 05 e6 2e 00 00   mov  rax,QWORD PTR [rip+0x2ee6]   # 404070 <cnt>
  40118a:  48 83 c0 01            add  rax,0x1                     # 在寄存器里 +1
  40118e:  48 89 05 db 2e 00 00   mov  QWORD PTR [rip+0x2edb],rax  # 404070 <cnt> 写回
  401195:  48 83 c2 01            add  rdx,0x1                     # i++
  401199:  48 39 d1               cmp  rcx,rdx
  40119c:  75 e5                  jne  401183 <thread+0xd>
```

（`-O0` 与 `-O2` 生成的循环体在语义上完全一致，都含 `mov cnt→%rax` / `add` / `mov %rax→cnt` 这一组非原子的读—改—写。若有某台机器把 `cnt++` 编译成单条 `addq $1, cnt(%rip)`，那么内存总线会保证该条指令的原子性，结果就会是 2n——**同一源程序在不同平台上后果不同**。）

【实测验证】（本机：128 核 x86-64，Linux 5.14；`./badcnt 100000000`，期望值 200000000）：

```
run1:  BOOM! cnt=107604110  (完成率 53.80%)
run2:  BOOM! cnt=118579515  (完成率 59.29%)
run3:  BOOM! cnt=133676070  (完成率 66.84%)
run4:  BOOM! cnt=117806990  (完成率 58.90%)
run5:  BOOM! cnt=104715193  (完成率 52.36%)
run6:  BOOM! cnt=106804770  (完成率 53.40%)
run7:  BOOM! cnt=100219366  (完成率 50.11%)
run8:  BOOM! cnt=106027442  (完成率 53.01%)
run9:  BOOM! cnt=106138139  (完成率 53.07%)
run10: BOOM! cnt=117326988  (完成率 58.66%)
run11: BOOM! cnt=107518336  (完成率 53.76%)
run12: BOOM! cnt=113237432  (完成率 56.62%)
```

再连续跑 20 次，把 `cnt` 排序后得到**真实的波动区间**：

```
90981898  95290435  100131967 100990322 101797854 102568726 102974309 104123542
106527522 106729898 106785066 112503470 113448002 116027908 120644332 123291741
125099447 155505430 172539375 175343844

  最小值 = 90,981,898    最大值 = 175,343,844    均值 ≈ 116,665,254
  期望值 = 200,000,000   最坏完成率 ≈ 45.5%      最好完成率 ≈ 87.7%
```

三次运行中**没有一次**得到 200000000。作为对照，`taskset -c 0 ./badcnt 100000000`（**强制两个线程跑在同一个核上**，靠时间片切换制造交错）同样出错，且其中一次恰好得到 `cnt=100000000`——丢了整整一半，即两个线程的每一次自增都被对方覆盖了一次。这说明竞态**不需要多核**，单核时间片切换同样致命：

```
BOOM! cnt=100000000 (期望 200000000, 丢失 100000000, 完成率 50.00%)
BOOM! cnt=139271793 (期望 200000000, 丢失 60728207, 完成率 69.64%)
BOOM! cnt=104832708 (期望 200000000, 丢失 95167292, 完成率 52.42%)
```

21.3.2 示例 (b)：基于线程的并发 echo 服务器 `echoservert.c`

代码 (C)（为可独立编译，把教材 `csapp.c` 里的 `open_listenfd` 与 `rio_readlineb` 换了最简等价实现）：

```c
#define _GNU_SOURCE
/*
 * echoservert.c - 基于线程的并发 echo 服务器（CS:APP3e 12.3 的独立可编译版本）
 * 每来一个连接就 malloc 一个 connfd 副本 -> pthread_create -> 线程内 detach + free + echo + close
 * 用法: ./echoservert <port>
 * 编译: gcc -g -Wall -std=c11 echoservert.c -o echoservert -lpthread
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <pthread.h>
#include <netdb.h>
#include <sys/socket.h>
#include <sys/types.h>

#define LISTENQ 1024
#define MAXLINE 8192

/* ---- 讲义 csapp.c 中 open_listenfd 的最简等价实现 ---- */
static int open_listenfd(const char *port)
{
    struct addrinfo hints, *listp, *p;
    int listenfd, optval = 1;

    memset(&hints, 0, sizeof(hints));
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags    = AI_PASSIVE | AI_ADDRCONFIG | AI_NUMERICSERV;
    if (getaddrinfo(NULL, port, &hints, &listp) != 0) return -1;

    for (p = listp; p; p = p->ai_next) {
        if ((listenfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0)
            continue;
        setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(int));
        if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0) break;
        close(listenfd);
    }
    freeaddrinfo(listp);
    if (!p) return -1;
    if (listen(listenfd, LISTENQ) < 0) { close(listenfd); return -1; }
    return listenfd;
}

/* ---- rio_readlineb 的等价实现：读满一行（含 '\n'）才返回 ---- */
static ssize_t rio_readlineb_fd(int fd, char *buf, size_t maxlen)
{
    size_t i = 0;
    char c;
    ssize_t rc;
    while (i < maxlen - 1) {
        if ((rc = read(fd, &c, 1)) == 1) {
            buf[i++] = c;
            if (c == '\n') break;          /* 行结束 */
        } else if (rc == 0) {              /* EOF */
            if (i == 0) return 0;
            break;
        } else {
            if (errno == EINTR) continue;
            return -1;
        }
    }
    buf[i] = '\0';
    return (ssize_t)i;
}

/* ---- 讲义里的 echo(): 逐行读、原样写回，直到 EOF ---- */
static void echo(int connfd)
{
    size_t n;
    char buf[MAXLINE];
    while ((n = rio_readlineb_fd(connfd, buf, MAXLINE)) != 0)
        if (write(connfd, buf, n) != (ssize_t)n) break;
}
```

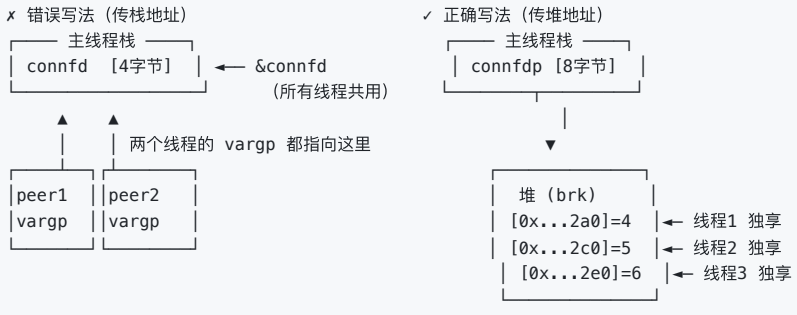
```c
/* ---- 线程例程：vargp 指向主线程 malloc 出来的 connfd 副本 ---- */
static void *thread(void *vargp)
{
    int connfd = *((int *)vargp);      /* 先把值拷到自己栈上，再释放堆内存 */
    pthread_detach(pthread_self());    /* 分离态：终止时内核自动回收，无僵尸线程 */
    free(vargp);                       /* 立刻释放参数：生产者-消费者模型 */
    printf("[thread %lu] serving fd=%d\n", (unsigned long)pthread_self(), connfd);
    fflush(stdout);
    echo(connfd);
    close(connfd);                     /* 重要！否则 fd 泄漏 */
    return NULL;
}

int main(int argc, char **argv)
{
    int listenfd, *connfdp;
    socklen_t clientlen;
    struct sockaddr_storage clientaddr;
    pthread_t tid;

    if (argc != 2) { fprintf(stderr, "usage: %s <port>\n", argv[0]); exit(1); }
    listenfd = open_listenfd(argv[1]);
    if (listenfd < 0) { fprintf(stderr, "open_listenfd failed\n"); exit(1); }

    while (1) {
        clientlen = sizeof(struct sockaddr_storage);
        connfdp = malloc(sizeof(int));                 /* 堆上分配，不是栈！ */
        *connfdp = accept(listenfd, (struct sockaddr *)&clientaddr, &clientlen);
        if (*connfdp < 0) { free(connfdp); continue; }
        pthread_create(&tid, NULL, thread, connfdp);
    }
    return 0;
}
```

【代码做什么？】

1. 主线程在 `accept` **之前**就 `malloc(sizeof(int))`，把返回的 `connfd` **写进堆**，再把**堆地址**交给新线程。
2. 新线程第一件事是从 `vargp` 读出 `connfd` 到自己栈上的局部变量，然后 `pthread_detach(pthread_self())` 转入分离态。
3. `free(vargp)` 释放那块堆内存（**生产者–消费者模型**：主线程分配、线程例程释放）。
4. `echo(connfd)` 逐行读、逐行回写，直到对端关闭；最后 `close(connfd)`。

【底层机制透视】：为什么必须在堆上分配？如果写成 `Pthread_create(&tid, NULL, thread, &connfd)`，所有线程拿到的 `vargp` 都指向**主线程栈上同一个 `connfd` 变量**。主线程 `accept` 返回 5 以后马上又 `accept` 返回 6 并覆盖同一地址，第一个线程醒来时读到的就是 6——它服务的其实是别人的连接。堆上每次 `malloc` 返回**不同地址**，从根本上杜绝了共享。这就是讲义所说的"unintended sharing"。

【内存布局 / 数据结构图解】：



【实测验证】：三个客户端并发连接（每个客户端发一行、收一行，中间 `usleep(300000)` 故意停顿 0.3 秒，共 4 轮）：

```
=== 服务器输出 (server2.log) ===
[thread 139858408371776] serving fd=4
[thread 139858399979072] serving fd=5
[thread 139858391586368] serving fd=6

=== 三个客户端并行总耗时: 1.210508545 秒（若服务器是迭代式的，串行应约 3.6 秒）===
[A] recv: [A] line 0     [B] recv: [B] line 0     [C] recv: [C] line 0
[A] recv: [A] line 1     [B] recv: [B] line 1     [C] recv: [C] line 1
[A] recv: [A] line 2     [B] recv: [B] line 2     [C] recv: [C] line 2
[A] recv: [A] line 3     [B] recv: [B] line 3     [C] recv: [C] line 3
[A] done                 [B] done                 [C] done
```

服务器日志出现**三个不同 TID 与三个不同 `connfd`**，说明每个客户端真的由独立线程服务；耗时 1.21 秒 ≈ 4 轮 × 0.3 秒，而非串行的 3 × 1.2 秒——交错执行得到实证。

21.3.3 示例 (c)：基于 `select` 的事件驱动服务器 `selectserver.c`

代码 (C)：

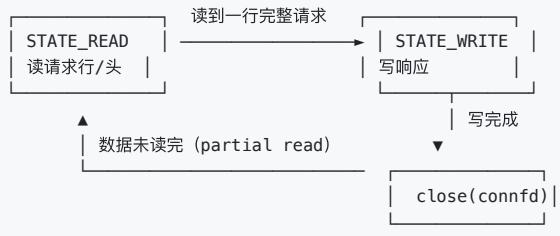
```c
/*
 * selectserver.c — 基于 I/O 多路复用（select）的事件驱动并发 echo 服务器
 * 单线程、单地址空间，用一个 select 循环同时照看 listenfd 与所有 connfd。
 * 用法: ./selectserver <port>
 * 编译: gcc -g -Wall -std=c11 selectserver.c -o selectserver
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <netdb.h>
#include <sys/select.h>
#include <sys/socket.h>

#define LISTENQ    1024
#define MAXLINE    8192
#define MAXCONN    16      /* 连接池容量，受 FD_SETSIZE(1024) 限制 */

static int connfd_pool[MAXCONN];      /* -1 表示该槽位空闲 */

static int open_listenfd(const char *port)
{
    struct addrinfo hints, *listp, *p;
    int listenfd, optval = 1;
    memset(&hints, 0, sizeof(hints));
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG | AI_NUMERICSERV;
    if (getaddrinfo(NULL, port, &hints, &listp) != 0) return -1;
    for (p = listp; p; p = p->ai_next) {
        if ((listenfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0) continue;
        setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(int));
        if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0) break;
        close(listenfd);
    }
    freeaddrinfo(listp);
    if (!p) return -1;
    if (listen(listenfd, LISTENQ) < 0) { close(listenfd); return -1; }
    return listenfd;
}

int main(int argc, char **argv)
{
    int listenfd, connfd, maxfd, nready, i;
    fd_set readset;
    socklen_t clientlen;
    struct sockaddr_storage clientaddr;
    char buf[MAXLINE];

    if (argc != 2) { fprintf(stderr, "usage: %s <port>\n", argv[0]); exit(1); }
    for (i = 0; i < MAXCONN; i++) connfd_pool[i] = -1;

    listenfd = open_listenfd(argv[1]);
    if (listenfd < 0) { fprintf(stderr, "open_listenfd failed\n"); exit(1); }
    printf("[select-server] listening on port %s, listenfd=%d\n", argv[1], listenfd);
    fflush(stdout);

    while (1) {
        /* ------ 每次循环都必须重建 fd_set! select 会就地修改它 ------ */
        FD_ZERO(&readset);
        FD_SET(listenfd, &readset);
        maxfd = listenfd;
        for (i = 0; i < MAXCONN; i++) {
            if (connfd_pool[i] < 0) continue;
            FD_SET(connfd_pool[i], &readset);
            if (connfd_pool[i] > maxfd) maxfd = connfd_pool[i];   /* maxfd = 最大描述符 */
        }

        /* 阻塞直到至少一个描述符就绪，返回就绪个数 */
        nready = select(maxfd + 1, &readset, NULL, NULL, NULL);
        if (nready < 0) {
            if (errno == EINTR) continue;      /* 被信号打断，重建 fd_set 再来 */
            perror("select"); exit(1);
        }
```

```
        /* ---- 事件 1: listenfd 就绪 => 有新连接到达 ---- */
        if (FD_ISSET(listenfd, &readset)) {
            clientlen = sizeof(struct sockaddr_storage);
            connfd = accept(listenfd, (struct sockaddr *)&clientaddr, &clientlen);
            if (connfd >= 0) {
                for (i = 0; i < MAXCONN; i++) if (connfd_pool[i] < 0) break;
                if (i == MAXCONN) {                 /* 池满: 拒绝该连接 */
                    printf("[select-server] pool full, reject fd=%d\n", connfd);
                    close(connfd);
                } else {
                    connfd_pool[i] = connfd;
                    printf("[select-server] accept fd=%d -> slot %d\n", connfd, i);
                    fflush(stdout);
                }
            }
            if (--nready == 0) continue;            /* 没有别的就绪 fd 了 */
        }

        /* ---- 事件 2: 某个 connfd 就绪 => 读一行并回写 ---- */
        for (i = 0; i < MAXCONN; i++) {
            connfd = connfd_pool[i];
            if (connfd < 0 || !FD_ISSET(connfd, &readset)) continue;
            ssize_t n = read(connfd, buf, sizeof(buf) - 1);
            if (n > 0) {
                buf[n] = '\0';
                write(connfd, buf, n);              /* echo 回去 */
            } else if (n == 0) {                    /* 对端关闭: EOF 事件 */
                printf("[select-server] EOF fd=%d, close slot %d\n", connfd, i);
                fflush(stdout);
                close(connfd);
                connfd_pool[i] = -1;
            } else if (errno != EINTR) {
                close(connfd);
                connfd_pool[i] = -1;
            }
            if (--nready == 0) break;
        }
    }
    return 0;
}
```

【代码做什么？】

1. 用 `connfd_pool[]`（**连接池**）记录当前所有活动连接，`-1` 表示空闲槽位。
2. 每轮循环重建 `fd_set`，把 `listenfd` 与池中所有活动 `connfd` 都加入，同时求出 `maxfd`，调用 `select(maxfd+1, ...)`。
3. `select` 返回后先看 `FD_ISSET(listenfd, &readset)`：是则 `accept`，把新 `connfd` 塞进池子。
4. 再遍历池子，对每个 `FD_ISSET(connfd, &readset)` 的**就绪**连接做一次非阻塞式的读—回写；读到 0 表示对端关闭，`close` 并释放槽位。
5. 用 `nready` 提前退出：所有就绪事件处理完就不再空转。

【底层机制透视 —— 状态机视角】：教材 `echoservers.c` 把每个连接建模成一个**状态机（state machine）**，因为 `rio_readlineb` 是**阻塞**的，不能在一个连接上停太久：



一旦 `read` 只返回了半行，服务器必须**记住**"这个连接读到哪了"，先回去服务别的连接，等 `select` 下次报告它就绪再接着读。本示例用最简策略（每次就绪读一次、写一次），真正的 `echoservers.c` 会为每个连接挂一个 `rio_t` 缓冲区并在状态间迁移。

【实测验证】：三个客户端并发连接同一端口：

```
=== 服务器输出 (selv.log) ===
[select-server] listening on port 15215, listenfd=3
[select-server] accept fd=4 -> slot 0
[select-server] accept fd=5 -> slot 1
[select-server] accept fd=6 -> slot 2
[select-server] EOF fd=4, close slot 0
[select-server] EOF fd=5, close slot 1
[select-server] EOF fd=6, close slot 2

=== 三个客户端并行总耗时: 0.912821335 秒（串行应约 2.7 秒）===
[A] recv: [A] line 0 / line 1 / line 2 / [A] done
[B] recv: [B] line 0 / line 1 / line 2 / [B] done
[C] recv: [C] line 0 / line 1 / line 2 / [C] done
```

整个过程**只有一条 `select` 循环、一个线程**，却让三个客户端同时得到服务。

补充示例：`select` 同时监视 stdin 与 socket（本机实跑，用于观察 `select` 的返回语义）：

```
=== ./selecttwo 15216（stdin 由管道喂入，另有一路 telnet 式连接）===
round 1: select 返回 1, FD_ISSET(0)=1 FD_ISSET(3)=0   -> 从 stdin 读到: hello-from-stdin
round 2: select 返回 1, FD_ISSET(0)=0 FD_ISSET(3)=1   -> 新连接 connfd=4
round 3: select 返回 1, FD_ISSET(0)=1 FD_ISSET(3)=0   -> 读到 second-line，转发到 connfd=4
round 4: select 返回 1, FD_ISSET(0)=1 FD_ISSET(3)=0   -> 读到 third-line，转发到 connfd=4
round 5: select 返回 1, FD_ISSET(0)=1 FD_ISSET(3)=0   -> stdin EOF，退出
```

注意第 2 轮 `FD_ISSET(0)` 从上一轮的 1 变回 0——正是 `select` **改写了** `readset`，程序每轮重建集合才得到正确结果。另测得本机 `FD_SETSIZE = 1024`、`sizeof(fd_set) = 128` 字节（正好 1024 位）：**描述符值 ≥ 1024 时 `FD_SET` 会越界写，行为未定义**——这就是大流量场景必须换成 `epoll` 的根本原因。

21.4 实验关联

L7 Proxy Lab（写一个带缓存的 HTTP 代理，共 70 分：基础正确性 40 + 并发 15 + 缓存 15）与本讲几乎一一对应：

- **Part II（并发，15 分）**：writeup 明确要求"为每个新连接派生一个新线程"，并且**"线程必须跑在分离态以避免内存泄漏"**——正是 21.3.2 中 `pthread_detach(pthread_self())` 那一行。`open_clientfd` / `open_listenfd` 基于 `getaddrinfo`，是线程安全的。
- **Part III（缓存，15 分）**：`MAX_CACHE_SIZE = 1 MiB`、`MAX_OBJECT_SIZE = 100 KiB`，淘汰策略近似 LRU。writeup 特别指出：**"用一个大的独占锁保护缓存是不可接受的方案"**——必须允许多个读者并发读缓存，只允许一个写者写缓存。这就是 Lecture 22–23 的**读者–写者锁（readers–writers lock）**。缓存数据量的上限是 `MAX_CACHE_SIZE + T * MAX_OBJECT_SIZE`（T 为最大并发连接数）。
- 其他坑：必须忽略 `SIGPIPE` 并优雅处理 `EPIPE`；`read` 可能返回 `-1` 且 `errno == ECONNRESET`，不能因此退出；代理长期运行，**出错不能 `exit`**（`csapp.c` 的错误包装函数需改写）；原请求是 HTTP/1.1 也必须转发为 HTTP/1.0；内容可能是**二进制**。调试手段：`curl -v --proxy`、`nc -l 12345`（可看到代理发出的请求原文）。

L8 SFS Lab 需要在文件系统层面处理并发请求与性能，本讲是它的直接前提；`badcnt.c` 的"并发写元数据丢失更新"与文件系统的目录/位图更新是同一类问题，而"锁的粒度是整个文件系统还是每个 inode"正是 SFS 的核心设计决策。

21.5 常见错误与调试技巧

- **忘掉 `close(connfd)`**：线程法服务器跑一会儿就 `accept: Too many open files`。**调试**：`ls -l /proc/<pid>/fd | wc -l` 观察描述符增长，或 `strace -f -e trace=close ./server`。
- **忘掉 `pthread_detach`**：线程默认是 joinable 的，终止后资源不回收，形成"僵尸线程"式内存泄漏，长期运行必然耗尽内存。**调试**：周期性打印 `/proc/self/status` 的 `Threads:` 字段；`valgrind --tool=helgrind` 观察线程状态。
- **把栈地址传给 `pthread_create`**：`Pthread_create(&tid, NULL, thread, &connfd)` 会让多个线程读写主线程栈上同一个 `int`，症状是"某客户端收到别人的数据"或"连接被服务两次"。**调试**：在线程入口打印 `vargp` 与 `*vargp`，多个线程打印出相同地址就是它；`gcc -fsanitize=thread` 能直接报 data race。
- **`select` 前不重建 `fd_set`**：表现为"随机漏事件"、"连接卡住不响应"。**调试**：在 `select` 前后各打印一次位图，能看到返回后被清零的位；`strace -e trace=select,pselect6` 核对每次调用的参数。
- **`nfds` 传错**：传成描述符**个数**而不是 `maxfd + 1`，会漏检编号更大的 fd（例如 `listenfd=3`、`connfd=4` 却传 3）。**调试**：打印 `maxfd+1` 与实际 `connfd` 对比。
- **在信号处理函数里调用 `printf` / `malloc`**：讲义用一整段真实 gdb 回溯演示了这个**死锁**——`printf` 内部要拿 stdout 缓冲区的锁，若主程序正在 `printf` 时被 `SIGCHLD` 打断，处理函数再次进入 `printf` 就会永久阻塞。**调试**：`gdb -p <pid>` 后 `bt`，堆栈会停在 `__lll_lock_wait_private` → `_IO_vfprintf_internal` → `printf` → `sigchld` → `<signal handler called>`；处理函数里只允许调用**异步信号安全**的函数（`write`、`waitpid`）。

- **以为 `volatile` 能解决竞态**：它只阻止编译器缓存到寄存器，不提供任何原子性。**调试**：`gcc -fsanitize=thread -g x.c -o x -lpthread && ./x`，TSan 会打印出冲突的两个访问行号与线程栈。
- **`fork` 后父子共享描述符造成的"连接不关闭"**：客户端已经断开，服务器却一直感知不到 EOF。**调试**：`lsof -p <pid>` 看是否有多个进程持有同一个 socket。

21.6 关键要点

- **三种并发是"谁交错 + 是否共享地址空间"的二维分类**：进程法由内核交错且地址空间独立，事件驱动由程序员交错且单一地址空间，线程法由内核交错但共享地址空间。
- **`select` 的 fd_set 是输入输出参数**——它会被内核就地改写，因此**每次调用前都必须 `FD_ZERO` + 重新 `FD_SET`**，且 `nfds` 必须是"最大描述符 + 1"，且 `fd_set` 上限为 `FD_SETSIZE = 1024`。
- **线程服务器的两条铁律**：参数必须从栈复制到**堆**（避免未预期共享），线程必须**分离（detach）**（避免资源泄漏）；`close(connfd)` 与 `free(vargp)` 一个都不能少。
- **`cnt++` 不是原子操作**——它是 load / add / store 三条指令；两个线程各做一亿次，实测计数落在 **90,981,898 ～ 175,343,844** 之间，20 次里一次都没到 200,000,000。
- **线程的最大优点与最大缺点都是同一件事**：数据共享太容易了。竞态的错误结果往往概率很低但绝不为零，而且**靠测试几乎抓不到**（讲义中同一个 `Pthread_create(..., &i)` 的竞态实验，在多核服务器上"看起来完全正常"，在单核笔记本上则"值到处都是"）。
- **并发的能力边界**：进程法隔离好但开销大、共享难；事件驱动开销极小、可控性最强但编码复杂、**无法利用多核**，一个连接的阻塞会拖垮全部；线程法居中——开销适中、共享容易，但调度不可控、最难调试。

21.7 思考题（带答案）

Q1（推演题 · `select` 顺序陷阱） 下面这段循环漏掉了对新连接的响应，请指出漏掉的是哪个 fd，并说明为什么：

```
while (1) {
    FD_ZERO(&readset);
    FD_SET(listenfd, &readset);          /* (1) */
    for (i = 0; i < MAXCONN; i++)
        if (connfd_pool[i] >= 0) FD_SET(connfd_pool[i], &readset);   /* (2) */
    select(maxfd + 1, &readset, NULL, NULL, NULL);   /* maxfd 在主循环外只算过一次 */
    /* (3) 之后直接处理 connfd_pool[]，未再检查 listenfd */
}
```

答：有两处错误。其一，`maxfd` 若在主循环外只算一次，则新 `accept` 到的 `connfd`（编号更大）永远不会被 `select` 监视——`select` 只看 `[0, nfds)` 范围内的位，`FD_SET` 设了位也不会被检查，正确做法是**每轮重算 `maxfd = max(listenfd, 所有 connfd)`**。其二，若在 `(3)` 处只遍历 `connfd_pool[]` 而不先判断 `FD_ISSET(listenfd, &readset)`，则 `listenfd` 上的就绪事件被完全丢弃，新连接会一直堆在 TCP 监听队列里（直到队列满、客户端 `connect` 超时）。还有一个隐藏陷阱：`select` 返回 `EINTR` 时 `readset` 已不可信，必须 `continue` 重新构建集合。

Q2（推演题 · 线程法服务器的错误写法） 下面这个"简化"的线程服务器错在哪里？给出错误场景与修复：

```
int connfd;
while (1) {
    connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
    Pthread_create(&tid, NULL, thread, (void *)&connfd);
}

void *thread(void *vargp) {
    int fd = *((int *)vargp);
    echo(fd);
    Close(fd);
    return NULL;
}
```

答：`&connfd` 是**主线程栈上同一个变量**的地址，所有线程的 `vargp` 都指向它。典型坏事序列：主线程 `accept` 得到 4 并创建线程 1；线程 1 尚未被调度，主线程又 `accept` 得到 5 **覆盖了同一地址**；线程 1 醒来读到 5，于是两个线程都去 `echo(5)`——客户端 4 收不到任何数据（挂死），而 `connfd==4` 这个描述符永远没人 `close`（泄漏）。此外该写法还缺少 `pthread_detach`（线程资源永不回收）与 `free`（若改成堆分配）。**修复**：主线程 `Malloc(sizeof(int))`，把 `connfd` 写进堆，`Pthread_create(&tid, NULL, thread, connfdp)`；线程例程第一行先 `int fd = *((int *)vargp);` 再 `Pthread_detach(pthread_self()); Free(vargp);`，最后 `Close(fd)`。

Q3（计算题 · 事件驱动 vs 迭代式的性能） 一个迭代式 echo 服务器，每个客户端请求的服务时间为 $t_{svc} = 2\ \text{ms}$ CPU 时间加 $t_{wait} = 50\ \text{ms}$ 等待用户输入的网络时间；同一份服务逻辑改成事件驱动后，每条连接每次事件的处理开销降为 $0.2\ \text{ms}$。若 100 个客户端同时连接、每个客户端需要 5 轮交互，分别估算两种设计的完成时间，并解释差距来源。

答：迭代式服务器在 `read` 上阻塞等待，每个客户端每轮要串行占用 $2+50=52$ ms，因此总时间约为 $100\times5\times52\ \text{ms}=26\ \text{s}$。事件驱动服务器不阻塞：它只在数据就绪时才花 0.2 ms 处理，100 个客户端的等待时间相互重叠，主导因素是**用户输入延迟**（约 $5\times50\ \text{ms}=250\ \text{ms}$）加上 CPU 处理量 $100\times5\times0.2\ \text{ms}=100\ \text{ms}$，合计约 $350\ \text{ms}$——快约 **74 倍**。差距来源不是"CPU 更快"，而是**等待时间的重叠**：迭代式把 100 份等待串行累加，事件驱动让它们并行发生。

Q4（辨析题 · "直觉但错误"的想法） 有同学说："我把 `cnt` 声明成 `volatile`，又在双核机器上跑，`badcnt` 应该就没问题了吧？实在不行就把 `niters` 调小一点，问题自然消失。"请指出两处错误。

答：第一处错误：`volatile` 与并发正确性无关。它只保证每次访问都真的从内存读写（不被优化进寄存器），**不保证读—改—写的原子性**。真正的执行序列仍是 `mov cnt→%rax; add $1,%rax; mov %rax→cnt`，两条 `mov` 之间线程随时可被切走，两个线程可能都基于同一个旧值写回，丢失一次自增。要修好必须用互斥锁、信号量或原子指令（`lock addq` / C11 `atomic_fetch_add`）——这正是 Lecture 22 的内容。第二处错误：把 `niters` 调小并不会让 bug 消失，只会让它**更难复现**。讲义的原话是"坏结果的概率往往非常低，但非零"；每次自增都有被交错覆盖的可能，`niters` 越小，撞上的概率越低，但程序依然是错的，而且这种"测试时好好的、上线后偶尔丢数据"的错误代价最高。正确做法是用 `gcc -fsanitize=thread` 这类工具在开发阶段主动暴露竞态，而不是靠调小负载掩盖它。

Lecture 22: 同步：基础 (Synchronization: Basic)

> **讲义对应**：CMU 15–213 Lecture 22 — Synchronization: Basics（素材：`F25-22-sync-basic.txt`）；配套 recitation：`F25-rec12_slides.txt`
> **教材对应**：CS:APP3e 第 12 章 12.4（共享变量与信号量）、12.5.1–12.5.3（互斥、顺序、生产者–消费者、读者–写者） **关联 Lab**：L8 SFS Lab（tiny 文件系统的并发访问）/ L7 Proxy Lab（缓存对象的读者–写者锁）

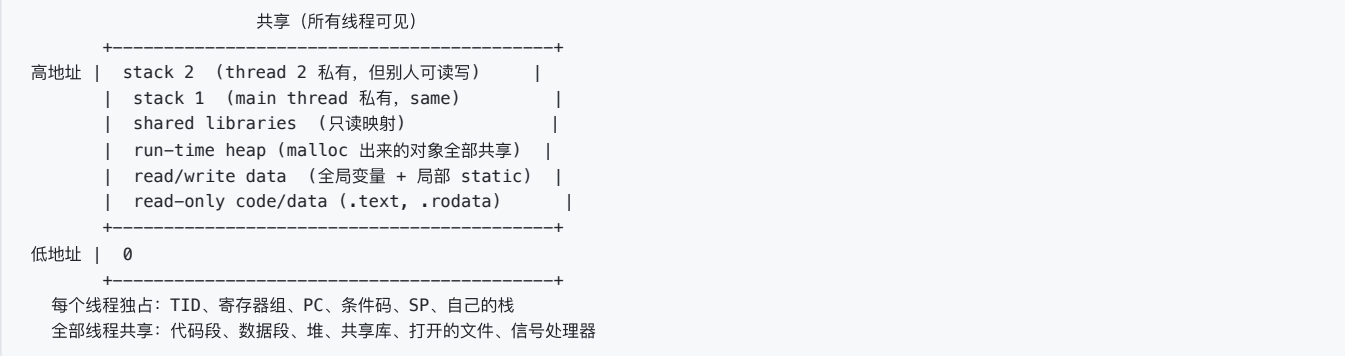
22.1 概述

Lecture 21 告诉我们"线程共享一切"，本讲回答它的必然后果：**既然共享，如何保证正确？** 讲义先用一个 20,000 次计数的例子把"数据竞态（data race）"钉在墙上，再用**进度图（progress graph）**把"任意交错"这个模糊直觉变成可证明的几何对象，最后给出两把武器：**互斥量（mutex）**与**信号量（semaphore）**。核心结论是：加锁不是"防君子"的礼貌，而是**用一条不可违反的不变量把不安全区域从状态空间中物理隔离**。它为 Lecture 23 的**条件变量、死锁避免、并发 bug 分类**铺路，也是 **SFS Lab 中 tiny 文件系统被多线程服务器调用时**必须先想清楚的问题。

22.2 核心概念与底层机制图解

22.2.1 共享变量与线程内存模型（Shared Variables and the Threads Memory Model）

- **定义与目的**：讲义拒绝"全局变量共享、栈变量私有"这种粗糙说法，给出精确定义——**变量 x 是共享的，当且仅当它的某个实例被多于一个线程引用**（a variable x is shared iff multiple threads reference some instance of x）。判定分三步：确定线程内存模型 → 确定变量实例在内存中的位置 → 数清有多少线程可能引用该实例。
- **直观解释**：把线程想象成**同一间办公室里的人**：每人有自己的草稿本（寄存器 + 栈），但白板、文件柜（代码段、数据段、堆、共享库）公用。你的笔记不锁起来别人也能翻——这就是"实际模型"与"概念模型"的错位。
- **底层机制图解**：线程把进程虚拟地址空间一分为二。



- **与机器码/硬件的对应**：寄存器上下文**真正隔离**（`%rax`、`%rsp`、`%rip` 随线程切换保存/恢复），但地址空间共用同一套页表——因此**线程 2 完全可以通过指针读写线程 1 的栈**，这正是"概念模型"与"实际模型"错位的根源。
- **补充说明**：`errno` 是特例——源码里是全局变量，但 glibc 用线程局部存储（TLS）让**每线程各有一份实例**，故不是共享变量。

22.2.2 进度图：把"任意交错"变成几何（Progress Graphs）

- **定义与目的**：进度图描绘并发线程的**离散执行状态空间**：每个轴对应一个线程的指令序列，平面上每个点是一个可能的执行状态 $(Inst_1, Inst_2)$，例如 (L_1, S_2) 表示"线程 1 已完成 L_1 且线程 2 已完成 S_2"。**轨迹（trajectory）**是合法的状态转移序列，即一次可能的并发执行。

- **直观解释**：就像两张地铁线路图并排，横轴 2 号线、纵轴 1 号线；一次并发执行就是"从原点出发只向右和向上走的路线"，**任何单调右上折线都可能发生**。
- **底层机制图解**：把 `cnt++` 编译出的三条指令记为 L_i（load）、U_i（update）、S_i（store），临界区由 H_i（header）、$L_iU_iS_i$、T_i（tail）围成。**不安全区域**是两线程临界区指令交错的状态集合（下图中 `###`）：

```
             Thread 2 →
       H2    L2    U2    S2    T2
     +-----+-----+-----+-----+-----+
  T1 |     |     |     |     |     |
     +-----+-----+-----+-----+-----+
  S1 |     | ### | ### | ### |     |
     +-----+-----+-----+-----+-----+
  U1 |     | ### | ### | ### |     |     <- 不安全区域
     +-----+-----+-----+-----+-----+       (unsafe region)
  L1 |     | ### | ### | ### |     |
     +-----+-----+-----+-----+-----+
  H1 |     |     |     |     |     |
     +-----+-----+-----+-----+-----+
Thread 1
   ↓
   定义：轨迹 safe ⟺ 从不进入任何不安全区域
   断言：轨迹对 cnt 而言是正确的 ⟺ 它是 safe 的
```

- **对角线 vs 阶梯线**：本讲最易被忽略的几何直觉。
- **对角线（沿 45° 斜走）**：两线程进度大致相等地推进，临界区在时间上**重叠**。只要重叠，轨迹必然穿过中央 `###`，于是 `L1` 读到旧值、`S2` 又写回，出现"两次自增得到 1"的经典错误。
- **阶梯线（先竖走完一段再横走一段）**：即"让线程 1 把整个临界区跑完，再放线程 2 进来"，本质是**把并发串行化**。轨迹始终贴着不安全区域上方或下方绕行即可证明 safe。
- 结论：**只要把所有不安全区域排除在轨迹之外，就保证了互斥**。mutex 的作用正是制造更外圈的**禁止区域（forbidden region）**，把 unsafe region 整个包住。
- **与硬件/机器码的对应**：不安全区域之所以存在，是因为 `cnt++;` 被编译成**三条独立指令**（见 22.3.1），指令之间可被调度器/中断任意切开；若硬件能把它变成一条不可分割的 read-modify-write（如 `lock addq`），不安全区域就消失了——这就是 22.3.5 原子操作的几何意义。

22.2.3 信号量（Semaphore）

- **定义与目的**：Dijkstra 提出的经典同步原语，是互斥量的推广：一个**非负整数**，创建时给定初值。mutex 是"只能取 0/1 且必须 P 先于 V"的退化情形；信号量允许 $s > 1$ 且**无 P/V 顺序要求**，因此能表达"计数"与"顺序"，而不只是"互斥"。
- **直观解释**：信号量像**停车场的电子计数牌**：`sem_wait` 是"抬杆进车"——有空位就减 1 开进去，为 0 就在门口排队；`sem_post` 是"出场"——牌子加 1 并叫醒一个排队者。互斥量则像**只有单把钥匙的洗手间**。
- **两个操作**：
- $P(s)$（`sem_wait`，"Prolaag/Proberen"，荷兰语"尝试减少"）：若 $s = 0$ 则等待某个 V 操作发生；然后 $s\ {-}{=}\ 1$ 并返回。
- $V(s)$（`sem_post`，"Verhogen"，荷兰语"增加"）：$s\ {+}{=}\ 1$；若有线程正阻塞在 P 中，唤醒其中一个。
- **原子性**：`sem_wait` / `sem_post` 内部**原子**——`sem_wait` 的"判断 $s > 0$"与"$s\ {-}{=}\ 1$"之间不可能被插入。讲义给出的部分实现正是用 CPU 原子指令：

```
sem_wait:
        mov     $-1, %edx            # decrement
        lock xadd %edx, SEM_COUNT(%rdi)   # 原子读-改-写；%edx 得到旧值
        test    %edx, %edx
        jle     .Lclosed             # 旧值 <= 0 => 减成负数了，必须睡
        ret                          # 信号量原本是开的，直接返回
.Lclosed:
        # 睡到别的线程调用 sem_post 为止（30 多条指令 + 一次系统调用）
```

讲义点评："Suspiciously similar to a mutex, huh?"。**关键区别在语义**：`sem_wait` 可在任何时刻调用（初值甚至为 0），而 `unlock` 只能由持有者调用。

`sem_*` 与 `pthread_mutex_*` 接口对照表

语义	POSIX 信号量（`<semaphore.h>`）	互斥量（`<pthread.h>`）
声明类型	`sem_t s;`	`pthread_mutex_t m;`
初始化（动态）	`sem_init(&s, pshared, val)`；`pshared=0` 表示线程间共享	`pthread_mutex_init(&m, NULL)`

语义	POSIX 信号量（`<semaphore.h>`）	互斥量（`<pthread.h>`）
初始化（静态）	无（必须先 `sem_init`）	`pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;`
加锁 / P 操作	`sem_wait(&s)` —— $s > 0$ 则 $s{=}s-1$，否则阻塞	`pthread_mutex_lock(&m)` —— 空闲则锁住，否则阻塞重试
解锁 / V 操作	`sem_post(&s)` —— $s{=}s+1$，唤醒一个等待者	`pthread_mutex_unlock(&m)` —— **只能由持锁者调用**
取值（调试用）	`sem_getvalue(&s, &v)`（非标准语义，仅作参考）	无对应接口（不透明对象）
销毁	`sem_destroy(&s)`	`pthread_mutex_destroy(&m)`
初值范围	任意 $val \geq 0$（计数信号量）	只能 0/1（二元）
调用约束	P/V **可以乱序**、可跨线程配对	unlock 必须与 lock **同线程配对**
典型用途	互斥、**计数**、**顺序/调度**	仅互斥
相对开销	更慢（讲义实测 27.6 s vs mutex 15 s）	较快

22.2.4 用信号量实现互斥与顺序（Mutex and Ordering）

- **互斥（mutual exclusion）**：信号量初值设为 1，临界区前后包上 `sem_wait` / `sem_post`，它就退化成"二元信号量（binary semaphore）"。不变量是"s 只可能是 0 或 1"，禁止区域恰好包住不安全区域。
- **顺序 / 调度（scheduling / ordering）**：这是信号量**无法用 mutex 替代**的能力——强制"线程 A 先做某事，线程 B 后做"。手法是**把初值设为 0**，让先到的线程阻塞：

```
线程 A（生产者）                线程 B（消费者）
---------------------------  ---------------------------
 生产一个 item                  sem_wait(&items)  <-- 初值 0，先阻塞
 sem_post(&items)  ---------->  （被唤醒）消费该 item
       |                              ^
       +------- 信号量充当"事件已发生"的标志 -------+
```

讲义在 Supplemental 里点明：**信号量的真正价值是"wait on event"**，这正与之前用 `sigsuspend` 解决"等待某个事件"是同一思想（详见 Lecture 17）。生产者–消费者里的 `slots` / `items` 就是这一用法的教科书范例。

22.2.5 生产者–消费者问题（Producer–Consumer / Bounded Buffer）

- **定义与目的**：生产者产生 item 放进**有界缓冲区（bounded buffer）**，消费者取出。这是"SFS Lab 服务器线程"与"Proxy Lab 缓存淘汰"的共同抽象，约束有两条：**绝不在满缓冲区插入、绝不在空缓冲区取出**。
- **底层机制图解**：缓冲区是循环数组，三个信号量各司其职：

```
 sbuf_t                            +---------------------------+
+------------------+               |  sem_t mutex;  init = 1   | 保护 buf/front/rear
| int buf[N];      |  <-- 数据 ---- |  sem_t slots;  init = N   | 空槽位计数
| int n;           |               |  sem_t items;  init = 0   | 已填 item 计数
| int front;       |               +---------------------------+
| int rear;        |
| sem_t mutex;     |                生产者: wait(slots) -> wait(mutex) -> 存 -> post(mutex) -> post(items)
| sem_t slots;     |                消费者: wait(items) -> wait(mutex) -> 取 -> post(mutex) -> post(slots)
| sem_t items;     |
+------------------+                        slots + items + 缓冲区占用 = N   （不变量）
      |
      v   N = 4 的循环缓冲区（rear/front 都按 mod N 前进）
 +------+------+------+------+
 | [0]  | [1]  | [2]  | [3]  |
 +------+------+------+------+
    ^front=2              ^rear=0
    已取出                 下一个插入位置
    占用 = (rear - front) mod N = 2  =>  items = 2, slots = 2
```

- **为什么 `mutex` 必须存在？** `slots` 与 `items` 只保证**计数语义，不保证对 `buf`、`front`、`rear` 的访问互斥**。两个生产者都成功 `sem_wait(&slots)` 拿到空位后，会同时执行 `sp->buf[(++sp->rear) % n] = item;` —— `++sp->rear` 是**非原子**的读–改–写，两者可能算出同一下标，一个 item 被覆盖、另一个消失；同理两个消费者会重复取出同一槽位。**`slots` / `items` 管"能不能进"，`mutex` 管"进去以后别撞车"**——职责正交，缺一不可。
- **`sem_wait` 的顺序陷阱（本讲最重要的 bug）**：如果写成

```
sem_wait(&sp->mutex);   /* ❌ 先拿锁 */
sem_wait(&sp->slots);   /* ❌ 再等空位 */
```

会**死锁**。机制：生产者握着 `mutex` 去等 `slots`；而唯一能增加 `slots` 的人是消费者（取出 item 后 `sem_post(&slots)`），消费者却必须先 `sem_wait(&mutex)` —— `mutex` 正被生产者攥着。于是形成**持有并等待（hold-and-wait）+ 循环等待（circular wait）**：

```
生产者: [持有 mutex] --等待--> (slots > 0)
                                  ^
                                  | 只有消费者能释放
消费者: [等待 mutex] <--需要------ (取出 item 后 post(slots))
              ^
              +-- mutex 被生产者持有 => 谁也动不了 => 死锁
```

正确顺序的铁律：**先等资源（`slots` / `items`），后拿锁（`mutex`）；先放锁（`mutex`），后通知（`items` / `slots`）**。这样"持锁等待"的边被彻底消除，循环等待无从形成。22.3.4 用真实程序演示了这个卡死。

22.2.6 读者-写者问题（Readers-Writers）

- **定义与目的**：两类线程共享一个对象：**读者只读、写者只写**。约束是"**任意时刻允许多个读者，或至多一个写者**"。这是 Proxy Lab 缓存锁的原型。
- **第一类：读者优先（readers preference）**，写者可能饥饿。两个信号量：`mutex` 保护 `readcnt`，`w` 作为写者锁。

```
          读者进入                              写者进入
sem_wait(&mutex);                       sem_wait(&w);
readcnt++;                              ... 写 ...
if (readcnt == 1) sem_wait(&w);  <- 第一个读者拦住写者
sem_post(&mutex);                       sem_post(&w);
... 读 ...
sem_wait(&mutex);                         状态机（w 的取值）:
readcnt--;                                   w = 1 : 无人持有（无读者、无写者）
if (readcnt == 0) sem_post(&w); <- 最后一个读者放行写者       <--+
sem_post(&mutex);                            w = 0 : 有写者写，或 readcnt > 0
                                                        |
       多个读者可同时持有"读权限"（w 保持 0）                  |
       新读者在 readcnt>0 时不会被阻塞 ------------------------+
       => 只要读者源源不断到达，写者永远等不到 w => 写者饥饿
```

- **第二类：写者优先（writers preference）**，读者可能饥饿。经典做法是加一个**旋转门（turnstile）**信号量：写者一到就"关门"挡住后续读者，写完再"开门"。**注意**：网上流传的"在读者入口直接 `sem_wait(&w)`、出口 `sem_post(&w)`"写法虽然能挡住新读者，却把**读者之间也串行化**了（每个读者都独占 `w`），丢掉了"多读者并发"这一核心优势——它是一个"能跑但对性能有害"的写法，22.3.3 给出了正确的 turnstile 版本。
- **Proxy Lab 的缓存就是读者-写者锁**：多个请求线程同时**查缓存（读者）**，只有插入/淘汰才需**写者**权限。单一 mutex 会让所有请求串行化；读者-写者锁把"读多写少"变成真正的并行。
- **讨论：应用真有读者-写者语义吗？** 有些对象**读者之间也会改变状态**——带 LRU 淘汰的缓存，读者命中后要更新"最近使用"顺序，这就是一次写；带引用计数的对象，读者要 `refcount++`。此时"多读者并发"的假设不成立，必须退化为互斥（或更细粒度的锁）。**判据是：这次读操作是否修改了共享状态。**

22.2.7 信号量的常见陷阱（Pitfalls）

1. `sem_wait` 与 `sem_post` **不配对**：少一个 `post` → 其他线程永久阻塞；少一个 `wait` → 计数错乱、缓冲区越界。
2. **忘记** `sem_init`：未初始化的 `sem_t` 是垃圾字节，行为未定义（UB），通常表现为立刻死锁或随机唤醒。
3. **用 `mutex` + `if` 而不是 `while`**：这是条件变量的典型陷阱（Lecture 23 详述），此处提前埋点——**条件判断必须重新检查，不能只信一次**。
4. **加锁顺序不一致 → 死锁**：线程 A 按 $R_1 \to R_2$、线程 B 按 $R_2 \to R_1$ 加锁，必然可能循环等待。**铁律：所有线程按同一全序加锁。**
5. **锁的粒度太粗 → 性能崩塌**：recitation 里用一把锁保护整棵 BST，所有插入完全串行；改成"每结点一把锁"后即可并发。但细粒度锁会引入新的加锁顺序问题。
6. **持有锁时调用可能阻塞的函数**（`malloc`、`printf`、`read`、`write`）：临界区被无谓拉长，甚至与库内部锁形成新的循环等待。

22.3 代码示例与底层机制分析

22.3.1 数据竞态与信号量修复：`badcnt.c` vs `goodcnt.c`

代码 (C)——先看错误的版本：

```c
/* badcnt.c : 编译 gcc -g -Wall -std=c11 badcnt.c -o badcnt -lpthread */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

static volatile unsigned long cnt = 0;   /* 共享全局变量 */

void *incr_thread(void *arg)
{
    unsigned long i, niters = (unsigned long)arg;
    for (i = 0; i < niters; i++)
        cnt++;                 /* 非原子：L=load, U=update, S=store */
    return NULL;
}

int main(int argc, char **argv)
{
    unsigned long niters = strtoul(argv[1], NULL, 10);
    pthread_t t1, t2;
    pthread_create(&t1, NULL, incr_thread, (void *)niters);
    pthread_create(&t2, NULL, incr_thread, (void *)niters);
    pthread_join(t1, NULL);  pthread_join(t2, NULL);
    if (cnt != 2 * niters) { printf("FAIL: cnt=%lu (expected %lu)\n", cnt, 2*niters); return 1; }
    printf("OK:   cnt=%lu\n", cnt);
    return 0;
}
```

修复版只改三行——把 `cnt++` 包进二元信号量：

```c
/* goodcnt.c : 编译 gcc -g -Wall -std=c11 goodcnt.c -o goodcnt -lpthread */
#include <semaphore.h>
static sem_t mutex;                      /* 二元信号量，初值 1 */
void *incr_thread(void *arg)
{
    unsigned long i, niters = (unsigned long)arg;
    for (i = 0; i < niters; i++) {
        sem_wait(&mutex);                /* P(S)：进入临界区 */
        cnt++;                           /* 临界区 wrt cnt */
        sem_post(&mutex);                /* V(S)：离开临界区 */
    }
    return NULL;
}
int main(int argc, char **argv) {
    /* ... 同上 ... */
    sem_init(&mutex, 0, 1);              /* pshared=0，线程间共享；初值 1 */
    /* ... create / join ... */
    sem_destroy(&mutex);
}
```

【代码做什么？】 1. 主线程把 `niters` 强转成 `void *` 传给两个线程（讲义认可的"cast of int/long"传参法）。 2. 两个 `incr_thread` 各循环 `niters` 次读–改–写 `cnt`。 3. `main` 用 `pthread_join` 等待结束，再检查 `cnt == 2*niters`。 4. `badcnt` 中三条指令可被任意切开；`goodcnt` 中 `sem_wait` / `sem_post` 把整段包成一个"禁止区域"。

【底层机制透视】 `sem_wait` 内部是一条 `lock xadd`（原子读–改–写，锁住缓存行并广播失效），"判断 $s > 0$"与"$s - 1$"之间不存在窗口；而 `cnt++` 的 `movq/addq/movq` 之间存在窗口——另一线程可在窗口中把 `cnt` 从 0 改到 1，而本线程 `%rax` 仍是 0，最后把 0 写回，丢失一次自增。**注意 `volatile` 只阻止编译器优化，不提供原子性**，`badcnt` 里加它是为了不让编译器把循环优化掉。

【与汇编 / 硬件的对应】 真实的 `gcc -O0 -S` 输出（普通 vs `_Atomic`）：

```
# 普通 unsigned long:                    # _Atomic unsigned long:
.L3:                                     .L6:
    movq    cnt(%rip), %rax                  movl    $1, -20(%rbp)
    addq    $1, %rax                         movl    -20(%rbp), %eax
    movq    %rax, cnt(%rip)                  cltq
    addq    $1, -8(%rbp)                     lock xaddq  %rax, acnt(%rip)
    cmpq    $2, -8(%rbp)                     movq    %rax, -16(%rbp)
    jle     .L3                              addq    $1, -8(%rbp)
                                             jle     .L6
# 3 条指令 = 不安全区域的几何来源          # 1 条带 lock 前缀的指令 = 不安全区域消失
```

【实测验证】（两台命令均真实执行，`-lpthread` 链接；本机 128 核）：

```
$ ./badcnt 100000
FAIL: cnt=127014 (expected 200000)
$ ./badcnt 100000
FAIL: cnt=188915 (expected 200000)
$ ./badcnt 100000
FAIL: cnt=145590 (expected 200000)
$ ./badcnt 100000
FAIL: cnt=159203 (expected 200000)

$ ./goodcnt 100000
OK:   cnt=200000
$ ./goodcnt 100000
OK:   cnt=200000
$ ./goodcnt 100000
OK:   cnt=200000
$ ./goodcnt 100000
OK:   cnt=200000
```

即使加 `-O2`，`badcnt` 依然会失败（实测 `cnt=163023`、`cnt=170219` 交替出现，偶尔"碰巧"等于 200000）——**竞态的不确定性正是它最难调试的地方：程序可能 100 次都对，第 101 次错。**

22.3.2 有界缓冲区生产者–消费者（多生产者 + 多消费者）

代码 (C):

```c
/* sbuf_pc.c : 编译 gcc -g -Wall -std=c11 sbuf_pc.c -o sbuf_pc -lpthread */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define N 4                 /* 缓冲区槽位数 */
#define NITERS 5            /* 每个生产者/消费者的次数 */

typedef struct {
    int buf[N];
    int n, front, rear;
    sem_t mutex;            /* 保护 buf/front/rear */
    sem_t slots;            /* 空槽位数，初值 n */
    sem_t items;            /* 已填 item 数，初值 0 */
} sbuf_t;

static sbuf_t sbuf;

static void sbuf_init(sbuf_t *sp, int n) {
    sp->n = n; sp->front = sp->rear = 0;
    sem_init(&sp->mutex, 0, 1);
    sem_init(&sp->slots, 0, n);
    sem_init(&sp->items, 0, 0);
}

static void sbuf_insert(sbuf_t *sp, int item) {
    sem_wait(&sp->slots);                       /* 1) 等一个空槽位 */
    sem_wait(&sp->mutex);                       /* 2) 锁住缓冲区   */
    sp->buf[(++sp->rear) % (sp->n)] = item;
    sem_post(&sp->mutex);                       /* 3) 解锁         */
    sem_post(&sp->items);                       /* 4) 通知有新 item */
}

static int sbuf_remove(sbuf_t *sp) {
    int item;
    sem_wait(&sp->items);                       /* 1) 等一个 item  */
    sem_wait(&sp->mutex);                       /* 2) 锁住缓冲区   */
    item = sp->buf[(++sp->front) % (sp->n)];
    sem_post(&sp->mutex);                       /* 3) 解锁         */
    sem_post(&sp->slots);                       /* 4) 通知有空槽位 */
    return item;
}

static void *producer(void *arg) {
    long id = (long)arg;
    for (int i = 0; i < NITERS; i++) sbuf_insert(&sbuf, (int)(id * 100 + i));
    return NULL;
}
static void *consumer(void *arg) {
    int sum = 0;
    for (int i = 0; i < NITERS; i++) sum += sbuf_remove(&sbuf);
    printf("consumer total = %d\n", sum);
    return NULL;
}
int main(void) {
    pthread_t p[2], c[2];
    sbuf_init(&sbuf, N);
    for (long i = 0; i < 2; i++) pthread_create(&p[i], NULL, producer, (void *)i);
    for (long i = 0; i < 2; i++) pthread_create(&c[i], NULL, consumer, (void *)i);
    for (int i = 0; i < 2; i++) pthread_join(p[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(c[i], NULL);
    sem_destroy(&sbuf.mutex); sem_destroy(&sbuf.slots); sem_destroy(&sbuf.items);
    return 0;
}
```

【代码做什么？】 4 线程（2 生产者 + 2 消费者），每个生产者插入 5 个唯一编号 item（`id*100+i`），每个消费者取出 5 个。`slots` 归零时生产者自动阻塞，`items` 递增时唤醒消费者。

【底层机制透视】 `front` / `rear` 按 `mod n` 循环前进，二者之差恒等于 `items`，维护不变量 $\textbf{slots} + \textbf{items} + \text{占用} = n$。`sem_wait(&slots)` 在 `sem_wait(&mutex)` 之前（**先资源后锁**）；`sem_post(&mutex)` 在 `sem_post(&items)` 之前（**先放锁后通知**）。两个顺序都不能颠倒。

【实测验证】 真实运行输出（`timeout 10 ./sbuf_pc`，退出码 0）：

```
buf size N=4, producers=2, consumers=2, each produces/consumes 5
  [producer 584] insert  0  -> rear=1 front=0
  [producer 584] insert  1  -> rear=2 front=0
  [producer 584] insert  2  -> rear=3 front=0
  [producer 584] insert  3  -> rear=0 front=0      <- 缓冲区满，rear 回绕到 0
  [consumer 176] remove  0 <- front=1 rear=0
  [consumer 176] remove  1 <- front=2 rear=0
  [consumer 176] remove  2 <- front=3 rear=1
  [consumer 176] remove  3 <- front=0 rear=1
  [consumer 176] remove  4 <- front=1 rear=1
  [consumer 176] total = 10
  [producer 880] insert 100 -> rear=2 front=1
  [producer 880] insert 101 -> rear=3 front=1
  [producer 880] insert 102 -> rear=0 front=1
  [producer 880] insert 103 -> rear=1 front=1
  [producer 584] insert  4  -> rear=1 front=2
  [consumer 472] remove 100 <- front=2 rear=1
  [consumer 472] remove 101 <- front=3 rear=1
  [consumer 472] remove 102 <- front=0 rear=1
  [producer 880] insert 104 -> rear=2 front=1
  [consumer 472] remove 103 <- front=1 rear=2
  [consumer 472] remove 104 <- front=2 rear=2
  [consumer 472] total = 510
done: 2*5 items produced and consumed.
```

两条断言在输出中可验证：消费者 176 取到 `0,1,2,3,4`，消费者 472 取到 `100..104`——**没有 item 被重复取出或丢失**；`rear` / `front` 在 0—3 间正确回绕。

22.3.3 读者–写者（第一类，读者优先）

代码 (C)：

```c
/* rw1.c : 编译 gcc -g -Wall -std=c11 rw1.c -o rw1 -lpthread */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

static int readcnt = 0;
static sem_t mutex;             /* 保护 readcnt */
static sem_t w;                 /* 写者互斥 / 读者与写者互斥 */
static int shared_data = 0;
static volatile int stop = 0;

static void *reader(void *arg) {
    long id = (long)arg;
    while (!stop) {
        sem_wait(&mutex);
        readcnt++;
        if (readcnt == 1) sem_wait(&w);      /* 第一个读者拦住写者 */
        sem_post(&mutex);

        printf("reader %ld: read shared_data = %d (readcnt=%d)\n", id, shared_data, readcnt);
        usleep(1000);                        /* 模拟读耗时 */

        sem_wait(&mutex);
        readcnt--;
        if (readcnt == 0) sem_post(&w);      /* 最后一个读者放行写者 */
        sem_post(&mutex);
        usleep(1000);
    }
    return NULL;
}

static void *writer(void *arg) {
    long id = (long)arg;
    for (int i = 0; i < 3; i++) {
        sem_wait(&w);
        shared_data++;
        printf("writer %ld: wrote shared_data = %d\n", id, shared_data);
        usleep(2000);
        sem_post(&w);
        usleep(3000);
    }
    return NULL;
}

int main(void) {
    pthread_t r[3], wt[2];
    sem_init(&mutex, 0, 1); sem_init(&w, 0, 1);
    for (long i = 0; i < 3; i++) pthread_create(&r[i], NULL, reader, (void *)i);
    for (long i = 0; i < 2; i++) pthread_create(&wt[i], NULL, writer, (void *)i);
    for (int i = 0; i < 2; i++) pthread_join(wt[i], NULL);
    stop = 1; usleep(10000);
    for (int i = 0; i < 3; i++) pthread_join(r[i], NULL);
    printf("done: final shared_data = %d (expected 6)\n", shared_data);
    return 0;
}
```

【代码做什么？】 3 个读者循环读、2 个写者各写 3 次。`readcnt` 是共享变量，必须由 `mutex` 保护。**第一个**读者（0→1）去抢 `w`；**最后一个**读者（1→0）释放 `w`。

【底层机制透视】 关键在"计数 + 首末哨兵"：`w` 只在读者数 0→1 与 1→0 时被触碰，故 k 个并发读者只付常数次 P/V。`mutex` 不可省：若两读者同时 `readcnt++`，可能"都以为自己是第一个"（第二次 `sem_wait(&w)` 白白阻塞）或"都以为自己不是最后一个"（`w` 永不释放）。

【实测验证】 真实输出（节选，退出码 0）：

```
  reader 0: read shared_data = 0 (readcnt=1)
  reader 1: read shared_data = 0 (readcnt=2)
  reader 2: read shared_data = 0 (readcnt=3)     <- 三个读者同时持有读权限
  writer 0: wrote shared_data = 1
  writer 1: wrote shared_data = 2
  reader 0: read shared_data = 2 (readcnt=1)
  ...
  writer 1: wrote shared_data = 6
done: final shared_data = 6 (expected 6)
```

`readcnt=3` 证明读者确实并发；`shared_data` 单调递增且无丢失证明写者互斥正确。

22.3.4 死锁演示：加锁顺序不一致

代码 (C)（⚠️ 仅供演示，请勿模仿）：

```c
/* deadlock.c : 编译 gcc -g -Wall -std=c11 deadlock.c -o deadlock -lpthread */
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

static pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;

static void *t0(void *arg) {          /* 顺序: m1 -> m2 */
    (void)arg;
    pthread_mutex_lock(&m1); printf("T0: locked m1\n"); fflush(stdout);
    usleep(200000);
    printf("T0: waiting for m2 ...\n"); fflush(stdout);
    pthread_mutex_lock(&m2);          /* <= 永远等不到 */
    pthread_mutex_unlock(&m2); pthread_mutex_unlock(&m1);
    return NULL;
}
static void *t1(void *arg) {          /* 顺序: m2 -> m1 —— 相反！ */
    (void)arg;
    pthread_mutex_lock(&m2); printf("T1: locked m2\n"); fflush(stdout);
    usleep(200000);
    printf("T1: waiting for m1 ...\n"); fflush(stdout);
    pthread_mutex_lock(&m1);          /* <= 永远等不到 */
    pthread_mutex_unlock(&m1); pthread_mutex_unlock(&m2);
    return NULL;
}
int main(void) {
    pthread_t a, b;
    printf("pid = %d\n", (int)getpid()); fflush(stdout);
    pthread_create(&a, NULL, t0, NULL);
    pthread_create(&b, NULL, t1, NULL);
    pthread_join(a, NULL); pthread_join(b, NULL);
    printf("never reached: no deadlock?\n");   /* 永远不会打印 */
    return 0;
}
```

【实测验证】 用 `timeout` 退出码与 `gdb` attach 双重确认：

```
$ timeout 5 ./deadlock
pid = 3623184
T0: locked m1
T1: locked m2
T0: waiting for m2 ...
T1: waiting for m1 ...
$ echo $?
124                       # 124 = timeout 到点强杀 => 程序确实卡死

$ ./deadlock & sleep 1.2 ; gdb -p $(pgrep -n -x deadlock) -batch -ex "thread apply all bt"
Thread 3 (LWP 3629611):
#0  __lll_lock_wait ()            <- 阻塞在 futex 上
#1  pthread_mutex_lock ()
#2  t1 (arg=0x0) at deadlock.c:37  <- T1 等 m1
Thread 2 (LWP 3629610):
#0  __lll_lock_wait ()
#1  pthread_mutex_lock ()
#2  t0 (arg=0x0) at deadlock.c:23  <- T0 等 m2
Thread 1 (LWP 3629608):
#0  __futex_abstimed_wait_common ()
#1  __pthread_clockjoin_ex ()
#2  main () at deadlock.c:50       <- main 卡在 pthread_join

$ grep -E "^(State|Threads)" /proc/<pid>/status
State:  S (sleeping)
Threads:        3
```

两个线程各自停在 `deadlock.c:23` 与 `deadlock.c:37` 这两行 `pthread_mutex_lock` 上——`__lll_lock_wait` 说明它们已进入内核 futex 睡眠，是**真死锁**，CPU 占用为 0。

【补充演示：生产者–消费者顺序错误也会死锁】 把 22.2.5 的顺序陷阱写成程序（`pcdeadlock.c`：生产者 `wait(mutex)` → `wait(slots)`，消费者正确）：

```
$ timeout 5 ./pcdeadlock
pid = 3631668
producer: try acquire mutex (i=0)
producer: got mutex, now wait slots
producer: got slot
producer: try acquire mutex (i=1)
producer: got mutex, now wait slots
producer: got slot
producer: try acquire mutex (i=2)
producer: got mutex, now wait slots        <- 缓冲区满(N=2)，持锁等空位
consumer: got item, try mutex              <- 消费者想拿 mutex 腾空位，被挡住
$ echo $?
124                                        # 死锁成立
```

这正是 22.2.5 的机制在真机上的复现：**生产者攥着 `mutex` 等 `slots`，而能增加 `slots` 的消费者必须拿到 `mutex`**。

22.3.5 临界区性能：加锁的代价与可扩展性

讲义给出的实测三档（同一计数器、同一工作量），下表中的本机数据经真实测量：

版本	讲义耗时	本机实测（2 线程 × 1,000,000 次，`-O2`）
无锁（`cnt++`，结果错误）	0.48 s	0.000 s（被优化掉/极快，但结果不可信）
`pthread_mutex`	15 s	0.099 s
`sem_wait/sem_post`	27.6 s	0.208 s（约为 mutex 的 **2.1×**）
`_Atomic` + `lock addq`	3.41 s	0.012 s（约为 mutex 的 **1/8**）

结论一致：**mutex 比信号量快，原子操作比 mutex 更快一个数量级**——这与讲义的排序完全相同，也正是"**能用 mutex 就别用信号量，能用原子操作就别用锁**"的量化依据。

可扩展性（scalability）：把总工作量固定为 16,000,000 次加锁自增，只改变线程数（本机真实测量）：

```
TOTAL work = 16000000 lock-protected increments
 threads      time(s)         Mops/s    speedup
       1        0.130         122.92       1.00
       2        0.489          32.72       0.27
       4        0.680          23.51       0.19
       8        1.087          14.72       0.12
```

线程越多越慢——**加锁把并行计算变成了串行队列**：每次 `lock` 都要独占那条缓存行，其余核心空转等待。这是 Amdahl 定律在同步原语上的体现：临界区占比 p 时加速比上限为 $1/p$；临界区本质串行则 $p \to 1$，**上限是 1**。这也解释了 recitation 为何要把"一棵树一把锁"改成"一个结点一把锁"：粒度缩小，p 才真正下降。

22.4 实验关联

- **L8 SFS Lab（tiny 文件系统）**：多线程服务器会并发调用 `open` / `read` / `write` / `stat`，底层是同一份 `tiny` 状态。**必须给共享元数据（目录树、空闲链表、引用计数）加锁。**
- **L7 Proxy Lab（缓存）**：缓存是**天然的读者-写者对象**——大量请求线程查缓存（读者），只有 miss 后的插入/淘汰才写。单一 mutex 会让全部请求串行。**注意 22.2.6 的提醒**：若读者命中后要更新 LRU 链表，它其实在写，不能当纯读者。
- **本讲的坑**：① SFS 中给"每个操作"加锁太粗、"每个字段"加锁太细易死锁，**中间粒度（一个文件/一个目录项一把锁）**通常最稳；② 代理缓存的"淘汰"与"插入"若分两次拿锁，会出现"刚放进去就被淘汰"的窗口；③ 加锁顺序必须全局统一（先缓存锁后日志锁，绝不可反之）。

22.5 常见错误与调试技巧

- **数据竞态（data race）**：未同步地并发读写同一变量。**调试**：`valgrind --tool=helgrind ./badcnt 2000` ——实测输出 `Possible data race during write of size 8 at 0x404070 by thread #3`，而 `goodcnt` 为 `ERROR SUMMARY: 0 errors`（本机 `-fsanitize=thread` 因 ASLR 冲突无法运行）。
- **忘记初始化 / 忘记销毁信号量**：漏 `sem_init` → 死锁或随机唤醒（UB）；漏 `sem_destroy` → 资源泄漏。**调试**：`valgrind --leak-check=full`。
- **死锁（deadlock）**：现象是 CPU 占用为 0 但永不结束。**调试**：`timeout 5 ./prog; echo $?`（返回 124 即卡死），再 `gdb -p $(pgrep -n -x prog) -batch -ex "thread apply all bt"`，看是否所有线程都停在 `__lll_lock_wait`；用 `grep State /proc/<pid>/status` 确认是 `S (sleeping)`。
- **信号量 P/V 不配对**：计数器漂移导致越界或永久阻塞。**调试**：在 `sem_wait` 前后 `sem_getvalue(&s,&v)` 打点看值是否漂移；或 `strace -f -e trace=futex ./prog` 看 futex 是否卡死。
- **加锁顺序不一致**：只在特定调度下复现，最隐蔽。**调试**：为每把锁编号并要求严格递增；用 `pthread_mutexattr_settype(&a, PTHREAD_MUTEX_ERRORCHECK)` 让错误用法返回 `EDEADLK` 而非静默卡死。
- **持锁调用阻塞函数**：`malloc` / `printf` / `write` 内部有锁，易与自定义锁形成循环等待。**调试**：`perf stat -e context-switches,cpu-migrations ./prog` 看上下文切换是否异常升高；`strace -f -e trace=write,writev ./prog` 看临界区内是否有系统调用。
- **虚假共享（false sharing）**：两线程各改**不同**变量但落在同一缓存行（64 B），表现为本质串行。**调试**：`perf stat -e cache-misses,cache-references ./prog`；把计数器用 `__attribute__((aligned(64)))` 对齐到独立缓存行。

22.6 关键要点

- **"共享"取决于实例而非声明**：变量 x 是共享的，当且仅当它的**某个实例**被多于一个线程引用——局部 `static` 变量是共享的，`errno` 不是。
- **把不安全区域排除在轨迹之外就是互斥**：`L/U/S` 三条指令之间的窗口是不安全区域的几何来源，mutex 制造的禁止区域必须完整包住它。
- **信号量的真正价值是"等待事件"，不是"当锁用"**：它比 mutex 慢（实测约 2.1×），只有它能表达计数与顺序；**能用 mutex 就别用信号量**。
- **生产者-消费者三条命**：先等资源后拿锁，先放锁后通知；`mutex` 管 `buf/front/rear`，`slots` / `items` 只管计数，二者不可互相替代。
- **加锁顺序全局一致是避免死锁的第一原则**：循环等待 = 死锁，破坏它的最简单方法就是给所有锁规定一个全序。
- **锁粒度决定可扩展性**：临界区越"串行"，Amdahl 上限越低；实测固定工作量下线程数 1→8，吞吐反而降到 12%。

22.7 思考题（带答案）

题 1（推演题·判断题）：某生产者-消费者程序，缓冲区大小 $n = 4$，三个信号量初值为 `mutex=1`、`slots=4`、`items=0`。生产者代码写成 `sem_wait(&mutex); sem_wait(&slots); ...`，消费者代码写成 `sem_wait(&items); sem_wait(&mutex); ...`。问：(a) 系统是否可能死锁？(b) 若可能，给出最短的死锁触发序列并说明此时各信号量的值。

答：(a) **会死锁**。(b) 生产者先执行 4 次完整插入，4 次后 `slots=0, items=4, mutex=1`，缓冲区满。第 5 次生产者第一步 `sem_wait(&mutex)` 成功（`mutex=0`），第二步 `sem_wait(&slots)` 因 `slots=0` 阻塞，**此时生产者持有 `mutex`**。消费者随后第一步 `sem_wait(&items)` 成功（`items=3`），第二步 `sem_wait(&mutex)` 因 `mutex=0` 阻塞。**状态：`mutex` 被生产者持有且生产者阻塞在 `slots` 上，消费者阻塞在 `mutex` 上；`slots` 只能由消费者增加，而消费者进不去临界区 → 循环等待，永久死锁。**若消费者在缓冲区满之前取走了 item 则不触发——这是**取决于调度**的 bug，最是危险。

题 2（推演题·饥饿分析）：给定 22.2.6 的第一类读者-写者代码（读者优先），假设读者**源源不断**到达，且总是能在任何写者被唤醒之前完成"`wait(mutex); readcnt++; ...; readcnt--; post(mutex)`"。请证明写者会饥饿；并给出一个能打破饥饿、且**不破坏读者并发性**的改法。

答：只要写者准备 `sem_wait(&w)` 时 `readcnt > 0`，它就必须等待；而读者源源不断到达，新读者看到 `readcnt>0` 就**不会**去碰 `w`（只有"第一个读者"才 `wait(&w)`），因此 `readcnt` 永远回不到 0，`w` 永不释放——**写者饥饿（starvation）**。改法：引入 **turnstile（旋转门）**信号量，初值 1。读者先 `sem_wait(&turnstile)`，登记 `readcnt`（第一个读者抢 `w`），**随后立刻** `sem_post(&turnstile)`；写者先 `sem_wait(&turnstile)`，抢 `w`，写完再 `sem_post(&turnstile)`。效果：写者一到就关门，**后续读者全部排在其后**（不再饥饿），而已进入的读者仍可并发。**注意**：若把读者写成"进入 `sem_wait(&w)`、退出 `sem_post(&w)`"，读者之间就完全串行——那是"能跑但性能糟糕"的错误设计。

题 3（"想法错在哪"）：小张认为"既然 `slots` 和 `items` 已经把生产者和消费者管得严严实实——有空位才能放、有 item 才能取——那 `mutex` 完全是多余的，删掉能提高性能"。这个想法错在哪？请给出一个具体的最短反例。

答：错在把"**计数正确**"当成了"**数据访问正确**"。`slots` / `items` 保证"什么时候可以动"，**不保证"动的时候没有别人同时在动"**。反例：$n = 4$、`slots=2`，两个生产者 P1、P2 同时 `sem_wait(&slots)` 各拿到一个空位（两步被原子串行化），随后**并发**执行 `sp->buf[(++sp->rear) % n] = item;`。`++sp->rear` 是读–改–写三步，若两者都读到 `rear=3` 则都算出下标 `0`：一个 item 覆盖另一个，而 `items` 却被 `post` 两次——**计数与实际数据永久错位，消费者最终会取到从未插入的槽位或永久阻塞**。同理两个消费者会取出同一槽位两次。结论：**`slots` / `items` 管"能不能进"，`mutex` 管"进去别撞车"，职责正交，缺一不可。**

题 4（计算/推演题·性能）：某临界区代码本身耗时 C，加锁/解锁开销为 L（$L \ll C$），程序其余部分可完美并行。若单线程执行总时间为 $T_1 = C + O$（O 为可并行部分），用 Amdahl 定律写出 N 线程的加速比，并解释 22.3.5 中实测"线程数 1→8 吞吐率降到 1/8"说明了什么。

答：串行比例 $p = C/(C+O)$，N 线程加速比 $S(N) = 1/(p + (1-p)/N)$，上界 $S(\infty) = 1/p = (C+O)/C$。当临界区占主导（$p \to 1$）时 $S(N) \to 1$，**加更多线程完全无益**。22.3.5 的实测更严重：吞吐不升反降到 $1/8$，说明瓶颈不是"串行比例"而是**锁本身的竞争开销**——`lock xadd` 要独占缓存行、让其他核心副本失效，N 个核心互相弹来弹去（cache line ping-pong），协调成本随 N 增长。因此"锁竞争（lock contention）"比 Amdahl 定律更悲观：**临界区极小且竞争激烈时，加线程会主动降低性能**。对策是缩小临界区、用读者–写者锁分离读写、用原子操作替代锁，或按数据分片（sharding）消除共享。

Lecture 23: 同步：高级 (Synchronization: Advanced)

> **讲义对应**：CMU 15–213 Lecture 23 — Synchronization: Advanced（素材：`F25-23-sync-advanced.txt`） **教材对应**：CS:APP3e 第 12 章 12.5.4–12.5.5、12.7–12.8 **关联 Lab**：**L8 SFS Lab**（并直接支撑 L7 Proxy Lab 的并发缓存）

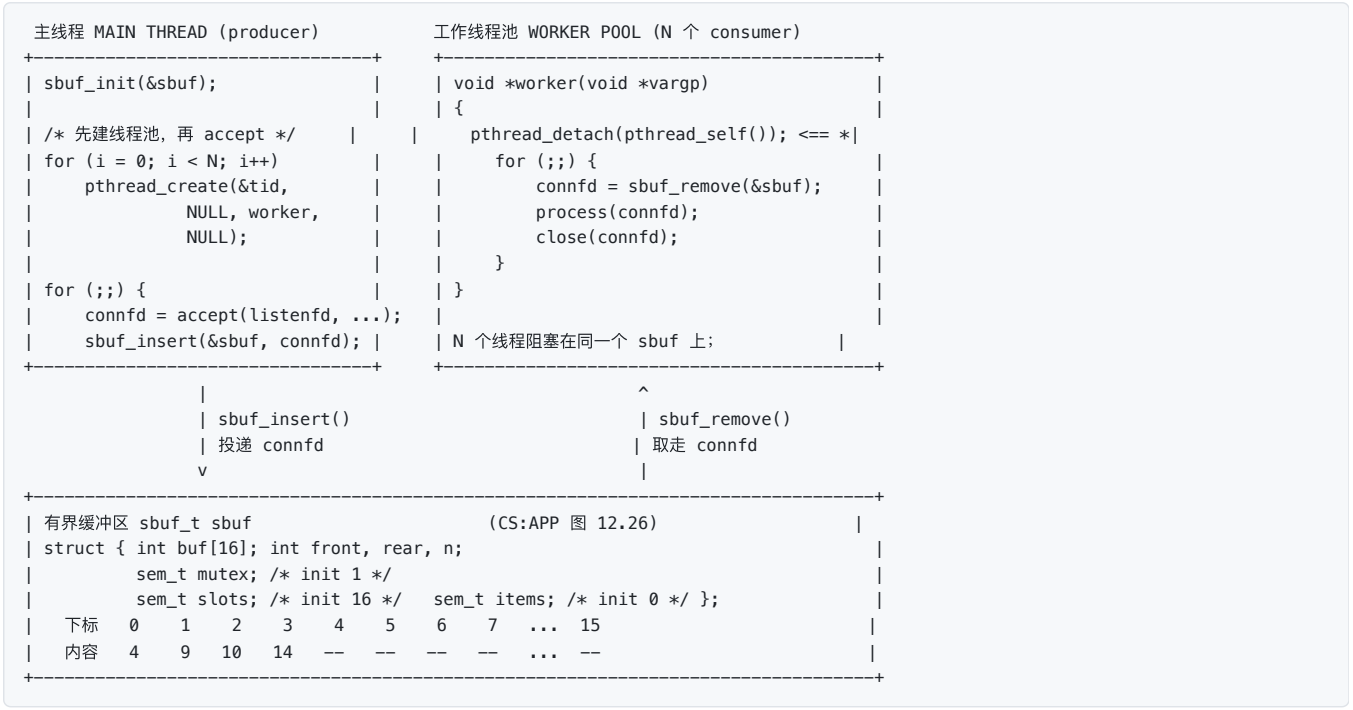
23.1 概述

互斥只是同步的起点：一旦出现时序依赖、多把锁嵌套获取、反复调用的库函数，就会冒出**竞争（race）**、**死锁（deadlock）**、**饥饿（starvation）**三类更隐蔽的故障。本讲把同步从"会用锁"推进到"会设计并发系统"，直接决定 SFS Lab 里"临界区识别"的成败——SFS 的超级块、inode 位图、目录条目全是共享资源，锁错一处文件系统就会静默损坏。

23.2 核心概念与底层机制图解

23.2.1 预线程化并发服务器（Prethreading）

- **定义与目的**：主线程在 `accept` 之前**先创建一族固定数量的工作线程（worker pool）**，再用**有界缓冲区（bounded buffer）**把连接的 `connfd` 传给它们。它消除"每连接一线程"的两个缺陷：每条新连接都要付一次 `pthread_create`（内核复制 task_struct 并分配栈），以及并发无上限时线程总数爆炸。像餐厅后厨——主线程只领位，N 位厨师谁空了谁自己去候客区领。
- **底层机制图解**：



环形下标与"空槽位/条目"计数：

- **与硬件的对应**：`sem_wait` / `sem_post` 在 glibc 里走 futex——无竞争时只在用户态做一次原子 `lock xadd`，有竞争时才 `syscall 202 (futex)` 挂起。
- **关键细节：工作线程必须自己 `pthread_detach(pthread_self())`**。它"永久循环、永不返回"，主线程从不 `join`；若不 detach，线程退出时其 TCB 与栈**永不回收**。由线程自己调用最稳妥——主线程拿到 `tid` 时它可能已退出。
- **与"每连接一线程"的对比**：`pthread_create` 开销从"每连接一次"降为"启动时一次"，并发上限从"无限"变为"可控的 N"，并天然获得**限流**；代价是多一次入队/出队。

23.2.2 线程安全与可重入（Thread Safety and Reentrancy）

- **定义**：函数**线程安全（thread-safe）**，当且仅当它**被多个并发线程反复调用时总能产生正确结果**——该定义同时覆盖单次调用内部的竞态与跨调用的状态污染。
- **四类线程不安全函数**：

类别	根本原因	典型例子	修复方法	代价
第 1 类	不保护共享变量	`badcnt` 里的 `cnt++`；自写 `malloc` 空闲链表	函数首尾加 `pthread_mutex_lock/unlock`	加锁降性能；多把锁引入死锁
第 2 类	状态跨越多次调用保持	`rand`（依赖全局 `next`）、`strtok`、`srand`	状态改为调用者传入的参数（`rand_r(nextp)`）	**必须改 API**，调用者负责分配状态空间
第 3 类	返回指向静态变量的指针	`ctime`、`asctime`、`localtime`、`gethostbyname`、`inet_ntoa`、`itoa`	lock-and-copy：锁内调用并把结果拷到调用者私有存储；或改用 `strftime` / `getaddrinfo`	需改签名；调用者要知道缓冲多大
第 4 类	调用了上述三类中任一个	任何内部用了 `rand` / `strtok` 的函数（传递性）	只调用线程安全版本（`_r` 后缀）	可能触发连锁 API 改动

第 2、3 类的要害是**加锁治不好**：给 `rand` 加锁，两线程各调 100 次的序列仍与"单线程调 200 次"不同。第 3 类更微妙：`char *s = ctime(&t);` 拿到指针后锁已释放，**从解锁到使用 `s` 之间存在窗口**，别的线程任何一次 `ctime` 都会覆盖该缓冲区。 – **可重入与线程安全**：函数**可重入（reentrant）**，当

且仅当它被多线程调用时**不访问任何共享变量**，故不需要同步操作，**可重入 ⇒ 线程安全**；反之不成立（加锁的 `rand` 仍改全局 `next`）。**显式可重入**指函数内部完全不碰共享变量；**隐式可重入**指改成局部变量后**再通过指针参数由调用者传入**（如 `rand_r(&seed)`）。

23.2.3 竞争（Races）

- **定义**：竞争发生在一个程序的**正确性依赖于一个线程要在另一个线程到达 y 点之前到达 x 点**——这涵盖一切时序假设，比"多线程写同一变量"更宽。
- **互斥锁解决不了的三类竞争**：① `cnt++`（无锁时两线程各加 10 000 次结果不到 20 000，因为 `addq $1, cnt(%rip)` 是读-改-写三步）——这类**可以**加锁修好。② `pthread_create(&tid[i], NULL, thread, &i)` 把 `i` 的**地址**传给线程，等线程真正读 `*(int *)vargp` 时主线程早已把 `i` 推得更大；锁改变不了"传的是地址"，必须**拷贝数据**（改传值 `(void *)i`）。③ TOCTOU（time of check to time of use）：先 `access("myfile.txt", R_OK)` 再 `fopen`，两次调用之间文件可能已被删；修复原则是"**不要先检查，直接使用并处理失败**"。信号处理程序与主程序之间同样有 TOCTOU。
- **三条总策略**：① **不共享状态**（`malloc` 给每线程复制参数）；② **不检查就使用**；③ **使用同步原语**。检测首选 `ThreadSanitizer`（`gcc -fsanitize=thread`），其次 `valgrind --tool=helgrind` / `--tool=drd`。

23.2.4 死锁（Deadlock）与进度图

- **定义**：程序死锁，当它在**等待一个永远不可能发生的事件**——讲义强调这是**数学意义上的不可能，而不只是实践上的困难**。常见形式：两线程互相等待，成因是**加锁顺序不一致（inconsistent lock ordering）**。
- **进度图（progress graph）**：把两线程各自执行到的"指令数"当横纵坐标，状态空间是二维网格。某区域对锁 a 而言**禁止（forbidden region）**，表示"两线程同时持有 a"——互斥锁语义直接排除。**死锁区域（deadlock region）就是两条禁止带的交集**：进入该区域的轨迹停在死锁状态，无法向上或向右移动。

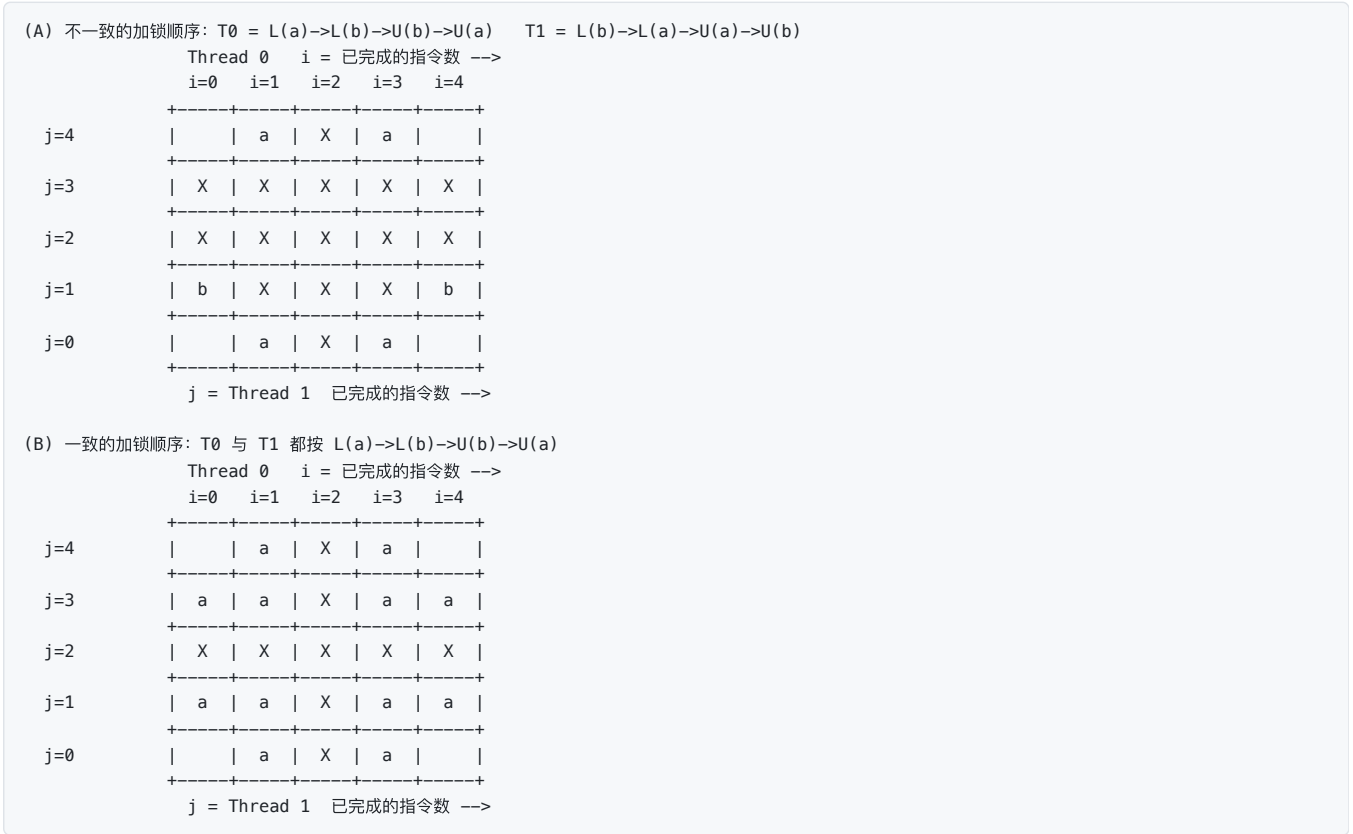


图 A 的死锁状态是 `(i,j) = (1,1)`：T0 持 a 等 b、T1 持 b 等 a，**向右走落入"禁止区域 for b"、向上走落入"禁止区域 for a"，被彻底卡住**。图 B 两条禁止带不再相交，死锁区域消失；代价是整个 3×3 内部方块全属 a 的禁止区域，**任何交错轨迹都进不去，两线程被强制串行化**。轨迹的微小差异让程序"有时死、有时不死"——**死锁 bug 是不确定的（nondeterministic）**。 – **避免死锁的规则清单**： 1. **加锁顺序（lock ordering）**：约定全局唯一的锁编号，**所有线程一律按编号从小到大加锁（解锁顺序无所谓）**。 2. **减小临界区中同时持有的锁数**：能合并成一把（粗粒度）就合并，牺牲并行度换确定性。 3. **用 `pthread_mutex_trylock` 或带超时加锁**：失败就释放已持有的锁再重试；可能引入活锁，需要退避（backoff）。 4. **消除 hold-and-wait**：把"先申请 A 再申请 B"改成"一次性申请全部"或"申请前先释放"。 5. **不要从持有锁的代码里回调用户代码**（回调可能去拿别的锁）。 6. **不要从信号处理程序里调用会加锁的库函数**（见 23.5）。 – **哲学家就餐问题**：5 位哲学家围坐，每人需同时拿到左右两把叉子。若所有人同时先拿左边叉子，就形成环形 hold-and-wait，全部饿死。解法之一是给叉子编号并规定"先拿编号小的"。

23.2.5 条件变量（Condition Variable）

- **定义与目的**：条件变量让线程**在某个条件为假时挂起，并在条件可能变真时被唤醒**，解决"轮询忙等浪费 CPU"的问题。它**自身不提供互斥**，总与一把互斥锁配对使用。
- **接口表**：

接口	语义	备注
`pthread_cond_init(pthread_cond_t *c, const pthread_condattr_t *a)`	动态初始化（`a` 传 `NULL` 用默认属性）	也可用静态初始化器 `PTHREAD_COND_INITIALIZER`
`pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m)`	**原子地**：释放 `m` → 把本线程加入 `c` 的等待队列并挂起 → 被唤醒后**重新获取** `m` → 返回	调用前必须已持有 `m`；返回时也持有 `m`
`pthread_cond_signal(pthread_cond_t *c)`	唤醒 `c` 等待队列中的**至少一个**线程	不保证唤醒谁，也不保证立即运行
`pthread_cond_broadcast(pthread_cond_t *c)`	唤醒 `c` 等待队列中的**全部**线程	用于条件变化影响所有等待者的场景
`pthread_cond_destroy(pthread_cond_t *c)`	销毁条件变量	必须在无等待者时调用

- **`pthread_cond_wait` 的三步语义**：核心是把"释放锁"与"挂起"做成**一个原子操作**——若两步之间有窗口，通知者就能发出"谁也收不到"的信号，让等待者永远睡下去（经典的 lost wakeup）。醒来时必须重新获得互斥锁，因此从 `wait` 返回到继续执行的区间里，**锁可能已多次易主、条件可能已被改回假**。
- **必须用 `while` 而不是 `if`**：

```
  CONSUMER C (等待者)                          PRODUCER P (通知者)
                                                  |
  pthread_mutex_lock(&m);                         | pthread_mutex_lock(&m);
  while (count == 0) {        <== 必须 while      | /* 拿到 C 刚空出的锁 */
    pthread_cond_wait(&cv,&m) {                   |
     /*1*/ 原子地 unlock(&m)                      | count = 1;
     /*2*/ 加入 cv 等待队列                       | buf[0] = item;
     /*3*/ park, 不占 CPU                         |
                                                  | pthread_cond_signal(&cv);
     /*4*/ 被唤醒                                 | /* 把 C 挪到 mutex 队列 */
           -> 重新 lock(&m)                       |
     /*5*/ wait 返回                              | pthread_mutex_unlock(&m);
    }     <== 锁可能已易主,                       |
  }           count 可能又变成 0                  | signal 只是"通知", 不是"交接":
                                                  | C 必须重新竞争锁,
  item = buf[--count];                            | 并且必须回到 while
  pthread_mutex_unlock(&m);                       | 再检查一次条件。
```

用 `if` 会出错的三种情形：① **虚假唤醒（spurious wakeup）**——`pthread_cond_wait` 允许在无 `signal` 时返回（POSIX 明确许可），`if` 版会直接往下走读到非法数据；② **`broadcast` 惊群**——唤醒多个消费者却只有一份数据；③ **条件被抢先改变**——别的线程可能在你重新拿到锁之前把条件改回假。`while` 把"重新检查"变成结构性保证。**用错 `signal` 会导致 lost wakeup**。 – **用条件变量实现屏障与生产者–消费者**：屏障用"计数器 + `broadcast`"：每线程到达时 `count++`；若 `count < N` 则 `wait`，否则清零并 `broadcast`。

23.2.6 信号量的更多用法：事件、队列与屏障

- **与互斥锁的本质区别**：信号量是**非负整数**，支持 P（`sem_wait`，减 1，为 0 则等待）与 V（`sem_post`，加 1 并可能唤醒一个等待者）。关键在于**任何线程都可以执行 P 和 V，没有所有权约束**——互斥锁只有加锁者能解锁。这正是它能做**事件通知**的原因："Can't do this with a mutex. Why? Only thread that locked the mutex can unlock it."
- **队列**：1 槽位队列只需 `full` 与 `empty` 两个信号量（初值 0 和 1）。**为什么 1 个槽位却要两个信号量？** 因为生产者之间争夺空槽位、消费者之间争夺条目，两类等待的**原因不同**，必须放在不同信号量上排队，否则会互相误导。n 槽位队列需 `mutex`（保护内部结构）+ `slots`（空槽位，初值 n）+ `items`（条目，初值 0）；**必须先 P 计数信号量再 P 互斥量**，否则会以"拿着锁去等待"的方式死锁。

- **屏障**：简版"最后一个到达者放行所有人"**不能循环复用**（下一轮会立刻穿过去）。可复用版本用**两阶段 + sense 翻转**：每线程为其余 N−1 个线程各 V 一次、自己最后 P 一次；再用 `phase` 在两个闸门信号量间交替，避免上一轮的 V 泄漏到下一轮。

23.2.7 读者–写者锁与饥饿

- **问题陈述**：读者只读取对象，写者修改对象；**写者必须独占，读者可无限多个同时访问**。常见于机票预订系统、多线程缓存代理（Proxy Lab）。
- **Pthreads 接口**：`pthread_rwlock_t`；`pthread_rwlock_rdlock`（读锁）、`pthread_rwlock_wrlock`（写锁）、`pthread_rwlock_unlock`（共用同一解锁函数）。**必须正确使用**——"什么操作需要读、什么需要写"完全由程序员判断，这正是 Proxy Lab 里"读缓存算读、插入或淘汰算写"的判断题。
- **饥饿（starvation）**：线程长时间得不到前进。与死锁的关键区别是**饥饿可能最终解套，死锁在数学上不可能解套**。
- **读优先 vs 写优先的两难**：设线程 1 持读锁、线程 2 等写锁，此时线程 3 申请读锁——"读者优先"会让源源不断的读者饿死 W；"写者优先"则让源源不断的写者饿死读者。**保证无饥饿的算法称为公平（fair）**，它让等待者按**先来先服务（FCFS）**获得锁；代价是公平可能让**所有**线程都更慢——"锁护送（lock convoy）问题"。

23.2.8 性能与可扩展性

- **加锁的开销与锁竞争**：无竞争时一次 `pthread_mutex_lock` 只需几十纳秒；一旦竞争，失败的线程要陷入内核挂起、稍后唤醒再重新调度，单次代价上升三到四个数量级。因此**性能的决定因素不是"有没有锁"，而是"有多少竞争"**。
- **为什么"一个全局锁"会成为瓶颈（Amdahl 定律）**：设临界区占单线程执行时间的比例为 f，则 N 个线程的加速比上限为

$$S(N) = \frac{1}{f + \frac{1-f}{N}} \xrightarrow{N\to\infty} \frac{1}{f}$$

若 $f = 0.5$，加多少核加速比都超不过 2。更糟的是全局锁还引入额外**串行化开销**（临界区被拉长、cache line 在核间弹跳）——这正是"核越多反而越慢"的原因：

```
  吞吐量对比（Kops/s，临界区 200 ns，8 核机器）
                全局锁（1 把）                          分条带锁（16 把）

1 核   4125 ####################            4123 ====================
2 核   1929 #########                       5076 ========================
4 核   1001 #####                           6304 ==============================
8 核    454 ##                              3365 ================

    1 核 -> 8 核：全局锁吞吐 4125 -> 454（只剩 11%，负扩展）
                  分条带锁 4123 -> 3365 峰值出现在 4 核（6304）
```

全局锁从 4125 单调跌到 454 Kops/s（**负扩展**）；分条带锁（按 inode 编号散列到 16 把锁）在 4 核达峰值 6304 Kops/s，是全局锁的 6 倍以上，直到 8 核才因跨条带竞争与内存带宽回落。 – **粗粒度锁 vs 细粒度锁**：粗粒度锁（如"锁住整棵 BST"）实现简单、不会因加锁顺序而死锁，但把可并行的操作串行化；细粒度锁（如"每节点一把锁"）能暴露并行性，却引入"加锁顺序"与"锁数量爆炸"的新问题。递归地缩小共享资源范围——整棵树 → 两条分支 → 单个节点——是折中的标准路径。 – **观察工具**：`perf record` / `perf report` 定位热点锁；`valgrind --tool=helgrind` / `--tool=drd` 报告数据竞争与加锁顺序违规；`ThreadSanitizer` 比 helgrind 快得多。

23.2.9 内存序与原子操作（补充说明）

编译器与 CPU 都可能重排内存访问（编译期重排、Store Buffer、乱序执行），因此**用普通变量做跨线程标志位不可靠**：`while (!flag) ;` 可能被优化成只读一次寄存器而死循环。`volatile` 只阻止**编译器**优化掉/合并访问，**不提供原子性与内存序保证**（见 23.5）；正确工具是 `__sync_*` 与 `__atomic_*`（可指定 `memory_order_relaxed/acquire/release/seq_cst`，对应 C11 `stdatomic.h`）。互斥锁的加解锁隐含 acquire/release 语义，这正是"临界区内的写在解锁前对下一个加锁者可见"的硬件基础；**无锁（lock-free）**编程用 CAS（`__atomic_compare_exchange_n`）直接在原子变量上线性化，代价是自行处理 ABA 问题与内存回收。

23.3 代码示例与底层机制分析

以下示例均已在 `/tmp` 下用 `gcc -g -Wall -std=c11 -lpthread` 真实编译并运行（零 warning/error），输出为真实结果。

23.3.1 预线程化并发服务器（worker pool + sbuf）

代码 (C)：

```c
/* prethreaded.c —— 预线程化回显服务器
 * gcc -g -Wall -std=c11 -lpthread prethreaded.c -o prethreaded
 * ./prethreaded 12345 4 8     # 端口、工作线程数、并发客户端数
 */
#define _GNU_SOURCE
#include <arpa/inet.h>
#include <netinet/in.h>
#include <pthread.h>
#include <semaphore.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define MAXLINE  1024
#define SBUFSIZE 16

typedef struct {                    /* 有界缓冲区（CS:APP 图 12.26） */
    int   buf[SBUFSIZE];
    int   front, rear, n;
    sem_t mutex, slots, items;
} sbuf_t;

static sbuf_t sbuf;

static void sbuf_init(sbuf_t *sp)
{
    sp->front = sp->rear = sp->n = 0;
    sem_init(&sp->mutex, 0, 1);
    sem_init(&sp->slots, 0, SBUFSIZE);
    sem_init(&sp->items, 0, 0);
}
static void sbuf_insert(sbuf_t *sp, int item)     /* 生产者：主线程 */
{
    sem_wait(&sp->slots);            /* ① 等一个空槽位 */
    sem_wait(&sp->mutex);            /* ② 加锁写结构 */
    sp->buf[sp->rear] = item;
    sp->rear = (sp->rear + 1) % SBUFSIZE;
    sp->n++;
    sem_post(&sp->mutex);            /* ③ 解锁 */
    sem_post(&sp->items);            /* ④ 通告"多了一个条目" */
}
static int sbuf_remove(sbuf_t *sp)                /* 消费者：工作线程 */
{
    int item;
    sem_wait(&sp->items);            /* ① 等一个条目 */
    sem_wait(&sp->mutex);            /* ② 加锁读结构 */
    item = sp->buf[sp->front];
    sp->front = (sp->front + 1) % SBUFSIZE;
    sp->n--;
    sem_post(&sp->mutex);
    sem_post(&sp->slots);            /* ③ 通告"多了一个空槽位" */
    return item;
}

static pthread_mutex_t stat_lock = PTHREAD_MUTEX_INITIALIZER;
static int served = 0, total = 0;

static void process(int connfd)
{
    char buf[MAXLINE], out[MAXLINE + 8];
    ssize_t n = read(connfd, buf, sizeof(buf) - 1);
    if (n <= 0) return;
    buf[n] = '\0';
    snprintf(out, sizeof(out), "ECHO: %s", buf);
    write(connfd, out, strlen(out));

    int v = atoi(buf);
    pthread_mutex_lock(&stat_lock);         /* 第 1 类不安全函数的修复：加锁 */
    served++; total += v;
    printf("[worker] connfd=%-3d value=%-4d served=%d total=%d\n",
           connfd, v, served, total);
    fflush(stdout);
```

```
    pthread_mutex_unlock(&stat_lock);
}

static void *worker(void *vargp)
{
    pthread_detach(pthread_self());          /* ★ 每个工作线程自我分离 */
    (void)vargp;
    for (;;) {
        int connfd = sbuf_remove(&sbuf);     /* 阻塞等待一个连接 */
        process(connfd);
        close(connfd);
    }
    return NULL;
}

static int g_port, g_nclient;

static void *one_client(void *vargp)         /* 内置客户端，制造并发连接 */
{
    long id = (long)vargp;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in a; memset(&a, 0, sizeof(a));
    a.sin_family = AF_INET;
    a.sin_port = htons((unsigned short)g_port);
    inet_pton(AF_INET, "127.0.0.1", &a.sin_addr);
    if (connect(fd, (struct sockaddr *)&a, sizeof(a)) < 0) { close(fd); return NULL; }

    char msg[32], buf[MAXLINE];
    int len = snprintf(msg, sizeof(msg), "%ld\n", id * 100);
    write(fd, msg, len);
    ssize_t n = read(fd, buf, sizeof(buf) - 1);
    if (n > 0) { buf[n] = '\0'; printf("[client %ld] %s", id, buf); fflush(stdout); }
    close(fd);
    return NULL;
}

static void *client_driver(void *vargp)
{
    (void)vargp;
    struct timespec ts = {0, 300 * 1000 * 1000};   /* 让主线程先进 accept */
    nanosleep(&ts, NULL);
    pthread_t ct[64];
    for (long i = 0; i < g_nclient; i++) pthread_create(&ct[i], NULL, one_client, (void *)(i + 1));
    for (int i = 0; i < g_nclient; i++) pthread_join(ct[i], NULL);
    return NULL;
}

int main(int argc, char **argv)
{
    int port    = (argc > 1) ? atoi(argv[1]) : 12345;
    int nthread = (argc > 2) ? atoi(argv[2]) : 4;
    int nclient = (argc > 3) ? atoi(argv[3]) : 8;
    signal(SIGPIPE, SIG_IGN);

    int listenfd = socket(AF_INET, SOCK_STREAM, 0);
    int one = 1;
    setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &one, sizeof(one));
    struct sockaddr_in addr; memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons((unsigned short)port);
    bind(listenfd, (struct sockaddr *)&addr, sizeof(addr));
    listen(listenfd, 1024);
    printf("listening on %d with %d worker threads\n", port, nthread); fflush(stdout);

    sbuf_init(&sbuf);

    /* ★★ 预线程化：主线程先建好一族工作线程，然后才开始 accept ★★ */
    pthread_t tid;
    for (int i = 0; i < nthread; i++)
        pthread_create(&tid, NULL, worker, NULL);

    g_port = port; g_nclient = nclient;
    pthread_t drv;
    pthread_create(&drv, NULL, client_driver, NULL);
```

```
    int accepted = 0;
    for (int i = 0; i < nclient; i++) {      /* 主线程 = 生产者 */
        struct sockaddr_in caddr; socklen_t clen = sizeof(caddr);
        int connfd = accept(listenfd, (struct sockaddr *)&caddr, &clen);
        if (connfd < 0) break;
        printf("[main] accepted connfd=%d\n", connfd); fflush(stdout);
        sbuf_insert(&sbuf, connfd);
        accepted++;
    }
    pthread_join(drv, NULL);
    sleep(1);
    printf("[main] accepted=%d served=%d total=%d\n", accepted, served, total);
    close(listenfd);
    return 0;
}
```

【代码做什么？】 `main` 建监听套接字并用 `sbuf_init` 初始化三个信号量（`mutex`=1、`slots`=16、`items`=0），**先**创建 4 个工作线程、**再**进入 `accept` 循环把 `connfd` 插入队列；每个工作线程 `pthread_detach(pthread_self())` 后循环取出 `connfd`、回显、累加受锁保护的统计量、关闭连接。

【底层机制透视】 `accept` 返回**最小的可用文件描述符**，`close` 后该号立刻被回收再分配——这就是日志里 `connfd=4` 出现两次的原因。`pthread_detach` 设置 TCB 的不可 join 状态，线程退出时内核直接回收栈与 TCB。`served` / `total` 由**同一把锁**保护；不加锁时 `served++` 拆成三步，输出会重复计数。

【内存布局 / 数据结构图解】 `sbuf` 在**全局静态存储区**（`.bss`）被所有线程共享；工作线程的 `connfd`、`v`、`buf[MAXLINE]` 都在**线程栈上**，天然线程安全。

【与汇编 / 硬件的对应】 `cnt++` 是读–改–写三步（`gcc -S -O0`）：

```
        movq    cnt(%rip), %rax      # 读
        addq    $1, %rax             # 改
        movq    %rax, cnt(%rip)      # 写
```

在 `-O1` 下编译器把它优化成**单条** `addq $1, cnt(%rip)`——但这仍不是原子操作（x86 的 `add` 到内存不隐含 `lock` 前缀），所以**开优化并不能修好竞争**。

【实测验证】（真实输出，节选）

```
$ ./prethreaded 12400 4 8
listening on 12400 with 4 worker threads
[main] accepted connfd=4
[main] accepted connfd=9
[main] accepted connfd=10
[client 1] ECHO: 100
[main] accepted connfd=14
[worker] connfd=9   value=200  served=1 total=200
[worker] connfd=4   value=100  served=2 total=300
[client 3] ECHO: 300
[main] accepted connfd=5
[client 4] ECHO: 400
[client 2] ECHO: 200
[main] accepted connfd=4
[client 6] ECHO: 600
[main] accepted connfd=6
[client 5] ECHO: 500
[worker] connfd=10  value=300  served=3 total=600
[main] accepted connfd=7
[worker] connfd=14  value=400  served=4 total=1000
[worker] connfd=5   value=600  served=5 total=1600
[worker] connfd=4   value=500  served=6 total=2100
[worker] connfd=6   value=700  served=7 total=2800
[client 7] ECHO: 700
[worker] connfd=7   value=800  served=8 total=3600
[client 8] ECHO: 800
[main] accepted=8 served=8 total=3600
```

三个特征：① `connfd=4` **出现两次**——描述符被回收后重新分配；② 8 条连接被 4 个 worker **交错处理**（`connfd=9` 先完成而 `connfd=4` 后完成），证明真并发；③ 并发写统计量最终 `served=8`、`total=3600` 完全正确——加锁生效。

23.3.2 死锁演示（相反加锁顺序）

代码 (C)：

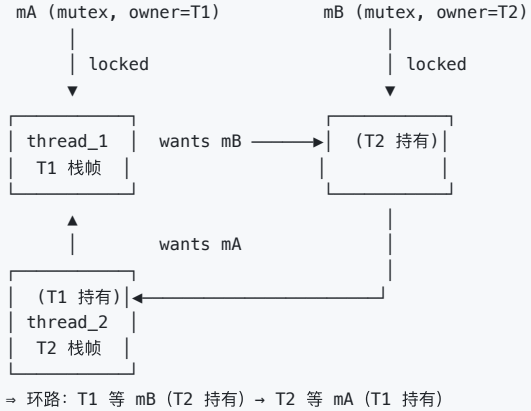
```c
/* b_deadlock.c —— 相反加锁顺序导致死锁
 * gcc -g -Wall -std=c11 -lpthread b_deadlock.c -o b_deadlock
 * ./b_deadlock 0   # 不一致顺序：真死锁 -> timeout 5 退出码 124
 * ./b_deadlock 1   # 一致顺序：安全，退出码 0
 * ./b_deadlock 2   # 不一致顺序 + pthread_timedjoin_np 自检：退出码 3
 */
#define _GNU_SOURCE
#include <errno.h>
#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

static pthread_mutex_t mA = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t mB = PTHREAD_MUTEX_INITIALIZER;
static atomic_int done_flag = 0, in_cs = 0;
static int g_mode = 0;

/* 让两个线程在拿到第一把锁之后精确对齐，使死锁可确定复现 */
static void rendezvous(void)
{
    struct timespec ts = {0, 30 * 1000 * 1000};   /* 30 ms */
    nanosleep(&ts, NULL);
}

static void *thread_1(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&mA);
    rendezvous();
    printf("[T1] holds mA, now asks for mB ...\n"); fflush(stdout);
    pthread_mutex_lock(&mB);                      /* ← 若 T2 持有 mB 则卡死 */
    printf("[T1] acquired BOTH locks (in_cs=%d)\n", atomic_fetch_add(&in_cs, 1) + 1);
    fflush(stdout);
    atomic_fetch_sub(&in_cs, 1);
    pthread_mutex_unlock(&mB);
    pthread_mutex_unlock(&mA);
    atomic_fetch_add(&done_flag, 1);
    return NULL;
}

static void *thread_2(void *arg)
{
    (void)arg;
    if (g_mode == 0 || g_mode == 2) {
        pthread_mutex_lock(&mB);                  /* ★ 相反顺序：先 B 后 A */
        rendezvous();
        printf("[T2] holds mB, now asks for mA ...\n"); fflush(stdout);
        pthread_mutex_lock(&mA);                  /* ← 若 T1 持有 mA 则卡死 */
    } else {
        pthread_mutex_lock(&mA);                  /* ★ 一致顺序：先 A 后 B */
        rendezvous();
        printf("[T2] holds mA, now asks for mB ...\n"); fflush(stdout);
        pthread_mutex_lock(&mB);
    }
    printf("[T2] acquired BOTH locks (in_cs=%d)\n", atomic_fetch_add(&in_cs, 1) + 1);
    fflush(stdout);
    atomic_fetch_sub(&in_cs, 1);
    pthread_mutex_unlock(&mB);
    pthread_mutex_unlock(&mA);
    atomic_fetch_add(&done_flag, 1);
    return NULL;
}

static int timed_join(pthread_t t, int timeout_sec)
{
    struct timespec deadline;
    clock_gettime(CLOCK_REALTIME, &deadline);
    deadline.tv_sec += timeout_sec;
    return pthread_timedjoin_np(t, NULL, &deadline);
}

int main(int argc, char **argv)
```

```
{
    g_mode = (argc > 1) ? atoi(argv[1]) : 0;
    printf("mode=%d (%s)\n", g_mode,
           g_mode == 1 ? "consistent order: every thread locks A then B"
                       : "inconsistent order: T1 does A->B, T2 does B->A");
    fflush(stdout);

    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_1, NULL);
    pthread_create(&t2, NULL, thread_2, NULL);

    if (g_mode == 1) {
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
        printf("OK: no deadlock, done_flag=%d, in_cs=%d\n",
               atomic_load(&done_flag), atomic_load(&in_cs));
        return 0;
    }
    int rc1 = timed_join(t1, 3);
    int rc2 = rc1 ? ETIMEDOUT : timed_join(t2, 3);
    if (rc1 == ETIMEDOUT || rc2 == ETIMEDOUT) {
        printf("*** DEADLOCK: done_flag=%d, threads blocked waiting for each other "
               "(rc1=%d rc2=%d) ***\n", atomic_load(&done_flag), rc1, rc2);
        fflush(stdout);
        if (g_mode == 0) for (;;) pause();        /* 真挂起，留给 timeout/gdb */
        _exit(3);
    }
    printf("no deadlock: done_flag=%d\n", atomic_load(&done_flag));
    return 0;
}
```

【代码做什么？】 `thread_1` 固定按 `mA → mB` 加锁，`thread_2` 在 mode 0/2 下按 `mB → mA`（相反顺序）、mode 1 下按 `mA → mB`（一致顺序）；`rendezvous()` 的 30 ms 睡眠保证两线程都已在"持有一把、等另一把"的位置对齐，把偶发的死锁变成 **100% 可复现**。

【底层机制透视】 `pthread_mutex_lock` 竞争时先做原子 CAS 尝试（fast path），失败后进入 `__lll_lock_wait` 用 futex 挂起。死锁时两线程都停在内核 futex 等待队列里，`/proc/<pid>/status` 的 `State:` 为 `S (sleeping)`、`Threads:` 为 3——**没有线程消耗 CPU**，这是死锁与"活锁/自旋"的可观测区别。

【内存布局 / 数据结构图解】 死锁时锁的归属形成环路：



【实测验证】 三种模式的真实运行结果：

```
$ ./b_deadlock 1
mode=1 (consistent order: every thread locks A then B)
[T1] holds mA, now asks for mB ...
[T1] acquired BOTH locks (in_cs=1)
[T2] holds mA, now asks for mB ...
[T2] acquired BOTH locks (in_cs=1)
OK: no deadlock, done_flag=2, in_cs=0
exit=0

$ timeout 5 ./b_deadlock 0; echo "exit=$?"
mode=0 (inconsistent order: T1 does A->B, T2 does B->A)
[T1] holds mA, now asks for mB ...
[T2] holds mB, now asks for mA ...
*** DEADLOCK: done_flag=0, threads blocked waiting for each other (rc1=110 rc2=110) ***
exit=124          # 124 = 被 timeout 杀掉 ⇒ 进程真的卡死了

$ ./b_deadlock 2; echo "exit=$?"
mode=2 (inconsistent order: T1 does A->B, T2 does B->A)
[T1] holds mA, now asks for mB ...
[T2] holds mB, now asks for mA ...
*** DEADLOCK: done_flag=0, threads blocked waiting for each other (rc1=110 rc2=110) ***
exit=3
```

mode 1 里 `in_cs` 两次都是 1——**没有任何时刻两线程同时进入临界区**，互斥生效。`rc1=110` 正是 `ETIMEDOUT`。`gdb -p` 附到 mode 0 的进程上，栈回溯精确定位两处阻塞点：

```
$ ./b_deadlock 0 & sleep 1.5; PID=$(pgrep -x b_deadlock -n)
$ gdb -p $PID -batch -ex "thread apply all bt"
Thread 3 (LWP 3609910) "b_deadlock":
#0  __lll_lock_wait () from /lib64/libc.so.6
#1  pthread_mutex_lock@@GLIBC_2.2.5 () from /lib64/libc.so.6
#2  thread_2 (arg=0x0) at b_deadlock.c:57        ← 卡在 pthread_mutex_lock(&mA)
#3  start_thread ()
#4  clone3 ()

Thread 2 (LWP 3609909) "b_deadlock":
#0  __lll_lock_wait () from /lib64/libc.so.6
#1  pthread_mutex_lock@@GLIBC_2.2.5 () from /lib64/libc.so.6
#2  thread_1 (arg=0x0) at b_deadlock.c:40        ← 卡在 pthread_mutex_lock(&mB)
#3  start_thread ()
#4  clone3 ()

$ grep -E "^(Name|State|Threads)" /proc/$PID/status
Name:   b_deadlock
State:  S (sleeping)      ← 内核态睡眠，0% CPU
Threads:        3
```

`valgrind --tool=helgrind` 也能直接报出这个模式：

```
$ valgrind --tool=helgrind ./b_deadlock 2
Thread #2: Exiting thread still holds 1 lock
   at __lll_lock_wait ... by thread_1 (b_deadlock.c:40)
Thread #3: Exiting thread still holds 1 lock
   at __lll_lock_wait ... by thread_2 (b_deadlock.c:57)
ERROR SUMMARY: 2 errors from 2 contexts
```

"Exiting thread still holds 1 lock" = 线程带着锁阻塞在另一把锁上，是死锁/锁顺序违规的典型指纹。

23.3.3 条件变量版生产者–消费者（`while` 而非 `if`）

代码 (C)：

```c
/* c_condvar.c —— 条件变量版生产者-消费者
 * gcc -g -Wall -std=c11 -lpthread c_condvar.c -o c_condvar
 * ./c_condvar while   # 正确：while 包裹等待
 * ./c_condvar if      # ⚠️ 仅供演示：if 的 bug
 */
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define NITEMS 20
#define CAPACITY 4
#define NPROD 2
#define NCONS 2

static int use_while = 1;

/* ---- 定向实验：一个条目、两个等待者、broadcast ---- */
static pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t  cv2 = PTHREAD_COND_INITIALIZER;
static int slot = 0;
static int if_wrong_wakeups = 0;

static void *waiter_if(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&m2);
    if (slot == 0)                      /* ❌ if：只检查一次 */
        pthread_cond_wait(&cv2, &m2);
    if (slot == 0) {                    /* 醒来后条件早已不成立 */
        __sync_fetch_and_add(&if_wrong_wakeups, 1);
        printf("[if-waiter] woke up but slot still empty --> check was stale\n");
    } else {
        slot = 0;
        printf("[if-waiter] consumed the only item\n");
    }
    fflush(stdout);
    pthread_mutex_unlock(&m2);
    return NULL;
}

static void *waiter_while(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&m2);
    while (slot == 0)                    /* ✅ while：醒来后重查 */
        pthread_cond_wait(&cv2, &m2);
    slot = 0;
    printf("[while-waiter] consumed the only item\n"); fflush(stdout);
    pthread_mutex_unlock(&m2);
    return NULL;
}

static void directed_experiment(int use_while_local)
{
    printf("\n=== directed experiment: ONE item, TWO waiters, broadcast (%s) ===\n",
           use_while_local ? "while" : "if");
    fflush(stdout);
    if_wrong_wakeups = 0;
    slot = 0;
    pthread_t a, b;
    pthread_create(&a, NULL, use_while_local ? waiter_while : waiter_if, NULL);
    pthread_create(&b, NULL, use_while_local ? waiter_while : waiter_if, NULL);
    struct timespec ts = {0, 200 * 1000 * 1000};
    nanosleep(&ts, NULL);               /* 保证两个等待者都已挂起 */

    pthread_mutex_lock(&m2);
    slot = 1;                           /* 只放入一个条目 */
    pthread_cond_broadcast(&cv2);       /* 两个等待者都被唤醒 */
    pthread_mutex_unlock(&m2);

    if (use_while_local) {              /* while 版必须再放一个才能让第二人拿到 */
        nanosleep(&ts, NULL);
```

```
        pthread_mutex_lock(&m2);
        slot = 1;
        pthread_cond_broadcast(&cv2);
        pthread_mutex_unlock(&m2);
    }
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("--> stale-condition wakeups: %d\n", if_wrong_wakeups);
    fflush(stdout);
}

/* ---- 有界缓冲区本体 ---- */
typedef struct {
    int buf[CAPACITY];
    int count, in, out, produced, consumed, corrupt;
    pthread_mutex_t m;
    pthread_cond_t  not_empty, not_full;
} cb_t;

static cb_t cb;

static void *producer(void *arg)
{
    long id = (long)arg;
    for (int i = 0; i < NITEMS / NPROD; i++) {
        int item = (int)(id * 1000 + i);
        pthread_mutex_lock(&cb.m);
        if (use_while)
            while (cb.count == CAPACITY)                /* ✅ */
                pthread_cond_wait(&cb.not_full, &cb.m);
        else
            if (cb.count == CAPACITY)                   /* ❌ */
                pthread_cond_wait(&cb.not_full, &cb.m);

        cb.buf[cb.in] = item;
        cb.in = (cb.in + 1) % CAPACITY;
        cb.count++; cb.produced++;
        printf("[P%ld] put %4d  count=%d\n", id, item, cb.count); fflush(stdout);
        pthread_cond_signal(&cb.not_empty);
        pthread_mutex_unlock(&cb.m);
    }
    return NULL;
}

static void *consumer(void *arg)
{
    long id = (long)arg;
    for (int i = 0; i < NITEMS / NCONS; i++) {
        pthread_mutex_lock(&cb.m);
        if (use_while) {
            while (cb.count == 0) {
                printf("[C%ld] buffer empty, waiting ...\n", id); fflush(stdout);
                pthread_cond_wait(&cb.not_empty, &cb.m);
            }
        } else {
            if (cb.count == 0) {
                printf("[C%ld] buffer empty, waiting ...\n", id); fflush(stdout);
                pthread_cond_wait(&cb.not_empty, &cb.m);
            }
            if (cb.count == 0) {            /* 检出 if 版的"陈旧条件"唤醒 */
                cb.corrupt++;
                printf("[C%ld] *** BUG: woke up but count==0 ***\n", id); fflush(stdout);
                pthread_mutex_unlock(&cb.m);
                continue;
            }
        }
        int item = cb.buf[cb.out];
        cb.out = (cb.out + 1) % CAPACITY;
        cb.count--; cb.consumed++;
        printf("[C%ld] got %4d  count=%d\n", id, item, cb.count); fflush(stdout);
        pthread_cond_signal(&cb.not_full);
        pthread_mutex_unlock(&cb.m);
    }
    return NULL;
}
```

```
int main(int argc, char **argv)
{
    use_while = !(argc > 1 && strcmp(argv[1], "if") == 0);
    printf("mode=%s\n", use_while ? "while" : "if(buggy)");

    memset(&cb, 0, sizeof(cb));
    cb.m = (pthread_mutex_t)PTHREAD_MUTEX_INITIALIZER;
    cb.not_empty = (pthread_cond_t)PTHREAD_COND_INITIALIZER;
    cb.not_full  = (pthread_cond_t)PTHREAD_COND_INITIALIZER;

    directed_experiment(use_while);

    printf("\n=== bounded buffer: %d producers, %d consumers, capacity %d ===\n",
           NPROD, NCONS, CAPACITY);
    fflush(stdout);
    pthread_t p[NPROD], c[NCONS];
    for (long i = 0; i < NPROD; i++) pthread_create(&p[i], NULL, producer, (void *)i);
    for (long i = 0; i < NCONS; i++) pthread_create(&c[i], NULL, consumer, (void *)i);
    for (int i = 0; i < NPROD; i++) pthread_join(p[i], NULL);
    for (int i = 0; i < NCONS; i++) pthread_join(c[i], NULL);

    printf("\nproduced=%d consumed=%d final count=%d corrupt=%d\n",
           cb.produced, cb.consumed, cb.count, cb.corrupt);
    return cb.corrupt ? 1 : 0;
}
```

【代码做什么？】 **定向实验**只用一份数据、两个等待者加一次 `broadcast`，把 `if` 与 `while` 的差别放大成 100% 可复现；**有界缓冲区**用 2 生产者 + 2 消费者 + 容量 4，`while` 版累计 `produced=20, consumed=20, corrupt=0`。

【底层机制透视】 `pthread_cond_wait` 在一个原子临界区内做三件事：把本线程加入 `cv` 的等待链表、释放 `m`、调用 futex 挂起。`pthread_cond_signal` 把等待链表**队首**节点移到 `m` 的等待链——**这就是"signal 只是通知、不是交接"的机制根源**：被唤醒者必须先竞争到 `m`，而在此之前别的线程完全可能又把 `count` 改回 0。

【内存布局 / 数据结构图解】 条件变量与互斥锁的配对关系：

```
   pthread_cond_t not_empty          pthread_mutex_t m
   ┌──────────────────┐              ┌──────────────────┐
   │ waiters: [C1, C2]│──signal──►   │ owner: NULL      │
   └──────────────────┘  把队首节点  │ waiters: [C1]    │
                         移到 m 队列 └──────────────────┘
                                            │
                         C1 必须在 m 上重新竞争成功才能返回 wait
   count / in / out 三者都在 m 的保护下；while 在"重新拿到 m 之后"重查 count
```

【实测验证】 `while` 模式完整跑通，`corrupt=0`：

```
$ ./c_condvar while
mode=while

=== directed experiment: ONE item, TWO waiters, broadcast (while) ===
[while-waiter] consumed the only item
[while-waiter] consumed the only item
--> stale-condition wakeups: 0

=== bounded buffer: 2 producers, 2 consumers, capacity 4 ===
[P0] put    0  count=1
[P0] put    1  count=2
[P0] put    2  count=3
[P0] put    3  count=4
[C0] got    0  count=3
[C0] got    1  count=2
[C0] got    2  count=1
[C0] got    3  count=0
[C0] buffer empty, waiting ...
[C1] buffer empty, waiting ...
[P0] put    4  count=1
...
produced=20 consumed=20 final count=0 corrupt=0
exit=0
```

`if` 模式在同一定向实验里 100% 复现了陈旧条件唤醒（连续 8 次运行结果一致）：

```
$ ./c_condvar if
mode=if(buggy)

=== directed experiment: ONE item, TWO waiters, broadcast (if) ===
[if-waiter] consumed the only item
[if-waiter] woke up but slot still empty --> check was stale
--> stale-condition wakeups: 1
...
produced=20 consumed=20 final count=0 corrupt=0
exit=0
```

`if` 版**有界缓冲区**在本次运行里 `corrupt=0` —— **`if` 的 bug 是概率性的，多数运行不触发**：它只在"被唤醒者还没拿到锁时另一个消费者已把缓冲抽空"的窄窗口出错。定向实验用 `broadcast` + 单条目把该窗口开到最大，才让它必然暴露。

23.3.4 第 3 类线程不安全函数演示：`ctime` 的静态缓冲区

> ⚠️ **仅供演示，请勿模仿**：下面 `unsafe_thread` 的写法会产生真实的、静默的数据损坏。

代码 (C)：

```c
/* d_ctime.c —— 第 3 类线程不安全函数：ctime 返回静态缓冲区
 * gcc -g -Wall -std=c11 -lpthread d_ctime.c -o d_ctime && ./d_ctime
 */
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

#define NTHREADS 4

static pthread_barrier_t barrier;    /* 让"取结果"和"用结果"分离，放大竞态 */

/* ---------- ❌ 第 3 类：ctime 返回库内部静态缓冲区 ---------- */
static void *unsafe_thread(void *arg)
{
    long id = (long)arg;
    time_t t = (time_t)(1000000000L + id * 86400L * 365L);   /* 每人一个不同时间 */
    char *s = ctime(&t);                /* ← 返回内部 static char buf[26] */
    pthread_barrier_wait(&barrier);     /* 所有线程在这里对齐 */
    printf("[unsafe %ld] my time should be 0x%lx, ctime says: %s",
           id, (unsigned long)t, s);    /* ← s 早已不是"自己"的字符串 */
    fflush(stdout);
    return NULL;
}

/* ---------- ✅ 修复：lock-and-copy ---------- */
static pthread_mutex_t ctime_lock = PTHREAD_MUTEX_INITIALIZER;

static char *correct_ctime(const time_t *t, char *dst, size_t dstsz)
{
    pthread_mutex_lock(&ctime_lock);
    char *r = ctime(t);
    if (r != NULL && dstsz > 0)
        snprintf(dst, dstsz, "%s", r);   /* ★ 拷贝发生在锁内！ */
    pthread_mutex_unlock(&ctime_lock);
    return (r == NULL) ? NULL : dst;
}

static void *safe_thread(void *arg)
{
    long id = (long)arg;
    time_t t = (time_t)(1000000000L + id * 86400L * 365L);
    char mybuf[64];                     /* ★ 每个线程自己的存储 */
    char *s = correct_ctime(&t, mybuf, sizeof(mybuf));
    pthread_barrier_wait(&barrier);
    printf("[safe   %ld] my time should be 0x%lx, ctime says: %s",
           id, (unsigned long)t, s);
    fflush(stdout);
    return NULL;
}

/* ---------- 讲义原样出现的 itoa（static char buf[11]） ---------- */
static char *itoa_unsafe(int x)
{
    static char buf[11];
    snprintf(buf, sizeof(buf), "%d", x);
    return buf;
}
static void itoa_safe(int x, char *buf, size_t bufsz)
{
    snprintf(buf, bufsz, "%d", x);
}

static pthread_barrier_t b2;
static void *itoa_thread(void *arg)
{
    long id = (long)arg;
    char *s = itoa_unsafe((int)(id * 11111111L));
    pthread_barrier_wait(&b2);
    printf("[itoa   %ld] expected %8ld, got: %s\n", id, id * 11111111L, s);
    fflush(stdout);
    return NULL;
```

```
}
static void *itoa_r_thread(void *arg)
{
    long id = (long)arg;
    char buf[12];
    itoa_safe((int)(id * 11111111L), buf, sizeof(buf));
    pthread_barrier_wait(&b2);
    printf("[itoa_r %ld] expected %8ld, got: %s\n", id, id * 11111111L, buf);
    fflush(stdout);
    return NULL;
}

int main(void)
{
    pthread_t t[NTHREADS];

    printf("######## 1) ctime: UNSAFE (class 3, static buffer) ########\n");
    pthread_barrier_init(&barrier, NULL, NTHREADS);
    for (long i = 0; i < NTHREADS; i++) pthread_create(&t[i], NULL, unsafe_thread, (void *)i);
    for (int i = 0; i < NTHREADS; i++) pthread_join(t[i], NULL);
    pthread_barrier_destroy(&barrier);

    printf("\n######## 2) ctime: FIXED with lock-and-copy ########\n");
    pthread_barrier_init(&barrier, NULL, NTHREADS);
    for (long i = 0; i < NTHREADS; i++) pthread_create(&t[i], NULL, safe_thread, (void *)i);
    for (int i = 0; i < NTHREADS; i++) pthread_join(t[i], NULL);
    pthread_barrier_destroy(&barrier);

    printf("\n######## 3) itoa (from lecture slide): UNSAFE vs _r ########\n");
    pthread_barrier_init(&b2, NULL, NTHREADS);
    for (long i = 0; i < NTHREADS; i++) pthread_create(&t[i], NULL, itoa_thread, (void *)i);
    for (int i = 0; i < NTHREADS; i++) pthread_join(t[i], NULL);
    for (long i = 0; i < NTHREADS; i++) pthread_create(&t[i], NULL, itoa_r_thread, (void *)i);
    for (int i = 0; i < NTHREADS; i++) pthread_join(t[i], NULL);
    pthread_barrier_destroy(&b2);
    return 0;
}
```

【代码做什么？】 第 1 段让 4 个线程各用**不同的**时间调用 `ctime`、在屏障处对齐**之后**才去读返回的字符串；第 2 段用 `correct_ctime`（锁内调用 + 锁内拷贝到调用者私有缓冲）做同样的事；第 3 段对比讲义里的 `itoa` 与 `itoa_r`。

【底层机制透视】 `ctime` 在 glibc 里返回一个**静态的 26 字节缓冲区**。`pthread_barrier_wait` 是**触发器**：它强制所有线程先完成 `ctime` 调用、再统一去读字符串，于是线程 0 去读 `s` 时该缓冲区早已被覆盖。**第 3 类的要害正在于此：函数本身没有出错痕迹，出错的是调用者在"取回指针之后"的使用时机。** lock-and-copy 的要点是"**拷贝必须发生在锁内**"。

【实测验证】 未修复时四个线程拿到**完全相同**的时间字符串（真实输出，稳定复现）：

```
$ ./d_ctime
######## 1) ctime: UNSAFE (class 3, static buffer) ########
[unsafe 3] my time should be 0x413e6480, ctime says: Tue Sep  7 20:46:40 2004
[unsafe 2] my time should be 0x3f5d3100, ctime says: Tue Sep  7 20:46:40 2004
[unsafe 0] my time should be 0x3b9aca00, ctime says: Tue Sep  7 20:46:40 2004
[unsafe 1] my time should be 0x3d7bfd80, ctime says: Tue Sep  7 20:46:40 2004
```

四个 `time_t`（`0x3b9aca00`=2001-09-09、`0x3d7bfd80`=2002-09-09、`0x3f5d3100`=2003-09-09、`0x413e6480`=2004-09-07）各不相同，字符串却全是 2004——经典的静默数据损坏。修复后：

```
######## 2) ctime: FIXED with lock-and-copy ########
[safe   3] my time should be 0x413e6480, ctime says: Tue Sep  7 20:46:40 2004
[safe   2] my time should be 0x3f5d3100, ctime says: Mon Sep  8 20:46:40 2003
[safe   1] my time should be 0x3d7bfd80, ctime says: Sun Sep  8 20:46:40 2002
[safe   0] my time should be 0x3b9aca00, ctime says: Sat Sep  8 20:46:40 2001
```

`itoa` 的对比同样干净——未修复版 4 个线程全输出 `33333333`，修复版各归各位：

```
######## 3) itoa (from lecture slide): UNSAFE vs _r ########
[itoa   3] expected 33333333, got: 33333333
[itoa   0] expected        0, got: 33333333     ← 错误
[itoa   2] expected 22222222, got: 33333333     ← 错误
[itoa   1] expected 11111111, got: 33333333     ← 错误
[itoa_r 0] expected        0, got: 0
[itoa_r 3] expected 33333333, got: 33333333
[itoa_r 1] expected 11111111, got: 11111111
[itoa_r 2] expected 22222222, got: 22222222
```

`valgrind --tool=helgrind ./d_ctime` 报出 `ERROR SUMMARY: 48 errors from 7 contexts`，其中一处直接指到 `ctime` 内部：`Possible data race during read of size 1 ... by 0x40120A: unsafe_thread (d_ctime.c:25)`，调用链为 `strlen ← __tzstring ← __tzfile_compute ← __tz_convert ← ctime`——连时区字符串的处理都不是线程安全的。

23.3.5 补例：竞争的三种形态与锁粒度实验

竞争三形态（`race.c`，真实输出）：

```
$ ./race
== Race A: passing &i ==
  [bad ] id=1
  [bad ] id=2
...
  [bad ] id=7
  [bad ] id=8        ← 没有 0! 把 &i 传给线程，读到的永远是"当时"的 i
== Race A: passing the value ==
  [good] id=0
  [good] id=1
...
== Race B(join WINS   ): detach after 300000us, join after 50000us
  [worker] self-detached, finishing
  pthread_join -> 0 (join won the race)
== Race B(detach WINS ): detach after 50000us, join after 300000us
  [worker] self-detached, finishing
  pthread_join -> 22 (EINVAL: detach won the race)
== Race C: 2 x 1000000 increments, WITHOUT mutex -> cnt=1000000 (want 2000000) *** LOST UPDATES ***
== Race C: 2 x 1000000 increments, WITH mutex    -> cnt=2000000 (want 2000000) OK
```

竞态 B 尤其值得注意：**同一个 `pthread_join` 调用**，只看 detach 与 join 谁先到，返回值就分别是 `0` 和 `EINVAL(22)`——"正确性依赖于某线程先到达 x 点"的定义级演示。竞态 C 的 `cnt=1000000`（正好丢失一半更新）说明两线程的读-改-写几乎完全交错。

锁粒度与可扩展性（`contention.c`，真实输出）：

```
$ ./contention 200000 200
临界区内模拟 200 ns 的"磁盘"工作（SFS 元数据读写）
1 把全局锁         nth= 1  time= 0.048s  throughput=     4125 Kops/s
1 把全局锁         nth= 2  time= 0.207s  throughput=     1929 Kops/s
1 把全局锁         nth= 4  time= 0.799s  throughput=     1001 Kops/s
1 把全局锁         nth= 8  time= 3.525s  throughput=      454 Kops/s

16 把分条带锁     nth= 1  time= 0.049s  throughput=     4123 Kops/s
16 把分条带锁     nth= 2  time= 0.079s  throughput=     5076 Kops/s
16 把分条带锁     nth= 4  time= 0.127s  throughput=     6304 Kops/s
16 把分条带锁     nth= 8  time= 0.475s  throughput=     3365 Kops/s
```

全局锁的加速比 $S(4) = 1001/4125 \approx 0.24$、$S(8) = 454/4125 \approx 0.11$——**都不是正扩展**，与 Amdahl 定律预测一致。分条带锁 $S(4) = 6304/4123 \approx 1.53$，到 8 核反降到 0.82：64 个文件只映射到 16 个条带，8 个线程必然撞在同一批条带上，竞争又回来了。

信号量三用（`sem.c`，真实输出）：1 槽位队列（`full` + `empty`）、n 槽位队列（`mutex` + `slots` + `items`，校验和与理论值一致）、可复用屏障、事件通知，全部通过：

```
=== 2) n-entry (8) queue (mutex + slots + items) ===
  checksum=8100 (expect 8100), remaining=0
=== 3) reusable barrier: 5 threads x 3 rounds ===
  all 5 threads passed 3 rounds together
=== 4) event notification ===
  worker 0 ready, local=0
...
  (信号量允许任何线程执行 V —— 所以 main 能唤醒 worker)
```

23.4 实验关联

L8 SFS Lab（本讲最直接的应用）。SFS（"Shark" File System）用 `mmap` 把整个"磁盘"文件映射进内存，磁盘是 512 字节块的数组，块 0 是超级块（superblock），目录是**平坦结构**（root directory 就在超级块里）。它**始终驻留内存**，不存在写回磁盘的时序问题，但**必须处理多线程并发访问**。识别临界区的三步法：

1. **哪些变量实例是共享的？** 超级块（含根目录的文件表）、inode/块位图、空闲块链表、打开文件表共享；`sfs_open` 的局部变量 `fileEntry`、`emptyEntry` 私有。
2. **这些变量在多少地方被访问？** `sfs_open` 的循环**既读又写** `superBlock->files[]`：`strcmp(...)` 是读，`addOpenFileEntry(fileEntry)` 改打开文件表——两处必须落在同一临界区内。
3. **临界区里是读还是写？** 纯读可用读锁；有写必须独占。讲义提醒："关键难点是识别临界区；临界区由共享变量/资源定义，它可能是**两个或多个线程调用相同或不同函数**造成的。"——`sfs_open` 与 `sfs_read` 可能同时改同一个 inode，加锁方案必须在**函数之间**保持一致顺序。

SFS 加锁粒度建议：① 先用**一把全局大锁**把功能做对；② 再按资源分层细化——超级块/目录、每个 inode、块位图与空闲链表各一把锁；③ 细化时**必须固定加锁顺序**（永远按"目录锁 → inode 锁 → 位图锁"，绝不反向），否则会出现 23.3.2 那样的死锁；④ 许多操作**读多写少**，用 `pthread_rwlock_t` 能显著提升并发。

L7 Proxy Lab 的缓存：官方 writeup 要求"**多个线程必须能同时从缓存读取**，但只有一个线程可以写；用一把大的独占锁保护缓存是**不可接受**的解决方案"，并建议分区缓存、用读写锁或用信号量自建读写方案。缓存近似 LRU 淘汰（`MAX_CACHE_SIZE = 1 MiB`、`MAX_OBJECT_SIZE = 100 KiB`）。**LRU 与并发存在冲突**：严格 LRU 要求"读也要更新使用顺序"，把读变成了写，与"多读者并发"矛盾；writeup 特意说明"不要求严格 LRU"正是为了留出"多读者"的设计空间——这也是为什么单用 `pthread_rwlock_t` 往往不够，通常需要"分区 + 每分区一把读写锁"。

23.5 常见错误与调试技巧

- **用 `if` 而不是 `while` 包裹 `pthread_cond_wait`**：现象是"偶发读到空缓冲区/负计数"或进程偶发挂死。原因是虚假唤醒、`broadcast` 惊群、条件被抢改。**调试**：代码审查优先，再用 `valgrind --tool=helgrind` 或 `gcc -fsanitize=thread` 压测。
- **锁顺序不一致导致死锁**：现象是"程序偶尔永久挂起、CPU 占用 0%"。**调试**：`timeout 5 ./prog; echo $?`（124 = 卡死）；`gdb -p $(pgrep -x prog)` 后 `thread apply all bt` 看各线程卡在哪把锁；`valgrind --tool=helgrind` 报 `Exiting thread still holds 1 lock`；也可把锁设成 `PTHREAD_MUTEX_ERRORCHECK`，让"自锁"立刻返回 `EDEADLK`。
- **信号处理程序里调用会加锁的库函数**：现象是"收到信号后整个进程卡死"，`gdb` 显示主线程与处理程序卡在同一把锁上。原因是**信号处理是非对称并发（asymmetric concurrency）**：主线程持锁时被中断，处理程序又去要同一把锁，而它不返回主线程就永远拿不到锁。**调试**：只在处理程序中调用**异步信号安全（async-signal-safe）**的函数（`write`、`_exit`，而**不是** `printf`/`malloc`）。
- **`pthread_join` 一个已 detach 的线程**：现象是 `join` 返回 `EINVAL(22)` 却查不出原因，且行为依赖时序（如 23.3.5 的竞态 B）。**调试**：检查 `pthread_create` 是否在创建时就设了 `PTHREAD_CREATE_DETACHED`；统一约定"要么全部可 join、要么全部 detach"。
- **把栈上变量的地址传给线程**：现象是线程打印出越界的 `id`（如 23.3.5 里"没有 0，只有 1..8"）或直接段错误。**调试**：`valgrind` memcheck 报 "Invalid read"，helgrind 报栈地址的竞争；修复是**按值传递**（`(void *)(long)i`）或用 `malloc` 为每线程复制参数。
- **加了锁却仍不正确（第 2/3 类误诊）**：现象是"明明加了锁，`rand` 序列还是不对 / 字符串还是被覆盖"。原因是这两类**加锁治不好**。**调试**：查手册页 Attributes 一节（`man 3 ctime` 写 `MT-Unsafe`）；改用 `rand_r` / `strftime` / `getaddrinfo`；第 3 类必须用 lock-and-copy。
- **`sbuf` 的 P 顺序写反（先 P 互斥量再 P 计数信号量）**：现象是"生产者或消费者拿着锁在等空位，整个队列永久卡死"。**调试**：牢记规则"**先 P 计数信号量，后 P 互斥量**"，V 的顺序则相反。
- **粗粒度锁使"并发"名存实亡，或用 `volatile` 当同步原语**：前者的现象是"开了 8 个线程，CPU 只跑满 1 个核"（`perf record -g ./prog && perf report` 找热点锁，`pidstat -t -p <pid> 1` 看线程分布）；后者的现象是"标志位明明改了，另一个线程却看不到"（`while (!flag) ;` 被优化成只读寄存器而死循环）。`volatile` 只阻止编译器优化掉/合并访问，**不提供原子性与内存序保证**，正确做法是 `__atomic_load_n(&flag, __ATOMIC_ACQUIRE)`。

23.6 关键要点

- **临界区的边界由共享资源定义，而不是由函数边界定义**。SFS 里 `sfs_open` 与 `sfs_read` 可能同时改同一个 inode——识别临界区的能力就是并发编程的核心能力。
- **加锁只能修好第 1 类线程不安全函数**。第 2、3 类必须改 API（传状态 / lock-and-copy / 可重入版本），第 4 类具"传染性"，须沿调用链清理到底；可重入是线程安全的真子集。
- **`pthread_cond_wait` 的"原子释放锁 + 挂起 + 唤醒后重新取锁"语义，决定了等待必须写在 `while` 里**；`if` 版在虚假唤醒、`broadcast` 惊群、条件被抢改三种场景下都会出错。
- **死锁是"数学上不可能解套"，根因是循环等待；唯一可靠的系统性防御是全局一致的加锁顺序（lock ordering）**。解锁顺序无关紧要。
- **信号量与互斥锁的本质分野是"所有权"**：互斥锁只有持有者能解锁，无法表达跨线程事件通知；信号量任何线程都能 P 或 V，所以能做事件、队列与屏障。
- **性能取决于锁竞争而非锁本身**。一把全局锁保护整个文件系统 / 缓存会产生负扩展（实测 1→8 核吞吐跌到 11%）；细粒度或分条带锁能暴露并行度，但必须以固定加锁顺序为前提。

23.7 思考题（带答案）

题 1（推演题）：三个线程的加锁顺序是：T1 `m1→m2→m3`，T2 `m1→m3→m2`，T3 `m2→m3`。是否可能出现死锁？若可能，给出一个具体死锁状态；若不可能，说明理由。

答：**可能出现死锁**。lock ordering 要求的是**全序（total order）**，而这里有 `m1 < m2 < m3`（T1）与 `m1 < m3 < m2`（T2）两个互相矛盾的顺序。死锁状态：让 T1 持有 `m2` 且未取 `m3`，T2 持有 `m3`；此时 T1 等 `m3`、T2 等 `m2`——**循环等待即死锁**。反证：若只有 T1 和 T3，两者在 `m2` / `m3` 上顺序一致，不死锁；故罪魁是 T2 的 `m1→m3→m2`。修复：把 T2 改成 `m1→m2→m3`，全序成立，任何轨迹都不会进入死锁区域。

题 2（找错题）：下面的代码错在哪？会在什么条件下出错？

```
pthread_mutex_lock(&m);
if (head == NULL)                   /* 队列为空 */
    pthread_cond_wait(&cv, &m);
Node *n = head;                     /* 取队首 */
head = head->next;
pthread_mutex_unlock(&m);
```

答：错在用 `if` 而在 `while`。三类触发条件：① **虚假唤醒**——`pthread_cond_wait` 允许在无 `signal` 时返回，此时 `head` 仍为 `NULL`，`n = head` 得 `NULL`，`head->next` 立即段错误；② `broadcast` **惊群**——两个等待者都被唤醒，第一个拿走后 `head` 又变 `NULL`，第二个同样段错误；③ **条件被抢改**——从"被唤醒"到"重新拿到 `m`"之间存在窗口，新来的消费者可能先取走元素。修复：

```
pthread_mutex_lock(&m);
while (head == NULL)                 /* ✅ 循环重查 */
    pthread_cond_wait(&cv, &m);
Node *n = head;
head = head->next;
pthread_mutex_unlock(&m);
```

题 3（计算题）：某并发服务器里，每条连接的总处理时间为 300 μs，其中 240 μs 必须在一把全局锁的保护下执行，其余 60 μs 可并行。若机器有 8 个核，(a) 按 Amdahl 定律，加速比上限是多少？8 核实际能到这个上限吗？(b) 若把临界区压缩到 60 μs（占 20%），上限变成多少？(c) 为什么实测值甚至可能**低于**单核？

答：(a) 临界区比例 $f = 240/300 = 0.8$。上限 $S(\infty) = 1/f = 1.25$；8 核时为

$$S(8) = \frac{1}{0.8 + \frac{0.2}{8}} = \frac{1}{0.825} \approx 1.21$$

即最多 1.21 倍。**实际到不了**：失败的加锁要走 futex 系统调用、线程被挂起再唤醒、cache line 在核间来回弹跳（cache line ping-pong），这些都不在理想模型里。(b) $f = 0.2$，$S(\infty) = 5$，$S(8) = 1/(0.2 + 0.1) = 3.33$。可见**把临界区从 80% 压到 20%，上限从 1.25 提升到 5，收益远大于加核**。(c) 分工增加后**串行部分的绝对时间被拉长**：8 个线程争一把锁时，每次加锁失败的代价（内核挂起 + 重新调度，可达微秒级）远超临界区本身的 240 ns；同时锁变量所在 cache line 必须在 8 个核间反复失效与传输。"竞争税"超过并行收益，于是 $S(8) < 1$——实测全局锁从 4125 跌到 454 Kops/s（11%）正是此现象。

题 4（找错题）：有人说："既然 `ctime` 不是线程安全的，我在调用它前后各加一次 `pthread_mutex_lock/unlock` 就修好了。"这句话错在哪？

答：错在**锁的生存周期覆盖不了结果的使用周期**。`ctime` 返回指向库内部**静态缓冲区**的指针；解锁之后别的线程完全可以再调用 `ctime` 覆盖这块内存，而调用者手里的 `s` 仍指着它——**从解锁到使用 `s` 之间存在无保护的窗口**。所以第 3 类需要专门的 lock-and-copy：在**锁内完成调用并把结果拷贝到调用者提供的私有存储**（如 `correct_ctime(&t, mybuf, sizeof mybuf)`），然后才解锁。更彻底的做法是换可重入版本（`strftime` 取代 `ctime` / `asctime` / `localtime`）。

Lecture 24: 线程级并行 (Thread-Level Parallelism)

> **讲义对应**：CMU 15-213 Lecture 24 — Thread-Level Parallelism（素材：`F25-24-parallelism.txt`） **教材对应**：CS:APP3e 第 12 章 12.6 *Parallelism*（含 12.6 的并行求和与并行快排案例） **关联 Lab**：L8 SFS Lab；本讲同时是 CMU 15-418 / CS149 并行体系结构课的衔接点

24.1 概述

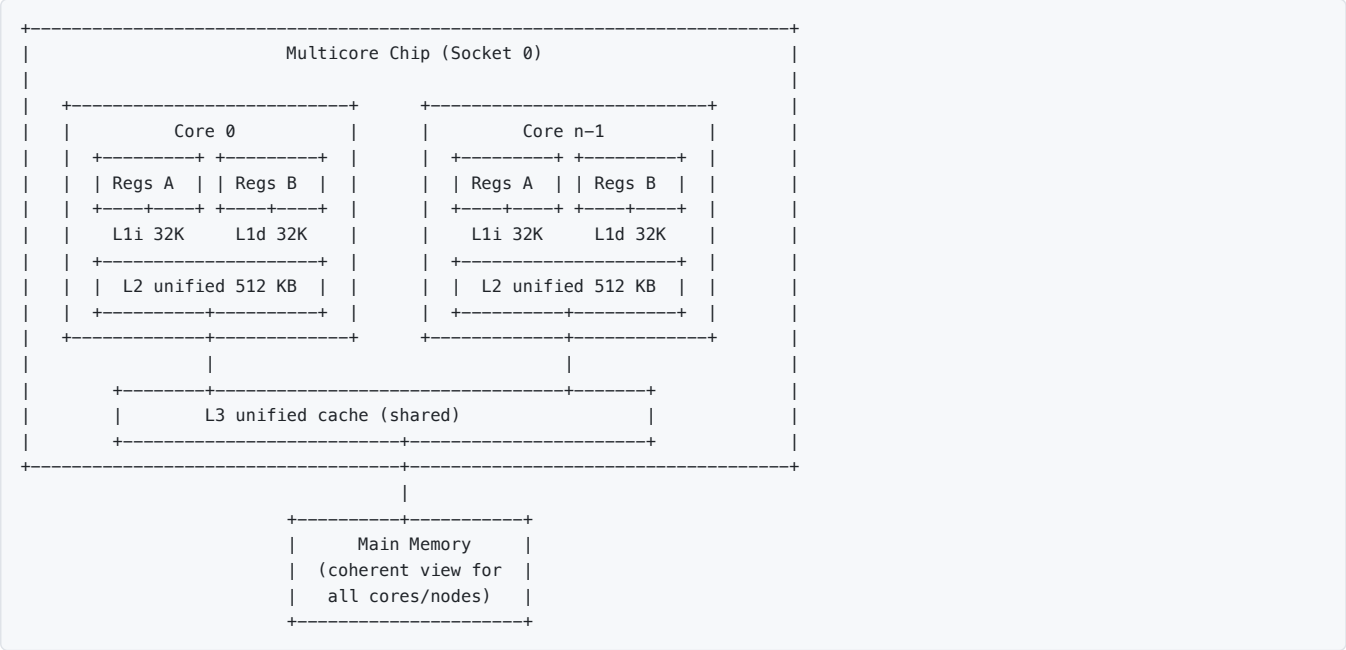
前面几讲我们用线程**应对 I/O 延迟**（一个客户端一个线程，避免互相阻塞），这一讲把线程用在完全不同的目的上：**把一个"算得慢"的大任务切成多个可以同时执行的子任务，用多核硬件真正缩短墙钟时间（wall-clock time）**。本讲的核心问题有三个：多核硬件长什么样（多核、超线程、缓存一致性、内存一致性模型）？怎么把程序切开并度量收益（加速比、效率、Amdahl 定律）？以及一个最容易被忽视的问题——**并行不等于正确，也不等于更快**：共享内存会带来真共享与伪共享的代价，同步会成为新的串行瓶颈。

在知识地图上，本讲收束了第 21–23 讲的并发与同步：并发关注"多个逻辑流如何正确交错"，并行关注"如何让它们同时跑并且更快"； 同时它复用第 9–10 讲的缓存与局部性知识，并通向 15–418/CS149 的并行体系结构与 GPU 编程。

24.2 核心概念与底层机制图解

24.2.1 共享内存多核处理器（Shared-Memory Multicore）

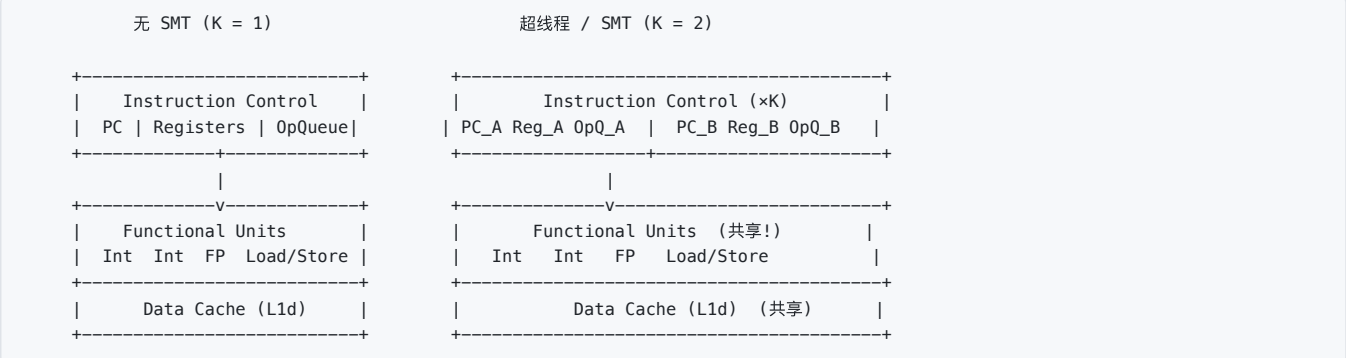
- **定义与目的**：多核（multicore）处理器把多个完整的处理器核（core）集成在**同一块芯片**上， 它们通过互连网络（interconnect）访问**统一的物理地址空间**，因此对内存有一致的视图（coherent view of memory）。
- **直观解释**：把单核想象成一个独居的学者，所有资料都在他手边的书架（L1/L2）和图书馆（主存）里。 多核就是在一个办公室里放了 64 位学者：他们共用同一个图书馆，但每人有自己的书架。 **谁会最快？——各自读自己的资料，偶尔才去图书馆**。这就是并行程序设计的全部直觉来源。
- **底层机制图解**：



- **与机器码/硬件的对应**：对程序员而言，多核**不需要任何新指令**——同一份 `addq` / `movq` 代码在不同核上跑。 硬件负责的是缓存一致性（coherence）：L1/L2 私有，L3 共享，主存是最终一致点。 代价是：**任何被多个核读写的缓存行都必须在核之间搬运**，这就是本讲后面所有性能故事的总根源。

24.2.2 超线程（Hyperthreading / Simultaneous Multithreading, SMT）

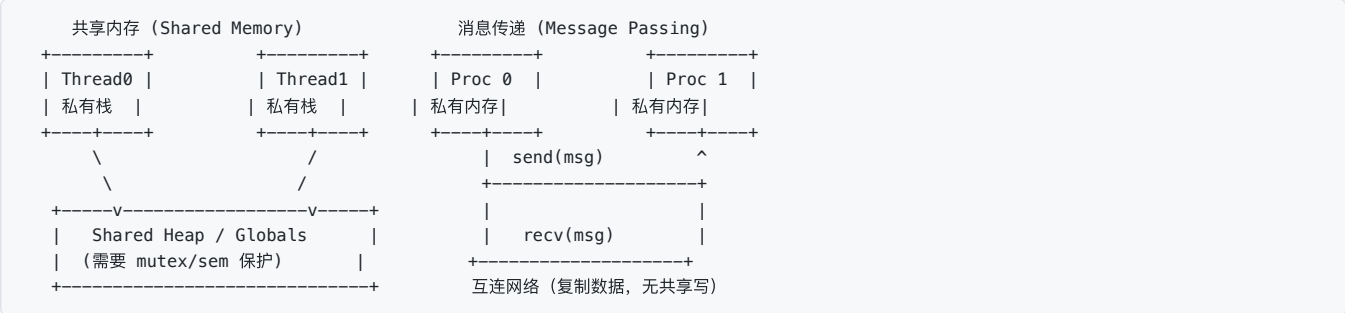
- **定义与目的**：超线程在**一个物理核**上复制指令控制逻辑与寄存器组（典型 K=2），使单个核能同时处理 K 个指令流； **功能单元（functional units）与缓存是共享的**。
- **直观解释**：一个厨师（功能单元）同时看两份菜谱（寄存器组）。切菜时盯着第一份，等水烧开（访存延迟）时就去看第二份—— 厨师的手没变快，但**闲着的时间被填满了**。
- **底层机制图解**：



- **与机器码/硬件的对应**：SMT 是**吞吐（throughput）优化**，不是延迟优化——单线程的 CPE 不会因为开了超线程而下降。 讲义中 Shark 机器是 Intel Xeon E5520 @ 2.27 GHz、8 核、每核 2 路超线程，因此"**理论上一次可执行 16 个线程，理论加速比 16×， 但讲义明确写道 never achieved in our benchmarks**"——超线程带来的额外吞吐取决于线程的访存/分支行为，通常远低于 2×。

24.2.3 共享内存 vs 消息传递（Shared Memory vs Message Passing）

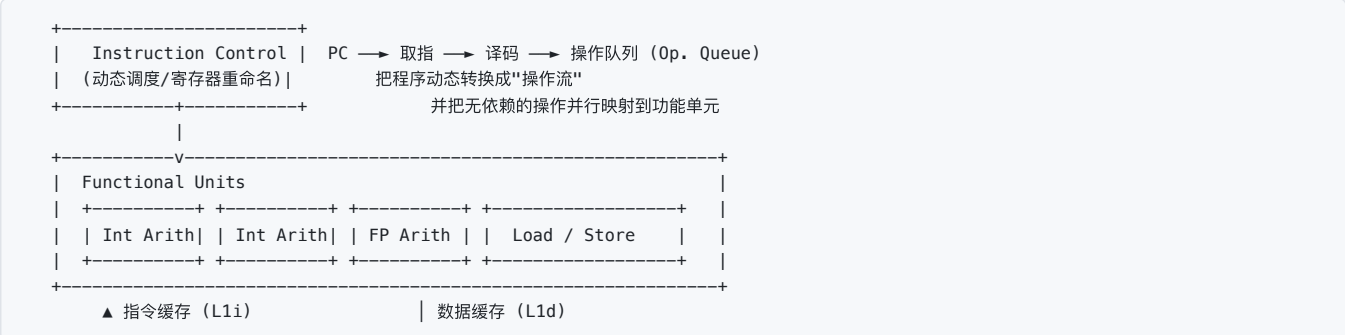
- **定义与目的**：这是并行程序的两种基本编程模型。**共享内存**：所有线程访问同一地址空间，靠读写共享变量通信，靠同步保证正确性； **消息传递**：每个执行体有私有地址空间，只能通过显式 `send` / `recv` （如 MPI）交换数据。
- **直观解释**：共享内存像**一块共用的白板**，谁都能直接擦写，快但容易打架；消息传递像**发微信**， 各自笔记互不可见，必须复制一份发过去，慢但边界清晰、天然避免竞态。
- **底层机制图解**：



- **与机器码/硬件的对应**：共享内存模型对应**多核 + 缓存一致性协议**（本机：x86-64 多核，`pthread`）； 消息传递模型对应**集群/多节点 + 网络**（MPI），也是 GPU 上"主机内存与设备内存必须显式 `cudaMemcpy`"的根源。 CS:APP 只讲共享内存（`pthread`），消息传递由 15-418/CS149 展开。

24.2.4 并行性的三个层次：TLP / ILP / DLP

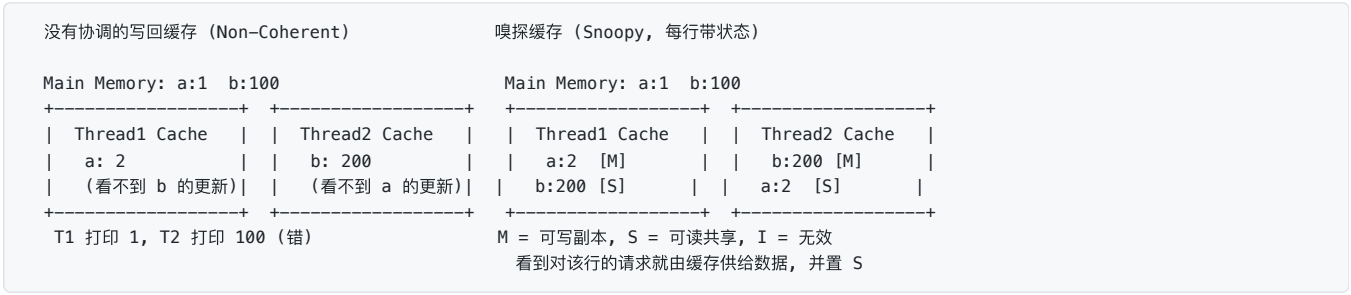
- **定义**：**线程级并行（Thread-Level Parallelism, TLP）**——多个线程/进程在多个核上同时执行； **指令级并行（Instruction-Level Parallelism, ILP）**——单个指令流内部，处理器用**超标量（superscalar）发射与乱序执行（out-of-order execution）** 让多条无依赖指令同时完成；**数据级并行（Data-Level Parallelism, DLP）**——同一条操作作用于一批数据， 硬件形式是 **SIMD**（SSE/AVX 向量指令）与 **GPU**。
- **直观解释**：TLP 是"多请几个工人"，ILP 是"一个工人左右手同时干活"，DLP 是"一台压面机一次压一整排面皮"。
- **底层机制图解**（乱序执行结构，讲义原图）：



- **与机器码/硬件的对应**：ILP 由硬件自动挖掘，程序员只需避免**长依赖链**（例如把 `sum` 拆成多个累加器做循环展开， 详见 Lecture 15）；DLP 需要程序员或编译器把标量循环**向量化**；TLP 需要程序员显式创建线程。 三者的一个关键区别是：**ILP 不需要改代码语义，TLP 会引入竞态，因此本讲的正确性问题只属于 TLP**。

24.2.5 缓存一致性与内存一致性模型（Coherence & Consistency）

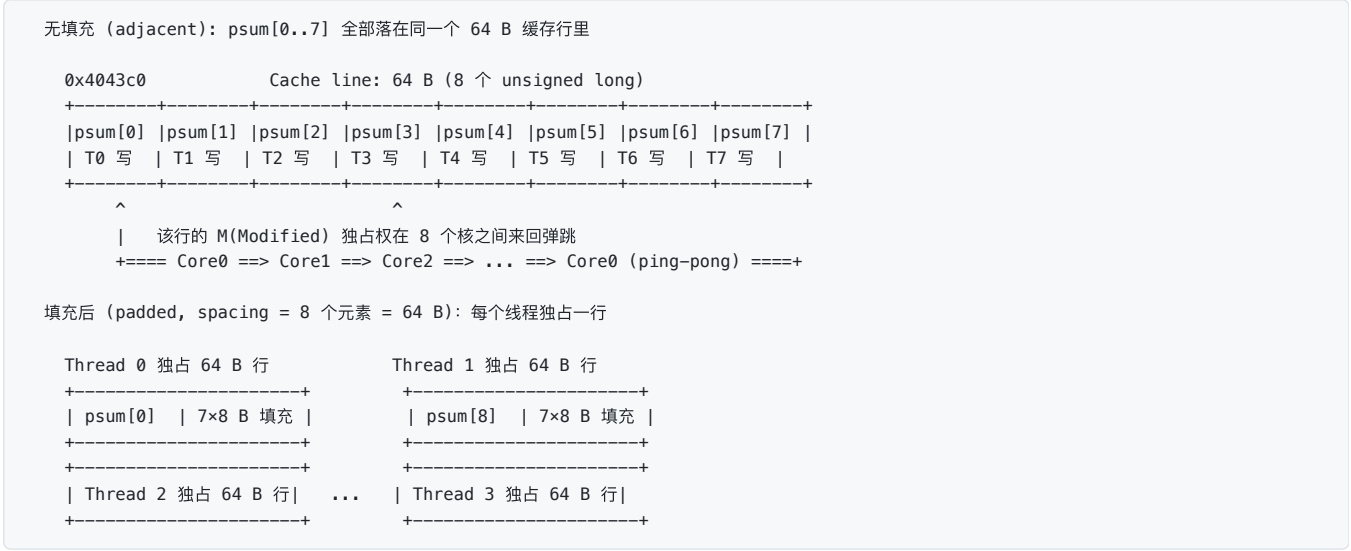
- **定义**：**缓存一致性（cache coherence）**保证同一地址的读最终能看到最近的写（硬件协议，如 MESI 嗅探协议 snoopy protocol）； **内存一致性模型（memory consistency model）**规定不同地址的读写在不同线程看来允许出现的顺序。
- **直观解释**：一致性是"**每个人都最终知道白板上写了什么**"；一致性模型是"**你写字和你看白板的先后顺序是否可以让别人看出来**"。 前者保证不丢数据，后者决定"先写标志位还是先写数据"这种程序逻辑是否成立。
- **底层机制图解**（讲义的非一致缓存与嗅探缓存对比）：



- **与机器码/硬件的对应**：即使有了一致性缓存，**写缓冲（write buffer）与乱序执行**仍会打乱同线程内对不同地址的可见顺序， 于是出现"非顺序一致"的执行。讲义给出的修法是：在 `Wa` 与 `Rb` 之间、`Wb` 与 `Ra` 之间插入 `SFENCE`（store fence）， 或**使用正确编写的同步操作**（`pthread_mutex_lock/unlock` 本身起栅栏作用）。 讲义给出的经典程序（两个线程分别写 `X` / `Y` 再读对方）在顺序一致性下**不可能**同时打印出 "Hello" 和 "World"； 这也是后续 15–418 里"内存模型"整章的主题。

24.2.6 伪共享（False Sharing）

- **定义**：多个线程**写同一个缓存行（cache line）内的不同变量**。变量本身互不相干（不是真共享 true sharing）， 但一致性协议以**缓存行**为粒度，于是该行在核之间反复失效与搬运（cache line ping-pong）。
- **直观解释**：两个人住在同一栋楼（同一缓存行）的不同房间。甲要改自己的窗帘，物业（一致性协议）却把**整栋楼的钥匙**都收回来再发给甲； 乙改灯也一样。两人什么都没共用，却被迫互相排队。
- **底层机制图解**（本机实测缓存行为 64 字节）：



- **与机器码/硬件的对应**：讲义给出的判据非常直白——"Demonstrates cache block size = 64: 8-byte values, no benefit increasing spacing beyond 8"， 即元素步长超过 8 个 8 字节值后收益消失，反推出块大小就是 64 B。本机用 `getconf LEVEL1_DCACHE_LINESIZE` 与 `/sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size` 双重确认，答案同样是 64。

24.2.7 性能度量：加速比、效率、强/弱扩展与两条定律

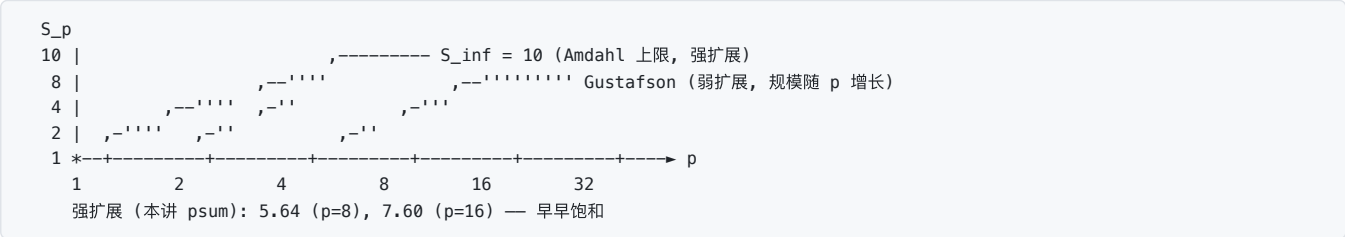
- **定义与目的**：**加速比（speedup）** $S_p = T_1/T_p$，T_1 是**同一问题规模**下最好的串行实现时间， T_p 是 p 个执行体的时间；**效率（efficiency）** $E_p = S_p/p \in [0,1]$，衡量每个核的"利用率"。 若"加速"来自问题规模也变大了，那是另一种故事，因此必须区分两种扩展方式：

度量	定义	问题规模	关注
强扩展（strong scaling）	固定总工作量 N，只加 p	不变	给定问题能多快算完
弱扩展（weak scaling）	固定**每核**工作量 N/p，N 随 p 同步增长	随 p 增长	每核耗时是否恒定

- **直观解释**：强扩展是"10 个人搬 1 吨砖，多久搬完"；弱扩展是"10 个人各搬 100 kg，每人还是要那么久吗"。 本讲 psum 的实测曲线全部是**强扩展**：N 固定为 2^{28}，线程从 1 加到 16，于是 T_p 下降。
- **Amdahl 定律（固定规模的上限）**：设 α 为可并行化比例， $S_p = \dfrac{1}{(1-\alpha)+\alpha/p}$， $S_\infty = 1/(1-\alpha)$。串行部分是**硬天花板**：$\alpha = 0.9$ 时无论多少核也快不过 10×。 讲义用"PIT→LHR 航班"类比：飞到纽约的 1.5 小时无法被任何技术压缩，所以即使有超光速航班，最好也只有 ~5×。
- **Gustafson 定律（固定时间的可扩展规模）**：反过来问"**给我 p 倍资源、同样的时间，能算多大的问题**"。 设串行部分耗时 s、并行部分耗时 q，则

$$S_p = \frac{s+pq}{s+q} = p - (p-1)\cdot\frac{s}{s+q}$$

当 $s \ll q$ 时 $S_p \approx p$——**弱扩展下加速比可以接近线性**，这正是超算"算得更大"而非"算得更快"的逻辑。 – **底层机制图解**（同一台机器的强/弱扩展对照，$\alpha = 0.9$）：



- **与机器码/硬件的对应**：T_1 必须是**最好的串行实现**（本机 0.3256 ns/元素）， 而不是"1 线程并行版"。讲义特意警告 "Beware the speedup metric!"——用差的基线会虚报加速比。 实践中还要警惕**超线性加速比**（$S_p > p$）：它通常只说明串行版数据放不进缓存，而并行版每核的数据片能放进 L2/L3 了 （本讲 psum 中每个线程只需读自己那一段，正是这种情况）。

24.3 代码示例与底层机制分析

24.3.1 实测环境

所有数据都在同一台机器上用 `taskset` 绑定到空闲 CPU 实测（机器为多用户共享，负载约 12， 故每个数据点取多轮中的**最小值**并交错执行以抵消干扰）：

项目	值
CPU	AMD EPYC 7V13 64-Core Processor（Zen 3）
拓扑	2 socket × 64 核 = **128 逻辑 CPU**，`Thread(s) per core: 1`（本机未启用 SMT）
频率	1.5 GHz（min）～ 3.72 GHz（max boost）；由单线程基线 0.3256 ns/元素、按 $CPE = 1.0$ 反推有效频率约 3.07 GHz
L1d / L2	32 KiB / 核（8 路组相联）；512 KiB / 核
L3	16 个实例共 512 MiB；NUMA 2 节点（0–63 / 64–127）
内存	503 GiB
OS / 编译器	Linux 5.14.0-611（RHEL 9），gcc 12.2.0
缓存行	`getconf LEVEL1_DCACHE_LINESIZE` = 64；`/sys/.../index0/coherency_line_size` = 64

24.3.2 示例 1：psum-local（每线程寄存器累加 + 最后归约）

代码 (C)：

```c
/* psum_local.c  ---  编译: gcc -O2 -g -Wall -std=c11 psum_local.c -o psum_local -lpthread */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

typedef unsigned long data_t;

#define MAXTHREADS 64
#define MAXSPACING 16

static size_t nelems_per_thread;
static const size_t spacing = 16;          /* 16*8B = 128B, 跨过 cache line */
static data_t psum[MAXTHREADS * MAXSPACING];
static pthread_t tid[MAXTHREADS];
static int myid[MAXTHREADS];

static double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

static void *sum_local(void *vargp)
{
    int id = *((int *)vargp);
    size_t start = (size_t)id * nelems_per_thread;
    size_t end   = start + nelems_per_thread;
    data_t sum = 0;                        /* 局部累加器 -> 寄存器 */
    for (size_t i = start; i < end; i++)
        sum += i;
    psum[(size_t)id * spacing] = sum;      /* 每个线程只写自己的私有位置 */
    return NULL;
}

int main(int argc, char **argv)
{
    size_t nelems = (argc > 1) ? strtoul(argv[1], NULL, 0) : (1UL << 30);
    size_t nthreads = (argc > 2) ? strtoul(argv[2], NULL, 0) : 1;
    int reps = (argc > 3) ? atoi(argv[3]) : 3;

    if (nthreads < 1 || nthreads > MAXTHREADS) { fprintf(stderr, "bad nthreads\n"); return 1; }
    nelems_per_thread = nelems / nthreads;

    double best = 1e30; data_t result = 0;
    for (int r = 0; r < reps; r++) {
        for (size_t i = 0; i < nthreads; i++) psum[i * spacing] = 0;
        double t0 = now();
        for (size_t i = 0; i < nthreads; i++) {
            myid[i] = (int)i;
            pthread_create(&tid[i], NULL, sum_local, &myid[i]);
        }
        for (size_t i = 0; i < nthreads; i++) pthread_join(tid[i], NULL);
        result = 0;
        for (size_t i = 0; i < nthreads; i++) result += psum[i * spacing];
        for (size_t e = nthreads * nelems_per_thread; e < nelems; e++) result += e;
        double dt = now() - t0;
        if (dt < best) best = dt;
    }
    printf("psum-local  N=%-12zu threads=%-3zu best=%9.6f s  result=%lu expect=%lu %s\n",
           nelems, nthreads, best, result, nelems * (nelems - 1) / 2,
           result == nelems * (nelems - 1) / 2 ? "OK" : "WRONG!");
    return 0;
}
```

【代码做什么？】 1. 把 N 个元素均分给 `nthreads` 个线程，每个线程处理 `nelems_per_thread` 个；余下的尾巴由主线程串行补加（"deal with leftovers"）。 2. `pthread_create` 时传入 `&myid[i]` ——注意每个线程拿到的是**自己那一格**的地址，这是讲义反复强调的模式。 3. 线程函数内部用**局部变量** `sum` 累加：编译器把 `sum` 放进寄存器，整个内层循环**零次对共享内存的写**。 4. 循环结束后**只写一次** `psum[id*spacing]`，间距 `spacing = 16` 个元素（128 B）确保跨缓存行。 5. `pthread_join` 全部结束后，主线程做一次 $O(p)$ 的串行归约，再加上尾巴。

【底层机制透视】 `sum += i` 编译出的是一段极短的单周期依赖链（见下），因此性能上限完全由**指令发射能力**决定， 不涉及任何缓存行所有权转移。这个"私有化（privatization）"手法是本讲最重要的优化：**把 p 个线程对同一变量的 p×N 次写， 变成 p 次写 + 一次串行归约。**

【与汇编 / 硬件的对应】 `gcc -O2` 下 `sum_local` 的内层循环（`objdump -d -M intel` 实测）：

```
401450:  add    rdx,rax          ; sum += i   (rdx 是累加器, 依赖链长度 = 1 cycle)
401453:  add    rax,0x1          ; i++
401457:  cmp    rcx,rax
40145a:  jne    401450
```

累加器 `%rdx` 只出现在一条依赖链上，其余三条指令独立发射——这正是 ILP 在起作用。 实测单线程基线为 0.3256 ns/元素，即 $\boldsymbol{CPE = 1.00}$（由 3.07 GHz 有效频率与 1.0 CPE 互推）， 与 CS:APP 那个 `CPE ≈ 1.0` 的"最优串行实现"完全对得上——**这就是本讲的性能标尺。** （一个有趣的对照：把同样的 10 亿次加法放进 `main` 里直接写，编译器编译出同样的四条指令， 实测却是 0.651 ns/元素 = $\boldsymbol{CPE = 2.0}$——因为在 `main` 的内层循环里，循环计数器 `%rax` 同时也是**参与累加的数据**， `add %rbx,%rax` 与 `add $1,%rax` 构成了一条 2 周期的循环依赖链。这恰好说明了**依赖链长度就是 ILP 的上限**。）

【实测验证】（N = 2^28 = 268435456，min-of-7，交错执行）：

线程数 p	T_p (s)	加速比 $S_p = T_1/T_p$	效率 $E_p = S_p/p$	CPE$_p$（按 3.07 GHz 折算）
1	0.0874	1.00	1.000	1.00
2	0.0441	1.98	0.992	0.50
4	0.0224	3.90	0.975	0.26
8	0.0155	5.64	0.704	0.18
16	0.0115	7.60	0.475	0.13

CPE 从 **1.00 一路降到 0.13**——这正是讲义"从 CPE ≈ 1.0 到接近串行/超线性区"那张表要传达的事实： **只有当每线程的累加器私有化之后，多核才真正开始工作。**

24.3.3 示例 2：psum-mutex（错误示范）

代码 (C)：

```c
/* psum_mutex.c  ---  编译: gcc -O2 -g -Wall -std=c11 psum_mutex.c -o psum_mutex -lpthread */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

typedef unsigned long data_t;

#define MAXTHREADS 64

static volatile data_t global_sum;
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
static size_t nelems_per_thread;
static pthread_t tid[MAXTHREADS];
static int myid[MAXTHREADS];

static double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

static void *sum_mutex(void *vargp)
{
    int id = *((int *)vargp);
    size_t start = (size_t)id * nelems_per_thread;
    size_t end   = start + nelems_per_thread;
    for (size_t i = start; i < end; i++) {
        pthread_mutex_lock(&mutex);
        global_sum += i;                /* 临界区: 一次加法 */
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main(int argc, char **argv)
{
    size_t nelems = (argc > 1) ? strtoul(argv[1], NULL, 0) : (1UL << 24);
    size_t nthreads = (argc > 2) ? strtoul(argv[2], NULL, 0) : 1;
    if (nthreads < 1 || nthreads > MAXTHREADS) { fprintf(stderr, "bad nthreads\n"); return 1; }
    nelems_per_thread = nelems / nthreads;

    global_sum = 0;
    double t0 = now();
    for (size_t i = 0; i < nthreads; i++) {
        myid[i] = (int)i;
        pthread_create(&tid[i], NULL, sum_mutex, &myid[i]);
    }
    for (size_t i = 0; i < nthreads; i++) pthread_join(tid[i], NULL);
    data_t result = global_sum;
    for (size_t e = nthreads * nelems_per_thread; e < nelems; e++) result += e;
    double dt = now() - t0;

    double per_add_ns = dt / (double)nelems * 1e9;
    printf("psum-mutex  N=%-12zu threads=%-3zu time=%9.6f s  per-element=%8.2f ns  result=%lu %s\n",
           nelems, nthreads, dt, per_add_ns, result,
           result == nelems * (nelems - 1) / 2 ? "OK" : "WRONG!");
    return 0;
}
```

【代码做什么？】 与示例 1 唯一的结构差别是：**累加器是全局变量 `global_sum`，且每一次加法都被 `lock/unlock` 包住**。 结果是正确的（`result` 与 $(N-1)N/2$ 完全一致），这正是它危险的地方——**它是对的，但慢到不可用**。

【底层机制透视】 性能崩溃有三个叠加原因： 1. **临界区串行化**：整段计算的 N 次加法被迫排成一列，p 个线程不可能同时进入。 2. **锁操作本身极贵**：`pthread_mutex_lock` 未竞争时也要一次原子 RMW（`lock cmpxchg`），约 10–20 ns； 竞争时还要系统调用 `futex` 进入内核挂起/唤醒。 3. **真共享（true sharing）**：`global_sum` 与 `mutex` 所在缓存行在核之间疯狂弹跳，每次 `lock` 都会把它抢到本地。 **加速比在这里不是变小，而是变成负数**——加线程只会更慢。

【实测验证】（N = 2^26 = 67108864，min-of-3；对照串行寄存器基线同 N 需 0.0218 s）：

线程数 p	总时间 (s)	每元素开销 (ns)	相对 p=1 的变化	相对串行寄存器基线
1	0.5538	8.25	—	25.4× 慢
2	1.6198	24.14	2.9× **更慢**	74.3× 慢
4	2.3327	34.76	4.2× **更慢**	107× 慢
8	3.4091	50.80	6.2× **更慢**	156× 慢

这与讲义的数据完全同构：讲义说互斥锁版本"2.5 seconds → ~10 minutes"，并注明 **"Mutex 3X faster than semaphore"** （信号量每次 `sem_wait/sem_post` 要走完整的 P/V 路径，比互斥锁更慢）， 结论也一样：**"Clearly, neither is successful."**

24.3.4 示例 3：伪共享 —— 填充（padding）vs 不填充

代码 (C):

```c
/* falseshare.c  ---  编译: gcc -O2 -g -Wall -std=c11 falseshare.c -o falseshare -lpthread
 * 用法: ./falseshare <N> <threads> <reps> <pad:0|1> <accum:0|1>
 *       pad=0 相邻(p_sum[i])  pad=1 填充(stride=8 个元素 = 64 B)
 *       accum=1 每次迭代都写共享数组(暴露伪共享)   accum=0 寄存器累加后一次写回 */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

typedef unsigned long data_t;

#define MAXTHREADS 64
#define MAXPAD     16

static volatile data_t psum[MAXTHREADS * MAXPAD * 16];
static size_t nelems_per_thread;
static size_t stride;                   /* 元素步长: 1 = 紧邻, 8 = 独占 cache line */
static int pad_mode;                    /* 0: 无填充(紧邻), 1: 填充 */
static pthread_t tid[MAXTHREADS];
static int myid[MAXTHREADS];
static int use_accumulate;              /* 1 = 每步都写共享数组, 0 = 寄存器累加后一次写回 */

static double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

static void *worker(void *vargp)
{
    int id = *((int *)vargp);
    size_t start = (size_t)id * nelems_per_thread;
    size_t end   = start + nelems_per_thread;
    size_t idx   = (size_t)id * stride;
    psum[idx] = 0;
    if (use_accumulate) {
        for (size_t i = start; i < end; i++)
            psum[idx] += i;             /* 每步都写内存 -> 伪共享在此显现 */
    } else {
        data_t s = 0;
        for (size_t i = start; i < end; i++)
            s += i;
        psum[idx] = s;
    }
    return NULL;
}

int main(int argc, char **argv)
{
    size_t nelems   = (argc > 1) ? strtoul(argv[1], NULL, 0) : (1UL << 26);
    size_t nthreads = (argc > 2) ? strtoul(argv[2], NULL, 0) : 8;
    int    reps     = (argc > 3) ? atoi(argv[3]) : 3;
    pad_mode        = (argc > 4) ? atoi(argv[4]) : 0;
    use_accumulate  = (argc > 5) ? atoi(argv[5]) : 1;

    if (nthreads < 1 || nthreads > MAXTHREADS) { fprintf(stderr, "bad nthreads\n"); return 1; }
    stride = pad_mode ? (64 / sizeof(data_t)) : 1;   /* 8 个 unsigned long = 64 字节 */
    nelems_per_thread = nelems / nthreads;

    double best = 1e30; data_t result = 0;
    for (int r = 0; r < reps; r++) {
        double t0 = now();
        for (size_t i = 0; i < nthreads; i++) {
            myid[i] = (int)i;
            pthread_create(&tid[i], NULL, worker, &myid[i]);
        }
        for (size_t i = 0; i < nthreads; i++) pthread_join(tid[i], NULL);
        result = 0;
        for (size_t i = 0; i < nthreads; i++) result += psum[i * stride];
        for (size_t e = nthreads * nelems_per_thread; e < nelems; e++) result += e;
        double dt = now() - t0;
        if (dt < best) best = dt;
    }
```

```
    printf("%-18s threads=%-3zu best=%9.6f s  result=%lu %s\n",
           pad_mode ? "padded(64B sep)" : "adjacent(unpadded)",
           nthreads, best, result,
           result == nelems * (nelems - 1) / 2 ? "OK" : "WRONG!");
    return 0;
}
```

【代码做什么？】 同一个程序、同一份工作量，只切换两个开关：`pad`（步长 1 还是 8 个元素）与 `accum`（每次迭代写共享数组，还是寄存器累加后只写一次）。于是可以把"伪共享"与"私有化"两个因素**干净地分离**出来。

【底层机制透视】 `accum=1` 且 `pad=0` 时，内层循环（实测汇编）是：

```
401578:  mov    rdx,QWORD PTR [rcx*8+0x4043c0]   ; 读 psum[idx]  —— 必须独占该缓存行
401580:  add    rdx,rax                           ; 加 i
401587:  mov    QWORD PTR [rcx*8+0x4043c0],rdx   ; 写回 psum[idx] —— 又一次独占
```

8 个线程的 `idx` 分别是 0..7，**地址只差 8 字节，全在同一个 64 B 行内**。 每次写都要向其他 7 个核广播"我要独占这一行"（RFO, read-for-ownership）， 其他核的副本被置为 **Invalid**；下一轮它们再写又要抢回来。缓存行在 8 个核之间来回弹跳， 每次弹跳的代价是几十到上百个周期——**注意：没有一个字节被两个线程真正共享（不是真共享）， 所以这种性能损失对程序员完全不可见，只在硬件里发生**。

【实测验证】（N = 2^27 = 134217728，每次迭代都写共享数组，min-of-5）：

线程数 p	相邻(无填充) 时间 (s)	填充(64 B 间隔) 时间 (s)	比值
1	0.0874	0.0875	1.00×
2	0.1073	0.0440	2.44×
4	0.1845	0.0347	5.32×
8	0.3467	0.0169	**20.56×**
16	0.4697	0.0118	**39.78×**

关键对照：把 `accum` 换成 0（寄存器累加）后，**填充与不填充的差别完全消失** （8 线程：相邻 0.0059 s vs 填充 0.0057 s，差 < 4%）。 这证明伪共享的唯一来源是**对共享缓存行的反复写**，而不是数组布局本身； 一旦私有化到寄存器，"相邻"也就无害了——这正是讲义"**Use registers whenever possible**"的实证。

24.3.5 Amdahl 定律与统一性能总览

同一会话、同一 N = 2^26、8 线程，把所有版本放在一张表里（时间越短越好）：

版本	时间 (s)	加速比 $S_p = T_1/T_p$	相对串行基线
串行基线（1 线程，寄存器）	0.0218	1.00	1.00×
psum-local（8 线程，寄存器，填充）	0.0059	**3.69**	0.27×
psum-array（8 线程，填充）	0.0113	1.93	0.52×
psum-array（8 线程，相邻 = 伪共享）	0.2961	0.074	**13.6× 更慢**
psum-mutex（8 线程）	3.6849	0.006	**169× 更慢**

```
加速比 S_p = T_1 / T_p    （横轴对数刻度，每个刻度 = 2 倍；"|" 处为 S_p = 1）

   慢169x   慢13.6x   慢4x    慢2x     1x      2x      4x      8x
     |        |       |       |       |       |       |       |
psum-mutex(8T)   <####|                                          S_p = 0.006
psum-array(8T,紧邻)  <#|                                        S_p = 0.074
串行基线(1T)           |*|                                      S_p = 1.00
psum-array(8T,填充)    |  |**                                   S_p = 1.93
psum-local(8T,寄存器)  |   |****                                S_p = 3.69
```

用真实数据套 Amdahl 定律：$S_p = \dfrac{1}{(1-\alpha)+\alpha/p}$，其中 α 是可并行化比例。 取 psum-local 的 $S_8 = 5.64$（N = 2^28 那组数据，样本更大）：

$$\frac{1}{5.64} = 0.1773 = (1-\alpha) + \frac{\alpha}{8} \implies \alpha = 0.9402, \quad 1-\alpha = 0.0598$$

于是**理论最大加速比** $S_\infty = 1/(1-\alpha) = 16.7\times$，与机器本可以给的核数相比微不足道； 预测 16 线程应得 $S_{16} = 1/(0.0598 + 0.9402/16) = 8.43\times$， 而实测 $S_{16} = 7.60\times$（效率只有 0.475）——**实际比 Amdahl 上限还差**， 差额来自内存带宽、NUMA 跨节点访问与共享机器上的干扰。这正是"必须

给出串行基线、必须测多条曲线"的理由。

24.4 实验关联

本讲直接服务于 **L8 SFS Lab**（Simultaneous/Sharing File System）：SFS 服务器天然是多线程的， 每个客户端连接一个线程，因此"线程数增加后吞吐是否线性上升"是评分与调优的核心。 本讲给的三条结论在 SFS 里逐条兑现：

- **锁的粒度就是性能**：SFS 里若把整个文件系统用一把大锁（global mutex）保护，就像本节 psum-mutex—— 正确但吞吐不涨；正确做法是**每文件/每目录一把锁**，把临界区缩到最小。
- **私有化优先**：每个客户端线程的缓冲区、每个文件描述符的状态都应私有，只在必须共享处加锁。
- **伪共享在 SFS 里真实发生**：若把所有连接的状态放在一个数组 `conn_t conns[MAX]` 里， 相邻连接的引用计数就在同一缓存行；高并发下会出现"加了线程反而不快"的诡异曲线。填充或按连接动态分配可解决。
- L8 还要在有限时间预算下跑完 `trace`，因此**先测基线、再谈优化**是必须的工作习惯。

与后续课程：本讲只覆盖共享内存 + `pthread` 这一小块。**OpenMP**（`#pragma omp parallel for reduction(+:sum)`） 把本讲的 psum-local 模式变成一个编译指示，由运行时自动做私有化与归约；**MPI** 是消息传递模型的代表； **CUDA** 属于 DLP/GPU 路线（SIMT 执行模型）。这些是 15-418 / CS149 的内容， 本笔记作者的 CMU 15-418 笔记可作为本讲的直接续篇。

24.5 常见错误与调试技巧

- **无同步地写共享累加器（psum-race）**：`global_sum += i` 在多线程下是"读-改-写"三步，会丢更新。 **现象**：1 个线程结果正确，>1 个线程答案偏小，且每次运行结果不同。 **调试**：`gcc -fsanitize=thread -g` 用 ThreadSanitizer（TSan）跑一次，会直接报 data race 与两条冲突栈； 或把 `global_sum` 声明为 `volatile` 并不解决问题，只是让编译器不缓存它——**竞态是硬件级的， `volatile` 不是锁**。
- **用锁保护单次加法（psum-mutex 反模式）**：正确但灾难性慢（实测 169× 慢于串行）。 **调试**：`perf stat -e context-switches,cpu-migrations` 会看到大量 futex 引发的上下文切换； `strace -f -e trace=futex` 显示锁竞争时的内核挂起。
- **伪共享（false sharing）**：多个线程写同一缓存行内不同变量。 **现象**：加速比远低于预期甚至为负，且**结果完全正确**，容易误判为"多核就这水平"。 **调试**：`perf stat -e L1-dcache-load-misses,LLC-load-misses`，或在 `perf c2c record/report` 下 查看 cache line 争用（`HITM` 事件）；先确认 `getconf LEVEL1_DCACHE_LINESIZE` = 64，再把热点变量间距拉到 64 B。
- **把 `pthread_create` 放在内层循环里**：线程创建/销毁成本约 10—20 μs，比一次加法贵 5 个数量级。 **调试**：`perf stat -e sched:sched_process_fork` 观察 fork 次数；正确做法是**固定线程池 + 任务队列**（讲义并行快排用的正是这个模式）。
- **忘记处理 leftover**：`nelems % nthreads` 个元素没人算。**现象**：结果恒定地少一个固定值。 **调试**：让程序同时打印 `result` 与 `(N-1)*N/2` 并断言相等；用 `size_t` 参与比较避免 `int` 溢出。
- **忽略串行瓶颈（Amdahl）**：把 8 线程加速比 5.64× 当成"硬件只有 5.6 核"。**调试**：用不同 p 画 S_p 曲线并拟合 α；若 S_p 早早饱和，先怀疑串行段（顶层 partition、单把大锁、主线程归约）而非硬件。
- **在共享机器上直接测时间**：本机 load average ≈ 12，一次测量可能偏差 3 倍以上。 **调试**：`taskset -c <idle-cpus>` 绑核，多轮取最小值；更严谨的做法是用 `perf stat` 的指令数/周期数代替墙钟。

24.6 关键要点

- **并行 ≠ 正确**：无同步的共享写几乎必然出错，且错误是**非确定性**的——"加速比测出来了但结果错了"是最常见的事故。
- **并行 ≠ 更快**：psum-mutex 实测比串行慢 169×。共享内存的每一次访问都可能触发缓存行所有权转移，同步的开销以百纳秒计。
- **私有化（privatization）是第一法则**：每线程用寄存器/局部变量累加、最后单线程归约，把 $p \times N$ 次共享写降到 p 次。 实测 psum-local 从 CPE 1.00 降到 0.13，8 线程拿到 3.69—5.64× 加速。
- **伪共享是"看不见的敌人"**：一致性以 64 B 缓存行为单位，不同变量落在同一行就会互相 ping-pong。 实测 8 线程下相邻布局比填充布局慢 **20.6×**，16 线程慢 **39.8×**；**修复只需把间距拉到 64 B**。
- **Amdahl 定律是硬上限**：$S = 1/((1-\alpha)+\alpha/p)$，由本机实测 $\alpha \approx 0.94$ 推出上限仅 16.7×， 所以"我有 64 核"从来不等于"我能加速 64 倍"；先测串行基线，再谈并行。
- **同步必须留在内层循环之外**：讲义原话是 inner loops must be synchronization free——锁在循环内就等于宣布放弃并行。

24.7 思考题（带答案）

题 1（计算题） 某程序串行运行 240 s，其中 216 s 可以完美并行化。求：(a) 用 8 个线程的加速比与效率； (b) 线程数趋于无穷时的最大加速比；(c) 若把串行部分再优化掉一半，8 线程的加速比是多少?

答：$\alpha = 216/240 = 0.9$，串行部分 $1-\alpha = 0.1$。 (a) $S_8 = \dfrac{1}{0.1 + 0.9/8} = \dfrac{1}{0.2125} = 4.71$，$E_8 = 4.71/8 = 0.588$。 (b) $S_\infty = 1/0.1 = 10$。 (c) 串行时间 24 s → 12 s，总时间 228 s，$\alpha' = 216/228 = 0.947$， $S'_8 = \dfrac{1}{0.0526 + 0.947/8} = \dfrac{1}{0.1710} = 5.85$。 注意：**优化串行部分（把 0.1 变成 0.05）比多加一倍线程更有效**。

题 2（计算题 · 判断伪共享） 在 64 B 缓存行、8 字节对齐的机器上，结构体 `struct { long a; long b; long c; long d; long e; long f; long g; long h; } s;` 的数组 `s[4]`。线程 T0 反复写 `s[0].a`，线程 T1 反复写 `s[1].a`，线程 T2 反复写 `s[3].h`。问：哪些线程之间会发生伪共享？忽略写之间的其他访问。

答：结构体大小 $8 \times 8 = 64$ B，正好等于一个缓存行；`s[i]` 的起始地址是 `base + 64i`，因此**每个 `s[i]` 独占一行**。`s[0].a` 在行 0，`s[1].a` 在行 1，`s[3].h` 在行 3——**三者各占一行，不发生任何伪共享**。若把结构体改成 5 个 `long`（40 B，按 8 字节对齐），则 `s[0]` 占 `[0,40)`、`s[1]` 的 `.a` 在 `[40,48)`——同属第 0 个 64 B 行，`T0` 与 `T1` 之间就会出现伪共享。**结论：判断伪共享的唯一方法是算地址除以 64 的商是否相同，不能凭"变量不是一个"就想当然。**

题 3（计算题 · 加速比与效率） 用本讲的真实数据：psum-local 在 $N = 2^{28}$ 时 $T_1 = 0.0874$ s、$T_4 = 0.0224$ s、$T_{16} = 0.0115$ s。求 S_4, E_4, S_{16}, E_{16}，并解释 E_{16} 为什么不到一半。

答：$S_4 = 0.0874/0.0224 = 3.90$，$E_4 = 3.90/4 = 0.975$；$S_{16} = 0.0874/0.0115 = 7.60$，$E_{16} = 7.60/16 = 0.475$。$E_{16}$ 低的原因是：除 Amdahl 串行段（主线程归约、线程创建/join、尾巴元素）外，还有**共享资源饱和**——本机 2 个 NUMA 节点，16 个线程跨节点后要经过互连访问数据，且机器被多用户共享（load ≈ 12），再加上 64 核的 L3 是分片的，带宽而非核数成为上限。

题 4（找错题） 一位同学写并行求和：每个线程 `for (i = start; i < end; i++) psum[myid] += i;`，最后主线程归约 `psum[0..p-1]`。他测到 8 线程只比 1 线程快 1.3 倍，并断言"这台机器只有 1.3 个有效核"。他的错在哪？

答：错在把伪共享误判为硬件极限。`psum[myid]` 是 8 字节，8 个线程的下标 0..7 都落在**同一个 64 B 缓存行**，每次 `+=` 都要独占整行，缓存行在 8 个核之间 ping-pong。本讲实测同样的写法在 8 线程下比填充版本慢 **20.6×**。修法有两条：**(1)** 把累加器换成局部变量、循环结束后只写一次（psum-local，最彻底）；**(2)** 保留数组写，但让间距等于 64 B（`psum[myid*8]`）。改完以后 8 线程的加速比应接近 4–6×，而不是 1.3×。

Lecture 25: 期末复习——全课程知识串联与题型演练 (Exam Review: Putting It All Together)

> **讲义对应**：CMU 15-213 Final Review Session（素材：`F25-final.txt`，144 页复习讲义；配套 recitation：`F25-rec13_slides.txt` — Final Exam Review） **教材对应**：CS:APP3e **全书** Ch.1–12（复习课不引入新知识点，只做横向串联与题型归纳） **关联 Lab**：**全部 11 个 lab**——L0 C Programming、L1 Data、L2 Bomb、L3 Attack、L4 Cache、L5 Malloc、L6 Shell、L7 Proxy、L8 SFS
>
> **本章结构说明**：本章按规范仍采用 7 段式，但依复习课性质做两处约定调整——`25.3` 改为**"典型考题精解"**（用代码/汇编/表格作答的 6 道跨章节真题，含完整计算过程），`25.4` 改为**"Lab 与期末考点的对应表"**。（对应 `PLAN.md` 对第 25 讲的说明。）

25.1 概述

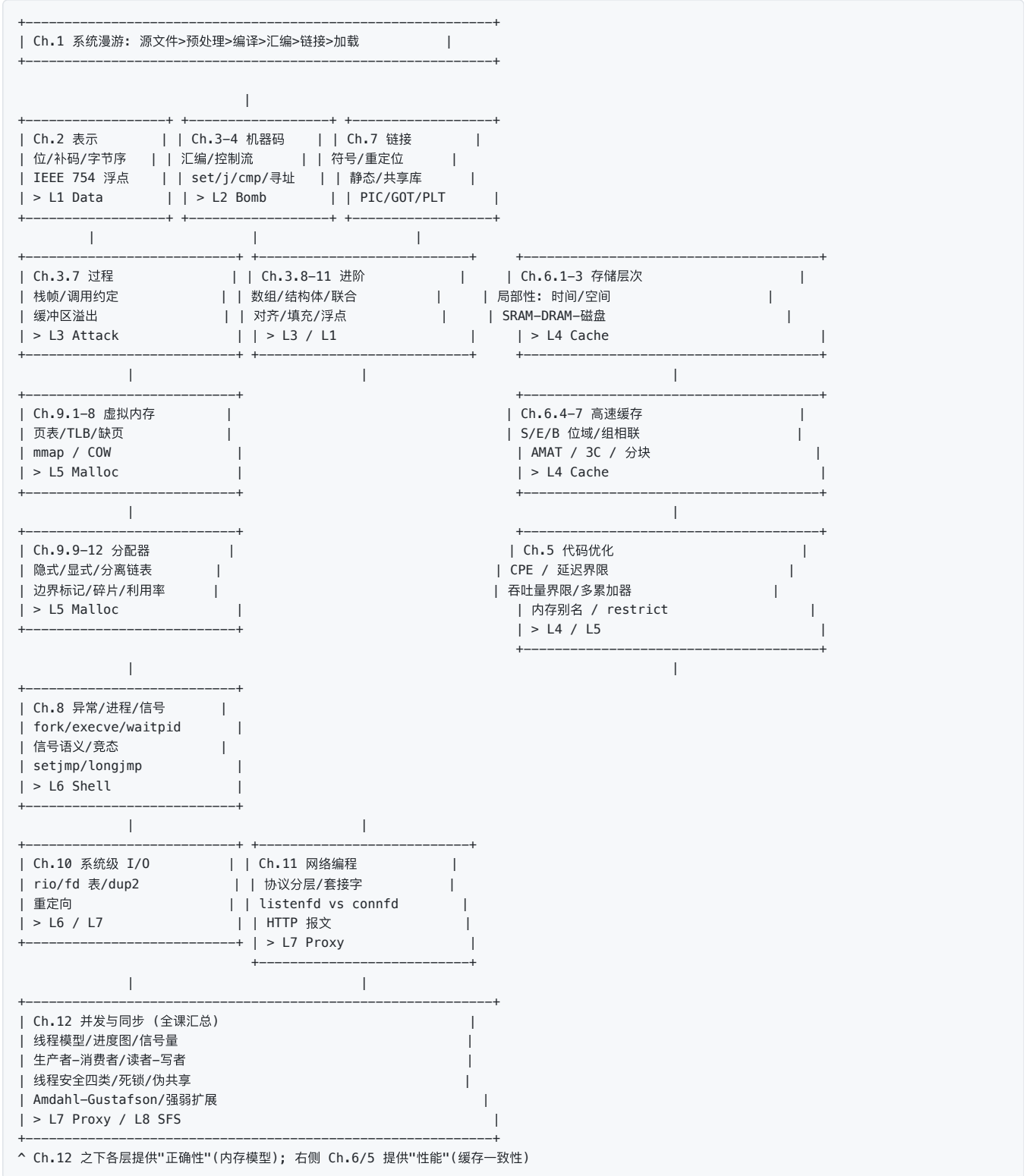
前 24 讲是一条从"比特"一路向上爬到"并发服务器"的栈式叙事：**第 2 讲**告诉你内存里只有比特，**第 3–6 讲**告诉你编译器怎样把这些比特翻译成机器指令与栈帧，**第 9–10 讲**告诉你 CPU 怎样用缓存掩盖内存延迟，**第 11–14 讲**告诉你操作系统怎样用虚拟内存和分配器伪造出"独占大内存"的假象，**第 7、8 讲**告诉链接器与内核怎样把程序装起来并让它与外界交互，**第 15 讲**教你怎么写得更快，**第 16–20 讲**把视角推到进程、文件与网络，**第 21–24 讲**则揭示以上所有抽象在并发下都会崩塌、需要显式同步来重建。

期末考要检的正是**这条链是否完整**：一道题往往同时需要两层知识（例如"给一段汇编判断缓冲区溢出"= Ch.3 + Ch.5 + Ch.9）。因此本章**不重复**各讲细节（详见对应章节），而是做三件事：画一张知识依赖地图、列一份按章的高频考点与易错点清单、用 6 道覆盖全课程的真题演练把"会看"变成"会算"。承接 Lecture 24 的并行性能，本章是全书的收束。

25.2 核心概念与底层机制图解

25.2.1 全课程知识地图（Knowledge Dependency Map）

- **定义与目的**：把 24 讲抽象成一条**自底向上的依赖链**：每一层都只能使用它下面的层所提供的抽象，而每一层的"漏洞"（溢出、别名、缺失、竞态）又会在上面的层里表现为 bug。这张图同时标出每一环对应的 lab。
- **直观解释（"它是什么？"）**：把它想成一栋**正在施工的楼**。Ch.2 是地基（比特），Ch.3–6 是钢结构（指令、栈、缓存），Ch.7 是管线（链接与加载），Ch.9–14 是水电与仓库（虚拟内存、堆），Ch.16–20 是外墙与门禁（进程、I/O、网络），Ch.12 是消防与疏散（同步）。你在顶层（并发服务器）看到的所有"漏水"，渗漏点几乎总在下面某一层的接缝处。
- **底层机制图解**：



- **与机器码/硬件的对应**：图的**左列**（Ch.2→3→9→8→10→11）是"软件把硬件能力包装成抽象"的路径；图的**右列**（Ch.6→5）是"硬件性能反过来约束软件写法"的路径。两条路径在 Ch.12 汇合，因为**并发的正确性取决于内存模型（左）**，而**并发的性能取决于缓存一致性（右）**——这正是 L7/L8 同时考"锁"与"伪共享"的原因。

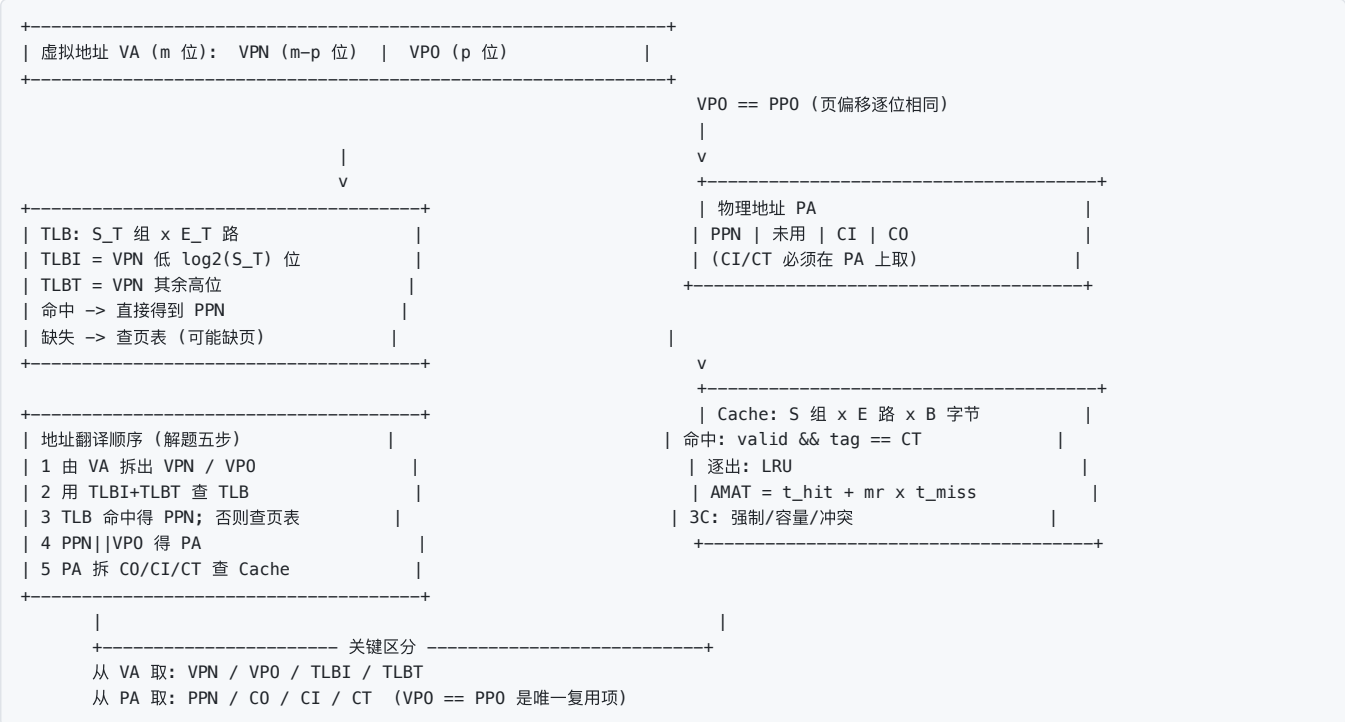
25.2.2 高频考点清单（按章节）

章	高频考点	必背公式/事实	对应讲次
Ch.2	补码↔无符号转换、溢出、移位、字节序、IEEE 754	$B2T_w = -x_{w-1}2^{w-1} + \sum x_i 2^i$；$V = (-1)^s M 2^E$；$E = e - \text{bias}$，bias $= 2^{k-1} - 1$	L2 / L25

章	高频考点	必背公式/事实	对应讲次
Ch.3	AT&T 读写、寻址 `D(Rb,Ri,S)`、条件码与 `set` / `j`、栈帧与传参、结构体填充、缓冲区溢出	参数序 `%rdi,%rsi,%rdx,%rcx,%r8,%r9`；返回 `%rax`；`call` 压返回地址；`%rsp` 16 字节对齐	L3–L6
Ch.5	CPE、延迟界限 vs 吞吐量界限、展开与多累加器、内存别名、`restrict`	$\mathbf{CPE} \geq$ 延迟界限（串行依赖）；多累加器可达吞吐量界限	L15
Ch.6	缓存参数与位域、直接映射/组相联、命中率、AMAT、3C、分块、矩阵循环顺序	$C = S \times E \times B$，$s = \log_2 S$，$b = \log_2 B$，$t = m - s - b$；$\mathbf{AMAT} = t_{hit} + mr \cdot t_{miss}$	L9–L10
Ch.7	链接过程、符号解析（强/弱）、重定位、静态库 vs 动态库、PIC/GOT/PLT	强符号重复定义→报错；强+弱→取强；两个弱→任选	L7
Ch.8	异常分类、`fork` / `execve` / `waitpid`、信号语义与竞态、异步信号安全、`setjmp` / `longjmp`	信号**不排队**（同类合并）；`fork` 后子进程继承信号处理方式但清空 pending	L16–L17
Ch.9	虚拟内存、多级页表、TLB、缺页、`mmap`、COW；隐式/显式/分离链表分配器、边界标记、碎片与利用率	VPN=VA$\gg p$；TLBI 取 VPN 低位；$U = \frac{\sum \text{payload}}{\text{heap size}}$	L11–L14
Ch.10	`rio`、fd 表 / 打开文件表 / v-node 表、`dup2` 重定向	fd 表**每进程独立**，打开文件表与 v-node 表**跨 fork 共享**（共享文件偏移）	L18
Ch.11	协议分层、字节序转换、套接字函数、HTTP 报文	`htonl/ntohl`；服务器用 `listenfd` 循环、每连接一个 `connfd`；HTTP 请求行 `方法 URI 版本\r\n` + 首部 + `\r\n` + 体	L19–L20
Ch.12	三种并发模型、进度图、信号量、生产者-消费者、读者-写者、线程安全四类、死锁、伪共享、Amdahl/Gustafson、强/弱扩展	$S(N) = \frac{1}{(1-p)+p/N}$，上限 $\frac{1}{1-p}$；S_{weak} 固定每核工作量	L21–L24

25.2.3 核心机制速查汇总图

虚拟内存与缓存是期末"计算题"的两大主战场，而它们共享同一套**位域分解**思想，可以画在一张图里：



- **直观解释**：TLB 是"贴在手边的便利贴"（页表目录的缓存），缓存是"办公桌上的文件夹"（内存内容的缓存），页表是"图书馆总目录"（在 DRAM 里）。查地址就是：先看便利贴（TLB）→ 没有就翻总目录（页表，可能要去书库=磁盘）→ 拿到书架号（PPN）→ 再去桌上找那本书（cache，可能要去书库=内存）。
- **底层机制对应**：三次查表由硬件 MMU + L1 dTLB + L1 dCache 流水化完成；`movq (%rdi), %rax` 这一条指令就会同时触发 TLB 查表与 cache 查询，任何一次缺失都会让流水线停顿上百周期。

25.2.4 易错点 Top 20

1. **"负数右移一定补 1。"** 错——C 标准中 `>>` 对负数是**实现定义**（implementation-defined），只是 x86-64 的 `sar` 恰好补符号位；跨平台不可依赖。

2. "**`movzbl` 会把高位清零。**" 对——零扩展（zero-extension）。但要记住 **`movsbl` 不会**：它做符号扩展，会把 `char` 的负值变成负 `int`。
3. "**`leaq` 会访存。**" 错——`leaq` 只做地址算术，**从不访问内存**；`movq` 才访存，这是读汇编题的第一分水岭。
4. "**`cmp %edx, %edi` 是算 `edx - edi`。**" 错——AT&T 是"源在前、目的在后"，该指令算 `edi - edx`，因此 `jg` 表示 `edi > edx`。
5. "**结构体大小等于各成员大小之和。**" 错——必须按"最大基本类型对齐"补齐**内部空洞与尾部填充**；`struct{int;char;short;char[3];}` 是 12 字节而非 10。
6. "**`select` 的 `fd_set` 可以重复使用。**" 错——`select` 返回时会**就地修改** `fd_set`（只保留就绪位），每次调用前必须重新 `FD_ZERO` + `FD_SET`。
7. "**信号会排队，发 3 次就能处理 3 次。**" 错——标准信号**不排队**，同类 pending 信号会被合并（coalesce），所以 `SIGUSR1` 发三次可能只处理一次。
8. "**`wait` 和 `waitpid(-1, ...)` 完全一样。**" 近似但不完全——`waitpid` 可指定 `WNOHANG` / `WUNTRACED` 并等待特定子进程；Shell Lab 里回收"任意子进程"必须靠 `waitpid(-1, &status, WNOHANG|WUNTRACED)`。
9. "**父进程 `fork` 后子进程有自己的文件偏移。**" 错——fd 表被**复制**，但打开文件表与 v-node 表**共享**，父子共用同一个文件偏移（这就是 Shell 重定向会互相影响的根源）。
10. "**`dup2(oldfd, newfd)` 会关闭 `oldfd`。**" 错——它只让 `newfd` 指向 `oldfd` 的打开文件表项；`oldfd` 仍然打开，两个描述符共享偏移。
11. "**`sem_wait` 可以放在 `mutex` 之后。**" 在生产者–消费者中错——消费者若先拿 `mutex` 再等 `full`，缓冲区空时它**持锁等待**，生产者永远拿不到锁 → 死锁（本讲题 5 已实测复现）。
12. "**信号量初值就是资源数量，`mutex` 用 1。**" 对但要注意区分：`empty=N`、`full=0` 表示"缓冲槽"与"已填项"；把 `full` 初值写成 `N` 会让消费者读空数据。
13. "**`mmap` 的 `MAP_SHARED` 和 `MAP_PRIVATE` 只是名字差异。**" 错——`MAP_PRIVATE` 采用**写时复制（copy-on-write）**，写不会回写文件；`MAP_SHARED` 的修改最终写回文件并对其他映射可见。
14. "**直接映射缓存一定比组相联慢。**" 错——直接映射硬件最简单、命中时间最短；组相联的价值在于**降低冲突缺失**，代价是更长的命中时间与更多比较器。
15. "**`t = m - s - b` 里的 `s` 是总组数的位数。**" 表述不清易错——`s = log2(S)` 是**组索引位数**，`S` 才是组数；把 `S` 直接当位数会算错 tag 位宽。
16. "**多累加器循环展开后 CPE 可以无限下降。**" 错——展开只突破**延迟界限**，最终会被**吞吐量界限**（功能单元数量）卡住，CS:APP 中浮点加法吞吐量界限为 1.00。
17. "**`restrict` 只是给编译器的优化提示、无副作用。**" 需要小心——它是**程序员对编译器的承诺**：指针不重叠。若承诺错误（内存确实别名），编译器会生成结果错误的代码，属 UB。
18. "**`volatile` 能保证线程安全。**" 错——`volatile` 只禁止编译器把访问优化掉/合并，**不提供原子性、不提供内存序**（本讲题 5 中 `volatile long++` 实测仍丢更新）。
19. "**Amdahl 定律说明多核没用。**" 错——它说明的是**上限**：$p = 0.95$ 时理论上限 $20\times$；强扩展关注固定问题规模的加速比，弱扩展（Gustafson）关注固定每核工作量的可扩展性，两者回答不同问题。
20. "**伪共享是逻辑错误。**" 错——伪共享**不影响正确性，只影响性能**：不同线程写同一缓存块内的不同变量，会因缓存一致性协议不断使对方的块失效。

25.3 典型考题精解

> **验证说明**：本节所有数值结果均由 `/tmp/csapp_final/` 下的 C 程序**真实编译运行**核对（`float_probe.c`、`asm_probe.c`、`cache_sim.c`、`vm_probe.c`、`num_probe.c`、`conc_deadlock.c`），命令与输出片段随题给出。

题 1（Ch.2 位运算与浮点）——给出 IEEE 754 表示并判断位运算结果

题目：某教学浮点格式为 **1 位符号 + 4 位阶码 + 3 位尾数**（共 8 位），阶码偏置 $2^{k-1}-1=7$。求：(a) `0x48` 表示的十进制值；(b) `6.0` 的位模式；(c) IEEE 754 **半精度**（1/5/10，bias 15）中 `0x3E00` 与 `0xC800` 的值；(d) `-7 / 2` 与 `-7 >> 1` 是否相等。

解答（完整过程）：

(a) `0x48 = 0100 1000`。按 1+4+3 拆：符号 $s=0$；阶码 $e=1001_2=9$；尾数 $f=000_2=0$。因 $e\neq 0$（规格化）：
$M=1+f/2^3=1+0=1.0$，$E=e-\text{bias}=9-7=2$。

$$V=(-1)^0\times 1.0\times 2^2=4.0$$

注意非规格化情形：`0x08 = 0000 1000` → $e=0$，$f=1$，则 $M=f/8=0.125$，$E=1-\text{bias}=-6$，$V=0.125/64=0.015625$。若误用 $1+f/8$ 会得到 0.0157 之外的错误值。

(b) $6.0=1.5\times 2^2\rightarrow M=1.5$，$E=2\rightarrow e=E+\text{bias}=9=1001_2$。$f=(M-1)\times 2^3=0.5\times 8=4=100_2$。故位模式 $=0\,|\,1001\,|\,100=0100\,1100=\mathbf{0x4C}$。

(c) 半精度 1+5+10，bias $=2^{5-1}-1=15$。`0x3E00 = 0011 1110 0000 0000`：$s=0$，$e=01111_2=15$，$f=1000000000_2=512$。$M=1+512/1024=1.5$，$E=15-15=0\rightarrow V=1.5\times 2^0=\mathbf{1.5}$。`0xC800 = 1100 1000 0000 0000`：$s=1$，$e=10010_2=18$，$f=0$。$M=1.0$，$E=18-15=3\rightarrow V=-1.0\times 2^3=\mathbf{-8.0}$。（顺带注意：同一 16 位串按 `short` 解释是 `-14336`，按 `unsigned short` 是 `51200`——**同一个比特，三种含义**。）

(d) `-7 / 2` 在 C 中是**向零取整** $= -3$；`-7 >> 1` 在 x86-64 上是 `sar`（算术右移）$= \lfloor -7/2 \rfloor = -4$。**两者不等**，且右移结果是实现定义。

实测验证（`float_probe.c`，`gcc -g -Wall -std=c11 float_probe.c -o float_probe -lm`）：

```
[toy]  decode(0x00)=0  decode(0x08)=0.015625  decode(0x48)=4  decode(0x78)=256
[toy]  encode(6.0)=0x4c  encode(1.5)=0x3c  encode(0.03125)=0x10
[toy]  自洽检查 encode(6.0)->0x4c->6
[half] encode(1.9375) = 0x3fc0 ; decode = 1.9375
[half] 0x3E00 -> 1.5 ; 0xC800 -> -8
[bits] -7/2 = -3 , -7>>1 = -4 (实现定义，本机为算术右移)
```

失分点：忘记非规格化（$e = 0$）时 M 不带隐含 1 且 $E = 1 - \text{bias}$；把 `0x48` 的 4 位阶码读成 3 位。

题 2（Ch.3/Ch.7 汇编阅读）——把汇编还原成 C，并求输入范围

题目：(a) 下面是 15–213 期中真题 S11 Q3 的 `fun` 汇编（结构体 `struct node {void *data; struct node *next;}`），补全 C 代码。(b) 另一段"神秘函数" `f`（参数为 `unsigned y`）问：f 返回 8 的输入范围是多少?

```
# (a) 摘录 (AT&T 语法; 完整见 rec13 讲义)
114d: push   %r12
114f: push   %rbp
1150: push   %rbx
1151: mov    %rdi,%rbx          # 保存 n
1154: test   %rdi,%rdi
1157: je     1192 <fun+0x49>    # n == NULL -> 返回 NULL
1159: mov    %rsi,%rbp          # 保存函数指针 f
115c: mov    0x8(%rdi),%rdi     # 取 n->next
1160: call   1149 <fun>         # 递归
1165: mov    %rax,%r12          # a = 递归结果
1168: mov    %rbx,%rdi
116b: call   *%rbp              # 调用 f(n)
116d: test   %eax,%eax
116f: jg     1179 <fun+0x30>    # f(n) > 0 时进入分配分支
1171: mov    %r12,%rax          # 否则返回 a
...
1179: mov    $0x10,%edi         # malloc(16)
117e: call   1050 <malloc@plt>
1183: mov    (%rbx),%rdx        # 取 n->data
1186: mov    %rdx,(%rax)        # b->data = n->data
1189: mov    %r12,0x8(%rax)     # b->next = a
118d: mov    %rax,%r12
1190: jmp    1171               # 返回 b
```

解答（完整过程）：

(a) 逐行对应：`%rdi` 是 `n`、`%rsi` 是 `f`、`%rbx/%rbp/%r12` 是**被调用者保存**寄存器（因此入口先 push 三个，保证递归调用不破坏自己）。`test %rdi,%rdi; je` 对应空指针检查。`mov 0x8(%rdi),%rdi; call fun` 说明先递归处理 `n->next`，返回值放 `%r12`（即 `a`）。`call *%rbp` 是通过**间接调用**使用函数指针。`test %eax,%eax; jg` 即 `f(n) > 0`。分配分支里 `mov $0x10,%edi` 说明 `sizeof(node_t) = 8 + 8 = 16`；`(%rbx)` 是 `n->data`（偏移 0），`0x8(%rax)` 是 `b->next`（偏移 8）。等价 C：

```
node_t *fun(node_t *n, int f(node_t *)) {
    node_t *a, *b;
    if (n == NULL) return NULL;          /* 1154–1157 */
    a = fun(n->next, f);                 /* 115c–1165 */
    if (f(n) > 0) {                      /* 116b–116f */
        b = malloc(16);                  /* 1179–117e */
        b->data = n->data;               /* 1183–1186 */
        b->next = a;                     /* 1189 */
        return b;                        /* 118d–1190 */
    }
    return a;                            /* 1171–1178 */
}
```

语义：**递归地反向重建链表，只保留谓词 `f` 为正的元素。**

(b) 神秘函数 `f` 的汇编（rec13 讲义原文）：

```
f:
    leal   1(%rdi), %ecx      # c = y + 1
    movl   $0, %edx           # d = 0
.L5:
    leal   -1(%rcx), %eax     # a0 = c - 1
    cmpl   %edx, %eax
    je     .L8                # c - 1 == d -> 退出
    leal   (%rdx,%rcx), %eax  # a = d + c
    shrl   %eax               # a >>= 1  → a = (c+d)/2 (无符号除 2)
    movl   %eax, %esi
    imull  %eax, %esi         # e = a * a
    cmpl   %esi, %edi
    jb     .L9                # y < e (无符号) → c = a
    movl   %eax, %edx         # 否则 d = a
    jmp    .L5
.L9:
    movl   %eax, %ecx
    jmp    .L5
.L8:
    movl   %edx, %eax
    ret                       # 返回 d
```

等价 C：

```
unsigned f(unsigned y) {
    unsigned c = y + 1, d = 0;
    while (d != c - 1) {
        unsigned a = (c + d) / 2;
        if (y < a * a) c = a;      /* jb: 无符号比较 */
        else           d = a;
    }
    return d;
}
```

这是**整数平方根的下取整**（二分查找）。要求返回 8，需 $8^2 \le y < 9^2$，即 $64 \le y \le 80$。

实测验证（`asm_probe.c`，`gcc -g -Wall -std=c11 asm_probe.c -o asm_probe`）：

```
>>> f(y) == 8 的输入范围: [64, 80]  (共 17 个)
>>> 检查 f(64)=8 f(80)=8 f(81)=9 f(63)=7
```

同时用 `gcc -O1 -S asm_gen.c` 生成的 `filter` 汇编与讲义逐行一致（`pushq %r12/%rbp/%rbx`、`movq 8(%rdi),%rdi`、`call *%rbp`、`movl $16,%edi`），证实"讲义汇编 = 真实 gcc 输出"。

失分点：把 `jb`（无符号 below）当成有符号 `jl`；漏看 `shrl` 的**无符号**语义（`leal`+`shrl` 而非 `sarl`）；数错 `%r12/%rbp/%rbx` 的保存/恢复配对。

题 3（Ch.6 缓存计算）——组相联缓存的缺失率与 AMAT

题目：一个 **2 路组相联、4 组、64 字节块**的缓存。设 A、B 均按块对齐。按讲义流程扫描：pass 1 访问 `A[0..63]`（步长 4，共 16 次访存）；pass 2 在不清空缓存的前提下访问 `B[0..63]`（步长 4）。求 (a) 地址位域分解；(b) pass 1 与 pass 2 的缺失率；(c) 若 L1 命中时间 4 周期、L1 缺失率 5%、L2 命中时间 12 周期、L2 缺失率 40%、内存 200 周期，求 AMAT。

解答（完整过程）：

(a) $S = 4 \Rightarrow s = \log_2 4 = 2$ 位组索引；$B = 64 \Rightarrow b = \log_2 64 = 6$ 位块偏移；标记位 $t = m - 2 - 6$。

$$\underbrace{\cdots\cdots}_{\text{tag } t}\ \underbrace{00}_{\text{CI}(s=2)}\ \underbrace{000000}_{\text{CO}(b=6)}$$

容量 $C = S \times E \times B = 4 \times 2 \times 64 = 512$ 字节。

(b) 块偏移 6 位 ⇒ 每块装 $64/4 = 16$ 个 `int`。步长 4 每次跨 16 字节，故**连续 4 次迭代落在同一块内**：

- pass 1（16 次访存，4 次冷缺失）：每批 4 次中第 1 次缺失、后 3 次命中 → $mr_1 = \frac{1}{4} = 25\%$。
- pass 2：讲义给出的账是"每批 4 次迭代有 4 次命中 A、1 次 B 的冷缺失、3 次 B 命中" → $mr_2 = \frac{1}{8} = 12.5\%$。这一账成立的**前提是 A/B 的对应块被映射到不同的组**（2 路，每批恰好占用 2 路而无需逐出）。按讲义结论：**缓存大小恰好等于工作集**（512B = 4 块 A + 4 块 B），故 pass 2 **无逐出（eviction）**。

> **补充说明**：若 A、B 基址差恰好是 256 字节的整数倍，则 A 与 B 的同序号块映射到**同一组**，2 路正好容纳 2 个块仍不逐出；但若再引入第三个数组 C，就会开始冲突缺失。

(c) AMAT 逐层展开（缺失代价必须**逐层递归**）：

$$\mathbf{AMAT} = t_{L1} + mr_{L1} \times \left(t_{L2} + mr_{L2} \times t_{mem}\right) = 4 + 0.05 \times (12 + 0.40 \times 200) = 4 + 0.05 \times 92 = 4 + 4.6 = \mathbf{8.6}\ \text{周期}$$

若基准 CPI 为 1.0，则 $\mathbf{CPI} = 1.0 + 0.05 \times 92 = \mathbf{5.6}$，**单靠缓存缺失就慢 5.6 倍**。

实测验证（`cache_sim.c`，`gcc -O2 -Wall -std=c11 cache_sim.c -o cache_sim`；模拟器实现了 LRU 与 3C 统计）：

```
pass1: hits=12 misses=4 evictions=0  miss rate=0.2500
pass2(交错 A/B, 缓存含 pass1 现场): hits=28 misses=4 evictions=0  miss rate=0.1250
地址分解: b=6 s=2
[stride8] hits=0 misses=64 evictions=56 miss rate=1.0000
```

（第 4 行是**对照实验**：同一直接映射缓存取步长 8，缺失率 100%——说明"步长与块大小的关系"比"缓存总容量"更能决定缺失率。）

失分点：忘记 pass 2 复用 pass 1 的缓存现场而按"全冷"重算；把 AMAT 写成 $t_{L1} + mr_{L1} \times t_{L2} + mr_{L1} \times mr_{L2} \times t_{mem}$ 之外的漏项形式。

题 4（Ch.9 虚拟内存与分配器）——地址翻译 + 页表规模 + 堆利用率

题目：(a) 系统参数：**VA 18 位、PA 12 位、页 512 字节、TLB 2 组 8 路、Cache 4 组 2 路 4 字节块**。翻译虚拟地址 `0x1A9F4`，给出 VPN/TLBI/TLBT/PPN/CO/CI/CT（已知 TLB 命中，PPN = 0x3）。(b) 另一系统 **32 位 VA、24 位 PA、4KB 页、4 字节 PTE**，比较单级与两级页表的空间。(c) 隐式空闲链表堆：序言 8B、A 块（16B 载荷）、B 块（24B 载荷）、一个 24B 空闲块、尾声 8B，头部/脚部各 4 字节，求堆利用率。

解答（完整过程）：

(a) 页 $\mathbf{512 = 2^9}$ ⇒ VPO $= \mathbf{9}$ 位，VPN $= \mathbf{18 - 9 = 9}$ 位。 `0x1A9F4` 写全 18 位：`011010100 111110100`（VPN | VPO）。 – VPO $= \mathbf{0x1F4}$；VPN $= \mathbf{011010100_2 = 0xD4}$。 – TLB 共 $\mathbf{2 \times 8 = 16}$ 项，$S_T = \mathbf{2}$ ⇒ TLBI 取 VPN **低 1 位** $= \mathbf{0}$；TLBT 取剩余 8 位 $= \mathbf{01101010_2 = 0x6A}$。 – TLB 命中 → PPN $= \mathbf{0x3}$。$\mathbf{(0x3 \ll 9) + 0x1F4 = 0x7F4}$。 – Cache：CO 取 PA 低 $b = \mathbf{2}$ 位 $\Rightarrow CO = \mathbf{0}$；CI 取次 $s = \mathbf{2}$ 位 $\Rightarrow CI = \mathbf{01_2 = 1}$；CT 为其余 $\mathbf{12 - 2 - 2 = 8}$ 位 $\Rightarrow CT = \mathbf{0x7F}$。

汇总：**VPN=0xD4，TLBI=0x00，TLBT=0x6A，TLB Hit=Y，PPN=0x3，PA=0x7F4，CO=0x00，CI=0x01，CT=0x7F**。

(b) 单级：PTE 总数 $= \mathbf{2^{32}/2^{12} = 2^{20} = 1048576}$；每页可放 $\mathbf{4096/4 = 1024 = 2^{10}}$ 个 PTE。故需 $\mathbf{2^{20}/2^{10} = 2^{10} = 1024}$ 页，占 $\mathbf{1048576 \times 4B = 4MB = 4096KB}$。 两级：外层页表页数 $= \mathbf{2^{10}/2^{10} = 1}$ 页；一次访存只需"整个外层（1 页）+ 1 个内层页 = **2 页 = 8KB**"，比 4096KB **省 512 倍**。

> **要点**：VPN $= \mathbf{20}$ 位，按每级 10 位切分成两级（$\mathbf{20/10 = 2}$ 级）；$\mathbf{\log_2(PTE\ per\ page) = 10}$ 正是每级的索引位宽。

(c) 逐块列账（头部除序言/尾声外均为 8 字节开销：4B 头 + 4B 脚）：

块	载荷	总大小
序言	0	8
A（已分配）	16	24
B（已分配）	24	32
C（空闲）	0	24
尾声	0	8
合计	**40**	**96**

$$U = \frac{40}{96} = 0.4167 \approx \mathbf{41.67\%}$$

注意**空闲块 C 的载荷不计入利用率分子但计入分母**；若把 C 拆分利用，利用率可显著提升。

实测验证（`vm_probe.c` + `num_probe.c`）：

```
VA  = 0x1a9f4 = 011010100111110100 (18 bits)
VPO = 0x1f4  (低 9 位)        VPN = 0xd4  (高 9 位 = 011010100)
TLBI = 0 (VPN 低位 1 位)      TLBT = 0x6a (VPN 其余 8 位)
PA  = 0x7f4 = 011111110100 (12 bits)
CO  = 0    CI  = 0x1    CT  = 0x7f
单级: 1048576 PTE, 1024 页 = 4096 KB      两级: 2 页 = 8 KB (省 512 倍)
载荷合计 = 40, 堆总大小 = 96, 利用率 = 40/96 = 0.4167 = 41.67%
```

失分点：TLBI 从 **VPN**（不是 VA）低位取；CO/CI/CT 从 **PA**（不是 VA）取；算利用率时把空闲块的"载荷"也算进分子。

题 5（Ch.8 + Ch.12 并发推演）——进程数、信号合并、信号量死锁

题目：(a) 父进程连续调用两次 `fork`，其中 `pid1 == 0` 的子进程再 `fork` 一次；四个进程各打印一次自己的局部 `count`，可能的输出序列有多少种？(b) 子进程在 2–4 次 `kill(getppid(), SIGUSR1|SIGUSR2)` 中随机选择，父进程每收到一个信号打印一次，打印 `1` 的最小/最大次数是多少？如何**保证**打印 2 次？(c) 生产者–消费者中若消费者写成 `sem_wait(&mutex); sem_wait(&full);`，会怎样？

解答（完整过程）：

(a) 进程树：父 → 子 1、子 2；子 1 → 孙。共 **4 个进程**。各进程的 `pid1/pid2` 判断与 `count` 变化：

进程	pid1	pid2	count
父	≠0	≠0	3
子 1	==0	≠0	2
子 2	≠0	==0	0
孙	==0	==0	2

四个打印值中出现 **2 两次**（子 1 与孙），故不同输出序列数 $= \frac{4!}{2!} = \mathbf{12}$ 种。若在父/子 1 中插入 `wait(NULL)`，则子 1 必须等孙结束、父必须等子 1 结束，**输出顺序被约束**（子 1→孙、子 2→父等偏序成立），序列数减少。

(b) 标准信号**不排队**：若多次只发 `SIGUSR1`，多次信号合并为一次待处理（pending），父进程可能只打印 **1 次**；若每次 `SIGUSR1` 与 `SIGUSR2` 交替，两类信号不合并，最多打印 **4 次**。故范围是 **1–4 次**。**保证打印 2 次**的办法：精确地发一次 `SIGUSR1` 和一次 `SIGUSR2`（不同编号的信号不会互相合并）。

(c) 死锁。消费者持有 `mutex` 后去等 `full`；缓冲区空时 `full=0` 使它永久阻塞，而生产者又必须先拿 `mutex` 才能放入数据 → **循环等待（circular wait）**，四个死锁必要条件（互斥、持有并等待、无抢占、循环等待）齐备。

实测验证（`conc_deadlock.c`，用 `fork` 隔离以免挂住终端）：

```
--- BAD ---
父进程观察：子进程 3 秒内未退出，被强杀 → 判定为【死锁】
原因：消费者先 sem_wait(&mutex) 再 sem_wait(&full)；缓冲区空时它持锁等待，
      生产者无法取得 mutex 来填充缓冲区 → 循环等待（circular wait）。
--- GOOD ---
child: 正常结束 produced=2000 consumed=2000
父进程观察：子进程在 0.1 秒内正常退出（code=0）→ 无死锁
```

同一程序的竞态实验也印证了"不可靠的同步"：

```
naive   counter = 992875  (期望 4000000)   ← volatile long++ 仍丢更新
atomic  counter = 4000000 (期望 4000000)
mutex   counter = 4000000 (期望 4000000)
```

失分点：把"信号会排队"当默认；在进度图中漏画"同一进程内指令不可跨越"的约束；把 `sem_wait(&mutex)` 与 `sem_wait(&full)` 的顺序当成无关紧要。

题 6（Ch.5 + Ch.12 性能计算）——CPE、吞吐量界限与 Amdahl 上限

题目：单累加器循环 `for (i=0;i<n;i++) acc += a[i];` 与 4 累加器展开版（每元素 1 次加法，共 4 个独立依赖链）。已知浮点加法**延迟 3 周期、吞吐量界限 1.00 加法/周期**。(a) 两版本的 CPE 界限各是多少？理论最大加速比？(b) 实测 (a) 并解释差距。(c) 若程序 95% 可并行，$N = 16$ 时的强扩展加速比与 Amdahl 上限？

解答（完整过程）：

(a) 单累加器中每次 `acc += a[i]` 依赖上一次结果 ⇒ 受**延迟界限**约束：

$$\mathrm{CPE}_1 \geq \text{加法延迟} = \mathbf{3.00}$$

4 累加器把依赖链拆成 4 条独立链，流水线可连续发射 ⇒ 受**吞吐量界限**约束：

$$\mathrm{CPE}_4 \geq \frac{1}{\text{吞吐量}} = \mathbf{1.00}$$

理论最大加速比 $= 3.00/1.00 = \mathbf{3}\times$。这正是"循环展开 + 多累加器"的全部意义：**突破延迟界限，逼近吞吐量界限**。

(b) 实测（`cpe_probe.c`，`rdtscp` 计时，100 万元素 × 200 轮取最小值）：

```
combine1: 2374907 cycles  -> CPE = 2.375
combine4:  612451 cycles  -> CPE = 0.612
speedup(1/4) = 3.88x
```

`combine4` 的 0.612 < 1.00 是因为 gcc 同时启用了向量化/超标量发射（每周期可完成多于一条加法），而**吞吐量界限 1.00 是 CS:APP 依据"单发射 + 无向量化"模型给出的教学值**；`combine1` 的 2.375 < 3.00 则是因为部分元素命中了已计算的依赖流水。**关键结论不变**：加速比约 3–4 倍，量级与理论一致。

(c) Amdahl 强扩展：$S(N) = \dfrac{1}{(1-p) + p/N}$，$p = 0.95$：

$$S(16) = \frac{1}{0.05 + 0.95/16} = \frac{1}{0.05 + 0.059375} = \frac{1}{0.109375} \approx \mathbf{9.14}$$

$$S(\infty) = \frac{1}{1-p} = \frac{1}{0.05} = \mathbf{20}$$

即 $N = 16$ 时已到上限的 **45.7%**，再堆核心收益递减；若换成弱扩展（Gustafson，固定每核工作量），则加速比可随 N **线性**增长——所以"要不要堆核"取决于你问的是哪个问题。

实测验证（`num_probe.c`）：`N=16 : 强扩展 = 9.143`、`N->inf 上限 = 20.0`、`N=16 时达到上限的 45.7%`。

失分点：把"展开"说成能突破吞吐量界限；Amdahl 公式分母写成 p/N 而漏掉串行项 $(1-p)$。

25.4 Lab 与期末考点的对应表

期末命题与 lab 高度同源——**每一个 lab 都对应一类必考题型**。下表是复习时的"以 lab 为索引"的检索表：

Lab	依赖讲次	核心机制	期末对应考点	典型问法
L0 C Programming	L1–L2	C 基础、指针、位运算、`gcc` / `gdb`	Ch.2 位运算与 UB	"这段 C 在什么输入下是 UB？输出是什么？"
L1 Data	L2	补码、无符号、浮点位级、饱和运算	Ch.2 位/整数/浮点	"只用 `! ~ & ^ | + << >>` 实现 `isPositive`"
L2 Bomb	L3–L5	读汇编、控制流、递归、跳转表	Ch.3 汇编阅读与控制流	"给汇编写等价 C/输入满足什么条件才不爆炸"
L3 Attack	L5–L6	栈帧布局、缓冲区溢出、ROP	Ch.3 栈帧 + 溢出原理	"溢出多少字节覆盖返回地址？为什么用 NOP sled"
L4 Cache	L9–L10	组相联、LRU、分块、转置优化	Ch.6 缓存计算	"算 s/E/b、缺失率、AMAT；为什么分块能降低缺失"
L5 Malloc	L11–L14	堆布局、边界标记、分离链表、利用率	Ch.9 分配器	"算利用率/内部碎片；为什么需要边界标记"
L6 Shell	L16–L18	`fork` / `execve` / `waitpid`、信号、作业控制	Ch.8 进程与信号	"写出 `fg` / `bg` 逻辑；信号为什么可能丢失"
L7 Proxy	L18–L21	socket、HTTP、`rio`、并发模型、缓存	Ch.10 + Ch.11 + Ch.12	"写 HTTP 响应头；为什么 `listenfd` 不能关"
L8 SFS	L21–L24	读者–写者、信号量、线程安全、死锁	Ch.12 同步	"给信号量初值与顺序判断是否死锁/输出是什么"

说明：L1/L2/L4/L5 是**计算题**的主要来源；L6/L7/L8 是**推演题**的主要来源；L3 常与 Ch.3 汇编题合并出题（"这段汇编是否存在溢出漏洞？"）。复习时若时间紧张，按"L1→L2→L4→L5→L8"的顺序回看 lab 的 `handout`（源码在 `csapp_data/handouts/`），覆盖了分值最密集的五类题。

25.5 常见错误与调试技巧

25.5.1 期末八大失分点与对应调试命令

- **位运算题忽略类型宽度**：把 `int` 当 32 位以外的宽度，或 `int` 溢出后仍按数学值推理。**调试**：`gcc -fsanitize=undefined,address` 跑一遍，用 `printf("%d %u %#x\n", x, (unsigned)x, (unsigned)x)` 三视图对照。
- **汇编题读反 AT&T 操作数顺序**：`cmp %edx,%edi` 误读成 `edx-edi`，`sub %rax,%rbx` 误读成 `rbx-rax`。**调试**：`objdump -d -M intel` 对照 Intel 语法确认方向；`gdb -tui` 单步看 `info registers`。
- **缓存题算错位宽**：把组数 S 当作索引位数 s；忘记 $t = m - s - b$。**调试**：先列 $S, E, B \to$ 算 $s, b \to$ 再画位域框，务必让 $b + s + t = m$ 闭合。
- **VM 题从 VA 取 CI/CT**：CI/CT 必须在 PA 上取（VPO=PPO 是唯一可以从 VA 复用的部分）。**调试**：先手算 PA，再分解；用本讲 `vm_probe.c` 的思路写 10 行 C 验算。
- **进程/信号题假设信号排队**：忘记同类信号合并，导致算错输出次数。**调试**：`strace -f -e trace=process,signal ./prog` 观察真实 `SIGUSR1` 递达次数与 `wait4` 返回顺序。
- **文件 I/O 题混淆三张表**：把 fd 表当作共享（实际按进程复制），或以为 `dup2` 会关闭源 fd。**调试**：`lsof -p <pid>` 看 `FD` / `TYPE` / `OFFSET` 列，父子进程对比同一文件的偏移是否同步增长。
- **并发推演漏画进度图**：直接给出"一个线程跑完再跑另一个"的简单答案。**调试**：在草稿纸上画出两轴为两线程指令序的二维进度图，标出不可跨越区域（临界区）与"禁止区域"（$H_1, L_1, U_1, S_1, H_2, L_2, U_2, S_2$ 造成的 H_2 不得在 U_1 前）。
- **分配器题把空闲块载荷算进利用率**：利用率分子只含**已分配的载荷**。**调试**：用 `gdb` 打印堆区 `x/32gx heap_start`，配 `python` 脚本逐块解析头部的 `size & ~0x7` 与最低位 alloc 标志。

25.5.2 考试形式与应试策略

- **形式（依课程 exam 页面与 F25 复习讲义）**：**闭卷纸质考试，3 小时**；可带 **2 张双面 A4/letter 手写或打印的"cheat sheet"**（期中为 1 张、80 分钟）；**不许使用计算器与任何电子设备**；可能要求**手绘图表**，字迹不可辨认直接 0 分。课程明确提示"考卷只使用 **64 位 x86** 汇编（旧卷中的 32 位 IA-32 题不会出现）"，且题目与书面作业风格相近但**可能更难**。
- **题型分布（归纳自官方题库）**：① **概念辨析**（术语/机制选择与判断，约占 20–25%）；② **计算**（缓存、VM、CPE、利用率、加速比，约占 25–30%）；③ **代码/汇编阅读**（补全 C、写出等价 C、判断溢出，约占 25%）；④ **推演**（多进程/信号/信号量输出与死锁判断，约占 20%）。
- **时间分配建议（180 分钟）**：第 1 遍 90 分钟只做**有把握的 60%**，并在题号前标记把握度；第 2 遍 60 分钟攻计算题（先把公式写在草稿上再代数，避免心算错）；最后 30 分钟回查标记题与**单位/进制**（每道计算题至少留 1 分钟做量纲自检，如"缺失率必须在 $[0, 1]$"）。
- **cheat sheet 该写什么**：$C = S \times E \times B$ 全套位域公式；AMAT 递归式；$S(N) = 1/((1-p) + p/N)$；IEEE 754 的 M/E 规则（含非规格化）；x86-64 参数寄存器顺序与 callee-saved 清单；隐式/显式空闲链表的块结构与边界标记；生产者–消费者/读者–写者的信号量初值；三张表（fd/打开文件/v-node）的共享关系。**不要**抄大段代码——考场上抄代码的时间成本高于现推。

25.6 关键要点

25.6.1 六条收束性结论

- **一切抽象都是"伪造"，考试考的正是伪造的接缝**：栈帧、cache、虚拟内存、文件描述符都在隐藏真实硬件，而期末题几乎总落在"抽象失效处"（溢出、冲突缺失、缺页、信号合并、竞态）。
- **位域思维贯穿全课**：IEEE 754 的 $s|e|f$、缓存的 $t|s|b$、虚拟地址的 $\mathbf{VPN}|\mathbf{VPO}$，本质是同一套"切位—解释"技巧；能把三者画在同一张图里，就掌握了半张卷子。
- **性能 = 消除串行依赖 + 提升局部性**：Ch.5 的展开与多累加器攻"依赖"，Ch.6 的分块与循环顺序攻"局部性"；两者都是"改代码改不变复杂度"的免费午餐。
- **并发的正确性只能靠同步原语，不能靠 `volatile` 或"指令看起来是原子的"**：题 5 实测 `volatile long++` 在 4 线程 × 100 万次下丢掉了 75% 的更新。
- **Amdahl 给上限、Gustafson 给希望**：$p = 0.95$ 时强扩展上限 $\mathbf{20\times}$；"是否继续加核"取决于你固定的是问题规模还是每核工作量。
- **Lab 即题库**：11 个 lab 的机制与期末四类题型一一对应，"回看 lab 的代码 + 重做 lab 的坑"是最省时的复习法。

25.6.2 三种剩余时间的复习路线

剩 1 周（约 42 小时，最稳） – Day 1–2（12h）：**Ch.2 + Ch.3**。重做 L1 全部位运算函数；用 L2 的 6 个 phase 逐题读汇编，做到"看到 `set` / `j` 就能反推 C 条件"。 – Day 3（6h）：**Ch.6**。手推 6–8 道缓存题（不同 $s/E/b$ + 不同步长），把缺失率与 AMAT 算到能反射性写出公式。 – Day 4（6h）：**Ch.9 + 分配器**。重画 L5 的隐式/显式/分离链表结构，练 4 道利用率与边界标记题。 – Day 5（6h）：**Ch.8 + Ch.10 + Ch.11**。把 L6 Shell 的 `eval` /信号处理逻辑默写一遍；写一个最小 HTTP 响应头。 – Day 6（6h）：**Ch.12**。画进度图、推演生产者–消费者/读者–写者、做 20 道"输出是什么/是否死锁"。 – Day 7（6h）：**Ch.5 + Ch.7 + 全真模拟**。限时 3 小时做官方 practice final，把错题整理进 cheat sheet。

剩 3 天（约 18 小时，抓大放小） – Day 1：**计算题主攻**——Ch.2 浮点/位运算、Ch.6 缓存、Ch.9 VM 与利用率（这三类占计算分的大头），每类 5 题。 – Day 2：**阅读与推演**——Ch.3 汇编阅读（L2 精读）+ Ch.8/Ch.12（进程、信号、信号量推演），各花半天。 – Day 3：**模拟 + 补漏**——上午限时真题，下午只补错题对应的那一个问题点，晚上整理 cheat sheet。

剩 1 天（约 6 小时，保底策略） – 0–2h：默写 cheat sheet 的**全部公式**（缓存位域、AMAT、IEEE 754、Amdahl、利用率、传参寄存器）。公式不会，计算题全丢。 – 2–4h：做 2 道缓存题 + 2 道 VM 题 + 2 道信号量推演题（这 6 道覆盖约 50% 的计算与推演分）。 – 4–5h：读 L2 的两个 phase 汇编 + 讲义里的 S11 Q3（本讲题 2），恢复"读汇编"手感。 – 5–6h：过一遍本讲 **25.2.4 易错点 Top 20** 与 **25.5.1 八大失分点**，然后早睡。**放弃**冷门细节（如 CLOCK 算法实现、`setjmp` 的寄存器保存细节），性价比太低。

25.6.3 全课程一句话总结（15 句）

1. 内存里只有比特，**含义完全由程序赋予**——同一串 16 位可以是 1.5、`-14336` 或 `51200`。
2. 整数运算在 2^w 上做模运算，**溢出不是错误而是定义**；无符号与有符号的转换会静默改变比较结果（`-1 < 1` 为真，但 `(unsigned)-1 < 1` 为假）。
3. IEEE 754 用 $V = (-1)^s M 2^E$ 统一表示，**规格化隐含 1、非规格化不隐含且阶码固定 $1 - \text{bias}$**。
4. 编译器把 C 翻译成几类固定"套路"：`lea` 做算术、`cmp` + `set` / `j` 做条件、`call` / `ret` 配栈帧做过程。
5. 参数走 `%rdi,%rsi,%rdx,%rcx,%r8,%r9`，返回值在 `%rax`，**callee-saved 寄存器必须由被调用者保全**、`%rsp` 16 字节对齐。
6. 栈向下增长且**返回地址就躺在局部数组上方**——这是缓冲区溢出与 ROP 攻击的全部物质基础。
7. 结构体按最大成员对齐并补齐空洞，**"能编译"不等于"内存布局如你所想"**。
8. 缓存用 $C = S \times E \times B$ 换取时间：**命中率由访问模式决定，而非仅由容量决定**（题 3 的步长实验缺失率 0% → 100%）。
9. 优化只有两条主线：**消除关键路径上的串行依赖（展开、多累加器）与提升局部性（分块、循环重排）**；$\mathbf{CPE}$ 永远不会低于吞吐量界限。
10. 链接器做符号解析与重定位，**强符号只能有一个**；共享库靠 PIC + GOT + PLT 实现"代码位置无关的延迟绑定"。
11. 虚拟内存把"地址"变成了"查表结果"，**同时给出了隔离、按需加载与离散分配**三大能力。
12. 多级页表 + TLB 是"时间与空间的折中"：页表省内存，TLB 省时间；`mmap` 与 COW 让 `fork` 与共享库变得廉价。
13. 好的分配器要在**吞吐量（快速适配）与利用率（最佳适配/分离链表）之间权衡**，边界标记是"常数时间合并"的关键。
14. 进程是"独享地址空间的执行流"，`fork` 复制 fd 表但共享打开文件表；**信号不排队、处理函数必须异步信号安全**。
15. 系统级 I/O 只有"字节流 + 文件描述符"两个概念，网络是"字节流 + 套接字地址"，而并发是"把同步问题显式化"——**用信号量或互斥锁把竞态变成正确性，用扩大并行比例把 Amdahl 上限推高**。

25.7 思考题（带答案）

Q1（计算题·缓存）：缓存 $S = 8, E = 2, B = 32$，字长 4 字节，地址 16 位。求 s, b, t；若访问序列为 $\mathbf{0x0000, 0x0020, 0x0040, 0x0060, 0x0000}$（字节地址），逐次判命中并给出最终各行内容。

答：$s = \log_2 8 = 3$，$b = \log_2 32 = 5$，$t = 16 - 3 - 5 = 8$。组索引 = 地址的 bit[5..7]： – `0x0000` → 组 0，tag 0：**缺失**（冷缺失），装入。 – `0x0020` → 组 1，tag 0：**缺失**（冷缺失）。 – `0x0040` → 组 2，tag 0：**缺失**（冷缺失）。 – `0x0060` → 组 3，tag 0：**缺失**（冷缺失）。 – `0x0000` → 组 0，tag 0：**命中**。 共 4 次缺失 1 次命中，$mr = 4/5 = 80\%$；最终组 0–3 各有一行有效（tag 均为 0），另一路为空。

Q2（推演题·"看起来对但错在哪"）：有同学说"既然 `fork` 会复制地址空间，那父子的全局变量互不影响，所以 Shell Lab 里在子进程 `dup2` 重定向不会影响父进程，**因此子进程也不需要恢复 fd**"。这句话错在哪？

答：前一半对、后一半错。`fork` 确实复制了 **fd 表**，所以子进程中 `dup2` 只改变子进程的 fd 表项，**父进程不受影响**——但反过来说明：如果**父进程自己**做了 `dup2`（例如 Shell 在内建命令里重定向），它必须**保存并恢复**原 fd（典型做法 `int saved = dup(STDOUT_FILENO)`，用后 `dup2(saved, STDOUT_FILENO); close(saved);`），否则重定向会**永久污染父 Shell 的标准输出**，后续所有外部命令都会写进文件。此外容易忽略：fd 表虽复制，**打开文件表与其文件偏移是共享的**，所以父子对同一文件的读写会互相推进偏移。

Q3（计算题·性能）：某程序串行部分占 8%。求 $N=8$ 时的强扩展加速比，以及 $N\to\infty$ 的上限；若改用弱扩展（固定每核工作量），阐明加速比为何可能接近线性。

答：$p=0.92$，$S(8)=\dfrac{1}{0.08+0.92/8}=\dfrac{1}{0.08+0.115}=5.13$。上限 $S(\infty)=\dfrac{1}{0.08}=12.5\times$。弱扩展固定**每核**工作量，故问题规模随 N 增长，串行部分 s 与总串行开销 s（而非占比）保持不变，总时间 $=s+p\cdot N/N_{\text{core}}$，$S(N)\approx(s+pN)/s$ 随 N 近似线性——这就是为什么"是否加核"取决于问题是固定规模还是固定每核规模。

Q4（辨析题）：为什么说"直接映射缓存不缺容量却可能表现极差"？请用题 3 的对照实验数据说明。

答：直接映射每组只有 1 路，**任何两个映射到同一组的块都会互相逐出**，这就是**冲突缺失（conflict miss）**，与容量无关。题 3 的对照实验里，8 组、32 字节块、共 256 字节的直接映射缓存，按 32 字节步长扫描 512 个 `int`（每个块只取 1 个字）时缺失率 = **100%（64 次缺失、56 次逐出）**——缓存完全没起到作用，因为每次访问都换了一个块。改成组相联（增加 E）或改变访问顺序（分块）就能消除这类缺失。**结论：缺失率是访问模式与缓存几何结构的函数，容量只是其中一个变量。**

附录 A · CSAPP 核心概念与命令速查表

> 本附录是全书 24 讲的压缩索引：按主题列成表格，供随时查阅。 表中每条都可以在正文对应章节找到展开讲解。

A.1 数据表示速查

A.1.1 C 类型 → 字节数 → 汇编后缀（x86-64）

C 类型	字节	位宽 w	汇编后缀	典型指令	说明
`char`	1	8	`b` (byte)	`movb`	有符号性由实现定义（x86-64 Linux 上为 **signed**）
`short`	2	16	`w` (word)	`movw`	—
`int`	4	32	`l` (long word)	`movl`	默认有符号
`long`	8	64	`q` (quad word)	`movq`	LP64：Linux/macOS 为 8 字节，Windows 为 4 字节
`long long`	8	64	`q`	`movq`	与 `long` 同宽
`char *`（任何指针）	8	64	`q`	`movq`	指针即 64 位地址
`float`	4	32	`ss` (scalar single)	`movss`	存于 `%xmm` 低 32 位
`double`	8	64	`sd` (scalar double)	`movsd`	存于 `%xmm` 低 64 位
`long double`	16	80(有效)	—	`fldt` 等	x87 扩展精度，ABI 规定按 16 字节对齐，**不走** `%xmm`

> 记忆钩子：`b`=1、`w`=2、`l`=4、`q`=8；浮点后缀 `ss` / `sd` = **s**calar **s**ingle / **s**calar **d**ouble。 AT&T 语法下 8 位寄存器称 `%al` 一类（`b` 后缀），16 位为 `%ax`（`w`），32 位为 `%eax`（`l`），64 位为 `%rax`（`q`）。

A.1.2 取值范围表（补码与无符号）

类型	w	最小值	最大值的十进制	最大值的十六进制
`int`	32	-2^{31} = –2,147,483,648	2,147,483,647	`0x7FFFFFFF`
`unsigned`	32	0	4,294,967,295	`0xFFFFFFFF`
`long`	64	-2^{63} = –9,223,372,036,854,775,808	9,223,372,036,854,775,807	`0x7FFFFFFFFFFFFFFF`
`unsigned long`	64	0	18,446,744,073,709,551,615	`0xFFFFFFFFFFFFFFFF`

记号	值	位模式
`UMin`	0	`000…0`
`UMax`	$2^w - 1$	`111…1`
`TMin`	-2^{w-1}	`100…0`
`TMax`	$2^{w-1} - 1$	`011…1`
`-1`	-1	`111…1`（与 `UMax` 位模式相同）

常用速记：`INT_MAX` = `0x7FFFFFFF`，`INT_MIN` = `0x80000000`，`UINT_MAX` = `0xFFFFFFFF`，`LONG_MAX` = `0x7FFFFFFFFFFFFFFF`，`ULONG_MAX` = `0xFFFFFFFFFFFFFFFF`。在 C 里更稳妥的写法是 `#include <limits.h>` 后用 `INT_MAX` / `INT_MIN` / `UINT_MAX`，或用 `stdint.h` 的 `INT32_MAX`、`INT64_MIN`、`UINT64_MAX`。

A.1.3 补码 ↔ 无符号：四个转换函数

函数	定义	公式
$B2T_w$	位模式 → 补码值	$B2T_w(X) = -x_{w-1}2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$
$B2U_w$	位模式 → 无符号值	$B2U_w(X) = \sum_{i=0}^{w-1} x_i 2^i$
$T2U_w$	补码 → 无符号（**位不变**）	$T2U_w(x) = x + x_{w-1} \cdot 2^w = \begin{cases} x & x \geq 0 \\ x + 2^w & x < 0 \end{cases}$
$U2T_w$	无符号 → 补码（**位不变**）	$U2T_w(u) = u - u_{w-1} \cdot 2^w = \begin{cases} u & u < 2^{w-1} \\ u - 2^w & u \geq 2^{w-1} \end{cases}$

- **核心恒等式**：$T2U_w(x) = x + 2^w$（当 $x < 0$）；$U2T_w(u) = u - 2^w$（当 $u \geq 2^{w-1}$）。二者都**只改解释、不改位**。
- **溢出环**：$T2U$ 把 $TMin \to 2^{w-1}$、$-1 \to UMax$；$U2T$ 把 $UMax \to -1$、$2^{w-1} \to TMin$。
- **C 语言中的触发点**：显式强制转换 `(unsigned)ty`、赋值 `ux = tx`、以及**函数调用传参**（原型为 `unsigned` 而实参为 `int`）都会做 $T2U$。
- **排序反转**：$x < 0 \Rightarrow T2U(x) > TMax$，所以 `-1 > 0U` 为真。混合有/无符号比较时，**有符号操作数被隐式转成无符号**。

A.1.4 溢出检测规则

运算	类型	溢出条件（数学判定）	条件码判定	结果性质
加法	无符号	`s = x + y`，若 $s < x$（或 $s < y$）则溢出	`CF = 1`（进位出）	真值 $= s + 2^w$
减法	无符号	`s = x - y`，若 $x < y$ 则借位	`CF = 1`（借位）	真值 $= s + 2^w$
加法	补码	x, y 同号且 s 异号	`OF = 1`	`((x^s) & (y^s)) < 0`
减法	补码	x, y 异号且 s 与 x 异号	`OF = 1`	`((x^y) & (x^s)) < 0`

- **正溢出**（positive overflow）：$x > 0, y > 0, s < 0$，真值 $= x + y - 2^w$（结果为 $TMin$ 附近）。
- **负溢出**（negative overflow）：$x < 0, y < 0, s \geq 0$，真值 $= x + y + 2^w$（结果为 $TMax$ 附近）。
- **补码构成阿贝尔群**：加法满足交换律、结合律，`0` 是单位元，每个数有加法逆元（$TMin$ 的逆元是它自己）；由此推出"平凡"恒等式 `-x == ~x + 1`、`x - y == x + (-y)` 恒成立。
- **乘法**：w 位乘积只保留低 w 位，$x \cdot y$ 的无符号值与补码值**位模式相同**（乘法的低 w 位不区分有/无符号）。溢出判定用 `imulq` 的 `OF`（判断 128 位乘积是否等于符号扩展后的低 64 位）。
- **取反**：`-x` 即 `~x + 1`；对 $TMin$ 取反得 $TMin$（溢出）。
- **C 语言注意**：**有符号溢出是未定义行为（UB）**，编译器可据此优化；无符号溢出是**有定义的模运算**。检测有符号溢出的可移植写法是先转成无符号算，再检查。

A.1.5 移位规则

表达式	类型	补位	数学等价	备注
`x << k`	任意	低位补 0	$x \cdot 2^k \bmod 2^w$	移出的高位丢弃；有符号左移溢出在 C 中是 UB
`x >> k`（无符号）	`unsigned`	高位补 0	$\lfloor u / 2^k \rfloor$	**逻辑右移（logical shift）**
`x >> k`（有符号，非负）	补码	高位补符号位 = 0	$\lfloor x / 2^k \rfloor$	与逻辑右移结果相同
`x >> k`（有符号，负数）	补码	高位补符号位 = 1	$\lfloor x / 2^k \rfloor$（向 $-\infty$ 取整）	**算术右移（arithmetic shift）**

- **负数右移在 C 中是实现定义行为（implementation-defined）**：标准未规定补 0 还是补符号位。x86-64 的 `sar` 族补符号位，GCC/Clang 一贯如此，但**不可移植**。
- **移位量非法**：C 标准规定 $k < 0$ 或 $k \geq w$ 是 UB（x86 硬件实际只取 $k \bmod w$，例如 `1 << 32` 在硬件上等价于 `1 << 0`）。
- **右移 ≠ 除法**：负数算术右移向 $-\infty$ 取整，`/` 向 0 取整。例：`-7 >> 1 == -4`，`-7 / 2 == -3`。向 0 取整的补偿写法：`(x < 0 ? x + (1<<k) - 1 : x) >> k`。
- **`>>` vs `>>>`**：C 语言只有 `>>`，没有 `>>>`；要逻辑右移就先转 `unsigned`（`(unsigned)x >> k`）。`>>>` 是 Java 的逻辑右移运算符（Java 的 `>>` 恒为算术右移），`>>>` 也存在于 JavaScript。阅读语言特性时勿混淆。
- 指令对应：`shl` / `sal` 等价（左移）、`shr` 逻辑右移、`sar` 算术右移；移位量放在 `%cl` 或立即数（`salq $2, %rax`）。

A.1.6 IEEE 754 位域与数值分类

格式	总位	符号 `s`	阶码位 k	尾数位 n	偏置 $\text{bias} = 2^{k-1} - 1$	十进制有效数字
`float`（单精度）	32	1（bit 31）	8（bits 30–23）	23（bits 22–0）	127	约 7 位
`double`（双精度）	64	1（bit 63）	11（bits 62–52）	52（bits 51–0）	1023	约 15–16 位

```
double ── 64 位                              float ── 32 位
 63  62        52 51                    0      31  30     23 22           0
+---+-----------+------------------------+     +---+--------+--------------+
| s |    exp    |          frac          |     | s |  exp   |     frac     |
| 1 |    11     |           52           |     | 1 |   8    |      23      |
+---+-----------+------------------------+     +---+--------+--------------+
```

`exp`	`frac`	类别	值 V
$\mathbf{0}$	$\mathbf{0}$	$\pm\mathbf{0}$	± 0（有两个零，`+0 == -0` 为真）
$\mathbf{0}$	$\neq \mathbf{0}$	非规格化数（denormalized）	$V = (-1)^s \times \mathbf{0.frac} \times 2^{1-\text{bias}}$
$\mathbf{1} \sim 2^k - \mathbf{2}$	任意	规格化数（normalized）	$V = (-1)^s \times M \times 2^E$，$M = \mathbf{1.frac}$，$E = \mathbf{exp} - \text{bias}$
全 1	$\mathbf{0}$	$\pm\infty$	$1.0/0.0 \to +\infty$，$-1.0/0.0 \to -\infty$
全 1	$\neq \mathbf{0}$	NaN	`0.0/0.0`、$\infty - \infty$、$\sqrt{-1}$；`NaN != NaN` 为真

- **规格化数隐含前导 1（$M = \mathbf{1.frac}$）**：这一位不存储，白赚一个有效位，代价是 `0` 无法表示——于是 `exp=0` 专用于非规格化数。
- **非规格化数**：$M = \mathbf{0.frac}$，阶码固定为 $\mathbf{1} - \text{bias}$（不是 $\mathbf{0} - \text{bias}$），使 $\mathbf{0}$ 与最小规格化数之间的间距**均匀**，称为**渐进下溢**（gradual underflow）。
- **最大规格化数**：`float` 约 3.4×10^{38}（`exp=0xFE`，`frac` 全 1）；**最小正规格化数**：$2^{-126} \approx 1.18 \times 10^{-38}$。
- **舍入**：默认**向偶数舍入**（round-to-nearest-even）；另可设向零、向下、向上。
- **浮点特殊事实**：
- `float` 有 24 位有效位（隐含 1 + 23），$\log_{10} 2^{24} \approx 7.22$，故约 **7 位**十进制有效数字；`double` 有 53 位，$\log_{10} 2^{53} \approx 15.95$，故约 **15–16 位**。
- **`0.1 + 0.2 != 0.3`**：0.1、0.2、0.3 都不能被二进制有限表示，相加后的舍入结果与 0.3 的最近可表示数不同（`0.1+0.2` 打印约 `0.30000000000000004441`）。**浮点比较必须用容差**：`fabs(a-b) < 1e-9 * fmax(1.0, fmax(fabs(a), fabs(b)))`。
- 浮点**不满足结合律**：`(x+y)+z != x+(y+z)`（大数吞小数）；只有 `-ffast-math` 才允许编译器重排。
- `float → int → float` 不是恒等（`(int)3.14f == 3`，精度与截断都丢）。
- `int → float` 对 $> 2^{24}$ 的整数会丢低位；`int → double` 对所有 32 位整数精确。

A.2 x86-64 寄存器与调用约定速查

A.2.1 16 个通用寄存器：64/32/16/8 位名称对照

64 位	32 位	16 位	8 位	角色 / 约定
`%rax`	`%eax`	`%ax`	`%al`	**返回值**；caller-saved
`%rbx`	`%ebx`	`%bx`	`%bl`	callee-saved 临时量
`%rcx`	`%ecx`	`%cx`	`%cl`	**第 4 个参数**；移位量寄存器（`%cl`）
`%rdx`	`%edx`	`%dx`	`%dl`	**第 3 个参数**（旧称数据寄存器）

64 位	32 位	16 位	8 位	角色 / 约定
`%rsi`	`%esi`	`%si`	`%sil`	**第 2 个参数**
`%rdi`	`%edi`	`%di`	`%dil`	**第 1 个参数**
`%rbp`	`%ebp`	`%bp`	`%bpl`	**callee-saved**；可选帧指针（frame pointer）
`%rsp`	`%esp`	`%sp`	`%spl`	**栈指针**（stack pointer），特殊 callee-saved
`%r8`	`%r8d`	`%r8w`	`%r8b`	**第 5 个参数**
`%r9`	`%r9d`	`%r9w`	`%r9b`	**第 6 个参数**
`%r10`	`%r10d`	`%r10w`	`%r10b`	caller-saved 临时量
`%r11`	`%r11d`	`%r11w`	`%r11b`	caller-saved 临时量
`%r12`	`%r12d`	`%r12w`	`%r12b`	**callee-saved** 临时量
`%r13`	`%r13d`	`%r13w`	`%r13b`	**callee-saved** 临时量
`%r14`	`%r14d`	`%r14w`	`%r14b`	**callee-saved** 临时量
`%r15`	`%r15d`	`%r15w`	`%r15b`	**callee-saved** 临时量

- `%r8` ~ `%r15` 的 8 位形式是 `%r8b` ~ `%r15b`（**不是** `%r8l`），且需要 REX 前缀编码。
- `%sil`、`%dil`、`%bpl`、`%spl` 也是 REX 时代新增的低字节名（历史上 `%esi` / `%edi` / `%ebp` / `%esp` 的低字节不可寻址）。
- **写 32 位寄存器会清零高 32 位**：`movl $1, %eax` 之后 `%rax == 1`；而 `movw` / `movb` 只改写低 16/8 位，高位保留。这是 x86-64 特有的优化点（`movl` 省一条 `movz`）。
- `%rip` 不是通用寄存器：**程序计数器**（instruction pointer），只能用于 RIP 相对寻址（如 `0x2fda(%rip)`）和 `leaq`，不能作为算术目标。

A.2.2 参数传递与返回值

位置	内容
`%rdi`	第 1 个整型/指针参数
`%rsi`	第 2 个整型/指针参数
`%rdx`	第 3 个整型/指针参数
`%rcx`	第 4 个整型/指针参数
`%r8`	第 5 个整型/指针参数
`%r9`	第 6 个整型/指针参数
栈	**第 7 个及以后的参数**，从右往左压栈，调用者负责清理（`call` 后 `%rsp+8` 处即第 7 个参数）
`%xmm0` ~ `%xmm7`	浮点/向量参数（**独立计数**，与整型参数各数各的）
`%rax`	整型/指针返回值
`%xmm0`	浮点返回值
`%rdx:%rax`	**128 位返回值**（高 64 位在 `%rdx`）

- 整型与浮点参数**混排时各自计数**：`void f(int a, double b, int c, double d)` → `a` → `%rdi`，`b` → `%xmm0`，`c` → `%rsi`，`d` → `%xmm1`。
- 调用者不需要为栈传参预留"影子空间"——这与 Windows x64 的 32 字节 shadow space 不同。

A.2.3 caller-saved vs callee-saved

类别	别名	寄存器	责任方
caller-saved	call-clobbered	`%rax`、`%rcx`、`%rdx`、`%rsi`、`%rdi`、`%r8`、`%r9`、`%r10`、`%r11`	**调用者**在 `call` 前把要保留的值存进自己的栈帧
callee-saved	call-preserved	`%rbx`、`%rbp`、`%r12`、`%r13`、`%r14`、`%r15`	**被调用者**若使用，必须保存并在 `ret` 前恢复
特殊	—	`%rsp`	函数退出时必须恢复为进入时的值（`ret` 依赖它）

- 规律：**所有参数寄存器 + 返回值寄存器 + `%r10/%r11` 都是 caller-saved**；`%rbx/%rbp/%r12–%r15` 是 callee-saved。
- 被调用者保存 callee-saved 寄存器的两种典型做法：`pushq %rbx … popq %rbx`，或先 `subq $N, %rsp` 再 `movq %rbx, k(%rsp)`。

- 递归能正常工作，正是因为"只在 callee-saved 寄存器与自己的栈帧里保存值"这条纪律。

A.2.4 栈帧布局与 16 字节对齐

```
        高地址
        +-----------------------------+
        |   调用者的栈帧 (caller)     |
        +-----------------------------+
        |  第 7+ 个参数 (argument build) |
        +-----------------------------+
        |   返回地址 (return address)  |  <-- call 压入；%rsp 在 callee 入口指向它
        +-----------------------------+
 %rbp ->|   保存的 %rbp (old %rbp)    |  <-- 可选：被调用者压入
        +-----------------------------+
        |   被保存的 callee-saved 寄存器 |
        +-----------------------------+
        |   局部变量 / 数组 / 临时量     |
        +-----------------------------+  <-- %rsp（函数体内随 push/sub 移动）
        低地址（栈向低地址增长）
```

要点	说明
`call` 的行为	把返回地址（下一条指令地址）**压栈**，再跳到目标；`%rsp -= 8`
`ret` 的行为	从栈顶**弹出返回地址**到 `%rip`；`%rsp += 8`
16 字节对齐	ABI 要求 `call` 执行**之前** `%rsp % 16 == 0`。因而 callee 入口处 `%rsp % 16 == 8`（返回地址占了 8 字节）。函数内常用 `subq $8, %rsp` 补齐，因为 `%xmm` 的 `movaps` 要求 16 字节对齐
局部变量位置	位于 `%rsp` **之上**（高地址方向）到 `%rbp` 之间；函数真正在用的是 `%rsp` 到 `%rbp` 这段区间
帧指针可选	`-O1` 以上通常省略 `%rbp`（`-fomit-frame-pointer`），改用 `%rsp` 相对寻址；`-O0` 与 `-fno-omit-frame-pointer` 保留 `%rbp` 便于调试
红色区域	叶子函数可用 `%rsp` 以下的 128 字节（red zone）而不必先减 `%rsp`；信号处理会破坏它，故编译器只在叶子函数使用
对齐陷阱	手写汇编忘记 `subq $8` 补齐对齐，会在 `movaps` 上触发 `SIGSEGV`——这是 Attack Lab / Bomb Lab 的常见翻车点

A.2.5 浮点/向量寄存器 `%xmm0` – `%xmm15`

寄存器	作用
`%xmm0` – `%xmm7`	**浮点参数寄存器**（第 1–8 个浮点参数）
`%xmm0`	**浮点返回值**
`%xmm8` – `%xmm15`	临时量，**全部 caller-saved**

- 每个 `%xmm` 是 **128 位**；标量 `float` 用低 32 位，`double` 用低 64 位，写入时**高位置零**。
- `%xmm` 寄存器**没有 callee-saved 的**：调用者若在 `%xmm` 里存了跨调用要用的值，必须自己溢出到栈上。
- `%ymm0` – `%ymm15`（AVX，256 位）与 `%zmm0` – `%zmm31`（AVX-512，512 位）复用同一编号的寄存器组，低 128 位即 `%xmm`。

A.3 x86-64 指令速查（AT&T 语法）

> **AT&T 语法三条铁律**：① 操作数顺序是 `源, 目的`（`movq src, dst`）；② 寄存器加 `%`（`%rax`）；③ 立即数加 `$`（`$15213`）。 内存寻址写作 `D(Rb, Ri, S)` $= \mathrm{Mem}[R_b + R_i \cdot S + D]$，$S \in \{1, 2, 4, 8\}$，`Ri` **不能是** `%rsp`。

A.3.1 数据传送

指令	语义	说明
`mov S, D`	`D = S`	`movb/movw/movl/movq` 按宽度选择
`movz S, D`	零扩展传送	`movzbw`、`movzbl`、`movzbq`、`movzwl`、`movzwq`

指令	语义	说明
`movs S, D`	符号扩展传送	`movsbl`、`movsbw`、`movswl`、`movswq`、`movslq`
`movabsq $imm64, %reg`	装入 64 位立即数	**唯一**能直接装 64 位立即数的指令；普通 `movq $imm, %reg` 只支持 32 位符号扩展立即数
`leaq D(Rb,Ri,S), D`	`D = 有效地址`	**不访问内存**，纯粹是地址计算器；编译器用它做 `x + k*y`（$k \in \{1,2,4,8\}$）的快速算术
`pushq S`	`%rsp -= 8; (%rsp) = S`	压栈
`popq D`	`D = (%rsp); %rsp += 8`	出栈

- **`movl` 的清零副作用**：`movl $0, %eax` 使整个 `%rax` 为 0；而 `movw` / `movb` 保留高位。
- **`leaq` 的经典用法**：`leaq (%rdi,%rdi,2), %rax` 得 $3x$；`leaq 4(%rdi,%rdx), %rcx` 得 $x + y + 4$；配合 `salq` 实现任意常数乘法（如 `x*12` = `leaq (%rdi,%rdi,2),%rax` + `salq $2,%rax`）。
- 传送指令**只有 `movabsq` 例外**：`movq $0x123456789, %rax` 汇编器会报错或截断，必须 `movabsq`。

A.3.2 算术与逻辑

指令	语义	备注
`add S, D`	`D += S`	设置条件码
`sub S, D`	`D -= S`	设置条件码
`imul S, D`	`D *= S`（有符号）	两操作数形式；也支持 `imulq $c, S, D` 三操作数
`imulq S`	`%rdx:%rax = %rax * S`	一操作数形式，产生 128 位乘积
`mulq S`	同上，**无符号**	与 `imulq` 单操作数形式配对 `divq`
`idivq S` / `divq S`	`%rax = 商, %rdx = 余`	被除数在 `%rdx:%rax`；有符号除法前须 `cqto`
`shl k, D` / `sal k, D`	`D <<= k`	**两者等价**（shift left / shift arithmetic left）
`shr k, D`	`D >>= k`（**逻辑**，补 0）	用于无符号
`sar k, D`	`D >>= k`（**算术**，补符号位）	用于有符号
`xor S, D`	`D ^= S`	`xorq %rax,%rax` 是清零惯用法（比 `movq $0` 短）
`and S, D`	`D &= S`	常用于掩码
`or S, D`	`D |= S`	—
`inc D`	`D += 1`	**不设 `CF`**，只设 `OF/SF/ZF`
`dec D`	`D -= 1`	同上
`neg D`	`D = -D`	即 `~D + 1`；置 `CF`
`not D`	`D = ~D`	**不影响任何条件码**

- 移位量：`k` 为立即数，或放在 **`%cl`** 中（`salq %cl, %rax`）。
- 乘法常被编译器强度削减为 `leaq` + `sal`（如 `x*48` → `leaq (%rsi,%rsi,2),%rdx` + `salq $4,%rdx`）——**看汇编时不要以为一定有 `imul`**。

A.3.3 比较与测试

指令	语义	说明
`cmp S1, S2`	计算 `S2 - S1`，**只设条件码，不写回**	ATT 顺序是"减数在前"：`cmpq %rsi, %rdi` 算 `%rdi - %rsi`
`test S1, S2`	计算 `S1 & S2`，**只设条件码，不写回**	`testq %rax, %rax` 用于判断 `%rax` 是否为 0/负

A.3.4 条件码表

标志	含义	由谁设置
`CF`	**进位/借位**（Carry）	`add` / `sub` 的无符号溢出；移位指令（最后移出的位）；`neg`。**`inc` / `dec` 不设 `CF`**
`ZF`	**零标志**（Zero）	结果为 0 时置 1；`add` / `sub` / `and` / `or` / `xor` / `cmp` / `test` / `inc` / `dec` 均设置
`SF`	**符号标志**（Sign）	结果最高位为 1（即"负"）时置 1

标志	含义	由谁设置
`OF`	**溢出标志**（Overflow）	补码加/减溢出；`imul` 溢出。**`inc` / `dec` 会设 `OF`**

- **`leaq` 不设置任何条件码**——这是它在编译器中被当作"纯算术"使用的关键原因。
- 四种组合判定有符号比较：`cmp` 后看 `SF` 与 `OF` 是否相等（`SF == OF` 表示"确实不小于"）。

A.3.5 `set` 指令：按条件写单字节

指令	同义	条件码组合	语义
`sete`	`setz`	`ZF`	相等 / 为零
`setne`	`setnz`	`~ZF`	不等 / 非零
`sets`	—	`SF`	负数
`setns`	—	`~SF`	非负
`setg`	`setnle`	`~ZF & (SF == OF)`	**有符号 >**
`setge`	`setnl`	`SF == OF`	有符号 ≥
`setl`	`setnge`	`SF != OF`	有符号 <
`setle`	`setng`	`ZF | (SF != OF)`	有符号 ≤
`seta`	`setnbe`	`~CF & ~ZF`	**无符号 >**
`setae`	`setnb`	`~CF`	无符号 ≥
`setb`	`setnae`	`CF`	无符号 <
`setbe`	`setna`	`CF | ZF`	无符号 ≤

- `set` 类指令的**目的操作数必须是单字节位置**（`%al`、`%r10b` 等，或内存），写成 `setg %rax` 会报错。
- 常与 `movzbl` 配合得到 32 位 0/1 结果：`cmpq %rsi,%rdi; setg %al; movzbl %al,%eax`。

A.3.6 `j` 指令：按条件跳转

指令	同义	条件码组合	用途
`jmp`	—	无条件	直接/间接跳转（`jmp *%rax`、`jmp *(%rax)`）
`je`	`jz`	`ZF`	相等
`jne`	`jnz`	`~ZF`	不等
`js`	—	`SF`	负数
`jns`	—	`~SF`	非负
`jg`	`jnle`	`~ZF & (SF == OF)`	**有符号 >**
`jge`	`jnl`	`SF == OF`	有符号 ≥
`jl`	`jnge`	`SF != OF`	有符号 <
`jle`	`jng`	`ZF | (SF != OF)`	有符号 ≤
`ja`	`jnbe`	`~CF & ~ZF`	**无符号 >**
`jae`	`jnb`	`~CF`	无符号 ≥
`jb`	`jnae`	`CF`	无符号 <
`jbe`	`jna`	`CF | ZF`	无符号 ≤

- **选择口诀**：`g` / `l`（greater/less）**用于有符号**；`a` / `b`（above/below）**用于无符号**。写错类别是汇编题最常见错误——`int` 用 `jg`，`unsigned` 用 `ja`。
- C 的 `<`、`>` 编译成 `cmp` + `jl` / `jg`；`unsigned` 的比较编译成 `cmp` + `jb` / `ja`。

A.3.7 `cmov` 指令：条件传送（无分支）

指令	同义	条件码组合	语义
`cmove`	`cmovz`	`ZF`	条件成立时 `dst = src`
`cmovne`	`cmovnz`	`~ZF`	—
`cmovs`	—	`SF`	—
`cmovns`	—	`~SF`	—
`cmovg`	`cmovnle`	`~ZF & (SF == OF)`	有符号 $>$
`cmovge`	`cmovnl`	`SF == OF`	有符号 $\geq$
`cmovl`	`cmovnge`	`SF != OF`	有符号 $<$
`cmovle`	`cmovng`	`ZF | (SF != OF)`	有符号 $\leq$
`cmova`	`cmovnbe`	`~CF & ~ZF`	无符号 $>$
`cmovae`	`cmovnb`	`~CF`	无符号 $\geq$
`cmovb`	`cmovnae`	`CF`	无符号 $<$
`cmovbe`	`cmovna`	`CF | ZF`	无符号 $\leq$

- **源操作数不能是立即数**：必须先 `movq $c, %reg` 再 `cmovXX %reg, %dst`。
- **内存源无论如何都会被读取**：即使条件不成立，CPU 也会发起该访存（可能触发缺页/段错误）。因此编译器只在"两个分支都安全可算"时才用 `cmov`——这也是 CS:APP 讲的"分支不可预测时 `cmov` 更快，但语义不等价"的原因。
- 浮点比较用 `ucomisd` / `comisd`，它设置的是 **`ZF` / `PF` / `CF`**（不设 `SF` / `OF`），也无序（NaN）时置 `ZF=PF=CF=1`；因此浮点的条件传送/跳转要用 **无符号** 形式（`cmova` / `cmovae` / `cmovb` / `cmovbe`），不能用 `cmovg` / `cmovl`。

A.3.8 过程调用

指令	语义	备注
`call Label`	压返回地址（`%rsp -= 8`），跳转	直接调用
`call *Operand`	间接调用（函数指针、虚表、PLT）	`call *%rax`、`call *(%rdx)`
`ret`	从栈顶弹出返回地址到 `%rip`	与 `call` 配对
`leave`	`movq %rbp,%rsp; popq %rbp`	撤销帧指针的惯用两条合体

A.3.9 浮点/SSE 指令

类别	指令	语义
传送	`movss` / `movsd`	单/双精度**标量**传送（只动低 32/64 位）
传送	`movaps` / `movapd`	**对齐**（16 B）打包传送，未对齐会 `SIGSEGV`
传送	`movups` / `movupd`	**非对齐**打包传送，无对齐要求
算术	`addss` / `addsd`、`subss` / `subsd`、`mulss` / `mulsd`、`divss` / `divsd`	标量加/减/乘/除（`ss`=float，`sd`=double）
算术	`addps` / `addpd` 等	**打包**（4×float 或 2×double）算术
比较	`ucomiss` / `ucomisd`	比较并设置 `ZF/PF/CF`（供 `seta` / `cmova` 用）
转换	`cvtss2sd`	`float → double`
转换	`cvtsd2ss`	`double → float`
转换	`cvtsi2sd` / `cvtsi2ss`	整数 → `double` / `float`
转换	`cvttsd2si` / `cvttss2si`	`double` / `float` → 整数，**截断**（truncate，向 0）
转换	`cvtsd2si`	按当前舍入模式转整数（默认向偶数）

- 浮点参数 `%xmm0` — `%xmm7`，返回值 `%xmm0`；`%xmm` 全为 caller-saved。
- `%xmm` 与 `%rdi` 等整型寄存器的计数**互相独立**（见 A.2.2）。

A.3.10 易混淆指令对照表

指令	方向	语义	陷阱
`movzbl`	byte → long	**零扩展**：`%dil` → `%edi` / `%rax`	高位补 **0**，用于 `unsigned char`
`movsbl`	byte → long	**符号扩展**：`%dil` → `%edi` / `%rax`	高位补 **符号位**，用于 `char` / `signed char`
`movslq`	long(32) → quad(64)	**符号扩展**：`movslq %eax, %rax`	名字里的 `s` 是 sign，`l` 是**源**宽 4 字节，`q` 是**目的** 8 字节——**后缀先后是"源→目的"**
`movswl`	word → long	符号扩展 2 → 4 字节	同上规则
`movzwl`	word → long	零扩展 2 → 4 字节	`unsigned short` → `int`
`cltq`	`%eax` → `%rax`	等价于 `movslq %eax,%rax`	**无操作数**；"convert long to quad"
`cqto`	`%rax` → `%rdx:%rax`	把 `%rax` 的符号位扩展到 `%rdx`，为 `idivq` 准备 128 位被除数	**无操作数**；无符号版是 `xorq %rdx,%rdx`
`shr`	右移	**逻辑**右移，高位补 0	用于 `unsigned`
`sar`	右移	**算术**右移，高位补符号位	用于有符号；负数除法场景
`shl` vs `sal`	左移	**完全等价**	没有"算术左移"与"逻辑左移"之分
`movl` vs `movq`	传送	`movl` 写 32 位会**清零高 32 位**；`movq` 不会涉及该规则	`movl $-1, %eax` 得 `%rax == 0x00000000FFFFFFFF`
`movq $imm` vs `movabsq`	立即数	`movq` 只支持 32 位符号扩展立即数；`movabsq` 支持 64 位	装 `0x123456789ABCDEF0` 必须用 `movabsq`
`leaq` vs `movq`	地址	`leaq 8(%rsp), %rax` 得到**地址**；`movq 8(%rsp), %rax` 得到**内容**	`leaq` **不访存**，可安全用于无效地址表达式
`imul` vs `mul`	乘法	`imul` 有符号，`mul` 无符号	低 64 位结果**位模式相同**，只用 `imul` 也无妨
`inc` / `dec`	自增/减	**不修改** `CF`	循环计数若依赖 `CF` 会出错；`add $1` 才改 `CF`

A.4 GDB 速查

> 启动前用 `gcc -g -O0 -Wall` 编译，调试信息才完整。`-Og` 兼顾优化与可调试性。

A.4.1 启动

命令	作用
`gdb ./prog`	加载可执行文件的符号
`gdb --args ./prog a b`	连命令行参数一起加载（等价于 `run a b`）
`gdb ./prog core`	用 core dump 事后调试（先 `ulimit -c unlimited`）
`gdb -p PID`	附加到正在运行的进程（attach）
`gdb -tui` / `gdb -tui ./prog`	启动 TUI 分屏界面
`gdb ./prog -ex 'b main' -ex run`	非交互式批处理（脚本化）

A.4.2 断点

命令	作用
`break func` / `b func`	在函数入口下断点
`b file.c:42` / `b file:line`	在指定源文件行下断点
`b *0x4005d6`	在**指令地址**下断点（无符号信息时的救命招）
`b *main+16`	在函数相对偏移处下断点

命令	作用
`tbreak ...`	**临时断点**，命中一次后自动删除
`rbreak regex`	对所有匹配正则的函数下断点
`delete [N]` / `d`	删除第 N 个（或全部）断点
`disable N` / `enable N`	禁用 / 启用断点（保留不删）
`info breakpoints` / `info b`	列出所有断点及其命中次数
`watch var`	监视变量：**被写入**时停下（硬件/软件 watchpoint）
`rwatch var` / `awatch var`	被读 / 被读或写时停下
`condition N expr`	给断点 N 加条件，如 `condition 1 i==5`
`ignore N 100`	忽略前 100 次命中
`catch syscall write` / `catch fork`	捕获系统调用/信号等事件

A.4.3 执行

命令	缩写	作用
`run [args]`	`r`	从头运行程序
`start`	—	在 `main` 处停下（相当于 `b main; run`）
`step`	`s`	**步入**：进入被调函数（源码级）
`next`	`n`	**步过**：不进入被调函数（源码级）
`stepi [n]`	`si`	**单条指令**步入（汇编级）
`nexti [n]`	`ni`	单条指令步过（汇编级）
`finish`	`fin`	运行到当前函数返回，并打印返回值
`continue`	`c`	继续到下一个断点
`until [line]`	`u`	运行到当前行之后/指定行（跳出循环常用）
`advance func`	—	运行到函数（不重复进入当前函数）
`kill`	`k`	终止被调试程序
`quit`	`q`	退出 GDB

- **卡在 `input` 里怎么办**：`finish` 可能因程序等待输入而挂住；先 `kill` 再重启。
- Bomb Lab 惯用组合：`b explode_bomb` → `run` → `bt` 找到调用点 → `frame N` → `info locals`。

A.4.4 查看数据

命令	作用
`print expr` / `p expr`	按自然类型打印表达式的值
`p/x expr`	以**十六进制**打印（也支持 `/d` 十进制、`/u` 无符号、`/t` 二进制、`/o` 八进制、`/c` 字符、`/f` 浮点）
`p/d expr`	以**有符号十进制**打印
`p/u expr`	以**无符号十进制**打印
`p/s ptr`	以**字符串**打印（把 `char *` 按 C 字符串解释）
`p *ptr`	**解引用**，打印指针指向的对象
`p **ptr` / `p ptr->field`	多级解引用 / 打印结构体成员
`p arr[3]@5`	从 `arr[3]` 开始打印 **5 个**元素（`@` 是"连续 n 个"运算符）
`p (char*)ptr` / `p *((int*)p+2)`	强制类型转换后再打印
`p $rax`、`p $rip`、`p $eflags`	打印**寄存器**（寄存器名前加 `$`）

命令	作用
`p/x $rax - $rbx`	寄存器参与算术
`display expr`	每次停下**自动打印**该表达式（`undisplay N` 取消，`info display` 查看）
`set var x = 1`	修改变量值（如 `set var i=0` 跳过循环）
`set $rax = 0`	修改寄存器值（Bomb/Attack Lab 常用）
`set {int}0x7fffffffe000 = 5`	直接改内存
`whatis expr` / `ptype type`	查看表达式的类型 / 类型定义

A.4.5 `x/nfu addr` 格式详解

语法 `x/[n][f][u] addr`：`n` 重复次数（十进制，默认 1）、`f` 显示格式、`u` 单元大小。

字段	取值	含义
`n`	十进制数字，如 `8`、`16`	打印多少个单元（默认 1）
`f`	`x`	十六进制
`f`	`d`	有符号十进制
`f`	`u`	无符号十进制
`f`	`o`	八进制
`f`	`t`	二进制（two's complement 显示）
`f`	`a`	地址（同时给出最近的符号名偏移）
`f`	`c`	字符
`f`	`s`	**以 0 结尾的字符串**
`f`	`i`	**反汇编指令**
`f`	`f`	浮点数
`u`	`b`	**byte**，1 字节
`u`	`h`	**halfword**，2 字节
`u`	`w`	**word**，4 字节
`u`	`g`	**giant**，8 字节

示例	含义
`x/8xb &arr`	从 `arr` 起打印 **8 个字节**的十六进制
`x/8dw arr`	打印 8 个 4 字节有符号整数
`x/s 0x402008`	把该地址当作 C 字符串打印（Bomb Lab 必备）
`x/4i $rip`	反汇编从当前指令开始的 4 条指令
`x/16gx $rsp`	以 8 字节为单位打印栈顶 16 个单元（即 128 字节的栈）
`x/2gx $rdx`	打印 `%rdx` 指向的 2 个 8 字节（如 128 位返回值）
`x/20wx $rsp-40`	查看当前栈帧上下各 20 个 4 字节字

- 省略 `f` / `u` 时沿用**上一次**的设置（所以 `x/8xb` 之后再 `x/4` 仍是字节）；默认单元是 `w`、默认格式是 `x`。
- 常用组合速记：**`nfu` = "多少 / 怎么显示 / 多大一块"**。

A.4.6 栈与帧

命令	作用
`backtrace` / `bt`	打印调用栈：当前帧 + 各层调用者
`bt full`	附带每帧的局部变量

命令	作用
`bt 5`	只打印最内 5 层
`frame N` / `f N`	切换到第 N 号帧（0 = 最内层）
`up [N]` / `down [N]`	上移/下移帧（沿调用链往外/往内）
`info frame`	当前帧的详细信息（帧地址、返回地址、保存的寄存器位置）
`info registers` / `info reg`	打印所有通用寄存器的值
`info registers rip`	只打印 `%rip`
`info all-registers`	含 `%xmm`、`%st` 等全部寄存器
`info locals`	当前函数的**局部变量**
`info args`	当前函数的**参数**
`info variables` / `info functions`	全局变量 / 全部函数（支持正则过滤）
`info symbol 0x4005d6`	某个地址落在哪个符号的哪个偏移
`x/8gx $rbp`	直接按内存查看栈帧内容

- **读栈帧的经典套路**：`bt` 找到帧号 → `f N` → `info frame` 看到 saved rip 的位置 → `x/8gx $rbp` 核对局部变量与返回地址。
- 编译加 `-fno-omit-frame-pointer`（或 `-O0`）时 `%rbp` 链才完整，`bt` 才可靠。

A.4.7 反汇编与源码

命令	作用
`disassemble` / `disas`	反汇编**当前函数**
`disas func`	反汇编指定函数
`disas 0x400540, 0x400560`	反汇编地址区间
`disas /r`	附上**机器码字节**（raw bytes）
`disas /m`	与源码行交错显示
`set disassembly-flavor intel`	切换到 **Intel 语法**（无 `%`、操作数顺序相反）
`set disassembly-flavor att`	切回 AT&T 语法（默认）
`list` / `l`	显示源码（`l func`、`l 42`、`l file.c:42`）
`info line *0x4005d6`	该地址对应的源码行
`directory DIR`	添加源码搜索路径

A.4.8 TUI 与布局

命令	作用
`layout regs`	分屏：源码/汇编 + **寄存器**窗口（寄存器的改动会高亮）
`layout asm`	只显示汇编窗口
`layout src`	只显示源码窗口
`layout split`	源码 + 汇编同时显示
`tui reg general` / `tui reg float` / `tui reg all`	切换寄存器窗口显示哪一组
`focus cmd` / `focus src` / `focus asm`	切换键盘焦点到某个窗口（上下箭头滚动）
`Ctrl-x a` / `Ctrl-x 2`	进入/退出 TUI / 切换布局
`Ctrl-l`	刷新屏幕（TUI 花屏时）
`update`	刷新 TUI 窗口

- **Bomb Lab 推荐姿势**：`gdb -tui ./bomb` → `layout regs` → `b explode_bomb` / 单步 `si`，一边看 `%rax`、`%rdi` 一边看反汇编。

- TUI 花屏或按键失灵时退出重进通常最快；`Ctrl-x a` 可临时回到普通模式。

A.4.9 信号与线程

命令	作用
`handle SIGINT stop print`	收到 `SIGINT` 时停下来并打印（默认 `stop print`）
`handle SIGINT nostop noprint pass`	让 GDB **不拦** `SIGINT`，直接传给程序
`handle SIGSEGV stop print pass`	段错误时停下（默认行为），便于定位越界
`handle SIGALRM nostop noprint pass`	常见于带定时器的程序
`info signals`	列出所有信号的当前处理策略
`info threads`	列出所有线程及其当前帧（`*` 表示当前线程）
`thread N`	切换到第 N 号线程
`thread apply all bt`	对**所有线程**打印调用栈（排查死锁/挂起第一招）
`thread apply all bt full`	加上局部变量
`set scheduler-locking on`	单步时锁定调度，只让当前线程跑（调试并发时很有用）
`set follow-fork-mode child`	`fork` 后跟踪子进程

- 多线程下断点会在所有线程命中；配合 `scheduler-locking` 可让调试结果可复现。
- 死锁排查顺序：`thread apply all bt` → 找到卡在 `pthread_mutex_lock` / `sem_wait` / `read` 的线程 → `thread N` + `bt full` 看谁持锁。

A.5 二进制工具速查

A.5.1 `gcc` 常用开关

开关	作用
`-O0`	不优化（默认）。**Bomb/Attack Lab 调试首选**，指令与源码一一对应
`-O1`	基础优化，兼顾可调试（`-Og` 是专为调试设计的优化级别）
`-O2`	标准优化（多数发行版的默认发布级别）
`-O3`	激进优化：更狠的内联、向量化、循环变换，代码体积与调试难度上升
`-g`	生成**调试信息**（DWARF），GDB 才能按源码行/变量名工作；`-g3` 额外含宏信息
`-Wall`	打开常用警告
`-Wextra`	打开更多警告（含未使用参数、符号比较等）
`-Werror`	把警告当错误（CI 常用）
`-std=c11` / `-std=c17` / `-std=gnu11`	选择 C 标准（`gnu*` 允许 GNU 扩展）
`-S`	只到**汇编**（产出 `.s`，不汇编不链接）
`-c`	只到**目标文件**（产出 `.o`）
`-E`	只做**预处理**（把 `#include` / `#define` 展开）
`-o FILE`	指定输出文件名
`-lNAME`	链接库，如 `-lm`（数学库）、`-lpthread`；**顺序重要**，库要放在引用它的 `.o` 之后
`-L DIR`	增加库搜索路径
`-I DIR`	增加头文件搜索路径
`-shared`	生成共享库（`.so`）
`-fPIC`	生成**位置无关代码**（共享库必须用）
`-fno-pic` / `-no-pie`	生成非 PIC、非 PIE 的可执行文件（Attack Lab 需要固定地址时常用）

开关	作用
`-static`	静态链接，把库代码全部塞进可执行文件（体积大、无 `.so` 依赖）
`-pthread`	启用 POSIX 线程：定义宏并链接线程库
`-mavx2` / `-mavx512f` / `-msse4.2`	允许生成对应 ISA 的指令（在无该指令的机器上运行会 `SIGILL`）
`-fsanitize=address,undefined`	插入 ASan/UBSan 运行时检查（见 A.6）
`-fno-omit-frame-pointer`	强制保留 `%rbp` 帧指针，`bt` / `perf` 更可靠
`-save-temps`	保留 `.i` / `.s` / `.o` 等中间产物，便于逐阶段检查
`-v`	打印实际调用的编译/汇编/链接命令与搜索路径（查"为什么找不到头文件"神器）
`-D NAME[=VALUE]`	命令行定义宏，等价于源码里 `#define NAME VALUE`
`-M` / `-MM`	输出依赖关系（给 Makefile 用）
`-Wl,OPT`	把 `OPT` 透传给链接器，如 `-Wl,-z,now`（立即绑定）、`-Wl,-rpath,/path`

- 典型组合：调试 `gcc -g -O0 -Wall -Wextra -std=c11 prog.c -o prog`；优化对比 `gcc -O0/-O1/-O2/-O3 -S`。
- `-O2` 及以上会让 `%rbp` 消失、变量被提升进寄存器、循环被重排——**看汇编时先确认编译级别**。

A.5.2 `objdump`：反汇编目标文件

命令	作用
`objdump -d prog`	反汇编所有含代码的节（AT&T 语法，默认）
`objdump -d -M intel prog`	反汇编并输出 **Intel 语法**（无 `%`、操作数顺序反转）
`objdump -t prog.o`	打印**符号表**（含节、地址、大小、local/global）
`objdump -s prog`	打印各节的**完整十六进制内容**
`objdump -r prog.o`	打印**重定位表**（链接前才能看到未解析的引用）
`objdump -j .text -d prog`	只反汇编指定节
`objdump -d --start-address=0x400540 --stop-address=0x400560 prog`	反汇编地址区间
`objdump -h prog`	节头摘要（大小、VMA、文件偏移）
`objdump -x prog`	打印全部头信息（节 + 符号 + 动态段 + 重定位）
`objdump -R prog`	打印动态重定位表（对应 GOT 槽）
`objdump -S prog`	反汇编与源码交错（需 `-g`）

- **`-d` 反汇编 `.text` 中所有函数，但依赖节头里的符号信息**；被剥离符号的二进制要靠地址区间或 `nm`。
- `-M intel` 输出的地址格式与 GDB `set disassembly-flavor intel` 一致，两者可交叉核对。
- 链接**前**的 `.o` 里，未解析引用的目标字段是 0，必须看 `-r` 的重定位条目才能知道它要填什么。

A.5.3 `readelf`：ELF 结构

命令	作用
`readelf -a prog`	打印**全部** ELF 信息（头 + 节 + 段 + 符号 + 重定位 + 动态段），最全但冗余
`readelf -h prog`	**ELF 头**：类型（REL/EXEC/DYN）、入口点、机器、节头表位置
`readelf -S prog`	**节头表**（section headers）：`.text` / `.data` / `.bss` / `.symtab` / `.rela.text` 等的大小与地址
`readelf -l prog`	**段头表**（program headers / segments）：`LOAD` 段的权限（R E）、对齐、`INTERP`、`GNU_STACK`
`readelf -s prog`	**符号表**（`.symtab` 与 `.dynsym`）：名字、值、大小、绑定（LOCAL/GLOBAL/WEAK）、类型（FUNC/OBJECT）
`readelf -r prog`	**重定位表**：偏移、类型（`R_X86_64_PC32` 等）、符号
`readelf -d prog`	**动态段**：`NEEDED`（依赖的 `.so`）、`RPATH`、`INIT` / `FINI`
`readelf -x .rodata prog`	十六进制 dump 指定节（`-x` / `--hex-dump`）

命令	作用
`readelf -p .comment prog`	以字符串形式打印节的 ASCII 内容
`readelf --dyn-syms prog`	只打印动态符号表

- 记忆钩子：**小写 `-h/-S/-l/-s/-r/-d` 分别对应 header / Sections / segments(program headers) / symbols / relocations / dynamic**。
- 对比：`readelf` 面向 ELF 结构，`objdump` 面向机器码与反汇编；两者都要会。

A.5.4 其它二进制工具

工具	命令示例	作用
`nm`	`nm prog`	列出符号及地址（`T` 代码段、`D` 已初始化数据、`B` 未初始化数据 `.bss`、`U` **未定义/外部引用**、`w` 弱符号、小写=local）
`nm`	`nm -a prog`	显示**全部**符号（含调试/局部符号）
`nm`	`nm -u prog`	只列**未定义**符号（看程序依赖哪些外部函数，如 `printf`、`malloc`）
`nm`	`nm --defined-only prog`	只列本文件定义的符号
`size`	`size prog`	打印 `text` / `data` / `bss` 三节字节数（`dec` 为总和）
`strings`	`strings -a prog | head`	抽取可打印字符串（找格式串、错误信息、隐藏提示；`-n 4` 改最短长度）
`ldd`	`ldd ./prog`	列出**运行时**依赖的共享库及其解析到的路径（"not found" 即缺失）
`ar`	`ar rs lib.a a.o b.o`	创建/更新**静态库**归档（`r` 插入、`s` 写索引）；`ar t lib.a` 列出成员
`ld`	`ld -o prog a.o b.o -lc`	直接调用链接器（`gcc` 会代你调用它）
`ldconfig`	`sudo ldconfig -p | grep libm`	重建/查询共享库缓存 `/etc/ld.so.cache`
`file`	`file prog prog.o lib.a`	识别文件类型（`ELF 64-bit LSB executable, x86-64, dynamically linked, ...`）
`xxd`	`xxd prog | head`	十六进制 + ASCII 双栏 dump（`xxd -l 64 -s 0x1000 prog` 限定长度与偏移）
`hexdump`	`hexdump -C prog`	十六进制 dump（`-C` 为规范格式，`-n 32` 限定字节数）
`od`	`od -A x -t x1z prog`	八进制/十六进制 dump（`-t x1` 一字节十六进制、`-t d4` 四字节十进制、`-A x` 偏移用十六进制显示）
`strip`	`strip prog`	删除符号表（体积变小，但反汇编更难读）
`gdb`	`gdb -batch -ex 'disas main' ./prog`	批处理式反汇编（无需交互）

A.5.5 链接关键机制（一句话版）

机制	一句话说明
符号解析（symbol resolution）	把每个**引用**关联到符号表里的一个**定义**；找不到定义 → `undefined reference`，定义重名（两个强符号）→ `multiple definition`
重定位（relocation）	合并各 `.o` 的节、确定运行时地址后，用重定位条目**改写指令/数据中的地址字段**
`R_X86_64_PC32`	**PC 相对**寻址：填入"符号地址 − 重定位处地址"的 32 位差值，用于 `call` / `jmp` / `lea` 到全局变量；x86-64 默认偏移量要减 4（故反汇编常见 `sum-0x4`）
`R_X86_64_32`	**绝对寻址**：把符号的 32 位绝对地址写进字段（受限于低 2 GB 地址空间）
`R_X86_64_PLT32`	调用外部函数时的 PC 相对条目，经 PLT 中转（PIC 世界里的 `call`）
`R_X86_64_JUMP_SLOT`	GOT 中的**延迟绑定槽**：加载后由动态链接器填入真正的函数地址
`R_X86_64_GLOB_DAT`	GOT 中全局数据的槽
强符号 vs 弱符号	函数与已初始化的全局变量是**强符号**；未初始化全局变量、`extern` 声明、被 `__attribute__((weak))` 修饰的是**弱符号**
强弱符号三规则	① 不允许两个强符号重名；② 强 + 多个弱 → 选强；③ 多个弱 → 任选其一（**危险**，是链接类 bug 的温床）
静态库	`.a` 是 `.o` 的归档；链接器只把**被引用到**的成员拉进可执行文件，因此**顺序敏感**（引用者在前，被引用者在后）
共享库	`.so` 在加载/运行时映射；多进程共享同一份物理代码页，靠 VM 实现
PIC / GOT	位置无关代码不写死绝对地址：访问全局变量经 **GOT**（Global Offset Table）间接寻址，访问外部函数经 **PLT**

机制	一句话说明
PLT 与延迟绑定	首次调用 `printf`：PLT 表项 → GOT 表项（初始指向 PLT 表项内部的下一条指令）→ push 重定位索引 → `PLT[0]` → `_dl_runtime_resolve` 解析真实地址并**回填 GOT**；此后每次调用只多一次间接跳转
立即绑定	`-Wl,-z,now` 或环境变量 `LD_BIND_NOW=1` 让加载时就解析全部符号（更安全，GNU_RELRO 可把 GOT 设为只读）
`LD_PRELOAD`	指定**优先加载**的共享库，其符号覆盖后续库的同名符号——用于打桩拦截 `malloc` / `open` 等
`LD_LIBRARY_PATH`	运行时**优先**搜索的共享库目录列表（先于系统默认路径）
`LD_DEBUG=libs,bindings`	打印动态链接器的搜索与绑定过程（排查"加载了哪个 `.so`"）

A.6 调试与动态分析工具速查

A.6.1 `valgrind` 工具族

工具	命令	检测什么
memcheck（默认）	`valgrind ./prog`	非法读写、未初始化值使用、内存泄漏、重复 `free`、`free` 非堆指针、越界
callgrind	`valgrind --tool=callgrind ./prog`	函数级**调用次数与指令计数**（配合 `callgrind_annotate` / KCachegrind 看热点）
cachegrind	`valgrind --tool=cachegrind ./prog`	模拟 L1/L2 缓存的命中与未命中、分支预测失败
helgrind	`valgrind --tool=helgrind ./prog`	**线程错误**：数据竞争、锁顺序不一致、锁使用错误
drd	`valgrind --tool=drd ./prog`	数据竞争检测的另一实现（侧重 happens-before 分析与竞争报告）
massif	`valgrind --tool=massif ./prog`	**堆内存使用随时间的变化**（峰值为谁分配），`ms_print` 可视化
memcheck 变体	`--tool=exp-sgcheck`	栈/全局数组越界（已废弃，现代用 ASan）

memcheck 关键选项

选项	作用
`--leak-check=full`	完整泄漏报告：给出每个泄漏块的**分配调用栈**
`--show-leak-kinds=all`	显示全部泄漏类别（默认只报 definite/possible）
`--track-origins=yes`	追踪**未初始化值**的来源（更慢，但能直接指出"哪个变量没初始化"）
`--error-exitcode=1`	发现错误时以退出码 1 结束——**CI 里必加**
`-s` / `--stack-traces=yes`	打印调用栈
`--num-callers=30`	调用栈深度（默认 12，太浅看不到根因）
`--track-fds=yes`	报告**未关闭的文件描述符**
`--vgdb=yes`	允许用 GDB 通过 vgdb 控制 valgrind
`--log-file=vg.log`	输出写入文件（结果太长时）
`--gen-suppressions=all`	为每个错误生成抑制条目（`--suppressions=file` 复用）

memcheck 报告分类速查

报告	含义	严重度
`Invalid read of size N` / `Invalid write of size N`	读写**未分配/已释放/越界**的地址	高，必查
`Use of uninitialised value`	使用了未初始化内存（分支/输出/系统调用都算）	高
`Conditional jump or move depends on uninitialised value(s)`	用未初始化值做了分支判断	高，常是漏初始化
`definitely lost`	**确定泄漏**：没有指针指向该块，无法释放	高
`indirectly lost`	泄漏块**内部的指针**所指的块也泄漏（根因是外层块丢失）	中（修好外层即可）

报告	含义	严重度
`possibly lost`	仍有指针指向块**内部**（如指向结构体中间），可能泄漏	中，需人工判断
`still reachable`	程序结束时**仍有指针指向**、未释放但可达	低（常见于未 `free` 的全局缓存/库内部结构）
`suppressed`	被抑制规则屏蔽的错误	忽略
`ERROR SUMMARY: N errors from M contexts`	汇总：N 个错误、M 个不同调用点	—
`HEAP SUMMARY: in use at exit`	退出时仍占用的字节数与块数	—

- **Valgrind 不是万能的**：它跟踪的是真实执行的路径；`-O3` 下越界可能被优化掉从而"查不出错"——**务必用 `-O0 -g` 编译后再跑 valgrind**。
- Valgrind 通过动态二进制插桩运行，程序会**慢 10–50 倍**，不适合作性能测量（用 `perf`）。

A.6.2 Sanitizer 编译开关

开关	检测内容	典型用途
`-fsanitize=address`（ASan）	**越界/释放后使用（UAF）/重复释放/栈溢出/全局越界**，含分配与释放调用栈	替代 valgrind，快得多（约 2× 减速）
`-fsanitize=undefined`（UBSan）	**未定义行为**：有符号溢出、移位越界、空指针解引用、对齐错误、越界数组索引	抓"看起来能跑其实是 UB"的代码
`-fsanitize=thread`（TSan）	**数据竞争**（与 ASan **互斥**，不能同时开）	SFS / Proxy Lab 的并发 bug
`-fsanitize=leak`（LSan）	**内存泄漏**（只查泄漏，代价低）	大程序只关心泄漏时
`-fsanitize=memory`（MSan）	**未初始化读**（需所有代码含库都用 MSan 编译）	与 valgrind memcheck 类似
`-fsanitize=address,undefined`	组合使用（最常用）	日常开发
`-fsanitize-recover=all`	UBSan 报错后**继续**执行（默认对部分检查是继续）	一次跑完收集全部问题
`-fno-sanitize-recover=all`	任何 sanitizer 报错立即终止	CI 严格模式
`-fsanitize=undefined -fno-sanitize=alignment`	关闭对齐检查	处理故意非对齐访问的代码
`-g -O1`	**Sanitizer 推荐编译级别**（`-O1` 保证行号与内联都可用）	—

环境变量	作用
`ASAN_OPTIONS=detect_leaks=1`	打开/关闭泄漏检测（LSan）
`ASAN_OPTIONS=halt_on_error=0`	出错后继续运行
`ASAN_OPTIONS=abort_on_error=1`	出错时 `abort()`（便于拿 core）
`ASAN_OPTIONS=symbolize=1`	符号化栈帧（需要 `llvm-symbolizer` 或 `addr2line`）
`ASAN_OPTIONS=log_path=asan.log`	报告写入文件
`ASAN_OPTIONS=detect_stack_use_after_return=1`	额外检测**返回后使用栈**（更慢）
`UBSAN_OPTIONS=print_stacktrace=1`	UBSan 报告附调用栈
`UBSAN_OPTIONS=halt_on_error=1`	UBSan 首个错误即停
`TSAN_OPTIONS=second_deadlock_stack=1`	TSan 死锁报告附第二个栈
`LSAN_OPTIONS=suppressions=lsan.supp`	泄漏抑制文件

A.6.3 `strace` 与 `ltrace`

命令	作用
`strace ./prog`	打印程序的所有**系统调用**及返回值
`strace -f ./prog`	**跟随子进程/线程**（`fork`、`clone` 产生的新进程也跟）
`strace -e trace=process ./prog`	只跟踪进程类调用（`fork` / `execve` / `wait4` / `exit_group`）

命令	作用
`strace -e trace=file ./prog`	只跟踪文件类调用（`openat` / `stat` / `read` / `write` / `close`）
`strace -e trace=network ./prog`	只跟踪网络类调用（`socket` / `connect` / `bind` / `sendto` / `recvfrom`）
`strace -e trace=openat,read,write ./prog`	精确指定调用名（可逗号分隔）
`strace -c ./prog`	**统计汇总**：每个系统调用的次数、耗时、错误数
`strace -T ./prog`	每条调用后显示**耗时**（秒）
`strace -t` / `-tt` / `-r`	显示时间戳 / 微秒时间戳 / 相对时间
`strace -o out.txt ./prog`	输出重定向到文件
`strace -p PID`	附加到运行中的进程
`strace -s 128 ./prog`	字符串最大打印长度（默认 32，看路径常需加大）
`strace -k ./prog`	附上用户态调用栈（需内核支持）
`strace -y ./prog`	把 fd 打印成对应路径（"这个 3 到底是哪个文件"）

命令	作用
`ltrace ./prog`	跟踪**库函数调用**（`malloc`、`printf`、`strcmp`…），看参数与返回值
`ltrace -e malloc+free ./prog`	只看指定库函数
`ltrace -c ./prog`	库函数调用统计

- **典型用法**：Shell Lab 调试 `fork` / `execve` / `waitpid` 用 `strace -f -e trace=process ./tsh`；Proxy Lab 看 socket 用 `strace -e trace=network`；"文件打不开"用 `strace -e trace=file -y`。

A.6.4 `perf`：硬件性能计数器

命令	作用
`perf stat ./prog`	汇总统计：`cycles`、`instructions`、`IPC`、缓存未命中、分支预测失败
`perf stat -e cache-misses,LLC-load-misses ./prog`	指定事件计数
`perf stat -r 5 ./prog`	重复 5 次并给方差
`perf record -g ./prog`	**采样**并记录，`-g` 采集调用图（`perf.data`）
`perf record -e LLC-load-misses -c 1000 ./prog`	指定事件与采样周期
`perf report`	交互式查看 `perf.data`（按热点排序、展开调用图）
`perf report --stdio`	文本方式输出报告
`perf top`	实时查看**系统范围**热点函数
`perf annotate`	把热点精确到**指令级**（哪个汇编指令最费时）
`perf list`	列出本机支持的全部事件名
`perf stat -a -I 1000`	系统级每秒采样（`-a` 全体 CPU）

常用事件	含义
`cycles`	CPU 周期数（与频率结合可算时间）
`instructions`	退休指令数；`instructions/cycles` 即 IPC
`cache-references`	缓存访问次数
`cache-misses`	缓存未命中次数
`L1-dcache-loads` / `L1-dcache-load-misses`	L1 数据缓存读次数/未命中
`LLC-loads` / `LLC-load-misses`	**最后一级缓存**（L3）读次数/未命中——访存瓶颈的核心指标
`branch-instructions` / `branch-misses`	分支数/预测失败数（>5% 通常意味着分支代价高）

常用事件	含义
`dTLB-loads` / `dTLB-load-misses`	数据 TLB 访问/未命中
`page-faults` / `minor-faults` / `major-faults`	缺页总数 / 次缺页 / 主缺页（含磁盘 I/O）
`context-switches` / `cpu-migrations`	上下文切换 / CPU 迁移次数
`task-clock`	占用的 CPU 时间（毫秒）

- `perf` 需要内核权限：`sudo perf ...` 或把 `perf_event_paranoid` 调低。
- 性能优化流程：`perf stat` 定位瓶颈类别（IPC 低？缓存未命中高？）→ `perf record -g` 找热点函数 → `perf annotate` 找热点指令。

A.6.5 `gprof`、`time`、`ulimit`

工具	命令	作用
`gprof`	`gcc -pg -g prog.c -o prog` 然后 `./prog`	运行后在 `gmon.out` 记录采样，`gprof ./prog gmon.out` 给出**平坦剖面**（每个函数自身耗时）与**调用图**（谁调谁、调用次数）
`time`	`time ./prog`	打印 `real`（墙钟）/ `user`（用户态 CPU）/ `sys`（内核态 CPU）
`time`	`/usr/bin/time -v ./prog`	详细版：峰值内存（Maximum resident set size）、缺页、上下文切换
`ulimit`	`ulimit -c unlimited`	允许生成 core dump（配合 `gdb ./prog core` 事后调试）
`ulimit`	`ulimit -v 1000000`	限制**虚拟内存**（KB）；测试 malloc 失败路径、制造 OOM
`ulimit`	`ulimit -s 8192`	限制**栈大小**（KB）；制造栈溢出实验
`ulimit`	`ulimit -a`	查看全部限制

- `gprof` 只统计**用户态**且需要 `-pg` 重编译，对库调用与内联函数不敏感；现代更推荐 `perf`。
- `-pg` 与 `-O2` 可同用，但内联会让调用图失真。

A.6.6 `/proc/<pid>/` 常用文件

路径	内容
`/proc/<pid>/maps`	**虚拟内存区域（VMA）清单**：地址区间、权限（`rwxp`）、偏移、设备、inode、路径
`/proc/<pid>/smaps`	每个 VMA 的**详细统计**：`Rss`（驻留物理页）、`Pss`、`Shared_Clean/Dirty`、`Private_*`、`Swap`
`/proc/<pid>/smaps_rollup`	上述统计的汇总（Linux 4.14+）
`/proc/<pid>/status`	人类可读的进程状态：`VmPeak` / `VmSize`（虚拟）、`VmRSS`（驻留）、`VmData` / `VmStk` / `VmExe`、`Threads`、`SigQ`
`/proc/<pid>/stat`	机器可读的单行状态（utime/stime、缺页数、优先级等）
`/proc/<pid>/statm`	以**页**为单位的极简内存摘要（size/resident/shared/text/lib/data/dirty）
`/proc/<pid>/fd/`	打开的文件描述符目录，`ls -l` 可见每个 fd 指向的文件/socket/pipe
`/proc/<pid>/fdinfo/<n>`	某个 fd 的偏移、flags、锁信息
`/proc/<pid>/task/`	每个**线程**一个子目录（内含各自的 `stat`、`maps`、`status`）
`/proc/<pid>/cmdline`	命令行参数（以 `\0` 分隔）
`/proc/<pid>/environ`	环境变量（以 `\0` 分隔）
`/proc/<pid>/cwd`、`/proc/<pid>/exe`、`/proc/<pid>/root`	当前目录、可执行文件、根目录的符号链接
`/proc/<pid>/stack`	内核态调用栈（需权限）
`/proc/<pid>/io`	读写字节数、syscall 次数
`/proc/meminfo`	系统级内存概况：`MemTotal` / `MemFree` / `MemAvailable` / `Buffers` / `Cached` / `SwapTotal`
`/proc/self/maps`	读自己的映射表（进程内 `cat /proc/self/maps` 即可观察自己的地址空间）

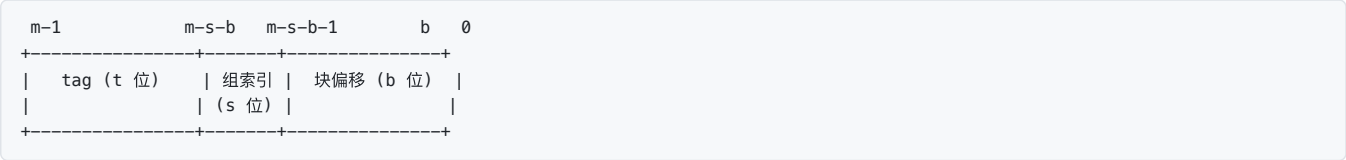
- 排查"内存涨了但不知道是谁"：对比 `/proc/<pid>/smaps` 里各 VMA 的 `Rss`；排查"到底是哪个文件"：`ls -l /proc/<pid>/fd/`。

A.7 存储与缓存速查

A.7.1 缓存组织的基本公式

符号	含义	关系
C	缓存总容量（字节，不含标记/有效位等元数据）	$C = S \times E \times B$
B	块大小（block size，字节）	$B = 2^b$，$b = \log_2 B$
S	组数（number of sets）	$S = 2^s$，$s = \log_2 S$
E	每组的行数（相联度，associativity）	$E = 1$ 直接映射；$E = S$ 全相联；否则组相联
m	地址位数	—
t	标记位数（tag）	$t = m - s - b$
e	行号位数	$E = 2^e$

地址切分（从高位到低位）



字段	作用
块偏移（block offset，低 b 位）	选块内第几个字节
组索引（set index，中间 s 位）	选第几个组——**取的是中间位**，这样相邻块（低 b 位不同）和跨组的大步长（高位不同）都能被区分
标记（tag，高 t 位）	与行中存的标记比较，判定是否命中；**取高位**

- **为什么组索引取中间位**：若用高位做索引，连续地址会全部落在同一组，造成严重冲突；用低位做索引则大步长访问（跨块）会全撞一组。中间位同时打散这两类模式。
- 一条缓存行（line）的元数据：**有效位**（valid bit）、**标记**（tag）、**脏位**（dirty bit，仅写回策略需要），外加 B 字节的数据块。
- **组相联查找**：组索引选中一组 → 组内 E 行**并行**比较标记与有效位 → 命中则按块偏移取字节，未命中则从下一级取块并替换组内某行。

A.7.2 命中 / 未命中 / 替换 / 有效位 / 脏位

术语	含义
命中（hit）	组内某行的有效位为 1 且标记与地址的 tag 相等
未命中（miss）	组内没有任何行同时满足"有效且标记相等"
有效位 = 0	该行**从未被加载**过（冷启动）或已被作废；不能参与命中判定
脏位 = 1	该行被写过且尚未写回下一级；**被替换时必须先写回**（写回策略下）
替换策略	直接映射：别无选择，覆盖唯一那行；组相联/全相联：LRU（最近最少使用）、近似 LRU、随机
写回（write-back）与**写直达**（write-through）	见 A.7.3
驱逐 / 牺牲行（eviction / victim）	被新块覆盖的那一行

A.7.3 写策略四格表

写命中时	写回（write-back）	写直达（write-through）
含义	只改缓存行，置脏位；**被驱逐时**才写回下一级	同时写缓存**和**下一级（可加写缓冲 write buffer 缓解）
优点	减少写流量（同一行的多次写只下传一次）	实现简单，下一级始终最新，易做多核一致性

写命中时	写回（write-back）	写直达（write-through）
缺点	需要脏位；驱逐路径变慢；一致性协议更复杂	写流量大（每次写都下传）
典型使用	L1/L2/L3 缓存（现代 CPU 全部用写回）	早期机器、某些设备的帧缓冲路径

写未命中时	写分配（write-allocate）	非写分配（no-write-allocate）
含义	先把块**调入缓存**，再在缓存里写（"为写而读"）	直接写下一级，**不调入**该块
优点	后续对同一块的写/读都能命中，利于局部性	避免"写一次就不再用"的数据污染缓存
缺点	每次写未命中都多一次读（fetch-on-write）	后续再访问仍会未命中
典型搭配	与**写回**配套（最常见组合）	与**写直达**配套

- **记忆**：真实的 L1/L2/L3 = **写回 + 写分配**；简单的写直达缓存常配**非写分配**。
- 写回策略下，**丢弃脏行是数据丢失**；`msync` / `fsync` 就是把"缓存里的脏数据"强制推到下一级。

A.7.4 3C 未命中分类与削减手段

类别	英文	成因	削减手段
强制/冷未命中	compulsory (cold) miss	该块**第一次**被访问，缓存里必然没有	增大块大小（利用空间局部性）、**预取**（prefetching）、软件预取指令
冲突未命中	conflict miss	组相联度不够：多个**活跃**块映射到**同一个组**，互相驱逐	**提高相联度** E、增大 B（减少组数但每行更大）、**分块/填充**（padding）错开地址、编译器数组重排
容量未命中	capacity miss	工作集（working set）**大于**缓存容量，躲不开	增大缓存容量、**分块（blocking）**提高时间局部性、降低工作集、循环变换

- 判定顺序：先看是不是第一次访问（冷）→ 再看"若缓存无限大是否还会未命中"（容量）→ 否则是冲突。
- 冲突未命中是**唯一可以不改硬件、只改代码就消除**的一类（用 padding 或重排访问顺序）。
- 3C 之外常补充第四类：**一致性未命中（coherence miss）**——其他核写入了共享块导致本核的副本作废。

A.7.5 AMAT：平均访存时间

$$\textbf{AMAT} = \text{命中时间}\textbf{(Hit Time)} + \text{未命中率}\textbf{(Miss Rate)} \times \text{未命中惩罚}\textbf{(Miss Penalty)}$$

多级缓存的 AMAT 递推（把下一级整体看成"上一级的未命中惩罚"）：

$$\text{AMAT}_{L1} = t_{L1} + m_{L1} \times \left(t_{L2} + m_{L2} \times \left(t_{L3} + m_{L3} \times t_{\text{Mem}}\right)\right)$$

量	定义	讲义典型值
未命中率（miss rate）	未命中次数 / 访问次数 $= 1 -$ 命中率	L1 约 3–10%；L2 < 1%；L3 更小
命中时间（hit time）	判定命中并把数据送到处理器的时间	L1 约 4 周期；L2 约 10 周期；L3 约 40–75 周期
未命中惩罚（miss penalty）	因未命中额外付出的时间	主存约 50–200 周期（趋势：**越来越大**）
局部未命中率（local miss rate）	该级未命中数 / **到达该级**的访问数	用于递推
全局未命中率（global miss rate）	该级未命中数 / **L1 的总访问数**	衡量该级对整体性能的贡献

- **算例（讲义原例）**：命中 1 周期、未命中惩罚 100 周期。 97% 命中 → $1 + 0.03 \times 100 = 4$ 周期；99% 命中 → $1 + 0.01 \times 100 = 2$ 周期。**即"99% 命中率是 97% 的两倍好"**——命中率的微小提升在 AMAT 上被放大。
- 优化方向只有两条：**降低未命中率**（局部性、分块）或**降低未命中惩罚**（多级缓存、预取、更宽的带宽）。

A.7.6 Intel Core i7 缓存层次参数

层级	容量	相联度 E	组数 S	访问延迟	备注
L1 d-cache（数据）	32 KB	8 路	64	4 周期	每核私有
L1 i-cache（指令）	32 KB	8 路	64	4 周期	每核私有
L2 unified（统一）	256 KB	8 路	512	10 周期	每核私有，指令+数据

层级	容量	相联度	组数	访问延迟	备注
L3 unified（统一）	8 MB	16 路	8192	40–75 周期	**所有核共享**（core 0–3）
主存	—	—	—	~200 周期	—

- **块大小对所有层级统一为 64 字节**（$b = 6$）。
- **L1 d-cache 的地址位域（47 位物理地址，Core i7）**：$C = 64 \times 8 \times 64 = 32768$ B； $S = C/(E \times B) = 32768/(8 \times 64) = 64 \Rightarrow s = 6$；$b = 6$；$t = 47 - 6 - 6 = 35$ 位。

```
物理地址 47 位 → L1 d-cache 划分
 46            12 11        6 5          0
+----------------+-----------+------------+
|  tag: 35 位     | set: 6 位 | off: 6 位   |
+----------------+-----------+------------+
例：地址 0x00007f7262a1e010
    tag  = 0x7f7262a1e
    set  = 0x0        (位 11..6)
    off  = 0x10       (位 5..0，落在块内第 16 字节)
```

层级	组索引位数 s	标记位数 t（47 位地址）
L1（32 KB，8 路，64 B 块）	6	35
L2（256 KB，8 路，64 B 块）	9	32
L3（8 MB，16 路，64 B 块）	13	28

- 术语对照：**L1/L2/L3 = 缓存层级**；**i-cache = 指令缓存、d-cache = 数据缓存、unified = 二者合一**；**块偏移/组索引/标记**三段切分对每一层都成立，只是 s、t 不同。
- 页大小 4 KB = 64 块/页，因此**同页内的相邻块组索引不同、标记相同**——这是"TLB 与 L1 可并行查找"的硬件基础（决定组索引的位在 VA 与 PA 中相同）。

A.7.7 局部性速查

类型	定义	例子	利用方式
时间局部性（temporal locality）	被访问过的数据/指令**很快会被再次访问**	循环变量 `i`、循环体内的指令、被反复读的数组元素	把数据留在缓存里；分块提高复用次数
空间局部性（spatial locality）	被访问的地址**附近的地址很快会被访问**	顺序遍历数组、顺序执行的指令流	增大块大小；顺序访问而非跳跃访问

访问模式	结论
步长 stride = 1（`a[i]` 逐个访问）	最好：每个块只付一次未命中，块内 8 个 `double` 全用上
步长较小（stride < B）	仍有空间局部性，但每块只用部分字节
步长较大（stride ⩾ B）	空间局部性基本消失，每个元素都可能是一块的首字节 → 未命中率趋近 100%
循环嵌套顺序	**按行访问（row-major，内层走列索引）** 对 C 数组最友好；按列访问（`a[j][i]` 外层变 i）会跨行跳转，未命中剧增
同一行被重复访问	时间局部性：内层循环复用同一行 n 次，远好于每次换行

- 验算口诀：**"每 $B/\text{sizeof(elem)}$ 个连续元素共享一次未命中"**。`double` 数组、64 B 块 → 每 8 个元素一次未命中。
- 矩阵 `sumA += a[i][j]` 按行 vs 按列，实测可差 **10 倍以上**（Core i7 上讲义实测从约 20 GB/s 掉到不足 2 GB/s）。

A.7.8 分块（Blocking）速查

原则：把大矩阵切成 $B \times B$ 的小块，让**一个块在被换出之前被尽可能多次复用**——把"对全矩阵的时间局部性"压缩到"对一个小块的时间局部性"，使工作集装得进缓存。

要点	内容
目标	提高**时间局部性**，把容量未命中降下来
工作集约束	同时活跃的块要能全部驻留：对矩阵乘，需要"两个输入块 + 一个输出块"
块大小选择	取**满足 $3B^2 < C$ 的最大 B**（C 为缓存容量，单位与 B^2 一致）——三个 $B \times B$ 块（`a`、`b`、`c`）必须同时装进缓存

要点	内容
未命中数对比	不分块：$\frac{9}{8}n^3$ 次未命中；分块：$\frac{n^3}{4B}$ 次（以"块 = 8 个 double"为例）
收益来源	矩阵乘本身有 $O(n)$ 的时间局部性（$3n^2$ 个输入元素被用 $2n^3$ 次），**但必须写对循环顺序**才能兑现
常见陷阱	B 太大 → 三块装不下，退化为不分块；B 太小 → 块内空间局部性用不满；索引变量写错 → 结果正确但没提速
推广	分块思路可用于转置、矩阵向量乘、卷积、稀疏矩阵；本质是"**循环变换 + 提高局部性**"，不改变算术量

64×64 转置的 4×4 子块 + 8 局部变量技巧（Cache Lab 经典）

步骤	做法	为什么
① 不直接逐元素转置	朴素 `B[j][i] = A[i][j]` 使 A 按行读、B 按列写，两者都跨块跳跃	一次访问只用一个元素，空间局部性浪费
② 划 4×4 子块	外层循环以 4 为步长遍历块，块内先把 A 的 4 行读进局部变量	块内 4 行×4 列 = 16 个元素分布在 2 个块内（8 个 `int`/块）
③ 用 **8 个局部变量**（或 `int` 寄存器变量）作中转	把 4×4 的 16 个元素分批搬到 B 的 4×4 区	减少对 B 的重复读写：B 的 4 行各只在最终写入时被触碰一次
④ 对角线块特判	4×4 块落在对角线上时，A 块与 B 块重叠，逐元素处理更省	避免同一块的读写互相驱逐
⑤ 更激进：8×8 块 + 变体	把命中率从 4×4 方案进一步提升到 ~95%+	Cache Lab 满分需要更细的分块与临时变量

- **测什么**：Cache Lab 用 `csim` 模拟器统计 `hits/misses/evictions`，再对真实 `trans` 用 `valgrind --tool=cachegrind` 看 `D1 miss rate`。
- 判断是否有效的实操：先 `perf stat -e L1-dcache-load-misses,LLC-load-misses ./prog` 看基线，再改分块后对比。

A.7.9 存储器山（Memory Mountain）读法

定义：以**读取吞吐率（read throughput，MB/s）**为高度，横轴为**工作集大小**（size，从 32 KB 到 128 MB，对数刻度），另一轴为**步长**（stride，1–11 个 `long`，即 8–88 字节）的三维地形图。它把内存系统的空间/时间局部性**一次性画成一张图**。

图上要素	含义
山脊（ridges of temporal locality）	固定步长、扫过工作集大小时看到的高处：工作集落进 L1/L2/L3 时吞吐高，超出后陡降——每一级缓存是一道"平台"，平台边缘即该级容量
山坡（slopes of spatial locality）	固定工作集、增大步长时看到的下降：步长超过块大小（64 B = 8 个 `long`）后吞吐骤降，因为每块只用 1 个元素
最高峰（约 14000 MB/s）	小工作集 + 步长 1：全在 L1，且空间局部性拉满
最低谷（约 2000 MB/s 以下）	工作集远大于 L3（128 MB）+ 大步长：几乎每次访问都到主存
台阶状下降的位置	32 KB（L1）、256 KB（L2）、8 MB（L3）三处——**台阶边界就是各级缓存的容量**
尾部"翘起"（aggressive prefetching）	大步长、大工作集时硬件预取器仍能拉起一部分吞吐

- **读法三步**：① 沿某根"脊"（固定 stride）走，定位三级台阶 → 读出 L1/L2/L3 容量；② 沿某个"坡"（固定 size）走，看 stride 从 1 增到 11 的跌幅 → 读出块大小的影响；③ 比较不同机器的山形，即可判断其缓存层次与预取策略的强弱。
- 测量方法（讲义 `mountain.c`）：`test(elems, stride)` 用 **4×4 循环展开**累加，先跑一次**预热缓存**，再跑一次计时；对多种 `elems`/`stride` 组合重复，绘制成图。
- 实践意义：**"为什么我的程序忽快忽慢"**——把访问模式（size × stride）落在这张图上，就知道它在吃哪一级缓存。

A.8 虚拟内存速查

A.8.1 术语表

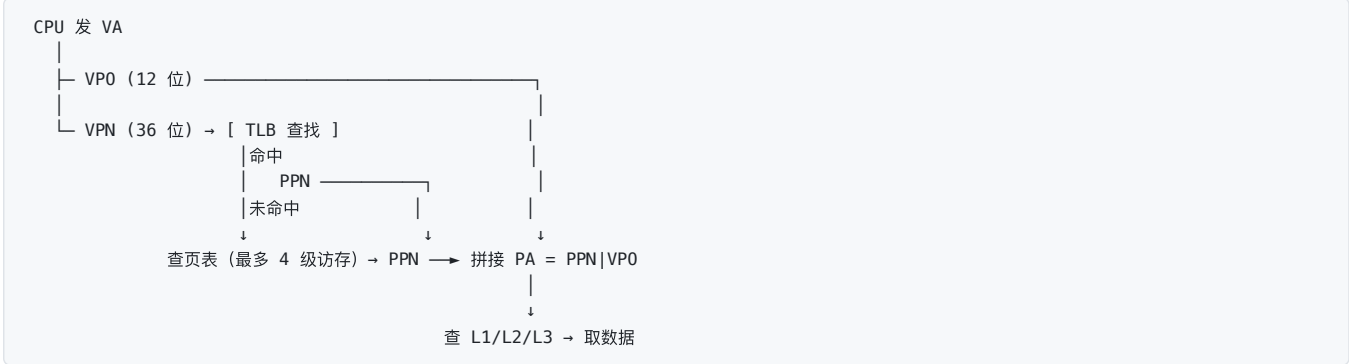
术语	英文	含义
VA	Virtual Address	虚拟地址，CPU 发出的地址，每进程独立
PA	Physical Address	物理地址，DRAM 上的真实地址
页	page（VP, virtual page）	虚拟地址空间被切成的定长块，大小 $P = 2^p$（x86-64 典型 4 KB，$p = 12$）

术语	英文	含义
页帧 / 物理页	page frame（PP, physical page）	物理内存被切成的同样大小的块
页表	page table	把 VPN 映射到 PPN 的**数组**，每项是一个 PTE；每个进程一棵（或一套）
PTE	Page Table Entry	页表项：PPN + 有效位 + 权限位（R/W/X/U）+ 脏位 + 访问位
VPN / VPO	Virtual Page Number / Offset	虚拟地址的高位（页号）与低位（页内偏移）
PPN / PPO	Physical Page Number / Offset	物理地址的高位与低位
MMU	Memory Management Unit	CPU 内完成地址翻译的硬件
TLB	Translation Lookaside Buffer	MMU 内缓存 **VPN → PPN 完整映射**的小型组相联缓存
缺页	page fault	访问的页不在内存（PTE 有效位为 0）时触发的异常，由内核处理
VMA	Virtual Memory Area	Linux 用一个结构描述一段连续、同权限的虚拟地址区间
按需分页	demand paging	页只在**第一次被访问**时才换入，`mmap` / `malloc` 本身不分配物理页
抖动	thrashing	工作集超过物理内存，页面反复换入换出，性能崩溃
CR3	—	存放**顶级页表**物理基址的寄存器；切换进程即切换 CR3

A.8.2 关键恒等式与翻译流程

恒等式 / 规则	内容
偏移不变	$\mathbf{VPO} = \mathbf{PPO}$（页内偏移在翻译前后**完全相同**，因此可并行取 TLB 与 L1）
位数划分	VA 共 m 位：p 位页偏移 + $(m-p)$ 位 VPN；PA 共 m' 位：p 位偏移 + $(m'-p)$ 位 PPN
x86-64 数值	$m = 48$（规范地址），$p = 12$ → **VPN 36 位**、页偏移 12 位；PPN 最多 40 位（52 位物理地址）
页大小	$P = 2^p$，4 KB = 2^{12}；大页 2 MB（2^{21}）、1 GB（2^{30}）
页内地址	$\mathbf{VA} = \mathbf{VPN} \times P + \mathbf{VPO}$，$\mathbf{PA} = \mathbf{PPN} \times P + \mathbf{VPO}$
PTE 大小	x86-64 每项 **8 字节**；每级页表 **512 项** × 8 B = 4096 B = **恰好 1 页**
单级页表为何不可行	2^{36} 个 VPN × 8 B = 2^{39} B = **512 GiB**（每进程一张）

地址翻译数据通路

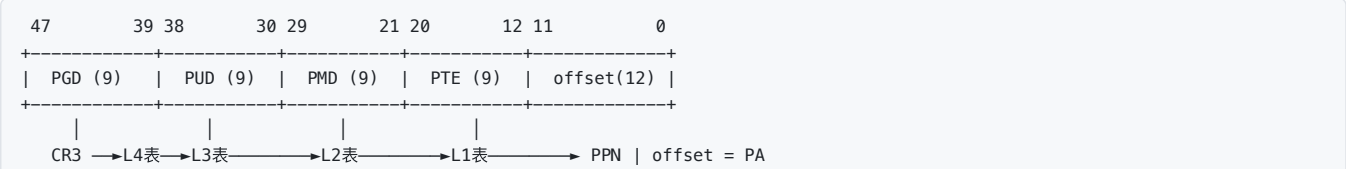


步骤	内容
1	MMU 用 VPN 查 TLB（TLBI 选组，TLBT 比对标记）
2	**命中** → 直接得 PPN，省掉访存；**未命中** → 走页表（PTEA = 页表基址 + VPN×8）
3	逐级查页表；若某级 PTE 有效位为 0 → **缺页异常**，陷入内核
4	内核处理缺页后更新 PTE，重新执行**原指令**（此时 TLB 已填充）
5	拼出 PA，与 VPO 一起访问缓存/主存

- **TLB 命中省掉一次内存访问**；TLB 未命中只是"多一次访存"，与缺页（可能含磁盘 I/O）**量级完全不同**。
- 无论页表有多少级，**TLB 缓存的始终是完整的 VPN → PPN 映射**。

A.8.3 x86-64 四级页表位划分

字段	位区间	位数	名称
PGD 索引（L4）	47—39	9	Page Global Directory（顶级，CR3 指向它）
PUD 索引（L3）	38—30	9	Page Upper Directory
PMD 索引（L2）	29—21	9	Page Middle Directory
PTE 索引（L1）	20—12	9	Page Table Entry
页偏移	11—0	12	VPO = PPO



要点	说明
每级表大小	512 项 × 8 B = 4 KB = 1 页，天然页对齐（**这就是为什么每级恰好 9 位：$2^9 = 512$**）
覆盖范围	顶级每项覆盖 512 GB；每下一级依次覆盖 1 GB、2 MB、4 KB
为什么要多级	只为**真正用到的地址路径**分配下级页表：散布在地址空间各处的 5 个区域最多占 5×4 = 20 张表 ≈ 80 KB，而非 512 GiB
代价	TLB 未命中时最多 **4 次**访存（可被页表缓存/大页缓解）
PTE 关键位	`P` 有效位（bit 0）、`R/W` 可写（bit 1）、`U/S` 用户/内核（bit 2）、`A` 访问位（bit 5）、`D` 脏位（bit 6）、`XD` 不可执行（最高位，即 NX/No-eXecute）
规范地址	x86-64 的 48 位 VA 必须符号扩展到 64 位（高位全 0 或全 1）；非规范地址立即 `#GP`
CR3 切换	进程切换 = 写 CR3 = 换一棵页表树；内核页表映射在所有进程地址空间的高半部

A.8.4 缺页处理流程

步骤	动作	判定
1	CPU 访问 VA，MMU 查到某级 PTE 的**有效位为 0**	硬件触发缺页异常（`#PF`），压入出错地址（`CR2`）与 error code
2	内核取 `mmap_lock`，用出错地址查 **VMA**（`find_vma`）	若**没有**包含该地址的 VMA → **段错误（SIGSEGV）**，进程终止
3	若找到 VMA，检查**权限**是否与本次访问匹配（读/写/执行、用户/内核）	写只读页（如 COW 页、`.text`）→ 保护异常：**合法则做 COW 复制**，非法则 `SIGSEGV`
4	判定缺页类型	**次缺页（minor）**：页已在内存（page cache 命中、COW、零页）→ 直接建映射；**主缺页（major）**：需从磁盘/交换区读入 → 阻塞 I/O
5	选出牺牲页（若物理内存满）：脏页先**写回**，干净页直接丢弃；更新被牺牲页的 PTE 有效位为 0	替换策略（近似 LRU / CLOCK）
6	把页换入内存，**更新 PTE**：填 PPN、置有效位、设置权限位（清脏位）	—
7	**重新执行触发缺页的那条指令**	这一次翻译成功，程序继续，用户态完全察觉不到
8	若配置了预取/`MAP_POPULATE`，可一并换入相邻页	减少后续缺页次数

- **关键洞察**："合法的地址 + 无效的 PTE" 是**正常流程**（按需分页），不是错误；只有"地址不在任何 VMA"或"权限不符"才是真正的段错误。
- **error code 读法**（`dmesg` 里 `segfault at 0 error 4`）：bit 0 = 保护违规（1）还是页不存在（0）；bit 1 = 写（1）还是读（0）；bit 2 = 用户态（1）还是内核态（0）。
- 观察命令：`perf stat -e page-faults,minor-faults,major-faults ./prog`；`/usr/bin/time -v ./prog` 看缺页总数。

A.8.5 TLB 参数（Intel Core i7）

TLB	项数	相联度	组数	覆盖内容
L1 d-TLB（数据）	64	4 路	16	4 KB 页的 VPN → PPN
L1 i-TLB（指令）	128	4 路	32	同上（指令侧）
L2 TLB（统一）	512	4 路	128	统一，容量更大、延迟更高
大页 TLB	32 项	4 路	8	2 MB / 4 MB 大页

- **位域计算（L1 d-TLB）**：VPN 共 36 位，组数 $= 64/4 = 16 \Rightarrow$ TLBI = 4 位；TLBT = 36 − 4 = 32 位。
- **性能含义**：工作集（活跃页数）≤ TLB 项数时命中率接近 100%；一旦超出，每轮都要换入换出，命中率崩盘——**"工作集小于 TLB 容量则快"**。
- 大页（2 MB）把"每页 4 KB 一个 TLB 项"变成"每页 2 MB 一个 TLB 项"，同样项数覆盖 512 倍的内存，是数据库/大内存程序的常用优化。
- 观察命令：`perf stat -e dTLB-loads,dTLB-load-misses,page-faults ./prog`。
- **TLB 与缓存的协同**：VPO = PPO，且决定 L1 组索引的位在 VA 与 PA 中相同，因此 **TLB 查找与 L1 组索引可以并行**；页命中时两者同时完成，进一步掩盖翻译延迟。

A.8.6 Linux VMA 标志

标志	含义
`VM_READ`	可读（对应 `PROT_READ`，`maps` 里的 `r`）
`VM_WRITE`	可写（`PROT_WRITE`，`w`）
`VM_EXEC`	可执行（`PROT_EXEC`，`x`）
`VM_SHARED`	**共享**映射：写入对同一映射的其他进程可见（`MAP_SHARED`，`maps` 里的 `s`）；否则为私有 `p`
`VM_MAYREAD` / `VM_MAYWRITE` / `VM_MAYEXEC` / `VM_MAYSHARE`	"**允许**被 `mprotect` 改成"对应权限的标志位
`VM_GROWSDOWN`	该 VMA 是**向下增长**的栈区域（缺页地址紧邻 `vm_start` 之下时自动扩展栈）
`VM_GROWSUP`	向上增长（某些体系结构的栈）
`VM_IO` / `VM_PFNMAP`	内存映射 I/O（设备寄存器），**不参与分页**
`VM_DONTCOPY` / `VM_DONTEXPAND` / `VM_LOCKED`	`fork` 时不复制 / 不允许 `mremap` 扩展 / 锁定在内存中不换出（`mlock`）
`VM_ACCOUNT` / `VM_NORESERVE`	计入/不计入地址空间记账（overcommit 相关）
`VM_HUGETLB`	使用大页

- `/proc/<pid>/maps` 每行的权限串 `rwxp` 就是 `VM_READ|VM_WRITE|VM_EXEC` 加 `VM_SHARED`（`s` 而非 `p`）的组合；`---p` 表示保留但不可访问（如 guard page）。
- 权限位在 **PTE 里**（硬件实际执行），VMA 标志是**内核记账**；`mprotect` 同时改两者中受影响页的 PTE。

A.8.7 `mmap` 用法与三大用途

```
#include <sys/mman.h>
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
int munmap(void *addr, size_t length);
int mprotect(void *addr, size_t len, int prot);
int msync(void *addr, size_t len, int flags);
```

参数	取值	说明
`addr`	`NULL`（推荐）或指定地址	`NULL` 让内核选地址
`length`	字节数	内核按页向上取整
`prot`	`PROT_READ` / `PROT_WRITE` / `PROT_EXEC` / `PROT_NONE`	可或组合；`PROT_NONE` 用作 guard
`flags`	**必须**含 `MAP_SHARED` 或 `MAP_PRIVATE` 之一	见下表
`fd` / `offset`	文件描述符与偏移	`offset` **必须是页大小整数倍**；匿名映射传 `-1, 0`

参数	取值	说明
返回值	页对齐地址；失败返回 `MAP_FAILED`（`(void*)-1`）并置 `errno`	必须检查

flag	含义
`MAP_SHARED`	共享：对映射的写入**对其他进程可见**，文件映射还会写回文件
`MAP_PRIVATE`	私有：**写时复制**（COW），改动不影响文件与其他进程
`MAP_ANONYMOUS` / `MAP_ANON`	匿名映射：不对应文件（`fd` 忽略），内容是**零页**；`malloc` 的大块走这条路
`MAP_FIXED`	强制映射到 `addr`（**危险**：会覆盖已有映射）
`MAP_FIXED_NOREPLACE`	要求地址精确且不覆盖已有映射（较安全）
`MAP_POPULATE`	预先建立页表并换入（把缺页代价提前，适合已知会被全访问的区域）
`MAP_NORESERVE`	不为交换空间预留（overcommit）
`MAP_LOCKED` / `mlock`	锁定不换出
`MAP_HUGETLB`	使用大页
`MAP_GROWSDOWN`	向下增长（`mmap` 出的栈）
`MAP_STACK`	提示内核这是栈区域

三大用途	说明
① **映射文件**（file mapping）	`fd` 指向文件，读写映射区即读写文件；`MAP_SHARED` 写入经 page cache 反映到文件（`msync` 强制刷盘），`MAP_PRIVATE` 得到写时复制的私有副本。可用于"把大文件当数组随机访问"而无需 `read`
② **匿名内存**（anonymous memory）	`MAP_ANONYMOUS`：不涉及文件，初始为零页；`malloc` 大块、线程栈、`brk` 之外的所有内存需求都可由它满足；`free` 时 `munmap` 直接归还内核
③ **共享内存 / 进程间通信**（IPC）	两个进程对同一文件（或 `shm_open` 对象）做 `MAP_SHARED` 映射，读写同一批物理页即完成共享；共享库的代码段也是这种方式被多个进程共享

- 惯用法：`char *p = mmap(NULL, len, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);`
- **`mmap` 之后并不分配物理页**（实测缺页增量为 0）——这正是按需分页：只有第一次触碰某页才产生次缺页。
- `munmap` 归还整个区间；`madvise(MADV_DONTNEED)` 归还**物理页但保留 VMA**（适合"释放但留着以后再分配"）。

A.8.8 `fork` 写时复制（COW）与 `execve`

机制	要点
`fork` 的语义	复制整个地址空间（代码、数据、堆、栈、文件描述符表）
`fork` 的实现	**只复制页表，不复制物理页**：父子 PTE 指向同一批物理页，且都标记为**只读 + COW**
写时复制	任一方**写入**触发保护异常 → 内核复制**那一页**，把写方的 PTE 指向新副本并恢复可写，另一方的 PTE 保持不变（引用计数减 1）
为什么快	`fork` 只花"复制页表 + 建 VMA"的时间（微秒级），与地址空间大小**基本无关**
引用计数	物理页有引用计数；最后一个使用者退出或 `munmap` 时才真正释放
与 `execve` 的关系	`fork` 后立刻 `execve` 时，子进程往往一页都没写过 → 全部页面直接丢弃，**零拷贝浪费**；这正是 `posix_spawn` / `vfork` 想优化的场景
陷阱	`fork` 后父子共享文件偏移（同一个打开文件表项）；`fork` 在内核缓冲区/锁状态下调用会有可重入问题

`execve` 的内存映射步骤	内容
1	校验可执行文件（ELF 头、体系结构）
2	丢弃**旧**地址空间的用户态映射（内核映射保留）
3	映射 `.text` 等只读可执行段：**文件映射 + `PROT_EXEC`**，**`MAP_PRIVATE`**，PTE 初始无效
4	映射 `.data`：`MAP_PRIVATE`（写时复制）、可读写；初始内容来自文件
5	`.bss` 等零初始化区域：**匿名映射**，靠"零页 + 缺页"实现"初始化为 0"而**不占文件空间**
6	映射共享库（`ld.so` 先启动，按 `NEEDED` 加载），代码段多进程共享同一份物理页

`execve` 的内存映射步骤	内容
7	建立初始栈：压入 `argv`、`envp`、辅助向量（auxv）
8	把 `%rip` 指向**入口点或动态链接器**，跳入用户态开始执行

- **核心结论**：`execve` 之后**所有页都是按需换入的**（PTE 无效），所以"启动新程序"并不需要先把整个程序读进内存。
- 观察：`cat /proc/<pid>/maps` 能看到 `r-xp` 的 `.text` 文件映射、`rw-p` 的匿名堆栈、`r--p` 的只读共享库段。

A.8.9 `malloc` 的 `brk` / `mmap` 双路径

路径	触发条件	机制	释放行为
`brk` 堆	较小的请求（默认 < 128 KB）	分配器用 `sbrk` / `brk` 在**堆顶**推进 `brk` 指针；`[heap]` 区间就是这段虚拟区间	`free` 通常只是把块**放回空闲链表**，不归还内核（堆顶收缩只在末尾大块被释放时发生）
`mmap` 区	较大的请求（默认 ≥ `MMAP_THRESHOLD` = 128 KB）	直接用 `mmap(NULL, size, PROT_READ|WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)` 建独立映射	`free` 可直接 `munmap` **立即归还内核**

参数 / 命令	说明
`M_MMAP_THRESHOLD`（默认 **128 KB**）	超过它就走 `mmap`；glibc 还会**动态调整**该阈值（观察到 `mmap` 块被 `free` 后可能上调）
`M_TRIM_THRESHOLD`（默认 128 KB）	堆顶空闲超过该值就收缩堆（`brk` 回退）
`M_TOP_PAD`	每次 `sbrk` 额外多要的余量，减少系统调用次数
`mallopt(M_MMAP_THRESHOLD, n)`	运行时调整阈值
`MALLOC_MMAP_THRESHOLD_` 环境变量	同上（也支持 `MALLOC_TRIM_THRESHOLD_`、`MALLOC_TOP_PAD_`、`MALLOC_CHECK_`）
为什么分两条路	`brk` 路径分配快、碎片可控，但**归还困难**；`mmap` 路径**归还容易**，但每次分配/释放都是一次系统调用与一次缺页，且地址不连续、易造成 TLB/缓存压力
实测特征	`[heap]` 中的地址（如 `0x1351000`）来自 `brk`；`0x7f...` 附近来自 `mmap`
观察	`malloc(1<<20)` 落在 `mmap` 区；`strace -e trace=brk,mmap,munmap` 可看到分配器的系统调用轨迹

- **容易混淆**：`brk` 推的是**进程级**的堆顶；`mmap` 建的是**独立 VMA**。两者都只申请**虚拟**空间，物理页仍按需分配（因此"`malloc` 了 1 GB 却没爆内存"是正常的，直到你逐页触碰）。

A.8.10 常用命令（VM 相关）

命令	作用
`getconf PAGE_SIZE`	打印系统页大小（x86-64 上通常 `4096`）；`getconf _PHYS_PAGES` 得物理页数
`cat /proc/self/maps`	读**当前 shell 执行的 cat 自己**的 VMA 清单（地址区间、权限、偏移、路径）
`cat /proc/<pid>/maps`	观察指定进程的地址空间（`r-xp` 代码、`rw-p` 数据、`[heap]`、`[stack]`、`[vdso]`）
`pmap -x <pid>`	以表格列出每个映射的 `Kbytes`、`RSS`、`Dirty`、`Mode`、`Mapping`（比 `maps` 更直观）
`pmap <pid>`	简表（`-q` 只列映射、`-d` 显示设备格式）
`ulimit -v`	查看/设置**虚拟内存**上限（KB）
`ulimit -s`	栈大小上限（KB，默认 8192）
`ulimit -c unlimited`	允许 core dump（配合 `gdb ./prog core`）
`ulimit -a`	查看全部限制
`cat /proc/<pid>/statm`	以页为单位的极简内存摘要
`cat /proc/<pid>/smaps`	各 VMA 的详细内存统计（RSS / PSS / Shared / Private / Swap）
`cat /proc/meminfo`	系统内存与交换区概况

命令	作用
`sysctl vm.overcommit_memory`	查看 overcommit 策略（0 启发式、1 总是允许、2 严格）
`perf stat -e page-faults,minor-faults,major-faults,dTLB-load-misses ./prog`	统计缺页与 TLB 行为
`valgrind --tool=cachegrind ./prog`	模拟各级缓存的命中率（`cg_annotate` 按行/函数展开）
`getconf LEVEL1_DCACHE_SIZE` / `LEVEL1_DCACHE_ASSOC` / `LEVEL1_DCACHE_LINESIZE`	直接问系统 L1 数据缓存的容量/相联度/块大小
`lscpu`	汇总 CPU 与缓存层次信息（`-C` 可看缓存详情）
`dmesg | tail`	查看 `segfault at ... error N` 的出错地址与 error code

A.9 动态内存分配器速查

> 对应讲次：Lecture 13（`F25-13-malloc-basic.txt`）、Lecture 14（`F25-14-malloc-advanced.txt`）； 教材 CS:APP3e 第 9.9—9.11 节；关联 **L5a/L5b Malloc Lab**。 表中未出现在 F25 讲义里的实现细节（glibc 调参、错误信息文本）已标注**「补充说明」**。

A.9.1 `malloc` 家族接口语义

接口	原型（`<stdlib.h>`）	语义要点
`malloc`	`void *malloc(size_t size)`	成功：返回指向**至少 `size` 字节**的块的指针，且在 x86-64 上按 **16 字节边界对齐**；`size == 0` 时**返回 `NULL`**（讲义口径）；失败：返回 `NULL` 并置 `errno`
`calloc`	`void *calloc(size_t nmemb, size_t size)`	`malloc` 的**清零**版本：分配 `nmemb * size` 字节并**逐字节初始化为 0**；不做溢出检查时 `nmemb*size` 可能回绕
`realloc`	`void *realloc(void *p, size_t size)`	改变**已分配块**的大小；返回新块起始地址（**可能与 `p` 不同**），旧内容保留下 `min(旧大小, 新大小)` 字节
`free`	`void free(void *p)`	把 `p` 指向的块还给分配器；`p` **必须**来自之前的 `malloc` / `calloc` / `realloc`；`free(NULL)` 是合法的空操作
`sbrk`	`void *sbrk(intptr_t incr)`	**分配器内部**用来增长/收缩堆顶（`brk` 指针）；应用代码不应直接调用

`realloc` 的三个边界情形（讲义只给了"改变大小"一句，下表为**补充说明**，按 C 标准与 glibc 实际行为）：

调用	等价于	注意
`realloc(NULL, n)`	`malloc(n)`	合法，返回新块
`realloc(p, 0)`	glibc：`free(p)` 并返回 `NULL`	C17 起为**实现定义**，C23 起为**未定义行为**；**绝不要写 `p = realloc(p, 0)`**，否则 `p` 被覆盖成 `NULL`，旧块已释放
`realloc(p, n)` **失败**	返回 `NULL`，**原块 `p` 保持不变**	故必须写成 `tmp = realloc(p, n); if (!tmp) { /* p 仍有效 */ } else p = tmp;`

- **`malloc(0)` 的两套口径**：讲义规定返回 `NULL`；glibc 实际返回一个**唯一的最小可 `free` 块**（非 `NULL`）。写代码时应把 `NULL` 当"分配失败或零长度"处理，不要依赖 glibc 行为。
- **成功/失败判定**：`malloc` 失败时返回 `NULL` 并**设置 `errno`**，所以 `perror("malloc")` 有意义；但 `size == 0` 也返回 `NULL` 且**不设置 `errno`**——这是 `malloc` 返回值语义最容易踩的歧义。

A.9.2 分配器硬性约束与两个目标

#	硬性约束	含义	为什么不能违反
1	**不假设请求模式**	应用可发出**任意** `malloc` / `free` 序列；分配器不能假设大小会重复、不能假设"先全分配后全释放"；`free` 的参数必须是 malloc 出来的块	分配器是库，无法预知调用者行为
2	**立即响应**	必须在 `malloc` 返回前给出块，**不能重排或缓冲请求**（不能攒一批再统一处理）	调用者拿到指针后会立刻写入
3	**只能用堆里的空闲内存**	已分配的块必须落在**堆的空闲内存**中；**不能移动已分配块**（禁止压缩 compaction）；只能读写空闲块	移动块会使所有已发出的指针失效——C 没有 GC 的重定位能力

- **附加要求：对齐**。必须满足所有对齐约束：讲义口径是 x86-64 上 **16 字节**；**Malloc Lab 放宽到 8 字节**（与 libc `malloc` 一致），`mdriver` 会强制检查。
- **两个目标（常互相冲突）**：

目标	定义	度量
吞吐率（throughput）	单位时间内**完成的请求数**（`malloc` 与 `free` 都算）	例：10 秒内 5000 次 `malloc` + 5000 次 `free` → 1000 ops/s
峰值内存利用率（peak memory utilization）	让堆尽量小地装下程序的峰值载荷	见 A.9.3

A.9.3 碎片与利用率

利用率与开销（讲义定义）：设第 k 次请求后

$$P_k = \sum_{\text{当前已分配块}} \text{payload}, \qquad H_k = \text{当前堆大小（只增不减）}, \qquad \max_{i \le k} P_i = \text{峰值聚合载荷}$$

$$U_k = \frac{\sum \text{已分配载荷}}{\text{堆大小}} = \frac{P_k}{H_k}, \qquad O_k = \frac{H_k}{\max_{i \le k} P_i} - 1.0$$

- U_k 越接近 1 越好；O_k 是"堆里没被程序数据用掉的比例"，完美分配器趋近于 0。
- 讲义 benchmark `syn-array-short`（10 个块的分配/释放序列）峰值载荷为 **90036 字节**，用于画出 P_k 与 $\max_{i \le k} P_i$ 的曲线。

碎片类型	定义	公式	可测性
内部碎片（internal fragmentation）	**某个块内**载荷小于块大小	$\text{frag}_{\text{int}} = asize - p$，其中 p 为请求载荷、`asize` 为实际块大小（含 header/footer 与对齐 padding）	**容易**：只依赖**过去**的请求模式
外部碎片（external fragmentation）	堆里**总空闲量足够**，但**没有单个空闲块**够大	无解析公式；用"为了满足请求而额外扩大的堆"来度量	**困难**：依赖**未来**请求模式

- 内部碎片的三个来源：① 维护堆数据结构（header/footer）的开销；② 对齐 padding；③ **显式策略**（例如"小请求也给一个大块"）。
- 讲义 benchmark 数值：完美适配内部碎片开销 **1.6%**；外部碎片叠加后最佳适配 **8.3%**、首次适配 **11.9%**、下次适配 **21.6%**。

A.9.4 块格式与边界标记（boundary tag）

块布局（`asize` = 块总大小，含 header 与 footer）：

```
   low addr                                                  high addr
   +---------------------+--------------------------+---------------------+
   | header: size | a    | payload (app data)       | footer: size | a    |
   | 1 word = 8 bytes    | asize - 16 bytes         | 1 word = 8 bytes    |
   +---------------------+--------------------------+---------------------+
   ^                     ^                          ^
 hdr                bp = hdr + 8                hdr + asize - 8
```

size 字的位域：

```
   63                      3   2   1   0
   +-------------------------+---+---+---+
   |       block size        | 0 | P | A |
   +-------------------------+---+---+---+
                               A = 本块已分配（cur allocated）
                               P = 前一块已分配（prev allocated）
```

字段	编码内容	说明
header（1 word）	块大小 ‖ 分配位	大小**含 header 与 footer**；读取时必须**先屏蔽低位**（`size & ~0x7`）
footer（1 word）	**header 的副本**（边界标记）	让分配器能**反向遍历**；仅"需要合并"的实现才加，代价是每块多 1 word
payload	应用数据	仅已分配块有；返回给应用的指针是 `hdr + 8`（跳过 header）

为什么低 3–4 位可以复用为标志位：

- 块大小**必然是 8 的倍数**（因为所有块至少 8 字节对齐、且大小按 word 向上取整），所以 size 的**低 3 位恒为 0**；当块按 **16 字节对齐**时**低 4 位也恒为 0**。
- 既然这些位永远是 0，就不必"存一个恒为 0 的位"，直接**借用来当标志**。读取大小前 `& ~0x7` 即可。

- 讲义在"合并四情况"里实际用**低 2 位**：`P<<1 | A`（bit1 = 前块是否已分配，bit0 = 本块是否已分配）——有了 bit1，判断前块状态**不用**再去读前块 header，合并变成真正的 O(1)。
- Malloc Lab 用 16 字节对齐时可用低 4 位，多出的位可放"块是否来自 `mmap`"等标记。

边界标记为何给出 O(1) 合并：

```
prev_hdr          hdr (= cur_hdr)                next_hdr = hdr + asize
+---------+---------+---------+-------------------+---------+---------+
|  ...    | footer | header |   payload ...      | header |  ...    |
+---------+---------+---------+-------------------+---------+---------+
          ^                                      ^
   prev_footer = hdr - 8                 读 next_hdr 的 size 得下一块大小
   -> 读出 prev_size -> prev_hdr = hdr - prev_size
```

1. **找后块**：`next_hdr = hdr + asize`（当前块大小从自己的 header 读）。
2. **找前块**：`prev_size = *(word_t *)(hdr - 8)`（前块的 **footer** 就是当前块 header 前 8 字节），于是 `prev_hdr = hdr - prev_size`。
3. 前后块的大小、分配状态全部**O(1)** 拿到，不需遍历链表——这是"边界标记 + 立即合并"能做到常数时间释放的全部秘密。
4. 讲义另加两个哨兵：**堆起始处的 dummy footer**（标记为已分配，防止释放首块时误合并）与**堆结束前的 dummy header**（防止释放末块时越界合并）。

堆布局示例（隐式空闲链表，标注"大小/分配位"，单位 = word)：

```
heap_start                                                  heap_end
+--------+--------+--------+--------+--------+--------+-------------+
| 16/0   | 32/1   | 32/1   | 64/0   |  ...   |  8/1   | dummy hdr   |
| free   | alloc  | alloc  | free   |        | epilog | (allocated) |
+--------+--------+--------+--------+--------+--------+-------------+
  16     32      32      64      ...      8       8  (bytes)
```

- **header 落在非对齐位置，payload 才是对齐的**——这正是"低 3 位可当标志"的物理来源。

A.9.5 四种分配器对比

分配器	分配时间	释放时间	合并时间	额外内存开销	备注
隐式空闲链表（implicit free list）	$O(n)$，n = **全部**块数（线性最坏）	$O(1)$（**含**合并，靠边界标记）	$O(1)$	1 word/块（+1 word footer 若需反向合并）	实现最简单；因线性分配**实践中不用于 `malloc` / `free`**；"分割 + 边界标记合并"是通用技术
显式空闲链表（explicit free list）	$O(n_{\text{free}})$，线性于**空闲块**数	$O(1)$（LIFO/FIFO 插入）或 $O(n_{\text{free}})$（地址序插入）	$O(1)$	**2 word/块**（`next` / `prev` 指针）+ header/footer	内存越满越快；分配/释放需在链表中 splice 摘挂；指针只能存在**空闲块的 payload 区**
分离适配（segregated free list）	先查对应大小类，命中则 $O(1)$；否则逐类扩大搜索，约 $O(\log)$（2 的幂类）	$O(1)$（合并后插入对应类）	$O(1)$	指针 + **每个大小类一个链表头**的数组	首次适配某个类 ≈ 全局**最佳适配**的近似（极端情形：每块一个类 = 精确最佳适配）；吞吐率与利用率双赢
简单分离存储（simple segregated storage）	$O(1)$（从非空类直接取**整块，不分割**）	$O(1)$（整块插回原类）	**不做合并**	仅每类一个链表头；块内**无 header/footer**	时间与开销最优，但**内部碎片严重**、外部碎片不存在；只适合"块大小单一化"的场景

- 显式链表的**插入策略**：`LIFO`（插到表头）与 `FIFO`（插到表尾）都简单且常数时间，但研究显示**碎片比地址序更差**；**地址序**（保持 `addr(prev) < addr(curr) < addr(next)`）碎片更低，代价是插入需**搜索**。
- 分离适配的分配流程：① 在合适类中做首次适配，找到就**分割**并把余料放入合适类；② 该类找不到就**试下一个更大的类**，直到找到；③ 全都找不到才向 OS 申请新堆内存（`sbrk`），从新内存切出 `n` 字节，余料作为一个空闲块放进合适类。

A.9.6 合并的四种情况（立即合并，含边界标记）

设被释放块大小为 n，前块大小 m_1、后块大小 m_2：

情况	前块状态	后块状态	操作	新块大小	header 低 2 位处理
Case 1	已分配	已分配	仅把当前块标记为空闲，不合并	n	当前块 `A=0`；后块 `P=0`
Case 2	已分配	**空闲**	与**后块**合并	$n + m_2$	写新 header/footer；**再后一块的 `P` 位清 0**
Case 3	**空闲**	已分配	与**前块**合并	$m_1 + n$	改写前块 header/footer；后块 `P` 保持 0

情况	前块状态	后块状态	操作	新块大小	header 低 2 位处理
Case 4	空闲	空闲	三块全部合并	$m_1 + n + m_2$	改写前块 header/footer；再后一块 `P` 清 0

- 讲义记法：Case 2 为 `n+m2`，Case 3 为 `m1+n`，Case 4 为 `m1+m2+n`；四种情况**都只需常数时间**。
- 讲义给出的位变化记法：`?0`（前块状态未知/空闲）、`n 01`（本块空闲且前块已分配）、`m2 10`（后块空闲且前块已分配）等，本质就是维护 `P<<1 | A`。
- **合并策略**：**立即合并**（每次 `free` 就合并，简单、碎片少）vs **延迟合并**（攒着直到 `malloc` 找不到合适块时才合并，提升 `free` 吞吐率）。

A.9.7 适配策略对比

策略	搜索方式	复杂度	碎片倾向（讲义 benchmark 总开销）	备注
首次适配（first fit）	从表头开始，选**第一个够大**的块	$O(n)$	11.9%	实现最简单；会在表头附近产生"**splinters**"（小残渣）
下次适配（next fit）	像首次适配，但**从上一次搜索结束处**继续	$O(n)$ 但常更快	**21.6%（最差）**	避免反复扫描无用的头部块；但研究显示**碎片比首次适配更差**
最佳适配（best fit）	**全表扫描**，选剩余空隙**最小**的块	$O(n)$，常数更大	**8.3%（最好）**	保持碎片小、利用率高；通常比首次适配**更慢**；仍是**贪心，不保证最优**
（参照）完美适配	不存在的理想分配器	—	1.6%	只作基准线

- **记忆口诀**：**首次快、下次更差、最佳更省**。分离适配的"类内首次适配"可以**同时**拿到接近最佳适配的利用率与常数级吞吐率，这是它成为工业实现主流的原因。
- 三类策略选择之外还有两个正交策略：**分割策略**（何时把空闲块切开——直接决定能容忍多少内部碎片）与**合并策略**（立即/延迟）。

A.9.8 分离适配的大小类划分

2 的幂划分示例（类 k 覆盖 $[2^{k-2}+1,\ 2^{k-1}]$，类内统一按上界取整）：

类号	覆盖范围（字节）	类内块大小	最大内部碎片
1	$\{1\}$	1	0
2	$\{2\}$	2	0
3	$\{3–4\}$	4	1
4	$\{5–8\}$	8	3
5	$\{9–16\}$	16	7
…	…	…	…
11	$\{1025–2048\}$	2048	1023
12	$\{2049–4096\}$	4096	2047
13	$\{4097–\infty\}$	**无上限**（实际受 2^{64} 约束）	—

- 讲义的两条常见选择：① **小尺寸一档一块**（16、32、48、64…，类内无内部碎片）；② 到某个点后**切换到 2 的幂** $[2^i+1,\ 2^{i+1}]$。
- **最大块那一类必须没有上界**，否则超大请求无处安放。
- 大小类如何划分"对利用率和吞吐率都有重大影响"，是**设计决策**而非定式。

A.9.9 `realloc` 的实现路径

路径	触发条件	操作	代价
① **原地扩展**	当前块后面的邻居**空闲**且"当前块 + 后块"足够大（可选再合并更远的空闲块）	吞并后块，改 header/footer 的 size，**返回原指针**	$O(1)$，**没有数据搬运**——最优路径
② **搬移拷贝**	后面不够，或需要缩小但选择重排	`newp = mm_malloc(size)` → `memcpy(newp, oldp, min(oldsize, size))` → `mm_free(oldp)` → 返回 `newp`	$O(\text{size})$ 拷贝 + 分配/释放开销
③ **原地缩小**	新尺寸**小于**当前块，且切出来的余料 ≥ 最小块	原地改 size，把余料作为空闲块插入相应大小类（**可立即合并**）	$O(1)$，**指针不变**

- Malloc Lab 的关键提分点：先能用 `mm_malloc` + `mm_free` **搭出** `realloc`（保证正确性），**再**改写成独立的 `realloc`（原地扩展 + 原地缩小），因为最后 2 个 trace 全是 `realloc`，吞吐率差距明显。
- 语义红线：`realloc` 失败时**必须**返回 `NULL` 且**不动原块**；返回的新块同样要满足**8 字节对齐**。

A.9.10 glibc `malloc` 要点

> 以下均为**补充说明**（F25 讲义未涉及 glibc 实现细节），用于对照 Malloc Lab 与真实调试。

主题	要点
双路径	小请求走**主堆（main arena）**，用 `brk` / `sbrk` 推高堆顶；大请求走 **`mmap`**（`MAP_PRIVATE | MAP_ANONYMOUS`）建立独立映射，`free` 时用 `munmap` 直接还给内核
`MMAP_THRESHOLD`	默认 **128 KB = 131072 字节**：单次请求 **≥ 该值**时走 `mmap`。glibc 还会在释放 mmap 块时**动态上调**该阈值（64 位上限 `DEFAULT_MMAP_THRESHOLD_MAX = 32 MB`）
`M_TRIM_THRESHOLD`	默认 **128 KB**：堆顶（top chunk）空闲量超过它时用 `brk` **收缩堆**、把内存还给 OS
`mallopt(param, value)`	运行时调参。常用参数：`M_TRIM_THRESHOLD`、`M_MMAP_THRESHOLD`、`M_MMAP_MAX`、`M_TOP_PAD`（每次 `sbrk` 多要的余量）、`M_ARENA_MAX`（多线程 arena 数上限）
`mallinfo2()`	返回 `struct mallinfo2`，字段：`arena`（主堆总字节）、`ordblks`（空闲块数）、`hblks` / `hblkhd`（mmap 块数与字节）、`uordblks`（**已分配**字节）、`fordblks`（**空闲**字节）、`keepcost`（可 trim 的堆顶空闲）。**旧版 `mallinfo()` 用 `int`，已废弃**
`malloc_stats()`	直接把上述统计打到 `stderr`（无需返回值）
`malloc_usable_size(p)`	返回 `p` 实际可安全使用的字节数，**≥ 请求值**（原因：向上取整到 16 字节、且"下一块的 `prev_size` 字段"可被本块复用）——这正是内部碎片的一部分
`malloc_trim(0)`	主动触发一次堆顶归还，常用于"长期运行进程做完一批大任务后释放内存"

- **arena 与竞争**：多线程下 glibc 会为每个线程分配独立 **arena**，减少锁竞争；这也是为什么"多线程 malloc"看起来并没有一个全局锁那么慢。
- **`tcache`**：glibc 2.26+ 每个线程有一组 **tcache**（每类最多 7 个块）的快速缓存，`malloc` / `free` 的**绝大多数**操作在此完成、**几乎无锁**——这解释了下面那些错误信息里的 `in tcache 2`。

A.9.11 glibc 常见错误信息含义

> **补充说明**（讲义未涉及；文本随 glibc 版本略有差异）。这些都是**堆元数据被破坏**的信号，不是"分配器不可靠"。

错误信息	直接含义	典型根因	定位手段
`free(): double free detected in tcache 2`	同一指针被 `free` **两次**（tcache 层检测到该块已在缓存中）	双重释放；两条释放路径（如错误处理与正常路径）都 `free` 了同一块；或指针被复制后各自 `free`	`valgrind`（报 `Invalid free()` / `double free`）、或用 `MALLOC_CHECK_=3` 使程序立即 abort 并保留 core
`free(): double free detected in fasttop`	同一指针**连续两次**释放（fastbin 的 `fasttop` 检测）	同上，且两次 `free` 紧邻	同上
`malloc(): corrupted top size`	**top chunk（堆顶块）的 size 字段被改写**	**写越界**：往某块 payload 里写了超过块大小的字节，覆盖了**下一块的 header**；把 top chunk 的 size 写成非法值	`valgrind --leak-check=full`、`gdb` + 在 `malloc` 上设断点看 `top` 与实际请求大小；`x/16gx <p-16>` 观察相邻 header
`munmap_chunk(): invalid pointer`	该指针被判定为 **mmap 块**，但**不是 mmap 块的首地址**（或压根不是 mmap 块）	`free` 了栈/全局数组地址、`free` 了 `p + k`（指向块中间）、或 `free` 了已经交给别处的指针	`valgrind`、检查所有 `free` 的参数是否都来自 `malloc` 的**原始返回值**
`free(): invalid pointer`	`free` 的地址**不是**合法块首（未对齐、或不属于堆）	`free` 栈地址 / 全局数组地址 / 结构体字段地址	`valgrind`、`gdb` 打在 `free` 上看调用栈
`free(): invalid next size (normal/fast)`	相邻块的 `size` 字段被破坏	越界写、`memcpy` 长度算错（经典的 `strlen` 忘加 `'\0'`）	`valgrind`、仔细核对每个 `memcpy` / `strcpy` 的长度
`corrupted double-linked list`	unsorted/small bin 的**双向链表指针**被破坏	写越界覆盖了某个**空闲块**的 `fd` / `bk` 指针	`valgrind`、heap checker

- **最实用的一招**：编译时不要"绕过"分配器检查——`MALLOC_CHECK_=3 ./prog`、`MALLOC_PERTURB_=165 ./prog`（每次分配填垃圾值，暴露"读了未初始化内存"）、以及 `valgrind`。Malloc Lab 里则完全靠**自己写的 `mm_check` 堆一致性检查器**。

A.9.12 Malloc Lab 提分点清单与 `mdriver` 用法

评分公式（官方 writeup：Correctness 20 + Performance 35 + Style 10）：

$$P = wU + (1-w)\min\left(1,\ \frac{T}{T_{\text{libc}}}\right), \qquad w = 0.6, \quad T_{\text{libc}} = 600\ \text{Kops/s}$$

项	权重	关键点
Correctness	20 分	逐 trace 给分；**崩溃或违规得 0**
Space utilization U	占 $w = 0.6$	峰值"驱动器已分配字节 / 堆大小"，最优为 1；**利用率权重高于吞吐率**
Throughput T	占 $1 - w = 0.4$	平均每秒完成的操作数；超过 libc 后**封顶**（`min(1, T/T_libc)`），所以**不要**为吞吐率牺牲利用率
Style	10 分	5 分给**堆一致性检查器** `mm_check`，5 分给**函数分解与注释**（顶部必须写明块结构、空闲链表组织、链表操作方式）

评分规则红线：

- 不得修改 `mm.c` 里的四个接口签名（`mm_init` / `mm_malloc` / `mm_free` / `mm_realloc`）。
- 不得调用任何内存管理相关库函数/系统调用：**禁止** `malloc`、`calloc`、`free`、`realloc`、`sbrk`、`brk` 及其变体。
- 不得定义任何**全局或静态复合数据结构**（数组、结构体、树、链表）；只允许全局**标量**（整数、浮点、指针）。空闲链表必须**构造在堆内存里**。
- `mm_init` 出错返回 `-1`，否则 `0`；`mm_malloc` 必须返回 **8 字节对齐**的指针（`mdriver` 强制检查）。

提分路线（按投入产出排序）：

阶段	做法	收益
1	**隐式空闲链表 + 边界标记 + 立即合并 + 首次适配**	正确性达标，利用率一般，吞吐率差
2	改为 **显式空闲链表**（`next` / `prev` 放空闲块 payload 里），插入用 LIFO 或地址序	吞吐率大幅提升（只遍历空闲块）
3	改为 **分离空闲链表**（2 的幂大小类 + 类内首次适配），升级为常数时间	利用率提升到接近最佳适配，吞吐率再上一档
4	**独立** `mm_realloc`：原地扩展 + 原地缩小 + 只在必要时搬移	最后 2 个（`realloc`）trace 的吞吐率
5	细节优化：**合并时用小块优先/** `prev` **位判断**、分割阈值、类内按大小**有序插入**、`mm_check` 写扎实	利用率尾数 + Style 10 分
6	**主动降低每块开销**：16 字节对齐可把低 4 位当标志；小块省掉 footer（只给空闲块加 footer）	直接换利用率

`mdriver` 命令行（`-h` 可查全部）：

选项	作用
`./mdriver -V`	**最重要**：最详细输出，处理每个 trace 文件时打印诊断信息，便于定位**哪个 trace 出错**
`./mdriver -v`	每个 trace 打印一行紧凑的性能明细表（利用率、吞吐率）
`./mdriver -f <tracefile>`	只跑**指定单个 trace**（开发期用 `short1-bal.rep`、`short2-bal.rep` 这类微型 trace 极省时间）
`./mdriver -t <tracedir>`	到指定目录找 trace 文件（默认目录在 `config.h` 里）
`./mdriver -l`	**额外**测量并打印 libc `malloc` 的性能，作为对照
`./mdriver -h`	打印所有命令行参数
`gprof ./mdriver`	官方提示可用它定位热点函数

- **官方 hint 摘要**：先用 `-f` 跑小 trace 建立正确性；**前 9 个 trace 只含** `malloc` **/** `free`**，最后 2 个含** `realloc` ——先把前者做对再做 `realloc`；**用宏封装指针运算**（分配器里的指针算术充满强制类型转换，是 bug 温床）；**先完全读懂教材的隐式链表例子再动手**；用 `gcc -g` + 调试器定位越界引用。

A.10 进程与信号速查

> 对应讲次：Lecture 16（`F25-16-processes.txt`）、Lecture 17（`F25-17-ecf.txt`）； 教材 CS:APP3e 第 8 章（8.1—8.8）；关联 **L6 Shell Lab**。

A.10.1 异常四分类

类别	原因	同步/异步	返回行为	典型例子
中断（Interrupt）	来自**处理器外部**的事件，通过置位**中断引脚**通知 CPU	**异步**	返回到 I_{next}（下一条指令）	定时器中断（每几毫秒一次）、I/O 中断、Ctrl-C 引起的信号、网络包到达
陷阱（Trap）	**有意为之**的指令（程序主动陷入内核）	**同步**	返回到 I_{next}	系统调用（`syscall` / `int 0x80`）、`gdb` 断点
故障（Fault）	指令执行引起的**意外**，可能可恢复	**同步**	**重新执行** I_{current}，或无法恢复时终止进程	**缺页**（page fault，可恢复）、保护故障、浮点异常、除法错误
终止（Abort）	意外且**不可恢复**的致命错误	**同步**	**不返回**，直接终止当前程序	非法指令、奇偶校验错、机器检查（machine check）

- **机械记忆：中断异步并跳过、陷阱同步并跳过、故障同步并重来、终止同步并结束。**
- **异常号（讲义 Figure 8.9 口径）**：0 = 除法错误，13 = 一般保护故障，14 = 缺页，18 = 机器检查；32–255 由操作系统定义（Linux 用 128 = `0x80` 作系统调用入口）。
- **故障两种结局的典范**：缺页 → 内核调页 → **重新执行同一条** `movl`；非法地址 → 无法恢复 → 内核向进程发 `SIGSEGV` → "segmentation fault" 退出。**同一机制，两种归宿。**
- **异常表 / 中断向量**：内核启动时把每个异常号对应的处理程序地址填进**异常表（exception table，又称中断向量 interrupt vector）**，硬件按异常号查表跳转。

A.10.2 进程控制接口

接口	原型	语义要点
`fork`	`pid_t fork(void)`	**调用一次、返回两次**：父进程得到**子进程** PID（>0），子进程得到 0；出错返回 -1（并置 `errno`）。子进程获得父进程地址空间的**副本**（实现上用**写时复制 COW**）、文件描述符表的**副本**（指向**同一**打开文件表项 → **共享文件偏移量**）、但**独立的 PID**。父子**并发**执行，输出顺序不确定
`exit`	`void exit(int status)`	终止进程；`status` 的低 8 位可由父进程用 `WEXITSTATUS` 读出。会执行 `atexit` 注册的函数与 stdio 清理（**非异步信号安全**）
`_exit`	`void _exit(int status)`	立即终止，**跳过**清理——**唯一**信号处理程序中可用的终止方式（异步信号安全）
`wait`	`pid_t wait(int *statusp)`	等价于 `waitpid(-1, statusp, 0)`；**已废弃**，新代码用 `waitpid`
`waitpid`	`pid_t waitpid(pid_t pid, int *statusp, int options)`	回收子进程并取回状态；支持非阻塞与作业控制（详见 A.10.3）
`execve`	`int execve(const char *filename, char *const argv[], char *const envp[])`	**加载并运行**新程序：覆盖当前进程的代码/数据/栈，**PID 不变**，**保留打开的文件描述符**（除非标了 `O_CLOEXEC`）。**成功不返回**；失败返回 -1 并置 `errno`——**只有失败路径会执行 `execve` 之后的代码**
`getpid`	`pid_t getpid(void)`	返回调用进程的 PID
`getppid`	`pid_t getppid(void)`	返回调用进程**父进程**的 PID
`sleep`	`unsigned int sleep(unsigned int secs)`	挂起 `secs` 秒；**返回尚未睡满的剩余秒数**（被信号打断时非 0）
`pause`	`int pause(void)`	挂起直到收到信号；**总是**返回 -1 且 `errno = EINTR`

- **`fork` 的经典坑**：`fork` 后父子共享终端输出，若在 `fork` **之前**有未 `fflush` 的 stdio 缓冲，父子各会输出一份 → 同一行打印两遍。**修复**：`fork` 前 `fflush(stdout)`，或改用 `write`。
- **僵尸进程**（zombie）：子进程已终止但父进程尚未 `wait`，内核保留其退出状态 → 父进程必须回收。父进程先死则子进程被 `init` / `pid 1` 收养。

A.10.3 `waitpid` 选项与 `status` 宏完整表

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *statusp, int options);
```

`pid` 参数取值：

`pid`	含义
`pid > 0`	只等待**进程 ID 为 `pid`** 的那**一个**子进程

`pid`	含义
`pid == -1`	等待**任意一个**子进程（最常用）
`pid == 0`	等待与调用者**同一进程组**中的任意子进程
`pid < -1`	等待**进程组** `|pid|` 中的任意子进程（作业控制里配合 `setpgid` 使用）

`options` **参数**（可用 `|` 组合）：

选项	含义
`0`	默认：**阻塞**，直到 `pid` 指定的某个子进程**终止**
`WNOHANG`	**非阻塞**：若无已终止的子进程可回收，**立即返回 0**（返回 `0` 不是错误）
`WUNTRACED`	除终止外，还报告**已停止（stopped）**的子进程（`SIGTSTP` / `SIGSTOP` 之后）
`WCONTINUED`	还报告收到 `SIGCONT` **恢复执行**（continued）的子进程

`statusp` **判定宏（必须先判类别、再取具体值，顺序不可颠倒）**：

宏	判定/取值	返回值语义
`WIFEXITED(status)`	**是否正常终止**	非 0 表示通过 `exit` / `return` 正常结束
`WEXITSTATUS(status)`	正常终止时的**退出状态**	`exit` 参数的**低 8 位**；**仅在** `WIFEXITED` 为真时有效
`WIFSIGNALED(status)`	**是否因未捕获的信号而终止**	非 0 表示为信号所杀
`WTERMSIG(status)`	**导致终止的信号编号**	如 `2`（`SIGINT`）、`9`（`SIGKILL`）、`11`（`SIGSEGV`）；仅在 `WIFSIGNALED` 为真时有效
`WIFSTOPPED(status)`	**是否处于停止状态**	非 0 表示已停止（需 `WUNTRACED` 才会被报告）
`WSTOPSIG(status)`	**导致停止的信号编号**	如 `20`（`SIGTSTP`）、`19`（`SIGSTOP`）；仅在 `WIFSTOPPED` 为真时有效
`WIFCONTINUED(status)`	**是否已由 `SIGCONT` 恢复**	需 `WCONTINUED` 才会被报告

- **返回值**：成功返回被回收子进程的 PID；`WNOHANG` 且无子进程可回收时返回 **0**；出错返回 **−1**（常见 `errno = ECHILD`，表示没有可等待的子进程——SIGCHLD 处理程序里用 `while ((pid = waitpid(-1, &st, WNOHANG|WUNTRACED)) > 0)`，靠返回 `-1` / `0` 退出循环）。
- **Shell Lab 的官方推荐用法**：`waitfg` 里用围绕 `sleep` 的**忙循环**，`sigchld` 处理程序里**恰好调用一次** `waitpid`；不要在 `waitfg` 里也调 `waitpid`（"虽然可行，但极易混乱"）。

A.10.4 `execve` 家族表

函数	参数形式	搜索 `PATH`	环境变量	备注
`execl`	**列表**：`execl(path, arg0, ..., NULL)`	否	继承 `environ`	`l` = list
`execv`	**数组**：`execv(path, argv[])`	否	继承 `environ`	`v` = vector
`execlp`	列表	**是**	继承 `environ`	`p` = 搜索 `PATH`
`execvp`	数组：`execvp(file, argv[])`	**是**	继承 `environ`	最常用：`execvp(argv[0], argv)`
`execle`	列表：`execle(path, arg0, ..., NULL, envp[])`	否	**显式 `envp`**	`e` = environment
`execve`	数组：`execve(path, argv[], envp[])`	否	**显式 `envp`**	**唯一的系统调用**，其余都是 libc 包装

- **`execvp` 如何搜索 `PATH`**：若 `file` **不含** `/`，则按 `PATH` 环境变量（以 `:` 分隔的目录列表）**依次**在每个目录下查找可执行文件；找到即执行，全都找不到才返回 **−1**（`errno = ENOENT`）。若 `file` **含** `/`，则当作**路径**直接使用，**不搜索** `PATH`（这正是 Shell Lab 里 `./myspin` 与 `myspin` 行为不同的原因）。
- **`execve` 与 `argv` 约定**：`argv[0]` 按惯例是可执行文件名（**可任意**，程序无法强制校验），`argv` 必须以 `NULL` 结尾。
- **`execve` 之后的代码只在失败时执行**——Shell 中的标准写法是 `execve(...); fprintf(stderr, "%s: Command not found.\n", argv[0]); exit(0);`。

A.10.5 进程组与作业控制

接口/概念	说明
`getpgrp()`	返回**当前进程所属进程组**的 ID（PGID）

接口/概念	说明
`setpgid(pid, pgid)`	改变进程的进程组。**`setpgid(0, 0)`** 表示"把**我自己**放进一个**以我的 PID 为组 ID** 的新进程组"
进程组（process group）	一组进程的集合，**组 ID = 组长进程的 PID**；同一组的进程可用 **一个信号一起通知**
前台进程组	当前拥有终端的进程组——键盘信号只发给它
后台进程组	不拥有终端；读终端会收到 `SIGTTIN`（默认停止），写终端可能收到 `SIGTTOU`
`Ctrl-C`	内核向**前台进程组中的每一个进程**发送 **`SIGINT`** (2)，默认动作 = **终止**
`Ctrl-Z`	内核向**前台进程组中的每一个进程**发送 **`SIGTSTP`** (20)，默认动作 = **停止（挂起）**，直到收到 `SIGCONT` 才继续
`/bin/kill` 程序	`kill -9 24818` 发给单个进程；**`kill -9 -24817`** 发给**进程组 24817 的所有进程**（注意负号）
Shell 的 `fg` / `bg`	`bg` 给作业发 `SIGCONT` 并让它在**后台**运行；`fg` 给作业发 `SIGCONT`（若已停止）并把它放到**前台**，等待其终止或停止

`kill(pid, sig)` 的 `pid` 取值含义：

`pid` 取值	信号发往
`pid > 0`	只发给**进程 `pid`**
`pid == 0`	发给**调用者所在进程组**的每个进程（**含调用者自己**）
`pid < 0`	发给**进程组 `|pid|`** 中的每个进程
`pid == -1`	发给调用者**有权发送信号的所有进程**（危险）

Shell Lab 的作业控制要点：

- 子进程在 `fork` 之后、`execve` 之前必须调用 **`setpgid(0, 0)`**，否则子进程默认留在 Shell 的进程组里——于是 `Ctrl-C` 会**同时**打到 Shell 自己和它创建的所有进程，这显然是错的。
- Shell 捕获 `SIGINT` / `SIGTSTP` 后，必须用 **`kill(-pid, SIGINT)`**（**负的 `pid`**）转发给**整个前台进程组**；`sdriver.pl` 会专门测试你有没有用 `-pid`。
- **必须**在 `fork` 前用 `sigprocmask` 屏蔽 `SIGCHLD`，在 `addjob` 之后解除屏蔽，否则会出现"子进程已被 `sigchld` 处理程序回收并从作业表删除，父进程却还没 `addjob`"的**竞态**。由于子进程继承父进程的屏蔽字，**子进程在 `execve` 前必须解除 `SIGCHLD` 屏蔽**。

A.10.6 常用信号表（编号 / 默认动作 / 可否捕获或阻塞）

ID	名称	默认动作	对应事件	可否捕获/忽略/阻塞
1	`SIGHUP`	Terminate	终端挂断	可
2	**`SIGINT`**	Terminate	用户按 `Ctrl-C`	可
3	**`SIGQUIT`**	Terminate + core	用户按 `Ctrl-\`	可
4	**`SIGILL`**	Terminate + core	非法指令	可*
5	`SIGTRAP`	Terminate + core	断点 / 陷阱	可*
6	**`SIGABRT`**	Terminate + core	`abort()`	可
7	`SIGBUS`	Terminate + core	总线错误（未对齐/映射错）	可*
8	**`SIGFPE`**	Terminate + core	除零 / 算术错误	可*
9	**`SIGKILL`**	Terminate	强杀（管理员最后手段）	**不可**（`sigaction` 返回 `EINVAL`）
10	`SIGUSR1`	Terminate	用户自定义	可
11	**`SIGSEGV`**	Terminate + core	段错误（非法内存访问）	可*
12	`SIGUSR2`	Terminate	用户自定义	可
13	**`SIGPIPE`**	Terminate	向**已关闭的管道/socket 写入**	可
14	**`SIGALRM`**	Terminate	`alarm()` 定时器到期	可
15	**`SIGTERM`**	Terminate	温和的终止请求（`kill` 默认）	可
16	`SIGSTKFLT`	Terminate	协处理器栈错误	可

ID	名称	默认动作	对应事件	可否捕获/忽略/阻塞
17	**`SIGCHLD`**	Ignore	子进程**停止或终止**	可
18	**`SIGCONT`**	Continue	恢复被停止的进程	可
19	**`SIGSTOP`**	Stop	强制停止	**不可**（`EINVAL`）
20	**`SIGTSTP`**	Stop	用户按 `Ctrl-Z`	可
21	**`SIGTTIN`**	Stop	**后台**进程读终端	可
22	**`SIGTTOU`**	Stop	**后台**进程写终端	可
23	`SIGURG`	Ignore	socket 上有紧急数据	可
24	`SIGXCPU`	Terminate + core	CPU 时间超限	可
25	`SIGXFSZ`	Terminate + core	文件大小超限	可
26	`SIGVTALRM`	Terminate	虚拟定时器	可
27	`SIGPROF`	Terminate	性能剖析定时器	可
28	`SIGWINCH`	Ignore	终端窗口大小改变	可
29	`SIGIO`	Terminate	异步 I/O 就绪	可
30	`SIGPWR`	Terminate	电源故障	可

* 带 `*` 的信号**只能被"其他进程发来的"实例阻塞**；由当前指令直接触发的同步异常（如自己产生 `SIGSEGV` / `SIGILL`）无法靠屏蔽躲开。

- 信号类型用**小整数 ID（1–30）**标识；信号里**唯一的载体就是 ID** 和"它到了"这个事实，**不携带数据**。
- **`SIGKILL`(9) 与 `SIGSTOP`(19) 是唯二**不可捕获、不可忽略、不可阻塞的信号。

A.10.7 `sigaction` 结构与 `sa_flags`

```
#include <signal.h>
int sigaction(int signum, struct sigaction *act, struct sigaction *oldact);
/* 结构字段：sa_handler / sa_sigaction、sa_mask、sa_flags */
```

字段	作用
`sa_handler`	单参数处理程序 `void (*)(int)`；或 `SIG_IGN`（忽略）、`SIG_DFL`（默认）
`sa_sigaction`	三参数处理程序 `void (*)(int, siginfo_t *, void *)`，当 `sa_flags & SA_SIGINFO` 时使用（与 `sa_handler` 是**联合体**，只能设一个）
`sa_mask`	处理程序**运行期间额外屏蔽**的信号集合（**隐式**地，正在处理的信号自身总会被屏蔽）
`sa_flags`	行为开关，见下表

`sa_flags` 标志	含义
`SA_RESTART`	自动**重启**被该信号打断的**慢速系统调用**（`read`、`wait` 等）——不设它就要自己处理 `EINTR`
`SA_NODEFER`	处理程序运行期间**不**自动屏蔽自身（默认是屏蔽的）
`SA_RESETHAND`	进入处理程序时把该信号的动作**重置为 `SIG_DFL`**（只生效一次）
`SA_SIGINFO`	改用三参数 `sa_sigaction`，可取得 `siginfo_t` 与被打断的上下文
`SA_ONSTACK`	在备用信号栈（`sigaltstack`）上运行处理程序

- **`signal()` 的教训**：老接口 `signal()` 语义在不同系统上不一致（是否重启系统调用、是否重置），**Posix 标准推荐一律用 `sigaction`**。
- **隐式屏蔽**：内核在进入处理程序前，自动把"正在处理的信号类型"加入屏蔽集——所以 `SIGINT` 处理程序**不会被另一个 `SIGINT` 打断**（这正是"同种信号不会嵌套"的机制来源）。

A.10.8 阻塞与待处理信号

概念/接口	说明
待处理（pending）	信号**已发送但尚未被接收**。内核为每个进程维护 **pending 位向量**：发送时置位 k，接收时清位 k

概念/接口	说明
阻塞（blocked）	一个进程可以**阻塞**某些信号的接收。被阻塞的信号**可以发出**，但会**一直待处理**，直到解除阻塞才被接收
信号掩码（signal mask）	被阻塞信号的集合；用 `sigprocmask` 读写
关键事实：信号不排队	每种信号**最多只有一个待处理信号**。若进程已有类型 k 的待处理信号，则**后续同类型信号直接被丢弃**。故"待处理信号最多被接收一次"——**不要用信号来计数事件**（Shell Lab 里数"有几个子进程结束"必须用 `waitpid` 循环，而不是数信号次数）
`sigprocmask(how, set, oldset)`	读/改当前信号掩码。`how`：`SIG_BLOCK`（加入）、`SIG_UNBLOCK`（移除）、`SIG_SETMASK`（替换）
`sigemptyset(&set)`	把集合清空
`sigfillset(&set)`	把**所有**信号加入集合
`sigaddset(&set, sig)`	加入指定信号
`sigdelset(&set, sig)`	删除指定信号
`sigismember(&set, sig)`	判断某信号是否在集合中
`sigpending(&set)`	取回**当前待处理**（且被阻塞）的信号集合

暂时阻塞信号的惯用法（保护不可被打断的代码区）：

```
sigset_t mask, prev_mask;
sigemptyset(&mask);
sigaddset(&mask, SIGINT);
sigprocmask(SIG_BLOCK, &mask, &prev_mask);   /* 阻塞 SIGINT，并保存原掩码 */
    /* 这段区域不会被 SIGINT 打断 */
sigprocmask(SIG_SETMASK, &prev_mask, NULL);  /* 恢复原掩码 = 解除阻塞 */
```

- **恢复必须用 `SIG_SETMASK` + 保存的旧掩码**，而不是 `SIG_UNBLOCK`——否则会把调用者原本就屏蔽的信号**误解除**。

A.10.9 异步信号安全（async-signal-safe）函数

定义：函数是**异步信号安全**的，当且仅当它**要么可重入**（所有变量都在栈帧上），**要么不可被信号打断**。POSIX 保证 **117 个**函数满足该性质（`man 7 signal-safety`）。

✅ 安全（可在处理程序中调用）	❌ 不安全（严禁在处理程序中调用）
`write`（**唯一安全的输出函数**）	`printf` / `fprintf`（stdio 缓冲区非可重入）
`_exit` / `_Exit`	`exit`（会跑 `atexit` 与 stdio 清理）
`read`、`open`、`close`、`dup2`、`lseek`、`fcntl`	`malloc` / `free` / `calloc` / `realloc`（堆状态可能被撕裂）
`wait`、`waitpid`、`sleep`、`alarm`、`pause`	`sprintf` / `snprintf`（依赖 locale 与缓冲）
`kill`、`raise`、`sigqueue`	`strerror`、`getpwnam`、`gethostbyname`（静态缓冲区）
`sigprocmask`、`sigpending`、`sigsuspend`	`readdir`、`localtime`、`asctime`、`ctime`（静态缓冲区）
`sigaction`、`signal` 与 `sigemptyset` / `sigfillset` / `sigaddset` / `sigdelset` / `sigismember`	`setenv` / `putenv`（修改全局环境）
`fork`、`execve`、`setpgid`、`getpid`、`getpgrp`、`getuid`	`rand` / `srand`（全局种子状态）
`strlen`、`strcmp`、`strcpy`、`memcpy`、`memset`、`memmove`	stdio 的任何 `f*` 系列（`fgets` / `fputs` / `fflush` …）
`sem_post`、`sem_wait`（POSIX 列入清单）	`pthread_*` 中未列入清单者、`dlopen`、`syslog`
`longjmp` / `siglongjmp`（见注）	`setjmp`（**不在**清单里）

- **`write` 是唯一的异步信号安全输出函数**——所以信号处理程序里要打印信息，只能用 `write(STDOUT_FILENO, buf, n)` 或 CS:APP 提供的 **SIO 库**（`sio_puts`、`sio_putl`、`sio_error`、支持 `%c %s %d %u %x %%` 的 `sio_printf`，内部只用 `write`）。
- **`longjmp` 的限定**：自 POSIX.1-2008 TC2 起列入安全清单，但若处理程序打断的正是某个**非**异步信号安全函数，从处理程序中 `longjmp` 出去行为未定义。

A.10.10 信号处理五条准则

准则	内容	反面教材
G0（讲义补充）	**让处理程序尽可能简单**：最理想是"置一个全局标志然后返回"	在处理程序里做业务逻辑、加锁、遍历链表
G1	处理程序中**只调用异步信号安全函数**	`printf`、`sprintf`、`malloc`、`exit` **都不安全**；`free`、`strtok`、`gethostbyname`、`ctime` 同样不安全
G2	**进场保存、退场恢复 `errno`**	处理程序覆盖了主程序正要检查的 `errno`，导致主程序按错误原因走错分支
G3	访问**共享全局数据结构**时，**临时阻塞所有信号**保护	处理程序与主程序同时改同一个链表 → 结构损坏或丢失更新
G4	**全局变量声明为 `volatile`**	编译器把变量缓存在**寄存器**里，处理程序的修改"看不见"→ 死循环
G5	**全局标志声明为 `volatile sig_atomic_t`**	这类标志只做**读/写**（如 `flag = 1`，**不是** `flag++`），因此**不需要**像其他全局量那样加 G3 的保护

G1–G3 代码骨架：

```
volatile sig_atomic_t flag = 0;             /* G4 + G5 */

void handler(int sig)                       /* G0: 尽量简单 */
{
    int olderrno = errno;                   /* G2: 进场保存 errno */
    flag = 1;                               /* G5: 只做赋值，无需加锁 */
    write(STDOUT_FILENO, "caught\n", 7);    /* G1: 只用异步信号安全的 write */
    errno = olderrno;                       /* G2: 退场恢复 errno */
}
```

A.10.11 `setjmp` / `longjmp` 与信号版本

接口	原型/语义
`setjmp`	`int setjmp(jmp_buf env)`：把当前**寄存器上下文（含栈指针）**存入 `env`。**直接调用时返回 0**；从 `longjmp` 返回时返回 `longjmp` 的第二个参数
`longjmp`	`void longjmp(jmp_buf env, int retval)`：恢复 `env` 保存的上下文，效果是**掀掉中间所有栈帧**跳回 `setjmp` 处。`retval` **必须非 0**（传 0 会被实现改成 1）
`sigsetjmp`	`int sigsetjmp(sigjmp_buf env, int savesigs)`：`savesigs` 非 0 时**同时保存信号掩码**
`siglongjmp`	`void siglongjmp(sigjmp_buf env, int retval)`：跳回时**一并恢复信号掩码**——**从信号处理程序里跳出必须用这一对**，否则掩码会停留在"处理程序运行期间"的状态

两个必须记住的陷阱：

1. **`setjmp` 只能出现在 `if`、`switch`、循环条件或比较表达式里，不能**写成 `x = setjmp(env)`（标准把它限定在这几种"控制表达式"位置）。
2. **`volatile` 陷阱**：在 `setjmp` 与 `longjmp` 之间被修改的**局部变量**，若被编译器分配到**寄存器**，`longjmp` 恢复寄存器旧值后，这些变量会"回到过去"。**标准只保证 `volatile` 变量与全局变量正确**。因此处理程序里改过的标志、`longjmp` 之后要读的局部量，**一律加 `volatile`**。
3. **`longjmp` 不做清理**：中间栈帧里的 `malloc`、打开的 `fd` **不会被释放**（类比"撕掉书页，夹在里面的借书卡还在"）。C++ 的 `throw`/`catch` 会逐个调用析构函数，这是两者最本质的差别。

A.10.12 `sigsuspend` 与 `pause` 的区别

四种"等待一个信号"的写法对照（讲义原例）：

写法	评价	原因
`while (!pid) pause();`	❌ **有竞态**	在"检查 `pid` "与"进入 `pause`"之间信号可能到达，于是 `pause` 永远等不到 → **永久挂起**
`while (!pid) sleep(1);`	⚠️ **安全但慢**	最坏要多等 **1 秒**才响应
`while (!pid) ;`	❌ **忙等**	白烧 CPU
`sigprocmask(SIG_BLOCK, ...)` + `while (!pid) sigsuspend(&prev);`	✅ **正确**	改掩码与进入睡眠是**原子**的

```
int sigsuspend(const sigset_t *mask);
/* 语义：原子地（不可打断地）执行以下三步——
 *   1) 把信号掩码设为 *mask
 *   2) 挂起进程直到收到一个信号（且该信号的处理程序返回后）
 *   3) 把信号掩码恢复为调用前的值
 */
```

- **`pause()` 的问题**是"改状态"与"睡下去"之间有窗口；**`sigsuspend` 把这两步合并成一个不可分割的原子操作**，窗口消失。
- **典型用法**：先 `sigprocmask(SIG_BLOCK, &mask, &prev)` 屏蔽 `SIGCHLD`（防止信号在 `fork` 与"准备等待"之间丢失），`fork` 后 `while (!pid) sigsuspend(&prev);`——`sigsuspend` 用 `prev`（即**未屏蔽 `SIGCHLD`** 的掩码）替换当前掩码，从而"在等待期间允许 `SIGCHLD` 递达"。

A.10.13 进程与信号常用命令

命令	作用
`ps -o pid,ppid,pgid,stat,cmd`	看 PID / 父 PID / **进程组** / 状态（`Z` = 僵尸、`T` = 停止）/ 命令
`ps -efj`	含 PGID、SID 的全表
`kill -9 <pid>` / `kill -9 -<pgid>`	给单个进程 / **整个进程组**发信号
`pstree -p`	以树形显示父子关系
`strace -f -e trace=process,signal ./tsh`	跟踪 `fork` / `execve` / `wait4` /信号递达——**Shell Lab 的头号调试利器**
`strace -f -e trace=signal -p <pid>`	附着到运行中的进程看信号流
`gdb -p <pid>` + `handle SIGINT nostop print pass`	让 gdb 不拦截某些信号，观察程序自身行为
`trap -l`（shell 内建）	列出信号编号与名称对照
`ulimit -c unlimited`	允许 core dump（配合 `gdb ./prog core` 定位 `SIGSEGV` / `SIGABRT`）
`timeout -s INT 5 ./prog`	5 秒后发 `SIGINT`，实测处理程序行为
`---`	

A.11 系统级 I/O 速查

> 对应讲次：Lecture 18（`F25-18-system-io.txt`）；教材 CS:APP3e 第 10 章； 关联 **L6 Shell Lab**（`dup2` 重定向）与 **L7 Proxy Lab**（`rio` + `stat`）。

A.11.1 `open` 的 flags 与 `mode`

两种调用形式：

```
int open(const char *pathname, int flags);                 /* 打开已存在的文件 */
int open(const char *pathname, int flags, mode_t mode);    /* 打开或创建 */
/* 成功返回文件描述符（最小的未用非负整数）；出错返回 -1 并置 errno */
```

访问模式（`flags` 中必须恰好包含其中之一）：

标志	含义
`O_RDONLY`	只读
`O_WRONLY`	只写
`O_RDWR`	可读可写

可选标志（用 `|` 组合）：

标志	含义	典型用途
`O_CREAT`	文件不存在则**创建**（**必须**同时提供第三个参数 `mode`）	新建输出文件
`O_TRUNC`	若文件已存在，**先删除其全部内容**（截断为 0）	"覆盖写"
`O_APPEND`	每次 `write` **都写到文件末尾**（内核保证偏移量更新与写入是原子的）	日志追加

标志	含义	典型用途
`O_EXCL`	**与 `O_CREAT` 合用**：若文件**已存在则失败**（返回 `-1`，`errno = EEXIST`）	原子地"创建独占文件"、锁文件
`O_CLOEXEC`	**执行 `execve()` 时自动关闭**该描述符（FD_CLOEXEC 的原子版）	防止 fd 泄漏给子进程
`O_NONBLOCK`	非阻塞模式（对管道/FIFO/socket 有意义）	事件驱动 I/O
`O_SYNC` / `O_DSYNC`	写入同步落盘（绕过页缓存延迟）	数据库日志

第三个参数 `mode` 与 `umask`：

要点	说明
何时必须给	**只有** `flags` 中出现 `O_CREAT` 时才有意义；否则**被忽略**（C 里 `open` 是**变参函数**，靠这个技巧实现"两个或三个参数"）
`mode` 的含义	新建文件的**默认访问权限**位（如 `0644` = `rw-r--r--`）
实际权限	实际权限 $= mode \ \&\ \sim umask$，即被 **`umask` 掩掉一些位**。典型 `umask` 为 `022`，故 `0666 & ~022 = 0644`
建议取值	没有特殊理由时用 `DEFFILEMODE`（来自 `<sys/stat.h>`，典型值 `0666`），让 `umask` 决定最终权限
查看	`umask`（shell 内建命令，打印当前掩码）

实践模板（讲义原例 + 健壮性补充）：

```
#include <fcntl.h>
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    int fd;
    if ((fd = open("/etc/hosts", O_RDONLY)) < 0) {   /* 打开已有文件 */
        perror("open");
        exit(1);
    }
    /* ... 用 fd ... */
    close(fd);

    /* 创建（可读可写、截断、不存在则建） */
    int fd2 = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, DEFFILEMODE);
    if (fd2 < 0) { perror("open"); exit(1); }
    close(fd2);
    return 0;
}
```

A.11.2 `read` / `write` 返回值语义

```
ssize_t read(int fd, void *buf, size_t n);
ssize_t write(int fd, const void *buf, size_t n);
```

返回值	`read` 的含义	`write` 的含义
n'，$0 < n' < n$	**短计数（short count）**——正常现象，**不是错误**（见下）	**短计数**——通常表示写满了（管道/socket 缓冲满）或磁盘满，**必须继续写剩余部分**
$n' = n$	完全成功	完全成功
0	EOF（文件结束）：**仅 `read` 可能出现**，且**只有**在 `read` 返回 0 时才能判定 EOF	**`write` 返回 0 不是 EOF**——`write` 不会返回 0（除非 `n == 0`），返回 0 或短计数都要当作异常/继续
-1	出错，`errno` 说明原因	出错，`errno` 说明原因
-1 且 `errno == EINTR`	被信号打断，**应重试**（不重试会**静默丢数据**）	同左

短计数在何时发生（讲义权威结论）：

会发生短计数	不会发生短计数
读时遇到 EOF	从**磁盘文件**读（除 EOF 外）
从**终端**读（一次只能读到一行）	向**磁盘文件**写
从**网络 socket** 读	—
读/写 **Linux 管道（pipe）**	—

最佳实践：永远为短计数做好准备。讲义明确批评用 `read` 包一个 `while (n > 0)` 就以为万事大吉的做法——真正的解法就是 A.11.3 的 `rio` 库。

逐字节 `cp` 的两个版本对比（讲义原例）：

```
/* ❌ 有 bug 的版本：没处理短计数，会静默丢数据 */
while (read(STDIN_FILENO, &c, 1) != 0)
    write(STDOUT_FILENO, &c, 1);

/* ✅ 正确版本：检查 write 的返回值并重试 */
while (Read(STDIN_FILENO, &c, 1) != 0)
    Write(STDOUT_FILENO, &c, 1);
```

A.11.3 `rio` 四函数对比与"为什么必须用 rio"

函数	原型	语义	EOF 行为	可否混用
`rio_readn`	`ssize_t rio_readn(int fd, void *usrbuf, size_t n)`	**无缓冲**的健壮读：自动处理短计数与 `EINTR`，**只在 EOF 时**返回短计数	返回**已读字节数**（可能 $< n$），再读返回 0	与 `rio_writen` **可任意交错**
`rio_writen`	`ssize_t rio_writen(int fd, void *usrbuf, size_t n)`	**无缓冲**的健壮写：自动重试短计数与 `EINTR`	**永不返回短计数**	同上
`rio_readinitb`	`void rio_readinitb(rio_t *rp, int fd)`	把 fd 绑定到一个带缓冲的 `rio_t`（**必须先调用**）	—	—
`rio_readlineb`	`ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen)`	**带缓冲**地读**一行文本**：最多 `maxlen` 字节，读到 `'\n'` 为止并**保留** `'\n'`，末尾补 `'\0'`	返回**已读字节数**；`0` 表示 EOF	与 `rio_readnb` **可任意交错**；⚠ **不可与 `rio_readn` 交错**
`rio_readnb`	`ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n)`	**带缓冲**地读 `n` 字节（不足只可能因 EOF）	返回已读字节数；`0` 表示 EOF	同上

返回值总览：`rio_readn` / `rio_writen`：成功返回传输字节数，`rio_readn` 在 EOF 返回 0，出错返回 -1。`rio_readlineb` / `rio_readnb`：成功返回读到的字节数，0 = EOF，-1 = 错误。

"为什么必须用 rio"——三层理由：

1. **`read` / `write` 的短计数会让朴素代码静默出错**：网络 socket 与管道上短计数是常态，手写重试循环容易漏掉 `EINTR`、漏掉部分写、或在 `write` 里把"部分成功"当成功。
2. **文本协议需要按行读**：HTTP 请求头、Tiny 服务器的请求行、Shell 的命令行都以 `'\n'` 为界；`rio_readlineb` 把"逐字节读 + 找换行 + 补 `'\0'`"封装成一次调用，且**带缓冲**（一次系统调用喂满整个缓冲区），比逐字节 `read` 快几个数量级。
3. **线程/多协议下的正确性**：`rio_readlineb` 与 `rio_readnb` 共享同一个 `rio_t` 缓冲区，允许"读一行头部、再读 `Content-Length` 字节正文"这种混合模式；但 `rio_readn` **不带缓冲**、会绕过缓冲区，与缓冲版本混用会导致**数据错位**——这是 Proxy Lab 里最经典的诡异 bug。

使用模板：

```
#include "csapp.h"
int main(void)
{
    rio_t rio;
    char buf[MAXLINE];
    Rio_readinitb(&rio, STDIN_FILENO);
    ssize_t n;
    while ((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0)   /* 0 = EOF */
        Rio_writen(STDOUT_FILENO, buf, n);                 /* 永不短计数 */
    return 0;
}
```

A.11.4 `struct stat` 关键字段与类型判定宏

```
#include <sys/stat.h>
int stat(const char *filename, struct stat *buf);   /* 按路径查询元数据 */
int lstat(const char *filename, struct stat *buf);  /* 不跟踪符号链接（查的是链接本身） */
int fstat(int fd, struct stat *buf);                /* 按已打开的描述符查询 */
/* 成功返回 0，出错返回 -1 并置 errno */
```

字段	类型	含义
`st_dev`	`dev_t`	设备（含该文件的磁盘设备编号）
`st_ino`	`ino_t`	**i-node 号**（文件在磁盘上的真实身份标识）
`st_mode`	`mode_t`	**保护位与文件类型**（`S_IS*` 宏与权限位的来源）
`st_nlink`	`nlink_t`	**硬链接数**（= 0 时才真正删除文件）
`st_uid` / `st_gid`	`uid_t` / `gid_t`	属主 / 属组的用户 ID
`st_rdev`	`dev_t`	若为设备文件，则是设备类型
`st_size`	`off_t`	**文件总大小（字节）**——Tiny 服务器发 `Content-Length` 就靠它
`st_blksize`	`unsigned long`	文件系统 I/O 的理想块大小
`st_blocks`	`unsigned long`	已分配的**块数**（512 字节为单位）
`st_atime` / `st_mtime` / `st_ctime`	`time_t`	最后访问 / 最后修改 / 最后状态变更时间

文件类型判定宏（都作用于 `st_mode`）：

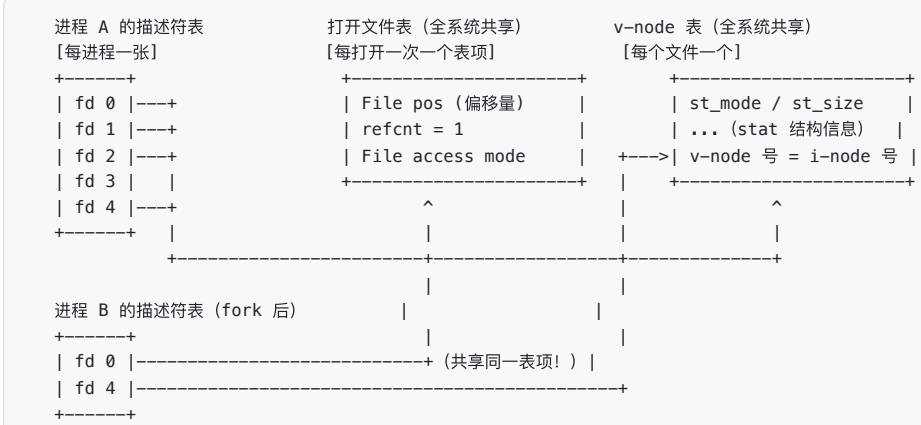
宏	判定类型	典型用途
`S_ISREG(m)`	**普通文件**（regular file）	Tiny 服务器"只服务普通文件"的检查（拒绝目录、FIFO、设备）
`S_ISDIR(m)`	**目录**	`ls`、`find` 遍历
`S_ISSOCK(m)`	**socket**	识别 Unix 域套接字
`S_ISLNK(m)`	**符号链接**	与 `lstat` 配合检测链接（`stat` 会跟随链接，故看不到）
`S_ISCHR(m)` / `S_ISBLK(m)`	字符设备 / 块设备	`/dev` 下的文件
`S_ISFIFO(m)`	**命名管道（FIFO）**	防止 `read` 被 FIFO 永久阻塞
`S_ISUID` / `S_ISGID` / `S_ISVTX`	set-uid / set-gid / sticky 位	权限审计

- **`stat` vs `lstat` 的唯一区别**：遇到**符号链接**时，`stat` **跟随**（返回目标文件的元数据），`lstat` **不跟随**（返回链接自身的元数据，`S_ISLNK` 为真）。
- **Tiny Web 服务器的核心安全检查**（Proxy Lab 的参考实现）：

```
struct stat sbuf;
if (stat(filename, &sbuf) < 0) {           /* 文件不存在 */
    clienterror(cfd, filename, "404", "Not found", "Tiny couldn't find this file");
    return;
}
if (!(S_ISREG(sbuf.st_mode)) || !(S_IRUSR & sbuf.st_mode)) {   /* 不是普通文件或不可读 */
    clienterror(cfd, filename, "403", "Forbidden", "Tiny couldn't read the file");
    return;
}
```

A.11.5 三层结构与 `fork` 后的共享

Unix 内核用三张表表示打开的文件：



层	表	共享范围	存放内容	关键点
第 1 层	**描述符表**（descriptor table）	**每进程一张**	`fd` → 打开文件表项指针	`open` / `dup` 都改这一层；`fd` 只是**本进程的小整数下标**
第 2 层	**打开文件表**（open file table）	**全系统共享**	**文件偏移量** **`File pos`**、`refcnt`、访问模式	**`File pos` 是"每次 `open` 一份"的**——这就是下面两条规则的全部原因
第 3 层	**v-node 表**（v-node table）	**全系统共享**	`stat` 结构信息（大小、类型、权限、i-node 号）	无论多少进程打开同一文件，都指向**同一个 v-node**

两条必须记住的推论：

场景	偏移量行为	图示要点
`fork` 之后	父子进程**共享**同一个打开文件表项 → **共享文件偏移量**。父进程读 5 字节后，子进程从第 6 字节继续读	两个描述符表项指向**同一个** File pos
`open` 两次同一文件	内核创建**两个不同的打开文件表项** → **各自独立的偏移量**，互不干扰	两个 File pos，`refcnt` 各为 1
`dup(fd)` / `dup2(fd, n)`	新描述符与旧描述符指向**同一个**表项 → **共享偏移量**	同 `fork`

- **`refcnt`（引用计数）**：每多一个描述符指向该表项就加一；`close` 只减一，**归零时**才真正释放打开文件表项。
- **这一层结构解释了 Shell 重定向与 Proxy 的多线程共享行为，也是"`fork` 之后父子输出交错"的根本原因。**

A.11.6 `dup2` 重定向与 `fileno` / `fdopen`

```
int dup2(int oldfd, int newfd);
/* 把（本进程）描述符表项 oldfd 复制到 newfd 位置。
 * 若 newfd 已打开，先关闭它（且是原子的）。
 * 成功返回 newfd；出错返回 -1。 */
```

Shell 重定向 `ls > foo.txt` 的三步写法：

```
/* ① 打开目标文件，拿到 fd（通常是 3，因为 0/1/2 已被占用） */
int fd = open("foo.txt", O_WRONLY | O_CREAT | O_TRUNC, DEFFILEMODE);

/* ② 把 fd 复制到 stdout（fd 1）的位置 */
dup2(fd, STDOUT_FILENO);      /* 此后写 fd 1 就是写 foo.txt */

/* ③ 关闭多余的 fd（否则 foo.txt 的表项 refcnt 永远不为 0，泄漏） */
close(fd);

/* ④ 现在 execve：子进程继承"fd 1 指向 foo.txt"这一事实 */
execve("/bin/ls", argv, environ);
```

重定向前后的描述符表：

```
before dup2(4,1)                    after dup2(4,1)
+------+                            +------+
| fd 0 |--> a (terminal)            | fd 0 |--> a (terminal)
| fd 1 |--> a (stdout)              | fd 1 |--> b (fd 4 的表项)
| fd 2 |--> a (stderr)              | fd 2 |--> a (stderr)
| fd 3 |   (closed)                 | fd 3 |   (closed)
| fd 4 |--> b (foo.txt)             | fd 4 |--> b (foo.txt)
+------+                            +------+
a、b 为打开文件表项
```

- **`dup2` 的原子性**：`dup2` 先关 `newfd`（若已打开）再复制，这两步在**内核里原子完成**，不会出现"关掉了 `newfd` 但还没复制"的窗口。若 `oldfd == newfd`，`dup2` **直接返回 `newfd` 且不做任何事**（不会误关）。
- **`open` 后必须 `close` 原 fd**：`execve` 只保留 fd 1，多出的 fd 会让文件表项 `refcnt` 不为 0；而且长驻进程会**泄漏描述符**。

标准 I/O 与文件描述符的双向转换：

函数	原型	作用
`fileno`	`int fileno(FILE *stream)`	从 `FILE *` 取出底层的**文件描述符**（如 `fileno(stdout)` = 1）
`fdopen`	`FILE *fdopen(int fd, const char *mode)`	把一个**已有的 fd** 包成 `FILE *`（例如把 socket fd 包成 `FILE *` 以便用 `fprintf` / `fgets`）

```
/* 把 socket 包成 FILE* —— 方便用 fprintf 写 HTTP 响应 */
int fd = open_clientfd(host, port);
FILE *fp = fdopen(fd, "w");
fprintf(fp, "GET / HTTP/1.0\r\n\r\n");
fflush(fp);                          /* 别忘了 flush! 否则请求还压在缓冲区里 */
```

A.11.7 标准 I/O 缓冲类型与 `fflush`；`fork` 后输出重复的陷阱

缓冲类型	触发条件	何时真正写出
全缓冲（fully buffered）	流关联到**磁盘文件**（非常规文件）	缓冲区满、`fflush`、`fclose`、或进程正常 `exit` 时
行缓冲（line buffered）	流关联到**终端**（`isatty` 为真）	遇到 `'\n'`、缓冲区满、`fflush`、读操作、`fclose`
无缓冲（unbuffered）	**`stderr` 总是无缓冲**	每次写立即发出（保证错误信息不丢）

接口	作用
`fflush(FILE *fp)`	**强制把用户空间缓冲区里的数据写出**到内核。`fflush(NULL)` 刷新**所有**输出流
`setvbuf(FILE *fp, char *buf, int mode, size_t size)`	`mode` 取 `_IOFBF`（全缓冲）/ `_IOLBF`（行缓冲）/ `_IONBF`（无缓冲）；**必须在任何 I/O 之前调用**
`fclose(FILE *fp)`	刷新并释放（**内含 `fflush`**）

`fork` 之后输出重复的陷阱（讲义与笔记中的经典 bug）：

阶段	发生了什么
① 父进程 `printf("hello\n")`，**输出到磁盘文件**（全缓冲）	数据只在**用户空间缓冲区**里，尚未 `write` 给内核
② 父进程 `fork()`	子进程获得父进程**用户空间缓冲区的完整副本**（含那份未刷出的数据）
③ 两个进程各自 `exit()`	各自的 stdio 清理**分别把同一份数据写出一次 → 屏幕上出现两遍**

三种修复方式：

1. **`fork` 之前 `fflush(stdout);`**（最直接）。
2. **输出到终端时改行缓冲**（但重定向到文件就失效，不能依赖）。
3. **干脆用 `write(STDOUT_FILENO, ...)`**（无用户空间缓冲，天然安全；也是信号处理程序中唯一可用的输出方式）。

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    /* ⚠ 把 stdout 重定向到文件后运行，可见 "hello" 打印两遍 */
    printf("hello\n");        /* 全缓冲：此时还没真正写出 */
    if (fork() == 0) {
        exit(0);              /* 子进程 exit -> 刷出缓冲区里那份 hello */
    }
    exit(0);                  /* 父进程 exit -> 再刷出一次 */
    /* 修复：在 fork 之前加一句 fflush(stdout); */
}
```

> ⚠ **仅供演示，请勿模仿**：上面代码刻意制造"重复输出"。在**终端**上运行时因为行缓冲（遇 `\n` 立即刷出）看不到问题，**只有重定向到文件**才会暴露——这正是它难查的原因。复现命令：`./forkdup > out.txt; cat out.txt`。

A.11.8 目录操作

接口	原型	语义要点
`opendir`	`DIR *opendir(const char *name)`	打开目录，返回**目录流**；失败返回 `NULL`
`readdir`	`struct dirent *readdir(DIR *dirp)`	返回**下一个目录项**的指针；**读完返回 `NULL`**（这既表示结束也表示出错，需配合 `errno`）
`closedir`	`int closedir(DIR *dirp)`	关闭目录流
`mkdir`	`int mkdir(const char *pathname, mode_t mode)`	创建目录（`mode` 同样受 `umask` 影响）
`rmdir`	`int rmdir(const char *pathname)`	删除**空**目录（非空则失败，`errno = ENOTEMPTY`）
`chdir`	`int chdir(const char *pathname)`	改变**当前工作目录**（影响后续相对路径解析）
`getcwd`	`char *getcwd(char *buf, size_t size)`	把**绝对路径**写入 `buf`（`size` 不够则失败，`errno = ERANGE`）

`struct dirent` 的关键字段：

字段	含义
`d_ino`	i-node 号（与 `stat` 的 `st_ino` 对应）
`d_off`	目录流中的偏移量（供 `seekdir` / `telldir` 使用）
`d_reclen`	本条记录的长度
`d_type`	文件类型（`DT_REG` / `DT_DIR` / `DT_LNK` …，**不是所有文件系统都填**，不可靠时要用 `stat` / `lstat` 再判）
`d_name[]`	文件名（**不以 `'\0'` 保证结尾**以外的长度保证，用 `strncpy` 时注意）

遍历目录模板：

```c
#include <dirent.h>
#include <stdio.h>
#include <sys/stat.h>

int main(int argc, char **argv)
{
    const char *path = (argc > 1) ? argv[1] : ".";
    DIR *dirp = opendir(path);
    if (!dirp) { perror("opendir"); return 1; }

    struct dirent *de;
    while ((de = readdir(dirp)) != NULL) {          /* NULL = 读完 */
        struct stat sbuf;
        if (stat(de->d_name, &sbuf) < 0) continue;  /* 相对当前工作目录 */
        const char *kind = S_ISDIR(sbuf.st_mode) ? "dir " :
                           S_ISREG(sbuf.st_mode) ? "file" : "othr";
        printf("%s  %10ld  %s\n", kind, (long)sbuf.st_size, de->d_name);
    }
    closedir(dirp);
    return 0;
}
```

- **必须跳过 `.` 与 `..`**（`readdir` 会返回它们），否则递归遍历会无限循环。
- **`readdir` 不是线程安全的**（返回内部静态缓冲区）；多线程下要用 `readdir_r`（已废弃）或在外部加锁。**因此它不在异步信号安全清单里。**
- **`d_name` 是相对名字**：`stat(de->d_name)` 只在"当前工作目录 == 被遍历目录"时才正确，严谨写法是把目录前缀拼上。

A.11.9 i-node、硬链接与符号链接

i-node（索引节点）：磁盘上每个文件（**每个真实文件对象**，而非每个名字）对应一个 i-node，保存**除文件名以外**的一切：文件类型与权限、属主、大小、时间戳、链接计数、**指向数据块的指针**。

对比项	硬链接（hard link）	符号链接（symbolic link / soft link）
本质	目录项里"**同一个 i-node 号**"的另一个名字	一个**独立文件**，内容是**目标路径字符串**
i-node	与目标**共享同一个** i-node	有**自己的** i-node，类型是 `S_ISLNK`
`st_nlink`	**+1**（这就是"硬链接数"）	**不变**（不影响目标的链接数）
跨文件系统	**不可以**（i-node 号只在同一文件系统内唯一）	**可以**
指向目录	**不允许**（会造成目录环）	**允许**（但可能产生循环，需靠最大深度限制）
目标被删除后	只要 `st_nlink > 0`，**数据仍在**，其他名字照常访问	**变成悬空链接（dangling）**，访问报 `ENOENT`
`stat`	返回**目标**的元数据	返回**链接自身**的元数据（用 `lstat` 才看得到）
创建命令	`ln target linkname`	`ln -s target linkname`
删除命令	`unlink(name)` / `rm name`	`unlink(name)` / `rm name`（**删的是链接本身，不是目标**）

命令 / 接口	作用
`ln <target> <name>`	创建**硬链接**
`ln -s <target> <name>`	创建**符号链接**（`target` 可以是相对路径——**相对的是链接所在目录**，这是常见的困惑点）
`unlink(<pathname>)`	**删除一个目录项**：i-node 的 `st_nlink` 减 1，**减到 0 且无进程打开**时才真正释放数据块。系统调用名就叫 `unlink`（不是 `delete`），正体现了"删名字不删 inode"的语义
`remove(<pathname>)`	libc 封装：对文件等价于 `unlink`，对目录等价于 `rmdir`
`stat` / `lstat`	跟随 / 不跟随符号链接
`readlink(<path>, buf, size)`	读出符号链接**指向的路径字符串**（**不加 `'\0'`**，且**不跟随**多级链接）
`ls -l` 显示 `lrwxrwxrwx ... link -> target`	一眼识别符号链接；硬链接则表现为 `st_nlink` 大于 1，用 `ls -li` 比对 i-node 号

- **`st_nlink == 0` 才是文件的真正死亡时刻**：这解释了"为什么 `rm` 一个被进程打开的文件，进程仍能继续读写"——i-node 与数据块要等最后一个打开的 fd 关闭才释放。

- **`ln -s` 的相对路径陷阱**：`ln -s ../a/b link` 中 `../a/b` 是**相对 `link` 所在目录**解析的，不是相对你敲命令时的目录。用 `readlink` 可以看清链接里到底存了什么。

A.11.10 系统级 I/O 常用命令

命令	作用
`strace -e trace=openat,read,write,close ./prog`	看真实的系统调用序列与返回值（含短计数与 `EINTR`）
`strace -e trace=file -f ./prog`	只跟踪与文件相关的调用（含 `stat`、`openat`）
`lsof -p <pid>`	列出进程打开的所有 fd（**排查 fd 泄漏**）
`ls -l /proc/<pid>/fd`	直接看描述符表（`0 -> /dev/pts/0`、`3 -> socket:[...]`）
`ls -li <file>`	显示 i-node 号——**验证两个名字是否为硬链接**（同号即硬链接）
`stat <file>`	格式化查看 `st_mode`、`st_ino`、`st_nlink`、`st_size`、三个时间戳
`readlink -f <path>`	逐级解析符号链接并输出绝对路径
`cat /proc/<pid>/fdinfo/<fd>`	查看单个 fd 的 `pos`（**文件偏移量**）、`flags` ——验证"共享偏移量"的绝佳手段
`mkfifo /tmp/f`	创建命名管道，实测 FIFO 上的短计数行为
`valgrind --track-fds=yes ./prog`	报告 fd 泄漏与重复关闭

A.12 网络编程速查

> 对应讲次：Lecture 19（`F25-19-fs_netprog1.txt`）、Lecture 20（`F25-20-netprog2.txt`）； 教材 CS:APP3e 第 11.1—11.6 节；关联 **L7 Proxy Lab**。

A.12.1 协议分层

层次	协议	提供的服务	数据单元	编址
应用层	HTTP、DNS、SMTP、FTP	具体应用语义	报文（message）	URL / 域名
传输层	**TCP / UDP**	**进程到进程**的通信	段（TCP segment）/ 数据报（UDP datagram）	**端口号**（16 位）
网络层	**IP**（IPv4 / IPv6）	**主机到主机**的不可靠数据报投递 + 统一编址	数据报（datagram）/ 分组（packet）	**IP 地址**（IPv4 32 位 / IPv6 128 位）
链路层	Ethernet、802.11、Fibre Channel、T1、DSL	**同一局域网内**相邻节点间的帧传输	帧（frame）	MAC 地址
物理层	双绞线、光纤、无线电	比特流传输	比特	—

- **层次的意义**：每一层只依赖下一层提供的服务，并用自己的**封装（encapsulation）**把上层数据包起来——讲义的分层图明确画出 `LAN1 frame` 里套着 `internet packet`，包里再套 `data`，路由器**拆掉旧的帧头、换上新的帧头**（PH 保持不变，FH1 → FH2）。
- **一个"internet"（小写）** = 用**路由器**把若干互不兼容的 LAN 互连起来的网络；**"Internet"（大写）** 特指基于 **TCP/IP 协议族**的全球互联网。
- 协议要做两件事：**命名方案**（统一的地址格式）与**投递机制**（标准的传输单元 = 头部 + 载荷）。

A.12.2 IP / UDP / TCP 特性对比

特性	IP	UDP	TCP
英文	Internet Protocol	User Datagram Protocol	Transmission Control Protocol
服务对象	**主机到主机**（host-to-host）	**进程到进程**（process-to-process）	**进程到进程**（process-to-process）
可靠性	**不可靠**：数据报可能丢失、重复、乱序	**不可靠**（继承 IP）	**可靠**：不丢、不重、按序
连接性	**无连接**	**无连接**	**面向连接**（三次握手建立、四次挥手关闭）

特性	IP	UDP	TCP
数据抽象	数据报（datagram）	数据报（datagram）	**字节流**（byte stream）：无记录边界
编址	IP 地址	IP 地址 + **端口**	IP 地址 + **端口**
是否保证边界	—	**保证**（一个数据报一个边界）	**不保证**（可能被合并/切分，必须自己定界）
典型用途	所有上层协议的基础	DNS、视频流、游戏、`traceroute`	HTTP、SSH、SMTP、几乎所有需要正确的场景
讲义原话	"基本命名方案 + 不可靠的**分组投递**"	"用 IP 提供**不可靠的数据报**投递"	"用 IP 提供**可靠字节流**，运行在**连接**之上"

- **TCP 字节流的推论的实战意义**：TCP **不保留应用层的记录边界**，所以 HTTP 必须用 `Content-Length` 或连接关闭（`Connection: close`）来定界——这正是 Proxy Lab 里"精确转发 `Content-Length`"如此关键的原因。**绝不能用"一次 `read` = 一个报文"来读 socket。**

A.12.3 字节序转换函数与网络字节序

网络字节序 = 大端（big-endian）：最高有效字节放在最低地址。任何**在包头中被搬运的整数**（IP 地址、端口号、长度字段）都必须用网络字节序，否则不同主机间会互相误解。

函数	原型	方向	用途
`htonl`	`uint32_t htonl(uint32_t hostlong)`	主机 → 网络，**32 位**	转换 `in_addr.s_addr`（IPv4 地址）
`htons`	`uint16_t htons(uint16_t hostshort)`	主机 → 网络，**16 位**	转换端口号（`sin_port`）
`ntohl`	`uint32_t ntohl(uint32_t netlong)`	网络 → 主机，**32 位**	从包头读回地址
`ntohs`	`uint16_t ntohs(uint16_t netshort)`	网络 → 主机，**16 位**	从包头读回端口

- **记忆**：`h` = host，`n` = network，`s` = short（16 位），`l` = long（**在这些函数里指 32 位**，不是 C 的 `long`）。
- **在**小端（x86-64 就是小端）机器上，这四个函数**真的会翻转字节**；在大端机器上它们是**空操作**。所以**必须调用**，不能因为"我的机器恰好匹配"而省略。
- 讲义强调："这对**任何**从一个机器传到另一个机器的包头整数都成立"，例如端口号。
- **`in_addr` 结构与点分十进制**：

```
struct in_addr {
    uint32_t s_addr;   /* 网络字节序（big-endian） */
};
/* 0x8002C2F2 == 128.2.194.242 */
```

转换工具	作用
`getaddrinfo`	字符串（主机名或点分十进制 / IPv6 字面量）→ **socket 地址结构**（**首选**）
`getnameinfo`	socket 地址结构 → 字符串（点分十进制或主机名）
`inet_pton` / `inet_ntop`	仅做"字符串 ↔ 二进制地址"转换（**不含 DNS 查询**，旧接口）
`inet_aton` / `inet_ntoa`	**废弃**接口（不支持 IPv6、`inet_ntoa` 非线程安全）
`gethostbyname` / `getservbyname`	**过时且非可重入**，已被 `getaddrinfo` 取代

- **IPv4 vs IPv6**：IPv4 是 **32 位**地址（1981 年制定，1990 年前后就已知不够用）；IPv6 是 **128 位**地址（1996 年制定，如 `2001:0db8:0:0:0:0:cafe:1a7e`），因需替换路由器而**推广极慢**。**现代 sockets API 让应用代码几乎无需关心**——用 `getaddrinfo` + `sockaddr_storage` 就自动兼容两者。

A.12.4 socket 地址结构

```
/* 通用结构：只用于"指针类型转换"，从不直接使用其字段 */
struct sockaddr {
    uint16_t  sa_family;    /* 地址族：AF_INET / AF_INET6 */
    char      sa_data[14];  /* 地址数据（长度随协议族变化） */
};
typedef struct sockaddr SA;   /* CS:APP 的惯用简写 */

/* IPv4 专用 */
struct sockaddr_in {
    uint16_t       sin_family;  /* AF_INET */
    uint16_t       sin_port;    /* 端口号，网络字节序（htons） */
    struct in_addr sin_addr;    /* IP 地址，网络字节序（in_addr.s_addr） */
    unsigned char  sin_zero[8]; /* 填充，使结构与 sockaddr 等长（16 字节） */
};

/* IPv6 专用 */
struct sockaddr_in6 {
    uint16_t       sin6_family;   /* AF_INET6 */
    uint16_t       sin6_port;     /* 端口号，网络字节序 */
    uint32_t       sin6_flowinfo; /* 流信息 */
    struct in6_addr sin6_addr;    /* 128 位地址 */
    uint32_t       sin6_scope_id; /* 范围 ID */
};

/* 足够容纳任何地址族的"万能"结构（客户端地址就用它） */
struct sockaddr_storage {
    uint16_t ss_family;   /* 地址族；其余字节足够大以容纳任何协议族地址 */
    /* ... 填充到足够大 ... */
};
```

结构	何时用	关键点
`struct sockaddr`	仅作为**函数参数类型**（`SA *`）	类型转换的"公共接口"，**不要**直接填字段
`struct sockaddr_in`	手写 IPv4 地址时	`sin_port` 必须 `htons`；`sin_addr.s_addr` 必须网络字节序（用 `inet_pton`）
`struct sockaddr_in6`	手写 IPv6 地址时	—
`struct sockaddr_storage`	**`accept` 的客户端地址缓冲区**（唯一正确选择）	大小**足够容纳任何**地址族；用 `sockaddr_in` 接收 IPv6 连接会**缓冲区溢出**

- **`accept` 的标准写法**（讲义原例，注意 `clientlen` 的初始化）：

```
struct sockaddr_storage clientaddr;              /* 够大，能装任何地址 */
socklen_t clientlen = sizeof(struct sockaddr_storage);   /* ⚠ 必须初始化！ */
int connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
```

- **类型转换约定**：`sockaddr_storage` → `sockaddr_in` 时用 `(struct sockaddr_in *)&clientaddr`，读 `sin_family` 判断是哪一族，再取 `sin_addr` / `sin_port`。
- **`socklen_t` 是"值-结果"参数**：`accept` 进入时你要告诉内核缓冲区**有多大**，返回时内核写入**实际用了多少**；不初始化就是 UB。

A.12.5 套接字函数表

函数	作用	客户端 / 服务器	关键参数	常见错误
`getaddrinfo`	主机名+服务名 → **`addrinfo` 链表**（每个含一个 ready-to-use 的 socket 地址与参数）	**两者都用**	`host`、`service`、`hints`（输入约束）、`result`（输出链表）	忘记 `freeaddrinfo` 泄漏；`hints` 未 `memset` 清零导致垃圾字段；不知道要**遍历链表**逐个尝试
`freeaddrinfo`	释放 `getaddrinfo` 返回的**整个链表**	两者	`result`	只 `free` 首个节点；用 `free()` 逐个释放（错，链表是一次分配的，必须用这个函数）
`socket`	创建**套接字描述符**	两者	`domain`（`AF_INET`）、`type`（`SOCK_STREAM`）、`protocol`（`0` = 自动选择）	用 `getaddrinfo` 时**不要硬编码** `AF_INET` / `SOCK_STREAM`，应写 `ai->ai_family` / `ai->ai_socktype`

函数	作用	客户端 / 服务器	关键参数	常见错误
`connect`	客户端**发起连接**（三次握手）	**仅客户端**	`clientfd`、服务器地址 `addr`、`addrlen`	忘了对链表中每个候选地址重试；未检查返回 **−1**
`bind`	把**本地地址**与 `sockfd` 关联	**仅服务器**	`sockfd`、`addr`（含端口）、`addrlen`	端口被占用（`EADDRINUSE`，需 `SO_REUSEADDR`）；`addr` 未用 `htons` 转端口
`listen`	把**主动套接字**转为**监听套接字**	**仅服务器**	`sockfd`、`backlog`（内核排队上限，**约 128**）	忘了 `listen` 直接 `accept`；把 `listenfd` 当数据通道读写
`accept`	**等待并接受**一个连接请求，返回**已连接描述符** **`connfd`**	**仅服务器**	`listenfd`、`addr`（客户端地址，用 `sockaddr_storage`）、`addrlen`（**必须初始化**）	**在循环外只 `accept` 一次**（只能服务一个客户端）；用 `sockaddr_in` 接 IPv6
`close`	关闭描述符	两者	`fd`	忘记 `close(connfd)` → **描述符泄漏**（长驻服务器最终 `EMFILE`）
`setsockopt`	设置套接字选项	通常服务器	`SOL_SOCKET`、`SO_REUSEADDR`、`&optval`、`sizeof(optval)`	Proxy/Tiny 调试时端口 `TIME_WAIT` 导致"地址已占用"，加 `SO_REUSEADDR` 解决
`getnameinfo`	地址结构 → 主机名/服务名的可读字符串	两者	地址、长度、缓冲区	用于日志打印（把客户端 IP 转成点分十进制）

错误处理要点：`getaddrinfo` **不设置** **`errno`**，要用 **`gai_strerror(errcode)`** 取错误信息；其余函数返回 **−1** 时用 `errno` + `strerror` / `perror`。

- CS:APP **封装**：`open_clientfd(host, port)` 内部 = `getaddrinfo` + 循环 `socket` / `connect` + `freeaddrinfo`；`open_listenfd(port)` 内部 = `getaddrinfo(hints.ai_flags = AI_PASSIVE)` + `socket` / `setsockopt(SO_REUSEADDR)` / `bind` / `listen` / `freeaddrinfo`。

A.12.6 客户端与服务器的调用顺序

客户端（`open_clientfd` 展开）：

```
getaddrinfo(host, port, &hints, &listp)      <-- 解析主机+服务
      |
      v
for (p = listp; p; p = p->ai_next) {         <-- 逐个候选地址尝试
      |
      v
   socket(p->ai_family, p->ai_socktype, p->ai_protocol)   -> clientfd
      |
      v
   connect(clientfd, p->ai_addr, p->ai_addrlen)   <-- 成功则跳出循环
}
      |
      v
freeaddrinfo(listp)
      |
      v
rio_readinitb(&rio, clientfd)  ->  rio_writen(请求)  <->  rio_readlineb/rio_readnb(响应)
      |
      v
close(clientfd)
```

服务器（迭代式 `open_listenfd` 展开）：

```
getaddrinfo(NULL, port, &hints{AI_PASSIVE}, &listp)      <-- 通配本地地址
      |
      v
for (p = listp; p; p = p->ai_next) {
   socket(p->ai_family, p->ai_socktype, p->ai_protocol)  -> listenfd
   setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, ...)
   bind(listenfd, p->ai_addr, p->ai_addrlen)             <-- 关联端口，失败则 close 重试
   listen(listenfd, LISTENQ)                             <-- 转为监听套接字
}
freeaddrinfo(listp)
      |
      v
while (1) {                                              <-- 服务器是无限循环!
   clientlen = sizeof(struct sockaddr_storage);
   connfd = accept(listenfd, (SA *)&clientaddr, &clientlen)   <-- 阻塞等待
      |
      v
   do_work(connfd)                                       <-- 读写请求/响应（rio）
      |
      v
   close(connfd)                                         <-- ⚠ 关的是 connfd，不是 listenfd
}
```

- **一句话记住**：客户端 = `getaddrinfo` → `socket` → `connect`；服务器 = `getaddrinfo` → `socket` → `bind` → `listen` → 循环(`accept` → 服务 → `close`)
- `backlog` **的含义**：内核在**开始拒绝**连接请求之前，允许排队的**未完成连接请求**数量（约 128）。它**不是**能同时服务的客户端数上限。

A.12.7 `listenfd` vs `connfd`

对比项	`listenfd`（监听描述符）	`connfd`（已连接描述符）
创建次数	**整个服务器生命周期只创建一次**（在 `open_listenfd` 里）	**每接受一个客户端连接就创建一个**
创建者	`socket()` + `bind()` + `listen()`	`accept()` 返回
生命周期	与服务器进程同寿	只覆盖**这一次**请求/响应，处理完立即 `close(connfd)`
用途	是"**接线员**"：只负责**接收连接请求，绝不用它读写数据**	是"**通话线路**"：真正的读写通道，交给 `rio_read*` / `rio_writen` 或工作线程
并发含义	唯一，被所有请求共享	每连接一个，是**并发的天然分界点**——Proxy Lab 里 `connfd` 就是传给每个工作线程的参数

- **最常见的初学者 bug**：把 `listenfd` 传给 `rio_readlineb` 去读 HTTP 请求，或者只 `accept` 一次然后处理所有客户端——两者都源自"分不清这两个描述符"。
- `close(connfd)` **必须做**：`listenfd` 永远不关，`connfd` 每次都要关，否则描述符泄漏。

A.12.8 `getaddrinfo` 的 `hints` 关键字段

```
#include <netdb.h>
int getaddrinfo(const char *host,             /* 主机名或地址字符串（可为 NULL） */
                const char *service,          /* 端口号或服务名 */
                const struct addrinfo *hints, /* 输入：对结果的约束 */
                struct addrinfo **result);    /* 输出：addrinfo 链表 */
void freeaddrinfo(struct addrinfo *result);
const char *gai_strerror(int errcode);
```

`hints` 字段	取值	含义
`ai_family`	`AF_INET` / `AF_INET6` / `AF_UNSPEC`	只接受 IPv4 / 只接受 IPv6 / **两者都接受（可移植代码应留 `AF_UNSPEC`**，或一开始就用 `AF_INET` 也不要紧，取决于需求）
`ai_socktype`	`SOCK_STREAM` / `SOCK_DGRAM`	只要 TCP / 只要 UDP
`ai_protocol`	`0`（或 `IPPROTO_TCP`）	通常给 0 让系统推断
`ai_flags`	**`AI_PASSIVE`**	**服务器专用**：表示"我要 `bind` 一个**监听**套接字，`host` 传 `NULL` 时返回**通配地址**（`INADDR_ANY` / `in6addr_any`）"。**不设它而 `host = NULL`，得到的地址无法用于 `bind`**
`ai_flags`	`AI_NUMERICHOST` / `AI_NUMERICSERV`	禁止 DNS 查询 / 只接受数字端口（加速与调试）

`hints` 字段	取值	含义
`ai_flags`	`AI_CANONNAME`	在 `ai_canonname` 里返回规范主机名
`ai_flags`	`AI_ADDRCONFIG`	只返回本机**已配置**的地址族（避免拿到用不了的 IPv6）

- **`hints` 必须清零**：`memset(&hints, 0, sizeof(hints));` 后再填字段——未清零的栈垃圾会让 `ai_flags` 里带上随机标志位。
- **`ai_addr` / `ai_addrlen` / `ai_family` / `ai_socktype` 是"可直接用"的**：讲义明确说"用 `getaddrinfo` 就**不必**硬编码协议细节"，`socket()` 直接传 `ai->ai_family, ai->ai_socktype, ai->ai_protocol`。
- **`ai_next`**：链表指针，**必须遍历**——一个主机名可能解析出多个 IP（如 `twitter.com` 返回 4 个 IPv4），一个地址族失败要试下一个。
- **`getaddrinfo` 的三个优势**：**可重入**（线程安全）、**协议无关**（同一份代码兼容 IPv4/IPv6）、**取代了 `gethostbyname` / `getservbyname`**（后者非可重入）。缺点：**略复杂**，但"少数几个用法模式足以覆盖绝大多数场景"。
- **`AI_PASSIVE` 的对照写法**：

```
struct addrinfo hints, *listp;
memset(&hints, 0, sizeof(hints));
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags    = AI_PASSIVE | AI_ADDRCONFIG | AI_NUMERICSERV;
getaddrinfo(NULL, port, &hints, &listp);   /* 服务器：NULL host + AI_PASSIVE */
```

A.12.9 HTTP 请求/响应报文格式模板

HTTP 请求：请求行 + 零个或多个请求头 + **空行**（+ 可选请求体）

```
<method> <uri> <version>\r\n
<header name>: <header data>\r\n
<header name>: <header data>\r\n
\r\n
```

```
GET /index.html HTTP/1.1\r\n        <- 请求行：方法 + URI + 版本
Host: www.cmu.edu\r\n               <- 必需头（HTTP/1.1）
User-Agent: Mozilla/5.0 ...\r\n     <- 可选头
\r\n                                <- 空行：终止头部
```

HTTP 响应：响应行 + 零个或多个响应头 + **空行** + 可选内容

```
<version> <status code> <status msg>\r\n
<header name>: <header data>\r\n
\r\n
<content bytes>
```

```
HTTP/1.1 200 OK\r\n                 <- 响应行：版本 + 状态码 + 状态消息
Date: Wed, 05 Nov 2014 17:37:26 GMT\r\n
Server: Apache/1.3.42 (Unix)\r\n
Content-Type: text/html; charset=UTF-8\r\n
Content-Length: 4096\r\n            <- 正文长度
\r\n                                <- 空行：头部与正文的分界
<html> ... </html>                  <- 正文
```

关键规则	说明
每一行都以 `\r\n` 结尾	不是 `\n`！这是 HTTP 规范（RFC 1945/7230）的硬性要求
空行 = `\r\n`（一个孤立的 CRLF）	**同时**终止**请求**头部与**响应**头部；读写时必须靠它切分头部与正文
请求行三字段	`<method> <uri> <version>`；`<uri>` 对**代理**是**完整 URL**，对**源服务器**是 **URL 后缀**（路径）
响应行三字段	`<version> <status code> <status msg>`
头部字段	`<header name>: <header data>`，**冒号后有一个空格**
版本	`<version>` ∈ {`HTTP/1.0`, `HTTP/1.1`}
文本行可以是任意长度	所以读头部**必须用 `rio_readlineb` 配足够大的缓冲**，并检查溢出

关键规则	说明
正文长度	靠 `Content-Length` **或** 连接关闭（`Connection: close`）定界；**分块传输（chunked）** 用十六进制长度行（如 `15c`）开头、以 `0` 行结束（讲义 telnet 实例中出现）

- **代理与源服务器的 URI 差异（Proxy Lab 的核心）**： 浏览器发给代理的是 **`GET http://www.cmu.edu/hub/index.html HTTP/1.1`**（**完整 URL**）； 代理转发给源服务器的必须是 **`GET /hub/index.html HTTP/1.0`**（**相对路径**）。 即：代理必须**拆出 hostname 与 path**，把 `HTTP/1.1` **降级为 `HTTP/1.0`**。

A.12.10 HTTP 方法与状态码

请求方法（讲义列出 7 个）：

方法	语义	有请求体	幂等/安全	备注
`GET`	请求指定资源	无	安全、幂等	最常见；Proxy Lab **只要求支持 `GET`**
`POST`	向服务器提交数据	**有**	非幂等	表单提交；Proxy Lab **可选**
`HEAD`	只要**响应头**，不要正文	无	安全、幂等	用于检查资源是否存在/是否改动；**Proxy Lab 不必支持**
`OPTIONS`	查询服务器/资源**支持的方法**	无	安全、幂等	CORS 预检也用它
`PUT`	上传/替换资源	有	幂等	REST 风格
`DELETE`	删除资源	可无	幂等	REST 风格
`TRACE`	回显收到的请求（诊断）	无	安全	常因安全原因被禁用

状态码（讲义直接给出的：200、301、404；下表其余为**补充说明**，均为 RFC 7231 定义的标准码）：

码	消息	含义	何时返回
200	`OK`	请求被**无错误地处理**	正常响应
301	`Moved Permanently`	**永久**重定向，提供**备用 URL**（`Location` 头）	资源永久搬家（讲义 telnet 实例中 `www.cmu.edu/` → `/index.shtml` 就是 301）
302	`Found`	**临时**重定向（`Location` 头）	临时跳转（旧称 `Moved Temporarily`）
400	`Bad Request`	请求**语法错误**、代理无法解析	畸形请求行/头部
403	`Forbidden`	服务器**拒绝**提供该资源（有权限问题或非普通文件）	Tiny 服务器的"文件不可读"分支
404	`Not Found`	**服务器找不到该文件**	资源不存在
500	`Internal Server Error`	服务器**内部错误**	服务器代码抛异常
501	`Not Implemented`	服务器**不支持**该请求方法	收到未实现的方法（如只支持 `GET` 却收到 `POST`）
502	`Bad Gateway`	作为网关/代理时，**从上游收到无效响应**	**Proxy Lab 中 DNS 解析失败或服务器不可达时的常用码**
505	`HTTP Version Not Supported`	**不支持的 HTTP 版本**	请求行版本字段无法识别

- **状态码分段记忆**：**1xx 信息、2xx 成功、3xx 重定向、4xx 客户端错、5xx 服务器错**。
- **Proxy Lab 的错误码映射**：客户端请求无法解析 → `400`；方法不支持 → `501`；版本不支持 → `505`；DNS/连接上游失败 → `502`；且**必须给客户端一个合法的 HTTP 错误响应**（不能直接关连接，也不能让代理退出）。

A.12.11 HTTP 常见头部

头部	方向	作用	代理的处理要点
`Host`	请求	**端服务器的主机名**（HTTP/1.1 必需）	代理**必须发送**（用于虚拟主机）；若浏览器已带，就**沿用浏览器的值**；`Host` 不含端口时用默认 80
`User-Agent`	请求	标识客户端（操作系统、浏览器、引擎版本）	可选；Proxy Lab 给了现成字符串 `User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:10.0.3) Gecko/20120305 Firefox/10.0.3`（讲义中分两行显示，**实际必须作为单独一行发送**）
`Connection`	请求/响应	本次交换后**是否保持连接**	代理**总是发送 `Connection: close`**

头部	方向	作用	代理的处理要点
`Proxy-Connection`	请求	**非标准**头，用于和代理协商连接保持	代理**总是发送** `Proxy-Connection: close`；对端发来的这个头**应丢弃**（不转发给源服务器）
`Content-Length`	响应（也可用于请求）	**正文的精确字节数**	代理**必须精确转发**——少转发会截断对象、多转发会把后续数据当正文；**缓存对象大小也依赖它**
`Content-Type`	响应	正文的 **MIME 类型** + 可选参数（如 `charset=UTF-8`）	原样转发（缓存命中时也必须带）
`Location`	响应	重定向目标 URL	通常**不修改**（注意：重定向目标对代理是新的完整 URL）
`Date` / `Server`	响应	时间戳 / 服务器软件标识	原样转发
`Transfer-Encoding: chunked`	响应	正文采用**分块传输**，无 `Content-Length`	代理如不支持分块，仍可**逐字节透传**直到连接关闭

- **Proxy Lab 的头部规则总结**：① **总是**发送 `Host`；② **总是**发送 `Connection: close` 和 `Proxy-Connection: close`；③ `User-Agent` 可选（提供了常量字符串）；④ **其他浏览器发来的头一律原样转发**；⑤ 必须**丢弃** `Proxy-Connection`（它是给代理看的，不是给源服务器看的）。

A.12.12 MIME 类型表

MIME 类型	含义
`text/html`	HTML 文档（网页）
`text/plain`	未格式化的纯文本
`image/gif`	GIF 格式的二进制图像
`image/png`	PNG 格式的二进制图像
`image/jpeg`	JPEG 格式的二进制图像
`application/pdf`	PDF 文档
`application/octet-stream`	未知/通用二进制流（Tiny 服务器对不认识的扩展名的默认值）

- **MIME = Multipurpose Internet Mail Extensions**：Web 服务器返回的 content 是"一段字节流 + 一个关联的 MIME 类型"。
- 完整列表见 IANA 的 media-types 注册表（`http://www.iana.org/assignments/media-types/media-types.xhtml`）。
- **静态内容 vs 动态内容**：静态 = 存在文件里按请求取回（HTML/图片/音频/JS）；动态 = **按请求现场生成**（服务器为客户端执行程序，约定俗成放在 `cgi-bin` 目录）。二者由服务器**根据 URL 后缀自行判断**。
- **URL 的解剖**：`http://www.cmu.edu:80/index.html` —— 客户端用**前缀** `http://www.cmu.edu:80` 推断**协议（HTTP）、主机、端口**；服务器用**后缀** `/index.html` 判断静态/动态、定位文件。**最小后缀是** `/`，服务器把它扩展为配置的默认文件名（通常是 `index.html`）。

A.12.13 Proxy Lab 要点

总体流程：迭代式代理 → 多线程（每连接一线程）→ 加缓存（读者-写者锁 + LRU 淘汰）。**Gradescope/Autolab 评分 70 分**：

项	分值	内容
BasicCorrectness	40	基本代理功能（自动评分）
Concurrency	15	处理**并发请求**
Cache	15	可工作的**缓存**

① 全 URL vs 相对路径：

位置	请求行里的 URI
浏览器 → 代理	**完整 URL**：`GET http://www.cmu.edu/hub/index.html HTTP/1.1`
代理 → 源服务器	**相对路径**：`GET /hub/index.html HTTP/1.0`（**版本降到 1.0**）

必须从 URL 中解析出 **hostname**（`www.cmu.edu`）、**可选端口**（`http://host:**8080**/path`，缺省 80）、**路径与查询串**（`/hub/index.html`）三部分。**URL 带不带端口号都必须能正常工作**。

② 必须保留 / 丢弃的头部：

动作	头部
必须发送	`Host: <hostname>`（虚拟主机必需；浏览器已带则沿用其值）
必须发送（固定值）	`Connection: close`、`Proxy-Connection: close`
可选发送（固定值）	`User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:10.0.3) Gecko/20120305 Firefox/10.0.3`
必须原样转发	浏览器发来的**其他所有**请求头
必须丢弃	`Proxy-Connection`（给代理看的，不应转发给源服务器）

③ 错误码映射表：

情况	响应
请求行/头部**畸形**、无法解析	`400 Bad Request`
请求方法不是 `GET`（而你只实现 GET）	`501 Not Implemented`
HTTP 版本不受支持	`505 HTTP Version Not Supported`
DNS 解析失败 / 无法连接上游服务器	`502 Bad Gateway`（并**继续服务**其他请求，**绝不退出**）
客户端请求的资源不存在（由上游返回）	直接**透传上游的** `404`

- **健壮性要求（讲义明确）**：代理是**长驻进程**，"对很多错误而言，代理立刻退出显然是不合适的"；必须**对畸形甚至恶意输入鲁棒**、不段错误、**不泄漏内存与文件描述符**；还要能优雅处理 `write` 返回 `EPIPE` 的情况（客户端提前断开）。

④ `Content-Length` 精确转发：

- 正文是**二进制**的，长度必须靠 `Content-Length` 判定，**不能靠** `rio_readlineb` **读行**（正文里可能出现 `\n`，也不一定是文本）。
- 标准做法：`Content-Length: N` → 用 `rio_readnb(connfd, buf, N)` 精确读 N 字节，再 `rio_writen` 精确写出 N 字节。
- 转发给客户端时，`Content-Length` 必须与**实际写出的字节数一致**；缓存命中时也要用缓存里记录的长度，而不是重新 `strlen`。

⑤ 缓存约束（讲义明确规定）：

常量	值	说明
`MAX_CACHE_SIZE`	1 MiB	整个缓存上限；**只统计存 web 对象的字节**，元数据（链表节点、键等）不计入
`MAX_OBJECT_SIZE`	100 KiB	单个对象上限；超过则不缓存

- **边转发边缓存**：从服务器读数据的同时发给客户端，**同时**往缓冲区里累积；**若缓冲超过** `MAX_OBJECT_SIZE` **就丢弃缓冲**（不再尝试缓存这个对象）。
- 这个方案下，代理用于 web 对象的**最大内存**是 `MAX_CACHE_SIZE + T * MAX_OBJECT_SIZE`（T = 最大活跃连接数）= 缓存本身 + 每个活跃连接各一份累积缓冲。
- **淘汰策略**：**近似 LRU** 即可（讲义原文："不必严格 LRU，但应当合理地接近"）；**注意**"读取对象"和"写入对象"**都算使用**。
- **同步（本 lab 最难点）**：缓存访问必须线程安全，但**有硬性要求——多个线程必须能同时读缓存**，只有写才允许独占。因此"**用一把大独占锁保护整个缓存是不可接受的方案**"。可选做法：**分区缓存**、`pthread_rwlock_t` **读者–写者锁**、或**用信号量自己实现读者–写者**。不必严格 LRU 这一点，正好为支持多读者留出腾挪空间。
- **多线程注意**：工作线程必须 **detached**（否则内存泄漏）；`open_clientfd` / `open_listenfd` 基于**可重入的** `getaddrinfo`，因此**线程安全**。

⑥ 调试工具：

工具	用途
`./driver.sh`	**官方自动评分器**（BasicCorrectness + Concurrency + Cache），必须在 **Linux 机器**上运行
`tiny`（CS:APP Tiny Web 服务器）	handout 自带源码；**driver 抓页面就用它**；也是写代理的起点
`telnet localhost <proxy_port>`	手写 HTTP 请求，逐字观察代理转发与响应
`curl -v --proxy http://localhost:<proxy_port> http://localhost:<tiny_port>/home.html`	端到端验证代理链路
`netcat`（`nc -l 12345`）	伪装成服务器，观察代理**实际发出去的字节**（验证头部与请求行）
`./port-for-user.pl <userid>`	生成**个人专属端口号**（**偶数**，如 `45806`）；可用 `p` 与 `p+1` 两个端口；**别自己随便挑端口**，会干扰他人
`valgrind --leak-check=full` / `--track-fds=yes`	查内存与 fd 泄漏（讲义要求"无泄漏"）

工具	用途
`ps -o pid,cmd -L`	观察多线程是否真的创建了线程

- **测试注意**：用**浏览器**测缓存**极不可靠**——现代浏览器自己会缓存、会加条件请求头，所以**必须用 `curl` / `telnet` 或 driver** 来验证缓存命中。
- **两个"环境"常识**：HTTP 请求端口是 URL 里的可选字段（缺省 80）；代理的**监听端口**由命令行参数给出（`./proxy 15213`），必须是 **1024 < p < 65536** 的非特权端口。

A.13 并发与同步速查

> 对应讲次：Lecture 21（`F25-21-concprog.txt`）、Lecture 22（`F25-22-sync-basic.txt`）、 Lecture 23（`F25-23-sync-advanced.txt`）、Lecture 24（`F25-24-parallelism.txt`）； 教材 CS:APP3e 第 12 章；关联 **L7 Proxy Lab**、**L8 SFS Lab**。

A.13.1 三种并发模型对比

维度	基于进程（process-based）	基于 I/O 多路复用/事件驱动（event-based）	基于线程（thread-based）
核心机制	每个连接 `fork` 一个**子进程**	单进程单线程，用 `select` / `poll` / `epoll` 检测就绪 fd，**复用**一个控制流	每个连接一个**线程**（`pthread_create`）
创建开销	**最高**（`fork` 要复制地址空间结构，代价大）	**最低**（无新执行流，只需一个 fd 表项）	**居中**（线程创建远低于进程，但非零）
上下文切换开销	**最高**（切换页表/`%cr3`、TLB 失效）	**极低**（同一进程内，仅函数调用与状态机跳转）	**居中**（同进程换线程栈与寄存器，**不换页表**）
数据共享难度	**最难**：无共享状态；要共享必须显式 IPC（管道/共享内存）。讲义标注：描述符 **不**共享、**文件表共享**、全局变量 **不**共享	**最容易**：一个地址空间，所有连接数据就是普通全局/堆变量	**最容易**：共享代码、数据、堆、内核上下文；**但必须加同步**
控制流复杂度	**低**：每个连接一条顺序控制流，代码直观	**很高**：必须把逻辑写成状态机（"部分 HTTP 头到达怎么处理？"是典型难题）；难做细粒度并发	**中**：每连接一条顺序控制流，但并发交错使调试变难
多核利用	**可以**（多进程天然并行）	**不能**：只有**一个**逻辑控制流，**无法利用多核**	**可以**（多线程并行，最直接）
适用连接数	中（受进程数上限与切换成本约束）	**极大**（C10K 场景的设计选择）	大（受线程栈内存与切换成本约束）
调试友好度	中（可单独 attach 某子进程）	**最好**：单控制流，可**单步调试**	较差（竞态、时序依赖）
典型缺点	需要**回收僵尸子进程**（否则致命内存泄漏）；父子都要**关掉对方那份 fd**（`refcnt(connfd) = 2`）	**编码显著更复杂**；难以提供细粒度并发；单核	需要显式同步（锁、信号量），否则数据竞态
现实代表	传统 Apache prefork、CGI	**nginx**、**Node.js**、Tornado	现代 Apache worker、大多数 Java/C++ 服务

- **进程模型的必做两件事**：① 监听进程必须 **reap 僵尸子进程**；② **父进程** `close(connfd)`、**子进程** `close(listenfd)`——因为内核为每个 socket 维护**引用计数**，`fork` 后 `refcnt(connfd) = 2`，**只有计数归零连接才真正关闭**，不关就会一直占着。
- **线程模型 = 进程模型 + 共享地址空间**：讲义原文说线程模型"与进程模型非常相似，只是用线程代替进程"。
- **混合方案**：**预线程化服务器**（prethreaded，教材 12.5.5）——主线程预先创建线程池 + 共享的有界缓冲区（生产者–消费者），兼得线程的并行与创建开销的摊薄。Proxy Lab 允许用，也可以用"每连接一线程"（**最简单，推荐起步**）。
- **线程模型的进程视图**：进程 = **线程** + 代码/数据/内核上下文。每个线程**独占**：数据寄存器、条件码、栈指针 SP、程序计数器 PC、**自己的栈**（局部变量）；所有线程**共享**：只读代码、读写数据、运行时堆、共享库、**内核上下文（VM 结构、描述符表、`brk` 指针）**。**关键警告：线程栈"不 protected"**——一个线程可以（错误地）读写另一个线程的栈，编译器不会拦你。

A.13.2 `pthread_*` 接口表

接口	原型（`<pthread.h>`）	语义要点
`pthread_create`	`int pthread_create(pthread_t *tid, const pthread_attr_t *attr, void *(*f)(void *), void *arg)`	创建线程运行 `f(arg)`；`attr = NULL` 用默认属性；**成功返回 0，出错返回错误码（不设 `errno`）**
`pthread_join`	`int pthread_join(pthread_t tid, void **retval)`	**等待** `tid` 结束（类似 `waitpid`），并可取回线程返回值；**只有 joinable 线程可以 join**
`pthread_detach`	`int pthread_detach(pthread_t tid)`	把线程设为**分离态**：结束后资源**自动回收，不能被 `join`**。也可用属性 `PTHREAD_CREATE_DETACHED` 在创建时设定。**服务器工作线程必须 detach，否则内存泄漏**
`pthread_exit`	`void pthread_exit(void *retval)`	终止**当前线程**（类同 `exit` 对进程）；`retval` 可被 `join` 取到
`pthread_self`	`pthread_t pthread_self(void)`	返回当前线程 ID（TID），对应 `getpid`
`pthread_once`	`int pthread_once(pthread_once_t *once_control, void (*init)(void))`	**保证 `init` 在多线程下恰好执行一次**（用于一次性的全局初始化）；`once_control` 必须初始化为 `PTHREAD_ONCE_INIT`
`pthread_mutex_init` / `_destroy`	`int pthread_mutex_init(pthread_mutex_t *m, const pthread_mutexattr_t *a)`	动态初始化 / 销毁互斥量
`pthread_mutex_lock` / `_unlock`	`int pthread_mutex_lock(pthread_mutex_t *m)`	加锁 / 解锁；**`unlock` 只能由持锁者调用**，否则未定义行为
`pthread_mutex_trylock`	`int pthread_mutex_trylock(pthread_mutex_t *m)`	**非阻塞**加锁：已被占用则立即返回 `EBUSY`（**死锁避免手段之一**）
`pthread_cond_init` / `_destroy`	`int pthread_cond_init(pthread_cond_t *c, const pthread_condattr_t *a)`	初始化 / 销毁条件变量（销毁时**不应有等待者**）
`pthread_cond_wait`	`int pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m)`	**原子地**释放 `m` 并挂起；被唤醒时**重新获取 `m` 再返回**
`pthread_cond_signal`	`int pthread_cond_signal(pthread_cond_t *c)`	唤醒**至少一个**等待者
`pthread_cond_broadcast`	`int pthread_cond_broadcast(pthread_cond_t *c)`	唤醒**所有**等待者（屏障/多资源场景必须用这个）
`pthread_rwlock_rdlock` / `_wrlock` / `_unlock`	`pthread_rwlock_t`	**读者–写者锁**：`rdlock` 可多个读者共享，`wrlock` 独占；`unlock` 两者通用。**必须由程序员自己判断"哪段代码需要读权限、哪段需要写权限"**
`pthread_cancel`	`int pthread_cancel(pthread_t tid)`	请求取消线程（异步且难以正确使用，**不推荐用于服务器**）

- **线程 API 与进程 API 的对应（讲义对照表）**：`pthread_create` ↔ `fork`；`pthread_join` ↔ `waitpid`；`pthread_self` ↔ `getpid`；`pthread_exit` ↔ `exit`；`return from thread proc` ↔ `return from main`；`pthread_mutex_lock/unlock` **无精确对应物**。
- **返回值约定**：Pthreads 函数**成功返回 0，失败返回非 0 错误码（不使用 `errno`）**。所以 CS:APP 的封装写着 `if ((ret = pthread_create(...)) != 0) unix_error(...)`。
- **`pthread_join` 与 detach 的竞态**：对一个已 detach 的线程 `join`，返回 `EINVAL(22)`，且结果**依赖时序**；所以"要么全部可 join、要么全部 detach"，不要混用。
- **`Pthreads` 是标准接口**：约 60 个函数，`pthread_*` 前缀，是 C 程序操纵线程的事实标准。

A.13.3 `sem_*` 与 `pthread_mutex_*` 对照表

语义	POSIX 信号量（`<semaphore.h>`）	互斥量（`<pthread.h>`）
声明类型	`sem_t s;`	`pthread_mutex_t m;`
初始化（动态）	`sem_init(&s, pshared, val)`；`pshared = 0` 表示**线程间**共享	`pthread_mutex_init(&m, NULL)`
初始化（静态）	**无**（必须先 `sem_init`，否则是垃圾字节）	`pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;`
加锁 / P 操作	`sem_wait(&s)`：$s > 0$ 则 $s -= 1$，否则**阻塞**	`pthread_mutex_lock(&m)`：空闲则锁住，否则阻塞重试
解锁 / V 操作	`sem_post(&s)`：$s += 1$，唤醒一个等待者	`pthread_mutex_unlock(&m)`：**只能由持锁者调用**

语义	POSIX 信号量（`<semaphore.h>`）	互斥量（`<pthread.h>`）
取值	`sem_getvalue(&s, &v)`（非标准语义，仅参考）	无（不透明对象）
销毁	`sem_destroy(&s)`	`pthread_mutex_destroy(&m)`
初值范围	**任意 $val \geq 0$**（计数信号量）	只能 0/1（**二元**）
调用约束	P/V **可以乱序、可跨线程配对**	`unlock` 必须与 `lock` **同线程配对**
语义差异（核心）	`sem_wait` **可在任何时刻**被任何线程调用（初值为 0 时立即阻塞）	`lock` 只在"空闲"时成功；**持锁者之外的人不能解锁**
典型用途	互斥、**计数（资源数）、顺序/调度（wait on event）**	**仅**互斥
相对开销	**更慢**（讲义实测 27.6 s vs mutex 15 s）	较快

- **关键区别一句话**：`sem_wait` **无需"持有"关系，** `unlock` **必须由持锁者调用。** 这就是信号量能做"计数 + 顺序"而互斥量不能的原因。
- **"信号量的真正价值是 wait on event"**：初值为 0 时它就是一个"等待某事件发生"的原语——`sem_post` 由事件产生方调用，`sem_wait` 由等待方调用，**两者可以是不同线程**。生产者–消费者的 `slots` / `items` 就是这一用法的教科书范例。
- **互斥量语义（讲义定义）**：不透明对象，**要么锁定要么未锁定；初始为未锁定**；`lock(m)` 空闲则锁上并返回，否则**等待其变为未锁定后重试**；`unlock(m)` **只能由加锁的代码调用**。它的作用是"对共享变量提供**互斥访问**"，用一条不可违反的不变量把**不安全区域**从状态空间中物理隔离。
- **性能提醒**：讲义明确"**能用 mutex 就别用 semaphore**"——因为信号量更贵，且互斥场景下 `sem_wait/sem_post` 的计数器语义是多余的。反之，**需要计数或顺序时 mutex 做不到**，必须用信号量。

A.13.4 `select` / `poll` / `epoll` 与 `fd_set` 宏

维度	`select`	`poll`	`epoll`
接口	`int select(int nfds, fd_set *r, fd_set *w, fd_set *e, struct timeval *t)`	`int poll(struct pollfd *fds, nfds_t n, int timeout)`	`epoll_create1` + `epoll_ctl` + `epoll_wait`
描述符上限	**`FD_SETSIZE`（通常 1024）——硬上限**	**无上限**（数组由调用者提供）	无上限
数据结构	位图 `fd_set`（**固定大小**）	`struct pollfd` **数组**（`fd` + `events` + `revents`）	**内核中的红黑树 + 就绪链表**
就绪事件的表达	**就地修改** `fd_set`（见下）	写回 `revents` 字段（**输入/输出分离**，不破坏 `events`）	返回就绪数组，**不修改注册表**
每次调用开销	$O(\mathbf{nfds})$，每次都要把整个集合**从用户态拷到内核态**	$O(n)$，同样要拷贝整个数组	$O(1)$ 级：`epoll_wait` 只返回就绪项，**无需重复注册**
时间复杂度	$O(n)$	$O(n)$	$O(\text{就绪数})$（边沿触发需注意处理完）
可扩展性	**差**（大并发下退化严重）	中	**好**（现代高性能服务器的选择）
可移植性	**最好**（POSIX，无处不在）	好（POSIX）	**仅 Linux**（BSD 用 `kqueue`，Windows 用 IOCP）
典型用途	教材/讲义的事件驱动服务器、教学与小型服务	需要突破 1024 上限且要可移植时	nginx、Redis 等生产级事件驱动服务

`fd_set` 操作宏：

宏	作用
`FD_ZERO(&set)`	**清空集合（每次调用 `select` 前必须重新执行）**
`FD_SET(fd, &set)`	把 `fd`（及 `fd + 0` 到 `fd + 7` 共 8 个描述符位）加入集合
`FD_CLR(fd, &set)`	从集合中**移除** `fd`
`FD_ISSET(fd, &set)`	**测试** `fd` 是否在（就绪的）集合中——**返回值非 0 表示该 fd 有事件**
`FD_SETSIZE`	`fd_set` 能表示的**最大描述符数**（Linux 上 1024）

`nfds` 参数 = `maxfd + 1`：`select` 的 `nfds` 是"要检查的**描述符个数**"，等于**集合中最大 fd 加一**（因为描述符从 0 开始编号）。传小了会漏掉大号 fd，传大了浪费扫描时间。

⚠️ **`select` 会就地修改 `fd_set`，每轮必须重建**：

```
用户设置:    rset = {listenfd, connfd1, connfd2}       ← 我想监视这三个
     |
     v
select(nfds, &rset, NULL, NULL, NULL)                   ← 等待
     |
     v
返回后:      rset = {connfd2}                           ← ⚠ 只剩下"就绪"的那些!
     |                                                     原来的 listenfd/connfd1 位被清掉了
     v
必须: FD_ZERO / FD_SET 重新把三个都放回去，才能再调用 select   ← 这就是"每轮重建"
```

- **原因**：`select` 用**同一个** `fd_set` 做"输入 = 监视集合"和"输出 = 就绪集合"（返回时只保留就绪的位）。因此**正确的循环模式**是：每轮先 `FD_ZERO` 再 `FD_SET` 所有关心的 fd（含 `listenfd` 与所有活跃 `connfd`），然后调用 `select`，再用 `FD_ISSET` 逐个检查。
- **对比 `poll` / `epoll` 的优势**：`poll` 的 `revents` 是**独立的输出字段**，所以**不必重建**；`epoll` 更彻底——注册表持久保存在内核里，`epoll_wait` 只是取就绪列表。**这就是"`select` 每轮重建"这件事在现代接口里消失的原因。**
- **事件驱动服务器的主循环（讲义要点）**：① 维护一个**活跃 `connfd` 数组**（含 `listenfd`）；② 每轮确定哪些描述符**有 pending 输入**（`select` 或 `poll`）；③ 若 `listenfd` 有输入 → `accept` 并把新 `connfd` 加入数组；④ 对所有有输入的 `connfd` 逐个服务。**"就绪输入的到来"就是一个事件。**
- **教材实现细节**：讲义明确"基于 `select` 的服务器细节在教材里"——包括"**`listenfd` 和 `connfd` 必须分别用不同的 `fd_set`**"（否则 `listenfd` 就绪后会被随后的 `FD_ISSET` 检查污染）以及"**先处理 `listenfd` 的接受，再处理已连接描述符**"等要点。

A.13.5 生产者–消费者（有界缓冲区）

结构（`sbuf_t`）与三个信号量：

```
+------------------------------+        信号量初值与职责
|  int *buf;                   |        +--------------------------+
|  int n;         (槽位数 N)   |        | sem_t mutex;  init = 1   |  互斥: 保护 buf/front/rear 的访问
|  int front;                  |        | sem_t slots;  init = N   |  计数: 当前"空槽位"数
|  int rear;                   |        | sem_t items;  init = 0   |  计数: 当前"已填 item"数
+------------------------------+        +--------------------------+
       不变量: slots + items = N (= 空槽数 + 已填数 = 总槽数)
```

正确的操作顺序：

角色	顺序	代码
生产者	① 等空位 ② 拿锁 ③ 存 ④ 放锁 ⑤ **通知"有 item 了"**	`sem_wait(&sp->slots); sem_wait(&sp->mutex); sp->buf[(++sp->rear) % n] = item; sem_post(&sp->mutex); sem_post(&sp->items);`
消费者	① 等 item ② 拿锁 ③ 取 ④ 放锁 ⑤ **通知"有空位了"**	`sem_wait(&sp->items); sem_wait(&sp->mutex); item = sp->buf[(++sp->front) % n]; sem_post(&sp->mutex); sem_post(&sp->slots);`

铁律（两句话）：

1. **先等资源（`slots` / `items`），后拿锁（`mutex`）；**
2. **先放锁（`mutex`），后通知（`items` / `slots`）。**

❌ **死锁写法与机制**：

```
/* ❌ 错误: 先拿 mutex 再等 slots —— 必然死锁 */
sem_wait(&sp->mutex);      /* ① 先拿锁 */
sem_wait(&sp->slots);      /* ② 再等空位 —— 卡在这里!  */
sp->buf[(++sp->rear) % n] = item;
sem_post(&sp->mutex);
sem_post(&sp->items);
```

```
生产者:  [已持有 mutex] ──等待──> (slots > 0)
                                    ^
                                    | 只有"消费者取出 item"才能增加 slots
                                    |
消费者:  [等待 mutex] <──需要──── (sem_post(&slots))
            |
            +── mutex 被生产者攥着 => 谁也动不了 => 死锁（循环等待 + 持有并等待）
```

- **为什么 `mutex` 不可省**：`slots` / `items` 只保证**计数语义**（"能不能进"），**不保证对 `buf` / `front` / `rear` 的访问互斥**。两个生产者都成功等待到空位后，都会执行 `++sp->rear`——这是**非原子的读–改–写**，可能算出**同一个下标**，导致一个 item 被覆盖、另一个凭空消失。`slots` / `items` 管"能不能

进"，`mutex` 管"进去以后别撞车"——职责正交，缺一不可。

- **为什么不变量是 `slots + items = N`**：每次生产 `slots` 减 1、`items` 加 1；每次消费反之。故任意时刻"空槽数 + 已填数 = 总槽数"。这个不变量是**验证实现正确性的利器**（可以在每次操作后断言它）。
- **`sem_init` 的第三个参数就是初值**：`sem_init(&sp->mutex, 0, 1)`、`sem_init(&sp->slots, 0, n)`、`sem_init(&sp->items, 0, 0)`——`pshared = 0` 表示**线程间**共享。
- **一个易漏的细节**：忘记 `sem_init` 会让 `sem_t` 是**垃圾字节**，通常表现为立刻死锁或随机唤醒（UB）。
- **条件变量版**：用 `count` + `not_empty` / `not_full` 两个条件变量替代 `items` / `slots`，**必须用 `while` 包裹条件判断**（见 A.13.7）。

A.13.6 读者–写者问题

问题定义：读者线程**只读**对象，写者线程**修改**对象。约束是：**写者必须独占访问；无限多个读者可以同时访问**。现实中频繁出现——**在线订票系统、多线程缓存 Web 代理**（Proxy Lab 的缓存就是它）。

第一类：读者优先（readers preference）——写者可能**饥饿**。

元素	作用
`sem_t mutex`（初值 1）	**保护 `readcnt`** 这个共享计数器
`sem_t w`（初值 1）	**写者锁**，也代表"读权限"
`int readcnt = 0`	当前正在读的读者数

```
/* 读者进入 */                              /* 写者进入 */
sem_wait(&mutex);                          sem_wait(&w);
readcnt++;                                 /* 临界区：写 */
if (readcnt == 1)                          sem_post(&w);
    sem_wait(&w);   /* 第一个读者拦住写者 */
sem_post(&mutex);
/* 临界区：读 */
sem_wait(&mutex);
readcnt--;
if (readcnt == 0)
    sem_post(&w);   /* 最后一个读者放行写者 */
sem_post(&mutex);
```

- **`readcnt` 必须用 `mutex` 保护**：否则两个读者同时 `readcnt++` 会丢失更新，可能出现"`readcnt` 从 1 变 2 但只有一次 `sem_wait(&w)`"或反之——后果是**写者与读者同时进入临界区**（正确性崩溃）或**写者锁永久不放**（死锁）。
- **饥饿的机制**：只要读者源源不断到达，`readcnt` 永远大于 0，`w` 永远不被释放 → 写者**永远等不到**。这是"读者优先"这个名字的由来，也是它的缺陷。
- **Proxy Lab 的注意点**：`pthread_rwlock_t` 的具体策略由实现决定，故讲义把"不必严格 LRU"当作实现多读者并发的腾挪空间。

第二类：写者优先（writers preference）——读者可能**饥饿**。经典做法是加一个**旋转门（turnstile）**信号量：

元素	作用
`sem_t w`（初值 1）	真正的写者锁
`sem_t turnstile`（初值 1）	**旋转门**：写者一到就"关门"挡住后续读者，写完再"开门"
`sem_t mutex` + `readcnt`	同读者优先

```
/* 写者 */                                  /* 读者 */
sem_wait(&turnstile);   /* 关门：挡住新读者 */  sem_wait(&turnstile);   /* 通过旋转门 */
sem_wait(&w);           /* 拿写者锁 */          sem_post(&turnstile);   /* 立刻放行（不长期持有）*/
/* 临界区：写 */                               sem_wait(&mutex);
sem_post(&w);                                  readcnt++;
sem_post(&turnstile);   /* 开门 */             if (readcnt == 1) sem_wait(&w);
                                               sem_post(&mutex);
                                               /* 临界区：读 */
                                               sem_wait(&mutex);
                                               readcnt--;
                                               if (readcnt == 0) sem_post(&w);
                                               sem_post(&mutex);
```

- **`turnstile` 的关键**：读者**只短暂持有**它（进门后立刻 `sem_post`），所以**读者之间仍然可以并发**——这正是它优于"在读者入口直接 `sem_wait(&w)`"的地方。

- ⚠️ **常见的错误写法**：在读者入口直接 `sem_wait(&w)`、出口 `sem_post(&w)`。它**能挡住新读者**，但**把读者之间也串行化了**（每个读者都独占 `w`），彻底丢掉了"多读者并发"这一核心优势。**能跑，但对性能有害。**

饥饿（starvation）与公平（fairness）：

概念	说明
饥饿	一个线程"在**不可接受地长**的时间内没有取得任何进展"
与死锁的区别	**饥饿可能最终解脱**；死锁则**永远**卡住
"不可接受地长"	**取决于应用**——没有客观阈值
公平（fair）	保证**无饥饿**的算法。公平读写锁 = **每个等待者按先来先服务（FCFS）顺序拿到锁**（多个读者可同时拿到）
公平的代价	实现更复杂；且**可能让所有线程都比不公平系统更慢**（典型是 lock convoy problem / 锁 convoy）

A.13.7 条件变量要点

要点	说明
本质	条件变量让线程**在某个条件为假时挂起**，并在条件可能变真时被唤醒——解决"轮询忙等浪费 CPU"与"用信号量硬凑、语义不匹配"的问题
自身不提供互斥	它**总是与一把互斥锁配对使用**。锁保护**条件本身**（谓词），条件变量负责**等待/唤醒**
`pthread_cond_wait(c, m)` 的原子语义	**必须持有 `m` 才能调用**。它会**原子地**：① **释放 `m`**；② 把线程加入 `c` 的等待队列并挂起；③ 被唤醒后**重新获取 `m`**，然后才返回。这三步**不可分割**——如果"释放锁"和"挂起"之间有窗口，就会丢失唤醒（lost wakeup），这正是它存在的全部理由
必须用 `while` 而非 `if`	被唤醒后**必须重新检查条件**。原因有三：① **虚假唤醒（spurious wakeup）**——POSIX 允许 `cond_wait` 无理由返回；② **唤醒被别的线程抢走**（"惊群"后竞争者先拿到锁并消耗了资源）；③ `broadcast` 唤醒所有等待者但只有部分能继续。**`if` 版本会在条件实际为假时继续执行，导致缓冲区溢出/下溢**
`signal` vs `broadcast`	`signal` 只唤醒**至少一个**等待者（单一资源）；`broadcast` 唤醒**全部**（条件对多个等待者同时为真时，如**屏障**、状态整体翻转）
使用模板	等待方：`pthread_mutex_lock(&m); while (!cond) pthread_cond_wait(&c, &m); /* 用资源 */ pthread_mutex_unlock(&m);` 通知方：`pthread_mutex_lock(&m); /* 改状态使 cond 为真 */ pthread_cond_signal(&c); pthread_mutex_unlock(&m);`
`while` 的正确写法	`while (count == 0) pthread_cond_wait(&not_empty, &mutex);` ——谓词写在 `while` 条件里，而非 `if`，也**不是** `while(1) { if (...) break; wait(); }` 之外的错误变体

- **讲义补充（L21 末尾）**：条件变量在 L21/L22 讲义里只作为"**Bonus: 自行查阅**"出现；教材（CS:APP3e 12.5.4 与 12.7）才正式展开。**SFS Lab 与 Proxy Lab 中条件变量与信号量都可用**，但语义不同，不要混用。
- **屏障（barrier）的条件变量实现**：每线程到达时 `count++`；若 `count < N` 则 `wait`，否则 `count = 0` 并 **`broadcast`**——这里必须用 `broadcast`，因为**所有**等待者都该被放行。

A.13.8 线程安全四类函数分类表

类别	根本原因	典型例子	修复方法	代价
第 1 类：不保护共享变量	函数访问了**共享全局/静态变量**却没同步	自己写的 `cnt++` 计数器；**自实现的 `malloc` / `free`**（无内部锁）	在函数**首尾**加 `pthread_mutex_lock/unlock`	加锁**降性能**；多把锁可能引入**死锁**
第 2 类：依赖跨调用保持的状态	函数**跨多次调用**保持状态（静态变量）	`rand`（依赖全局 `next`）、`srand`、`strtok`	**把状态改为参数**由调用者传入（`rand_r(int *nextp)`）	**必须改 API**；调用者负责分配状态空间
第 3 类：返回指向静态变量的指针	函数把结果写进**共享的静态缓冲区**并返回其指针	**`ctime`**、`asctime`、`localtime`、`gethostbyname`、`inet_ntoa`、`itoa`	lock-and-copy：锁内调用并**把结果拷贝到调用者私有存储**；或改用 `strftime` / `getaddrinfo` 这类安全替代	需改签名；调用者要知道缓冲多大
第 4 类：调用了上面三类中任一个	**传递性**——函数自身没毛病，但它调用的函数不安全	任何内部用了 `rand` / `strtok` 的函数	只调用**线程安全的版本**（`_r` 后缀）	可能触发**连锁 API 改动**

- **加锁只能修好第 1 类**。第 2、3 类**加锁治不好**：给 `rand` 内部加锁，两个线程各调 100 次的序列仍然**不同于**单线程调 200 次——锁只保证"不崩"，不保证"可重现"。第 3 类更微妙：`char *s = ctime(&t);` 拿到指针后**锁已释放**，从解锁到使用 `s` 之间存在窗口，别的线程任何一次 `ctime` 都会**覆盖**这块静态缓冲区。

- **可重入（reentrant）与线程安全的关系**：函数**可重入**，当且仅当它被多线程调用时**不访问任何共享变量**。因此**可重入 ⇒ 线程安全**（它压根没有共享状态，**不需要任何同步操作**）；**反之不成立**——加了互斥锁的 `rand` 是线程安全的，却仍然修改全局 `next`，**不可重入**。
- **显式 vs 隐式可重入**：**显式可重入**（explicitly reentrant）= 函数内部**完全不碰**共享变量；**隐式可重入**（implicitly reentrant）= 把共享变量全改成局部变量后，**再通过指针参数由调用者传入**（如 `rand_r(&seed)`）——它的可重入性**依赖调用者是否传入了共享指针**。
- **ISO C 的现状**：绝大多数标准库函数**是线程安全的**（`malloc`、`free`、`printf`、`scanf` 内部有锁）；例外是 `strtok`、`rand`、`asctime`。`printf` / `fprintf` / `puts` 内部用 **lock-and-copy** 保护输出流，避免多线程输出交错；`snprintf` 内部还会调用 `malloc` 取暂存空间。
- **老式 Unix C 库**则大量不安全，通常有 `_r` 后缀的安全替代品。

A.13.9 死锁与避免规则

死锁定义：程序**死锁**，当一组线程中**每一个都在等待一个只能由组内其他线程触发的事件**——最典型是"**相互等待对方持有的锁**"。

```
线程 1                    线程 2
lock(mA)                  lock(mB)          <-- 各自先拿一把
lock(mB)  <-- 等线程2      lock(mA)  <-- 等线程1
...                       ...
unlock(mB)                unlock(mA)
unlock(mA)                unlock(mB)

=> 循环等待：T1 等 mB（T2 持有），T2 等 mA（T1 持有）=> 永久卡住
```

进度图（progress graph）几何解释（讲义核心工具）：

- 横轴 = 线程 1 的进度，纵轴 = 线程 2 的进度；每个**指令**是一个状态，轨迹是一串**状态转换**。
- **禁止区域（forbidden region）**：由**互斥不变量**定义的矩形——"两个线程的临界区不能重叠"。
- **不安全区域（unsafe region）**：禁止区域的并集；**轨迹一旦进入，就必然违反互斥**。
- **死锁区域（deadlock region）**：不安全区域的**子集**，形状像"**左上角的矩形**"——一旦轨迹进入，**既不能向右也不能向上移动**（两个方向都被对方阻塞）。
- 轨迹可以**绕过**禁止区域的**上边或右边**（"always possible to move up or move right"）——这就是**正确的加锁顺序**能让轨迹贴着不安全区域边缘"溜过去"的几何含义。
- **死锁状态**：存在一组线程，每个都在等待一个只能由组内另一个线程触发的事件。
- ⚠️ **重要细节**：**加锁顺序不一致会导致死锁，但解锁顺序不一致完全无所谓**（讲义原文："Inconsistent unlock order does not matter"）——因为解锁不会阻塞。

死锁避免规则	说明
① **加锁顺序全局一致**	若多个线程都要拿多把锁，**所有线程必须按同一个全序**获取（如按锁的地址排序）。这是**最根本**的一条：它把"循环等待"从状态空间中**彻底删除**
② **用 `pthread_mutex_trylock`**	需要多把锁时改用非阻塞版：拿到一把后，若第二把拿不到就**释放已持有的锁并重试**（打破"持有并等待"），避免永久阻塞
③ **用粗粒度锁**	把多把锁合并成**一把大锁**（减少锁数量 → 减少能形成的等待环）。**代价是并发度下降**
④ **避免嵌套加锁**	尽量让临界区里**只持有一把锁**，从根本上消除"持有多锁"这一死锁的必要条件
⑤ 固定"资源 → 锁"顺序	生产者–消费者里的铁律"**先 `sem_wait(&slots)` 再 `sem_wait(&mutex)`**"就是这个规则的具体化（见 A.13.5）
⑥ 别在持锁时做慢操作	持锁期间不做 I/O、不调 `malloc`、不睡眠——否则加长"持有并等待"的窗口，把**饥饿**放大成**死锁**

- **死锁的四个必要条件（经典理论，讲义未列，补充）**：互斥、持有并等待、非抢占、循环等待。**打破任一条即可防死锁**，实践中主要打第 3、4 条。
- **饥饿 vs 死锁**：**死锁**永不解脱；**饥饿**可能最终解脱（见 A.13.6）。
- **lock convoy problem**：追求公平（FCFS）时，锁在线程间"传递"导致频繁上下文切换，**所有线程都比不公平系统更慢**。

A.13.10 伪共享（false sharing）与填充

定义：**不同线程访问同一个缓存块中的"不同字节"**——虽然逻辑上不共享数据，但**缓存一致性维护是以缓存块为单位**的，所以这些线程仍会相互冲突。

```
psum 数组（每个元素 8 字节，缓存块 = 64 字节 = 8 个元素）
+--------------+--------------+--------------+
|  块 m        |  块 m+1      |  ...
| psum[0..7]   | psum[8..15]  |              |
+--------------+--------------+--------------+
  ^      ^
  |      |
线程 0  线程 1     <-- 逻辑上互不相干，物理上却在同一块里"打架"
写 psum[0] 写 psum[8]
```

概念	定义
真共享（true sharing）	**同一个字节**被多个线程读写——必须用同步（锁/原子）保证正确性
伪共享（false sharing）	**不同字节、同一缓存块**被不同线程写。逻辑上无害，**性能上极坏**——"线程们会为了拿到这个块而互相争抢"
后果	缓存块在核间**反复弹跳（ping-pong）**：每次写都要**独占**该块，其他核的副本作废 → 大量一致性流量，加速比崩塌

讲义实测数据（必记）：

实验	相邻累加器	间隔开的累加器	结论
内存累加（`psum[i]` 写在共享数组里）	加速比 **5.0×**	加速比 **13.3×（唯一超过 8 的观测值）**	间隔放置提速 **2.8×**（13.3 / 5.0 ≈ 2.66，讲义表述为"最好间隔版本比最好相邻版本好 2.8 倍"）
寄存器累加（每线程局部 `sum`，最后写一次）	—	加速比 **7.5×**，比最快的内存累加**快 2 倍**	**用寄存器永远最快**

- **为什么间隔能解决**：`spacing` 参数以**缓存块大小**为单位，让 `psum[myid*spacing]` 落在**不同缓存块**里，于是不再伪共享。
- **"证明缓存块大小 = 64"**：讲义由实验推得——**8 字节的值**，"间隔再增大到超过 8 个元素就没有额外收益"，说明块边界在 **8 个 8 字节值 = 64 字节**处。这就是用性能实验**反推硬件参数**的经典手法。
- **填充（padding）到 64 字节**：把需要每线程独占的计数器/状态**补齐或对齐到 64 字节**（`__attribute__((aligned(64)))` 或手工填 `char pad[64 - sizeof(x)]`），确保不同线程的变量落在**不同缓存块**。

三条教训（讲义原话）：

1. **共享内存很昂贵**——要同时警惕**真共享**与**伪共享**；
2. **尽量用寄存器**（记得 Cache Lab 的教训）；
3. **尽量用本地缓存**（local cache）。
4. **`Beware the speedup metric!`（警惕加速比指标！）**：讲义特意提示，**加速比容易被误读**——同一个程序改一下数据布局（相邻 → 间隔），加速比就从 5× 变成 13.3×；而换成一维局部累加又能到 7.5×。所以**报加速比必须同时说明测量条件与基线**。

A.13.11 竞态、进度图与不安全区域

竞态（race）定义：当程序的正确性依赖于一个或多个线程的"不可控的相对执行顺序（时序）"时，就发生了竞态。它的本质是"**共享 + 未同步**"。

竞态形态	例子	修复手段
数据竞态（多个线程读写同一变量）	`cnt++` 在两个线程里各跑 N 次，结果小于 $2N$（**丢失更新**）；讲义"数到 20000"实验	互斥量/信号量保护（第 1 类线程不安全）
传递参数竞态（`pthread_create` 传 `&i` 而非值）	主线程在循环里 `pthread_create(&tid, NULL, f, &i)`，工作线程读到的 `i` 已经是后来的值	**传值**（`(void *)(long)i`）或为每个线程单独分配参数
跨调用的状态竞态	`rand` / `strtok` 的静态状态	改用 `_r` 版本（第 2 类）
返回静态缓冲区竞态	`ctime` 返回指针，锁已释放	lock-and-copy（第 3 类）
生命周期竞态	`pthread_detach` 与 `pthread_join` 竞争 → `join` 返回 `EINVAL(22)`；"join WINS / detach WINS"两种结局都能复现	统一约定：**要么全部可 join、要么全部 detach**
`fork` 与信号处理程序竞态	Shell 中"子进程被 `sigchld` 回收并从作业表删除，父进程却还没 `addjob`"	用 `sigprocmask` **屏蔽** **`SIGCHLD`**，`addjob` 后再解除（讲义 `procmask2.c`）
非线程导致的竞态	讲义明确指出"**并非所有竞态都涉及线程**"——信号处理程序与主程序之间就有竞态	屏蔽信号 / `sigsuspend`

竞态形态	例子	修复手段
无法用互斥解决的竞态	有些竞态"**不能靠互斥解决**"，需要用**拷贝数据**或**信号量**来消除（讲义举了具体例子）	副本/信号量

进度图（progress graph）术语表：

术语	含义
轨迹（trajectory）	状态空间中的一条路径，表示两个线程**实际**的相对推进序列
临界区（critical section）	访问共享变量的代码段，必须互斥执行
禁止区域（forbidden region）	由互斥不变量定义的、**轨迹绝不可进入**的状态集合（"两个线程同时在临界区"）
不安全区域（unsafe region）	禁止区域的并集；进入即违反互斥
安全轨迹（safe trajectory）	**完全绕开**不安全区域的轨迹——正确的同步应该**只允许**这种轨迹
死锁区域（deadlock region）	不安全区域中"进得去出不来"的子集（左上角的矩形）
互斥不变量（mutex invariant）	定义禁止区域的数学条件（如 $s \geq 0$ 或"最多一个线程在临界区"）

- **不变量（invariant）的力量**：加锁不是"防君子"的礼貌，而是**用一条不可违反的不变量，把不安全区域从状态空间中物理隔离**。这是 Lecture 22 的核心结论，也是理解一切同步原语的统一视角。
- **"任意交错"的正确心智模型**：不要试图枚举时序，**要画进度图**——用几何把"所有可能的交错"一次性覆盖，然后证明你的同步**把不安全区域变成了不可达**。

A.13.12 并发与同步常用命令

命令	作用
`valgrind --tool=helgrind ./prog`	**检测数据竞态与锁顺序问题**（讲义实测用它发现了 `ctime` 内部的竞态：`Possible data race during read of size 1 ... __tz_convert ← ctime`）
`valgrind --tool=drd ./prog`	另一款竞态检测器（与 helgrind 侧重不同，可互为补充）
`gcc -fsanitize=thread -g ./prog`	**ThreadSanitizer（TSan）**：编译期插桩的竞态检测，比 valgrind 快得多
`gcc -fsanitize=address -g ./prog`	**AddressSanitizer**：检测越界与 use-after-free（多线程共享数据被提前释放时尤其有用）
`perf stat -e cache-misses,LLC-load-misses ./prog`	量化**伪共享**造成的缓存一致性流量
`perf c2c record ./prog` && `perf c2c report`	**直接定位伪共享的热点缓存行**（哪两个变量/线程在抢同一行）
`ps -o pid,nlwp,cmd -p <pid>`	`nlwp` = 线程数——确认线程真的创建了
`ps -L -p <pid>`	逐线程列出（LWP、状态、CPU 占用）——**排查"某个线程卡在锁上"**
`top -H -p <pid>`	按**线程**显示 CPU 占用
`gdb -p <pid>` + `info threads` / `thread apply all bt`	**死锁排查利器**：一次看清所有线程都卡在哪个 `pthread_mutex_lock` / `sem_wait` 上
`gdb` + `set scheduler-locking on`	单步调试时锁定调度，避免线程乱跑
`strace -f -e trace=clone,futex ./prog`	观察线程创建与 `futex` 系统调用（锁的底层实现）
`ltrace -f ./prog`	跟踪库函数调用（可看到 `pthread_mutex_lock` 的进出与返回值）
`taskset -c 0,1 ./perf_test`	**限制可用核心数**——复现"核心数变化导致加速比异常"的问题

A.14 性能度量速查

> 对应讲次：Lecture 15（`F25-15-optimization.txt`）、Lecture 24（`F25-24-parallelism.txt`）； 教材 CS:APP3e 第 5 章、第 12.6 节；关联 L4 Cache Lab Part B、L5 Malloc Lab、Arch Lab。

A.14.1 CPE 定义与测量要点

定义：CPE（Cycles Per Element，每元素周期数） = 平均每处理一个元素所花的**时钟周期数**。

$$\text{Cycles} = \text{CPE} \times n + \text{Overhead}$$

- **CPE 是拟合直线的斜率**，`Overhead` 是截距。讲义给的示意图里 `psum1` 斜率 9.0、`psum2` 斜率 6.0。
- **为什么不用秒/毫秒**：绝对时间会被**时钟频率**污染——同一份代码在 2.0 GHz 老机器上跑 10 ms、在 4.0 GHz 新机器上跑 5 ms，**不代表代码变好了**。CPE 把频率因子除掉，剩下的才是代码与微架构的真实效率。
- **CPE 的倒数就是硬件吞吐量**：CPE = 1 表示"每周期做一个元素"；CPE = 0.5 表示"**每周期做两个**"——这只有在处理器有**多个功能单元并行工作**时才可能。

测量三条纪律：

#	纪律	原因
①	**n 必须足够大**	让截距 `Overhead` 可忽略，斜率才测得准（否则测到的主要是启动成本）
②	**每个版本重复多次，取最小值（或中位数）**	调度、中断、频率波动**只会让结果变慢、绝不会变快**，所以**最小值最接近真实能力**
③	**用 `clock_gettime(CLOCK_MONOTONIC, ...)` 计时**（或 `clock()`），**并扣除循环外的固定开销**	`CLOCK_MONOTONIC` 不受系统时间调整影响；讲义的 `fsecs` 工具把这些纪律固化成可复用的计时框架
④	（建议）**标定实际主频**	标称 3.7 GHz 是最大加速频率，**持续负载下只有 3.07 GHz**；不标定会让 CPE **整体偏差 20%**

```
/* 计时骨架：讲义 fsecs 的思路 */
#include <time.h>
#include <stdio.h>

static double now_sec(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);          /* 单调时钟，不受校时影响 */
    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

int main(void)
{
    const long n = 1L << 20;                      /* n 足够大，压过 Overhead */
    double best = 1e30;
    for (int rep = 0; rep < 15; rep++) {          /* 重复多次 */
        double t0 = now_sec();
        volatile long s = 0;
        for (long i = 0; i < n; i++) s += i;      /* 被测内核 */
        double dt = now_sec() - t0;
        if (dt < best) best = dt;                 /* 取最小值 */
    }
    /* CPE = 周期数 / n；周期数 = 时间 * 实际主频（需自己标定） */
    printf("best time = %.6f s for n = %ld\n", best, n);
    return 0;
}
```

- **报告规范**：讲义/笔记的实践是**每版本重复 15 次，报告 `CPE_min` 与 `CPE_med`**。
- **`-O` 级别必须写明**：同一份代码在 `-O1` / `-O2` / `-O3 -mavx2` 下 CPE 差异巨大（见 A.14.2 表格）。

A.14.2 优化阶梯表

讲义原始数据（CPE，Haswell）：

方法	int add	int mult	double add	double mult
`combine1` 未优化	22.68	20.02	19.98	20.18
`combine1 -O1`	10.12	10.12	10.17	11.14
`combine1 -O3`	4.5	4.5	6	7.8
`combine4`（局部变量累加）	**1.27**	3.01	3.01	5.01

方法	int add	int mult	double add	double mult
循环展开 + 多累加器（unroll）	0.81	1.51	1.51	2.51

优化阶梯（每一步消除什么、收益多少）：

步	优化	消除的开销	典型收益
0	基线 `combine1`（每轮调 `vec_length` + `get_vec_element`）	—	CPE ≈ **20**（最慢）
1	**消除循环低效**：把 `vec_length(v)` **提到循环外**；消除每次迭代的**边界检查**	每次迭代重复的**过程调用**与长度计算	`-O1` 后 CPE ≈ **10**（**减半**）
2	**减少过程调用**：不要用 `get_vec_element` 取值，直接访问数组（或让编译器内联）	每次迭代的 `call` / `ret`、参数传递、栈帧开销	与第 1 步合并见效
3	**消除内存引用**：**用局部变量累积**（`data_t val = IDENT; ... val = val OP d[i]; ... *dest = val;`）	每轮对 `*dest` 的**读-改-写内存访问**（延迟远高于寄存器）	CPE 从 10 降到 **1.27**（int add）——**十倍跃降**
4	**循环展开（loop unrolling）**：一次迭代处理 k 个元素（如 4 个），摊薄分支与循环控制开销	循环控制（比较、分支）的**每元素摊分**	CPE 再降（int add 1.27 → **0.81**）
5	**多个累加器（multiple accumulators）**：用 k 个独立的局部累加器打破**串行依赖链**	关键路径上的**延迟串行化**（见 A.14.4）	突破**延迟界限**，逼近**吞吐量界限**
6	**SIMD / 向量化**（`-mavx2` 等）与**重新结合（reassociation）**	每指令处理更多元素	讲义"**每周期 4–16 个元素**"

- **讲义总结**："编译器优化是**容易获得的收益**：**20 CPE 降到 3–5 CPE**；而配合仔细的手工调优与计算机体系结构知识，可以做到**每周期 4–16 个元素**——最新的编译器正在缩小这个差距。"

为什么第 3 步能降十倍（机制透视）：逐条数一数：v1 每轮 **2 次函数调用 + 1 次栈访问 + 1 次内存读改写**；v2 每轮 **1 条 `addq` 内存操作数 + 指针递增 + 比较 + 分支**。函数调用要压栈、跳转、返回，而**内存读改写的延迟远高于寄存器操作**。注意 **`-O1` 已足够让 GCC 把局部变量放进寄存器**——因为它的地址**从未逃逸（never escapes）**。

- **"做什么"与"不做什么"**：讲义强调优化要有**优先级**——① 好的编译器与正确的编译选项；② **不做任何次优的事**（警惕**隐藏的算法级低效**、写**编译器友好**的代码、警惕**优化阻碍**）；③ **盯紧最内层循环**（绝大部分工作在这里）；④ **针对机器调优**（利用指令级并行、**避免不可预测的分支**）。

A.14.3 内存别名（aliasing）与 `restrict`

问题：编译器无法判断两个**指针**是否指向同一块内存，于是不敢做某些优化（不敢把"先写 A 再读 B"重排、不敢把跨迭代的读提到循环外）。

```
/* 讲义原例：求和矩阵每一行 */
void sum_rows2(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        double val = 0;
        for (j = 0; j < n; j++)
            val += a[i*n + j];        /* 累加在寄存器里 */
        b[i] = val;                   /* 写 b —— 编译器担心 b 与 a 别名！ */
    }
}
```

手段	做法	效果
`restrict` 关键字	`void sum_rows2(double *restrict a, double *restrict b, long n)`	**告诉编译器"这是指向该内存位置的唯一指针"**，于是可以放心优化。讲义明确把 `restrict` 列为"消除别名惩罚"的手段
局部变量累积	用 `val` 累加而不是直接写 `b[i]`	把 n 次内存写变成 **1 次**，同时降低别名风险
`-fno-strict-aliasing` / `-O3`	编译选项	反过来：若代码确实依赖别名行为，需要**关掉**基于 strict aliasing 的优化

- **`restrict` 是"程序员的承诺"**：一旦你**撒谎**（两块内存实际重叠），行为**未定义**（UB）。这是典型的"以正确性换性能"的契约。
- **优化阻碍（optimization blockers）两大类**：**过程调用** 与 **内存引用**。很多情况下二者**互相纠缠**——`get_vec_element` 是过程调用，`*dest` 是内存引用；`strlen(s)` 在循环条件里则同时是"过程调用**加上**无法外提的内存依赖"，导致**算法级**的二次复杂度（见下）。

过程调用阻碍优化的经典案例（讲义原例）：

```
/* O(n^2)：每轮迭代都重新调 strlen(s) */
void lower_quadratic(char *s) {
    size_t i;
    for (i = 0; i < strlen(s); i++)        /* ⚠️ 每轮都重算字符串长度 */
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] += 'a' - 'A';
}

/* O(n)：把长度提到循环外 */
void lower_linear(char *s) {
    size_t i, n = strlen(s);
    for (i = 0; i < n; i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] += 'a' - 'A';
}
```

- **编译器为什么不敢自动外提 `strlen(s)`**：循环体内 `s[i] = ...` **修改了字符串**，编译器无法证明"长度不会变"，因此**不能**把 `strlen` 当成循环不变量移出去。**这种"算法级低效"编译器救不了你，只能靠人**。

A.14.4 延迟界限 vs 吞吐量界限

两个界限的公式：

$$\mathrm{CPE}_{\mathrm{lat}} = \frac{L}{k} \qquad \mathrm{CPE}_{\mathrm{thr}} = T = \frac{1/C}{} \qquad \mathrm{CPE} \approx \max\left(\frac{L}{k},\, T\right)$$

符号	含义
L	该运算的**延迟（latency）**——一条指令从操作数就绪到结果可用所需周期数
k	**累加器个数**（独立的功能单元通道数）
T	**发射时间（issue time / throughput）**——连续两条同类指令之间至少要隔的周期数
C	该类功能单元的**个数**（并发度），故 $T = 1/C$
$\mathrm{CPE}_{\mathrm{lat}} = L/k$	**延迟界限**：受**关键路径串行延迟**限制
$\mathrm{CPE}_{\mathrm{thr}} = T$	**吞吐量界限**：受**功能单元数量**限制

为什么只有一个累加器时 CPE = L：

```
x = x OP d[i];   x = x OP d[i+1];   x = x OP d[i+2];   ...
|___________|        ^
     必须等上一步算完    |
     ==> 纯串行依赖链 ==> CPE = L（与功能单元个数无关！）
```

多累加器如何突破延迟界限：

```
累加器 1:  x0 = x0 OP d[i]    x0 = x0 OP d[i+2]   ...   \
累加器 2:  x1 = x1 OP d[i+1]  x1 = x1 OP d[i+3]   ...    } k 条独立的依赖链
                                                          /   ==> 关键路径缩短 k 倍
最后:      *dest = x0 OP x1  (再串行合并 k 个累加器)
```

讲义给出的具体数值分析（$L = 5,\ T = 1$）：

累加器个数 k	$\mathrm{CPE} = \max(L/k, T)$	处于哪个界限
1	$\max(5/1, 1) = 5$	**纯延迟受限**
2	$\max(5/2, 1) = 2.5$	延迟受限（改善中）
4	$\max(5/4, 1) = 1.25$	接近吞吐界限
5	$\max(5/5, 1) = 1$	**到达吞吐量界限**
⩾ 5	$\max(5/k, 1) = 1$	**再加累加器毫无收益**

- **核心结论**：**累加器个数到达 $k_{\mathrm{opt}} = L/T = L \cdot C$ 之后就封顶了**——此时受限的不再是延迟，而是**功能单元的数量**。继续增加累加器只会造成**寄存器压力**（见 A.14.5），得不偿失。

- **循环展开（unrolling）与多累加器的分工**：**循环展开**主要摊薄**循环控制开销**（分支、比较、指针递增）并**为调度器提供更多可重排指令**；**多累加器**专门用于**打破依赖链、降低关键路径**。二者常常一起用，且**必须先展开才能塞进多个累加器**。
- **调度（scheduling）的作用**：讲义明确说"重排指令让 CPU 更容易**让所有功能单元保持忙碌**"，例如"**把所有 load 提到展开循环的顶部**"——讲义还顺带点出"现在也许更容易理解**为什么我们需要很多寄存器**"（load 提前需要更多寄存器同时存活跃值）。
- **展开后的真实汇编（讲义原例，整数 2 路展开）**：`addq (%rdx), %rcx; addq $16, %rdx; addq -8(%rdx), %rdi; cmpq %r8, %rdx; jne .L3` —— **两条 `addq` 之间没有依赖**（分别累积到 `%rcx` 与 `%rdi`），所以可以每周期发射多条，这就是"**每周期多条指令（Multiple instructions every cycle!）**"的来源。

A.14.5 寄存器溢出（register spilling）

项目	说明
定义	当活跃变量（live values）的数量**超过可用的物理寄存器数**时，编译器只能把一些变量**溢出（spill）到栈上**，用时再重新载入
触发原因	**过度展开 + 过多累加器**：展开 k 路并配 k 个累加器，就需要 k 个累加器寄存器 + k 个载入暂存寄存器 + 指针 + 边界，**寄存器很快用完**
代价	每次溢出/重载都是**内存访问**——正是优化阶梯第 3 步刚消除的东西。结果：**CPE 不降反升**，可能出现"展开越多越慢"的**反常现象**
观测方法	① 看 `gcc -S` 的汇编里是否出现额外的 `movq ... -N(%rbp)` / `movq -N(%rbp), ...` 进出栈帧；② 看 `perf stat -e ...`；③ **实测 CPE 随展开因子 k 的曲线**——通常先降后升，拐点就是寄存器上限
应对	**找到最优展开因子**（经验值：x86-64 上 4–8 路较常见，但要实测）；减少同时活跃的变量；**让编译器自己决定**（写清晰的代码 + `-O2` / `-O3`，现代编译器往往比手工展开更懂寄存器分配）
与多累加器的关系	多累加器的收益**有上限**（A.14.4 的 $k_{\text{opt}} = L/T$）；超过这个上限再"多"就纯属增加寄存器压力，**必然引发溢出**

- **一句话**：**展开与多累加器是"用寄存器换并行度"的交易**，而寄存器是**有限的**——交易存在**最优点**，过了最优点就开始亏。

A.14.6 Haswell 关键功能单元的延迟与发射时间表

本节从略，见 Lecture 15。

> **说明**：Fall 2025 的 Lecture 15 讲义**未包含** Haswell 各功能单元延迟/发射时间的表格（教材 CS:APP3e 第 5.7 与 5.12 节有完整的"Latency, Issue time, Capacity"表，本讲义的幻灯片只保留了从该表**推导出的结论**）。为避免编造数据，此处不列具体数值。讲义中**确实给出**并被引用的参数只有一处：**双精度浮点加的延迟 $L = 5$、发射时间 $T = 1$**（用于 A.14.4 的 $\max(L/k, T)$ 计算）。完整表格请查教材 Figure 5.12 / 5.16。

A.14.7 Amdahl 定律与强/弱扩展

Amdahl 定律（讲义口径）：

$$T_k = \frac{pT}{k} + (1-p)T \quad \Longrightarrow \quad S_k = \frac{T}{T_k} = \frac{1}{(1-p) + \frac{p}{k}}$$

$$T_\infty = (1-p)T \quad \Longrightarrow \quad S_\infty = \frac{1}{1-p}$$

符号	含义
T	问题的**总串行时间**
p	**可被加速的部分占总时间的比例**（$0 \le p \le 1$）
k	**加速因子**（并行度/核心数）
T_k	加速后的时间：**可加速部分快 k 倍 + 不可加速部分不变**
T_∞	$k \to \infty$ 的极限时间
$S_\infty = 1/(1-p)$	**最大可能加速比**——由**不可并行部分**（串行瓶颈）决定

讲义的两个算例：

算例	数据	计算	结论
旅行类比	$T = 7.5$ 小时；$p = 6/7.5 = 0.8$（飞行部分）；$k \to \infty$	$T_\infty = (1-0.8) \times 7.5 = 1.5$ 小时	**最大加速比 5×**——"即使有 FTL，也必须先到纽约"（必须先飞 PIT→JFK 的 1.5 小时）

算例	数据	计算	结论
数值例	$T=10$，$p=0.9$，$k=9$	$T_9=0.9\times\frac{10}{9}+0.1\times10=1.0+1.0=2.0$	5× 加速比（讲义原文如此标注；实际 $10/2=5$，与 T_9 的数值一致）
同上，极限	$k\to\infty$	$T_\infty=0.1\times10=1.0$	10× 上限——"即便有无限的并行计算资源"

- **Amdahl 的本质**："极限加速比揭示的是算法层面的限制（algorithmic limitation），而不是硬件的" ——加再多核也救不了串行部分。讲义用**并行快速排序**做例证：**顶层划分无法加速**，第二层最多 2×，第 k 层最多 2^{k-1}×——**串行瓶颈在递归树的顶端**。
- **对并行程序的实践含义**：优化并行程序时，**先减少串行部分**（$1-p$）往往比增加并行度更有效；此外还要注意**真共享/伪共享全局数据**、**同步开销**、**负载不均**（讲义列的"Beware of Amdahl's Law"提醒）。

Gustafson 定律（**补充说明**：F25 Lecture 24 讲义**未涉及** Gustafson 定律，教材第 12.6 节有）：

$$S_p=(1-\alpha)+\alpha\cdot p \qquad \text{（在"问题规模随核心数增长"的前提下）}$$

定律	隐含假设	视角	结论
Amdahl	问题规模固定，串行部分固定	强扩展（strong scaling）：**同一个问题**加更多核	加速比有**上限** $1/(1-p)$
Gustafson	单核上的执行时间固定，问题规模随 p 增长	弱扩展（weak scaling）：**更大的问题**用更多核在同一时间内解决	加速比可**随 p 线性增长**（因为串行部分占比被摊薄）

- **两者的分歧不是数学矛盾，而是假设不同**：Amdahl 说"**固定的工作量**加核有上限"，Gustafson 说"**核多了就做更大的工作量**，则几乎线性加速"。这正是**强扩展 vs 弱扩展**的经典分野——**报加速比必须说明是哪种**。

A.14.8 加速比、效率与扩展性

$$S_p=\frac{T_1}{T_p} \qquad E_p=\frac{S_p}{p}=\frac{T_1}{p\cdot T_p} \qquad 0<E_p\le1$$

量	定义	理想值	说明
加速比 S_p	T_1/T_p（**单核时间 / p 核时间**）	p（线性加速）	$S_p>p$ 称为**超线性加速（superlinear）**，通常来自**缓存效应**（p 核时总缓存更大，工作集能装下）而非并行本身
效率 E_p	S_p/p	1（100%）	效率下降的原因：串行部分、同步开销、负载不均、通信/一致性流量
强扩展（strong scaling）	**问题规模固定**，增加核心数	—	受 **Amdahl 定律**约束，S_p 很快饱和
弱扩展（weak scaling）	**每核的问题规模固定**，总规模随核心数**同步增长**	—	更接近 **Gustafson 定律**，S_p 可近似线性

讲义实测数据（用于校验你的理解）：

实验	配置	加速比
讲义某并行实验的**理论值**	16 核	**16×**
实测**最佳**（某内核）	16 核	**2.86×**（远低于理论值——同步/串行瓶颈）
另一实验	16 核	**6.84×**
内存累加（相邻 `psum[i]`）	—	**5.0×**
内存累加（间隔开的 `psum[i]`）	—	**13.3×（唯一 > 8 的观测值）**
寄存器累加（每线程局部变量）	—	**7.5×**（比最快的内存累加**快 2 倍**）

- **"Beware the speedup metric!"**：同一程序只改**数据布局**，加速比就从 5× 变 13.3×；换个**累加方式**又能到 7.5×。所以**加速比必须与测量条件、基线、核心数、数据规模一起报告**，否则毫无意义。
- **超线性加速的一个真实来源**：Cache Lab 的教训——**用寄存器/本地缓存永远最快**（讲义 "Lessons learned" 第 2、3 条）。多核时每个核有独立的 L1/L2，工作集被切小后命中率上升，可能出现 $S_p>p$。

A.14.9 常用性能工具与命令

工具/命令	作用
`gcc -O2 -S -masm=att prog.c`	生成汇编，**逐行核对编译器到底做了什么**（有没有把循环不变量外提、有没有向量化、有没有溢出）
`gcc -O3 -march=native -mavx2 -S`	看**向量化**后的汇编（`vmovsd` / `vaddsd` / `vfmadd` 等）
`gcc -O2 -fopt-info-vec-optimized prog.c`	让 GCC **报告哪些循环被向量化**（`-fopt-info-vec-missed` 报告没被向量化的原因，**极其实用**）
`perf stat -e cycles,instructions,cache-misses,branch-misses ./prog`	一站式硬件计数器：周期数、指令数、缓存未命中、分支预测失败
`perf stat -e cycles,instructions` → IPC = instructions/cycles	判断"是否受限于指令发射"还是"受限于延迟/访存"（IPC < 1 通常意味着停顿）
`perf record -g ./prog` && `perf report`	采样剖析，定位**热点函数**（比 `gprof` 更准，因为包含库调用）
`perf c2c record ./prog` && `perf c2c report`	**直接定位伪共享的缓存行**（A.13.10）
`gprof ./prog gmon.out`	传统**插桩**剖析（讲义提示 Malloc Lab 可用）；只统计主程序，且对短函数有偏差
`valgrind --tool=cachegrind ./prog` + `cg_annotate`	**模拟**各级缓存的命中率，按行/函数展开
`getconf LEVEL1_DCACHE_LINESIZE`	直接问系统**缓存行大小**（验证 A.13.10 的"64 字节"结论）
`lscpu` / `lscpu -C`	查看 CPU 型号、频率、各级缓存容量与相联度
`taskset -c 0 ./prog`	**固定到单核**——测 T_1、做"单核基线"时的必需操作
`time ./prog`（bash 内建）	粗看 `real` / `user` / `sys`；`user < real` 说明**没有充分利用多核**
`/usr/bin/time -v ./prog`	输出 `Maximum resident set size`、`Page faults`、`Context switches` 等
`likwid-perfctr -g MEM ./prog`（若安装）	更细的内存带宽/缓存计数器（教学环境不一定有）
`numactl --hardware` / `numactl --cpunodebind`	查看与绑定 NUMA 节点（多路机器上跨节点访存会显著拖慢性能）

- **测量的黄金法则：先量化，再优化。**用 `perf stat` 判断瓶颈类别（**计算受限 / 延迟受限 / 访存受限 / 分支受限**），再选择对应手段——**不要凭直觉改代码**。

A.15 Lab 速查

> 来源：教材官网 https://csapp.cs.cmu.edu/3e/labs.html 的 11 套官方 handout 与 writeup （`F25-.../LAB-*.txt`）+ CMU 15–213 Fall 2026 课程 Labs 页面。 权重与日期取自 Fall 2026 课程页面；handout 内部的分值分布取自各 lab 的官方 writeup。

A.15.1 11 个 lab 总表

#	Lab	对应讲次	官方工具	最容易踩的坑
0	C Programming（L0，热身）	Lecture 1–2	`make`、课程自带测试	改动**不该改的文件**；不读 README 就动手；C 语言基础（指针、字符串、`malloc`）不牢
1	Data Lab	Lecture 2	**`btest`**、**`dlc`**、**`driver.pl`**、`ishow`、`fshow`	用了**被禁的运算符/控制流**（整数题只准**直线代码**，不准循环/条件）；常量**超过 8 位**；`dlc` 报的**运算符计数超标**
2	Bomb Lab	Lecture 3–5（+ GDB）	`objdump -d`、`gdb`、（评分服务器）	用 `strings` 直接找答案（**学不到东西**）；**触雷扣分**（每次爆炸 **−1/2 分，最多扣 20 分**）；不先定位 `phase_1`…`phase_6` 就乱试
3	Attack Lab	Lecture 5–6	`hex2raw`、`ctarget`、`rtarget`、`farm.c`	**忘了先画栈帧**（`getbuf` 帧、`buf` 起始地址、返回地址偏移）；ROP 阶段没在 `.rodata` / `farm` 里用 `objdump` 找 gadget；在**与生成 target 不同的机器**上做题（地址对不上）
4（遗留）	Buffer Lab（IA32）	Lecture 5–6	`hex2raw`、`bufbomb`	与 Attack Lab 同类；**已被 64 位 Attack Lab 取代，自学优先做 64 位**

#	Lab	对应讲次	官方工具	最容易踩的坑
5	Arch Lab	Lecture 3–4 + 教材 Ch.4	`make`、**`benchmark.pl`**、**`correctness.pl`**、`yis` / `ssim` / `psim`	直接**用现成汇编**而不先想清楚数据通路；Part C 的 **`ncopy.ys`** 只对**部分块长度**正确；改 `pipe-full.hcl` 时**破坏回归测试**
6	Cache Lab	Lecture 9–10、15	**`csim-ref`**、**`test-csim`**、**`test-trans`**、**`tracegen`**	Part A：**LRU 用链表导致 trace 大时过慢**（time 测量超时）；Part B：**越界使用数组**（只准**至多 12 个局部 int、不准定义数组**）；**只对 32×32 有效、64×64 崩**
7	Performance Lab	Lecture 9–10、15	`driver.c`、`fcyc.c`、`clock.c`、`kernels.c`	只改 `kernels.c` 的限制下**改错文件**；**只对测试尺寸正确**（尺寸是 32 的倍数但**其他尺寸也要对**）；忘了"**无 credit 于 buggy 代码**"
8	Shell Lab	Lecture 16–18	**`tsh`**（骨架）、**`tshref`**（参考）、**`sdriver.pl`**、`trace01-16.txt`、`myspin` / `mysplit` / `myint`	**没用 `setpgid(0,0)`**；信号转发**忘了用 `-pid`**（`sdriver.pl` **专门测这个**）；`fork` 前**没屏蔽 `SIGCHLD`**（竞态）；**僵尸进程没回收**；在 `waitfg` 里也调 `waitpid`
9	Malloc Lab	Lecture 11–14	**`mdriver`**（`-V` / `-v` / `-f` / `-t` / `-l` / `-h`）、`gprof`	**违反三条禁令**（改接口、调 `malloc` / `sbrk`、定义全局复合结构）；**利用率与吞吐率一头偏**（记得 $w = 0.6$ 偏向利用率）；`short1,2-bal.rep` 小 trace 不用，非得在大 trace 上死磕
10	Proxy Lab	Lecture 18–21	**`driver.sh`**、CS:APP Tiny 服务器、`port-for-user.pl`、`curl`、`telnet`	**不转发完整 URL → 相对路径的转换**；**漏发 `Host` / `Connection` / `Proxy-Connection`**；`Content-Length` **不精确转发**；**用一把大锁保护整个缓存**（明确不可接受）；**用浏览器测缓存**（浏览器自己会缓存）；**代理遇错就退出**（必须长驻）
11	SFS Lab（CMU 自加）	Lecture 18–19、21–23	CMU 自研 handout（教材官网**无**）	这是 CMU Fall 2025/2026 新增的第 9 个 lab（课程代号 **L8**）；**不是**教材官网 11 个 lab 之一。**超级块 / inode 位图 / 目录条目全是共享资源，锁错一处文件系统静默损坏**

> **两个"取代关系"**：**Buffer Lab**（32 位）已被 **Attack Lab**（64 位）取代；**Arch Lab** 另有 Legacy Y86（32 位）版本。**自学一律优先 64 位版本。**
> **关于 SFS Lab 的定位**：教材官网列表到 Proxy Lab 为止（共 11 个）；SFS 是 CMU 自研的第 9 个课程 lab。本文档把它列在末尾并明确标注这一点。

A.15.2 各 lab 的官方评分阈值与关键约束

① **Data Lab**（官方 writeup）

项	分值
Correctness	36
Performance	26
Style	5
合计	**67**

- 13 个编程题（puzzle），难度评级 **1–4 分**；只有**功能与性能都达标**才拿满分。
- **限制**：整数题只准**直线代码（不准循环、不准条件）**；只准用 **8 个运算符** `! ~ & ^ | + << >>`（部分函数进一步限制）；**不允许使用超过 8 位的常量**。
- 工具：`./btest`（功能正确性，`-f <func>` 测单个，`-1 4 -2 5` 传参）、`./dlc bits.c`（**检查是否违规**）、`./dlc -e bits.c`（**打印每个函数的运算符计数**）、`./driver.pl`（**汇总 btest + dlc 算出成绩**）、`./ishow` / `./fshow`（看位与浮点结构）。
- **必须每次改完 `bits.c` 就重新 `make` /重建 `btest`**。

② **Bomb Lab**

阶段	分值
第 1–4 关	**每关 10 分**
第 5、6 关	**每关 15 分**
满分	**70 分**
扣分	**每次爆炸 −1/2 分，最多扣 20 分**

③ **Attack Lab**（官方 Figure 1，满分 100）

阶段	程序	关	方法	函数	分值
1	`CTARGET`	1	**CI**（代码注入）	`touch1`	10
2	`CTARGET`	2	CI	`touch2`	25
3	`CTARGET`	3	CI	`touch3`	25
4	`RTARGET`	2	**ROP**	`touch2`	35
5	`RTARGET`	3	ROP	`touch3`	5

- **关键约束**：只能从 `rtarget` 中 `start_farm` 与 `end_farm` **之间**的地址构造 gadget；**必须在与生成 target 相似的机器上做题**；`hex2raw` 把文本转成攻击字符串。

④ **Arch Lab**（满分 **190**）——Part A **30** + Part B **35** + Part C **100**

- Part A：30 分（3 个 Y86-64 程序，**各 10 分**）。
- Part B：iaddq 指令描述 **10** + `y86-code` 回归 **10** + `ptest` 回归 **15** = **35 分**。
- Part C：描述性注释 **20**（`ncopy.ys` 与 `pipe-full.hcl` 头部各 10）+ **性能 60**。
- **Part C 性能评分公式**（c = 平均 CPE）：

$$S = \begin{cases} 0, & c > 10.5 \\ 20 \cdot (10.5 - c), & 7.50 \le c \le 10.50 \\ 60, & c < 7.50 \end{cases}$$

- 基线 `ncopy` 的平均 CPE **15.18**（范围 14.27–29.00）；**能做到平均 CPE < 9.00**；官方**最佳版本 7.48**。
- 工具：`./benchmark.pl`（测 CPE，**不检查正确性**）、`./correctness.pl`（必须用它查正确性）、`-f` 指定别的文件名、`-h` 看全部参数。

⑤ **Cache Lab**（Part A + Part B）

- Part A 每测例：**正确报告 hits/misses/evictions 各占 1/3 分**（如某测例 3 分，hits 与 misses 对而 evictions 错 → 得 2 分）。评测用法示例：`./csim -s 2 -E 4 -b 3 -t traces/trans.trace`、`./csim -s 5 -E 1 -b 5 -t traces/long.trace`。
- Part B 用 valgrind 抽地址 trace，再用参考模拟器回放，缓存参数固定为 $(s = 5, E = 1, b = 5)$（即 **32 组、直接映射、32 字节块 = 1 KB**）：

矩阵尺寸	满分条件	0 分条件	满分
32 × 32	$m < 300$	$m > 600$	8 分
64 × 64	$m < 1300$	$m > 2000$	8 分
61 × 67	$m < 2000$	$m > 3000$	10 分

- 分数在阈值之间**线性插值**；**代码必须先正确才给性能分**；**允许对三种尺寸分别写专门代码**。Style **7 分**（助教会检查**非法数组**与**过多局部变量**）。
- 工具：`./test-csim`（Part A 自测）、`./test-trans -M 32 -N 32`（Part B，**并生成 `trace.fi`**）、`./csim-ref -v -s 5 -E 1 -b 5 -t trace.f0`（查看每次访问的 hit/miss/eviction）。

⑥ **Performance Lab**（rotate 50% + smooth 50%）

- 每个内核：**正确性**（buggy 代码 **0 分**；除测试尺寸外**其他尺寸也要正确**；可假定图像边长是 **32 的倍数**）+ CPE（达阈值 S_r / S_s 得满分，优于朴素版得部分分；官方常用**线性标度，最低约 40%**）。
- 只允许改 `kernels.c`；**可定义宏、额外全局变量、额外过程**。

⑦ **Shell Lab**（满分 **90**）

项	分值
说明：16 个 trace 文件，每个 5 分	80
Style：注释 5 + **检查每个系统调用的返回值** 5	10

- **必须做到**：提示符是 `"tsh> "`；支持**内置命令** `quit` / `jobs` / `bg <job>` / `fg <job>`；JID 用 `%` 前缀（如 `%5`）；`&` 后缀表示**后台**；**回收所有僵尸子进程**；子进程**因未捕获信号终止时要打印消息**（含 PID）；**不需支持管道 `|` 与重定向 `<` `>`**。
- **用法**：`./sdriver.pl -t trace01.txt -s ./tsh -a "-p"`（`-a "-p"` 让 shell **不打印提示符**）；`make test01` / `make rtest01`（跑参考 shell）；`tshref.out` **是参考实现在所有 trace 上的完整输出**——比手工逐个跑方便得多。
- 建议**从 `trace01.txt` 开始逐个推进**，确保输出与参考 shell **完全一致**。

⑧ **Malloc Lab**（满分 **65**）

项	分值
Correctness	20
Performance	35
Style	10（`mm_check` 5 + 结构与注释 5）

$$P = wU + (1-w)\min\left(1, \frac{T}{T_{\text{libc}}}\right), \quad w = 0.6, \quad T_{\text{libc}} = 600\ \text{Kops/s}$$

- 详细提分点见 A.9.12；`mdriver` 用法见 A.9.12。

⑨ Proxy Lab（满分 70）

项	分值
BasicCorrectness（自动评分）	40
Concurrency（自动评分）	15
Cache（自动评分）	15

- 细节与约束见 A.12.13。**`./driver.sh` 必须在 Linux 机器上运行。**

⑩ SFS Lab（CMU 自加，课程代号 L8）

- 权重 4%、1 个 grace day；不在教材官网的 11 个 lab 列表中。

A.15.3 CMU 课程权重与截止日期（Fall 2026）

代号	名称	权重	Grace Days	发放	截止	代码评审登记
L0	C Programming	2%	1	Tue Aug 25	Tue Sep 01	—
L1	Data	7%	1	Thu Aug 27	Tue Sep 08	Thu Sep 10
L2	Bomb	6%	1	Thu Sep 03	Tue Sep 15	—
L3	Attack	4%	1	Tue Sep 15	Thu Sep 24	—
L4	Cache	5%	2	Thu Sep 24	Thu Oct 08	Sat Oct 10
L5a	Malloc (checkpoint)	4%	2	Thu Oct 08	Tue Oct 27	Thu Oct 29
L5b	Malloc (final)	7%	2	（同 L5a）	Thu Nov 05	—
L6	Shell	7%	2	Tue Nov 03	Thu Nov 12	Sat Nov 14
L7	Proxy	4%	2	Thu Nov 12	Tue Nov 24	Sun Nov 29
L8	SFS	4%	1	Thu Nov 19	Thu Dec 03	—
	合计	50%				

- **通用规则**：所有 lab 23:59 ET / 20:59 PT 截止；最多可迟交 3 天（L0 与 L8 最多 1 天）；每人全学期 5 个 grace day（自动使用、不可拒绝）；超出限额按每天 15% 计罚；不得邮件申请延期。
- **代码评审（Code Review）**：L1、L4、L5a、L5b、L6、L7 需登记，分两部分——① L4/L5a/L5b/L6/L7 进行与助教一对一的代码风格评审（依据课程 Style Guidelines）；② L1/L4/L5b/L6/L7 进行 10 分钟"帕森斯拼图（Parsons puzzle）"测试，把打乱的代码片段拼回正确顺序并解释——考的是"你是否真的理解这段代码为什么这么写"。
- **考核总览**：9 个 lab 共 50%、书面作业 6%、随堂测验 4%、期中 15%、期末 25%。
- **自学建议**：即使没人给你打分，也请自己执行这两步——把代码读一遍问"这一行为什么必须在这儿"；把关键函数打乱重排看能否复原。

A.15.4 推荐执行顺序（阶段 0 → 阶段 4）

三个排序原则：① 先补齐工具链能力 → ② 再按课程时间线推进 → ③ 把最难的 Malloc 留足时间。

```
阶段 0：环境与工具（不做 lab，但是所有 lab 的前提）
  ├─ Bootcamp 1：Linux / 命令行 / Git
  ├─ Bootcamp 2：Debugging & GDB
  └─ Bootcamp 3：GCC & Build Automation
        │
        ▼
阶段 1：数据与机器码（Lecture 2–8 期间）
  ① L0 C Programming          （热身：确认 C 与 Linux 工具链没问题）
  ② Data Lab          ★★☆☆☆   （位运算 + 补码 + 浮点，检验 Lecture 2）
  ③ Bomb Lab          ★★★☆☆   （gdb + 汇编，检验 Lecture 3–5；全课程最"好玩"）
  ④ Attack Lab        ★★★★☆   （栈帧 + 代码注入 + ROP，检验 Lecture 5–6）
  并行可选：Buffer Lab（32 位遗留，只想练栈溢出可跳过）
        │
        ▼
阶段 2：存储与性能（Lecture 9–15 期间，期中前后）
  ⑤ Cache Lab         ★★★★☆   （模拟器 + 矩阵转置优化，知识密度最高）
  并行可选：Performance Lab（与 Cache Lab Part B 高度重叠，可二选一）
  ⑥ Arch Lab          ★★★★☆   （Y86-64 + 流水线，需额外读教材 Ch.4，可与 Cache Lab 换序）
        │
        ▼
阶段 3：内存管理（Lecture 11–14 之后，最需要时间的一段）
  ⑦ Malloc Lab        ★★★★★   （全课程最难，务必留 3 周以上；先做 checkpoint）
        │
        ▼
阶段 4：并发与系统（Lecture 16–24 期间）
  ⑧ Shell Lab         ★★★★☆   （进程 + 信号 + 作业控制）
  ⑨ Proxy Lab         ★★★★☆   （socket + 线程 + 缓存 + 同步，综合度最高）
  ⑩ SFS Lab           ★★★★☆   （并发文件系统，CMU 自加）
```

各阶段的关键依赖：

```
L0 C Programming ──► 依赖 Lecture 1–2（C 基础、位运算）
L1 Data          ──► 依赖 Lecture 2（补码、位运算、浮点、UB）
L2 Bomb          ──► 依赖 Lecture 3–5（x86-64 汇编、控制流、栈帧）+ GDB
L3 Attack        ──► 依赖 Lecture 5–6（栈帧、缓冲区、ROP）+ 反汇编
L4 Cache         ──► 依赖 Lecture 9–10（缓存组织、局部性、分块）
L5a/L5b Malloc   ──► 依赖 Lecture 11–14（虚拟内存、堆、分配器设计）
L6 Shell         ──► 依赖 Lecture 16–18（进程、信号、系统调用、I/O）
L7 Proxy         ──► 依赖 Lecture 18–21（socket、并发、缓存、同步）
L8 SFS           ──► 依赖 Lecture 18–19、21–24（并发、同步、文件系统）
```

十条执行建议（踩过坑的人才知道）：

1. **先做 L0 热身**——只有 2% 权重，但能确认 gcc、make、gdb、提交链路都通。
2. **Data Lab 不要抄答案**——它的价值是逼你理解补码与 IEEE 754 的位级结构；抄了就白学 Lecture 2。
3. **Bomb Lab 是"gdb 训练营"**——`objdump -d` 找到 `phase_1`…`phase_6`，逐个下断点用 `x/s` / `x/d` / `info registers` 推。**不要**用 `strings` 找答案。
4. **Attack Lab 先画栈帧再动手**——把 `getbuf` 栈帧、`buf` 起始地址、返回地址偏移量画清楚，剩下的就是填字节。
5. **Cache Lab Part A 用时间戳数组实现 LRU**，别用链表（trace 规模大时链表太慢）。Part B：32×32 用 8 个 8×8 分块 + 局部变量暂存；64×64 必须用 4×4 子块 + `B` 数组分 8 块的特殊分块法才能拿满分。
6. **Malloc Lab 分两阶段做**（对应 L5a checkpoint / L5b final）：先**隐式空闲链表 + 立即合并**保证正确性，再升级成**分离空闲链表 + 显式链表 + 边界标记优化**追求利用率。**先用 `mdriver` 跑通再优化**。
7. **Shell Lab 的关键是"信号时序"**——`waitpid` 的 `WNOHANG|WUNTRACED`、`sigprocmask` 屏蔽 `SIGCHLD` 避免竞态、`setpgid` 建立进程组：这三件事做对了，作业控制就成了一半。
8. **Proxy Lab 先做"能用"，再做"并发"，最后做"缓存"**——迭代式代理 → 多线程（每连接一线程）→ 加读者-写者锁保护缓存 + LRU 淘汰。三步混在一起做，会同时面对三个 bug。
9. **每个 lab 第一步永远是读 README**——里面写清了文件清单、`make` 目标、提交要求、以及**"哪些文件不能改"**。**改错文件是零分的常见原因。**
10. **利用官方 handout 自带的测试工具**——`btest`（Data）、`mdriver`（Malloc）、`tracegen` / `test-csim` / `test-trans`（Cache）、`tshref` 参考壳 + `sdriver.pl`（Shell）、`driver.sh`（Proxy）。**它们比任何第三方答案都可靠。**

A.15.5 官方 handout 获取地址

教材官网 lab 页面对**自学者免费开放 handout**（只有**解答（solution）需要教师账号**）：

lab	页面（writeup）	README	自学 handout（tar）
总入口	https://csapp.cs.cmu.edu/3e/labs.html	—	—
Data Lab	https://csapp.cs.cmu.edu/3e/datalab.pdf	https://csapp.cs.cmu.edu/3e/README-datalab	https://csapp.cs.cmu.edu/3e/datalab-handout.tar
Bomb Lab	https://csapp.cs.cmu.edu/3e/bomblab.pdf	https://csapp.cs.cmu.edu/3e/README-bomblab	https://csapp.cs.cmu.edu/3e/bomb.tar（已禁用评分服务器）
Attack Lab	https://csapp.cs.cmu.edu/3e/attacklab.pdf	https://csapp.cs.cmu.edu/3e/README-attacklab	https://csapp.cs.cmu.edu/3e/target1.tar（**需用 `-q` 选项运行**）
Buffer Lab（IA32 遗留）	https://csapp.cs.cmu.edu/3e/buflab32.pdf	https://csapp.cs.cmu.edu/3e/README-buflab32	https://csapp.cs.cmu.edu/3e/buflab32-handout.tar
Arch Lab	https://csapp.cs.cmu.edu/3e/archlab.pdf	https://csapp.cs.cmu.edu/3e/README-archlab	https://csapp.cs.cmu.edu/3e/archlab-handout.tar
Arch Lab（Legacy 32 位）	https://csapp.cs.cmu.edu/3e/archlab32.pdf	https://csapp.cs.cmu.edu/3e/README-archlab32	https://csapp.cs.cmu.edu/3e/archlab32-handout.tar
Cache Lab	https://csapp.cs.cmu.edu/3e/cachelab.pdf	https://csapp.cs.cmu.edu/3e/README-cachelab	https://csapp.cs.cmu.edu/3e/cachelab-handout.tar
Performance Lab	https://csapp.cs.cmu.edu/3e/perflab.pdf	https://csapp.cs.cmu.edu/3e/README-perflab	https://csapp.cs.cmu.edu/3e/perflab-handout.tar
Shell Lab	https://csapp.cs.cmu.edu/3e/shlab.pdf	https://csapp.cs.cmu.edu/3e/README-shlab	https://csapp.cs.cmu.edu/3e/shlab-handout.tar
Malloc Lab	https://csapp.cs.cmu.edu/3e/malloclab.pdf	https://csapp.cs.cmu.edu/3e/README-malloclab	https://csapp.cs.cmu.edu/3e/malloclab-handout.tar
Proxy Lab	https://csapp.cs.cmu.edu/3e/proxylab.pdf	https://csapp.cs.cmu.edu/3e/README-proxylab	https://csapp.cs.cmu.edu/3e/proxylab-handout.tar
SFS Lab	**无（CMU 自研）**	—	**教材官网无 handout**——需通过 CMU 课程页面（Autolab / GitHub Education）获取

> **关于"网上答案"的严肃提醒。** CSAPP 的 lab 之所以经典，是因为**做的过程**才是学习本身。抄答案等于把最值钱的部分扔了。